System Design

A Complete Guide

Master Scalable Architecture — From Fundamentals to Designing Twitter, YouTube, and Uber

2025 Edition • 34 Chapters

Table of Contents

Chapter 1 — What Is System Design
Chapter 2 — Scaling Basics
Chapter 3 — Load Balancing
Chapter 4 — Caching
Chapter 5 — Content Delivery Networks (CDN)
Chapter 6 — Databases: SQL and NoSQL
Chapter 7 — Database Replication
Chapter 8 — Database Sharding
Chapter 9 — Message Queues
Chapter 10 — API Design
Chapter 11 — Rate Limiting
Chapter 12 — Consistent Hashing
Chapter 13 — CAP Theorem
Chapter 14 — Microservices
Chapter 15 — Monolith to Microservices
Chapter 16 — Event-Driven Architecture
Chapter 17 — Blob Storage
Chapter 18 — Search Systems
Chapter 19 — Notification System
Chapter 20 — Logging and Monitoring
Chapter 21 — Design: URL Shortener
Chapter 22 — Design: Paste Bin
Chapter 23 — Design: Twitter
Chapter 24 — Design: Instagram
Chapter 25 — Design: Chat System
Chapter 26 — Design: YouTube
Chapter 27 — Design: Uber

Chapter 28 — Design: Notification Service
Chapter 29 — Design: Rate Limiter
Chapter 30 — Design: Web Crawler
Chapter 31 — Estimation Math
Chapter 32 — Interview Framework
Chapter 33 — Common Patterns
Chapter 34 — Glossary

Chapter 1: What Is System Design?

What You Will Learn

- What system design actually means and why it is different from writing code
- The difference between an architect and a builder in software
- How to separate functional requirements from non-functional requirements
- Core concepts: latency, throughput, availability, consistency, and partition tolerance
- How to do quick back-of-envelope calculations
- A repeatable 4-step framework for tackling any system design problem

Why This Chapter Matters

Imagine you are asked to build a house. You could start laying bricks right away. But without a blueprint, you might end up with a kitchen where the bathroom should be, walls that cannot support a second floor, and doors that open into walls.

System design is the blueprint for software. It is the process of deciding how the pieces of a system fit together before you start writing code. Whether you are preparing for a technical interview or building a real product, system design thinking separates engineers who build things that work from engineers who build things that work at scale.

Every major outage, every slow application, every system that crumbles under traffic -- these are usually not code bugs. They are design problems.

1.1 What Is System Design?

System design is the process of defining the architecture, components, modules, interfaces, and data flow of a system to satisfy specified requirements.

In simpler terms: it is deciding what pieces you need and how they talk to each other.

The Architect vs The Builder

Think of constructing a skyscraper.

- The architect decides how many floors, where the elevators go, how the plumbing runs, and what materials can handle the load. They think about earthquakes, wind, and thousands of people using the building daily.
- The builder takes those plans and executes them. They pour concrete, wire electricity, and install fixtures.

In software:

- The system designer (architect) decides: Should we use one database or three? How do users authenticate? What happens when a server crashes? How do we handle 10 million requests per second?
- The developer (builder) writes the code that implements those decisions.

```
+-----------------------------------------------------+
|              SYSTEM DESIGN PROCESS                  |
+-----------------------------------------------------+
|                                                     |
|   Requirements --> Architecture --> Components      |
|        |               |               |            |
|        v               v               v            |
|   "What does it    "How do the    "What does       |
|    need to do?"    pieces fit?"   each piece do?"  |
|                                                     |
+-----------------------------------------------------+
```

Most engineers start their careers as builders. System design is the skill that turns builders into architects.

Real-World vs Interview System Design

System design shows up in two major places:

In interviews:

- You are given an open-ended problem: "Design Twitter" or "Design a URL shortener"
- You have 45-60 minutes to define requirements, sketch architecture, and discuss trade-offs
- The interviewer is evaluating your thinking process, not looking for one correct answer

In the real world:

- You are building a feature and need to decide how it fits into the existing system
- Your system is falling over under load and you need to figure out why
- You are starting a new project and need to choose the right database, message queue, and deployment strategy

The skills are the same. The difference is that in real life, you have more time and more information. In interviews, you have to make reasonable assumptions quickly.

1.2 Functional vs Non-Functional Requirements

Every system has two types of requirements. Confusing them is one of the most common mistakes in system design.

Functional Requirements: What the System Does

These describe the features and behaviors of the system. They answer: "What can a user do?"

Examples for a social media app:

- Users can create an account
- Users can post text and images
- Users can follow other users
- Users see a feed of posts from people they follow
- Users can like and comment on posts

Non-Functional Requirements: How the System Performs

These describe the qualities of the system. They answer: "How well does it work?"

Examples for the same social media app:

- The feed loads in under 200 milliseconds
- The system handles 50,000 concurrent users
- The system is available 99.99% of the time
- Data is never lost, even if a server crashes
- The system works for users in 30 countries

The Iceberg Analogy

Functional requirements are the tip of the iceberg -- they are what users see. Non-functional requirements are the massive hidden part underneath -- they are what keeps the system from sinking.

```
           Functional Requirements
               (What users see)
          ___________________________
         /    Create account          \
        /     Post content             \
       /      Follow users              \
~~~
```

L2 cache reference	7 ns
Main memory reference	100 ns
SSD random read	150 us
HDD random read	10 ms
Send packet CA to Netherlands	150 ms
Read 1 MB from memory	250 us
Read 1 MB from SSD	1 ms
Read 1 MB from HDD	20 ms
Read 1 MB over 1 Gbps network	10 ms

Key insight: Latency is about speed for one request. Low latency means fast responses.

Throughput: How Much Work Gets Done

Definition: The number of requests or operations a system can handle per unit of time.

Analogy: Back at the coffee shop. Throughput is the number of lattes the shop can make per hour. A shop might have low latency (each latte takes 2 minutes) but low throughput (only one barista, so 30 lattes per hour). Or it might have 5 baristas and make 150 lattes per hour.

Key insight: Throughput is about volume. High throughput means handling many requests.

Latency vs Throughput:

```
Latency (Speed)                Throughput (Volume)
+----------+                   +----------+
| Request |---> 50ms           | 1000     |
+----------+                   | req/sec |
                               +----------+
"How fast is one?"             "How many per second?"
```

You can have low latency but low throughput (a very fast single-lane road). You can have high throughput but high latency (a cargo ship carries a lot but takes weeks). The best systems optimize both.

Availability: How Often It Works

Definition: The percentage of time a system is operational and accessible.

Analogy: A vending machine. If you walk up to it 100 times and it works 99 times, its availability is 99%.

Availability is measured in "nines":

Availability	Downtime per year	Downtime per month	Downtime per day
99% (two nines)	3.65 days	7.3 hours	14.4 minutes
99.9% (three nines)	8.77 hours	43.8 minutes	1.44 minutes
99.99% (four nines)	52.6 minutes	4.38 minutes	8.64 seconds
99.999% (five nines)	5.26 minutes	26.3 seconds	864 ms

Key insight: Each additional "nine" is exponentially harder and more expensive to achieve. Going from 99.9% to 99.99% does not sound like much, but it means reducing downtime from 8.7 hours/year to 52 minutes/year.

Consistency: Does Everyone See the Same Data?

Definition: Whether all users see the same data at the same time after a write operation.

Analogy: Imagine a shared Google Doc. If you type a sentence and your colleague immediately sees it, that is strong consistency. If there is a delay and they see the old version for a few seconds, that is eventual consistency.

Types of consistency:

- Strong consistency: After a write, all subsequent reads return the updated value. Everyone always sees the latest data.
- Eventual consistency: After a write, reads might return old data for a while, but eventually all copies will converge to the latest value.
- Causal consistency: If event A causes event B, everyone will see A before B. But unrelated events can appear in different orders.

Partition Tolerance: What Happens When the Network Breaks?

Definition: The system continues to operate even when network communication between parts of the system fails.

Analogy: You have two offices in different cities connected by a phone line. Partition tolerance means work continues even if the phone line goes down. Each office keeps working independently and syncs up when the connection is restored.

The CAP Theorem (Simplified)

The CAP theorem states that a distributed system can only guarantee two out of three properties:

- Consistency: Every read gets the most recent write
- Availability: Every request gets a response
- Partition tolerance: The system works despite network failures

```
              Consistency (C)
                   /\
                  /  \
                 /    \
                / CP   \
               /  systems\
              /    (e.g.,  \
             /   HBase,     \
            /    MongoDB*)    \
           /                   \
          /_____________________\
         /\                    /\
        /  \      CA          /  \
       / AP \   systems     /    \
      / systems (traditional\    \
     / (Cassandra, RDBMS on  \   \
    /   DynamoDB) one node)   \  \
   /_________________________________\
  Availability (A)    Partition Tolerance (P)

  * In practice, P is not optional in distributed
    systems. Networks WILL fail. So the real choice
    is between CP and AP.
```

The practical takeaway: Since network partitions are inevitable in distributed systems, you are really choosing between consistency and availability. Different parts of your system might make different choices.

1.4 Back-of-Envelope Estimation

System design problems often require you to estimate scale. You do not need exact numbers. You need reasonable approximations.

Why Estimates Matter

If your system needs to handle 100 requests per second, a single server might work fine. If it needs to handle 1 million requests per second, you need a completely different architecture. Estimation helps you choose the right approach.

Key Numbers to Remember

Users and traffic:

- 1 million seconds is about 12 days
- 1 billion seconds is about 31.7 years
- 1 day has about 86,400 seconds (round to 100,000 for estimation)
- 1 month has about 2.5 million seconds

Data sizes:

- 1 character (ASCII) = 1 byte
- 1 character (Unicode/UTF-8) = 1-4 bytes
- A typical tweet-length message = ~200 bytes
- A typical web page = ~2 MB
- A photo (compressed) = ~200 KB to 2 MB
- A short video = ~50 MB

Storage:

- 1 KB = 1,000 bytes (10^3)
- 1 MB = 1,000,000 bytes (10^6)
- 1 GB = 10^9 bytes
- 1 TB = 10^12 bytes
- 1 PB = 10^15 bytes

Example: Estimating Storage for a Photo Sharing App

Given:

- 500 million users
- 10% are daily active users (50 million DAU)
- Each active user uploads 2 photos per day
- Average photo size: 500 KB

Calculation:

```
Photos per day    = 50M users x 2 photos = 100M photos/day
Storage per day   = 100M x 500 KB = 50 TB/day
```

```
Storage per year  = 50 TB x 365 = ~18 PB/year
```

That is a LOT of storage. This tells you immediately that you need distributed storage, a CDN for serving photos, and probably a strategy for compressing or tiering old photos.

The Power of Round Numbers

Do not get caught up in exact math. Round aggressively:

- 365 days? Call it 400 (or 300 for conservative estimates)
- 86,400 seconds in a day? Call it 100,000
- 2.5 million seconds in a month? Call it 2 or 3 million

The goal is to land in the right order of magnitude, not the right decimal.

1.5 How to Approach a System Design Problem

Here is a repeatable 4-step framework that works for interviews and real-world design.

Step 1: Clarify Requirements (5 minutes)

Do not start designing immediately. Ask questions first.

Questions to ask:

- Who are the users? How many?
- What are the core features? (Narrow down to 3-5)
- What are the scale requirements? (Users, requests per second, data volume)
- What are the non-functional requirements? (Latency, availability, consistency)
- Are there any constraints? (Budget, existing technology, regulatory)

Example for "Design a URL Shortener":

- How many URLs shortened per day? (100 million)
- How many redirects per day? (10 billion -- 100:1 read/write ratio)
- How long should shortened URLs be kept? (10 years by default)
- Should analytics be tracked? (Yes, click counts)
- URL format? (As short as possible, alphanumeric)

Step 2: High-Level Design (10 minutes)

Sketch the major components and how they connect. Do not go deep yet.

```
+--------+      +---------------+      +-----------+
| Client | ---> | Load Balancer | ---> | Web Server|
+--------+      +---------------+      +-----------+
                                              |
                                              v
                                       +-----------+
                                       |  Database |
                                       +-----------+
```

Identify the main flows:

- How does a user shorten a URL?
- How does a user get redirected from a short URL?
- How are analytics collected?

Step 3: Deep Dive (20 minutes)

Pick the most interesting or challenging components and design them in detail.

For the URL shortener, you might deep dive into:

- URL generation: How do you create short, unique IDs? (Base62 encoding, hash functions, pre-generated IDs)
- Database schema: What tables? What indexes?
- Read optimization: How do you serve 10 billion redirects per day quickly? (Caching, database read replicas)

Step 4: Wrap Up (5 minutes)

Discuss:

- Bottlenecks: Where will the system fail under extreme load?
- Trade-offs: What did you sacrifice and why?
- Future improvements: What would you add with more time?
- Monitoring: How do you know the system is healthy?

The Framework Visualized

```
+-----------------------------------------------------------+
|          SYSTEM DESIGN FRAMEWORK (4 STEPS)                |
+-----------------------------------------------------------+
|                                                           |
|  STEP 1: CLARIFY (5 min)                                  |
|  +-------------------------------------------------------+ |
|  | Users? Scale? Features? Constraints? Non-functional? | |
|  +-------------------------------------------------------+ |
|                          |                                |
|                          v                                |
|  STEP 2: HIGH-LEVEL DESIGN (10 min)                       |
|  +-------------------------------------------------------+ |
|  | Major components, APIs, data flow, basic diagram     | |
|  +-------------------------------------------------------+ |
|                          |                                |
|                          v                                |
|  STEP 3: DEEP DIVE (20 min)                               |
|  +-------------------------------------------------------+ |
|  | Pick 2-3 components. Detail schema, algorithms,      | |
|  | scaling strategies, failure handling                  | |
|  +-------------------------------------------------------+ |
|                          |                                |
|                          v                                |
|  STEP 4: WRAP UP (5 min)                                  |
|  +-------------------------------------------------------+ |
|  | Bottlenecks, trade-offs, monitoring, future work     | |
|  +-------------------------------------------------------+ |
|                                                           |
+-----------------------------------------------------------+
```

Trade-Offs Table

Decision	Option A	Option B	When to Choose A	When to Choose B
Consistency vs Availability	Strong consistency	Eventual consistency	Financial data, inventory	Social feeds, likes
Latency vs Throughput	Optimize for latency	Optimize for throughput	Real-time apps, gaming	Batch processing, analytics
Simplicity vs Scalability	Simple architecture	Complex distributed system	Early stage, < 1000 users	Millions of users, growth expected
Cost vs Performance	Budget-friendly	High performance	Startups, MVPs	Revenue-critical systems

Build vs Buy	Build custom	Use managed service	Unique requirements, competitive advantage	Standard problems (auth, payments)

Common Mistakes

1. Jumping into design without clarifying requirements. You cannot design a system if you do not know what it needs to do. Always ask questions first.
2. Over-engineering from the start. Not every system needs microservices, Kubernetes, and a distributed cache. Start simple and scale when needed.
3. Ignoring non-functional requirements. A system that has all the features but cannot handle 100 users is not a good design.
4. Forgetting about failure. Servers crash. Networks fail. Disks die. Your design must account for these realities.
5. Not considering data volume. "Store user data" sounds simple until you realize you have 500 million users each with 10 GB of data.
6. Treating system design as memorization. It is a thinking skill, not a trivia contest. Understanding trade-offs matters more than knowing specific technologies.

Best Practices

1. Always start with requirements. Spend the first few minutes understanding what you are building and for whom.
2. Design for the expected scale, not infinite scale. If your system serves 1,000 users, a single database is fine. Do not add complexity you do not need yet.
3. Think in terms of trade-offs, not best solutions. Every decision has pros and cons. Acknowledge them.
4. Use back-of-envelope math early. Quick calculations prevent you from building a solution that is wildly under or over-provisioned.
5. Draw diagrams. A picture is worth a thousand words in system design. Visualize data flow and component interactions.
6. Consider failure modes. For each component, ask: "What happens if this fails?"

Quick Summary

System design is the process of defining how the pieces of a software system fit together. It is the difference between an architect who plans the building and a builder who constructs it. Every system has functional requirements (what it does) and non-functional requirements (how well it does it). Five core concepts -- latency, throughput, availability, consistency, and partition tolerance -- form the vocabulary of system design. Back-of-envelope estimation helps you understand the scale of a problem. The 4-step framework (clarify, high-level design, deep dive, wrap up) gives you a repeatable approach to any system design challenge.

Key Points

- System design is about deciding how pieces fit together, not writing code
- Functional requirements define features; non-functional requirements define quality
- Latency = speed of one request; throughput = volume of requests
- Availability is measured in "nines" -- each nine is exponentially harder
- The CAP theorem forces a choice between consistency and availability in distributed systems
- Back-of-envelope estimation helps you understand scale before you design
- The 4-step framework: Clarify, High-Level Design, Deep Dive, Wrap Up

---

Practice Questions

1. You are designing a messaging app like WhatsApp. List 5 functional requirements and 5 non-functional requirements. How would you prioritize them?
2. A system has 99.95% availability. How many minutes of downtime does that allow per month? If the system has 3 independent components each with 99.95% availability, what is the overall system availability?
3. You are told a system handles 5,000 requests per second with an average latency of 20ms. The product team wants to add a feature that increases latency to 50ms per request. What is the impact on throughput, and what might you do about it?

4. Estimate the storage requirements for an email service with 1 billion users, where the average user sends 10 emails per day and the average email is 50 KB (including attachments). How much storage do you need per year?
5. For each of the following systems, would you prioritize consistency or availability, and why?
    - A banking transaction system
    - A social media news feed
    - An airline seat booking system
    - A content recommendation engine

Exercises

Exercise 1: Design Requirement Gathering

Pick any application you use daily (Instagram, Spotify, Uber, etc.). Write down:

- 10 functional requirements
- 5 non-functional requirements
- Identify which non-functional requirements are most critical and explain why

Exercise 2: Back-of-Envelope Practice

Estimate the following:

- How many photos does Instagram store in total? (Hint: start with number of users, daily active users, photos per user per day, and how long Instagram has existed)
- How much bandwidth does YouTube need to serve video? (Hint: start with number of daily viewers, average video length, average video quality/bitrate)

Show your work step by step. Remember: the goal is the right order of magnitude.

Exercise 3: Framework Practice

Apply the 4-step framework to this problem: "Design a parking lot system." Spend 5 minutes on each step. Focus on:

- Clarifying questions you would ask
- High-level components
- One area to deep dive into
- Trade-offs you made

What Is Next?

Now that you understand what system design is and have a framework for approaching problems, the next step is to learn about the most fundamental challenge: scaling. In Chapter 2, we will start with a single server and explore what happens as your system grows. You will learn the difference between vertical and horizontal scaling, understand stateless vs stateful architectures, and see how session handling works. These are the building blocks that every other concept in this book builds upon.

Chapter 2: Scaling Basics

What You Will Learn

- How a single server architecture works and where it breaks down
- The difference between vertical scaling (bigger machine) and horizontal scaling (more machines)
- Why stateless servers are easier to scale than stateful servers
- How to handle user sessions in a scaled environment
- When database becomes the bottleneck and what to watch for
- How to choose between scaling up and scaling out

Why This Chapter Matters

Every successful application starts small. One server, one database, a handful of users. Then something wonderful and terrifying happens: people start using your product.

Suddenly, your single server is sweating. Response times climb. Users see errors. And you face the question that every growing system must answer: how do we handle more load?

Scaling is the foundation that everything else in this book builds upon. Load balancers, caches, CDNs, database replication -- they all exist to solve scaling problems. Before you can use those tools effectively, you need to understand why scaling is necessary and what your options are.

2.1 Single Server Architecture

Let us start at the beginning. The simplest possible web architecture looks like this:

```
+--------+         +---------------------------+
|        |  HTTP   |        Single Server       |
| User's | ------> |                            |
| Browser|         |  +-------+   +----------+ |
|        | <------ |  | Web   |   | Database | |
+--------+         |  | App   |-->|          | |
                    |  +-------+   +----------+ |
                    +---------------------------+
```

Everything runs on one machine:

- The web application that handles requests
- The database that stores data
- Static files (images, CSS, JavaScript)
- Maybe even a background job processor

How It Works

1. A user types your URL into their browser
2. DNS resolves the domain name to your server's IP address
3. The browser sends an HTTP request to that IP
4. Your web application processes the request
5. It reads from or writes to the database
6. It sends back an HTML page (or JSON response)

When One Server Is Enough

A single server can handle more than you might think:

- A modern server with 8 CPU cores and 32 GB RAM can handle thousands of concurrent connections
- If your application is well-written, one server can serve 1,000-10,000 requests per second
- For a blog, small business site, or early-stage startup with a few hundred users, this is perfectly fine

When One Server Breaks Down

A single server has hard limits:

- CPU saturation: When all cores are busy, new requests wait in a queue

- Memory exhaustion: When RAM is full, the OS starts swapping to disk, and everything slows to a crawl
- Disk I/O limits: Databases need to read and write to disk, and disks have finite speed
- Network bandwidth: One network card can only move so much data
- Single point of failure: If this one server dies, everything dies

```
         Load Increasing Over Time

Requests  |                         X Server crashes
per       |                       X
second    |                     X
          |                   X
          |               X     <-- Response times
          |            X            start degrading
          |         X
          |      X
          |   X
           +----------------------------------
                      Time -->

         One server can only take so much.
```

You have two options when you hit these limits: make the server bigger, or add more servers.

2.2 Vertical Scaling (Scaling Up)

Vertical scaling means making your existing server more powerful.

The "Bigger Desk" Analogy

Imagine you work at a desk and it is getting cluttered. You have papers everywhere, your monitor barely fits, and you keep knocking over your coffee.

Vertical scaling is like replacing your desk with a bigger desk. Same room, same person, just more workspace.

```
BEFORE (Small Server)           AFTER (Bigger Server)
+------------------+            +---------------------------+
|  4 CPU cores     |            |  32 CPU cores             |
|  16 GB RAM       |   ====>   |  256 GB RAM               |
|  500 GB HDD      |            |  2 TB NVMe SSD            |
|  1 Gbps network  |            |  10 Gbps network          |
+------------------+            +---------------------------+
```

```
Same architecture, more powerful hardware.
```

How to Vertically Scale

- Add more RAM
- Upgrade to faster CPUs (more cores, higher clock speed)
- Replace HDD with SSD (or SSD with NVMe)
- Upgrade network interface
- Move to a larger cloud instance (e.g., from AWS t3.medium to r5.4xlarge)

Advantages of Vertical Scaling

Advantage	Why It Matters
Simple	No code changes needed. Your application does not know or care that the hardware changed.
No distributed complexity	One server means no network latency between components, no data synchronization issues.
Strong consistency	All data is in one place. No need to worry about replicas being out of sync.
Lower operational cost	One server is easier to monitor, maintain, and debug than many.
Transactions are simple	Database transactions work naturally on a single machine.

Limits of Vertical Scaling

Limitation	Reality
Hardware ceiling	You cannot add infinite RAM or CPUs. The biggest servers top out at a few terabytes of RAM and a few hundred cores.
Cost grows non-linearly	Doubling performance often costs more than double the price. A server with 256 GB RAM costs much more than twice a 128 GB server.

Single point of failure	No matter how powerful, it is still one machine. If it fails, everything goes down.
Downtime during upgrades	Changing hardware usually means shutting the server down.
Diminishing returns	After a certain point, adding more resources gives smaller and smaller improvements due to software bottlenecks.

Real-World Vertical Scaling Example

AWS EC2 instance sizes (approximate, for illustration):

```
Instance Type    vCPUs    RAM       Cost/Month
--------------------------------------------
t3.micro           2      1 GB        ~$8
t3.large           2      8 GB       ~$60
r5.xlarge          4     32 GB      ~$180
r5.4xlarge        16    128 GB      ~$720
r5.16xlarge       64    512 GB    ~$2,880
u-24tb1.metal    448     24 TB   ~$160,000

Notice how cost grows much faster than capacity.
```

2.3 Horizontal Scaling (Scaling Out)

Horizontal scaling means adding more servers to share the work.

The "Hiring More Workers" Analogy

Going back to the desk analogy. Instead of buying a bigger desk, you hire more people. Each person gets their own desk. You split the work among them.

The work gets done faster, and if one person calls in sick, the others can cover.

```
BEFORE (One Server)              AFTER (Multiple Servers)

+--------+    +---------+       +--------+    +---------+
| Client |--->| Server  |       | Client |--->| Load    |
+--------+    +---------+       +--------+    | Balancer|
                                              +----+----+
```

```
                                                  |
                                       +----------+----------+
                                       |          |          |
                                  +--------+ +--------+ +--------+
                                  |Server 1| |Server 2| |Server 3|
                                  +--------+ +--------+ +--------+
```

How Horizontal Scaling Works

1. You run multiple copies of your application on different servers
2. A load balancer sits in front of the servers and distributes incoming requests
3. Each server handles a portion of the traffic
4. If you need more capacity, you add more servers
5. If a server fails, the others keep working

Advantages of Horizontal Scaling

Advantage	Why It Matters
(Nearly) unlimited scaling	Need more capacity? Add more servers. There is no hard ceiling.
Fault tolerance	If one server dies, the others continue serving. Users might not even notice.
No downtime for scaling	Add or remove servers without shutting anything down.
Cost-effective	Many small servers can be cheaper than one massive server.
Geographic distribution	You can place servers in different regions, closer to users.

Challenges of Horizontal Scaling

Challenge	Why It Is Hard
Data consistency	With data spread across servers, keeping it in sync is complex.
Session management	If user state lives on one server, what happens when they hit a different server?

Operational complexity	More servers means more to monitor, deploy to, and debug.
Application changes	Your code needs to be designed to run on multiple servers (stateless design).
Database bottleneck	Even with many app servers, they all talk to the same database.

2.4 Vertical vs Horizontal: Side by Side

```
VERTICAL SCALING                    HORIZONTAL SCALING
(Scale Up)                          (Scale Out)

+-------------+                    +-----+ +-----+ +-----+
|             |                    |     | |     | |     |
|   BIGGER    |                    |Small| |Small| |Small|
|   SERVER    |                    |  1  | |  2  | |  3  |
|             |                    |     | |     | |     |
|             |                    +-----+ +-----+ +-----+
+-------------+

Like getting a                     Like hiring more
bigger desk                        workers

Pros:                              Pros:
- Simple                           - No ceiling
- No code changes                  - Fault tolerant
- Strong consistency               - Cost effective

Cons:                              Cons:
- Has a ceiling                    - More complex
- Expensive at scale               - Data sync issues
- Single point of failure          - Code must be stateless
```

When to Use Which

Start with vertical scaling when:

- You are early stage with few users
- Your architecture is simple (monolith)
- You need strong consistency
- You want to move fast without infrastructure complexity

Switch to horizontal scaling when:

- You hit hardware limits
- You need fault tolerance (cannot afford downtime)
- Your user base is growing rapidly
- You need to serve users in multiple geographic regions

In practice, most systems use both: a reasonable server size (vertical) multiplied across many instances (horizontal).

2.5 Stateless vs Stateful Servers

The ability to scale horizontally depends heavily on whether your servers are stateful or stateless. This distinction is critical.

Stateful Servers

A stateful server remembers information about each user between requests. User session data, shopping cart contents, or authentication status is stored in the server's memory.

```
STATEFUL SERVER PROBLEM

User Alice's session is on Server 1.

+-------+      +-----------+      +----------+
| Alice | ---> | Server 1  | ---> | Server 1 |
+-------+      | (has her  |      | knows who|
               | session)  |      | Alice is |
               +-----------+      +----------+

What if Server 1 dies? Or what if the load
balancer sends Alice to Server 2?

+-------+      +-----------+      +----------+
| Alice | ---> | Server 2  | ---> | Server 2 |
+-------+      | (no idea  |      | "Who is  |
               | who Alice |      |  Alice?" |
               |   is)     |      |          |
               +-----------+      +----------+

Alice gets logged out. Her cart is empty.
She is not happy.
```

Problems with stateful servers:

- Users are "stuck" to specific servers
- Scaling out is hard because new servers do not have existing session data
- Server failures cause data loss and user disruption
- Uneven load distribution (some servers have more active sessions)

Stateless Servers

A stateless server does not store any user-specific data between requests. Every request contains all the information the server needs to process it. Session data is stored externally (in a database, cache, or the client itself).

```
STATELESS SERVER -- ANY SERVER CAN HANDLE ANY REQUEST

+-------+      +---------------+      +----------+
| Alice | ---> | Load Balancer | ---> | Server 1 | ---+
+-------+      +---------------+      +----------+    |
                     |                                 |
                     +----------> +----------+         |
                                  | Server 2 | ---+    |
                                  +----------+    |    |
                                                  |    |
                                                  v    v
                                            +--------------+
                                            | Shared State |
                                            | (Redis /     |
                                            |  Database)   |
                                            +--------------+

Any server can handle Alice's request because
the session data is in the shared store, not
on any individual server.
```

Benefits of stateless servers:

- Any server can handle any request
- Easy to add or remove servers
- Server failures do not lose user data
- Load is distributed evenly
- Auto-scaling becomes straightforward

2.6 Session Handling Strategies

When you move to stateless servers, you need a strategy for handling user sessions. Here are the three main approaches.

Strategy 1: Sticky Sessions

The load balancer routes all requests from the same user to the same server. The server stores the session in its own memory.

```
+-------+                          +----------+
| Alice | ---- always goes to --> | Server 1 |
+-------+                          +----------+

+-------+                          +----------+
|  Bob  | ---- always goes to --> | Server 2 |
+-------+                          +----------+
```

How it works: The load balancer uses a cookie or the user's IP address to consistently route them to the same backend server.

Pros:

- Simple to implement (load balancer configuration only)
- No external session store needed
- Low latency for session reads (data is in local memory)

Cons:

- Uneven load distribution (some servers get "heavier" users)
- If a server dies, all sessions on that server are lost
- Hard to auto-scale (removing a server means losing sessions)
- Not truly stateless

Strategy 2: External Session Store

Store all session data in an external shared store like Redis or Memcached. Servers read and write session data from this store.

```
+----------+     +----------+     +----------+
| Server 1 |     | Server 2 |     | Server 3 |
+----+-----+     +----+-----+     +----+-----+
     |                |                |
     +-------+--------+--------+-------+
             |                 |
             v                 v
     +---------------+  +---------------+
```

```
     | Redis Primary |  | Redis Replica |
     +---------------+  +---------------+

All servers read/write sessions to Redis.
Any server can handle any user's request.
```

Pros:

- True stateless servers -- any server can handle any request
- Server failures do not lose sessions
- Easy to auto-scale (just add/remove app servers)
- Session data survives server restarts

Cons:

- Extra infrastructure (Redis/Memcached cluster)
- Network latency for every session read/write
- Session store becomes a critical dependency (needs its own high availability)

Example: Express.js with Redis sessions:

```javascript
const session = require('express-session');
const RedisStore = require('connect-redis').default;
const { createClient } = require('redis');

const redisClient = createClient({
  host: 'redis-session-store.example.com',
  port: 6379
});

app.use(session({
  store: new RedisStore({ client: redisClient }),
  secret: 'your-secret-key',
  resave: false,
  saveUninitialized: false,
  cookie: {
    maxAge: 1800000  // 30 minutes
  }
}));
```

Strategy 3: Client-Side Sessions (JWT)

Store session data in the client itself, usually as a signed JSON Web Token (JWT) in a cookie or header.

```
+--------+                        +----------+
| Client |  --- sends JWT with -> | Server   |
| (has   |  --- every request --> | (verifies|
|  JWT)  |                        |  JWT,    |
+--------+                        |  needs   |
                                  |  no      |
                                  |  storage)|
                                  +----------+

The server does not store anything.
The client carries its own session.
```

Pros:

- Zero server-side storage needed
- Works great with microservices (any service can verify the token)
- No session store infrastructure to manage
- Scales infinitely on the server side

Cons:

- Cannot easily revoke a session (the token is valid until it expires)
- Token size increases with more data (sent with every request)
- Sensitive data should not go in the token (it is encoded, not encrypted)
- Token refresh logic adds complexity

Comparison Table

Aspect	Sticky Sessions	External Store (Redis)	Client-Side (JWT)
Server statefulness	Stateful	Stateless	Stateless
Fault tolerance	Low	High	High
Scalability	Limited	Good	Excellent
Complexity	Low	Medium	Medium
Session revocation	Easy	Easy	Hard
Latency overhead	None	Network hop to Redis	Token verification
Infrastructure	None extra	Redis cluster	None extra

2.7 Database Bottleneck Preview

Even with perfect horizontal scaling of your application servers, you will eventually hit a wall: the database.

```
+----------+  +----------+  +----------+  +----------+
| Server 1 |  | Server 2 |  | Server 3 |  | Server 4 |
+----+-----+  +----+-----+  +----+-----+  +----+-----+
     |             |             |             |
     +------+------+------+-----+------+------+
            |             |            |
            v             v            v
         +--------------------------------+
         |        SINGLE DATABASE         |
         |     (This is the bottleneck)   |
         +--------------------------------+

You scaled the app servers, but they all
compete for the same database resources.
```

Why the Database Becomes the Bottleneck

- Connection limits: Each database has a maximum number of concurrent connections (e.g., PostgreSQL defaults to 100)
- Write contention: Only one write can happen to a given row at a time
- Disk I/O: Even SSDs have limits on reads and writes per second
- Query complexity: As data grows, queries take longer without proper indexing
- Single server limits: The database runs on one machine with all the same limits we discussed

Signs Your Database Is the Bottleneck

- Application servers have low CPU usage but responses are slow
- Database CPU or I/O is consistently high
- Query execution times are increasing
- Connection pool is frequently exhausted
- Replication lag is growing (if using replicas)

What You Can Do About It (Preview)

We will cover these topics in detail in later chapters:

1. Read replicas: Send read queries to copies of the database

2. Caching: Put frequently accessed data in Redis or Memcached (Chapter 4)
3. Database sharding: Split data across multiple database servers
4. Query optimization: Better indexes, query rewriting, denormalization
5. Connection pooling: Reuse database connections instead of creating new ones
6. Choosing the right database: SQL vs NoSQL has scaling implications (Chapter 6)

Trade-Offs Table

Decision	Option A	Option B	When to Choose A	When to Choose B
Vertical vs Horizontal	Scale up (bigger machine)	Scale out (more machines)	Early stage, simple app, strong consistency needed	Growing traffic, need fault tolerance, global users
Stateful vs Stateless	Stateful servers	Stateless servers	Very simple app, no scaling needed	Any system that needs to scale
Session: Sticky vs Shared	Sticky sessions	External session store	Quick fix, low traffic	Production systems, need reliability
Session: Server vs Client	Server-side sessions	JWT/client-side sessions	Need session revocation, sensitive data	Microservices, API-first, mobile apps
Single DB vs Multiple	One database	Replicas/shards	Small data, low traffic	Read-heavy workloads, large datasets

Common Mistakes

1. Premature horizontal scaling. Adding load balancers and multiple servers when a single, slightly bigger server would work fine. Complexity has a cost.
2. Keeping state in application servers. Storing session data, uploaded files, or caches in server memory makes horizontal scaling painful. Externalize everything.
3. Ignoring the database. Scaling application servers is easy. The database is usually the real bottleneck, and it is harder to scale.

4. Not planning for failure. Scaling is not just about handling more load. It is also about surviving when things go wrong. A single server is a single point of failure.
5. Scaling before measuring. Always identify the bottleneck before scaling. If your database is the problem, adding more app servers will not help.
6. Forgetting about data locality. When you have servers in multiple regions, data access patterns matter. A server in Tokyo hitting a database in Virginia will have high latency regardless of how many servers you have.

Best Practices

1. Start simple. Begin with one server. It is good enough longer than you think.
2. Measure before scaling. Use monitoring to identify the actual bottleneck (CPU, memory, disk, network, database).
3. Make servers stateless early. Even if you only have one server, design your application as if you have many. It is much easier to do this from the start than to retrofit it later.
4. Use an external session store. Redis or Memcached is cheap and eliminates a whole class of scaling problems.
5. Separate static and dynamic content. Serve images, CSS, and JavaScript from a CDN (Chapter 5), not your application server.
6. Plan for the database. Think about read/write ratios, data volume, and query patterns early. These inform your database scaling strategy.
7. Automate scaling. Use auto-scaling groups (in cloud environments) to automatically add servers during peak traffic and remove them during quiet periods.

Quick Summary

Every system starts with a single server. As traffic grows, you face a choice: make the server bigger (vertical scaling) or add more servers (horizontal scaling). Vertical scaling is simple but has a ceiling and a single point of failure. Horizontal scaling is more flexible but requires your application to be stateless. Session handling is the first challenge of horizontal scaling, and the main strategies are sticky sessions, external session stores, and client-side tokens. Even after scaling your application servers, the database often becomes the bottleneck, which sets the stage for topics covered in the rest of this book.

Key Points

- A single server works for small applications but has hard limits on CPU, memory, disk, and network
- Vertical scaling (bigger machine) is simple but expensive and has a ceiling
- Horizontal scaling (more machines) is flexible but requires stateless design
- Stateful servers store user data in memory, making scaling difficult
- Stateless servers store no user data, enabling any server to handle any request
- Session strategies: sticky sessions (simple but fragile), external store (robust), JWT (no server storage)
- The database is usually the first bottleneck you hit after scaling application servers

---

Practice Questions

1. Your single-server application is seeing 95% CPU utilization during peak hours. Response times have doubled. Would you scale vertically or horizontally first? Explain your reasoning.
2. You have a stateful application server storing user shopping carts in memory. You need to add a second server. What are your options for handling the shopping cart data, and what are the trade-offs of each?
3. A startup launches with 100 users and plans to reach 1 million in 2 years. Should they start with a scalable, horizontally scaled architecture from day one? Why or why not?
4. Explain why a JWT-based session system is harder to "log out" a user from compared to a Redis-based session store. What are some workarounds?
5. Your monitoring shows application server CPU is at 20% but the database CPU is at 90%. Would adding more application servers help? What would you do instead?

Exercises

Exercise 1: Architecture Evolution

Draw the architecture for each stage of growth for a photo-sharing app:

- Stage 1: 100 users (single server)
- Stage 2: 10,000 users (need to separate concerns)
- Stage 3: 1,000,000 users (need horizontal scaling)

For each stage, identify: what changed, why, and what new problems the change introduces.

Exercise 2: Stateless Refactoring

You have a Flask (Python) web application that stores login sessions in a Python dictionary in memory:

```python
sessions = {}

@app.route('/login', methods=['POST'])
def login():
    user = authenticate(request.form['username'], request.form['password'])
    session_id = generate_session_id()
    sessions[session_id] = {'user_id': user.id, 'login_time': time.time()}
    response = make_response(redirect('/dashboard'))
    response.set_cookie('session_id', session_id)
    return response
```

Refactor this to use Redis as an external session store. What changes do you need to make? What new failure modes does this introduce?

Exercise 3: Capacity Planning

Your e-commerce site gets the following traffic:

- Average: 500 requests/second
- Black Friday peak: 15,000 requests/second
- Each app server can handle 1,000 requests/second

Calculate:

- How many servers do you need for average traffic?
- How many for Black Friday?
- If each server costs $200/month, what is the cost difference between always running peak capacity vs auto-scaling?

What Is Next?

You now understand that horizontal scaling requires distributing requests across multiple servers. But how does a user's request reach the right server? And how do you make sure the

load is spread evenly? In Chapter 3, we will dive into load balancing -- the traffic cop that sits between users and your servers, directing each request to the best available server. We will cover algorithms, health checks, L4 vs L7 load balancing, and how to avoid making the load balancer itself a single point of failure.

Chapter 3: Load Balancing

What You Will Learn

- What a load balancer does and why every scaled system needs one
- The most common load balancing algorithms and when to use each
- The difference between Layer 4 and Layer 7 load balancing
- How health checks keep traffic away from broken servers
- Active-passive vs active-active load balancer setups
- Software load balancers (Nginx, HAProxy) vs hardware appliances
- Global Server Load Balancing (GSLB) for multi-region deployments
- How to prevent the load balancer itself from becoming a single point of failure

Why This Chapter Matters

In the previous chapter, you learned that horizontal scaling means running multiple servers. But that raises an immediate question: how does a user's request find the right server?

If you just give users the IP address of one server, you are back to a single point of failure. If you give them a list of IPs and say "pick one," you have chaos.

You need a traffic cop -- something that stands at the front door, accepts every incoming request, and routes it to the best available server. That is a load balancer, and it is one of the most critical components in any production system.

3.1 What Is a Load Balancer?

A load balancer is a device or software that distributes incoming network traffic across multiple servers. It ensures no single server bears too much load while keeping unhealthy servers out of the rotation.

The Traffic Cop Analogy

Imagine a busy intersection with four lanes leading to a shopping mall's parking garage. Without a traffic cop, all cars might try to enter through the same lane, causing a jam. Three lanes sit empty.

A traffic cop stands at the intersection and directs cars: "You go left. You go right. You go straight. You wait -- that lane is full."

A load balancer does exactly this for network requests.

```
                   +--------+
                   | Client |
                   +---+----+
                       |
                       v
               +------+-------+
               | LOAD BALANCER|
               | (Traffic Cop)|
               +------+-------+
                       |
          +------------+------------+
          |            |            |
          v            v            v
     +--------+   +--------+   +--------+
     |Server 1|   |Server 2|   |Server 3|
     | (25%)  |   | (25%)  |   | (50%)  |
     +--------+   +--------+   +--------+

     The load balancer decides which server
     handles each incoming request.
```

What a Load Balancer Does

1. Distributes traffic: Spreads requests across servers using a chosen algorithm
2. Monitors health: Checks if servers are alive and removes dead ones from rotation
3. Provides a single entry point: Users connect to one IP/domain, unaware of multiple backend servers
4. Enables scaling: You can add or remove servers without users noticing
5. Improves availability: If a server fails, traffic is automatically rerouted

Where Load Balancers Sit

Load balancers can be placed at multiple tiers in your architecture:

```
+--------+
| Client |
+---+----+
    |
    v
+---+---+  <-- LB Tier 1: Between clients and web servers
| LB #1 |
+---+---+
    |
+---+---+---+
|   |   |   |
v   v   v   v
+--+ +--+ +--+ +--+
|W1| |W2| |W3| |W4|   Web Servers
+--+ +--+ +--+ +--+
    |       |
    v       v
  +-+-+   +-+-+  <-- LB Tier 2: Between web and app servers
  |LB2|   |LB3|
  +-+-+   +-+-+
    |       |
  +-+-+   +-+-+
  |A1 |   |A2 |       App Servers
  +---+   +---+
    |       |
    v       v
  +-+-------+-+  <-- LB Tier 3: Between app and database servers
  |   LB #4   |
  +-----+-----+
        |
  +-----+-----+
  |     |     |
  v     v     v
+---+ +---+ +---+
|DB1| |DB2| |DB3|    Database Servers
+---+ +---+ +---+
```

3.2 Load Balancing Algorithms

How does the load balancer decide which server gets the next request? That is determined by the algorithm. Each has trade-offs.

Round Robin

The simplest algorithm. Requests go to servers in order: 1, 2, 3, 1, 2, 3, and so on.

```
Request 1 --> Server 1
Request 2 --> Server 2
Request 3 --> Server 3
Request 4 --> Server 1  (back to start)
Request 5 --> Server 2
Request 6 --> Server 3
...
```

Pros: Simple, fair distribution, no state needed. Cons: Ignores server capacity and current load. If Server 1 is slower than Server 2, it still gets equal traffic.

Best for: Servers with identical specs and requests that take roughly equal time.

Weighted Round Robin

Like round robin, but servers with more capacity get more requests.

```
Server weights: Server 1 = 3, Server 2 = 1, Server 3 = 1

Request 1 --> Server 1
Request 2 --> Server 1
Request 3 --> Server 1
Request 4 --> Server 2
Request 5 --> Server 3
Request 6 --> Server 1  (cycle repeats)
...
```

Pros: Accounts for different server capacities. Cons: Weights are static. Does not react to real-time load.

Best for: Mixed hardware environments where some servers are more powerful.

Least Connections

Each new request goes to the server with the fewest active connections.

```
Current state:
  Server 1: 12 active connections
  Server 2:  5 active connections  <-- next request goes here
  Server 3:  8 active connections
```

Pros: Adapts to real-time load. Handles requests of varying duration well. Cons: Requires tracking connection counts. Slightly more overhead.

Best for: Applications where request processing times vary widely (e.g., some requests take 10ms, others take 5 seconds).

Weighted Least Connections

Combines least connections with server weights. A server with weight 2 and 10 connections is considered less loaded than a server with weight 1 and 8 connections.

Best for: Mixed hardware with variable request durations.

IP Hash

The client's IP address is hashed to determine which server handles the request. The same IP always goes to the same server.

```
hash("192.168.1.10") % 3 = 0 --> Server 1
hash("192.168.1.11") % 3 = 2 --> Server 3
hash("192.168.1.12") % 3 = 1 --> Server 2

Same client always hits the same server.
```

Pros: Ensures session persistence without sticky session cookies. Good for caching (same user always hits the same cache). Cons: Uneven distribution if IP addresses are not uniformly distributed. Adding/removing servers changes the mapping (consistent hashing helps here).

Best for: Systems that benefit from server affinity without explicit session tracking.

Random

Each request is assigned to a randomly chosen server.

Pros: Dead simple. Surprisingly effective with many servers (the law of large numbers evens it out). Cons: No guarantee of even distribution, especially with few servers.

Best for: Large server pools where statistical distribution is acceptable.

Algorithm Comparison Table

Algorithm	Complexity	Adapts to Load	Session Affinity	Best Use Case
Round Robin	Very Low	No	No	Identical servers, uniform requests

Weighted Round Robin	Low	No	No	Mixed hardware
Least Connections	Medium	Yes	No	Variable request durations
Weighted Least Conn	Medium	Yes	No	Mixed hardware + variable durations
IP Hash	Low	No	Yes	Session affinity, caching
Random	Very Low	No	No	Large pools, simple setup

3.3 Layer 4 vs Layer 7 Load Balancing

Load balancers can operate at different layers of the network stack. The two most common are Layer 4 (transport) and Layer 7 (application).

Layer 4 Load Balancing

Operates at the TCP/UDP level. It sees IP addresses and port numbers but does not look inside the actual request content.

```
L4 LOAD BALANCER

Client request:
  Source IP: 192.168.1.10
  Dest IP:   10.0.0.1 (LB)
  Dest Port: 443

LB decision based on: IP + Port
(Does NOT read HTTP headers, URL, cookies, etc.)

Forwards raw TCP connection to chosen server.
```

Pros:

- Very fast (no need to parse request content)
- Protocol agnostic (works with HTTP, WebSocket, gRPC, databases, anything TCP/UDP)
- Lower resource consumption

Cons:

- Cannot make routing decisions based on URL path, headers, or cookies
- Cannot modify request/response content
- Limited health check options (TCP connect vs actual HTTP response)

Layer 7 Load Balancing

Operates at the HTTP/HTTPS level. It reads and understands the full request: URL, headers, cookies, request body.

```
L7 LOAD BALANCER

Client request:
  GET /api/users HTTP/1.1
  Host: example.com
  Cookie: session=abc123
  Content-Type: application/json

LB can route based on:
  - URL path (/api/* vs /static/*)
  - Host header (api.example.com vs www.example.com)
  - Cookie values
  - HTTP method (GET vs POST)
  - Request headers
  - Even request body content
```

Pros:

- Smart routing based on content (URL path, headers, cookies)
- Can cache responses
- Can compress/decompress content
- Can terminate SSL (handle HTTPS)
- Better health checks (can check HTTP response codes and content)
- Can modify requests and responses (add headers, rewrite URLs)

Cons:

- Slower than L4 (must parse every request)
- More resource intensive
- Only works with HTTP/HTTPS (and sometimes other specific protocols)

L4 vs L7 Comparison

```
L4 LOAD BALANCER                    L7 LOAD BALANCER
+------------------+                +------------------+
```

```
| Sees:            |                | Sees:             |
| - IP addresses   |                | - Everything L4   |
| - Port numbers   |                |   sees PLUS:      |
| - TCP/UDP        |                | - URL path        |
|                  |                | - HTTP headers    |
| Cannot see:      |                | - Cookies         |
| - URL path       |                | - Request body    |
| - HTTP headers   |                | - HTTP method     |
| - Cookies        |                |                   |
| - Request body   |                | Can do:           |
|                  |                | - Content routing |
| Fast, simple,    |                | - SSL termination|
| protocol agnostic|                | - Caching         |
+------------------+                +-------------------+
```

Real-World Example: Content-Based Routing with L7

```
L7 Load Balancer Routing Rules:

/api/*          --> API server pool (4 servers)
/images/*       --> Image server pool (2 servers)
/static/*       --> Static file server pool (2 servers)
/admin/*        --> Admin server (1 server, internal only)
everything else --> Web server pool (3 servers)

+--------+
| Client |
+---+----+
    |
    v
+---+-----------+
| L7 LB         |
| /api/* ?      +---> API Servers [S1] [S2] [S3] [S4]
| /images/* ?   +---> Image Servers [S5] [S6]
| /static/* ?   +---> Static Servers [S7] [S8]
| /admin/* ?    +---> Admin Server [S9]
| /* (default)  +---> Web Servers [S10] [S11] [S12]
+---------------+
```

3.4 Health Checks

A load balancer is only useful if it knows which servers are healthy. Health checks are periodic tests that verify server availability.

How Health Checks Work

```
HEALTH CHECK CYCLE

Load Balancer sends check every N seconds:

  LB ---[GET /health]--> Server 1: 200 OK      (healthy)
  LB ---[GET /health]--> Server 2: 200 OK      (healthy)
  LB ---[GET /health]--> Server 3: 503 Error   (unhealthy!)
  LB ---[GET /health]--> Server 4: timeout      (unhealthy!)

After M consecutive failures:
  Server 3 is removed from pool
  Server 4 is removed from pool

Traffic now goes only to Servers 1 and 2.

After Server 3 passes K consecutive checks:
  Server 3 is added back to pool.
```

Types of Health Checks

TCP Check (Layer 4):

- Tries to open a TCP connection to the server
- If the connection succeeds, the server is healthy
- Simple and fast, but does not verify the application is actually working

HTTP Check (Layer 7):

- Sends an HTTP request (usually GET /health)
- Checks the response status code (expecting 200)
- Can also check response content
- More accurate but more overhead

Application-Level Check:

- The /health endpoint verifies that the application can reach its dependencies
- Checks database connectivity, cache availability, disk space
- Most thorough but slowest

Example: A Good Health Check Endpoint

```python
@app.route('/health')
def health_check():
```

```
    checks = {}

    # Check database connectivity
    try:
        db.execute("SELECT 1")
        checks['database'] = 'ok'
    except Exception:
        checks['database'] = 'failed'

    # Check Redis connectivity
    try:
        redis_client.ping()
        checks['redis'] = 'ok'
    except Exception:
        checks['redis'] = 'failed'

    # Check disk space
    disk_usage = get_disk_usage_percent()
    checks['disk'] = 'ok' if disk_usage < 90 else 'warning'

    # Determine overall health
    all_ok = all(v == 'ok' for v in checks.values())
    status_code = 200 if all_ok else 503

    return jsonify({
        'status': 'healthy' if all_ok else 'unhealthy',
        'checks': checks
    }), status_code
```

Health Check Configuration Parameters

Parameter	Description	Typical Value
Interval	Time between checks	5-30 seconds
Timeout	Max wait for response	2-5 seconds
Unhealthy threshold	Consecutive failures to mark unhealthy	2-3
Healthy threshold	Consecutive successes to mark healthy	2-5
Path	URL to check (for HTTP checks)	/health

3.5 Active-Passive vs Active-Active

What happens if the load balancer itself fails? After all, it is also a server that can crash.

Active-Passive (Failover)

One load balancer handles all traffic (active). A second load balancer sits idle, monitoring the first (passive). If the active one fails, the passive one takes over.

```
ACTIVE-PASSIVE

Normal operation:
+--------+                          +----------+
| Client | -----> [Active LB] ---> | Servers  |
+--------+       /                  +----------+
                /
         [Passive LB]
         (monitoring,
          ready to
          take over)

After active LB fails:
+--------+                          +----------+
| Client | -----> [Passive LB] --> | Servers  |
+--------+   (now active)         +----------+

         [Dead LB]
         (being repaired)
```

How it works:

- Both LBs share a virtual IP (VIP) address
- The passive LB sends heartbeats to the active LB
- If heartbeats stop, the passive LB claims the VIP
- This switchover typically takes a few seconds

Pros: Simple, clear roles. Cons: Wasteful (passive LB sits idle). Switchover causes brief downtime.

Active-Active

Both load balancers handle traffic simultaneously. If one fails, the other absorbs all traffic.

```
ACTIVE-ACTIVE

Normal operation:
+--------+     [LB #1] --+---> +----------+
| Client | --->           |     | Servers  |
+--------+     [LB #2] --+---> +----------+
```

```
Both LBs handle traffic. DNS or an upstream
router distributes clients across both.

After LB #1 fails:
+--------+                      +----------+
| Client | ---> [LB #2] ------> | Servers  |
+--------+                      +----------+

LB #2 handles all traffic until LB #1 recovers.
```

Pros: No wasted resources. Better throughput (both handle traffic). Faster failover. Cons: More complex configuration. Need to ensure both LBs have consistent state.

3.6 Software vs Hardware Load Balancers

Software Load Balancers

Software that runs on standard servers. The two most popular are Nginx and HAProxy.

Nginx Configuration Example:

```nginx
upstream backend_servers {
    least_conn;  # Use least connections algorithm

    server 10.0.1.1:8080 weight=3;
    server 10.0.1.2:8080 weight=2;
    server 10.0.1.3:8080 weight=1;

    # Health check parameters
    server 10.0.1.4:8080 backup;  # Only used if others are down
}

server {
    listen 80;
    server_name example.com;

    location / {
        proxy_pass http://backend_servers;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

        # Timeouts
```

```
        proxy_connect_timeout 5s;
        proxy_read_timeout 30s;
    }

    location /health {
        access_log off;
        return 200 "OK";
    }
}
```

HAProxy Configuration Example:

```
frontend http_front
    bind *:80
    default_backend http_back

backend http_back
    balance roundrobin
    option httpchk GET /health

    server server1 10.0.1.1:8080 check weight 3
    server server2 10.0.1.2:8080 check weight 2
    server server3 10.0.1.3:8080 check weight 1
    server server4 10.0.1.4:8080 check backup
```

Hardware Load Balancers

Dedicated physical appliances from vendors like F5 (BIG-IP) and Citrix (NetScaler). They are purpose-built for high-performance traffic distribution.

Software vs Hardware Comparison

Aspect	Software (Nginx/HAProxy)	Hardware (F5/Citrix)
Cost	Free or low cost	$10,000 - $100,000+
Performance	Very good (millions of req/s)	Exceptional (specialized chips)
Flexibility	Highly configurable	Vendor-dependent
Scalability	Scale by adding servers	Buy bigger appliance
Cloud compatibility	Excellent	Limited
Setup complexity	Moderate	High (specialized knowledge)
Maintenance	Community + self	Vendor support contracts

Cloud Load Balancers

In cloud environments, managed load balancers are the most common choice:

- AWS: Elastic Load Balancer (ALB for L7, NLB for L4)
- Google Cloud: Cloud Load Balancing
- Azure: Azure Load Balancer

These are software load balancers managed by the cloud provider. You do not maintain the infrastructure, and they scale automatically.

3.7 Global Server Load Balancing (GSLB)

When your system spans multiple data centers or regions, you need a way to route users to the nearest or best-performing data center. This is GSLB.

How GSLB Works

GSLB typically uses DNS to route users to the best data center before a request even reaches a local load balancer.

```
GLOBAL SERVER LOAD BALANCING

User in Tokyo requests example.com

1. DNS query: "Where is example.com?"

2. GSLB DNS server considers:
   - User's geographic location (Tokyo)
   - Health of each data center
   - Current load at each data center
   - Latency from user to each data center

3. DNS response: "example.com is at 203.0.113.10"
   (IP of the Tokyo data center)

4. User connects to Tokyo data center

+----------+                              +-----------+
| User in  |  DNS: example.com?           | GSLB DNS  |
| Tokyo    | ---------------------------> | Server    |
+----------+                              +-----------+
```

```
      |                                          |
      |    Answer: 203.0.113.10 (Tokyo DC)     |
      | <----------------------------------------+
      |
      +---> Tokyo Data Center
            +--------+    +--------+
            | LB     |--->|Servers |
            +--------+    +--------+

Meanwhile...

+----------+                              +-----------+
| User in  |  DNS: example.com?           | GSLB DNS  |
| London   | ---------------------------> | Server    |
+----------+                              +-----------+
      |                                          |
      |    Answer: 198.51.100.20 (London DC)   |
      | <----------------------------------------+
      |
      +---> London Data Center
            +--------+    +--------+
            | LB     |--->|Servers |
            +--------+    +--------+
```

GSLB Routing Methods

Method	How It Works	Best For
Geographic	Route to nearest data center	Latency-sensitive apps
Latency-based	Route to lowest-latency data center	Performance optimization
Weighted	Distribute traffic by percentage across regions	Gradual rollouts
Failover	Route to primary, switch to backup if primary fails	Disaster recovery
Round Robin DNS	Alternate between data center IPs	Simple distribution

---

3.8 Avoiding the Single Point of Failure

A load balancer that is itself a single point of failure defeats its purpose. Here is how to prevent that:

```
REDUNDANT LOAD BALANCER SETUP

                     +----------+
                     |   DNS    |
                     | (points  |
                     | to VIP)  |
                     +----+-----+
                          |
                    Virtual IP (VIP)
                     shared between
                     both LBs
                          |
               +----------+----------+
               |                     |
          +----+----+          +----+----+
          | LB #1   |          | LB #2   |
          | (Active) | <------> | (Standby)|
          +----+----+ heartbeat+----+----+
               |                     |
               +----------+----------+
                          |
               +----------+----------+
               |          |          |
          +----+--+ +----+--+ +----+--+
          |Server1| |Server2| |Server3|
          +-------+ +-------+ +-------+

If LB #1 fails, LB #2 claims the VIP
and starts handling all traffic.
Users see no change (same IP address).
```

Strategies for LB High Availability

1. Virtual IP (VIP) with failover: Two LBs share a floating IP. Tools like Keepalived manage the failover.
2. DNS-based: Multiple LB IPs in DNS. If one goes down, DNS health checks remove it.
3. Cloud managed: AWS ALB, GCP Cloud LB, etc., are inherently highly available (the cloud provider handles redundancy).
4. Anycast IP: The same IP is advertised from multiple locations. Network routing handles failover.

Trade-Offs Table

Decision	Option A	Option B	When to Choose A	When to Choose B
L4 vs L7	Layer 4 (TCP)	Layer 7 (HTTP)	Non-HTTP protocols, raw speed needed	Content routing, SSL termination, caching
Round Robin vs Least Conn	Round Robin	Least Connections	Uniform requests, same servers	Variable request times, mixed hardware
Active-Passive vs Active-Active	Active-Passive	Active-Active	Simpler setup, lower cost	Maximum uptime, better resource use
Software vs Hardware LB	Software (Nginx/HAProxy)	Hardware (F5)	Cost-sensitive, cloud environment	Ultra-high throughput, enterprise compliance
Local vs Global LB	Local LB only	GSLB + Local LB	Single region deployment	Multi-region, global user base

Common Mistakes

1. Using the load balancer as the only line of defense. A load balancer distributes traffic but does not fix application bugs or database bottlenecks.
2. Not setting up health checks. Without health checks, the load balancer sends traffic to dead servers, causing errors for users.
3. Overly aggressive health checks. Checking every 1 second with a 1-failure threshold means a single slow response removes a server from rotation. Transient issues become outages.
4. Ignoring SSL/TLS termination. If every backend server handles its own SSL, you waste CPU and complicate certificate management. Let the load balancer handle it.
5. Single load balancer without redundancy. You added a load balancer to avoid a single point of failure in your servers, but the load balancer itself is now one.
6. Choosing the wrong algorithm. Round robin when your servers have different capacities, or least connections when all requests take the same time (unnecessary overhead).

Best Practices

1. Always configure health checks. Both TCP (is the port open?) and HTTP (does the application respond correctly?).
2. Use L7 load balancing for web applications. The ability to route based on URL path, terminate SSL, and inspect headers is worth the small performance cost.
3. Set up redundant load balancers. Active-passive at minimum. Active-active if you can manage the complexity.
4. Terminate SSL at the load balancer. Manage certificates in one place, offload encryption from your application servers.
5. Start with round robin or least connections. These cover 90% of use cases. Only get fancy when you have a specific need.
6. Monitor your load balancer. Track connection counts, error rates, latency, and backend server health. The LB is the front door -- if it is slow, everything is slow.
7. Use cloud-managed load balancers when possible. They handle redundancy, scaling, and SSL certificate management for you.

Quick Summary

A load balancer distributes incoming traffic across multiple servers, acting as a traffic cop for your system. The most common algorithms are round robin, weighted round robin, least connections, and IP hash. Layer 4 load balancers work at the TCP level (fast but limited), while Layer 7 load balancers understand HTTP (smarter but heavier). Health checks ensure traffic only goes to healthy servers. Active-passive setups provide failover redundancy, while active-active setups use both load balancers simultaneously. For global deployments, GSLB uses DNS to route users to the nearest data center. The load balancer itself must be made redundant to avoid becoming the very single point of failure it was meant to eliminate.

Key Points

- A load balancer distributes traffic across multiple servers and removes unhealthy servers from rotation
- Round robin is simplest; least connections adapts to real-time load; IP hash provides session affinity
- L4 load balancing is fast and protocol-agnostic; L7 load balancing is smarter and can route by content

- Health checks are essential -- without them, broken servers still receive traffic
- Always make the load balancer itself redundant (active-passive or active-active)
- Cloud-managed load balancers (AWS ALB, GCP LB) handle redundancy and scaling automatically
- GSLB routes users to the best data center using DNS-based routing

---

Practice Questions

1. You have 3 backend servers. Server A has 16 GB RAM, Server B has 8 GB RAM, and Server C has 8 GB RAM. Which load balancing algorithm would you choose and why? What weights would you assign?
2. Your application uses WebSocket connections for real-time chat. Should you use L4 or L7 load balancing? What are the implications of each choice?
3. A load balancer is configured with 5-second health check intervals and a 2-failure threshold. Server 2 starts returning 503 errors. How long (at minimum) before the server is removed from rotation? What happens to requests sent to Server 2 during that window?
4. Explain why an active-active load balancer setup is better for availability but more complex than active-passive. What specific challenges does active-active introduce?
5. Your system serves users in North America, Europe, and Asia. You have data centers in Virginia, Frankfurt, and Tokyo. Describe how you would set up GSLB to route users to the nearest data center, and what happens if the Frankfurt data center goes down.

Exercises

Exercise 1: Nginx Configuration

Write an Nginx configuration that:

- Listens on port 443 with SSL
- Routes `/api/*` requests to a pool of 3 API servers using least connections
- Routes `/static/*` requests to a pool of 2 static file servers using round robin
- Includes a health check endpoint that returns 200
- Has a backup server that only receives traffic if all primary servers are down

Exercise 2: Algorithm Selection

For each of the following scenarios, choose the best load balancing algorithm and explain your reasoning:

- A video streaming service where videos take varying amounts of time to transcode
- A static website served by 10 identical servers
- An e-commerce site where keeping users on the same server improves cache hit rates
- A microservice that processes both quick (5ms) and slow (30s) requests

Exercise 3: Failure Scenario Analysis

Draw the architecture of a system with two active-passive load balancers, six backend servers, and a GSLB for two data centers. Then walk through what happens when:

- One backend server crashes
- The active load balancer in Data Center 1 crashes
- The entire Data Center 1 goes offline
- The network link between the two data centers goes down

For each scenario, describe the user impact, the automated recovery, and how long users might be affected.

What Is Next?

Load balancers distribute traffic, but they do not reduce the amount of work each server does. If every request requires a database query that takes 50 milliseconds, adding servers only gets you so far. In Chapter 4, we will explore caching -- the art of keeping frequently accessed data close at hand so you can serve it in microseconds instead of milliseconds. Caching is one of the most powerful tools in system design, and it works at every level of the stack.

Chapter 4: Caching

What You Will Learn

- What caching is and why it is the single biggest performance lever in most systems
- The difference between a cache hit and a cache miss
- Core caching strategies: cache-aside, read-through, write-through, write-behind, and write-around
- How TTL (Time To Live) controls data freshness
- Eviction policies: LRU, LFU, FIFO, and when to choose each
- Redis vs Memcached and how to pick between them
- CDNs as a caching layer
- Distributed caching and its challenges
- Cache stampede, cache penetration, and cache avalanche -- and how to prevent them

Why This Chapter Matters

Every millisecond of latency costs you users. Amazon found that every 100ms of latency costs them 1% in sales. Google found that an extra 0.5 seconds in search page generation dropped traffic by 20%.

Caching is the single most effective technique for reducing latency. It works by storing a copy of frequently accessed data in a faster storage layer so future requests can be served without going back to the slow original source.

If load balancing (Chapter 3) is about spreading work across servers, caching is about avoiding the work altogether. A request that never hits your database is infinitely faster than one that does.

4.1 What Is Caching?

Caching is the practice of storing copies of data in a temporary, fast-access storage layer so that future requests for the same data can be served more quickly.

The Bookshelf Analogy

Imagine you are a student writing a research paper. You need to reference the same five books over and over.

Without caching: Every time you need a quote, you walk to the university library, find the book on the shelf, copy the passage, walk back to your desk, and use it. If you need the same quote ten minutes later, you make the same trip.

With caching: You borrow those five books and keep them on your desk. Now when you need a quote, you reach over and grab the book. No walking, no searching, no waiting.

Your desk is the cache. The library is the database. The books on your desk are cached data.

```
WITHOUT CACHING:

  You --> Walk to Library --> Find Book --> Copy Quote --> Walk Back
  You --> Walk to Library --> Find Book --> Copy Quote --> Walk Back
  You --> Walk to Library --> Find Book --> Copy Quote --> Walk Back

  Total time: 30 minutes (3 trips x 10 min each)


WITH CACHING:

  You --> Grab Book from Desk --> Copy Quote
  You --> Grab Book from Desk --> Copy Quote
  You --> Grab Book from Desk --> Copy Quote

  Total time: 30 seconds (3 lookups x 10 sec each)
```

Where Caching Appears in a System

Caching is not a single technology -- it happens at every layer:

```
+----------+     +--------+     +-----------+     +--------+     +--------+
|  Browser |---->|  CDN   |---->|   Load    |---->|  App   |---->|  DB    |
|  Cache   |     |  Cache |     |  Balancer |     | Server |     |        |
+----------+     +--------+     +-----------+     +---+----+     +--------+
                                                      |
                                                      v
```

```
                                                    +----------+
                                                    | In-Memory|
                                                    |  Cache   |
                                                    | (Redis)  |
                                                    +----------+

Layer 1: Browser cache (images, CSS, JS)
Layer 2: CDN cache (static assets at the edge)
Layer 3: Application cache (Redis/Memcached)
Layer 4: Database query cache
Layer 5: CPU caches (L1, L2, L3)
```

Speed Comparison

Understanding why caching works requires knowing how fast different storage layers are:

```
+---------------------+------------------+------------------+
| Storage Layer       | Access Time      | Analogy          |
+---------------------+------------------+------------------+
| CPU L1 Cache        | ~1 nanosecond    | Blink of an eye  |
| CPU L2 Cache        | ~4 nanoseconds   | One heartbeat    |
| RAM                 | ~100 nanoseconds | Snap your fingers|
| SSD Read            | ~100 microseconds| Deep breath      |
| HDD Read            | ~10 milliseconds | Make a coffee    |
| Network (same DC)   | ~0.5 milliseconds| Walk to mailbox  |
| Network (cross DC)  | ~50 milliseconds | Drive to store   |
+---------------------+------------------+------------------+

RAM is roughly 1,000x faster than SSD
SSD is roughly 100x faster than HDD
```

4.2 Cache Hit and Cache Miss

Every cache lookup results in one of two outcomes.

Cache Hit

The requested data is found in the cache. The system returns it directly without going to the slower backend store. This is the fast path.

Cache Miss

The requested data is NOT in the cache. The system must fetch it from the original source (database, API, disk), and then typically stores a copy in the cache for future requests.

```
CACHE HIT:
                                  +-------+
  Client ----> Request ---+       | Cache |
                          |  +--->| (HIT) |---+
                          |  |    +-------+   |
                          +--+                |
                             |                v
                             |          Return Data
                             |          (FAST: ~1ms)
                             |
                             X  (Database never touched)


CACHE MISS:
                                  +-------+
  Client ----> Request ---+       | Cache |
                          |  +--->| (MISS)|
                          |  |    +-------+
                          +--+        |
                                      | Not found!
                                      v
                                 +----------+
                                 | Database |
                                 +----+-----+
                                      |
                                      v
                                 Fetch Data
                                      |
                          +-----------+-----------+
                          |                       |
                          v                       v
                     Store in Cache          Return Data
                     (for next time)        (SLOW: ~50ms)
```

Cache Hit Ratio

The hit ratio is the percentage of requests served from cache:

```
Hit Ratio = Cache Hits / (Cache Hits + Cache Misses) x 100

Example:
  950 hits + 50 misses = 1000 total requests
  Hit Ratio = 950 / 1000 = 95%
```

A good cache hit ratio is typically 90% or higher. Even going from 90% to 95% cuts your database load in half (from 10% misses to 5% misses).

4.3 Caching Strategies

There is no single "right" way to use a cache. Different strategies suit different access patterns. Here are the five most important ones.

Cache-Aside (Lazy Loading)

This is the most common caching strategy. The application manages the cache explicitly.

How it works:

1. Application checks the cache first
2. If hit, return cached data
3. If miss, read from database
4. Write the result into cache
5. Return data to client

```
CACHE-ASIDE PATTERN:

         +----------+
         |  Client  |
         +----+-----+
              |
              v
         +----+-----+
         |   App    |
         |  Server  |
         +----+-----+
              |
         1. Check cache
              |
              v
         +----+-----+
         |   Cache  |-----> HIT? Return data (done!)
         +----+-----+
              |
              | MISS
              v
         +----+-----+
    2.   | Database |
         +----+-----+
              |
         3. Read data
              |
              v
```

```
          +----+-----+
     4.   |   Cache  |  <-- Store for next time
          +----+-----+
               |
          5. Return to client
```

Pros:

- Only requested data is cached (no wasted memory)
- Cache failure does not break the system (just slower)
- Simple to implement

Cons:

- First request is always a cache miss (cold start)
- Data can become stale (cache has old data while database has new data)
- Three round trips on a miss (check cache, read database, write cache)

Best for: Read-heavy workloads where you can tolerate slightly stale data.

Read-Through Cache

Similar to cache-aside, but the cache itself is responsible for loading data from the database on a miss. The application only talks to the cache.

```
READ-THROUGH PATTERN:

          +----------+
          |  Client  |
          +----+-----+
               |
               v
          +----+-----+
          |   App    | -- Only talks to cache
          |  Server  |
          +----+-----+
               |
               v
          +----+-----+         +----------+
          |   Cache  |-------->| Database |
          +----------+         +----------+
             |    ^                  |
             |    |    On miss:      |
             |    +--- Cache loads --+
             |         data itself
             v
        Return data
```

Pros:

- Application code is simpler (no database-read-then-cache-write logic)
- Consistent data loading pattern

Cons:

- First request still has a miss penalty
- Cache library must support the read-through feature
- Harder to debug (logic hidden inside cache layer)

Best for: When you want to simplify application code and your cache library supports it.

Write-Through Cache

Every write goes to the cache AND the database before confirming to the client. The cache always has fresh data.

```
WRITE-THROUGH PATTERN:

          +----------+
          |  Client  |
          +----+-----+
               |
          Write request
               |
               v
          +----+-----+
          |   App    |
          |  Server  |
          +----+-----+
               |
               v
          +----+-----+    Synchronous    +----------+
          |   Cache  |----- Write ------>| Database |
          +----+-----+                    +----------+
               |         Both must succeed
               |         before confirming
               v
          Return "OK"
```

Pros:

- Cache is always consistent with the database
- No stale reads
- Good when combined with read-through

Cons:

- Higher write latency (must write to both cache and database)
- Writes to data that is never read waste cache space

- More complex failure handling

Best for: Systems where data consistency is critical and you read the same data you write.

Write-Behind (Write-Back) Cache

Writes go to the cache immediately, and the cache asynchronously flushes changes to the database later. The client gets a fast confirmation.

```
WRITE-BEHIND PATTERN:

         +----------+
         |  Client  |
         +----+-----+
              |
         Write request
              |
              v
         +----+-----+
         |   App    |
         |  Server  |
         +----+-----+
              |
              v
         +----+-----+       +----------+
    1.   |   Cache  |       | Database |
         +----+-----+       +----+-----+
              |                  ^
         2. Return "OK"          |
         (immediately!)          |
              |                  |
              +--- 3. Later ---->+
                   (async batch write)
```

Pros:

- Very low write latency (cache write is fast)
- Batch writes to database improve throughput
- Absorbs write spikes

Cons:

- Risk of data loss if cache crashes before flushing to database
- Complex to implement correctly
- Eventual consistency between cache and database

Best for: Write-heavy workloads where you can tolerate some data loss risk (counters, analytics, session data).

Write-Around Cache

Writes go directly to the database, bypassing the cache. The cache is only populated when data is read.

```
WRITE-AROUND PATTERN:

         +----------+
         |  Client  |
         +----+-----+
              |
       +------+------+
       |             |
    Read           Write
       |             |
       v             v
  +----+-----+  +----+-----+
  |   Cache  |  | Database |  <-- Writes skip the cache
  +----+-----+  +----------+
       |
       | MISS? Load from DB
       v
  +----------+
  | Database |
  +----------+
```

Pros:

- Cache is not flooded with write data that may never be read
- Good for write-once, read-maybe workloads

Cons:

- Recently written data always causes a cache miss on first read
- Higher read latency for recently written data

Best for: Situations where data is written frequently but read infrequently.

Strategy Comparison

```
+------------------+--------+--------+-----------+------------+
| Strategy         | Read   | Write  | Data Loss | Complexity |
|                  | Speed  | Speed  | Risk      |            |
+------------------+--------+--------+-----------+------------+
| Cache-Aside      | Fast*  | Normal | None      | Low        |
| Read-Through     | Fast*  | Normal | None      | Medium     |
| Write-Through    | Fast   | Slow   | None      | Medium     |
| Write-Behind     | Fast   | Fast   | Yes       | High       |
| Write-Around     | Slow** | Fast   | None      | Low        |
```

```
+------------------+--------+--------+-----------+------------+

*  After first miss
** For recently written data
```

4.4 TTL: Time To Live

A TTL is a timer attached to each cached entry. When the timer expires, the entry is removed (or marked stale). TTL controls how long data lives in the cache.

Why TTL Matters

Without a TTL, cached data lives forever. If the underlying data changes, your cache serves stale results indefinitely.

```
WITHOUT TTL:
  t=0:   Cache stores user_123.name = "Alice"
  t=10m: User changes name to "Bob" in database
  t=1hr: Cache STILL returns "Alice"  <-- STALE!
  t=1day: Cache STILL returns "Alice" <-- VERY STALE!

WITH TTL of 5 minutes:
  t=0:   Cache stores user_123.name = "Alice" (TTL=5min)
  t=5m:  Cache entry expires, removed automatically
  t=6m:  Request comes in --> cache miss --> reads "Bob" from DB
         Cache stores "Bob" (TTL=5min)  <-- FRESH!
```

Choosing the Right TTL

```
+----------------------+----------------+---------------------------+
| Data Type            | Suggested TTL  | Why                       |
+----------------------+----------------+---------------------------+
| User session         | 30 minutes     | Security + memory         |
| Product catalog      | 1-6 hours      | Changes infrequently      |
| Stock price          | 1-5 seconds    | Changes constantly        |
| Config/feature flags | 1-5 minutes    | Needs fast rollout        |
| Static assets (CSS)  | 1 year         | URL-versioned, never stale|
| Search results       | 5-15 minutes   | Good enough freshness     |
| DNS records          | 5 min - 48 hrs | Varies by provider        |
+----------------------+----------------+---------------------------+
```

The Freshness vs Performance Trade-off

```
Short TTL (seconds)                    Long TTL (hours/days)
<-------------------------------------------------------->
Fresh data                             Stale data
More DB load                           Less DB load
Lower hit ratio                        Higher hit ratio
Higher latency                         Lower latency
```

There is no universally correct TTL. It depends on how stale your data can be before users notice or care.

4.5 Cache Eviction Policies

Your cache has limited memory. When it is full and a new entry needs to be stored, the cache must decide which existing entry to remove. This decision is the eviction policy.

LRU: Least Recently Used

Remove the entry that has not been accessed for the longest time.

The analogy: Your desk can hold 5 books. When you need a 6th, you put away the one you have not opened in the longest time.

```
LRU EXAMPLE (capacity = 4):

Access: A, B, C, D, E, A, F

Step 1: [A]              -- Add A
Step 2: [A, B]           -- Add B
Step 3: [A, B, C]        -- Add C
Step 4: [A, B, C, D]     -- Full!
Step 5: [B, C, D, E]     -- Add E, evict A (least recently used)
Step 6: [C, D, E, A]     -- Access A (fetched from DB), evict B
Step 7: [D, E, A, F]     -- Add F, evict C (least recently used)

Most recently used -->  [D, E, A, F]  <-- Least recently used
                         ^                 ^
                         |                 |
                     Keep these        Evict this next
```

Pros: Simple, works well for most workloads Cons: A one-time scan of many items can pollute the cache (push out frequently used items)

Used by: Redis (approximated LRU), Memcached, most CPU caches

LFU: Least Frequently Used

Remove the entry that has been accessed the fewest times.

The analogy: You keep count of how many times you open each book. When your desk is full, you put away the one you have opened the fewest times.

```
LFU EXAMPLE (capacity = 3):

Action                    Cache State              Access Counts
------                    -----------              -------------
Access A                  [A]                      A:1
Access A                  [A]                      A:2
Access B                  [A, B]                   A:2, B:1
Access C                  [A, B, C]  (Full!)       A:2, B:1, C:1
Access A                  [A, B, C]                A:3, B:1, C:1
Access D                  [A, D, C] or [A, B, D]   A:3, D:1, ...
                          ^-- Evict B or C (both have count=1)
```

Pros: Keeps genuinely popular items in cache Cons: New items may be evicted too quickly (they start with count=1). Items that were popular long ago linger.

Used by: Redis (LFU mode), some CDN caches

FIFO: First In, First Out

Remove the oldest entry, regardless of how recently or frequently it was accessed.

```
FIFO EXAMPLE (capacity = 3):

Access: A, B, C, D, B, E

Step 1: [A]
Step 2: [A, B]
Step 3: [A, B, C]     -- Full!
Step 4: [B, C, D]     -- Add D, evict A (first in)
Step 5: [B, C, D]     -- B is already cached (hit)
Step 6: [C, D, E]     -- Add E, evict B (first in)
                          ^-- Even though B was just accessed!
```

Pros: Very simple, predictable Cons: Does not consider access patterns at all; can evict hot data

Used by: Some simple caching layers, message queues

Random Replacement

Remove a random entry. Surprisingly effective in some workloads.

Pros: No overhead for tracking access patterns Cons: Unpredictable; can evict hot data

Eviction Policy Comparison

```
+---------+------------+---------------+----------+------------------+
| Policy  | Tracks     | Overhead      | Quality  | Best For         |
+---------+------------+---------------+----------+------------------+
| LRU     | Last access| Medium        | Good     | General purpose  |
| LFU     | Frequency  | Higher        | Better*  | Stable hotspots  |
| FIFO    | Insert time| Lowest        | Fair     | Simple systems   |
| Random  | Nothing    | Lowest        | OK       | When simplicity  |
|         |            |               |          | matters most     |
+---------+------------+---------------+----------+------------------+

* Better when popular items remain popular over time
```

4.6 Redis vs Memcached

These are the two most popular in-memory caching systems. Both store data in RAM for fast access, but they differ in important ways.

Redis

Redis (Remote Dictionary Server) is an in-memory data store that supports rich data structures.

Key features:

- Data structures: strings, hashes, lists, sets, sorted sets, bitmaps, streams
- Persistence: can snapshot to disk (RDB) or log every write (AOF)
- Replication: leader-follower replication for high availability
- Clustering: built-in sharding across multiple nodes
- Pub/Sub: built-in messaging
- Lua scripting: atomic operations
- TTL support per key

Memcached

Memcached is a high-performance, distributed memory caching system. It does one thing and does it well: key-value caching.

Key features:

- Simple key-value store (string keys, string/binary values)
- Multi-threaded architecture (uses all CPU cores)
- No persistence (pure cache, data lost on restart)
- No replication (each node is independent)
- Simple protocol (easy to implement clients)
- Slab-based memory allocation (predictable memory usage)

Comparison

```
+--------------------+-------------------+-------------------+
| Feature            | Redis             | Memcached         |
+--------------------+-------------------+-------------------+
| Data structures    | Rich (lists, sets | Simple (string    |
|                    | hashes, sorted    | key-value only)   |
|                    | sets, etc.)       |                   |
+--------------------+-------------------+-------------------+
| Persistence        | Yes (RDB + AOF)   | No                |
+--------------------+-------------------+-------------------+
| Replication        | Yes (leader-      | No                |
|                    | follower)         |                   |
+--------------------+-------------------+-------------------+
| Clustering         | Yes (built-in)    | Client-side only  |
+--------------------+-------------------+-------------------+
| Threading          | Single-threaded*  | Multi-threaded    |
+--------------------+-------------------+-------------------+
| Max value size     | 512 MB            | 1 MB (default)    |
+--------------------+-------------------+-------------------+
| Memory efficiency  | Higher overhead   | Lower overhead    |
+--------------------+-------------------+-------------------+
| Pub/Sub            | Yes               | No                |
+--------------------+-------------------+-------------------+
| Scripting          | Lua scripts       | No                |
+--------------------+-------------------+-------------------+

* Redis 6+ has I/O threading for network operations
```

When to Choose Which

Choose Redis when:

- You need data structures beyond simple key-value (leaderboards, queues, sessions)
- You need persistence (data survives restarts)
- You need replication or clustering
- You need pub/sub or message brokering

Choose Memcached when:

- You need simple, fast key-value caching
- You want to maximize memory efficiency for caching
- You want multi-threaded performance on a single node
- Your caching needs are straightforward

In practice, Redis has become the default choice for most teams because of its versatility. Memcached remains popular for large-scale simple caching (Facebook uses it extensively).

4.7 CDN as a Cache

A CDN (Content Delivery Network) is essentially a geographically distributed cache. It stores copies of your content on servers around the world so users can access it from a nearby location.

We will cover CDNs in depth in Chapter 5. Here is how they fit into the caching picture:

```
WITHOUT CDN:
  User in Tokyo --> Request travels to Server in New York
  Latency: ~200ms

WITH CDN:
  User in Tokyo --> CDN Edge Server in Tokyo (cached copy)
  Latency: ~20ms

                     +-- Edge: Tokyo
                     |
  Origin Server -----+-- Edge: London
  (New York)         |
                     +-- Edge: Sydney
                     |
                     +-- Edge: Sao Paulo

  Each edge server caches content from the origin.
  Users connect to the nearest edge.
```

CDNs primarily cache static content (images, CSS, JavaScript, videos) but can also cache API responses and dynamically generated pages.

4.8 Distributed Caching

When your application runs on multiple servers, you have a choice: each server has its own local cache, or all servers share a distributed cache.

Local Cache vs Distributed Cache

```
LOCAL CACHE (each server has its own):

  +----------+     +----------+     +----------+
  | Server 1 |     | Server 2 |     | Server 3 |
  | +------+ |     | +------+ |     | +------+ |
  | |Cache | |     | |Cache | |     | |Cache | |
  | |A,B,C | |     | |A,D,E | |     | |B,F,G | |
  | +------+ |     | +------+ |     | +------+ |
  +----------+     +----------+     +----------+

  Problem: Data duplicated. Server 1 updates A,
  but Server 2 still has the old copy of A.


DISTRIBUTED CACHE (shared by all servers):

  +----------+     +----------+     +----------+
  | Server 1 |     | Server 2 |     | Server 3 |
  +-----+----+     +-----+----+     +-----+----+
        |                |                |
        +--------+-------+--------+-------+
                 |                |
           +-----+-----+   +-----+-----+
           |  Cache    |   |  Cache    |
           |  Node 1   |   |  Node 2   |
           |  A,B,C,D  |   |  E,F,G    |
           +-----------+   +-----------+

  All servers see the same cache.
  Data is partitioned across cache nodes.
```

Benefits of Distributed Cache

- Consistency: All application servers see the same cached data
- Capacity: Cache can be larger than any single machine's RAM
- Survivability: If one app server restarts, the cache is not lost

- Scalability: Add more cache nodes as data grows

Challenges of Distributed Cache

- Network latency: Cache is accessed over the network (~0.5ms) vs in-process (~0.001ms)
- Serialization cost: Data must be serialized/deserialized for network transfer
- Operational complexity: Another system to deploy, monitor, and maintain
- Partitioning: You need a strategy to distribute keys across nodes (see Chapter 12: Consistent Hashing)

4.9 Cache Problems and Solutions

Caching introduces its own set of problems. Understanding these is critical for building reliable systems.

Cache Stampede (Thundering Herd)

The problem: A popular cache entry expires. Hundreds of requests simultaneously find a cache miss and all hit the database at once, overwhelming it.

```
CACHE STAMPEDE:

  t=0: Cache entry for "hot_product" expires

  t=0.001s:  Request 1 --> Cache MISS --> Query DB
  t=0.002s:  Request 2 --> Cache MISS --> Query DB
  t=0.003s:  Request 3 --> Cache MISS --> Query DB
  ...
  t=0.050s:  Request 100 --> Cache MISS --> Query DB

  Database receives 100 identical queries simultaneously!

            +------+------+------+------+------+
            | Req1 | Req2 | Req3 | ...  |Req100|
            +--+---+--+---+--+---+--+---+--+---+
               |      |      |      |      |
               v      v      v      v      v
            +-------------------------------+
            |          DATABASE             |
            |      (OVERWHELMED!)           |
            +-------------------------------+
```

Solutions:

1. Locking: Only the first request on a miss fetches from the database. Other requests wait for the cache to be repopulated.

```
WITH LOCKING:
  Request 1 --> Cache MISS --> Acquire lock --> Query DB --> Update cache
  Request 2 --> Cache MISS --> Lock taken, WAIT...
  Request 3 --> Cache MISS --> Lock taken, WAIT...
  ...
  Cache updated! --> Requests 2, 3, ... --> Cache HIT
```

2. Early expiration (stale-while-revalidate): Start refreshing the cache before it actually expires. Serve slightly stale data while fetching fresh data in the background.
3. Jittered TTL: Add a random offset to TTLs so entries do not all expire at the same time.

```
Without jitter: All entries expire at t=300s
With jitter:    Entry A expires at t=287s
                Entry B expires at t=312s
                Entry C expires at t=295s
```

Cache Penetration

The problem: Repeated requests for data that does not exist. Each request misses the cache and hits the database, which also returns nothing. The cache never gets populated.

```
CACHE PENETRATION:

  Request for user_id=999999 (does not exist)
  --> Cache MISS
  --> Database query returns EMPTY
  --> Nothing to cache!
  --> Next request: same thing again

  Attacker sends millions of requests for non-existent IDs
  --> Every single one hits the database!
```

Solutions:

1. Cache null results: Store a "not found" marker in the cache with a short TTL.

```
Cache key: "user:999999" --> value: "NULL" (TTL=60s)
```

2. Bloom filter: A space-efficient data structure that can quickly tell you if an item definitely does NOT exist. Check the Bloom filter before checking the cache or database.

```
WITH BLOOM FILTER:

  Request for user_id=999999
  --> Check Bloom filter: "Definitely not in database"
  --> Return "not found" immediately
  --> Database never touched!
```

Cache Avalanche

The problem: A large number of cache entries expire at the same time (or the entire cache goes down), causing a massive spike in database load.

```
CACHE AVALANCHE:

  t=0: All cache entries set with TTL=3600s
  t=3600s: EVERYTHING expires at once!

  +---+---+---+---+---+---+---+---+
  | A | B | C | D | E | F | G | H |  ALL EXPIRED!
  +---+---+---+---+---+---+---+---+
    |   |   |   |   |   |   |   |
    v   v   v   v   v   v   v   v
  +-----------------------------------+
  |            DATABASE               |
  |          (CRUSHED!)               |
  +-----------------------------------+
```

Solutions:

1. Jittered TTL: Spread expiration times randomly
2. Multi-level caching: Use a local cache as a fallback when the distributed cache is down
3. Cache warm-up: Pre-populate the cache before it goes live
4. Circuit breaker: If the database is overwhelmed, fail fast instead of queuing more requests

---

4.10 Cache Invalidation

Cache invalidation -- deciding when to remove or update cached data -- is one of the hardest problems in computer science.

> "There are only two hard things in Computer Science: cache invalidation and naming things." -- Phil Karlton

Common Invalidation Strategies

1. TTL-based expiration: Set a timer. Data is removed when the timer expires. Simple but blunt.

2. Event-driven invalidation: When the source data changes, actively delete or update the cache entry.

```
EVENT-DRIVEN INVALIDATION:

  1. User updates profile
  2. App writes new data to database
  3. App deletes cache entry for that user
  4. Next read triggers cache miss --> loads fresh data

  +--------+    2. Write    +----------+
  |  App   |--------------->| Database |
  +---+----+                +----------+
      |
      | 3. Delete cache entry
      v
  +--------+
  |  Cache |  key "user:123" DELETED
  +--------+
```

3. Version-based invalidation: Include a version number in the cache key. When data changes, increment the version.

```
Version-based:
  v1: cache key = "product:42:v1"
  v2: cache key = "product:42:v2"  <-- New version, old key ignored
```

Common Mistakes

1. Caching everything. Only cache data that is read frequently. Caching rarely-read data wastes memory.
2. No TTL. Always set a TTL. Without one, stale data lives forever.
3. Ignoring cache warm-up. After a deploy or restart, the cache is empty. All requests hit the database. Pre-populate the cache with hot data.
4. Not monitoring cache hit ratio. If your hit ratio drops, something is wrong. Monitor it.

5. Treating cache as a source of truth. A cache is temporary. Data can be evicted at any time. Your system must work (slowly) without the cache.
6. Same TTL for all keys. Different data types need different TTLs. A stock price and a user profile should not share the same expiration.
7. No plan for cache failure. If Redis goes down, can your system still serve requests? If not, you have a single point of failure.

Best Practices

1. Start with cache-aside. It is the simplest and most flexible strategy. Add complexity only when needed.
2. Set appropriate TTLs. Balance freshness and performance for each data type.
3. Add jitter to TTLs. Prevent mass expiration by adding random offsets.
4. Monitor hit ratio, latency, and eviction rate. These three metrics tell you if your cache is healthy.
5. Use consistent hashing for distributed caches. See Chapter 12 for details.
6. Cache at multiple levels. Browser cache, CDN, application cache, and database cache each serve different purposes.
7. Design for cache failure. Your system should degrade gracefully, not crash, when the cache is unavailable.
8. Use the right eviction policy. LRU is a safe default. Switch to LFU if you have stable hotspots.
9. Keep cached objects small. Smaller objects mean faster serialization, lower memory usage, and better hit ratios.
10. Invalidate on writes. When data changes, invalidate or update the cache immediately rather than waiting for TTL expiration.

Quick Summary

Caching stores copies of frequently accessed data in a fast storage layer to reduce latency and database load. The main strategies are cache-aside (application manages the cache), read-through and write-through (cache manages itself), write-behind (async writes for speed), and write-around (writes bypass cache). TTL controls how long data stays cached. When the cache is full, eviction policies (LRU, LFU, FIFO) decide what to remove. Redis and

Memcached are the most popular cache systems. Watch out for cache stampede (many requests hitting DB on expiry), cache penetration (requests for non-existent data), and cache avalanche (mass expiration). Cache invalidation remains one of the hardest problems in computing.

Key Points

- A cache hit serves data from fast storage; a cache miss requires fetching from the slow original source
- Cache-aside is the most common strategy: check cache first, fall back to database on miss
- Write-through ensures consistency but adds write latency; write-behind is fast but risks data loss
- TTL controls freshness -- short TTLs mean fresher data but more DB load
- LRU (evict least recently used) is the best default eviction policy
- Redis is more feature-rich; Memcached is simpler and more memory-efficient for pure caching
- Cache stampede, penetration, and avalanche are the three main cache failure modes
- Always design your system to work without the cache -- just slower

---

Practice Questions

1. You have an e-commerce site where product prices change once a day but product pages are viewed millions of times. Which caching strategy would you use, and what TTL would you set? Why?
2. Your cache hit ratio drops from 95% to 60% after a new feature deployment. What are three possible causes, and how would you investigate each?
3. You are designing a social media feed. Users see posts from people they follow. Should you cache the feed per user, or cache individual posts and assemble feeds on the fly? What are the trade-offs of each approach?
4. Explain why write-behind caching is risky for financial transactions but acceptable for view counters.
5. Your Redis cache is running at 95% memory capacity. What steps would you take to address this before it starts evicting important data?

Exercises

Exercise 1: Design a Cache Layer You are building a news website. Articles are written by editors and published once. After publication, an article rarely changes but is read millions of times. Design the caching strategy. Specify: which caching strategy, TTL, eviction policy, and how you handle article updates.

Exercise 2: Cache Stampede Simulation Draw a timeline showing what happens when a cache entry with 1000 requests per second expires. Then draw a second timeline showing the same scenario with a locking mechanism. Label the database load at each point.

Exercise 3: Redis vs Memcached Decision You are building three different systems: (a) a session store for a web app, (b) a simple page cache for a static website, (c) a real-time leaderboard for a game. For each, recommend Redis or Memcached and explain your reasoning.

What Is Next?

In this chapter, you learned how caching speeds up your system by keeping frequently accessed data close to where it is needed. One of the most powerful forms of caching is the CDN -- a globally distributed cache that brings content physically closer to your users. In Chapter 5, you will learn how CDNs work, when to use them, and how they fit into your overall system architecture.

Chapter 5: Content Delivery Networks (CDNs)

What You Will Learn

- What a CDN is and why it dramatically improves performance for global users
- How CDNs use DNS to route users to the nearest edge server
- The difference between push CDN and pull CDN architectures
- Static content vs dynamic content caching
- Cache-control headers and how they govern CDN behavior
- Cache invalidation and purging strategies
- Major CDN providers: CloudFront, Cloudflare, and Akamai
- When to use a CDN and when not to
- How CDN request flow works from end to end

Why This Chapter Matters

In Chapter 4, you learned how caching reduces latency by keeping data in a faster storage layer. But there is a type of latency that no amount of server-side caching can fix: the speed of light.

If your server is in New York and your user is in Tokyo, every request must travel roughly 11,000 miles each way. Even at the speed of light, that is about 70 milliseconds of round-trip time -- and real-world networks add much more due to routing hops, congestion, and protocol overhead.

A CDN solves this by placing copies of your content on servers around the world. Instead of every user fetching data from your single origin server, they fetch it from a nearby CDN server (called an edge server). This is one of the simplest and highest-impact improvements you can make to any web application.

5.1 What Is a CDN?

A CDN (Content Delivery Network) is a geographically distributed network of servers that delivers content to users from the location nearest to them.

The Warehouse Analogy

Imagine you run an online store that ships physical products. Your only warehouse is in Chicago.

- A customer in Chicago gets their order in 1 day.
- A customer in Los Angeles waits 4 days.
- A customer in London waits 10 days.
- A customer in Tokyo waits 14 days.

Now imagine you open small warehouses (fulfillment centers) in Los Angeles, London, and Tokyo. You stock each warehouse with your most popular products.

- Customer in Chicago: 1 day (from Chicago warehouse)
- Customer in Los Angeles: 1 day (from LA warehouse)
- Customer in London: 1 day (from London warehouse)
- Customer in Tokyo: 1 day (from Tokyo warehouse)

A CDN does exactly this, but for digital content. Your origin server is the main warehouse. CDN edge servers are the regional fulfillment centers.

```
WITHOUT CDN:

  User (Tokyo)  --------11,000 miles-------->  Server (New York)
  User (London) ---------3,500 miles-------->  Server (New York)
  User (Sydney) --------10,000 miles-------->  Server (New York)
  User (NYC)    ----------5 miles----------->  Server (New York)

  Latency depends on distance from origin.


WITH CDN:

  User (Tokyo)  ----> Edge (Tokyo)      ~5ms
  User (London) ----> Edge (London)     ~5ms
  User (Sydney) ----> Edge (Sydney)     ~5ms
  User (NYC)    ----> Edge (New York)   ~2ms
```

```
                    +---- Edge: Tokyo
                    |
Origin Server ----+---- Edge: London
(New York)          |
                    +---- Edge: Sydney
                    |
                    +---- Edge: Sao Paulo
                    |
                    +---- Edge: Mumbai

Each edge caches content from the origin.
Users connect to the nearest edge.
```

What a CDN Stores

CDNs primarily cache static content -- files that do not change between requests:

- Images (JPEG, PNG, WebP, SVG)
- CSS stylesheets
- JavaScript files
- Fonts
- Video and audio files
- PDF documents
- HTML pages (when they rarely change)

Modern CDNs can also cache dynamic content -- API responses, personalized pages -- with shorter TTLs and smarter invalidation.

5.2 How CDN Routing Works

When a user requests content, how does their browser know to connect to a nearby CDN edge server instead of your origin? The answer is DNS.

DNS-Based Routing

```
CDN REQUEST FLOW (Step by Step):

1. User types "www.example.com/logo.png" in browser

2. Browser asks DNS: "What is the IP of www.example.com?"
```

```
3. DNS sees that www.example.com has a CNAME pointing to
   "d1234.cdn-provider.net"

4. CDN's DNS server receives the query. It knows the user's
   approximate location from their DNS resolver's IP address.

5. CDN DNS returns the IP of the nearest edge server.

6. Browser connects to the edge server.

7. Edge server checks: "Do I have /logo.png cached?"
   - YES (cache hit): Return it immediately
   - NO (cache miss): Fetch from origin, cache it, return it


Step-by-step:

  +--------+     +-------+     +---------+     +------+     +--------+
  | User's |     |  DNS  |     | CDN DNS |     | Edge |     | Origin |
  |Browser |     |Resolver|    | Server  |     |Server|     | Server |
  +---+----+     +---+---+     +----+----+     +--+---+     +---+----+
      |              |              |             |              |
      | 1. Resolve   |              |             |              |
      | example.com  |              |             |              |
      |------------->|              |             |              |
      |              | 2. CNAME to  |             |              |
      |              | cdn-provider |             |              |
      |              |------------->|             |              |
      |              |              |             |              |
      |              | 3. Return IP |             |              |
      |              |  of nearest  |             |              |
      |              |  edge server |             |              |
      |              |<-------------|             |              |
      |              |              |             |              |
      | 4. IP address|              |             |              |
      |<-------------|              |             |              |
      |              |              |             |              |
      | 5. Request /logo.png        |             |              |
      |-------------------------------->          |              |
      |              |              |            |              |
      |              |              |    6a. HIT: |              |
      |              |              |    Return   |              |
      |              |              |    cached   |              |
      |<---------------------------------|       |             |
      |              |              |             |              |
      |              |              |   6b. MISS: |              |
      |              |              |   Fetch from|              |
      |              |              |   origin    |              |
      |              |              |             |------------->|
      |              |              |             |<-------------|
      |              |              |   Cache it  |              |
      |              |              |   + return  |              |
      |<---------------------------------|       |             |
```

Routing Methods

CDNs use several techniques to direct users to the optimal edge server:

1. GeoDNS: Routes based on the geographic location of the user's DNS resolver. Simple but not always accurate (VPN users, corporate DNS resolvers in different regions).

2. Anycast: Multiple edge servers share the same IP address. The internet's routing protocols (BGP) naturally direct packets to the nearest server. This is very effective and widely used by Cloudflare.

3. Latency-based routing: CDN measures actual latency from various locations and routes to the lowest-latency server, not just the geographically closest one.

```
ANYCAST ROUTING:

  All edge servers announce the same IP: 1.2.3.4

  User in Tokyo:
    Internet routing --> Shortest BGP path --> Edge in Tokyo

  User in London:
    Internet routing --> Shortest BGP path --> Edge in London

  Same IP address, different physical servers!
```

5.3 Push CDN vs Pull CDN

There are two fundamentally different approaches to getting content onto CDN edge servers.

Pull CDN (Origin Pull)

The CDN pulls content from your origin server on demand, the first time it is requested.

```
PULL CDN:

  1. User requests /image.jpg from edge server
  2. Edge checks: "Do I have it?" --> NO (first request)
  3. Edge fetches /image.jpg from origin server
  4. Edge caches it and returns it to user
  5. Next user requests /image.jpg from same edge
  6. Edge checks: "Do I have it?" --> YES
  7. Edge returns cached copy (origin not contacted)
```

```
Timeline:
+---------+-----------------------------------------+
| Request | What Happens                            |
+---------+-----------------------------------------+
| 1st     | Edge MISS --> Fetch from origin --> Cache |
| 2nd     | Edge HIT  --> Return cached copy         |
| 3rd     | Edge HIT  --> Return cached copy         |
| ...     | Edge HIT  --> Return cached copy         |
| TTL exp | Edge MISS --> Fetch from origin --> Cache |
+---------+-----------------------------------------+
```

Pros:

- Simple setup (just point DNS to CDN)
- Only requested content is cached (no wasted storage)
- No need to upload content to CDN manually

Cons:

- First request to each edge is slow (origin pull)
- Origin must handle pull requests from many edges
- Sudden traffic spikes can overwhelm origin

Best for: Most websites, APIs, and applications. This is the default for most CDN providers.

Push CDN

You explicitly upload content to the CDN. The CDN stores it and serves it directly. The origin is only involved during uploads.

```
PUSH CDN:

  1. You upload /video.mp4 to CDN (ahead of time)
  2. CDN distributes it to edge servers
  3. User requests /video.mp4
  4. Edge already has it --> Returns immediately

  +--------+    Upload     +----------+   Replicate   +-------+
  | Origin |-------------->| CDN Core |-------------->| Edges |
  +--------+   (you push)  +----------+  (CDN handles)+-------+
                                                          |
                                                     User requests
                                                     served here
```

Pros:

- No cold-miss penalty (content is already on edges)

- Origin is not contacted for serving (fully offloaded)
- You control exactly what is cached

Cons:

- You must manage uploads and deletions
- You pay for storage on all edge servers, even for rarely accessed content
- More operational complexity

Best for: Large static files (videos, software downloads), content with predictable access patterns.

Comparison

```
+----------------+------------------+------------------+
| Feature        | Pull CDN         | Push CDN         |
+----------------+------------------+------------------+
| Setup effort   | Low              | Higher           |
| First request  | Slow (origin     | Fast (already    |
|                | fetch)           | cached)          |
+----------------+------------------+------------------+
| Storage cost   | Lower (only      | Higher (all      |
|                | popular content) | uploaded content)|
+----------------+------------------+------------------+
| Origin load    | Moderate (pulls) | Minimal          |
+----------------+------------------+------------------+
| Best for       | Websites, APIs   | Video, large     |
|                |                  | file downloads   |
+----------------+------------------+------------------+
| Most providers | Default mode     | Optional feature |
+----------------+------------------+------------------+
```

5.4 Static vs Dynamic Content

Static Content

Static content is the same for every user: images, CSS, JavaScript, fonts, videos. This is the bread and butter of CDN caching.

```
Static content: Same file for everyone

  User A requests /style.css --> Gets the same file
  User B requests /style.css --> Gets the same file
```

```
  User C requests /style.css --> Gets the same file

  Cache-Control: public, max-age=31536000
  (Cache for 1 year -- use versioned URLs to bust cache)
```

Dynamic Content

Dynamic content varies by user, time, or context: personalized dashboards, search results, API responses.

```
Dynamic content: Different for each user

  User A requests /api/feed --> Gets User A's feed
  User B requests /api/feed --> Gets User B's feed
  User C requests /api/feed --> Gets User C's feed

  Can you cache this? Maybe, with the right strategy:
  - Cache by user (Vary: Cookie or Vary: Authorization)
  - Cache with short TTL (Cache-Control: max-age=10)
  - Cache at edge with ESI (Edge Side Includes)
```

Edge Side Includes (ESI)

Some CDNs support ESI, which lets you cache parts of a page separately:

```
PAGE WITH ESI:

  +-----------------------------------------------------+
  | Header (cached 1 day -- same for everyone)          |
  +-----------------------------------------------------+
  | <esi:include src="/api/user-greeting"/>             |
  | (cached 5 min -- per user)                          |
  +-----------------------------------------------------+
  | Product listing (cached 1 hour -- same for all)     |
  +-----------------------------------------------------+
  | Footer (cached 1 day -- same for everyone)          |
  +-----------------------------------------------------+

  The edge assembles the page from cached fragments,
  each with its own TTL and cache key.
```

5.5 Cache-Control Headers

Cache-Control headers are HTTP headers that tell the CDN (and browser) how to cache content. These are your primary tool for controlling CDN behavior.

Key Cache-Control Directives

```
+----------------------+---------------------------------------------+
| Directive            | Meaning                                     |
+----------------------+---------------------------------------------+
| public               | Any cache (CDN, browser) can store this      |
| private              | Only the user's browser can cache (not CDN)  |
| no-cache             | Cache can store it, but must revalidate      |
|                      | with origin before serving                   |
| no-store             | Do not cache at all, anywhere                |
| max-age=N            | Cache for N seconds                          |
| s-maxage=N           | CDN-specific max-age (overrides max-age      |
|                      | for shared caches, browser uses max-age)     |
| must-revalidate      | Once expired, must check with origin          |
| stale-while-         | Serve stale content while fetching fresh      |
|   revalidate=N       | copy in background (for N seconds)            |
| stale-if-error=N     | Serve stale content if origin returns error   |
+----------------------+---------------------------------------------+
```

Common Cache-Control Patterns

```
STATIC ASSETS (images, CSS, JS with versioned URLs):
  Cache-Control: public, max-age=31536000, immutable
  --> Cache for 1 year. URL changes when content changes.

API RESPONSES (semi-dynamic):
  Cache-Control: public, s-maxage=60, stale-while-revalidate=300
  --> CDN caches for 60s, serves stale for 5 min while refreshing.

PRIVATE USER DATA:
  Cache-Control: private, no-cache
  --> Only browser can cache. Must revalidate every time.

SENSITIVE DATA (banking, health records):
  Cache-Control: no-store
  --> Never cache anywhere. Not on CDN, not in browser.

HTML PAGES:
  Cache-Control: public, max-age=0, must-revalidate
  --> Always check with origin. If unchanged (304), use cache.
```

ETag and Conditional Requests

ETags let the CDN check if content has changed without downloading it again:

```
CONDITIONAL REQUEST FLOW:

  1. First request:
     Edge --> Origin: GET /page.html
     Origin --> Edge: 200 OK
                      ETag: "abc123"
                      Body: <html>...</html>

  2. Subsequent request (after max-age expires):
     Edge --> Origin: GET /page.html
                      If-None-Match: "abc123"

     Origin checks: Has /page.html changed?

     NOT changed:
       Origin --> Edge: 304 Not Modified (no body!)
       Edge uses its cached copy. Saves bandwidth.

     Changed:
       Origin --> Edge: 200 OK
                        ETag: "def456"
                        Body: <html>new content</html>
       Edge updates its cache.
```

5.6 Cache Invalidation and Purging

Sometimes you need to remove content from the CDN before its TTL expires. This is called invalidation or purging.

Why Invalidate?

- You published an article with a typo and need to fix it immediately
- A product price changed and stale prices would cause problems
- A security vulnerability requires an immediate asset update
- A legal takedown request requires content removal

Invalidation Methods

1. Purge by URL: Remove a specific URL from all edge servers.

```
PURGE BY URL:
```

```
  API Call: PURGE /images/product-42.jpg

  +--------+       +--------+       +--------+
  | Edge 1 |       | Edge 2 |       | Edge 3 |
  | Tokyo  |       | London |       | NYC    |
  +--------+       +--------+       +--------+
       |               |               |
       v               v               v
  DELETE            DELETE            DELETE
  product-42.jpg    product-42.jpg    product-42.jpg

  Next request triggers a fresh pull from origin.
```

2. Purge by tag/key: Group related content with a tag and purge all content with that tag at once.

```
PURGE BY TAG:

  Tag: "product-42"
  Applies to: /images/product-42.jpg
              /api/product/42
              /pages/product-42.html

  API Call: PURGE tag="product-42"
  --> All three URLs purged from all edges
```

3. Purge everything: Nuclear option. Removes all cached content from all edges.

4. URL versioning: Instead of purging, change the URL when content changes. The old URL naturally expires; the new URL is a cache miss.

```
URL VERSIONING:

  Before change: /static/style.v1.css  (cached)
  After change:  /static/style.v2.css  (new URL, cache miss)

  The old URL is never purged -- it just expires naturally.
  No purge needed! This is the most reliable strategy.
```

Invalidation Propagation Time

Purging is not instant. It takes time to propagate across all edge servers:

```
+------------------+---------------------------+
| CDN Provider     | Typical Purge Time        |
+------------------+---------------------------+
```

```
| Cloudflare       | ~2-5 seconds (very fast)  |
| CloudFront       | ~5-60 seconds             |
| Akamai           | ~5-10 seconds             |
+------------------+---------------------------+
```

During propagation, some users may see old content while others see new content.

5.7 Major CDN Providers

Amazon CloudFront

- Integrated with AWS ecosystem (S3, EC2, Lambda@Edge)
- 450+ edge locations worldwide
- Supports Lambda@Edge for running code at the edge
- Pay-per-use pricing (per GB transferred + per request)
- Good for: AWS-heavy architectures, video streaming

Cloudflare

- Global anycast network with 300+ cities
- Free tier available (generous for small sites)
- Built-in DDoS protection, WAF, and DNS
- Workers (serverless at the edge) for dynamic content
- Good for: Websites of all sizes, security-focused deployments

Akamai

- The original CDN, largest network (4,000+ edge locations)
- Enterprise-focused with advanced features
- Strong in media delivery and security
- Higher cost, longer contracts
- Good for: Large enterprises, media companies, gaming

Comparison

```
+------------------+-----------+------------+----------+
| Feature          | CloudFront| Cloudflare | Akamai   |
+------------------+-----------+------------+----------+
| Edge locations   | 450+      | 300+ cities| 4,000+   |
| Free tier        | No*       | Yes        | No       |
| DDoS protection  | Shield    | Built-in   | Kona     |
| Edge compute     | Lambda    | Workers    | EdgeWorkers|
|                  | @Edge     |            |          |
| Pricing model    | Pay-per-  | Plans +    | Contract |
|                  | use       | pay-per-use| based    |
| Best for         | AWS users | Everyone   | Enterprise|
+------------------+-----------+------------+----------+

* AWS Free Tier includes limited CloudFront usage for 12 months
```

5.8 When to Use a CDN

Use a CDN When:

- Your users are geographically distributed
- You serve static assets (images, CSS, JS, videos)
- You want to reduce load on your origin server
- You need DDoS protection
- You want to improve page load times globally
- You serve large files (software downloads, media)

Skip the CDN When:

- All your users are in one region close to your server
- Your content is highly personalized and cannot be cached
- You have strict data residency requirements (content must stay in one country)
- You are in early development and latency optimization is premature
- Your traffic is very low and the added complexity is not justified

CDN Impact on Performance

```
BEFORE CDN (origin in US-East):

  User Location    | Latency (first byte) | Page Load
```

```
  -----------------+---------------------+-----------
  New York         | 20ms                | 1.2s
  Los Angeles      | 70ms                | 2.1s
  London           | 90ms                | 2.8s
  Tokyo            | 180ms               | 4.5s
  Sydney           | 220ms               | 5.2s


AFTER CDN:

  User Location    | Latency (first byte) | Page Load
  -----------------+---------------------+-----------
  New York         | 5ms                 | 0.8s
  Los Angeles      | 8ms                 | 0.9s
  London           | 10ms                | 0.9s
  Tokyo            | 12ms                | 1.0s
  Sydney           | 15ms                | 1.1s

  Everyone gets a fast, consistent experience.
```

5.9 CDN Architecture: Full Request Flow

Here is the complete flow of a request through a CDN-enabled system:

```
COMPLETE CDN REQUEST FLOW:

  +--------+
  | User   |  1. Types www.example.com/photo.jpg
  +---+----+
      |
      | 2. DNS lookup: www.example.com
      v
  +---+------+
  | DNS      |  3. Returns CNAME: cdn.example.com
  | Resolver |  4. Resolves cdn.example.com to nearest edge IP
  +---+------+
      |
      | 5. Connect to edge server
      v
  +---+------+
  | CDN Edge |  6. Check local cache for /photo.jpg
  | Server   |
  +---+------+
      |
      +-------- HIT? ---------> 7a. Return cached photo
      |                              (fast: ~10ms)
      |
      +-------- MISS? --------> 7b. Forward request to origin
```

```
      |
      v
  +---+------+
  | Origin   |  8. Origin returns photo.jpg
  | Server   |
  +---+------+
      |
      v
  +---+------+
  | CDN Edge |  9. Cache photo.jpg locally (with TTL)
  | Server   | 10. Return photo.jpg to user
  +----------+     (slower: ~100ms, but next request is fast)


COMPLETE ARCHITECTURE:

  +--------+     +--------+     +--------+     +--------+
  |  User  |     |  CDN   |     |  Load  |     | Origin |
  |        |     |  Edge  |     |Balancer|     | Server |
  +---+----+     +---+----+     +---+----+     +---+----+
      |              |              |              |
      |  Request     |              |              |
      |------------->|              |              |
      |              |              |              |
      |         HIT? |              |              |
      |<-------------|              |              |
      |   (done!)    |              |              |
      |              |              |              |
      |        MISS? |              |              |
      |              |   Forward    |              |
      |              |------------->|              |
      |              |              |   Route to   |
      |              |              |   server     |
      |              |              |------------->|
      |              |              |              |
      |              |              |   Response   |
      |              |              |<-------------|
      |              |   Response   |              |
      |              |<-------------|              |
      |              |              |              |
      |              | Cache + send |              |
      |<-------------|              |              |
```

5.10 Advanced CDN Topics

CDN and HTTPS

CDNs terminate TLS connections at the edge, which has two benefits:

1. Users get a faster TLS handshake (shorter round trip to nearby edge)
2. The connection between edge and origin can use a persistent, optimized connection

```
TLS WITH CDN:

  User <--TLS--> Edge (fast handshake, nearby)
                 Edge <--TLS--> Origin (persistent connection)

  Without CDN:
  User <--TLS--> Origin (slow handshake, far away)
```

CDN for API Acceleration

Modern CDNs can accelerate API traffic even when responses are not cacheable:

- Connection pooling: Edge keeps persistent connections to your origin
- TCP optimization: Edge uses optimized TCP settings for the long-haul connection
- Route optimization: CDN's private backbone may be faster than the public internet
- Compression: Edge can compress responses (gzip, Brotli) closer to the user

Edge Computing

Modern CDNs let you run code at the edge:

```
EDGE COMPUTING:

  Traditional:
  User --> Edge (cache only) --> Origin (all logic)

  With Edge Computing:
  User --> Edge (cache + logic) --> Origin (only when needed)

  Examples:
  - A/B testing decisions at the edge
  - URL rewriting and redirects
  - Authentication checks
  - Geolocation-based content
  - Image resizing on the fly
```

Common Mistakes

1. Not setting Cache-Control headers. Without explicit headers, CDN behavior is unpredictable. Always set headers on your origin responses.
2. Using the same TTL for everything. Static assets can be cached for months; API responses may need seconds. Differentiate.
3. Forgetting about cache busting. If you cache CSS for 1 year but update it, users see the old version. Use versioned URLs (style.v2.css) or content hashes (style.a1b2c3.css).
4. Caching private data on a shared CDN. If you do not set Cache-Control: private for user-specific data, the CDN may serve one user's data to another.
5. Ignoring the Vary header. If your response varies by Accept-Encoding or Cookie, you must include the Vary header or the CDN may serve wrong content.
6. Over-purging. Frequent purges negate the benefit of caching. Use versioned URLs instead.
7. Not monitoring CDN hit ratio. If your CDN has a low hit ratio, you are paying for a CDN without getting much benefit. Investigate and fix.

Best Practices

1. Use versioned or hashed URLs for static assets. This eliminates the need for purging and ensures users always get the correct version.
2. Set long TTLs for truly static content. Images, fonts, and versioned JS/CSS can safely be cached for 1 year.
3. Use s-maxage to control CDN caching separately from browser caching. This lets you set a short browser cache but a long CDN cache.
4. Enable stale-while-revalidate. This lets the CDN serve slightly stale content while fetching fresh content in the background, preventing latency spikes.
5. Use a CDN for all static assets, even in development. It catches caching bugs early.
6. Monitor CDN hit ratio, bandwidth savings, and origin offload. These metrics tell you if your CDN is working well.
7. Consider edge computing for simple logic. Authentication, redirects, and A/B testing can run at the edge, reducing origin load.
8. Use HTTPS between CDN and origin. Do not let the internal connection be unencrypted.

Quick Summary

A CDN is a geographically distributed network of edge servers that caches content close to users, reducing latency from hundreds of milliseconds to single digits. CDNs use DNS-based routing (GeoDNS or Anycast) to direct users to the nearest edge. Pull CDNs fetch content from your origin on demand (most common); push CDNs require you to upload content in advance. Cache-Control headers govern what gets cached and for how long. Invalidation can be done via purging or URL versioning (preferred). Major providers include CloudFront (AWS integrated), Cloudflare (free tier, anycast), and Akamai (largest network, enterprise). Use a CDN whenever you have geographically distributed users and static content to serve.

Key Points

- A CDN places copies of your content on servers worldwide so users connect to a nearby edge instead of a distant origin
- DNS routing (GeoDNS or Anycast) directs users to the optimal edge server
- Pull CDNs fetch content on demand (simple, most common); push CDNs require manual uploads (good for large files)
- Cache-Control headers are the primary mechanism for controlling CDN behavior
- Use versioned URLs for static assets to avoid purge-related issues
- s-maxage controls CDN caching independently from browser caching
- stale-while-revalidate prevents latency spikes during cache refreshes
- CDN hit ratio is the key metric to monitor

Practice Questions

1. Your website serves users globally, but you notice that users in Asia consistently experience 3-4 second load times while US users see 0.5 seconds. Explain how a CDN would help and what steps you would take to implement it.
2. You have an e-commerce site where product images change occasionally but product prices change multiple times per day. How would you configure CDN caching differently for images vs API responses? Include specific Cache-Control headers.
3. A developer on your team sets Cache-Control: public, max-age=86400 on an API endpoint that returns user-specific data. What is the security risk, and how would you fix it?

4. Compare Anycast routing and GeoDNS routing for a CDN. What are the trade-offs? When might one be preferred over the other?
5. Your CDN hit ratio is only 40%. List three possible causes and how you would investigate and improve it.

Exercises

Exercise 1: CDN Configuration You are building a news website with these content types: (a) article text that updates after publication, (b) article images that never change, (c) the homepage that changes every few minutes, (d) user comment sections. For each, decide: should it be cached on the CDN? If yes, write the Cache-Control header you would use. If no, explain why.

Exercise 2: Draw the Request Flow Draw a detailed ASCII diagram showing what happens when a user in Sydney requests a page from your origin server in London. Include DNS resolution, CDN edge lookup (miss then hit), origin fetch, and caching. Show the difference in latency for the first request vs subsequent requests.

Exercise 3: CDN Cost Analysis Your site serves 10 million image requests per day. Average image size is 200KB. Without a CDN, all traffic hits your origin server. With a CDN at 90% hit ratio, only 10% of requests reach origin. Calculate: (a) daily bandwidth saved on origin, (b) if your cloud provider charges $0.09/GB for origin bandwidth, how much do you save per month?

What Is Next?

Now that you understand how CDNs cache and deliver content at the edge, it is time to dive into the backbone of any system: the database. In Chapter 6, you will explore SQL and NoSQL databases -- their strengths, weaknesses, and when to use each. Understanding databases is essential because nearly every system design decision revolves around how data is stored, queried, and scaled.

Chapter 6: Databases -- SQL and NoSQL

What You Will Learn

- A solid recap of SQL databases: tables, joins, ACID transactions, and indexes
- The four types of NoSQL databases: key-value, document, wide-column, and graph
- ACID vs BASE: two fundamentally different consistency models
- Schema vs schemaless: the trade-offs of rigid structure vs flexibility
- A decision flowchart for choosing SQL vs NoSQL
- Polyglot persistence: using multiple database types in one system
- When each database type shines and when it struggles

Why This Chapter Matters

Every system you will ever design stores data. The database is the heart of your architecture. Choose the wrong one, and you will fight it at every turn -- poor performance, painful migrations, and limitations that force ugly workarounds.

For decades, SQL databases (PostgreSQL, MySQL, Oracle) were the only serious option. Then in the mid-2000s, companies like Google, Amazon, and Facebook hit scale problems that traditional SQL databases could not solve. They built new database systems that traded some of SQL's guarantees for massive scalability. These became known as NoSQL databases.

Today, the best architects understand both worlds and know when to use each -- or even when to use both together. This chapter gives you that understanding.

6.1 SQL Databases: A Recap

SQL (Structured Query Language) databases store data in tables with rows and columns. They enforce a strict schema, support complex queries, and guarantee data consistency through ACID transactions.

The Spreadsheet Analogy

Think of a SQL database as a collection of interconnected spreadsheets. Each spreadsheet (table) has defined columns (schema), and you can look up data across spreadsheets using common values (joins).

```
USERS TABLE:
+----+----------+-------------------+
| id | name     | email             |
+----+----------+-------------------+
| 1  | Alice    | alice@example.com |
| 2  | Bob      | bob@example.com   |
| 3  | Charlie  | charlie@ex.com    |
+----+----------+-------------------+

ORDERS TABLE:
+----+---------+------------+--------+
| id | user_id | product    | amount |
+----+---------+------------+--------+
| 1  | 1       | Laptop     | 999.99 |
| 2  | 1       | Mouse      | 29.99  |
| 3  | 2       | Keyboard   | 79.99  |
+----+---------+------------+--------+

JOIN: "Show me all orders with user names"

SELECT users.name, orders.product, orders.amount
FROM orders
JOIN users ON orders.user_id = users.id;

+----------+----------+--------+
| name     | product  | amount |
+----------+----------+--------+
| Alice    | Laptop   | 999.99 |
| Alice    | Mouse    | 29.99  |
| Bob      | Keyboard | 79.99  |
+----------+----------+--------+
```

ACID Properties

ACID is the set of guarantees that SQL databases provide for transactions:

```
A - Atomicity
    All operations in a transaction succeed, or none do.
    "Transfer $100 from Account A to Account B"
    --> Debit A AND credit B, or neither happens.

C - Consistency
    Every transaction moves the database from one valid
    state to another. Constraints are never violated.
    --> Balance cannot go below zero (if that is a rule).

I - Isolation
    Concurrent transactions do not interfere with each other.
    Each transaction sees a consistent snapshot of the data.
    --> Two people buying the last item: only one succeeds.

D - Durability
    Once a transaction is committed, it survives crashes,
    power failures, and restarts. Data is written to disk.
    --> Your bank transfer does not vanish if the server reboots.
```

Why ACID Matters

ACID is critical for systems where correctness cannot be compromised:

- Banking: A transfer must debit one account AND credit another. Never just one.
- Inventory: If two users buy the last item simultaneously, only one should succeed.
- Bookings: A hotel room should not be double-booked.

SQL Strengths

- Complex queries: JOINs, aggregations, subqueries, window functions
- Data integrity: Foreign keys, constraints, triggers ensure valid data
- ACID transactions: Strong consistency guarantees
- Mature tooling: Decades of optimization, monitoring, and backup tools
- Standardized language: SQL is universal across vendors

SQL Weaknesses

- Vertical scaling: Traditional SQL databases scale by adding more power to a single server (bigger CPU, more RAM). There is a ceiling.
- Rigid schema: Changing the schema (adding/removing columns) on a large table can be slow and disruptive.
- Impedance mismatch: Object-oriented code does not map naturally to flat tables. ORMs help but add complexity.

- Horizontal scaling is hard: Distributing a SQL database across multiple machines (sharding) is complex and breaks some guarantees (see Chapter 8).

Popular SQL Databases

```
+---------------+-------------------------------------------+
| Database      | Notable Features                          |
+---------------+-------------------------------------------+
| PostgreSQL    | Most feature-rich open-source SQL DB.     |
|               | JSON support, extensions, very reliable.  |
+---------------+-------------------------------------------+
| MySQL         | Most popular open-source SQL DB. Fast     |
|               | reads, widely deployed. Used by Facebook. |
+---------------+-------------------------------------------+
| SQLite        | Embedded database (no server). Perfect    |
|               | for mobile apps and small tools.          |
+---------------+-------------------------------------------+
| SQL Server    | Microsoft's enterprise DB. Deep Windows   |
|               | integration, strong BI tools.             |
+---------------+-------------------------------------------+
| Oracle        | Enterprise heavyweight. Expensive but     |
|               | extremely capable for large organizations.|
+---------------+-------------------------------------------+
```

6.2 NoSQL Databases

NoSQL ("Not Only SQL") databases sacrifice some of SQL's guarantees to gain flexibility, scalability, or performance. There are four main types, each designed for different data access patterns.

Type 1: Key-Value Stores

The simplest NoSQL database. Data is stored as key-value pairs, like a giant hash map.

```
KEY-VALUE STORE:

  +------------------+----------------------------------+
  | Key              | Value                            |
  +------------------+----------------------------------+
  | user:1           | {"name":"Alice","email":"a@b.c"} |
  | session:abc123   | {"logged_in":true,"cart":[1,2]}  |
  | config:timeout   | "30"                             |
  +------------------+----------------------------------+
```

```
  Operations:
    GET user:1          --> Returns Alice's data
    SET user:1 {...}    --> Stores/updates Alice's data
    DELETE user:1       --> Removes Alice's data

  That is it. No queries by value, no joins, no indexes.
  You look up by key or not at all.
```

Examples: Redis, Amazon DynamoDB (also document), Memcached, Riak

Best for: Caching, session storage, user preferences, shopping carts, leaderboards

Not for: Complex queries, relationships between data, reporting

Type 2: Document Stores

Store data as documents (usually JSON or BSON). Each document can have a different structure. You can query by any field, not just the key.

```
DOCUMENT STORE:

  Collection: "users"

  Document 1:
  {
    "_id": "user_1",
    "name": "Alice",
    "email": "alice@example.com",
    "addresses": [
      {"type": "home", "city": "New York"},
      {"type": "work", "city": "Boston"}
    ]
  }

  Document 2:
  {
    "_id": "user_2",
    "name": "Bob",
    "email": "bob@example.com",
    "phone": "+1-555-0100"       <-- Different fields! OK!
  }

  Queries:
    Find where name = "Alice"           --> Document 1
    Find where addresses.city = "Boston" --> Document 1
    Find where phone exists             --> Document 2
```

Examples: MongoDB, Amazon DynamoDB, Couchbase, CouchDB

Best for: Content management, catalogs, user profiles, event logging, any data that is naturally hierarchical

Not for: Highly relational data with many cross-references, complex transactions across multiple documents

Type 3: Wide-Column Stores

Store data in tables with rows and columns, but unlike SQL, each row can have a different set of columns. Designed for massive-scale, write-heavy workloads.

```
WIDE-COLUMN STORE:

  Row Key: "user_1"
  +-----------------+--------------------+-----------------+
  | Column Family: personal               | Column Family:  |
  |                                       | activity        |
  +----------+---------+-----------------++---------+-------+
  | name     | email   | phone           | last_login| posts |
  | "Alice"  | "a@b.c" | "+1-555-0100"   | "2024-01" | "42"  |
  +----------+---------+-----------------+-----------+-------+

  Row Key: "user_2"
  +-----------------+--------------------+-----------------+
  | Column Family: personal               | Column Family:  |
  |                                       | activity        |
  +----------+---------+-----------------++----------+------+
  | name     | email   |                 | last_login |     |
  | "Bob"    | "b@c.d" |  (no phone!)    | "2024-02"  |     |
  +----------+---------+-----------------+----------+------+

  Each row can have different columns.
  Columns are grouped into "column families."
  Optimized for reads/writes by row key.
```

Examples: Apache Cassandra, HBase, Google Bigtable

Best for: Time-series data, IoT sensor data, event logging, recommendation engines, any workload with massive write volume

Not for: Small datasets, ad-hoc queries, systems needing strong consistency

Type 4: Graph Databases

Store data as nodes (entities) and edges (relationships). Optimized for traversing relationships.

```
GRAPH DATABASE:

  Nodes and Edges:

    (Alice)---[FRIENDS_WITH]--->(Bob)
       |                          |
  [WORKS_AT]                [WORKS_AT]
       |                          |
       v                          v
    (Google)                   (Meta)
       |
  [LOCATED_IN]
       |
       v
  (Mountain View)

  Queries:
  "Find all of Alice's friends" --> Traverse FRIENDS_WITH edges
  "Find friends of friends"     --> Two-hop traversal
  "Shortest path Alice to Meta" --> Graph traversal algorithm

  These queries are FAST in a graph DB (follow pointers)
  but SLOW in SQL (multiple JOINs on large tables).
```

Examples: Neo4j, Amazon Neptune, JanusGraph, ArangoDB

Best for: Social networks, recommendation engines, fraud detection, knowledge graphs, network topology, dependency analysis

Not for: Simple CRUD operations, tabular data, high-volume writes

NoSQL Type Comparison

```
+----------------+------------+----------+----------+----------+
| Feature        | Key-Value  | Document | Wide-    | Graph    |
|                |            |          | Column   |          |
+----------------+------------+----------+----------+----------+
| Data model     | Key:Value  | JSON/BSON| Row +    | Nodes +  |
|                | pairs      | documents| columns  | edges    |
+----------------+------------+----------+----------+----------+
| Query by       | Key only   | Any field| Row key  | Traversal|
|                |            |          | + column |          |
+----------------+------------+----------+----------+----------+
| Schema         | None       | Flexible | Flexible | Flexible |
+----------------+------------+----------+----------+----------+
| Joins          | No         | Limited  | No       | Native   |
|                |            |          |          |(traversal|
|                |            |          |          | = join)  |
+----------------+------------+----------+----------+----------+
| Best for       | Cache,     | CMS,     | Time     | Social,  |
|                | sessions   | catalogs | series,  | fraud,   |
```

```
|                 |            |          | IoT      | recs     |
+-----------------+------------+----------+----------+----------+
| Scale           | Excellent  | Very good| Excellent| Good     |
+-----------------+------------+----------+----------+----------+
| Complexity      | Lowest     | Low      | Medium   | Higher   |
+-----------------+------------+----------+----------+----------+
```

6.3 ACID vs BASE

SQL databases follow ACID. Many NoSQL databases follow a different model called BASE.

BASE Properties

```
BA - Basically Available
     The system guarantees availability: it will respond
     to every request, even if the data is not the
     latest version.

S  - Soft state
     The state of the system may change over time, even
     without new input, as data propagates between nodes.

E  - Eventually consistent
     If you stop writing, all replicas will eventually
     converge to the same value. But at any given moment,
     different replicas might return different values.
```

ACID vs BASE Comparison

```
+------------------+-------------------------+-------------------------+
| Property         | ACID                    | BASE                    |
+------------------+-------------------------+-------------------------+
| Consistency      | Strong: every read      | Eventual: reads may     |
|                  | returns the latest      | return stale data       |
|                  | committed write         | temporarily             |
+------------------+-------------------------+-------------------------+
| Availability     | May sacrifice           | Prioritizes             |
|                  | availability for        | availability over       |
|                  | consistency             | consistency             |
+------------------+-------------------------+-------------------------+
| Transactions     | Full multi-operation    | Usually single-item     |
|                  | transactions            | operations              |
```

```
+------------------+-------------------------+-------------------------+
| Scaling          | Harder to scale         | Designed to scale       |
|                  | horizontally            | horizontally            |
+------------------+-------------------------+-------------------------+
| Use cases        | Banking, inventory,     | Social feeds, analytics,|
|                  | booking systems         | shopping carts, logs    |
+------------------+-------------------------+-------------------------+
```

The Real-World Trade-off

```
ACID (Bank Transfer):

  Account A: $500     Account B: $300
  Transfer $100 from A to B:

  BEGIN TRANSACTION
    UPDATE accounts SET balance = 400 WHERE id = A;
    UPDATE accounts SET balance = 400 WHERE id = B;
  COMMIT

  Result: A=$400, B=$400  (ALWAYS correct, never A=$400 B=$300)


BASE (Social Media Like Count):

  Post has 1000 likes. User clicks "Like."

  Server 1 records: 1001 likes
  Server 2 still shows: 1000 likes  (stale for a few seconds)
  Server 3 still shows: 1000 likes  (stale for a few seconds)

  After a few seconds, all servers show: 1001 likes

  Is this OK? Yes! Nobody notices if a like count is
  off by 1 for a few seconds.
```

6.4 Schema vs Schemaless

Schema (SQL)

The database enforces the structure of your data. Every row in a table has the same columns. You must define the schema before inserting data. Changing the schema requires a migration.

```
SCHEMA-BASED (SQL):

  CREATE TABLE users (
    id    INT PRIMARY KEY,
    name  VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    age   INT
  );

  INSERT INTO users (id, name, email, age)
  VALUES (1, 'Alice', 'alice@ex.com', 30);

  -- This FAILS:
  INSERT INTO users (id, name, email, age, phone)
  VALUES (2, 'Bob', 'bob@ex.com', 25, '555-0100');
  -- Error: column "phone" does not exist!

  -- Must ALTER table first:
  ALTER TABLE users ADD COLUMN phone VARCHAR(20);
  -- On a table with 100M rows, this can take minutes to hours!
```

Schemaless (NoSQL)

Each document/record can have a different structure. No upfront schema definition needed. Great flexibility, but the application must handle inconsistent data.

```
SCHEMALESS (MongoDB):

  // Document 1
  {
    "_id": 1,
    "name": "Alice",
    "email": "alice@ex.com",
    "age": 30
  }

  // Document 2 -- different fields, no problem!
  {
    "_id": 2,
    "name": "Bob",
    "email": "bob@ex.com",
    "phone": "555-0100",
    "preferences": {"theme": "dark", "lang": "en"}
  }

  // No ALTER TABLE needed. Just store what you want.
```

The Trade-off

```
+------------------+---------------------------+---------------------------+
| Aspect           | Schema (SQL)              | Schemaless (NoSQL)        |
+------------------+---------------------------+---------------------------+
| Data quality     | High (DB enforces rules)  | Depends on app code       |
+------------------+---------------------------+---------------------------+
| Flexibility      | Low (must migrate schema) | High (store anything)     |
+------------------+---------------------------+---------------------------+
| Schema changes   | ALTER TABLE (can be slow) | Just write new structure  |
+------------------+---------------------------+---------------------------+
| Documentation    | Schema IS documentation   | Must document separately  |
+------------------+---------------------------+---------------------------+
| Debugging        | Predictable data shape    | "What fields does this    |
|                  |                           |  document have?"          |
+------------------+---------------------------+---------------------------+
| Best for         | Stable, well-understood   | Evolving, varied, or      |
|                  | data models               | semi-structured data      |
+------------------+---------------------------+---------------------------+
```

Important note: "Schemaless" is somewhat misleading. In practice, your application code always expects some structure. The question is whether the database or your application enforces it. Many teams use schema validation libraries even with NoSQL databases.

6.5 SQL vs NoSQL: Decision Flowchart

Use this flowchart when choosing a database for a new project:

```
START
  |
  v
Do you need ACID transactions across
multiple records/tables?
  |
  +-- YES --> Do you need complex JOINs
  |           and aggregations?
  |             |
  |             +-- YES --> SQL DATABASE
  |             |           (PostgreSQL, MySQL)
  |             |
  |             +-- NO --> SQL DATABASE
  |                        (but consider NewSQL like
  |                         CockroachDB, TiDB for scale)
  |
  +-- NO --> What is your primary access pattern?
             |
             +-- Simple key lookups
             |   (get by ID, session store)
```

```
               |   --> KEY-VALUE STORE
               |       (Redis, DynamoDB)
               |
               +-- Flexible documents with
               |   varying structure
               |   --> DOCUMENT STORE
               |       (MongoDB, DynamoDB)
               |
               +-- Massive write volume,
               |   time-series data
               |   --> WIDE-COLUMN STORE
               |       (Cassandra, HBase)
               |
               +-- Complex relationships,
                   graph traversal
                   --> GRAPH DATABASE
                       (Neo4j, Neptune)
```

Quick Decision Table

```
+-----------------------------+-------------------+
| If your data looks like...  | Consider...       |
+-----------------------------+-------------------+
| Tables with relationships   | PostgreSQL, MySQL |
| JSON documents              | MongoDB, DynamoDB |
| Key-value pairs             | Redis, DynamoDB   |
| Time-series / IoT logs      | Cassandra, HBase  |
| Social graph / network      | Neo4j, Neptune    |
| Full-text search            | Elasticsearch     |
| Everything (you are unsure) | PostgreSQL        |
+-----------------------------+-------------------+

When in doubt, start with PostgreSQL. It handles JSON,
full-text search, and relational data well. Specialize later.
```

6.6 SQL vs NoSQL: Detailed Comparison

```
+-------------------+----------------------+----------------------+
| Dimension         | SQL                  | NoSQL                |
+-------------------+----------------------+----------------------+
| Data model        | Tables (rows/columns) | Varies (documents,   |
|                   |                      | key-value, graph, etc.)|
+-------------------+----------------------+----------------------+
| Schema            | Fixed, enforced by DB | Flexible or none     |
```

```
+--------------------+-----------------------+-----------------------+
| Query language     | SQL (standardized)    | Varies by database    |
+--------------------+-----------------------+-----------------------+
| Transactions       | Multi-row ACID        | Usually single-item   |
|                    |                       | (some support multi)  |
+--------------------+-----------------------+-----------------------+
| Joins              | Native, optimized     | Usually not supported |
|                    |                       | (denormalize instead) |
+--------------------+-----------------------+-----------------------+
| Scaling            | Vertical (scale up)   | Horizontal (scale out) |
|                    | Horizontal is hard    | Built-in sharding     |
+--------------------+-----------------------+-----------------------+
| Consistency        | Strong (ACID)         | Eventual (BASE)       |
|                    |                       | or tunable            |
+--------------------+-----------------------+-----------------------+
| Normalization      | Normalized (reduce    | Denormalized (embed   |
|                    | redundancy)           | related data)         |
+--------------------+-----------------------+-----------------------+
| Maturity           | 40+ years             | ~15-20 years          |
+--------------------+-----------------------+-----------------------+
| Tooling/ecosystem  | Very mature           | Growing rapidly       |
+--------------------+-----------------------+-----------------------+
```

Normalization vs Denormalization

This is one of the most important differences in practice:

```
NORMALIZED (SQL):
  Users Table:        Orders Table:          Products Table:
  +----+-------+      +----+--------+----+   +----+--------+-------+
  | id | name  |      | id | user_id|prod|   | id | name   | price |
  +----+-------+      +----+--------+----+   +----+--------+-------+
  | 1  | Alice |      | 1  | 1      | 10 |   | 10 | Laptop | 999   |
  +----+-------+      +----+--------+----+   +----+--------+-------+

  To get "Alice's order": JOIN three tables
  Pro: No duplicate data. Update product name in ONE place.
  Con: Requires JOINs (slower at scale).


DENORMALIZED (NoSQL):
  Orders Collection:
  {
    "order_id": 1,
    "user_name": "Alice",           <-- Embedded, not referenced
    "user_email": "alice@ex.com",   <-- Duplicated
    "product_name": "Laptop",       <-- Duplicated
    "product_price": 999            <-- Duplicated
  }

  To get "Alice's order": Read ONE document
```

```
  Pro: Single read, very fast. No JOINs needed.
  Con: If product name changes, must update ALL orders.
```

6.7 Polyglot Persistence

Modern systems rarely use just one database. Different parts of the system have different data needs, and different databases excel at different things. Using multiple database types in one system is called polyglot persistence.

Example: E-Commerce System

```
POLYGLOT PERSISTENCE IN E-COMMERCE:

  +-------------------+     +-------------------+
  |   Product Catalog |     |   User Accounts   |
  |   (MongoDB)       |     |   (PostgreSQL)    |
  |   - Flexible      |     |   - ACID for      |
  |     product attrs |     |     payments       |
  |   - Varies by     |     |   - Strong        |
  |     category      |     |     consistency    |
  +-------------------+     +-------------------+

  +-------------------+     +-------------------+
  |   Shopping Cart   |     |   Product Search  |
  |   (Redis)         |     |   (Elasticsearch) |
  |   - Fast R/W      |     |   - Full-text     |
  |   - Expirable     |     |     search        |
  |   - Key-value     |     |   - Faceted       |
  |     perfect fit   |     |     filtering     |
  +-------------------+     +-------------------+

  +-------------------+     +-------------------+
  |   Recommendations |     |   Analytics/Logs  |
  |   (Neo4j)         |     |   (Cassandra)     |
  |   - "Users who    |     |   - High write    |
  |     bought X also |     |     throughput     |
  |     bought Y"     |     |   - Time-series   |
  |   - Graph         |     |     data          |
  |     traversal     |     |                   |
  +-------------------+     +-------------------+
```

Benefits of Polyglot Persistence

- Each database is optimized for its use case

- Better performance for specialized access patterns
- Independent scaling of each data store

Costs of Polyglot Persistence

- Operational complexity: More databases to deploy, monitor, backup, and upgrade
- Data consistency: Keeping data in sync across databases is hard
- Team knowledge: Developers must understand multiple database technologies
- Increased infrastructure cost: Each database system requires its own resources

When to Use Polyglot Persistence

- Do use it when different parts of your system have fundamentally different data access patterns
- Do not use it just because it sounds modern. Start with one database (usually PostgreSQL) and add specialized databases only when you hit clear limitations

---

6.8 NewSQL: The Best of Both Worlds?

A newer category of databases attempts to combine SQL's ACID guarantees with NoSQL's horizontal scalability.

```
+-------------------+-----------------------------------------+
| NewSQL Database   | Key Feature                             |
+-------------------+-----------------------------------------+
| CockroachDB       | Distributed SQL, survives datacenter    |
|                   | failures, PostgreSQL-compatible          |
+-------------------+-----------------------------------------+
| TiDB              | MySQL-compatible distributed SQL,        |
|                   | scales horizontally                      |
+-------------------+-----------------------------------------+
| Google Spanner    | Globally distributed, strongly consistent,|
|                   | Google-scale (managed service)           |
+-------------------+-----------------------------------------+
| YugabyteDB        | PostgreSQL-compatible, distributed,      |
|                   | cloud-native                             |
+-------------------+-----------------------------------------+
```

NewSQL databases are a good option when you need both ACID compliance and horizontal scaling, but they come with their own trade-offs: higher operational complexity, potential latency from distributed consensus, and relative immaturity compared to traditional SQL

databases.

Common Mistakes

1. Choosing NoSQL because it is trendy. Most applications work perfectly well with PostgreSQL. Choose NoSQL for specific technical reasons, not hype.
2. Using MongoDB as a relational database. If you find yourself doing many cross-collection lookups (JOINs), you probably need a SQL database.
3. Ignoring data modeling. NoSQL does not mean "no thinking about data." You must model your data around your access patterns, which requires even more upfront design than SQL.
4. Assuming NoSQL is always faster. SQL databases with proper indexes are very fast. NoSQL is faster for specific access patterns, not universally.
5. Not considering operational cost. Running Cassandra, MongoDB, Elasticsearch, AND PostgreSQL means your team must be expert in all of them. That is expensive.
6. Over-normalizing in SQL. If every query requires five JOINs, your schema might be too normalized. Some denormalization is OK for read-heavy workloads.
7. Forgetting about backups and recovery. NoSQL databases have different backup and recovery characteristics. Understand them before going to production.

Best Practices

1. Start with PostgreSQL when unsure. It handles relational data, JSON documents, full-text search, and more. It is the safest default choice.
2. Model data around access patterns, not just entities. In NoSQL, design your data model based on the queries you need to run, not the shape of your domain objects.
3. Use the right database for the right job. Do not force a key-value store to do complex queries, and do not force a SQL database to do high-throughput time-series writes.
4. Evaluate operational complexity. A simpler database that your team knows well may be better than a theoretically superior one nobody understands.
5. Plan for schema evolution. In SQL, use migration tools (Flyway, Alembic). In NoSQL, version your documents and handle old formats gracefully.
6. Benchmark with realistic data. Toy benchmarks are misleading. Test with production-like data volumes and access patterns.

7. Consider managed services. Running databases yourself is complex. AWS RDS, DynamoDB, Atlas (MongoDB), and Aiven reduce operational burden significantly.

Quick Summary

SQL databases store data in structured tables with strong ACID guarantees, enforced schemas, and powerful query capabilities through JOINs and aggregations. They are the right default for most applications. NoSQL databases come in four flavors: key-value (simplest, for lookups), document (flexible JSON storage), wide-column (massive write throughput), and graph (relationship traversal). NoSQL trades consistency (BASE model) for horizontal scalability. Schema enforcement in SQL catches data errors early but makes changes harder; schemaless NoSQL is flexible but shifts validation to application code. Polyglot persistence uses multiple database types in one system, matching each data need to the best tool. When in doubt, start with PostgreSQL and add specialized databases only when specific limitations emerge.

Key Points

- SQL databases provide ACID transactions, enforced schemas, and powerful JOINs -- ideal for data integrity
- NoSQL databases come in four types: key-value, document, wide-column, and graph
- ACID guarantees strong consistency; BASE provides eventual consistency with better availability
- Schema enforcement catches errors early but makes changes harder; schemaless is flexible but riskier
- Choose SQL when you need transactions, complex queries, and data integrity
- Choose NoSQL when you need horizontal scaling, flexible schemas, or specific access patterns
- Polyglot persistence uses multiple databases, each optimized for its use case
- When in doubt, start with PostgreSQL -- it is the safest default

Practice Questions

1. You are designing a social media platform. User profiles are well-structured (name, email, bio), but posts can include text, images, polls, links, or any combination. Would you use SQL, NoSQL, or both? Explain your reasoning.
2. An online game needs to store player scores and display real-time leaderboards. Players can also form guilds, and you need to query "friends of friends." Which database type(s) would you use and why?
3. Explain why a banking system should use ACID transactions rather than BASE. Give a specific example of what could go wrong with eventual consistency in a banking context.
4. Your company currently uses PostgreSQL for everything. The analytics team complains that their queries on 500 million event records are too slow and blocking production traffic. What would you recommend?
5. A startup is building a product catalog where each product category has completely different attributes (electronics have specs like RAM and screen size; clothing has size, color, material). Compare how you would model this in SQL vs a document database.

Exercises

Exercise 1: Database Selection For each scenario, choose the most appropriate database type and justify your choice: (a) storing user sessions that expire after 30 minutes, (b) a content management system where articles have different structures (blog posts, news, tutorials), (c) a fraud detection system that needs to find connections between accounts, (d) a ride-sharing app that logs every GPS coordinate from every ride.

Exercise 2: Data Modeling Take a simple e-commerce domain (users, products, orders, reviews). Model it in two ways: (a) a normalized SQL schema with at least 4 tables and foreign keys, (b) a denormalized MongoDB document schema. For each, write pseudo-queries for: "Get all orders for user Alice" and "Get the average review score for product X." Compare the complexity.

Exercise 3: Migration Planning Your startup has been running on MongoDB for 2 years. You now realize that most of your data is highly relational (users, organizations, permissions, projects, tasks). You want to migrate to PostgreSQL. Outline a step-by-step migration plan that minimizes downtime and risk. Consider: data transformation, dual-write period, testing, and rollback strategy.

What Is Next?

Now that you understand the fundamentals of SQL and NoSQL databases, the next chapters will dive deeper into how databases handle scale. Chapter 7 covers database replication (keeping copies of your data for availability and read performance), and Chapter 8 covers database sharding (splitting your data across multiple machines for write performance). Together with this chapter, they complete your understanding of how databases work in large-scale systems.

Chapter 7: Database Replication

What You Will Learn

- Why databases need replication and what problems it solves
- How leader-follower (master-slave) replication works
- How leader-leader (multi-master) replication works
- The difference between synchronous and asynchronous replication
- What replication lag is and why it matters
- How read replicas improve performance
- What failover means and how it works (manual vs automatic)
- The split-brain problem and how to avoid it

Why This Chapter Matters

Imagine you run a popular online bookstore. Your single database handles every customer order, every search, and every product page load. One day, that server dies. Your entire business goes offline. Every second costs you money and customer trust.

Now imagine you had an exact copy of that database on another server, ready to take over instantly. That is replication.

In system design interviews, replication comes up in almost every question. Whether you are designing Twitter, Instagram, or a payment system, the interviewer expects you to know how data stays available when servers fail. This chapter gives you that knowledge.

What Is Database Replication?

Database replication is the process of keeping copies of the same data on multiple database servers. Think of it like making photocopies of an important document and storing them in different locations. If one copy gets destroyed, you still have others.

```
Original Document          Copies
+----------------+    +----------------+
|                |    |                |
|   Important    |--->|   Copy in      |
|   Contract     |    |   Office Safe  |
|                |    |                |
+----------------+    +----------------+
       |
       |               +----------------+
       |               |                |
       +-------------->|   Copy at      |
       |               |   Bank Vault   |
       |               |                |
       |               +----------------+
       |
       |               +----------------+
       |               |                |
       +-------------->|   Copy at      |
                       |   Lawyer       |
                       |                |
                       +----------------+
```

Each copy is called a replica. The collection of all replicas is called a replica set.

Why Replicate?

There are three main reasons to replicate your database:

1. High Availability

If one server crashes, another replica can serve requests. Your application stays online. In the bookstore example, customers keep shopping even when a server fails.

2. Better Read Performance

Most applications read data far more often than they write it. Think about Instagram: millions of people view posts every second, but relatively few are uploading new photos at any given moment. By spreading read requests across multiple replicas, each server handles a smaller share of the traffic.

3. Geographic Distribution

If your users are spread across the world, you can place replicas closer to them. A user in Tokyo gets data from a server in Japan instead of waiting for a response from a server in Virginia.

Leader-Follower Replication (Master-Slave)

This is the most common replication setup. One server is the leader (also called the master or primary). All other servers are followers (also called slaves, secondaries, or read replicas).

The rule is simple:

- All writes go to the leader
- All reads can go to the leader or any follower
- The leader sends data changes to all followers

```
                    WRITES
                      |
                      v
             +---------------+
             |               |
             |    LEADER     |
             |   (Primary)   |
             |               |
             +-------+-------+
                     |
        +-----------+-----------+
        |           |           |
        v           v           v
+-----------+ +-----------+ +-----------+
|           | |           | |           |
| FOLLOWER  | | FOLLOWER  | | FOLLOWER  |
|   (R1)    | |   (R2)    | |   (R3)    |
|           | |           | |           |
+-----------+ +-----------+ +-----------+
     ^             ^             ^
     |             |             |
+----+--------------+-------------+----+
|              READS                    |
+---------------------------------------+
```

How It Works Step by Step

1. A client sends a write request (for example, "insert new order").
2. The leader processes the write and stores it in its database.
3. The leader records the change in a replication log (also called a write-ahead log or binary log).

4. Each follower reads the replication log and applies the same changes to its own copy of the data.
5. When a client sends a read request, it can be routed to any follower (or the leader).

```
Client: "Insert Order #1234"
          |
          v
+-------------------+
| LEADER            |
| 1. Process write  |
| 2. Store data     |
| 3. Write to log   |-----> Replication Log
+-------------------+       [INSERT Order #1234]
                                     |
                       +------------+------------+
                       |            |            |
                       v            v            v
                  +---------+  +---------+  +---------+
                  |Follower1|  |Follower2|  |Follower3|
                  |Read log |  |Read log |  |Read log |
                  |Apply    |  |Apply    |  |Apply    |
                  +---------+  +---------+  +---------+
```

Advantages of Leader-Follower

- Simple to understand and implement
- No write conflicts (only one server accepts writes)
- Easy to scale reads by adding more followers
- Well-supported by most databases (MySQL, PostgreSQL, MongoDB)

Disadvantages of Leader-Follower

- The leader is a single point of failure for writes
- All writes must go through one server, which can become a bottleneck
- Followers may have slightly stale data (replication lag)

Leader-Leader Replication (Multi-Master)

In leader-leader replication, two or more servers can accept writes. Each leader sends its changes to the other leaders. Every server is both a leader and a follower at the same time.

```
            WRITES                    WRITES
              |                         |
              v                         v
      +---------------+        +---------------+
      |               |------->|               |
      |   LEADER A    |        |   LEADER B    |
      |  (US East)    |<-------|  (EU West)    |
      |               |        |               |
      +-------+-------+        +-------+-------+
              |                         |
        +-----+-----+             +-----+-----+
        |           |             |           |
        v           v             v           v
    +---------+ +---------+ +---------+ +---------+
    |Follower | |Follower | |Follower | |Follower |
    |  A1     | |  A2     | |  B1     | |  B2     |
    +---------+ +---------+ +---------+ +---------+
```

When to Use Multi-Master

Multi-master replication makes sense when:

- You have users in multiple geographic regions and want low-latency writes everywhere
- You need write availability even if one data center goes down
- You are running an application like Google Docs where multiple users edit simultaneously

The Write Conflict Problem

The biggest challenge with multi-master replication is write conflicts. What happens when two users update the same record at the same time on different leaders?

```
Time   Leader A (US)              Leader B (EU)
----   ----------------           ----------------
T1     User sets price            User sets price
       to $10                     to $15
T2     Sends change to B -------> Sends change to A
T3     CONFLICT!                  CONFLICT!
       Should price be            Should price be
       $10 or $15?                $10 or $15?
```

Common strategies to resolve conflicts:

Strategy	How It Works	Downside

Last Write Wins (LWW)	The write with the latest timestamp wins	Can lose data silently
Merge Values	Combine both values (for example, keep both edits)	Only works for some data types
Custom Logic	Application decides which write to keep	Adds complexity
Conflict Avoidance	Route all writes for a record to one leader	Reduces multi-master benefits

Advantages of Leader-Leader

- No single point of failure for writes
- Better write performance across regions
- Higher write availability

Disadvantages of Leader-Leader

- Write conflicts are hard to resolve correctly
- More complex to set up and maintain
- Risk of data inconsistency
- Harder to debug when things go wrong

Synchronous vs Asynchronous Replication

When the leader sends changes to followers, it can do so in two ways.

Synchronous Replication

The leader waits for the follower to confirm it received and stored the change before telling the client the write was successful.

```
Client          Leader            Follower
  |                |                 |
  |--- Write ----->|                 |
  |                |--- Replicate --->|
  |                |                 |
  |                |    (Follower    |
```

```
    |                |     writes to    |
    |                |     disk)        |
    |                |                  |
    |                |<-- ACK ----------|
    |                |                  |
    |<-- Success ----|                  |
    |                |                  |

  Total time: Write time + Network round trip + Follower write time
```

Pros:

- The follower is guaranteed to have the latest data
- If the leader crashes, the follower has everything
- No data loss

Cons:

- Slower writes (the client waits for the follower)
- If the follower is slow or down, the leader is blocked
- Higher latency

Asynchronous Replication

The leader confirms the write to the client immediately. It sends the change to followers in the background.

```
  Client          Leader            Follower
    |                |                  |
    |--- Write ----->|                  |
    |                |                  |
    |<-- Success ----|                  |
    |                |                  |
    |                |--- Replicate --->|
    |                |                  |
    |                |    (Follower     |
    |                |     writes to    |
    |                |     disk)        |
    |                |                  |
    |                |<-- ACK ----------|

  Total time for client: Write time only (much faster)
```

Pros:

- Faster writes (the client does not wait for followers)
- The leader is never blocked by slow followers

- Works even when followers are temporarily unavailable

Cons:

- Risk of data loss if the leader crashes before replicating
- Followers may serve stale data

Semi-Synchronous Replication

A practical middle ground: the leader waits for at least one follower to confirm, then acknowledges the client. The remaining followers replicate asynchronously.

```
Client          Leader         Follower 1     Follower 2
  |               |               |               |
  |--- Write ----->|               |               |
  |                |-- Replicate ->|               |
  |                |               |               |
  |                |<---- ACK -----|               |
  |<-- Success ----|               |               |
  |                |               |               |
  |                |-- Replicate --|-------------->|
  |                |               |          (async)
```

This approach is used by many production systems. You get durability (at least one follower has the data) without waiting for every follower.

Comparison Table

Feature	Synchronous	Asynchronous	Semi-Synchronous
Write Speed	Slow	Fast	Medium
Data Safety	Highest	Lowest	Good
Complexity	Medium	Low	Medium
Availability	Lower (blocked by followers)	Higher	Good
Common Use	Financial systems	Social media, analytics	Most production databases

---

Replication Lag

Replication lag is the delay between when data is written to the leader and when it appears on the followers. With asynchronous replication, this lag is unavoidable.

Real-World Example

Imagine you post a photo on social media. You upload it (write goes to the leader), and immediately refresh your profile page (read goes to a follower). But the follower has not received your photo yet. Your photo is missing from your own profile. Confusing, right?

```
Timeline:

T0: User uploads photo
    Leader: [Photo stored]     Follower: [No photo yet]

T1: User refreshes profile (reads from follower)
    Leader: [Photo stored]     Follower: [No photo yet]

    User sees: "Where is my photo?!"

T2: Follower catches up
    Leader: [Photo stored]     Follower: [Photo stored]

    User sees: "Oh, there it is."
```

Types of Consistency Guarantees

To deal with replication lag, systems offer different consistency models:

Read-After-Write Consistency (Read-Your-Writes)

After a user writes something, they are guaranteed to see their own write. Other users might still see stale data. Implementation: route reads for the user's own data to the leader.

Monotonic Reads

A user will never see data go backward in time. If they saw a value at time T, they will not see an older value in a subsequent read. Implementation: route a user's reads to the same replica consistently.

Consistent Prefix Reads

If a sequence of writes happens in a certain order, any reader will see them in the same order. This matters for conversations:

```
Without consistent prefix reads:
```

```
Alice writes: "How are you?"     (T1)
Bob writes:   "I am fine!"       (T2)

Observer might see:
Bob: "I am fine!"
Alice: "How are you?"

That makes no sense!
```

Read Replicas

Read replicas are followers specifically designated to handle read traffic. They are one of the simplest ways to scale a database.

How Read Replicas Help

```
WITHOUT Read Replicas          WITH Read Replicas

All traffic -> 1 server        Writes -> Leader
                               Reads  -> Multiple Replicas


+--------+                     +--------+
| Leader | <-- 10,000 req/s    | Leader | <-- 1,000 writes/s
+--------+                     +--------+
                                    |
                               +----+----+----+
                               |    |    |    |
                               v    v    v    v
                              R1   R2   R3   R4
                                        |
                                  9,000 reads/s
                                  split across 4
                                  = 2,250 each
```

Setting Up Read Replicas

Most cloud databases make this easy:

- AWS RDS: Create up to 15 read replicas with a few clicks
- Google Cloud SQL: Add read replicas across zones or regions
- Azure Database: Create read replicas in different regions

When to Use Read Replicas

Read replicas work best when your application is read-heavy. Many applications fit this pattern:

Application	Read:Write Ratio	Good Fit?
Social media feed	100:1	Excellent
E-commerce product pages	50:1	Excellent
Banking transactions	1:1	Poor (use synchronous replication instead)
Analytics dashboard	1000:1	Excellent
Chat application	5:1	Moderate

Failover

Failover is the process of switching from a failed server to a backup server. In the context of replication, failover usually means promoting a follower to become the new leader when the old leader fails.

Manual Failover

A human operator detects the failure and manually promotes a follower. This is the safest approach but the slowest.

```
Step 1: Leader fails

+--------+     +-----------+     +-----------+
|LEADER  |     |FOLLOWER 1 |     |FOLLOWER 2 |
|  XXXX  |     |           |     |           |
| (dead) |     |           |     |           |
+--------+     +-----------+     +-----------+

Step 2: Operator promotes Follower 1

+--------+     +-----------+     +-----------+
|OLD     |     |NEW LEADER |     |FOLLOWER 2 |
|LEADER  |     |(promoted) |     |           |
| (dead) |     |           |     |           |
```

```
+--------+     +-----+-----+     +-----------+
                      |                 |
                      +----Replicates--+

Step 3: Reconfigure clients to point to new leader
```

Pros: Lower risk of mistakes, operator can verify state Cons: Slow (minutes to hours of downtime), requires on-call staff

Automatic Failover

The system detects the failure and promotes a follower without human intervention. This is faster but riskier.

```
Automatic Failover Flow:

+------------------+
| Health Monitor   |
| checks every 5s  |
+--------+---------+
         |
         v
+------------------+
| Leader missed    |
| 3 health checks? |
+--------+---------+
         | YES
         v
+------------------+
| Select best      |
| follower         |
| (most up-to-date)|
+--------+---------+
         |
         v
+------------------+
| Promote follower |
| to leader        |
+--------+---------+
         |
         v
+------------------+
| Update routing   |
| (DNS / config)   |
+--------+---------+
         |
         v
+------------------+
| Other followers  |
| follow new leader|
+------------------+
```

Pros: Fast recovery (seconds to minutes), no human needed Cons: Risk of false positives, potential data loss, split-brain risk

Challenges of Automatic Failover

1. False Positives: The leader might be slow due to a temporary network issue, not actually dead. If the system promotes a follower too quickly, you now have two leaders (split-brain).
2. Data Loss: If the old leader had unreplicated writes (asynchronous replication), those writes are lost when a follower takes over.
3. Stale Reads: The promoted follower might be behind the old leader, serving slightly old data until it catches up.

The Split-Brain Problem

Split-brain is one of the most dangerous situations in distributed databases. It occurs when two servers both believe they are the leader and accept writes independently.

```
SPLIT-BRAIN SCENARIO

Network partition splits the cluster:

     Network Partition
          |||
+---------|||----------+
|         |||          |
| +------+|||+------+  |
| |LEADER||||LEADER|  |   <-- TWO LEADERS!
| |  A   |||||  B   |  |
| +---+--+|||+--+---+  |
|     |   |||   |      |
|  +--+-- |||--++--+   |
|  |F1|F2||||F3||F4|   |
|  +--+--+|||+-+--++   |
|         |||          |
+---------|||----------+

Leader A accepts: UPDATE price = $10
Leader B accepts: UPDATE price = $15

When network heals: WHICH PRICE IS CORRECT?
```

Why Split-Brain Is Dangerous

- Two copies of data diverge, and merging them is hard or impossible
- Clients connected to different leaders see different data
- Financial systems could process duplicate transactions
- Auto-incrementing IDs could produce duplicates

How to Prevent Split-Brain

1. Quorum-Based Decisions

A server can only become leader if a majority of nodes agree. Since a majority can only exist on one side of a partition, only one leader can emerge.

```
5-node cluster: Need 3 votes to be leader

Partition:
Side A: 3 nodes -> CAN elect leader (has majority)
Side B: 2 nodes -> CANNOT elect leader (no majority)
```

2. Fencing (STONITH)

When a new leader is promoted, the old leader is forcefully shut down. STONITH stands for "Shoot The Other Node In The Head."

3. Lease-Based Leadership

The leader holds a time-limited lease. If it cannot renew the lease (because of a network partition), it must step down. Only one server can hold the lease at a time.

4. External Coordination Service

Use a system like ZooKeeper or etcd to manage leader election. These systems are designed to handle exactly this kind of problem.

Replication in Practice

MySQL Replication

MySQL uses binary log (binlog) replication. The leader writes all changes to the binary log, and followers read from it.

```
MySQL Replication Architecture

+-----------------+
|     LEADER      |
|                 |
| +-----+ +-----+ |
| | InnoDB| |Binlog| |
| |Engine| | Log | |
| +-----+ +--+--+ |
+-------------|----+
              |
      +-------+-------+
      |               |
      v               v
+-----------+   +-----------+
| FOLLOWER1 |   | FOLLOWER2 |
| +-------+ |   | +-------+ |
| |Relay  | |   | |Relay  | |
| |Log    | |   | |Log    | |
| +---+---+ |   | +---+---+ |
|     |     |   |     |     |
| +---v---+ |   | +---v---+ |
| |InnoDB | |   | |InnoDB | |
| |Engine | |   | |Engine | |
| +-------+ |   | +-------+ |
+-----------+   +-----------+
```

PostgreSQL Replication

PostgreSQL uses Write-Ahead Log (WAL) shipping. The leader streams WAL records to followers.

MongoDB Replication

MongoDB uses replica sets with automatic leader election. If the primary fails, the remaining members vote to elect a new primary.

Trade-Offs

Decision	Option A	Option B
Topology	Leader-Follower (simple, no conflicts)	Leader-Leader (multi-region writes, complex)

Replication Mode	Synchronous (safe, slow)	Asynchronous (fast, risk of data loss)
Failover	Manual (safe, slow recovery)	Automatic (fast recovery, risk of split-brain)
Number of Replicas	Few (less overhead, limited read scaling)	Many (better read scaling, more lag management)
Consistency	Strong (correct but slow)	Eventual (fast but stale reads possible)

Common Mistakes

1. Using multi-master replication without handling conflicts. If two leaders accept conflicting writes, data corruption follows. Always have a conflict resolution strategy.
2. Ignoring replication lag in application code. Your application must handle the case where a read replica returns stale data. Do not assume followers are always up to date.
3. No monitoring on replication lag. If lag grows, reads become increasingly stale. Set up alerts when lag exceeds acceptable thresholds.
4. Treating failover as fully automatic and forgetting about it. Even with automatic failover, you need runbooks, testing, and monitoring. Simulate failures regularly.
5. Not testing failover before you need it. The worst time to discover your failover does not work is during an actual outage. Practice with chaos engineering.

Best Practices

1. Start with leader-follower replication. It covers the vast majority of use cases and avoids the complexity of multi-master conflict resolution.
2. Use semi-synchronous replication for important data. At least one follower should confirm writes synchronously to prevent data loss.
3. Route read-after-write queries to the leader. When a user writes data and immediately reads it back, send that read to the leader to avoid confusion from replication lag.
4. Monitor replication lag continuously. Set alerts for lag exceeding your acceptable threshold (often 1 to 5 seconds for most applications).

5. Test failover regularly. Schedule periodic failover drills. Netflix popularized this with Chaos Monkey, which randomly terminates instances in production.
6. Use connection pooling and routing layers. Tools like ProxySQL (MySQL) or pgBouncer (PostgreSQL) can automatically route reads to replicas and writes to the leader.
7. Plan for cross-region replication latency. A replica in another continent will always have higher lag. Design your application to tolerate this.

Quick Summary

Database replication keeps copies of your data on multiple servers. Leader-follower replication is the most common pattern: one server handles writes, and followers handle reads. Leader-leader replication allows writes on multiple servers but introduces conflict resolution challenges. Synchronous replication guarantees data safety at the cost of speed; asynchronous replication is faster but risks data loss. Failover promotes a follower when the leader fails. The split-brain problem occurs when two servers think they are the leader, and it is prevented using quorums, fencing, or coordination services.

Key Points

- Replication provides high availability, read scaling, and geographic distribution
- Leader-follower is simpler and avoids write conflicts; leader-leader supports multi-region writes but requires conflict resolution
- Synchronous replication guarantees follower has the data; asynchronous is faster but can lose data
- Replication lag means followers may serve stale data; use read-after-write consistency for user's own data
- Failover can be manual (safe, slow) or automatic (fast, risky)
- Split-brain means two leaders accepting writes independently, causing data divergence
- Semi-synchronous replication is a practical middle ground for most production systems

---

Practice Questions

1. Your e-commerce application writes 100 orders per second and serves 10,000 product page views per second. You currently have a single database. How would you use replication to handle this load? Which topology would you choose and why?
2. A user updates their profile picture and immediately sees the old picture on their profile page. What is happening, and how would you fix it?
3. Your database cluster has one leader and four followers with asynchronous replication. The leader crashes and has 30 seconds of unreplicated writes. What are your options, and what are the trade-offs of each?
4. You are designing a system that operates in three regions: US, Europe, and Asia. Each region needs low-latency writes. Would you use leader-follower or leader-leader replication? What challenges would you face?
5. Explain why the split-brain problem is especially dangerous for a banking application. How would you prevent it?

Exercises

Exercise 1: Design a Replication Strategy

You are building a social media platform with 10 million daily active users. The read-to-write ratio is 100:1. Users are distributed 60% in North America, 30% in Europe, and 10% in Asia. Design a replication topology. Draw the architecture, specify synchronous vs asynchronous for each link, and explain your failover strategy.

Exercise 2: Replication Lag Simulation

Imagine you have a leader and two followers. The leader processes writes at 1,000 per second. Follower A can apply 800 writes per second. Follower B can apply 1,200 writes per second. After 10 minutes of continuous operation, how far behind is Follower A? What would you do about it?

Exercise 3: Failover Runbook

Write a step-by-step runbook for handling a leader failure in a leader-follower setup with three followers. Include: how to detect the failure, how to choose which follower to promote, how to handle unreplicated writes, how to reconfigure the remaining followers, and how to verify the system is healthy after failover.

What Is Next?

Now that you understand how to replicate data across multiple servers, the next question is: what happens when a single database grows so large that even the leader cannot handle all the writes or store all the data? The answer is database sharding, which splits your data across multiple independent databases. In the next chapter, you will learn how to partition data, choose a shard key, and handle the unique challenges that sharding introduces.

Chapter 8: Database Sharding

What You Will Learn

- Why a single database cannot handle billions of rows
- What horizontal partitioning (sharding) means
- How to choose a sharding strategy: hash-based, range-based, geographic, and directory-based
- Why shard key selection is the most critical decision
- The problems sharding introduces: cross-shard joins, hotspots, and resharding
- What consistent hashing is and why it matters for sharding
- The celebrity problem (hot partitions)

Why This Chapter Matters

Your application is a hit. Millions of users are signing up. Your database, even with replicas handling reads, is struggling. The single leader server that handles all writes is running out of disk space, CPU, and memory. You cannot make the server bigger forever. There is a physical limit to how powerful a single machine can be.

Sharding is the answer. It splits your data across multiple independent databases, each handling a portion of the total data. Every large-scale system, from Instagram to Uber to Stripe, uses sharding. In system design interviews, sharding is one of the most frequently discussed topics. Understanding it deeply sets you apart from other candidates.

The Problem: Why Shard?

Let us say you run an e-commerce platform. Your orders table has grown to 2 billion rows. Here is what happens:

```
Single Database Problem:

+-----------------------------------+
|         SINGLE DATABASE           |
|                                   |
|  Orders Table: 2 billion rows     |
|  Size: 4 TB                       |
|  Indexes: 800 GB                  |
|                                   |
|  Problems:                        |
|  - Queries take 30+ seconds       |
|  - Backups take 12 hours          |
|  - Schema migrations = downtime   |
|  - Single disk is full            |
|  - CPU at 95% constantly          |
+-----------------------------------+
```

Vertical scaling (buying a bigger server) has limits. The most powerful single server you can buy might have 128 CPU cores and 4 TB of RAM, but eventually your data outgrows even that. And those machines are extremely expensive.

Horizontal scaling (sharding) is the solution. Instead of one giant database, you split the data across multiple smaller databases:

```
After Sharding:

+------------+  +------------+  +------------+  +------------+
|  SHARD 1   |  |  SHARD 2   |  |  SHARD 3   |  |  SHARD 4   |
|            |  |            |  |            |  |            |
| 500M rows  |  | 500M rows  |  | 500M rows  |  | 500M rows  |
| 1 TB       |  | 1 TB       |  | 1 TB       |  | 1 TB       |
|            |  |            |  |            |  |            |
| Fast       |  | Fast       |  | Fast       |  | Fast       |
| queries!   |  | queries!   |  | queries!   |  | queries!   |
+------------+  +------------+  +------------+  +------------+
```

Each shard is an independent database. It has its own CPU, memory, disk, and indexes. Queries against 500 million rows are much faster than queries against 2 billion rows.

Sharding Strategies

The most important decision in sharding is: how do you decide which data goes to which shard? This is determined by the shard key (also called partition key) and the sharding strategy.

1. Hash-Based Sharding

Take the shard key, run it through a hash function, and use the result to determine the shard.

```
Hash-Based Sharding:

shard_number = hash(shard_key) % number_of_shards

Example with 4 shards:

User ID: 1001  ->  hash(1001) % 4 = 2  ->  Shard 2
User ID: 1002  ->  hash(1002) % 4 = 0  ->  Shard 0
User ID: 1003  ->  hash(1003) % 4 = 3  ->  Shard 3
User ID: 1004  ->  hash(1004) % 4 = 1  ->  Shard 1

+----------+    +----------+    +----------+    +----------+
| Shard 0  |    | Shard 1  |    | Shard 2  |    | Shard 3  |
|          |    |          |    |          |    |          |
| User 1002|    | User 1004|    | User 1001|    | User 1003|
| User 1006|    | User 1008|    | User 1005|    | User 1007|
| ...      |    | ...      |    | ...      |    | ...      |
+----------+    +----------+    +----------+    +----------+
```

Pros:

- Distributes data evenly (good hash functions spread values uniformly)
- Simple to implement
- Works well for key-value lookups

Cons:

- Adding or removing shards requires rehashing almost everything
- Range queries are impossible (data for consecutive IDs is scattered)
- Cannot do efficient "get all users between ID 1000 and 2000"

2. Range-Based Sharding

Divide data into contiguous ranges based on the shard key value.

```
Range-Based Sharding:

Shard by User ID ranges:
```

```
+------------------+  +------------------+  +------------------+
|    Shard A       |  |    Shard B       |  |    Shard C       |
|                  |  |                  |  |                  |
| User IDs:       |  | User IDs:       |  | User IDs:       |
| 1 - 1,000,000   |  | 1,000,001 -     |  | 2,000,001 -     |
|                  |  | 2,000,000       |  | 3,000,000       |
+------------------+  +------------------+  +------------------+

Query: "Get users 500 to 600"
-> Only needs to go to Shard A!
```

Pros:

- Range queries are efficient (data in a range lives on the same shard)
- Simple to understand
- Easy to split a shard when it gets too large

Cons:

- Can create hotspots (if most new users have high IDs, the last shard gets all the writes)
- Uneven distribution is common
- Requires careful range planning

3. Geographic Sharding

Route data to shards based on geographic location. This is common for applications with strong regional data locality requirements.

```
Geographic Sharding:

+------------------+  +------------------+  +------------------+
|   US Shard       |  |   EU Shard       |  |  APAC Shard     |
|                  |  |                  |  |                  |
| US users' data   |  | EU users' data   |  | Asian users'    |
| US East DC       |  | Frankfurt DC     |  | data            |
|                  |  |                  |  | Singapore DC    |
+------------------+  +------------------+  +------------------+

User in Paris -> EU Shard (low latency)
User in Tokyo -> APAC Shard (low latency)
User in NYC   -> US Shard (low latency)
```

Pros:

- Low latency for users (data is close to them)
- Helps with data residency laws (GDPR requires EU data to stay in EU)

- Natural partitioning for many business models

Cons:

- Uneven distribution (some regions have far more users)
- Cross-region queries are slow
- Users who travel may experience higher latency

4. Directory-Based Sharding

Maintain a lookup table that maps each shard key to its shard. A central directory service knows where every piece of data lives.

```
Directory-Based Sharding:

+---------------------+
|   LOOKUP TABLE      |
|                     |
| User 1001 -> Shard2 |
| User 1002 -> Shard1 |
| User 1003 -> Shard3 |
| User 1004 -> Shard1 |
| ...                 |
+----------+----------+
           |
   +-------+-------+-------+
   |       |       |       |
   v       v       v       v
+------+ +------+ +------+ +------+
|Shard1| |Shard2| |Shard3| |Shard4|
+------+ +------+ +------+ +------+
```

Pros:

- Maximum flexibility (can move any record to any shard)
- Easy to rebalance without changing the hashing strategy
- Can accommodate custom routing logic

Cons:

- The lookup table is a single point of failure
- Every query requires a lookup first (extra latency)
- The lookup table itself can become a bottleneck

Comparison of Strategies

Strategy	Distribution	Range Queries	Flexibility	Complexity
Hash-Based	Even	Not supported	Low	Low
Range-Based	Can be uneven	Efficient	Medium	Medium
Geographic	Varies by region	Regional only	Medium	Medium
Directory-Based	Controllable	Depends	High	High

Shard Key Selection: The Most Critical Decision

Choosing the right shard key can make or break your system. A bad shard key leads to hotspots, inefficient queries, and painful migrations. Here is how to think about it.

What Makes a Good Shard Key?

A good shard key has three properties:

1. High cardinality: Many distinct values so data spreads across shards. User ID is good (millions of values). Boolean "is_active" is terrible (only two values).
2. Even distribution: Values should distribute data evenly. A timestamp shard key is bad because all recent data goes to one shard.
3. Query alignment: The shard key should match your most common query patterns. If you always query by user ID, shard by user ID.

Examples of Good and Bad Shard Keys

```
E-Commerce Application

GOOD shard key: user_id
- High cardinality (millions of users)
- Even distribution
- Most queries are per-user ("show my orders")

BAD shard key: order_status
- Low cardinality (pending, shipped, delivered)
- Uneven ("delivered" has 80% of orders)
- Creates massive hotspots

BAD shard key: created_at
- All new orders go to one shard
```

```
- Creates a "write hotspot" on the newest shard

+------------------+------------------+------------------+
|                  |                  |                  |
| Shard by         | Shard by         | Shard by         |
| user_id          | order_status     | created_at       |
|                  |                  |                  |
| [====]  [====]   | [==]  [==]       | [=]   [=]        |
| [====]  [====]   | [========]       | [=]   [========] |
|                  | (hotspot!)       | (hotspot!)       |
| Even!            | Uneven!          | Uneven!          |
+------------------+------------------+------------------+
```

Compound Shard Keys

Sometimes a single field is not enough. You can combine fields to create a compound shard key.

For a messaging application, you might use `(user_id, date)` as the compound key. This distributes data by user and also keeps recent messages together for efficient queries.

Problems with Sharding

Sharding introduces several challenges that do not exist with a single database.

1. Cross-Shard Joins

With a single database, joining the `users` table with the `orders` table is simple. With sharding, user data might be on Shard 1 and their orders on Shard 3.

```
Cross-Shard Join Problem:

Query: "Get user name and their orders"

+------------------+          +------------------+
|    Shard 1       |          |    Shard 3       |
|                  |          |                  |
| Users table:     |          | Orders table:    |
| ID=1001, Alice   |   ???    | User 1001,       |
|                  | <------> | Order #5555      |
+------------------+          +------------------+

JOIN across shards requires:
```

```
1. Query Shard 1 for user data
2. Query Shard 3 for order data
3. Combine results in application code
4. Much slower than a single-DB join!
```

Solutions:

- Denormalize data (store user name alongside orders)
- Keep related data on the same shard (shard both tables by user_id)
- Use an application-level join (query both shards and merge in code)

2. Hotspots

Even with a good shard key, certain records may receive disproportionately more traffic. This is the celebrity problem.

```
The Celebrity Problem:

Instagram sharded by user_id:

+------------------+  +------------------+  +------------------+
|    Shard 1       |  |    Shard 2       |  |    Shard 3       |
|                  |  |                  |  |                  |
| Regular users    |  | Kim Kardashian   |  | Regular users    |
| 100 req/s each   |  | 1,000,000 req/s  |  | 100 req/s each   |
|                  |  |                  |  |                  |
| Total: 10K req/s |  | Total: 1M req/s  |  | Total: 10K req/s |
|                  |  |  (OVERLOADED!)   |  |                  |
+------------------+  +------------------+  +------------------+
```

Solutions:

- Add a random suffix to the shard key for hot records (split one celebrity's data across multiple shards)
- Cache celebrity data aggressively
- Assign dedicated resources to hot shards

3. Resharding

When you need to change the number of shards (because data grew or shrank), you need to redistribute data. With simple hash-based sharding, this means moving most of your data.

```
Resharding Problem (hash-based):
```

```
BEFORE: 4 shards
hash(key) % 4 = shard_number

AFTER: 5 shards
hash(key) % 5 = shard_number

Key "user_1001":
  hash(1001) = 7834
  7834 % 4 = 2  (was on Shard 2)
  7834 % 5 = 4  (now belongs on Shard 4)

~80% of data needs to move!
```

This is why consistent hashing was invented (covered in Chapter 12). With consistent hashing, adding a new shard only requires moving about `1/n` of the data, where `n` is the number of shards.

4. Auto-Incrementing IDs

With a single database, auto-incrementing IDs are easy. With multiple shards, each shard generates its own IDs, causing collisions.

```
ID Collision Problem:

Shard 1: INSERT -> ID = 1, 2, 3, 4...
Shard 2: INSERT -> ID = 1, 2, 3, 4...   <-- Same IDs!
```

Solutions:

Solution	How It Works	Downside
UUID	Generate random 128-bit IDs	Large, not sortable
Snowflake IDs	Combine timestamp + machine ID + sequence	Requires clock synchronization
Range allocation	Each shard gets a range (Shard 1: 1-1M, Shard 2: 1M-2M)	Requires coordination
Central ID service	One service generates all IDs	Single point of failure

5. Transactions Across Shards

Single-database transactions are straightforward. Cross-shard transactions (distributed transactions) are slow and complex.

```
Cross-Shard Transaction:

"Transfer $100 from Account A (Shard 1) to Account B (Shard 3)"

Shard 1: Debit $100 from A   \
                              } Must BOTH succeed or BOTH fail
Shard 3: Credit $100 to B    /

Requires: Two-Phase Commit (2PC) or Saga pattern
Both add significant complexity.
```

Consistent Hashing Preview

Consistent hashing solves the resharding problem. Instead of `hash(key) % n`, it uses a hash ring where both servers and keys are mapped to positions on a circle.

```
Consistent Hashing Ring (Preview):

         0
         |
   315 --+-- 45
       /   \
  270 -+     +- 90
       \   /
   225 --+-- 135
         |
        180

Servers at positions: S1=45, S2=135, S3=270
Keys map to positions and go to the next server clockwise.

Adding S4 at 200:
- Only keys between 135 and 200 move to S4
- Everything else stays put!
```

We will explore consistent hashing in detail in Chapter 12.

Sharded Architecture in Practice

Here is what a complete sharded architecture looks like:

```
Complete Sharded Architecture:

+------------------+
|   Application    |
|    Servers       |
+--------+---------+
         |
         v
+------------------+
|  Shard Router /  |
|  Proxy Layer     |
|                  |
| "Which shard     |
|  has this data?" |
+--------+---------+
         |
   +-----+-----+-----+-----+
   |     |     |     |     |
   v     v     v     v     v
+----+ +----+ +----+ +----+ +----+
| S1 | | S2 | | S3 | | S4 | | S5 |
+----+ +----+ +----+ +----+ +----+
| L  | | L  | | L  | | L  | | L  |  <-- Each shard has
| |  | | |  | | |  | | |  | | |  |      its own leader
| F F| | F F| | F F| | F F| | F F|      and followers
+----+ +----+ +----+ +----+ +----+      (replication!)

L = Leader    F = Follower

Each shard is independently replicated for high availability.
```

The shard router (also called a proxy or coordinator) is the component that knows which shard holds which data. It can be:

- A dedicated proxy service (like Vitess for MySQL or mongos for MongoDB)
- A library embedded in the application
- A middleware layer

Trade-Offs

Decision	Option A	Option B
Sharding vs Not Sharding	Shard (handles scale, adds complexity)	Single DB with replicas (simpler, limited scale)

Hash vs Range	Hash (even distribution, no range queries)	Range (range queries, risk of hotspots)
Shard Key	user_id (per-user queries are fast)	order_id (per-order queries are fast, cross-user analytics hard)
Number of Shards	Few large shards (simpler, resharding sooner)	Many small shards (longer runway, more operational burden)
Routing	Proxy-based (centralized, extra hop)	Client-side (faster, clients need shard map)

Common Mistakes

1. Sharding too early. Sharding adds enormous complexity. Many systems never need it. Optimize queries, add indexes, use read replicas, and cache aggressively before considering sharding.
2. Choosing the wrong shard key. Changing the shard key after the system is in production is extremely painful, often requiring a full data migration. Spend significant time on this decision.
3. Forgetting about cross-shard queries. If your application needs complex joins or aggregations across all data, sharding will make those operations much slower or impossible.
4. Ignoring the celebrity problem. Even with hash-based sharding, a single hot key can overwhelm one shard. Plan for uneven traffic patterns.
5. Not planning for resharding. Your data will grow. If you use simple modular hashing, adding shards means moving most of your data. Use consistent hashing from the start.

Best Practices

1. Delay sharding as long as possible. Use vertical scaling, read replicas, caching, and query optimization first. Shard only when you truly need to.
2. Choose the shard key based on your query patterns. The most common query determines the best shard key. If 90% of queries are "get data for user X," shard by user_id.

3. Keep related data on the same shard. If users and their orders are always queried together, shard both tables by user_id so joins stay local.
4. Plan for at least 2x your current data. If you have 100 GB now, design for 200 GB or more. This gives you room to grow before resharding.
5. Use consistent hashing. It makes adding and removing shards much less painful. Most modern systems use this approach.
6. Each shard should be replicated. Sharding alone does not provide high availability. Each shard needs its own leader-follower replication setup.
7. Monitor shard sizes and query patterns. Uneven shards indicate a bad shard key or hotspot. Catch this early.
8. Build a shard management tool. You will need the ability to split shards, merge shards, and move data between shards. Automate these operations.

Quick Summary

Database sharding splits data across multiple independent databases (shards) to handle scale beyond what a single server can manage. The shard key determines how data is distributed. Hash-based sharding gives even distribution but prevents range queries. Range-based sharding supports range queries but risks hotspots. Geographic sharding keeps data close to users. Directory-based sharding offers maximum flexibility but adds a lookup overhead. Sharding introduces challenges including cross-shard joins, hotspots (the celebrity problem), resharding complexity, ID generation, and distributed transactions. Start with the simplest approach that works, and always replicate each shard for availability.

Key Points

- Shard when a single database cannot handle the data size or write throughput, not before
- The shard key is the most important decision; it determines data distribution and query efficiency
- Hash-based sharding distributes evenly but scatters range queries; range-based supports ranges but risks hotspots
- Cross-shard joins are expensive; keep related data on the same shard

- The celebrity problem causes hotspots even with good shard keys; use caching or key splitting
- Resharding with modular hashing moves most data; consistent hashing moves only 1/n of data
- Each shard should be independently replicated for high availability
- Delay sharding as long as possible; it adds significant operational complexity

---

Practice Questions

1. You are building a messaging application with 500 million users. Each user sends an average of 50 messages per day. How would you shard the messages table? What would you choose as the shard key, and why?
2. Your e-commerce platform is sharded by user_id. A product manager asks for a report: "Show me total revenue per product category across all users." How would you generate this report? What are the challenges?
3. A social media platform uses hash-based sharding with 8 shards. A celebrity with 100 million followers posts an update. How does this create a problem, and what would you do about it?
4. Compare hash-based and range-based sharding for a time-series database that stores IoT sensor readings. Which would you choose and why?
5. You currently have 4 shards and need to add a fifth. With simple modular hashing, approximately what percentage of data needs to move? How does consistent hashing improve this?

Exercises

Exercise 1: Shard Key Analysis

For each of the following applications, propose a shard key and justify your choice. Consider query patterns, distribution, and potential hotspots.

- A ride-sharing application (like Uber)
- An email service (like Gmail)
- A video streaming platform (like YouTube)

Exercise 2: Design a Sharding Migration Plan

Your application currently uses a single PostgreSQL database with 500 GB of data. You need to shard it into 4 shards with zero downtime. Write a step-by-step migration plan. Consider: how to set up the new shards, how to copy data, how to handle writes during migration, and how to switch traffic.

Exercise 3: Cross-Shard Query Optimization

You have a users table sharded by user_id and an orders table also sharded by user_id. The business needs a daily report that ranks the top 100 products by total revenue across all users. Design an efficient way to generate this report without querying all shards synchronously.

What Is Next?

You now know how to replicate databases for availability and shard them for scale. But what happens between your services? When Service A needs to tell Service B that something happened, should it call Service B directly? What if Service B is down? What if it takes too long? The answer is message queues, the backbone of asynchronous communication in modern systems. In the next chapter, you will learn how message queues decouple services, buffer traffic, and make your system more resilient.

Chapter 9: Message Queues

What You Will Learn

- What a message queue is and why it exists
- How producers and consumers work
- The difference between point-to-point and publish/subscribe patterns
- The key benefits: decoupling, async processing, buffering, and ordering
- How Kafka, RabbitMQ, and SQS compare
- What dead letter queues are and when to use them
- The difference between at-least-once, at-most-once, and exactly-once delivery
- What back pressure is and how to handle it
- Real-world use cases: order processing, email sending, event streaming

Why This Chapter Matters

Imagine you walk into a busy post office. You do not wait for your letter to be delivered before you leave. You hand it to the clerk, get a receipt, and go about your day. The postal system takes care of delivery in the background. If the delivery truck is full, letters wait in the sorting facility. If the recipient is not home, the carrier tries again later.

A message queue works exactly like this. It sits between the sender and receiver, holding messages until the receiver is ready to process them. Without message queues, modern distributed systems would be fragile, slow, and tightly coupled. Every system design interview that involves multiple services will benefit from your understanding of message queues.

What Is a Message Queue?

A message queue is a form of middleware that allows different parts of a system to communicate by sending and receiving messages through a shared buffer. Think of it as a to-do list that one part of the system writes to and another part reads from.

```
Without a Message Queue:

+----------+    Direct call    +----------+
|          |------------------>|          |
| Service A|   (must wait)    | Service B|
|          |<-----------------|          |
+----------+    Response       +----------+

If Service B is slow or down, Service A is stuck.


With a Message Queue:

+----------+    +-------+    +----------+
|          |--->|       |--->|          |
| Service A|    | QUEUE |    | Service B|
|  (done!) |    |       |    |(whenever)|
+----------+    +-------+    +----------+

Service A drops the message and moves on.
Service B processes it when ready.
```

Core Terminology

- Message: A unit of data sent from one service to another. It could be a JSON object, a binary blob, or a simple string.
- Producer (Publisher/Sender): The service that creates and sends messages to the queue.
- Consumer (Subscriber/Receiver): The service that reads and processes messages from the queue.
- Queue (Topic/Channel): The buffer where messages wait to be processed.
- Broker: The server that manages queues, stores messages, and delivers them to consumers. Examples: RabbitMQ, Kafka, ActiveMQ.

```
Message Queue Components:

+----------+                           +----------+
| PRODUCER |---+                   +--->| CONSUMER |
+----------+   |    +---------+    |    +----------+
                +--->|         |----+
+----------+   |    | BROKER  |    |    +----------+
| PRODUCER |---+    |         |    +--->| CONSUMER |
```

```
+----------+        | +-----+ |         +----------+
                     | |QUEUE| |
                     | +-----+ |
                     | |QUEUE| |
                     | +-----+ |
                     +---------+
```

Point-to-Point vs Publish/Subscribe

There are two fundamental messaging patterns.

Point-to-Point (Queue)

Each message is consumed by exactly one consumer. Once a consumer reads a message, it is removed from the queue. This is like a task list: each task is done by one worker.

```
Point-to-Point:

Producer --> [Message A, Message B, Message C] --> Queue

Consumer 1 picks up Message A
Consumer 2 picks up Message B
Consumer 3 picks up Message C

Each message processed ONCE by ONE consumer.

+----------+     +---+---+---+     +----------+
|          |     | C | B | A |---->| Consumer1| (gets A)
| Producer |---->|   |   |   |     +----------+
|          |     +---+---+---+---->| Consumer2| (gets B)
+----------+                 |    +----------+
                             +--->| Consumer3| (gets C)
                                  +----------+
```

Use cases: Task distribution, work queues, order processing.

Publish/Subscribe (Pub/Sub)

Each message is delivered to all subscribers. A message published to a topic is copied to every consumer that has subscribed. This is like a newspaper: every subscriber gets their own copy.

```
Publish/Subscribe:

Publisher --> [Message A] --> Topic

Subscriber 1 gets Message A
Subscriber 2 gets Message A
Subscriber 3 gets Message A

Every subscriber gets EVERY message.

+----------+     +---------+     +-------------+
|          |     |         |---->| Subscriber 1| (gets A)
|Publisher |---->|  TOPIC  |     +-------------+
|          |     |         |---->| Subscriber 2| (gets A)
+----------+     |         |     +-------------+
                 |         |---->| Subscriber 3| (gets A)
                 +---------+     +-------------+
```

Use cases: Event notifications, real-time feeds, broadcasting updates.

Comparison

Feature	Point-to-Point	Pub/Sub
Message delivered to	One consumer	All subscribers
Use case	Task processing	Event broadcasting
Scaling	Add more consumers to share load	Each subscriber processes independently
Message removal	After consumption	Retained for a period (Kafka) or after all subscribers read it

---

Benefits of Message Queues

1. Decoupling

Services do not need to know about each other. The producer sends a message to the queue. It does not care who reads it, how many consumers there are, or what language they are written in.

```
Tight Coupling (Without Queue):

Order Service ---> Email Service
              ---> Inventory Service
              ---> Analytics Service
              ---> Notification Service

Order Service must know about ALL downstream services.
If Inventory Service changes its API, Order Service breaks.


Loose Coupling (With Queue):

Order Service ---> [ORDER PLACED queue]
                        |
            +-----------+-----------+-----------+
            |           |           |           |
       Email Svc   Inventory   Analytics   Notification
                      Svc         Svc         Svc

Order Service knows nothing about consumers.
Add or remove consumers freely.
```

2. Asynchronous Processing

The producer does not wait for the work to be done. It sends the message and moves on. The consumer processes the message whenever it is ready.

Example: When a user places an order, the order service immediately confirms the order. Sending the confirmation email, updating inventory, and generating an invoice all happen asynchronously via message queues.

```
Synchronous (slow):
User -> Place Order -> Send Email -> Update Inventory -> Response
Total time: 50ms + 200ms + 100ms = 350ms

Asynchronous with queue (fast):
User -> Place Order -> [Queue messages] -> Response
Total time: 50ms + 5ms = 55ms

Email and inventory happen in the background.
```

3. Buffering and Load Leveling

Message queues absorb traffic spikes. If a flash sale generates 100,000 orders per minute but your payment system can only handle 10,000, the queue holds the excess until the payment system catches up.

```
Traffic Spike Without Queue:

100K req/min ----> Payment Service (10K capacity)
                   80K REQUESTS DROPPED!

Traffic Spike With Queue:

100K req/min ----> [QUEUE] ----> Payment Service (10K capacity)
                   Queue holds excess.
                   Payment processes at its own pace.
                   No requests lost.

Queue Depth Over Time:

Depth
10K |    ****
    |   *    *
 5K |  *      *
    | *        *
  0 |*          ********
    +--------------------> Time
     Spike    Catching up   Caught up
```

4. Message Ordering

Many queues guarantee that messages are delivered in the order they were sent. This is critical for operations that must happen in sequence, like financial transactions.

5. Retry and Fault Tolerance

If a consumer fails while processing a message, the message goes back to the queue and another consumer can pick it up. The message is not lost.

Kafka vs RabbitMQ vs SQS

These are the three most widely used message queue systems. Each has different strengths.

Apache Kafka

Kafka is a distributed event streaming platform. It stores messages in an ordered, immutable log. Messages are not deleted after consumption; they are retained for a configurable period.

```
Kafka Architecture:

+-----------------------------------------------+
|              KAFKA CLUSTER                    |
|                                               |
|  Topic: "orders"                              |
|  +--------+ +--------+ +--------+            |
|  |Partn 0 | |Partn 1 | |Partn 2 |            |
|  |[M1][M4]| |[M2][M5]| |[M3][M6]|            |
|  +--------+ +--------+ +--------+            |
|                                               |
|  Each partition is an ordered, immutable log |
+-----------------------------------------------+

Consumer Group A:  C1 reads P0, C2 reads P1, C3 reads P2
Consumer Group B:  C4 reads P0+P1+P2 (all partitions)

Messages are NOT deleted after reading.
Multiple consumer groups can read the same data.
```

Key features:

- Extremely high throughput (millions of messages per second)
- Messages retained for days/weeks (replayable)
- Partitioned topics for parallelism
- Consumer groups for scalable consumption
- Built for event streaming and log aggregation

RabbitMQ

RabbitMQ is a traditional message broker. It routes messages from producers to queues using exchanges and bindings. Messages are deleted after successful consumption.

```
RabbitMQ Architecture:

+----------+     +-----------+     +---------+     +----------+
| Producer |---->| Exchange  |---->|  Queue  |---->| Consumer |
+----------+     +-----------+     +---------+     +----------+
                      |
                      |            +---------+     +----------+
                      +----------->|  Queue  |---->| Consumer |
                                   +---------+     +----------+

Exchange types:
- Direct:  Route by exact routing key
- Topic:   Route by pattern matching
- Fanout:  Broadcast to all queues
- Headers: Route by message headers
```

Key features:

- Flexible routing with exchanges
- Message acknowledgment and rejection
- Priority queues
- Dead letter exchanges
- Built for task distribution and RPC patterns

Amazon SQS

SQS is a fully managed queue service from AWS. No servers to manage, no clusters to configure.

```
SQS Architecture:

+----------+     +------------------+     +----------+
| Producer |---->|                  |---->| Consumer |
+----------+     |   AWS SQS       |     +----------+
                 |                  |
+----------+     |  Fully managed   |     +----------+
| Producer |---->|  Auto-scaling    |---->| Consumer |
+----------+     |  Pay-per-message |     +----------+
                 +------------------+

Two types:
- Standard Queue: Best-effort ordering, at-least-once delivery
- FIFO Queue:     Strict ordering, exactly-once processing
```

Key features:

- Zero operational overhead (fully managed)
- Automatic scaling
- Pay only for what you use
- Integrates with AWS Lambda for serverless consumption
- Visibility timeout prevents duplicate processing

Comparison Table

Feature	Kafka	RabbitMQ	SQS
Throughput	Very High (1M+ msg/s)	High (50K msg/s)	High (auto-scales)
Message Retention	Configurable (days/weeks)	Until consumed	Up to 14 days

Ordering	Per partition	Per queue	FIFO queues only
Delivery	At-least-once	At-least-once or at-most-once	At-least-once (Standard) or exactly-once (FIFO)
Replay	Yes (consumers can re-read)	No (message deleted after ack)	No
Managed Service	Confluent Cloud, AWS MSK	CloudAMQP, AWS MQ	AWS native
Best For	Event streaming, logs, high volume	Task queues, complex routing	Simple queuing, serverless
Operational Burden	High (cluster management)	Medium	None

When to Choose What

- Kafka: You need event streaming, message replay, high throughput, or multiple consumer groups reading the same data.
- RabbitMQ: You need flexible message routing, task distribution, or request-reply patterns.
- SQS: You want zero operational overhead and are already in the AWS ecosystem.

Dead Letter Queues

A dead letter queue (DLQ) is a special queue where messages go when they cannot be processed successfully after multiple attempts. Think of it as the "undeliverable mail" bin at the post office.

```
Dead Letter Queue Flow:

+----------+     +---------+     +----------+
| Producer |---->|  Main   |---->| Consumer |
+----------+     |  Queue  |     +----+-----+
                 +----+----+          |
                      |          Process fails!
                      |          Retry 1... fail
                      |          Retry 2... fail
                      |          Retry 3... fail
                      |               |
                      |               v
```

```
                    +----+----+    +----------+
                    |  Dead   |<---|  Message  |
                    | Letter  |    |  moved    |
                    |  Queue  |    |  to DLQ   |
                    +---------+    +----------+

  Later: Engineer investigates DLQ messages
  - Fix the bug
  - Reprocess the messages
```

Why Use a DLQ?

1. Prevent infinite retry loops: Without a DLQ, a poison message (one that always fails) blocks the queue forever.
2. Preserve failed messages: Instead of losing them, you keep them for debugging.
3. Alerting: Monitor the DLQ. If messages pile up, something is wrong.
4. Reprocessing: After fixing the bug, you can move messages back to the main queue.

Delivery Guarantees

How many times will a consumer receive each message? This is one of the most important properties of a messaging system.

At-Most-Once Delivery

The message is delivered zero or one time. It may be lost, but it will never be delivered twice. The producer sends the message and does not retry if delivery fails.

```
  At-Most-Once:

  Producer --> Broker --> Consumer
               |
  If broker crashes here, message is LOST.
  No retry. No duplicate. Possible data loss.

  Use when: Logging, metrics (losing a few data points is acceptable)
```

At-Least-Once Delivery

The message is delivered one or more times. The producer retries until it gets an acknowledgment. The consumer may receive duplicates.

```
At-Least-Once:

Producer --> Broker --> Consumer
                        |
                    Consumer processes message
                    but ACK is lost
                        |
Broker: "No ACK? Redeliver!"
                        |
                    Consumer gets same message AGAIN

Use when: Order processing, payments (but consumer must be idempotent)
```

Exactly-Once Delivery

The message is delivered exactly one time. This is the hardest guarantee to provide and is often more accurately described as "effectively once" because it typically requires cooperation between the producer, broker, and consumer.

```
Exactly-Once:

Achieved through:
1. Idempotent producers (detect and reject duplicate sends)
2. Transactional consumers (process + ACK atomically)
3. Deduplication at the broker level

This is EXPENSIVE in terms of performance and complexity.

Use when: Financial transactions, inventory updates
```

Comparison

Guarantee	May Lose Messages?	May Duplicate?	Performance	Use Case
At-Most-Once	Yes	No	Fastest	Metrics, logs
At-Least-Once	No	Yes	Fast	Most applications
Exactly-Once	No	No	Slowest	Financial systems

Making At-Least-Once Work Like Exactly-Once

The practical approach: use at-least-once delivery with idempotent consumers. An idempotent consumer produces the same result regardless of how many times it processes the same message.

```
Idempotent Consumer:

Message: "Charge order #1234 for $50"

Non-idempotent consumer:
  First time:  Charge $50  (correct)
  Second time: Charge $50  (DOUBLE CHARGE!)

Idempotent consumer:
  First time:  Check: order #1234 already charged? No. Charge $50.
  Second time: Check: order #1234 already charged? Yes. Skip.
```

The key is to give each message a unique ID and track which IDs have been processed.

Back Pressure

Back pressure is a mechanism for a consumer to signal that it is overwhelmed and cannot handle more messages. Without back pressure, a fast producer can crash a slow consumer.

```
Without Back Pressure:

Producer: 10,000 msg/s ----> Consumer: 1,000 msg/s capacity
                                  |
                           Consumer overwhelmed!
                           Out of memory!
                           CRASH!


With Back Pressure:

Producer: 10,000 msg/s ----> [QUEUE FULL] ----> Consumer: 1,000 msg/s
                                     |
                          Queue signals: "SLOW DOWN"
                                     |
                          Producer: reduces rate or waits
```

Back Pressure Strategies

Strategy	How It Works	Pros	Cons
Queue size limit	Reject new messages when queue is full	Simple	Producer must handle rejections
Rate limiting	Limit producer send rate	Predictable	May be too restrictive
Consumer pull	Consumer requests messages when ready (Kafka model)	Consumer controls pace	Slightly higher latency
Credit-based flow	Consumer grants "credits" to producer (RabbitMQ)	Precise control	More complex

Real-World Use Cases

1. Order Processing

```
E-Commerce Order Flow:

+--------+    +---------------+    +-----------+
| User   |--->| Order Service |--->| Order     |
| places |    | (validates,   |    | Queue     |
| order  |    |  saves order) |    +-----+-----+
+--------+    +---------------+          |
                                    +-----+-----+-----+
                                    |           |     |
                                    v           v     v
                               +---------+ +-------+ +--------+
                               |Payment  | |Email  | |Inventory|
                               |Service  | |Service| |Service  |
                               +---------+ +-------+ +--------+

Benefits:
- User gets instant confirmation
- Payment processing is async (may take seconds)
- If email service is down, messages wait in queue
- Each service scales independently
```

2. Email Sending

```
Email Pipeline:

Any Service           Email Queue              Email Worker
+----------+     +------------------+     +----------------+
| "Send    |     |                  |     | Pick message   |
|  welcome |---->| [email1]         |---->| Render template|
|  email"  |     | [email2]         |     | Call SMTP      |
+----------+     | [email3]         |     | Handle bounce  |
                 | [email4]         |     +----------------+
+----------+     | [email5]         |
| "Send    |---->|                  |     Rate: 100 emails/s
|  receipt"|     +------------------+     (SES/SendGrid limit)
+----------+

Without queue: Services blocked waiting for email to send.
With queue:    Services are instant. Emails sent at controlled rate.
```

3. Event Streaming

```
Event Streaming with Kafka:

+------------+
| Web App    |--+
+------------+  |    +-----------+    +-------------+
                +--->| Kafka     |--->| Analytics   |
+------------+  |    | Topic:    |    | (real-time  |
| Mobile     |--+    | "clicks"  |    |  dashboard) |
| App        |  |    |           |--->+-------------+
+------------+  |    |           |
                |    |           |    +-------------+
+------------+  |    |           |--->| ML Pipeline |
| IoT        |--+    +-----------+    | (training   |
| Devices    |                        |  models)    |
+------------+                        +-------------+

Every click event is stored in Kafka.
Multiple consumers read the SAME events.
Analytics dashboard reads in real-time.
ML pipeline replays historical events for training.
```

4. Microservice Communication

```
Without Queue (Synchronous):

Service A --> Service B --> Service C --> Service D
If C is slow, everything is slow.
If C is down, everything is down.
```

```
With Queue (Asynchronous):

Service A --> [Q1] --> Service B --> [Q2] --> Service C --> [Q3] --> Service D
Each service works independently.
If C is slow, messages buffer in Q2.
If C is down, Q2 holds messages until C recovers.
```

Trade-Offs

Decision	Option A	Option B
Sync vs Async	Synchronous (simple, immediate response)	Async with queue (resilient, decoupled, eventual)
Queue Technology	Kafka (high throughput, replay, complex)	SQS (managed, simple, no replay)
Delivery Guarantee	At-least-once (fast, needs idempotent consumer)	Exactly-once (slow, complex, safer)
Queue per Service	One queue per consumer (isolated, more queues)	Shared queue (simpler, less isolation)
Pull vs Push	Pull (consumer controls pace)	Push (lower latency, risk of overwhelming consumer)

Common Mistakes

1. Using a message queue when a simple function call would work. If two services always run on the same server and need synchronous responses, a queue adds unnecessary complexity.
2. Not making consumers idempotent. With at-least-once delivery (the most common guarantee), your consumer will eventually receive duplicates. If it is not idempotent, you will process orders twice, send duplicate emails, or double-charge customers.
3. Ignoring dead letter queues. Without a DLQ, poison messages that always fail can block your entire queue. Always configure a DLQ and monitor it.

4. Not monitoring queue depth. A growing queue means consumers cannot keep up. This is an early warning sign. Set alerts on queue depth and consumer lag.
5. Choosing Kafka for everything. Kafka is powerful but complex to operate. If you just need a simple task queue with 100 messages per second, SQS or RabbitMQ is a better fit.

Best Practices

1. Design consumers to be idempotent. Use unique message IDs and track processed messages to handle duplicates gracefully.
2. Always configure dead letter queues. After a configurable number of retries (usually 3 to 5), move failed messages to a DLQ for investigation.
3. Monitor queue depth and consumer lag. Alert when queue depth exceeds a threshold or when consumer lag grows. These are signs of scaling problems.
4. Use message schemas. Define the structure of your messages (using Avro, Protobuf, or JSON Schema) so producers and consumers agree on the format. Version your schemas.
5. Set message TTL (Time to Live). Messages should not live in the queue forever. Set an expiration time appropriate for your use case.
6. Handle poison messages gracefully. If a message fails repeatedly, log it, move it to the DLQ, and continue processing other messages.
7. Start with at-least-once delivery. It is the best balance of reliability and performance. Add exactly-once semantics only if your use case truly requires it.
8. Scale consumers horizontally. If the queue is growing, add more consumer instances rather than making one consumer faster.

Quick Summary

A message queue is a buffer between producers (senders) and consumers (receivers) that enables asynchronous communication. Point-to-point queues deliver each message to one consumer; pub/sub topics deliver to all subscribers. Message queues provide decoupling, async processing, buffering, and fault tolerance. Kafka excels at high-throughput event streaming with replay capability. RabbitMQ offers flexible routing for task distribution. SQS provides zero-ops simplicity. Dead letter queues catch messages that fail repeatedly.

At-least-once delivery with idempotent consumers is the practical choice for most systems. Back pressure prevents fast producers from overwhelming slow consumers.

Key Points

- Message queues decouple producers from consumers; neither needs to know about the other
- Point-to-point delivers each message to one consumer; pub/sub delivers to all subscribers
- Queues buffer traffic spikes so downstream services are not overwhelmed
- Kafka is best for event streaming and replay; RabbitMQ for routing; SQS for simplicity
- Dead letter queues prevent poison messages from blocking the queue
- At-least-once delivery with idempotent consumers is the most practical guarantee
- Back pressure prevents fast producers from crashing slow consumers
- Monitor queue depth and consumer lag as key health indicators

Practice Questions

1. Your e-commerce platform processes 1,000 orders per minute normally but spikes to 50,000 during flash sales. Your payment gateway can handle 2,000 transactions per minute. How would you use a message queue to solve this?
2. You are designing a notification system that sends push notifications, emails, and SMS. Should you use one queue or three separate queues? What are the trade-offs?
3. A consumer processes a payment message but crashes before sending an acknowledgment. What happens with at-least-once delivery? How do you prevent the customer from being charged twice?
4. Compare Kafka and SQS for a system that needs to process 10 million events per day from IoT sensors. The data must be available for replay up to 7 days. Which would you choose and why?
5. Your message queue has 500,000 messages backed up and growing. What steps would you take to diagnose and fix the problem?

Exercises

Exercise 1: Design an Order Processing Pipeline

Design a message queue architecture for an online food delivery service. When a customer places an order, the following must happen: validate the order, charge the payment, notify the restaurant, assign a delivery driver, and send the customer a confirmation. Draw the architecture showing queues, producers, and consumers. Identify which steps can happen in parallel.

Exercise 2: Implement Idempotent Processing

You have a consumer that processes payment messages from a queue. Each message contains: `{order_id, amount, currency, timestamp}`. Design the consumer logic to ensure that even if the same message is delivered three times, the payment is only processed once. Describe the data structure you would use to track processed messages and how you would handle the deduplication check.

Exercise 3: Dead Letter Queue Investigation

You wake up to an alert: your dead letter queue has 10,000 messages. Write a step-by-step investigation plan. What would you check first? How would you determine the root cause? How would you reprocess the messages after fixing the issue? What monitoring would you add to prevent this from happening again?

What Is Next?

You now know how services communicate asynchronously through message queues. But how do services communicate synchronously? When a mobile app needs to fetch data from your server, it calls an API. In the next chapter, you will learn how to design clean, scalable, and developer-friendly APIs using REST, GraphQL, and gRPC. You will understand versioning, pagination, authentication, and the principles that make APIs a pleasure to use.

Chapter 10: API Design

What You Will Learn

- REST principles and how to design clean, predictable APIs
- HTTP methods (GET, POST, PUT, PATCH, DELETE) and when to use each
- HTTP status codes and what they communicate to clients
- API versioning strategies and their trade-offs
- Pagination approaches: offset-based vs cursor-based
- Filtering, sorting, and searching patterns
- Rate limiting headers and how they protect your API
- Idempotency and why it matters for reliable APIs
- Authentication and authorization: API keys, OAuth 2.0, JWT
- GraphQL vs REST vs gRPC: when to use each
- OpenAPI (Swagger) for documentation and code generation

Why This Chapter Matters

An API (Application Programming Interface) is the contract between your system and its clients. It is how mobile apps, web frontends, third-party services, and other internal services communicate with your backend.

A well-designed API is a joy to use. Developers can guess how it works without reading documentation. Errors are clear and actionable. It scales without breaking existing clients.

A poorly designed API is a nightmare. Endpoints are inconsistent, error messages are cryptic, and every new feature breaks old integrations. Bad APIs generate endless support tickets and slow down every team that depends on them.

In system design interviews, API design is often the first thing you do: define the interface before designing the internals. Getting it right sets the foundation for everything that follows.

10.1 REST Principles

REST (Representational State Transfer) is an architectural style for designing web APIs. It was defined by Roy Fielding in his 2000 doctoral dissertation. Most web APIs today claim to be RESTful, though few follow every principle strictly.

Core REST Principles

1. Resources, not actions. URLs represent things (nouns), not operations (verbs).

```
GOOD (resource-oriented):
  GET    /users/42          Get user 42
  POST   /users             Create a new user
  PUT    /users/42          Replace user 42
  DELETE /users/42          Delete user 42

BAD (action-oriented):
  GET    /getUser?id=42
  POST   /createUser
  POST   /deleteUser?id=42
  GET    /fetchAllUsers
```

2. Use HTTP methods for operations. The verb is in the method, not the URL.

3. Stateless. Each request contains all the information needed to process it. The server does not store client session state between requests.

4. Uniform interface. All resources follow the same patterns. If you know how `/users` works, you can guess how `/products` works.

5. HATEOAS (Hypermedia As The Engine Of Application State). Responses include links to related resources. In practice, few APIs implement this fully.

Resource Naming Conventions

```
+---------------------------+-------------------------------+
| Convention                | Example                       |
+---------------------------+-------------------------------+
| Use plural nouns          | /users (not /user)            |
| Use lowercase             | /users (not /Users)           |
| Use hyphens for multi-word| /user-profiles (not           |
|                           |  /userProfiles)               |
```

```
| Nest for relationships    | /users/42/orders              |
| Limit nesting depth       | /users/42/orders (not         |
|                           |  /users/42/orders/7/items/3)  |
| Use query params for      | /users?role=admin             |
| filtering                 |                               |
+---------------------------+-------------------------------+
```

10.2 HTTP Methods

Each HTTP method has specific semantics. Using them correctly makes your API predictable.

```
+--------+------------+-------------+-----------+-------------+
| Method | Purpose    | Request Body| Idempotent| Safe (no    |
|        |            |             |           | side effects|
+--------+------------+-------------+-----------+-------------+
| GET    | Read       | No          | Yes       | Yes         |
| POST   | Create     | Yes         | No        | No          |
| PUT    | Replace    | Yes         | Yes       | No          |
| PATCH  | Partial    | Yes         | No*       | No          |
|        | update     |             |           |             |
| DELETE | Delete     | Optional    | Yes       | No          |
+--------+------------+-------------+-----------+-------------+

* PATCH can be made idempotent with careful design
```

When to Use Each

```
GET /users
  --> List all users (with pagination)

GET /users/42
  --> Get user with ID 42

POST /users
  Body: {"name": "Alice", "email": "alice@ex.com"}
  --> Create a new user. Server assigns the ID.

PUT /users/42
  Body: {"name": "Alice Smith", "email": "alice@ex.com", "age": 30}
  --> Replace user 42 entirely. All fields must be present.

PATCH /users/42
  Body: {"name": "Alice Smith"}
  --> Update only the name. Other fields unchanged.
```

```
DELETE /users/42
  --> Delete user 42.
```

PUT vs PATCH

This is a common source of confusion:

```
Current state of user 42:
  {"name": "Alice", "email": "alice@ex.com", "age": 30}

PUT /users/42 with body {"name": "Alice Smith"}
  Result: {"name": "Alice Smith"}
  --> email and age are GONE! PUT replaces the entire resource.

PATCH /users/42 with body {"name": "Alice Smith"}
  Result: {"name": "Alice Smith", "email": "alice@ex.com", "age": 30}
  --> Only name changed. Other fields preserved.
```

10.3 HTTP Status Codes

Status codes tell the client what happened. Using them correctly eliminates ambiguity.

The Five Categories

```
1xx - Informational (rarely used in APIs)
2xx - Success
3xx - Redirection
4xx - Client Error (the client did something wrong)
5xx - Server Error (the server failed)
```

Essential Status Codes for APIs

```
SUCCESS CODES:
+------+---------------------+----------------------------------------+
| Code | Name                | When to Use                            |
+------+---------------------+----------------------------------------+
| 200  | OK                  | Successful GET, PUT, PATCH, DELETE     |
| 201  | Created             | Successful POST that created a         |
```

```
|      |                      | resource. Include Location header.    |
| 204  | No Content           | Successful DELETE with no response    |
|      |                      | body                                  |
+------+----------------------+---------------------------------------+

CLIENT ERROR CODES:
+------+----------------------+---------------------------------------+
| 400  | Bad Request          | Malformed request, invalid JSON,      |
|      |                      | missing required fields                |
| 401  | Unauthorized         | No authentication provided or         |
|      |                      | invalid credentials                    |
| 403  | Forbidden            | Authenticated but not authorized      |
|      |                      | for this resource                      |
| 404  | Not Found            | Resource does not exist                |
| 405  | Method Not Allowed   | POST to a read-only endpoint           |
| 409  | Conflict             | Resource state conflict (e.g.,         |
|      |                      | duplicate email)                       |
| 422  | Unprocessable Entity| Valid JSON but semantically invalid    |
|      |                      | (e.g., age = -5)                       |
| 429  | Too Many Requests    | Rate limit exceeded                    |
+------+----------------------+---------------------------------------+

SERVER ERROR CODES:
+------+----------------------+---------------------------------------+
| 500  | Internal Server Err | Unexpected server failure              |
| 502  | Bad Gateway          | Upstream service returned bad response |
| 503  | Service Unavailable | Server is overloaded or in maintenance |
| 504  | Gateway Timeout      | Upstream service timed out             |
+------+----------------------+---------------------------------------+
```

Error Response Format

Always return structured error responses. Never return a bare string or an empty body.

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/json

{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Request validation failed",
    "details": [
      {
        "field": "email",
        "message": "Must be a valid email address"
      },
      {
        "field": "age",
        "message": "Must be a positive integer"
      }
    ]
```

```
  }
}
```

10.4 API Versioning

APIs evolve. You need to make breaking changes without breaking existing clients. Versioning gives you that safety.

Versioning Strategies

1. URL Path Versioning (most common)

```
GET /v1/users/42
GET /v2/users/42

Pros: Simple, visible, easy to route
Cons: Not truly RESTful (version is not a resource)
Used by: Twitter, Stripe, GitHub (partially)
```

2. Query Parameter Versioning

```
GET /users/42?version=1
GET /users/42?version=2

Pros: Clean URLs, optional parameter
Cons: Easy to forget, harder to route
Used by: Google APIs (some)
```

3. Header Versioning

```
GET /users/42
Accept: application/vnd.myapi.v1+json

GET /users/42
Accept: application/vnd.myapi.v2+json

Pros: Clean URLs, proper content negotiation
Cons: Not visible in URL, harder to test in browser
Used by: GitHub API
```

4. No Versioning (evolve carefully)

```
Add new fields without removing old ones.
Make new fields optional.
Deprecate old fields gradually.

Pros: Simplest, no version management
Cons: Limits the changes you can make
Used by: Slack API (partially)
```

Versioning Best Practices

```
+-------------------------------------+----------------------------+
| Practice                            | Why                        |
+-------------------------------------+----------------------------+
| Use URL versioning (v1, v2) unless  | Simplest for most teams    |
| you have a strong reason not to     |                            |
+-------------------------------------+----------------------------+
| Support at least 2 versions         | Give clients time to       |
| simultaneously                      | migrate                    |
+-------------------------------------+----------------------------+
| Announce deprecation early          | 6-12 months notice is      |
| (sunset headers)                    | standard                   |
+-------------------------------------+----------------------------+
| Version major changes only          | Adding a new optional field|
| (breaking changes)                  | does not need a new version|
+-------------------------------------+----------------------------+
```

10.5 Pagination

When a resource has many items (thousands of users, millions of orders), you cannot return them all in one response. Pagination breaks the result into manageable pages.

Offset-Based Pagination

The simplest approach. Client specifies `offset` (skip N items) and `limit` (return M items).

```
OFFSET-BASED PAGINATION:

  GET /users?offset=0&limit=20    --> Items 1-20
  GET /users?offset=20&limit=20   --> Items 21-40
  GET /users?offset=40&limit=20   --> Items 41-60
```

```
  Response:
  {
    "data": [...],
    "pagination": {
      "offset": 20,
      "limit": 20,
      "total": 1000
    }
  }

  Database query: SELECT * FROM users ORDER BY id LIMIT 20 OFFSET 20;
```

Pros:

- Simple to implement
- Client can jump to any page
- Total count is easy to provide

Cons:

- Slow for large offsets (OFFSET 1000000 must scan 1 million rows)
- Inconsistent results if data changes between pages (items can be skipped or duplicated)

Cursor-Based Pagination

Client provides a cursor (usually the ID or timestamp of the last item seen). The server returns items after that cursor.

```
CURSOR-BASED PAGINATION:

  GET /users?limit=20
  --> Returns items 1-20, cursor="user_20"

  GET /users?limit=20&after=user_20
  --> Returns items 21-40, cursor="user_40"

  GET /users?limit=20&after=user_40
  --> Returns items 41-60, cursor="user_60"

  Response:
  {
    "data": [...],
    "pagination": {
      "next_cursor": "eyJpZCI6IDQwfQ==",
      "has_more": true
    }
  }

  Database query:
```

```
SELECT * FROM users WHERE id > 40 ORDER BY id LIMIT 20;
(Uses index, fast regardless of position!)
```

Pros:

- Consistent performance regardless of position (no large OFFSET)
- Stable results even if data changes between pages
- Better for infinite scroll UIs

Cons:

- Cannot jump to arbitrary pages ("show me page 50")
- More complex to implement
- Cursor must encode the sort order

Comparison

```
+--------------------+-----------------+------------------+
| Feature            | Offset-Based    | Cursor-Based     |
+--------------------+-----------------+------------------+
| Performance at     | Degrades with   | Constant         |
| large offsets      | offset size     |                  |
+--------------------+-----------------+------------------+
| Jump to page N     | Yes             | No               |
+--------------------+-----------------+------------------+
| Data consistency   | Items can shift | Stable           |
| between pages      |                 |                  |
+--------------------+-----------------+------------------+
| Implementation     | Simple          | More complex     |
+--------------------+-----------------+------------------+
| Best for           | Small datasets, | Large datasets,  |
|                    | admin panels    | feeds, timelines |
+--------------------+-----------------+------------------+
```

10.6 Filtering, Sorting, and Searching

Filtering

Allow clients to narrow results using query parameters:

```
GET /products?category=electronics
GET /products?category=electronics&price_min=100&price_max=500
GET /products?status=active&created_after=2024-01-01
GET /orders?user_id=42&status=shipped
```

Sorting

Allow clients to control the order of results:

```
GET /products?sort=price           --> Ascending by price
GET /products?sort=-price          --> Descending by price (minus prefix)
GET /products?sort=category,-price --> By category asc, then price desc
```

Searching

For full-text search, use a dedicated query parameter:

```
GET /products?q=wireless+keyboard
GET /users?search=alice
```

Combined Example

```
GET /products?category=electronics&price_min=50&sort=-rating&limit=20

Translation: "Give me the top 20 electronics over $50,
              sorted by rating (highest first)"
```

10.7 Rate Limiting Headers

Rate limiting protects your API from abuse (see Chapter 11 for algorithms). The API communicates rate limits to clients through HTTP headers.

```
RATE LIMIT HEADERS:

HTTP/1.1 200 OK
X-RateLimit-Limit: 100          <-- Max requests allowed per window
```

```
X-RateLimit-Remaining: 73       <-- Requests remaining in current window
X-RateLimit-Reset: 1672531200   <-- Unix timestamp when window resets

When exceeded:
HTTP/1.1 429 Too Many Requests
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1672531200
Retry-After: 30                 <-- Seconds to wait before retrying

{
  "error": {
    "code": "RATE_LIMIT_EXCEEDED",
    "message": "Rate limit exceeded. Try again in 30 seconds."
  }
}
```

Rate Limit Tiers

```
+------------------+---------+---------+--------------------+
| Tier             | Limit   | Window  | Who                |
+------------------+---------+---------+--------------------+
| Free             | 60/hr   | 1 hour  | Unauthenticated    |
| Basic            | 1000/hr | 1 hour  | Free tier users    |
| Pro              | 10000/hr| 1 hour  | Paid users         |
| Enterprise       | Custom  | Custom  | Contract customers |
+------------------+---------+---------+--------------------+
```

10.8 Idempotency

An operation is idempotent if performing it multiple times has the same effect as performing it once.

Why Idempotency Matters

Network failures happen. When a client sends a request and does not get a response, it does not know if the server processed it or not. The safe thing is to retry. But if the operation is not idempotent, retrying can cause problems.

```
WITHOUT IDEMPOTENCY:
```

```
  Client: POST /payments  {"amount": 100, "to": "Bob"}
  Server: Processes payment, but response is lost in network
  Client: No response received. Retry!
  Client: POST /payments  {"amount": 100, "to": "Bob"}
  Server: Processes payment AGAIN.

  Result: Bob received $200 instead of $100!


WITH IDEMPOTENCY:

  Client: POST /payments  {"amount": 100, "to": "Bob"}
          Header: Idempotency-Key: "abc-123"
  Server: Processes payment, stores result with key "abc-123"
          Response lost in network
  Client: No response received. Retry!
  Client: POST /payments  {"amount": 100, "to": "Bob"}
          Header: Idempotency-Key: "abc-123"
  Server: Sees key "abc-123" already processed.
          Returns the original result without processing again.

  Result: Bob received exactly $100. Correct!
```

Idempotency by HTTP Method

```
+--------+-----------+--------------------------------------+
| Method | Idempotent| Explanation                          |
+--------+-----------+--------------------------------------+
| GET    | Yes       | Reading does not change state        |
| PUT    | Yes       | Replacing with same data = same result|
| DELETE | Yes       | Deleting twice = still deleted       |
| POST   | No        | Creating twice = two resources       |
| PATCH  | Maybe     | Depends on the operation             |
+--------+-----------+--------------------------------------+

POST is the dangerous one. Use idempotency keys for
POST requests that must not be duplicated (payments,
order placements, account creation).
```

Implementing Idempotency Keys

```
IDEMPOTENCY KEY FLOW:

  +--------+                    +--------+           +---------+
  | Client |                    | Server |           | ID Key  |
  |        |                    |        |           | Store   |
  +---+----+                    +---+----+           +----+----+
```

```
      |                              |                     |
      | POST /orders                 |                     |
      | Idempotency-Key: xyz-789     |                     |
      |----------------------------->|                     |
      |                              | Check key "xyz-789" |
      |                              |-------------------->|
      |                              |   Not found         |
      |                              |<--------------------|
      |                              |                     |
      |                              | Process order       |
      |                              | Store result        |
      |                              |  with key "xyz-789" |
      |                              |-------------------->|
      |   201 Created                |                     |
      |<-----------------------------|                     |
      |                              |                     |
      | RETRY (same key)             |                     |
      | POST /orders                 |                     |
      | Idempotency-Key: xyz-789     |                     |
      |----------------------------->|                     |
      |                              | Check key "xyz-789" |
      |                              |-------------------->|
      |                              |   Found! Return     |
      |                              |   stored result     |
      |                              |<--------------------|
      |   201 Created (same result)  |                     |
      |<-----------------------------|                     |
```

10.9 Authentication and Authorization

API Keys

The simplest authentication method. The client includes a secret key with each request.

```
GET /users
Authorization: ApiKey sk_live_abc123def456

Or in a custom header:
X-API-Key: sk_live_abc123def456
```

Pros: Simple to implement and use Cons: If leaked, anyone can use it. No user context (key represents an app, not a user).

Best for: Server-to-server communication, public APIs with rate limiting

OAuth 2.0

A protocol that lets users grant third-party apps limited access to their resources without sharing passwords.

```
OAUTH 2.0 FLOW (Authorization Code):

  +------+     +--------+     +-----------+     +----------+
  | User |     | Client |     | Auth      |     | Resource |
  |      |     | (App)  |     | Server    |     | Server   |
  +--+---+     +---+----+     +-----+-----+     +----+-----+
     |            |                |                  |
     | 1. Click   |                |                  |
     | "Login with |               |                  |
     |  Google"   |                |                  |
     |------------>|               |                  |
     |            |                |                  |
     |            | 2. Redirect to |                  |
     |            |    auth server |                  |
     |<------------|               |                  |
     |            |                |                  |
     | 3. User logs in and         |                 |
     |    approves access          |                 |
     |---------------------------->|                |
     |            |                |                  |
     |            | 4. Auth code   |                  |
     |            |<---------------|                  |
     |            |                |                  |
     |            | 5. Exchange    |                  |
     |            |  code for token|                  |
     |            |--------------->|                  |
     |            |                |                  |
     |            | 6. Access token|                  |
     |            |<---------------|                  |
     |            |                |                  |
     |            | 7. Use token   |                  |
     |            |  to access API |                  |
     |            |---------------------------------->|
     |            |                |                  |
     |            | 8. Protected   |                  |
     |            |    resource    |                  |
     |            |<----------------------------------|
```

Best for: When users authorize third-party access to their data (social logins, API integrations)

JWT (JSON Web Tokens)

A compact, self-contained token that encodes user information and is signed to prevent tampering.

```
JWT STRUCTURE:

  Header.Payload.Signature

  Header:   {"alg": "HS256", "typ": "JWT"}
  Payload:  {"user_id": 42, "role": "admin", "exp": 1672531200}
  Signature: HMAC-SHA256(header + "." + payload, secret_key)

  Encoded:
  eyJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjo0Mn0.abc123signature

  Usage:
  GET /users
  Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...


  +--------+                      +--------+
  | Client |                      | Server |
  +---+----+                      +---+----+
      |                               |
      | POST /login                   |
      | {"email":"a@b.c","pass":"x"} |
      |------------------------------>|
      |                               | Verify credentials
      |                               | Generate JWT
      |   {"token": "eyJ..."}       |
      |<------------------------------|
      |                               |
      | GET /users                    |
      | Authorization: Bearer eyJ... |
      |------------------------------>|
      |                               | Verify JWT signature
      |                               | Extract user_id from payload
      |                               | (No database lookup needed!)
      |   200 OK                      |
      |<------------------------------|
```

Pros: Stateless (server does not need to store sessions), contains user info Cons: Cannot be revoked before expiry (unless you add a blacklist), size grows with claims

Comparison

```
+------------------+----------+-----------+----------+
| Feature          | API Keys | OAuth 2.0 | JWT      |
+------------------+----------+-----------+----------+
| Complexity       | Low      | High      | Medium   |
| User context     | No       | Yes       | Yes      |
| Revocable        | Yes      | Yes       | Hard*    |
| Stateless        | Yes      | No**      | Yes      |
| Best for         | Server-  | Third-    | Stateless|
|                  | to-server| party     | auth in  |
```

```
|                  |          | access    | your APIs|
+------------------+----------+-----------+----------+

*  JWT requires blacklist or short expiry + refresh tokens
** OAuth requires storing tokens server-side
```

10.10 GraphQL vs REST vs gRPC

REST is not the only API style. Here are the three major approaches and when to choose each.

REST

```
REST API:

  GET /users/42           --> Get user
  GET /users/42/orders    --> Get user's orders
  GET /orders/7/items     --> Get order items

  Problems:
  1. Over-fetching: GET /users/42 returns ALL fields,
     even if you only need the name.

  2. Under-fetching: To show a user's profile page, you
     need 3 separate requests (user, orders, reviews).

  3. Multiple round trips:
     Client --> GET /users/42
     Client --> GET /users/42/orders
     Client --> GET /users/42/reviews
     (3 network round trips!)
```

GraphQL

GraphQL lets the client specify exactly what data it needs in a single request.

```
GRAPHQL:

  Single request, client specifies the shape:

  POST /graphql
  {
```

```
    query {
      user(id: 42) {
        name
        email
        orders(last: 5) {
          id
          total
          items {
            name
            price
          }
        }
      }
    }
  }

  Response matches the query shape exactly:
  {
    "data": {
      "user": {
        "name": "Alice",
        "email": "alice@ex.com",
        "orders": [
          {
            "id": 1,
            "total": 129.99,
            "items": [
              {"name": "Laptop Stand", "price": 49.99},
              {"name": "USB Hub", "price": 29.99}
            ]
          }
        ]
      }
    }
  }

  One request, exactly the data needed. No over-fetching,
  no under-fetching.
```

gRPC

gRPC uses Protocol Buffers (protobuf) for binary serialization and HTTP/2 for transport. It is much faster than JSON-based REST.

```
gRPC:

  1. Define service in .proto file:

     service UserService {
       rpc GetUser (UserRequest) returns (UserResponse);
       rpc ListUsers (ListRequest) returns (stream UserResponse);
```

```
    }

    message UserRequest {
      int32 id = 1;
    }

    message UserResponse {
      int32 id = 1;
      string name = 2;
      string email = 3;
    }

  2. Generate client and server code from .proto
  3. Call methods like local functions:

     user = client.GetUser(UserRequest(id=42))
     print(user.name)  // "Alice"

  Features:
  - Binary protocol (smaller, faster than JSON)
  - HTTP/2 (multiplexing, streaming)
  - Bi-directional streaming
  - Strong typing (code generation from .proto)
```

Comparison

```
+-------------------+-------------+-------------+-------------+
| Feature           | REST        | GraphQL     | gRPC        |
+-------------------+-------------+-------------+-------------+
| Protocol          | HTTP/1.1    | HTTP/1.1    | HTTP/2      |
| Data format       | JSON        | JSON        | Protobuf    |
|                   |             |             | (binary)    |
+-------------------+-------------+-------------+-------------+
| Fetching          | Fixed       | Client      | Fixed       |
|                   | response    | specifies   | response    |
|                   | shape       | shape       | shape       |
+-------------------+-------------+-------------+-------------+
| Performance       | Good        | Good        | Excellent   |
+-------------------+-------------+-------------+-------------+
| Browser support   | Native      | Via library | Limited*    |
+-------------------+-------------+-------------+-------------+
| Streaming         | Limited     | Subscriptions| Native     |
+-------------------+-------------+-------------+-------------+
| Learning curve    | Low         | Medium      | Medium-High |
+-------------------+-------------+-------------+-------------+
| Caching           | HTTP caching| Complex     | Not built-in|
|                   | built-in    | (POST-based)|             |
+-------------------+-------------+-------------+-------------+
| Best for          | Public APIs,| Mobile apps,| Microservice|
|                   | CRUD apps   | complex UIs | to micro-   |
|                   |             |             | service     |
```

```
+-------------------+-------------+-------------+-------------+

* gRPC-Web adds browser support, but it is not native
```

When to Choose Which

```
Choose REST when:
  - Building a public API consumed by third parties
  - Your resources map cleanly to CRUD operations
  - You want maximum compatibility (any HTTP client works)
  - HTTP caching is important

Choose GraphQL when:
  - Your frontend needs flexible, nested data
  - Mobile apps need to minimize data transfer
  - You have many client types with different data needs
  - You want to avoid managing many endpoint versions

Choose gRPC when:
  - You have internal service-to-service communication
  - You need high performance and low latency
  - You need bi-directional streaming
  - You want strong typing and code generation
```

10.11 OpenAPI (Swagger)

OpenAPI is a specification for describing REST APIs. It produces machine-readable API documentation that can generate client libraries, server stubs, and interactive documentation.

```
OPENAPI EXAMPLE (YAML):

  openapi: "3.0.0"
  info:
    title: User API
    version: "1.0"
  paths:
    /users/{id}:
      get:
        summary: Get a user by ID
        parameters:
          - name: id
            in: path
            required: true
```

```
            schema:
              type: integer
        responses:
          "200":
            description: User found
            content:
              application/json:
                schema:
                  type: object
                  properties:
                    id:
                      type: integer
                    name:
                      type: string
                    email:
                      type: string
          "404":
            description: User not found


OPENAPI TOOLING:

  +------------------+     +-----------------------+
  | OpenAPI Spec     |---->| Swagger UI            |
  | (YAML/JSON)      |     | (interactive docs)    |
  +------------------+     +-----------------------+
          |
          +--------------->+-----------------------+
          |                | Client SDK Generator  |
          |                | (Python, JS, Java...) |
          |                +-----------------------+
          |
          +--------------->+-----------------------+
                           | Server Stub Generator |
                           | (Spring, Express...)  |
                           +-----------------------+
```

Common Mistakes

1. Using verbs in URLs. `/getUser/42` should be `GET /users/42`. The HTTP method is the verb.
2. Returning 200 for errors. If something failed, use a 4xx or 5xx status code. Returning 200 with `{"error": "not found"}` breaks HTTP conventions and caching.
3. Not paginating list endpoints. Returning 10,000 items in one response is slow and wasteful. Always paginate.

4. Breaking changes without versioning. Removing a field or changing a response structure breaks existing clients. Version your API or use additive changes only.
5. Inconsistent naming. If one endpoint uses `userId` and another uses `user_id`, your API is confusing. Pick one convention and stick to it.
6. No rate limiting. Without rate limits, a single misbehaving client can take down your API. Always implement rate limiting.
7. Overly nested URLs. `/companies/1/departments/2/teams/3/members/4` is hard to use. Flatten where possible: `/members/4?team_id=3`.
8. Not using idempotency keys for critical operations. Payment processing, order creation, and other non-idempotent operations need idempotency keys.

Best Practices

1. Use consistent, predictable naming. Plural nouns, lowercase, hyphens. Follow one convention everywhere.
2. Return meaningful error responses. Include an error code, human-readable message, and field-level details.
3. Always paginate list endpoints. Default to a reasonable page size (20-50 items) with a maximum limit.
4. Use cursor-based pagination for large datasets. Offset-based is fine for small, admin-facing datasets.
5. Version your API from day one. Even if you only have `v1`, it sets the pattern.
6. Document your API with OpenAPI. Auto-generated documentation stays in sync with your code.
7. Use HTTPS everywhere. Never serve an API over plain HTTP.
8. Implement idempotency keys for POST requests that create resources. Especially for financial operations.
9. Set rate limits and communicate them via headers. Clients should know their limits before hitting them.
10. Design for backwards compatibility. Add new fields, do not remove old ones. Mark deprecated fields clearly.

Quick Summary

API design is about creating clear, consistent, predictable interfaces. REST uses HTTP methods (GET, POST, PUT, PATCH, DELETE) on resource-oriented URLs. Status codes communicate outcomes (2xx success, 4xx client error, 5xx server error). APIs should be versioned (URL path is simplest), paginated (cursor-based for large datasets), and protected with rate limiting. Idempotency keys prevent duplicate operations from network retries. Authentication options include API keys (simple), OAuth 2.0 (third-party access), and JWT (stateless auth). GraphQL lets clients request exactly the data they need. gRPC uses binary protobuf for high-performance service-to-service communication. OpenAPI provides machine-readable documentation. The best API is one that developers can use correctly without reading documentation.

Key Points

- Use resource-oriented URLs with HTTP methods as verbs: GET /users/42, not GET /getUser?id=42
- Return appropriate status codes: 201 for creation, 404 for not found, 429 for rate limiting
- Version your API from the start -- URL path versioning (/v1/) is the simplest approach
- Use cursor-based pagination for large datasets; offset-based for small ones
- Implement idempotency keys for non-idempotent operations to handle retries safely
- Choose REST for public APIs, GraphQL for flexible client needs, gRPC for internal services
- Always rate limit your API and communicate limits through response headers
- Document with OpenAPI to keep documentation in sync with implementation

---

Practice Questions

1. You are designing an API for a ride-sharing app. Define the REST endpoints for: requesting a ride, tracking a ride in progress, completing a ride, and rating a driver. Include HTTP methods, URLs, request bodies, and response codes.
2. A client reports that they were charged twice for the same order. Explain how idempotency keys could have prevented this. Design the implementation, including key storage, expiration, and race condition handling.
3. Your API serves both a mobile app (needs minimal data, slow network) and an admin dashboard (needs full data, fast network). Compare how you would handle this with

REST (multiple endpoints or field selection) vs GraphQL. Which approach would you recommend and why?

4. You have an endpoint `GET /products` that returns 2 million products. A client complains that fetching page 50,000 is extremely slow. Explain why offset-based pagination causes this and how cursor-based pagination solves it.
5. Compare API keys, OAuth 2.0, and JWT for a SaaS platform that needs to support both first-party mobile apps and third-party developer integrations. Which authentication method(s) would you use for each, and why?

Exercises

Exercise 1: Design a REST API Design a complete REST API for a blog platform. Define endpoints for: users, posts, comments, and tags. For each endpoint, specify: HTTP method, URL, request body (if applicable), response body, and status codes. Include pagination, filtering by author, and sorting by date.

Exercise 2: Idempotency Implementation Write pseudocode for an idempotency middleware that: (a) checks for an Idempotency-Key header on POST requests, (b) returns the cached result if the key was seen before, (c) processes the request and stores the result if the key is new, (d) handles race conditions when two requests with the same key arrive simultaneously.

Exercise 3: API Comparison Take one feature from a real API you use (e.g., creating a tweet, placing an order). Design the interface three ways: REST, GraphQL, and gRPC (using proto definitions). Compare them on: number of round trips, data transferred, type safety, and ease of implementation.

What Is Next?

You now know how to design clean, reliable APIs. But how do you protect those APIs from being overwhelmed by too many requests? In Chapter 11, you will learn about rate limiting -- the algorithms, strategies, and implementations that keep your API healthy under heavy load. Rate limiting is one of the most common system design topics in interviews and a critical component of any production API.

Chapter 11: Rate Limiting

What You Will Learn

- Why rate limiting is essential for protecting any production API
- Five rate limiting algorithms: token bucket, leaky bucket, fixed window counter, sliding window log, and sliding window counter
- How each algorithm works, with step-by-step ASCII diagrams
- The trade-offs between accuracy, memory, and complexity
- Implementation levels: application, API gateway, load balancer, CDN
- HTTP 429 responses and rate limit headers
- Distributed rate limiting across multiple servers
- How to design rate limits that protect your system without frustrating legitimate users

Why This Chapter Matters

Without rate limiting, a single user or bot can overwhelm your entire system. One misbehaving client sending 10,000 requests per second can:

- Crash your servers
- Exhaust your database connections
- Spike your cloud bill
- Deny service to all other users
- Enable brute-force attacks on login endpoints
- Scrape your entire dataset

Rate limiting is not optional. Every production API needs it. And in system design interviews, rate limiting is one of the most frequently asked design questions because it combines algorithms, distributed systems, and practical engineering judgment.

11.1 What Is Rate Limiting?

Rate limiting is the practice of controlling the rate at which clients can send requests to your system. When a client exceeds the allowed rate, their requests are rejected (or throttled) until the rate drops back to an acceptable level.

The Amusement Park Analogy

Imagine an amusement park ride that can safely handle 20 people per cycle, with a cycle every 5 minutes.

- Without rate limiting: 200 people rush to the ride at once. The line is chaotic, people get pushed, and the ride operator is overwhelmed.
- With rate limiting: A gate lets exactly 20 people through every 5 minutes. Others wait in an orderly queue. Everyone gets a ride, safely and predictably.

```
WITHOUT RATE LIMITING:

  Requests:  ||||||||||||||||||||||||||||||||||||  (all at once)
                          |
                          v
                     +----------+
                     |  Server  |  <-- OVERWHELMED!
                     +----------+


WITH RATE LIMITING:

  Allowed:   |||| |||| |||| |||| |||| ||||  (steady flow)
  Rejected:                     XXXXX         (excess dropped)
                          |
                          v
                     +----------+
                     |  Server  |  <-- Happy and healthy
                     +----------+
```

Types of Rate Limits

```
+--------------------+-------------------------------------------+
| Type               | Example                                   |
+--------------------+-------------------------------------------+
| User-based         | User X can make 100 requests per minute   |
| IP-based           | IP address can make 50 requests per minute|
| API key-based      | API key can make 1000 requests per hour   |
| Endpoint-based     | /api/login allows 5 attempts per minute   |
| Global             | Server accepts max 10,000 requests/sec    |
```

```
+--------------------+-------------------------------------------+
```

11.2 Token Bucket Algorithm

The token bucket is one of the most widely used rate limiting algorithms. It is simple, memory-efficient, and allows short bursts of traffic.

How It Works

Imagine a bucket that holds tokens. Tokens are added at a fixed rate. Each request takes one token from the bucket. If the bucket is empty, the request is rejected.

```
TOKEN BUCKET:

  Bucket capacity: 4 tokens
  Refill rate: 1 token per second

  Time    Bucket      Request     Result
  ----    ------      -------     ------
  t=0     [****]      --          4 tokens (full)
  t=0     [***_]      Request 1   ALLOWED (3 tokens left)
  t=0     [**__]      Request 2   ALLOWED (2 tokens left)
  t=0     [*___]      Request 3   ALLOWED (1 token left)
  t=0     [____]      Request 4   ALLOWED (0 tokens left)
  t=0     [____]      Request 5   REJECTED (empty!)
  t=1     [*___]      --          1 token added (refill)
  t=1     [____]      Request 6   ALLOWED (0 tokens left)
  t=2     [*___]      --          1 token added
  t=3     [**__]      --          1 token added
  t=4     [***_]      --          1 token added


ASCII DIAGRAM:

           Tokens added at fixed rate
                    |
                    v
           +-------+-------+
           |               |
           |    BUCKET     |  Capacity: max tokens
           |   [* * * *]   |
           |               |
           +-------+-------+
                   |
                   v
```

```
             Take 1 token
             per request
                    |
            +-------+-------+
            |               |
            | Tokens > 0 ?  |
            |               |
            +---+-------+---+
                |       |
               YES      NO
                |       |
                v       v
            ALLOW    REJECT
            request  request
```

Key Properties

- Burst friendly: A full bucket allows a burst of requests up to the bucket capacity
- Smooth average: Over time, the average rate equals the refill rate
- Memory efficient: Only needs to store token count and last refill timestamp (2 values per user)

Real-World Usage

Amazon API Gateway and Stripe use token bucket. It is the default choice for most rate limiters.

```
PARAMETERS:
  bucket_size = 10       (allows bursts of up to 10)
  refill_rate = 2/sec    (sustained rate of 2 requests/second)

  Scenario 1: Steady traffic (2 req/sec)
  --> All requests allowed, bucket stays near full

  Scenario 2: Burst of 10 requests, then silence
  --> All 10 allowed (bucket drains)
  --> Next request must wait for refill

  Scenario 3: 5 req/sec sustained
  --> First 10 allowed (burst)
  --> Then only 2/sec allowed (refill rate)
  --> 3/sec rejected
```

11.3 Leaky Bucket Algorithm

The leaky bucket processes requests at a constant rate, like water leaking from a bucket at a fixed drip rate. Requests that arrive when the bucket is full are discarded.

How It Works

```
LEAKY BUCKET:

  Think of a bucket with a hole at the bottom.
  Water (requests) pours in from the top.
  Water leaks out (is processed) at a constant rate.
  If the bucket overflows, water (requests) is discarded.

           Requests pour in
                |
                v
         +------+------+
         |  ~~~~~~~~   |  <-- Bucket (queue)
         |  ~~~~~~~~   |      Capacity: max queue size
         |  ~~~~~~~~   |
         |  ~~~~~~~~   |
         +------+------+
                |
                | Leak at constant rate
                v
         Process requests
         (fixed rate output)


  TIMELINE:

  Input rate:     ||||  ||    ||||||||||  ||
  (variable)

  Queue:          [####][####][##########][##]
                                ^^^^^^^^
                                overflow --> REJECTED

  Output rate:    | | | | | | | | | | | | | |
  (constant)     (steady, predictable processing)
```

Token Bucket vs Leaky Bucket

```
+--------------------+------------------+------------------+
| Feature            | Token Bucket     | Leaky Bucket     |
+--------------------+------------------+------------------+
| Burst handling     | Allows bursts    | Smooths out      |
```

```
|                    | (up to bucket    | bursts into      |
|                    | capacity)        | steady flow      |
+--------------------+------------------+------------------+
| Output rate        | Variable         | Constant         |
|                    | (bursty)         | (steady)         |
+--------------------+------------------+------------------+
| When bucket is full| Requests rejected| Requests queued  |
|                    | immediately      | then rejected    |
|                    |                  | if queue is full |
+--------------------+------------------+------------------+
| Implementation     | Counter + timer  | Queue + timer    |
+--------------------+------------------+------------------+
| Memory             | Very low         | Higher (stores   |
|                    | (2 values)       | queued requests)  |
+--------------------+------------------+------------------+
| Best for           | APIs where       | Traffic shaping, |
|                    | bursts are OK    | steady processing|
+--------------------+------------------+------------------+
```

Real-World Usage

Leaky bucket is used in network traffic shaping (controlling bandwidth). Shopify uses a variation for their API rate limiter.

11.4 Fixed Window Counter

The simplest counting algorithm. Divide time into fixed windows (e.g., 1-minute intervals) and count requests in each window. Reject when the count exceeds the limit.

How It Works

```
FIXED WINDOW COUNTER:

  Limit: 5 requests per minute

  Window 1          Window 2          Window 3
  [00:00 - 00:59]   [01:00 - 01:59]   [02:00 - 02:59]

  Timeline:
  00:00  Req 1 --> Count=1  ALLOWED
  00:15  Req 2 --> Count=2  ALLOWED
  00:30  Req 3 --> Count=3  ALLOWED
  00:45  Req 4 --> Count=4  ALLOWED
```

```
  00:50  Req 5 --> Count=5  ALLOWED
  00:55  Req 6 --> Count=6  REJECTED (limit=5!)
  01:00  -- New window, count resets to 0 --
  01:05  Req 7 --> Count=1  ALLOWED


  +--------+--------+--------+
  | Window1| Window2| Window3|
  |Count: 5|Count: 3|Count: 0|
  +--------+--------+--------+
  ^        ^        ^
  00:00    01:00    02:00
```

The Boundary Problem

Fixed windows have a critical flaw: traffic can spike at window boundaries.

```
BOUNDARY PROBLEM:

  Limit: 5 requests per minute

  Window 1                    Window 2
  [00:00 -------- 00:59]     [01:00 -------- 01:59]

  00:50  Req 1  ALLOWED       01:00  Req 6  ALLOWED
  00:51  Req 2  ALLOWED       01:01  Req 7  ALLOWED
  00:52  Req 3  ALLOWED       01:02  Req 8  ALLOWED
  00:53  Req 4  ALLOWED       01:03  Req 9  ALLOWED
  00:54  Req 5  ALLOWED       01:04  Req 10 ALLOWED

  10 requests in 14 seconds!
  (5 at end of Window 1 + 5 at start of Window 2)

  |      Window 1      |      Window 2      |
  |                |||||*****|               |
  |                     ^                    |
  |              10 requests in              |
  |              this 14-second span!        |
  |                                          |

  The rate limit is 5/minute, but 10 requests
  passed in a burst at the boundary.
```

Key Properties

- Simple: Just a counter per window (1 value per user per window)
- Memory efficient: Very low memory
- Boundary problem: Can allow 2x the limit at window boundaries

- Used when: Approximate limiting is acceptable

11.5 Sliding Window Log

Fixes the boundary problem by tracking the exact timestamp of every request.

How It Works

```
SLIDING WINDOW LOG:

  Limit: 5 requests per minute
  Window: last 60 seconds (slides with current time)

  Keep a log of all request timestamps:

  Time    Log                          Count  Result
  ----    ---                          -----  ------
  00:10   [00:10]                      1      ALLOWED
  00:25   [00:10, 00:25]               2      ALLOWED
  00:40   [00:10, 00:25, 00:40]        3      ALLOWED
  00:50   [00:10, 00:25, 00:40, 00:50] 4      ALLOWED
  00:55   [00:10, 00:25, 00:40,         5     ALLOWED
           00:50, 00:55]
  01:05   [00:25, 00:40, 00:50,         4     ALLOWED
           00:55, 01:05]
           ^-- 00:10 expired (more than 60 seconds ago)

  At any moment, count = timestamps within last 60 seconds.


  SLIDING WINDOW VISUALIZATION:

  Timeline:  00:00    00:20    00:40    01:00    01:20
             |--------|--------|--------|--------|
                 *        *       **  *
                00:10   00:25  00:40 00:50
                               00:55

  At t=00:55:
     Window: [00:55 - 60s] = [23:55 to 00:55]
             --> All 5 timestamps in window
             --> Count = 5, limit reached

  At t=01:05:
     Window: [01:05 - 60s] = [00:05 to 01:05]
             --> 00:10 expired!
             --> Count = 4, request allowed
```

Key Properties

- Accurate: No boundary problem, exact rate tracking
- Memory intensive: Must store every request timestamp (could be thousands per user)
- Slower: Must scan and clean up old timestamps on each request
- Used when: Exact accuracy is required (security-critical endpoints)

11.6 Sliding Window Counter

Combines the low memory of fixed window with the accuracy of sliding window. It estimates the count using a weighted average of the current and previous windows.

How It Works

```
SLIDING WINDOW COUNTER:

  Limit: 10 requests per minute
  Current time: 01:15 (15 seconds into the current window)

  Previous window [00:00-00:59]: 8 requests
  Current window  [01:00-01:59]: 3 requests so far

  Sliding window position:
  |---Previous Window---|---Current Window---|
  |########             |###                 |
  |  8 requests         | 3 requests         |
                  ^
            We are here: 01:15
            (25% into current window = 75% of prev window counts)

  Estimated count = (prev_count * overlap%) + current_count
                  = (8 * 0.75) + 3
                  = 6 + 3
                  = 9

  9 < 10 limit --> ALLOWED


  ANOTHER EXAMPLE at 01:45 (75% into current window):

  Estimated count = (8 * 0.25) + current_count
                  = 2 + current_count

  Much more room because most of the previous window
  has "slid out."
```

Step-by-Step Diagram

```
SLIDING WINDOW COUNTER VISUALIZATION:

  Previous Window (00:00-00:59)    Current Window (01:00-01:59)
  +-------------------------------+-------------------------------+
  |  ########                     | ###                           |
  |  8 requests                   | 3 requests                   |
  +-------------------------------+-------------------------------+
                                  ^
                             Window boundary

  At t=01:15 (15 sec into current window):
  Overlap with previous = 1 - (15/60) = 0.75 (75%)

  +-------------------------------+-------------------------------+
  |                  |############|####|                           |
  |   Already slid   | 75% of     |Curr|   Rest of current window  |
  |   out (25%)      | prev (6)   | (3)|                           |
  +-------------------------------+-------------------------------+
  <--- 60 second sliding window ---->

  Estimated rate = 6 + 3 = 9 requests in the sliding window
```

Key Properties

- Good accuracy: Much better than fixed window, close to sliding log
- Low memory: Only 2 counters per user per window (previous count + current count)
- Fast: Simple arithmetic, no timestamp scanning
- Slight inaccuracy: The estimate assumes previous window requests are evenly distributed (they might not be)

This is the recommended algorithm for most production rate limiters.

11.7 Algorithm Comparison

```
+------------------+----------+--------+---------+-----------+
| Algorithm        | Accuracy | Memory | Speed   | Burst     |
|                  |          |        |         | Handling  |
```

```
+------------------+----------+--------+---------+-----------+
| Token Bucket     | Good     | Low    | Fast    | Allows    |
|                  |          | (2 val)| O(1)    | bursts    |
+------------------+----------+--------+---------+-----------+
| Leaky Bucket     | Good     | Medium | Fast    | Smooths   |
|                  |          | (queue)| O(1)    | bursts    |
+------------------+----------+--------+---------+-----------+
| Fixed Window     | Poor     | Low    | Fast    | 2x burst  |
| Counter          | (boundary| (1 val)| O(1)    | at edges  |
|                  |  issue)  |        |         |           |
+------------------+----------+--------+---------+-----------+
| Sliding Window   | Exact    | High   | Slower  | No burst  |
| Log              |          | (all   | O(n)    | issues    |
|                  |          | stamps)|         |           |
+------------------+----------+--------+---------+-----------+
| Sliding Window   | Very Good| Low    | Fast    | Minor     |
| Counter          | (approx) | (2 val)| O(1)    | approx    |
+------------------+----------+--------+---------+-----------+

Recommendation:
  - Default choice: Token Bucket or Sliding Window Counter
  - Need bursting: Token Bucket
  - Need smoothing: Leaky Bucket
  - Need exact accuracy: Sliding Window Log
  - Need simplicity: Fixed Window Counter
```

11.8 Implementation Levels

Rate limiting can be applied at different points in your architecture:

```
RATE LIMITING AT DIFFERENT LAYERS:

  +--------+
  | Client |
  +---+----+
      |
      v
  +---+--------+   Layer 1: CDN / Edge
  |  CDN       |   (Cloudflare rate limiting)
  |  Rate Limit|   Catches bot traffic, DDoS
  +---+--------+
      |
      v
  +---+--------+   Layer 2: API Gateway
  |  API       |   (Kong, AWS API Gateway)
  |  Gateway   |   Per-API-key rate limiting
  +---+--------+
      |
```

```
      v
  +---+--------+   Layer 3: Load Balancer
  |  Load      |   (Nginx, HAProxy)
  |  Balancer  |   Connection rate limiting
  +---+--------+
      |
      v
  +---+--------+   Layer 4: Application
  |  App       |   (middleware in your code)
  |  Server    |   Fine-grained, per-endpoint limits
  +---+--------+
      |
      v
  +---+--------+   Layer 5: Database
  |  Database  |   Connection pooling
  |            |   Query rate limiting
  +---+--------+
```

Where to Implement

```
+------------------+-------------------------------------------+
| Layer            | Best For                                  |
+------------------+-------------------------------------------+
| CDN/Edge         | DDoS protection, bot blocking,            |
|                  | geographic filtering                      |
+------------------+-------------------------------------------+
| API Gateway      | Per-key limits, plan-based tiers,          |
|                  | centralized rate limiting                  |
+------------------+-------------------------------------------+
| Load Balancer    | Connection limits, basic request rate      |
+------------------+-------------------------------------------+
| Application      | Complex business rules (e.g., "5 login    |
|                  | attempts per account per minute")          |
+------------------+-------------------------------------------+
```

Most production systems use multiple layers. The CDN handles DDoS-scale abuse. The API gateway enforces per-customer limits. The application handles business-specific rate limiting.

11.9 HTTP 429 and Rate Limit Headers

When a request is rate limited, your API should return HTTP 429 (Too Many Requests) with headers that tell the client what happened and when to retry.

```
RATE LIMITED RESPONSE:

  HTTP/1.1 429 Too Many Requests
  Content-Type: application/json
  X-RateLimit-Limit: 100
  X-RateLimit-Remaining: 0
  X-RateLimit-Reset: 1672531260
  Retry-After: 30

  {
    "error": {
      "code": "RATE_LIMIT_EXCEEDED",
      "message": "You have exceeded the rate limit of 100
                  requests per minute. Please wait 30 seconds.",
      "retry_after": 30
    }
  }


SUCCESSFUL RESPONSE (with rate limit info):

  HTTP/1.1 200 OK
  Content-Type: application/json
  X-RateLimit-Limit: 100
  X-RateLimit-Remaining: 73
  X-RateLimit-Reset: 1672531260

  {
    "data": { ... }
  }


HEADER REFERENCE:
+------------------------+-----------------------------------+
| Header                 | Meaning                           |
+------------------------+-----------------------------------+
| X-RateLimit-Limit      | Max requests per window           |
| X-RateLimit-Remaining  | Requests remaining in window      |
| X-RateLimit-Reset      | Unix timestamp when window resets |
| Retry-After            | Seconds to wait (in 429 response) |
+------------------------+-----------------------------------+
```

Client Behavior

Good API clients respect rate limits:

```
CLIENT-SIDE RATE LIMIT HANDLING:

  1. Check X-RateLimit-Remaining before making requests
  2. If remaining is low, slow down
  3. If you get a 429, wait for Retry-After seconds
```

```
  4. Use exponential backoff for retries:
     - 1st retry: wait 1 second
     - 2nd retry: wait 2 seconds
     - 3rd retry: wait 4 seconds
     - 4th retry: wait 8 seconds
     - Add jitter (random offset) to avoid thundering herd
```

11.10 Distributed Rate Limiting

When your API runs on multiple servers, rate limiting becomes much harder. Each server sees only a fraction of the traffic.

The Problem

```
PROBLEM: LOCAL RATE LIMITING

  Limit: 100 requests per minute per user

  +----------+     +----------+     +----------+
  | Server 1 |     | Server 2 |     | Server 3 |
  | Count: 40|     | Count: 40|     | Count: 40|
  +----------+     +----------+     +----------+

  Each server thinks the user has sent 40 requests.
  Each server allows the user to continue.
  But the user has actually sent 120 requests total!
  The rate limit is ineffective.
```

Solution 1: Centralized Rate Limiter (Redis)

Use a shared counter in Redis that all servers read and write to.

```
CENTRALIZED RATE LIMITER:

  +----------+     +----------+     +----------+
  | Server 1 |     | Server 2 |     | Server 3 |
  +-----+----+     +-----+----+     +-----+----+
        |                |                |
        +--------+-------+--------+-------+
                 |                |
           +-----+-----+         |
           |   Redis    |         |
```

```
            |  Count:120 |  <------+
            |  REJECTED! |
            +------------+

  All servers check the same counter in Redis.
  Atomic increment: INCR user:42:rate_limit
```

```
REDIS IMPLEMENTATION (pseudocode):

  function is_allowed(user_id, limit, window_seconds):
      key = "rate:" + user_id + ":" + current_window()
      count = redis.INCR(key)
      if count == 1:
          redis.EXPIRE(key, window_seconds)
      return count <= limit
```

Pros: Accurate, simple logic Cons: Redis becomes a single point of failure. Added latency for every request (network round trip to Redis).

Solution 2: Sticky Sessions

Route all requests from the same user to the same server. Each server can use local rate limiting.

```
STICKY SESSIONS:

  Load Balancer routes by user_id:

  User 42 --> Always Server 1
  User 43 --> Always Server 2
  User 44 --> Always Server 3

  Each server has the complete count for its users.
  No shared state needed!
```

Pros: No shared state, fast Cons: Uneven load distribution. If a server restarts, counts are lost.

Solution 3: Loose Coordination

Each server tracks locally but periodically syncs with a central store. This allows slight over-limiting but avoids the latency of checking Redis on every request.

```
LOOSE COORDINATION:

  +----------+                +----------+
  | Server 1 |                | Server 2 |
  | Local: 15|  Sync every    | Local: 12|
  +-----+----+  5 seconds     +-----+----+
        |                           |
        +--------> Redis <----------+
                   Total: 27
                   (approximate)
```

Choosing a Strategy

```
+-------------------+----------+----------+-------------+
| Strategy          | Accuracy | Latency  | Complexity  |
+-------------------+----------+----------+-------------+
| Redis (central)   | High     | +1-2ms   | Low         |
| Sticky sessions   | High     | None     | Medium      |
| Loose coordination| Moderate | Minimal  | High        |
+-------------------+----------+----------+-------------+

For most systems, centralized Redis is the right choice.
Redis is fast enough (~1ms) that the added latency is
acceptable, and accuracy is important.
```

11.11 Rate Limiting Strategies

Different Limits for Different Endpoints

Not all endpoints are equal. Protect sensitive endpoints more aggressively:

```
+------------------------+-------------------+-----------------+
| Endpoint               | Rate Limit        | Why             |
+------------------------+-------------------+-----------------+
| GET /products          | 1000/min          | Read-only, safe |
| POST /orders           | 20/min            | Creates data    |
| POST /login            | 5/min per account | Brute force     |
|                        |                   | protection      |
| POST /password-reset   | 3/hour            | Abuse prevention|
| GET /search            | 30/min            | Expensive query |
| POST /upload           | 10/min            | Resource heavy  |
+------------------------+-------------------+-----------------+
```

Tiered Rate Limits

Different limits for different customer tiers:

```
+------------------+---------+----------+--------------------+
| Tier             | Limit   | Window   | Monthly Cost       |
+------------------+---------+----------+--------------------+
| Free             | 60/hr   | 1 hour   | $0                 |
| Starter          | 600/hr  | 1 hour   | $29                |
| Professional     | 6000/hr | 1 hour   | $99                |
| Enterprise       | Custom  | Custom   | Contact sales      |
+------------------+---------+----------+--------------------+
```

Graceful Degradation

Instead of hard rejecting, you can degrade service quality:

```
GRACEFUL DEGRADATION:

  0-50%   of limit: Full service
  50-80%  of limit: Serve from cache (skip DB)
  80-100% of limit: Return simplified responses
  >100%   of limit: 429 Too Many Requests

  This lets high-traffic users still get some value
  instead of being completely cut off.
```

Common Mistakes

1. No rate limiting at all. Every production API needs rate limiting. It is not optional.
2. Limiting only by IP. Multiple users can share an IP (corporate NAT, mobile carriers). Limit by authenticated user ID when possible.
3. Too aggressive limits. If your limit is too low, legitimate users are frustrated. Start generous and tighten based on data.
4. Not returning rate limit headers. Clients cannot respect limits they do not know about. Always include X-RateLimit-* headers.
5. Local-only rate limiting with multiple servers. Each server sees only its portion of traffic. Use centralized rate limiting (Redis) for accuracy.
6. Same limits for all endpoints. A login endpoint needs much stricter limits than a product listing endpoint.

7. Not handling the clock synchronization problem. In distributed systems, server clocks may differ. Use a central time source (Redis) rather than local server time.
8. Forgetting about legitimate spikes. If a user goes viral and their content is shared widely, sudden traffic is legitimate. Have a plan for temporary limit increases.

Best Practices

1. Use token bucket or sliding window counter. These provide the best balance of accuracy, memory, and performance.
2. Rate limit at multiple layers. CDN for DDoS, API gateway for per-key limits, application for business rules.
3. Return clear 429 responses with Retry-After. Help clients understand and respect your limits.
4. Use Redis for distributed rate limiting. It is fast, atomic, and battle-tested for this use case.
5. Set different limits for different endpoints. Expensive operations need stricter limits.
6. Monitor rate limit metrics. Track how often limits are hit, by whom, and for which endpoints. This data informs limit adjustments.
7. Provide rate limit headers on every response. Let clients see their remaining quota before they hit the limit.
8. Allow limit overrides for special cases. Internal services, load testing, and trusted partners may need higher limits. Build in the mechanism to override.
9. Use exponential backoff on the client side. Clients should not retry immediately after a 429; they should wait and increase the wait time on successive retries.
10. Log rate-limited requests. You need to distinguish between abuse (block the user) and legitimate overuse (increase the limit).

Quick Summary

Rate limiting controls how many requests a client can make in a given time period. The five main algorithms are: token bucket (allows bursts, simple, most popular), leaky bucket (smooths traffic), fixed window counter (simple but has boundary issues), sliding window log (exact but memory-heavy), and sliding window counter (good balance of accuracy and efficiency). Rate limiting should happen at multiple levels: CDN, API gateway, and application.

Return HTTP 429 with rate limit headers when limits are exceeded. For multi-server deployments, use centralized rate limiting with Redis for accurate counting. Set different limits for different endpoints and customer tiers.

Key Points

- Rate limiting protects your system from abuse, DDoS, brute force, and runaway clients
- Token bucket and sliding window counter are the best default algorithms
- Fixed window counter has a boundary problem that allows 2x burst at window edges
- Sliding window log is exact but uses too much memory for high-traffic systems
- Use Redis for distributed rate limiting -- local-only counters are inaccurate with multiple servers
- Return HTTP 429 with X-RateLimit-* headers and Retry-After
- Rate limit at multiple layers: CDN, API gateway, and application
- Different endpoints need different limits -- protect login and payment endpoints aggressively

---

Practice Questions

1. Explain the boundary problem with fixed window counters. A user has a limit of 100 requests per minute. Show a scenario where they make 200 requests in 2 minutes but 150 of them fall within a single 60-second span.
2. Your API serves 50,000 users with a rate limit of 100 requests per minute per user. You use the sliding window log algorithm, storing timestamps in Redis. Estimate the memory usage. Then calculate the memory for the same scenario using the sliding window counter. Which would you choose?
3. A customer complains that they are being rate limited even though they believe they are under the limit. You have 5 API servers behind a load balancer, each with its own local rate counter. Explain the problem and propose a solution.
4. Design a rate limiting system for a login endpoint that prevents brute-force attacks but does not lock out legitimate users. Consider: per-IP limits, per-account limits, CAPTCHA triggers, and gradual lockout.
5. Compare token bucket and leaky bucket for an API that receives bursty traffic (e.g., a mobile app that syncs data every few minutes). Which algorithm would be better and

why?

Exercises

Exercise 1: Implement Token Bucket Write pseudocode for a token bucket rate limiter. Include: initialization (bucket size, refill rate), the `allow_request()` function, and token refill logic. Handle the case where multiple seconds have passed since the last request (add multiple tokens at once, capped at bucket size).

Exercise 2: Distributed Rate Limiter Design Design a distributed rate limiting service using Redis. Draw the architecture with multiple API servers, a Redis cluster, and the request flow. Address: how to handle Redis being temporarily unavailable (fail open or fail closed?), how to minimize the number of Redis calls per request, and how to handle clock skew.

Exercise 3: Rate Limit Configuration You are building a SaaS API with three tiers: Free, Pro, and Enterprise. Design the rate limit configuration for these endpoints: GET /items (list), POST /items (create), GET /items/:id (detail), DELETE /items/:id (delete), POST /items/bulk (batch create), GET /search (full-text search). For each tier and endpoint, specify the limit and window, and explain your reasoning.

What Is Next?

In this chapter, you learned how to protect your API from excessive traffic. But rate limiting is just one piece of the puzzle. When you need to distribute data across multiple servers -- whether for a cache, a database, or a rate limiter -- you need a way to decide which server handles which data. In Chapter 12, you will learn about consistent hashing, the algorithm that solves this distribution problem elegantly and is used by virtually every distributed system.

Chapter 12: Consistent Hashing

What You Will Learn

- Why simple modular hashing breaks down when servers are added or removed
- How consistent hashing solves this problem using a hash ring
- The clock analogy for understanding the hash ring
- Virtual nodes and why they are essential for even distribution
- A step-by-step walkthrough with ASCII ring diagrams
- Real-world use cases: DynamoDB, Cassandra, CDN routing, load balancing
- How to handle server failures and additions gracefully
- Trade-offs and alternatives to consistent hashing

Why This Chapter Matters

Whenever you distribute data across multiple servers -- whether it is a distributed cache, a sharded database, or a CDN -- you need a way to decide which server handles which piece of data. The naive approach (modular hashing) works until you add or remove a server, at which point nearly all your data needs to be moved. This is catastrophic at scale.

Consistent hashing is the algorithm that solves this problem. It minimizes data movement when servers change. It is used by almost every major distributed system: Amazon DynamoDB, Apache Cassandra, Akamai CDN, Discord, and many more.

If you understand consistent hashing, you understand the foundation of data distribution in distributed systems.

12.1 The Problem with Modular Hashing

The simplest way to assign data to servers is modular hashing: take the hash of the key and compute `hash(key) % number_of_servers`.

How Modular Hashing Works

```
MODULAR HASHING:

  3 servers: Server 0, Server 1, Server 2

  hash("user_A") = 7    -->  7 % 3 = 1  --> Server 1
  hash("user_B") = 12   --> 12 % 3 = 0  --> Server 0
  hash("user_C") = 5    -->  5 % 3 = 2  --> Server 2
  hash("user_D") = 9    -->  9 % 3 = 0  --> Server 0
  hash("user_E") = 14   --> 14 % 3 = 2  --> Server 2
  hash("user_F") = 3    -->  3 % 3 = 0  --> Server 0

  Server 0: user_B, user_D, user_F
  Server 1: user_A
  Server 2: user_C, user_E
```

This works well with a fixed number of servers. But what happens when you add a server?

The Catastrophe: Adding a Server

```
ADDING SERVER 3 (now 4 servers):

  hash("user_A") = 7    -->  7 % 4 = 3  --> Server 3  (was 1!)
  hash("user_B") = 12   --> 12 % 4 = 0  --> Server 0  (same)
  hash("user_C") = 5    -->  5 % 4 = 1  --> Server 1  (was 2!)
  hash("user_D") = 9    -->  9 % 4 = 1  --> Server 1  (was 0!)
  hash("user_E") = 14   --> 14 % 4 = 2  --> Server 2  (same)
  hash("user_F") = 3    -->  3 % 4 = 3  --> Server 3  (was 0!)

  MOVED:
    user_A: Server 1 --> Server 3  MOVED
    user_B: Server 0 --> Server 0  stayed
    user_C: Server 2 --> Server 1  MOVED
    user_D: Server 0 --> Server 1  MOVED
    user_E: Server 2 --> Server 2  stayed
    user_F: Server 0 --> Server 3  MOVED

  4 out of 6 keys moved! That is 67%.

  With millions of cache keys, this means:
  - 67% cache miss rate (data is on the wrong server)
  - Massive database load as cache repopulates
  - Potential system outage
```

The Math Is Brutal

When you go from N servers to N+1 servers with modular hashing:

```
Fraction of keys that must move ≈ N / (N+1)

  3 --> 4 servers: ~75% of keys move
  9 --> 10 servers: ~90% of keys move
  99 --> 100 servers: ~99% of keys move

  The more servers you have, the WORSE it gets!
```

This is unacceptable in a production system. You need a way to add or remove servers while moving as few keys as possible. That is exactly what consistent hashing does.

12.2 Consistent Hashing: The Hash Ring

Consistent hashing maps both servers and keys onto a circular ring (a "hash ring"). Each key is assigned to the nearest server on the ring, going clockwise.

The Clock Analogy

Think of a clock face. The hours 1 through 12 are positions on the ring. Servers are placed at specific positions on the clock. Each key is hashed to a position on the clock, and it is assigned to the next server you encounter going clockwise.

```
THE HASH RING (Clock Analogy):

  Imagine the ring as a clock with positions 0 to 359 (degrees).

  Servers are placed at specific positions:
    Server A at position 90   (3 o'clock)
    Server B at position 210  (7 o'clock)
    Server C at position 330  (11 o'clock)

                      0 / 360
                        |
                    330 |
                  C *   |
                 /      |
               /        |
             /          |
  270 ---               +--- 90   * A
```

```
          \           |
            \         |
              \       |
               *      |
                 210  |
                  B   |
                      |
                    180

  How keys are assigned:
    hash("user_1") = 50   --> Clockwise to A (90)
    hash("user_2") = 120  --> Clockwise to B (210)
    hash("user_3") = 250  --> Clockwise to C (330)
    hash("user_4") = 340  --> Clockwise to A (90)*
    hash("user_5") = 100  --> Clockwise to B (210)

    * 340 passes 360/0 and the next server is A at 90
```

Step-by-Step: How It Works

```
STEP 1: Create the ring

  The ring represents the full range of the hash function.
  For example, if using SHA-256, the ring goes from 0 to 2^256.
  We will use 0-359 for simplicity.

      0
      |
  330 *C           * A  90
      |           |
      |           |
      |           |
  270 +-----------+ 180
      |
      * B
      210


STEP 2: Place servers on the ring

  Hash each server's identifier to get its position:
    hash("Server_A") = 90
    hash("Server_B") = 210
    hash("Server_C") = 330


STEP 3: Place keys on the ring

  Hash each key to get its position:
    hash("key1") = 50
    hash("key2") = 150
```

```
    hash("key3") = 250

          0
          |
     k1 . |
    330 *C |          * A  90
          |         |
          |  . k2   |
          | 150     |
    270 --+---------+-- 180
          |
     k3 . |
     250  * B
          210


STEP 4: Assign keys to servers

  Walk clockwise from each key to find the first server:
    key1 (50)  --> clockwise --> Server A (90)
    key2 (150) --> clockwise --> Server B (210)
    key3 (250) --> clockwise --> Server C (330)

  Server A: key1
  Server B: key2
  Server C: key3
```

12.3 Adding and Removing Servers

The magic of consistent hashing is what happens when servers change.

Adding a Server

```
ADDING SERVER D at position 150:

  BEFORE:                          AFTER:
       0                                0
       |                                |
  330 *C     * A 90               330 *C     * A 90
       |     |                          |     |
       |     |                          |  *D |
       |     |                          | 150 |
  270 -+-----+- 180              270 --+-----+-- 180
       |                                |
       *B                               *B
      210                              210
```

```
  Keys reassigned:
    key1 (50)  --> Still Server A (90)    NO CHANGE
    key2 (150) --> Now Server D (150)     MOVED (was B)
    key3 (250) --> Still Server C (330)   NO CHANGE

  Only key2 moved! (1 out of 3 keys = 33%)

  In general, when adding a server with N existing servers:
    Only K/N keys move (where K = total keys)
    That is 1/N of all keys, which is the MINIMUM possible.
```

Removing a Server

```
REMOVING SERVER B (at position 210):

  BEFORE:                           AFTER:
       0                                 0
       |                                 |
  330 *C     * A 90                 330 *C     * A 90
       |     |                           |     |
       |     |                           |     |
       |     |                           |     |
  270 -+-----+- 180                270 --+-----+-- 180
       |                                 |
      *B  <-- removed                    |
     210                                 |

  Keys reassigned:
    key1 (50)  --> Still Server A (90)    NO CHANGE
    key2 (150) --> Now Server C (330)     MOVED (was B)
    key3 (250) --> Now Server C (330)     MOVED (was B)

  Only keys that were on Server B moved to the
  next server clockwise (Server C).
  Other keys are unaffected.
```

Comparison: Modular vs Consistent Hashing

```
KEYS MOVED WHEN ADDING 1 SERVER:

+------------------+----------------------+----------------------+
| Total Servers    | Modular Hashing      | Consistent Hashing   |
|                  | (keys moved)         | (keys moved)         |
+------------------+----------------------+----------------------+
| 3 --> 4          | ~75%                 | ~25%                 |
| 9 --> 10         | ~90%                 | ~10%                 |
```

```
| 99 --> 100       | ~99%                  | ~1%                   |
| 999 --> 1000     | ~99.9%               | ~0.1%                 |
+------------------+-----------------------+-----------------------+

Consistent hashing: Only K/N keys move (minimal disruption)
Modular hashing:    ~N/(N+1) keys move (catastrophic at scale)
```

12.4 The Problem with Basic Consistent Hashing

Basic consistent hashing has a serious flaw: uneven distribution. With only a few servers, the ring segments are likely to be very unequal in size, meaning some servers handle far more keys than others.

```
UNEVEN DISTRIBUTION:

  Imagine 3 servers that happen to hash close together:

           0
           |
      340 *C
      350 |*A
           |  *B  20
           |
  270 -----+----- 90
           |
           |
           |
           |
          180

  Server A handles: keys from 350 to 20  (30 degrees)
  Server B handles: keys from 20 to 340  (320 degrees!)
  Server C handles: keys from 340 to 350 (10 degrees)

  Server B handles ~80% of all keys!
  Servers A and C are nearly idle.
  This defeats the purpose of distributing load.
```

The solution is virtual nodes.

12.5 Virtual Nodes

Instead of placing each server at one position on the ring, place each server at multiple positions. Each position is called a virtual node (vnode). A server with 100 virtual nodes appears at 100 different positions on the ring.

How Virtual Nodes Work

```
VIRTUAL NODES:

  Instead of:
    hash("Server_A") --> 1 position

  Use:
    hash("Server_A_1") --> position 45
    hash("Server_A_2") --> position 120
    hash("Server_A_3") --> position 200
    hash("Server_A_4") --> position 310

  Server A now "owns" 4 segments of the ring.


  WITH 4 VIRTUAL NODES PER SERVER (3 servers = 12 vnodes):

  Server A vnodes: A1(45), A2(120), A3(200), A4(310)
  Server B vnodes: B1(30), B2(95),  B3(170), B4(280)
  Server C vnodes: C1(60), C2(150), C3(230), C4(345)

           0
           |
      C4.345  B1.30
           | A1.45
      A4.310  C1.60
           |  B2.95
           | A2.120
  270 -----+----- 90
           | C2.150
           | B3.170
           |A3.200
           |C3.230
           |B4.280
          180

  Now the ring is much more evenly divided!
  Each server handles roughly 1/3 of the keys.
```

Why Virtual Nodes Work

```
DISTRIBUTION WITH DIFFERENT VNODE COUNTS:
```

```
+---------------------+-----------------------------------+
| VNodes per Server   | Load Distribution                 |
+---------------------+-----------------------------------+
| 1 (no vnodes)       | Highly uneven (10x difference)    |
| 10                  | Moderately even (2-3x difference)|
| 100                 | Very even (~10% variation)        |
| 200                 | Nearly perfect (~5% variation)    |
+---------------------+-----------------------------------+

More virtual nodes = more even distribution
But also more memory to store the ring
```

Virtual Nodes and Heterogeneous Servers

Virtual nodes also let you handle servers with different capacities. A powerful server gets more virtual nodes and handles more keys.

```
HETEROGENEOUS SERVERS:

  Server A: 16 GB RAM --> 100 virtual nodes
  Server B: 32 GB RAM --> 200 virtual nodes
  Server C: 64 GB RAM --> 400 virtual nodes

  Server C handles ~4x the keys of Server A,
  which matches its ~4x capacity.
```

12.6 Step-by-Step Example

Let us walk through a complete example with key operations.

Initial Setup

```
SETUP: 3 servers, 3 virtual nodes each = 9 ring positions

  hash("A-1")=10,  hash("A-2")=120, hash("A-3")=240
  hash("B-1")=50,  hash("B-2")=160, hash("B-3")=300
  hash("C-1")=80,  hash("C-2")=200, hash("C-3")=330

  Ring (sorted positions):
  10(A), 50(B), 80(C), 120(A), 160(B), 200(C), 240(A), 300(B), 330(C)

       0
```

```
      |
330(C)  10(A)
      |   50(B)
300(B)   80(C)
      |  120(A)
      |  160(B)
240(A)  200(C)
      |
     180
```

Assigning Keys

```
hash("order_1") = 15   --> Clockwise to 50(B)  --> Server B
hash("order_2") = 90   --> Clockwise to 120(A) --> Server A
hash("order_3") = 210  --> Clockwise to 240(A) --> Server A
hash("order_4") = 310  --> Clockwise to 330(C) --> Server C
hash("order_5") = 5    --> Clockwise to 10(A)  --> Server A
hash("order_6") = 155  --> Clockwise to 160(B) --> Server B

Result:
  Server A: order_2, order_3, order_5
  Server B: order_1, order_6
  Server C: order_4
```

Adding Server D

```
New virtual nodes:
hash("D-1")=70, hash("D-2")=190, hash("D-3")=260

Updated ring (sorted):
10(A), 50(B), 70(D), 80(C), 120(A), 160(B), 190(D),
200(C), 240(A), 260(D), 300(B), 330(C)

Re-evaluate affected keys:
hash("order_1") = 15  --> 50(B)  --> Server B  (unchanged)
hash("order_2") = 90  --> 120(A) --> Server A  (unchanged)
hash("order_3") = 210 --> 240(A) --> Server A  (unchanged)
hash("order_4") = 310 --> 330(C) --> Server C  (unchanged)
hash("order_5") = 5   --> 10(A)  --> Server A  (unchanged)
hash("order_6") = 155 --> 160(B) --> Server B  (unchanged)

In this example, no keys moved! This is because D's
virtual nodes landed in positions that did not "steal"
any existing key assignments. In practice, about K/N
keys would move (where N is the new total server count).
```

Server B Fails

```
Remove all of B's virtual nodes: 50, 160, 300

Updated ring:
10(A), 70(D), 80(C), 120(A), 190(D), 200(C),
240(A), 260(D), 330(C)

Re-evaluate keys that were on Server B:
hash("order_1") = 15  --> was 50(B) --> now 70(D) --> Server D
hash("order_6") = 155 --> was 160(B)--> now 190(D)--> Server D

Only keys on Server B moved. All other keys stay.

Keys that were on B are distributed to the next
servers clockwise from B's old positions, which
means they spread across D, C, and A -- not all
dumped on one server.
```

12.7 Real-World Usage

Amazon DynamoDB

DynamoDB uses consistent hashing to distribute data across storage nodes. Each table's partition key is hashed to determine which node stores it. When nodes are added or removed, only a fraction of partitions are rebalanced.

```
DYNAMODB PARTITIONING:

  Table: Orders
  Partition Key: order_id

  hash("order_001") --> Node 3
  hash("order_002") --> Node 1
  hash("order_003") --> Node 5

  When Node 4 is added:
  Only keys in the ring segment "taken over" by
  Node 4 are moved. Other keys stay on their nodes.
```

Apache Cassandra

Cassandra uses consistent hashing as its core data distribution mechanism. Each node is assigned a range of tokens on the ring. Data is placed on the node that owns the token range for its partition key.

```
CASSANDRA TOKEN RING:

  Node A: owns tokens 0 - 85
  Node B: owns tokens 86 - 170
  Node C: owns tokens 171 - 255

  Cassandra also uses virtual nodes (vnodes) by default.
  Each node owns 256 small token ranges instead of 1 large one.
  This makes rebalancing faster and load distribution more even.
```

CDN Routing

CDNs use consistent hashing to decide which edge server caches which content. When an edge server goes down, only its content needs to be re-routed to other edges. Other content is unaffected.

```
CDN CONSISTENT HASHING:

  URL: /images/cat.jpg
  hash("/images/cat.jpg") --> Edge Server in Tokyo

  Tokyo edge goes down:
  hash("/images/cat.jpg") --> Next server clockwise
                          --> Edge Server in Seoul

  All other URLs continue to be served by their
  original edge servers. No mass cache invalidation.
```

Discord

Discord uses consistent hashing to distribute chat messages and user state across its backend servers. When servers are added to handle growth, only a fraction of users are moved to the new servers.

Load Balancers

Some load balancers use consistent hashing to route requests. This ensures that the same user always hits the same backend server (session affinity) while minimizing disruption when servers change.

```
CONSISTENT HASH LOAD BALANCING:

  hash(user_session_id) --> Backend Server

  Advantage over round-robin:
  - Same user always hits same server (good for caching)
  - Adding/removing servers only affects some users
  - Better cache hit ratios on backend servers
```

12.8 Implementation Details

Choosing a Hash Function

The hash function must produce uniformly distributed values. Common choices:

```
+------------------+------------------+--------------------+
| Hash Function    | Speed            | Distribution       |
+------------------+------------------+--------------------+
| MD5              | Medium           | Excellent          |
| SHA-1            | Medium           | Excellent          |
| MurmurHash       | Fast             | Very good          |
| xxHash           | Very fast        | Very good          |
| CRC32            | Very fast        | Acceptable         |
+------------------+------------------+--------------------+

MurmurHash and xxHash are the most common choices for
consistent hashing because they are fast and have good
distribution without the overhead of cryptographic hashes.
```

Data Structure for the Ring

The ring is typically implemented as a sorted array or a balanced binary search tree (like a red-black tree or a TreeMap in Java).

```
RING LOOKUP:

  Sorted array of positions: [10, 50, 80, 120, 160, 200, 240, 300, 330]
  Each position maps to a server: [A, B, C, A, B, C, A, B, C]

  To find the server for a key:
  1. Compute hash(key) = position
  2. Binary search for the smallest position >= key's position
```

```
  3. If no position >= key's position, wrap around to the first position

  Time complexity: O(log N) where N = number of virtual nodes
  Space complexity: O(N)

  With 3 servers and 200 vnodes each = 600 positions
  Binary search on 600 elements is very fast (~10 comparisons)
```

Replication with Consistent Hashing

Many distributed systems replicate data to multiple servers for fault tolerance. With consistent hashing, data is replicated to the next N servers clockwise on the ring.

```
REPLICATION (replicas = 3):

  hash("key1") = 85

  Ring: ... 80(C), 120(A), 160(B), 200(C), ...

  Primary:  Server C (position 80 is closest but we go to next: 120? No, 80 is before 85)
  Actually: Walk clockwise from 85:
    Replica 1: 120 --> Server A  (primary)
    Replica 2: 160 --> Server B
    Replica 3: 200 --> Server C

  Data is stored on A, B, and C.
  If Server A fails, data is still available on B and C.

  With virtual nodes, you must ensure replicas are on
  DIFFERENT physical servers, not just different vnodes
  of the same server.
```

12.9 Alternatives and Variations

Jump Consistent Hash

A simpler algorithm that produces balanced distribution without a ring. It maps keys to buckets using a deterministic sequence of random jumps.

Pros: Very fast, zero memory, perfectly balanced Cons: Only works when adding/removing servers at the end (cannot remove a server from the middle)

Rendezvous Hashing (Highest Random Weight)

Each key is assigned to the server that produces the highest hash value for that key-server combination.

```
RENDEZVOUS HASHING:

  For key "user_1":
    hash("user_1" + "Server_A") = 0.72
    hash("user_1" + "Server_B") = 0.31
    hash("user_1" + "Server_C") = 0.89  <-- Highest!
    --> Assign to Server C

  For key "user_2":
    hash("user_2" + "Server_A") = 0.65  <-- Highest!
    hash("user_2" + "Server_B") = 0.42
    hash("user_2" + "Server_C") = 0.18
    --> Assign to Server A
```

Pros: Simple, no ring data structure, good distribution Cons: Must compute hash for every server on every lookup (O(N) per lookup)

Comparison

```
+---------------------+----------+---------+----------+---------+
| Algorithm           | Lookup   | Memory  | Balance  | Remove  |
|                     | Speed    |         |          | Any Node|
+---------------------+----------+---------+----------+---------+
| Consistent Hashing  | O(log N) | O(N)    | Good     | Yes     |
| (with vnodes)       |          | (vnodes)|  (vnodes)|         |
+---------------------+----------+---------+----------+---------+
| Jump Hash           | O(log N) | O(1)    | Perfect  | No*     |
+---------------------+----------+---------+----------+---------+
| Rendezvous Hash     | O(N)     | O(1)    | Good     | Yes     |
+---------------------+----------+---------+----------+---------+

* Jump Hash only supports adding/removing at the end
```

Common Mistakes

1. Using consistent hashing without virtual nodes. Without virtual nodes, the distribution is extremely uneven. Always use virtual nodes (100-200 per server is a common starting point).

2. Not accounting for heterogeneous servers. If servers have different capacities, assign proportionally more virtual nodes to more powerful servers.
3. Replicating to virtual nodes of the same server. When replicating, walk clockwise but skip virtual nodes that belong to a physical server you have already replicated to.
4. Using a poor hash function. A hash function with clustering (non-uniform output) defeats the purpose of consistent hashing. Use MurmurHash or xxHash.
5. Ignoring hot keys. Consistent hashing distributes keys evenly, but it does not solve the problem of one key being extremely popular (e.g., a viral tweet). You need additional mechanisms (replication, caching) for hot keys.
6. Too few virtual nodes. With fewer than 100 virtual nodes per server, load imbalance can be significant. More virtual nodes improve balance but use more memory.
7. Not handling ring changes atomically. When adding or removing servers, all nodes must agree on the new ring configuration at the same time. Otherwise, different nodes route the same key to different servers.

Best Practices

1. Use 100-200 virtual nodes per server. This provides good balance without excessive memory usage.
2. Use MurmurHash or xxHash for fast, well-distributed hashing.
3. Store the ring in a sorted data structure (TreeMap, sorted array) for efficient O(log N) lookups.
4. Replicate data to N distinct physical servers, not just N positions on the ring.
5. Handle hot keys separately. For extremely popular keys, use caching or split the key across multiple sub-keys.
6. Use a coordination service (ZooKeeper, etcd, Consul) to maintain the ring configuration and ensure all nodes have a consistent view.
7. Scale virtual nodes with server capacity. More powerful servers should have more virtual nodes to receive proportionally more traffic.
8. Monitor key distribution. Periodically check that keys are distributed evenly across servers. Adjust virtual node counts if needed.

Quick Summary

Modular hashing (hash % N) is simple but catastrophic when servers change: nearly all keys must be remapped. Consistent hashing solves this by placing both servers and keys on a circular hash ring. Each key is assigned to the first server encountered clockwise. When a server is added or removed, only K/N keys move (where K is total keys and N is total servers), which is the theoretical minimum. Virtual nodes (multiple ring positions per server) solve the uneven distribution problem by spreading each server's responsibility across many small segments. Real-world systems like DynamoDB, Cassandra, and CDN routers all use consistent hashing. Alternatives include jump consistent hash (simple, but limited) and rendezvous hashing (no ring, but O(N) lookup).

Key Points

- Modular hashing causes ~N/(N+1) of keys to move when a server is added -- catastrophic at scale
- Consistent hashing uses a ring where keys are assigned to the nearest server clockwise
- Adding or removing a server moves only K/N keys (the minimum possible disruption)
- Virtual nodes (100-200 per server) are essential for even distribution across servers
- Heterogeneous servers get proportionally more virtual nodes based on their capacity
- Used by DynamoDB, Cassandra, CDN routers, Discord, and many distributed caches
- Use MurmurHash or xxHash for fast, uniform hash distribution
- Replication works by assigning data to the next N distinct physical servers on the ring

---

Practice Questions

1. You have 5 cache servers and 1 million keys distributed using modular hashing. One server fails and you remove it. How many keys need to be remapped? Now answer the same question for consistent hashing with virtual nodes. What is the practical impact on cache hit ratio in each case?
2. You are running a distributed cache with consistent hashing and 3 servers. Server A has 16GB RAM, Server B has 32GB RAM, and Server C has 64GB RAM. How would you use virtual nodes to ensure each server handles traffic proportional to its capacity? How many virtual nodes would you assign to each?
3. Explain why consistent hashing alone does not solve the hot key problem. Give an example of a hot key in a social media system and propose a solution that works

alongside consistent hashing.

4. Compare consistent hashing, jump hash, and rendezvous hashing for a CDN with 200 edge servers that occasionally adds and removes servers. Which would you choose and why?
5. In a Cassandra cluster using consistent hashing with virtual nodes, a new node is added. Describe step by step what happens to the data: which data moves, how it is transferred, and how the cluster maintains availability during the transition.

Exercises

Exercise 1: Build a Hash Ring Implement a consistent hash ring in pseudocode. Support these operations: `add_server(server_id, num_vnodes)`, `remove_server(server_id)`, and `get_server(key)`. Use a sorted array for the ring. Show the state of the ring after each operation for a scenario with 3 servers being added and then 1 being removed.

Exercise 2: Distribution Analysis Using 3 servers and 1000 randomly generated keys, calculate the number of keys per server with: (a) modular hashing, (b) consistent hashing with 1 vnode per server, (c) consistent hashing with 10 vnodes per server, (d) consistent hashing with 200 vnodes per server. Show the standard deviation of keys per server for each approach.

Exercise 3: Replication on the Ring Design a replication scheme using consistent hashing for a distributed key-value store with a replication factor of 3. Draw the hash ring with 4 servers (3 vnodes each). Show which servers hold each replica for 5 different keys. Handle the case where consecutive vnodes belong to the same physical server (skip them for replica placement).

What Is Next?

Consistent hashing is a fundamental building block for distributed systems. It answers the question "which server handles this data?" In the next chapter, you will learn about the CAP theorem, which answers a deeper question: "what guarantees can a distributed system provide?" The CAP theorem explains the fundamental trade-offs between consistency, availability, and partition tolerance -- the constraints that every distributed system must navigate.

Chapter 13: The CAP Theorem

What You Will Learn

- What the CAP theorem is and why it matters for distributed systems
- The meaning of Consistency, Availability, and Partition Tolerance
- Why you can only guarantee two out of three properties at a time
- The difference between CP and AP systems with real examples
- How the PACELC extension adds nuance to the CAP theorem
- Strong consistency vs eventual consistency and when to use each
- How real databases like DynamoDB, Cassandra, MongoDB, and PostgreSQL make their trade-offs

Why This Chapter Matters

Every distributed system you design will force you to make a choice: what happens when part of your network fails? Do you show users stale data to keep things running? Or do you reject requests until the system is back in sync?

The CAP theorem is the foundation of that decision. If you skip this chapter, you will build systems that behave in ways you did not predict. You will pick databases without understanding their guarantees. And when things go wrong at 3 AM, you will not know why your system made the choices it did.

Understanding CAP is not just academic. It is the lens through which experienced engineers evaluate every distributed database, every replication strategy, and every architecture decision.

The CAP Theorem Explained

In 2000, computer scientist Eric Brewer proposed a conjecture. In 2002, Seth Gilbert and Nancy Lynch proved it. The idea is simple but powerful.

In any distributed data store, you can only guarantee two out of three properties:

- C — Consistency
- A — Availability
- P — Partition Tolerance

Let us define each one clearly.

Consistency (C)

Every read receives the most recent write or an error. All nodes in the system see the same data at the same time.

If you write "balance = $500" and then immediately read, you get $500. Not $400. Not "unknown." You get the latest value.

Availability (A)

Every request receives a response (not an error). The system keeps working even if some nodes are slow or busy.

Note: availability here does not mean the response is the most up-to-date. It just means you get a response.

Partition Tolerance (P)

The system continues to operate even when network communication between nodes is lost. Messages between nodes can be dropped, delayed, or lost entirely.

A partition is when two parts of your system cannot talk to each other. Think of it as a broken phone line between two offices.

The Real-Life Analogy: Two Offices, One Phone Line

Imagine you run a bank with two branch offices: Office A in New York and Office B in London.

Both offices can accept deposits and withdrawals. They share customer account balances by calling each other on a single phone line after every transaction.

```
+------------------+          Phone Line          +------------------+
|   Office A       |<----------------------------->|   Office B       |
|   (New York)     |     Account Balance Sync      |   (London)       |
|                  |                               |                  |
|  Balance: $1000  |                               |  Balance: $1000  |
+------------------+                               +------------------+
```

Everything works fine when the phone line is up. A customer deposits $200 in New York, Office A calls Office B, and both update to $1200.

Now the phone line goes down. This is a partition.

A customer walks into Office A and deposits $200. Another customer walks into Office B and withdraws $300. Neither office can tell the other what happened.

You now have a choice:

Choice 1: Stay Consistent (CP)

Both offices stop processing transactions until the phone line is restored. Customers are turned away. This is frustrating, but when the line comes back, the balances are correct.

"Sorry, our system is down. Please come back later."

Choice 2: Stay Available (AP)

Both offices keep processing transactions using their local copy of the balance. Customers are happy — they can deposit and withdraw. But the balances are now different in each office:

- Office A thinks the balance is $1200
- Office B thinks the balance is $700

When the phone line comes back, you have a conflict. Which balance is correct? The answer is: neither. You need a reconciliation strategy.

Why Not Choose CA (Consistent + Available, No Partition Tolerance)?

In a distributed system, network partitions will happen. Cables get cut. Switches fail. Data centers lose connectivity. You cannot choose to ignore partitions.

If all your data is on a single machine, you do not have partition tolerance to worry about — but you also do not have a distributed system. A single PostgreSQL server is effectively a CA system, but it does not help when you need to scale beyond one machine.

```
          The CAP Triangle

              C
             / \
            /   \
           /     \
          / CP  CA \
         /   or   \
        /   AP     \
       /             \
      A ------------- P

  You must pick a side.
  In distributed systems, P is mandatory.
  So the real choice is: CP or AP?
```

CP Systems: Consistency Over Availability

A CP system prioritizes consistency. When a partition occurs, the system will refuse to respond rather than return potentially stale data.

How It Works

```
Client Request: "Read balance for account #123"

+----------+          PARTITION          +----------+
| Node A   |      X  X  X  X  X         | Node B   |
| (Primary)|     Cannot reach B          | (Replica)|
|          |                             |          |
| Has the  |                             | May have |
| latest   |                             | stale    |
| write    |                             | data     |
+----------+                             +----------+
     |
     v
  Node A responds: "Error - cannot confirm consistency"
  OR
  Node A responds with latest data (if it is the primary)
```

Real-World CP Example: Banking Systems

When you transfer $500 from your checking to your savings account, the system must ensure:

1. The $500 is deducted from checking
2. The $500 is added to savings
3. Both operations succeed or both fail
4. No one reads a state where $500 has left checking but has not arrived in savings

If there is a network issue, the bank shows you an error rather than risk showing you the wrong balance. This is CP behavior.

CP Databases

- MongoDB (with majority write concern): Writes require acknowledgment from a majority of nodes. If a partition isolates the primary from the majority, the primary steps down.
- HBase: Uses a single master for consistency. If the master fails, the system is unavailable until a new one is elected.
- etcd / ZooKeeper: Used for coordination and configuration. They choose consistency because correct configuration is more important than always being available.

AP Systems: Availability Over Consistency

An AP system prioritizes availability. When a partition occurs, every node keeps serving requests, even if some nodes have stale data.

How It Works

```
Client Request: "Read post likes count"

+----------+          PARTITION          +----------+
| Node A   |      X  X  X  X  X         | Node B   |
|          |     Cannot sync data        |          |
|          |                             |          |
| Likes:   |                             | Likes:   |
| 1,042    |                             | 1,039    |
+----------+                             +----------+
     |                                        |
     v                                        v
  Responds: 1,042                     Responds: 1,039
```

```
  Both responses are "valid" even though
  they disagree. They will converge later.
```

Real-World AP Example: Social Media Likes

When you like a post on social media, the like count might temporarily differ depending on which server handles your request. One user might see 1,042 likes while another sees 1,039.

This is acceptable. Nobody loses money. The counts will eventually converge when the partition heals.

AP Databases

- Cassandra: Designed for availability. Every node can accept writes. Uses tunable consistency (you can adjust per-query).
- DynamoDB: By default, reads are eventually consistent. The system always responds, even during partitions.
- CouchDB: Designed for availability and eventual consistency. Uses multi-version concurrency control to handle conflicts.

The Consistency Spectrum

Consistency is not binary. It exists on a spectrum from strong to weak.

```
Strong                                                    Weak
Consistency                                           Consistency
   |                                                       |
   v                                                       v
+--------+    +-----------+    +----------+    +----------+
| Linear-|    | Sequential|    | Causal   |    | Eventual |
| izable |    | Consistency|   | Consist. |    | Consist. |
+--------+    +-----------+    +----------+    +----------+
   |               |               |               |
   v               v               v               v
 Every read      Operations     Causally        Reads may
 sees the        appear in      related ops     return stale
 latest          some total     are ordered;    data, but
 write.          order,         unrelated ops   will converge
 Slowest.        consistent     may be out      eventually.
                 across all     of order.       Fastest.
                 clients.
```

Strong Consistency

Every read returns the most recent write. Period. If User A writes "name = Alice" and User B reads immediately after, User B gets "Alice."

This is expensive. The system must coordinate across all nodes before acknowledging a write.

Use when: Financial transactions, inventory systems, anything where stale reads cause real harm.

Eventual Consistency

If no new writes happen, all replicas will eventually converge to the same value. But "eventually" could be milliseconds or seconds — the system does not guarantee when.

Use when: Social media feeds, view counters, recommendation engines, DNS.

Causal Consistency

Operations that are causally related are seen in the correct order. If User A posts a message and then User B replies to it, everyone sees the original message before the reply.

But two unrelated operations might appear in different orders to different observers.

Use when: Chat applications, collaborative editing, comment threads.

How Real Systems Make Their Choice

No real-world system is purely CP or purely AP. Most systems let you tune the trade-off.

DynamoDB

DynamoDB defaults to eventual consistency for reads (AP behavior). But you can request a strongly consistent read that costs more and takes longer. This lets you choose per-query.

```
+------------------+     +------------------+
|  Application     |     |  Application     |
|  (Read: eventual)|     |  (Read: strong)  |
+--------+---------+     +--------+---------+
         |                        |
```

```
        v                          v
+------+------+          +------+------+
| Any replica |          | Leader node |
| can respond |          | must respond|
| Fast, cheap |          | Slow, costly|
+-------------+          +-------------+
```

Cassandra

Cassandra uses tunable consistency. You set the consistency level per operation:

- `ONE`: Write/read from one node (fast, AP behavior)
- `QUORUM`: Write/read from majority of nodes (balanced)
- `ALL`: Write/read from all nodes (slow, CP behavior)

```
Cassandra Tunable Consistency

Write to 3 replicas (Replication Factor = 3)

Consistency    Nodes that    Nodes that    Behavior
Level          must ACK      can fail
-------------------------------------------------------
ONE            1             2             AP (fast)
QUORUM         2             1             Balanced
ALL            3             0             CP (slow)
```

The common pattern is `QUORUM` for both reads and writes. If you have 3 replicas and use QUORUM (2 of 3), you guarantee that at least one node in your read set has the latest write. This gives you strong consistency without needing ALL nodes.

MongoDB

MongoDB uses a primary-secondary replication model. Writes go to the primary. By default, reads can go to secondaries (which may be slightly behind).

You can configure:

- Write Concern: How many nodes must acknowledge a write (e.g., `majority`)
- Read Concern: Whether reads must come from the primary or can come from secondaries
- Read Preference: Which node to read from (primary, secondary, nearest)

With `writeConcern: majority` and `readConcern: majority`, MongoDB behaves as a CP system.

PostgreSQL

A single PostgreSQL instance is consistent and available but not partition tolerant (it is a single machine). When you add replication:

- Synchronous replication: The primary waits for the replica to confirm. Strong consistency, but slower and less available (CP).
- Asynchronous replication: The primary does not wait. Faster and more available, but replicas may be behind (AP-ish).

PACELC: Beyond CAP

The CAP theorem only talks about what happens during a partition. But what about normal operation? That is where PACELC comes in.

PACELC says:

> If there is a Partition, choose between Availability and Consistency.
>
> Else (when the system is running normally), choose between Latency and Consistency.

```
PACELC Decision Framework

Is there a partition?
      |
 +----+----+
 |         |
YES        NO
 |         |
 v         v
PAC       ELC
Pick:     Pick:
A or C    L or C

Examples:

System       During Partition    Normal Operation    PACELC
-------      ----------------    ----------------    ------
DynamoDB     Availability        Latency             PA/EL
Cassandra    Availability        Latency             PA/EL
MongoDB      Consistency         Consistency         PC/EC
PostgreSQL   Consistency         Consistency         PC/EC
(sync repl.)
```

This is more useful in practice because partitions are rare. Most of the time, your system is running normally, and the real trade-off is between latency and consistency.

PA/EL Systems (DynamoDB, Cassandra)

- During partition: Stay available, accept stale reads
- Normal operation: Prioritize low latency, serve reads from nearest replica (may be slightly stale)

PC/EC Systems (MongoDB with majority, PostgreSQL with sync replication)

- During partition: Reject requests to stay consistent
- Normal operation: Prioritize consistency even if it means higher latency (reads may need to go to the primary)

Eventual Consistency in Practice

Eventual consistency sounds scary, but it is everywhere. Here are patterns that make it work.

Read-Your-Own-Writes

After a user writes data, ensure that the same user sees the updated data on subsequent reads (even if other users see stale data briefly).

```
User writes "name = Alice"
         |
         v
+--------+--------+
|    Primary       |
| name = "Alice"   |
+--------+---------+
         |
         | Replication (async)
         v
+--------+---------+
|    Replica       |
| name = "Bob"     |  <-- Still has old value
+------------------+

Strategy: Route the user's reads to the primary
```

```
for a few seconds after a write.
```

Conflict Resolution Strategies

When two nodes accept conflicting writes during a partition, you need a strategy:

1. Last-Write-Wins (LWW): The write with the latest timestamp wins. Simple but can lose data.
2. Merge: Combine both writes (e.g., for shopping carts, merge items from both versions).
3. Application-level resolution: Show the conflict to the user and let them decide (like Git merge conflicts).

Anti-Entropy and Repair

Background processes periodically compare data across replicas and fix differences. Cassandra calls this "anti-entropy repair." It runs in the background to ensure eventual convergence.

Common Mistakes

1. Thinking CAP means you literally pick two and ignore one. In practice, you tune trade-offs. Systems are not purely CP or AP.
2. Ignoring partitions because "our network is reliable." Network partitions happen in every cloud provider, every data center, every system at scale. Design for them.
3. Choosing strong consistency everywhere "just to be safe." Strong consistency is expensive in latency and availability. Use it where it matters (money, inventory), not where it does not (likes, views).
4. Confusing CAP consistency with ACID consistency. CAP consistency means all nodes see the same data. ACID consistency means the database enforces rules (constraints, triggers). They are different concepts.
5. Thinking eventual consistency means "inconsistent forever." Eventual consistency means the system converges. The window of inconsistency is usually milliseconds to seconds.

Best Practices

1. Classify your data by consistency requirements. User account balance? Strong consistency. Like count? Eventual is fine. Make deliberate choices per data type.
2. Use tunable consistency when available. Databases like Cassandra and DynamoDB let you choose consistency per query. Use strong reads for critical paths and eventual reads for everything else.
3. Design for partition recovery. Have a clear strategy for what happens when a partition heals. How do you reconcile conflicting writes?
4. Use the PACELC model for real-world decisions. CAP only covers partitions. PACELC also covers normal operation, which is where your system spends 99.99% of its time.
5. Test partition behavior. Use chaos engineering tools (like Chaos Monkey or Toxiproxy) to simulate partitions and verify your system behaves as expected.
6. Document your consistency guarantees. Make it clear to your team what consistency level each service provides. This prevents bugs caused by wrong assumptions.

Quick Summary

The CAP theorem states that a distributed system can guarantee at most two of three properties: Consistency, Availability, and Partition Tolerance. Since partitions are unavoidable in distributed systems, the real choice is between consistency (CP) and availability (AP) during failures. The PACELC extension adds that during normal operation, the trade-off is between latency and consistency. Real systems like DynamoDB, Cassandra, and MongoDB offer tunable consistency, letting you make different choices for different data.

Key Points

- The CAP theorem limits distributed systems to two of three guarantees: Consistency, Availability, and Partition Tolerance.
- Network partitions are inevitable, so the practical choice is CP (consistency) vs AP (availability).
- CP systems reject requests during partitions to maintain correctness (e.g., banking).
- AP systems serve requests during partitions but may return stale data (e.g., social media).

- Consistency exists on a spectrum: linearizable, sequential, causal, and eventual.
- PACELC extends CAP to cover normal operation: the trade-off between latency and consistency.
- Real databases (DynamoDB, Cassandra, MongoDB) offer tunable consistency levels per query.
- Eventual consistency works well with patterns like read-your-own-writes and conflict resolution.
- CAP consistency (all nodes see the same data) is different from ACID consistency (enforcing rules).

Practice Questions

1. You are designing a system for an airline seat reservation. Should it be CP or AP? What happens if two users try to book the same seat during a network partition?
2. A social media platform shows post like counts. The count is slightly different for users in different regions. Is this a bug or expected behavior? What consistency model is at work?
3. Your Cassandra cluster has a replication factor of 5. You use QUORUM for both reads and writes. How many nodes must acknowledge each operation? Can you tolerate node failures?
4. Explain why a single-server PostgreSQL database does not face the CAP trade-off. What changes when you add replication?
5. Your system uses DynamoDB with eventual consistency reads. A user updates their profile and immediately refreshes the page but sees old data. How would you fix this?

Exercises

Exercise 1: Classify Real Systems

For each system below, determine whether it leans CP or AP, and explain why:

- A bank's core ledger system
- A DNS system
- A shopping cart service
- A distributed lock service (like ZooKeeper)

- A content delivery network

Think about what happens during a partition for each one.

Exercise 2: Design a Consistency Strategy

You are building a ride-sharing application. Different parts of the system have different consistency needs:

- Driver location updates (sent every 2 seconds)
- Ride fare calculation
- Payment processing
- Rider-driver matching

For each component, choose a consistency level (strong, causal, or eventual) and justify your choice. What databases or patterns would you use?

Exercise 3: Simulate a Partition

Set up a simple system with two data stores (even just two files on disk). Write a script that:

1. Writes data to both stores (simulating replication)
2. Simulates a partition (stop syncing)
3. Writes different data to each store
4. Restores the partition
5. Implements a reconciliation strategy (last-write-wins or merge)

Document what happens at each step and what data you end up with.

What Is Next?

Now that you understand the CAP theorem and how distributed systems make consistency trade-offs, you are ready to explore how organizations structure their systems. In the next chapter, we will dive into Microservices — how to break a large application into smaller, independent services and the new set of challenges that approach introduces.

Chapter 14: Microservices

What You Will Learn

- What microservices are and how they differ from monolithic architecture
- The benefits of microservices: independent deployment, scaling, and technology diversity
- The challenges: network complexity, data consistency, and debugging difficulty
- How to define service boundaries using bounded contexts from Domain-Driven Design
- Synchronous vs asynchronous inter-service communication (REST, gRPC, message queues)
- What a service mesh is and how the sidecar pattern works
- When a monolith is actually the better choice

Why This Chapter Matters

Microservices are one of the most discussed topics in modern software engineering. Companies like Netflix, Amazon, and Uber have built their platforms on microservices. But microservices are not free. They trade one set of problems (big codebase, slow deployments) for another (network failures, distributed debugging, data inconsistency).

If you adopt microservices without understanding the trade-offs, you will build a distributed monolith — all the complexity of microservices with none of the benefits. This chapter gives you the knowledge to make that decision wisely.

Monolith vs Microservices: The Big Picture

The Monolith: One Big Building

A monolithic application is a single deployable unit. All features live in one codebase, one process, one deployment.

Think of it as one large office building. Everyone works under the same roof. The accounting department, engineering, sales, and customer support all share the same hallways, the same cafeteria, and the same elevator.

```
+-------------------------------------------------------------+
|                    MONOLITH APPLICATION                       |
|                                                              |
|  +----------+  +----------+  +---------+  +----------+  |
|  |  User    |  |  Order   |  | Payment |  | Shipping |  |
|  |  Module  |  |  Module  |  | Module  |  | Module   |  |
|  +----+-----+  +----+-----+  +----+----+  +----+-----+  |
|       |             |             |             |        |
|  +----+-------------+-------------+-------------+----+   |
|  |              Shared Database                      |   |
|  +---------------------------------------------------+   |
|                                                          |
|  Single Codebase | Single Deployment | Single Process    |
+-------------------------------------------------------------+
```

Microservices: Separate Houses

A microservices architecture breaks the application into small, independent services. Each service owns its data, runs in its own process, and communicates with others over the network.

Think of it as a neighborhood of houses. Each house is independent. It has its own plumbing, its own electricity, and its own mailbox. Residents communicate by sending letters or making phone calls.

```
+------------+    +------------+    +------------+    +------------+
|  User      |    |  Order     |    |  Payment   |    |  Shipping  |
|  Service   |    |  Service   |    |  Service   |    |  Service   |
|            |    |            |    |            |    |            |
| +--------+ |    | +--------+ |    | +--------+ |    | +--------+ |
| |  Own   | |    | |  Own   | |    | |  Own   | |    | |  Own   | |
| |  DB    | |    | |  DB    | |    | |  DB    | |    | |  DB    | |
| +--------+ |    | +--------+ |    | +--------+ |    | +--------+ |
+------+-----+    +------+-----+    +------+-----+    +------+-----+
       |                |                |                |
       +--------+-------+--------+-------+--------+------+
                |                |                |
         +------+------+  +-----+------+  +------+------+
```

```
        |   REST API  |  |  gRPC      |  | Message     |
        |   Calls     |  |  Calls     |  | Queue       |
        +-------------+  +------------+  +-------------+

Each service: own codebase, own database, own deployment
```

Benefits of Microservices

1. Independent Deployment

Each service can be deployed independently. The payment team can ship a fix at 2 PM without waiting for the shipping team to finish their feature. No coordinated releases. No "deployment trains."

```
Monolith Deployment          Microservices Deployment

All teams must              Each team deploys
deploy together:            independently:

+-----------+               +------+  +------+  +------+
| User      |               | User |  | Order|  | Pay  |
| Order     |  --> Deploy   |      |  |      |  |      |
| Payment   |     ALL       | v2.1 |  | v3.0 |  | v1.5 |
| Shipping  |               +------+  +------+  +------+
+-----------+                  |          |         |
                            Deploy     Deploy    Deploy
                            Mon 2pm   Tue 10am  Wed 4pm
```

2. Independent Scaling

If your search service gets 100x more traffic than your payment service, you can scale search independently. No need to scale the entire application.

```
Scaling a Monolith           Scaling Microservices

Must scale everything:       Scale only what needs it:

+-------+ +-------+         +---+ +---+ +---+ +---+ +---+
| ALL   | | ALL   |         | S | | S | | S | | S | | S |  <-- 5 search
| code  | | code  |         +---+ +---+ +---+ +---+ +---+     instances
+-------+ +-------+
                              +---+                           <-- 1 payment
```

```
2 full copies                | P |                              instance
(wasteful)                   +---+
```

3. Technology Diversity

Each team can pick the best technology for their service. The search team can use Elasticsearch. The payment team can use a relational database. The analytics team can use Python while everyone else uses Java.

4. Fault Isolation

If the recommendation service crashes, the rest of the application keeps running. Users can still browse, search, and purchase. In a monolith, one bad module can bring down the entire application.

5. Team Autonomy

Small teams (2-pizza teams, as Amazon calls them) own their services end to end. They make their own technology choices, set their own release schedules, and are responsible for their own operations.

Challenges of Microservices

1. Network Complexity

In a monolith, function calls are local and fast. In microservices, they become network calls that can fail, time out, or be slow. Every call between services is a potential point of failure.

```
Monolith: In-Process Call     Microservices: Network Call

orderService.create(order)    POST http://order-service/orders

- Always succeeds              - Can time out
- Nanoseconds                  - Milliseconds
- No serialization             - JSON/Protobuf serialization
- No network issues            - Network can fail
- Type-safe                    - Schema mismatch possible
```

2. Data Consistency

In a monolith, you can use a database transaction to update multiple tables atomically. In microservices, each service has its own database. There are no cross-service transactions.

```
Monolith: Single Transaction

BEGIN TRANSACTION
  UPDATE orders SET status = 'confirmed'
  UPDATE inventory SET quantity = quantity - 1
  UPDATE payments SET status = 'charged'
COMMIT

------------------------------------------

Microservices: Distributed Operations

Order Service:    "Order confirmed"    --> Success
Inventory Service: "Reduce stock"      --> Success
Payment Service:   "Charge customer"   --> FAILED!

Now what? Order is confirmed, stock is reduced,
but payment failed. You need a saga pattern or
compensation logic to undo the partial work.
```

3. Debugging Difficulty

A single user request might touch 10 different services. When something goes wrong, you need distributed tracing to follow the request across services. Logs are spread across dozens of machines.

```
User Request: "Place Order"

API Gateway --> User Service --> Order Service --> Inventory Service
                                        |
                                        +--> Payment Service
                                        |
                                        +--> Notification Service
                                        |
                                        +--> Shipping Service

Where did the request fail? Which service was slow?
You need distributed tracing (Chapter 20) to find out.
```

4. Operational Overhead

Instead of deploying one application, you now deploy 50. Each needs monitoring, logging, health checks, CI/CD pipelines, and on-call rotations. The infrastructure cost is significant.

5. Testing Complexity

Integration testing becomes harder. How do you test that the order service works correctly with the payment service, the inventory service, and the shipping service — all together?

Defining Service Boundaries

The hardest part of microservices is deciding where to draw the lines. If you split too fine, you get "nanoservices" — services so small that they cannot do anything useful without calling other services. If you split too coarse, you get a distributed monolith.

Bounded Contexts from Domain-Driven Design (DDD)

Domain-Driven Design gives us the concept of bounded contexts. A bounded context is a boundary within which a particular model is defined and applicable.

Think of it this way: the word "account" means different things in different parts of a business.

- In User Management, an account is a login with email and password.
- In Billing, an account is a payment profile with credit card and invoices.
- In Analytics, an account is a set of usage metrics and behavior data.

Each of these is a separate bounded context, and each could be a separate microservice.

```
Bounded Contexts for an E-Commerce System

+-------------------+  +-------------------+  +-------------------+
|  User Context     |  |  Order Context    |  |  Payment Context  |
|                   |  |                   |  |                   |
|  - User profile   |  |  - Order          |  |  - Payment method |
|  - Authentication |  |  - Order items    |  |  - Transaction    |
|  - Preferences    |  |  - Order status   |  |  - Invoice        |
|                   |  |                   |  |  - Refund          |
|  "Account" =      |  |  "Account" =      |  |  "Account" =      |
|  login + email    |  |  customer who      |  |  payment profile  |
|                   |  |  placed orders     |  |                   |
+-------------------+  +-------------------+  +-------------------+
       |                      |                      |
       +--- User Service -----+--- Order Service -----+--- Payment Service
```

Guidelines for Drawing Boundaries

1. High cohesion within a service: Things that change together should live together.
2. Low coupling between services: Changing one service should rarely require changing another.
3. Align with team structure: Each service should be owned by a single team (Conway's Law).
4. One service, one business capability: A service should map to a business function, not a technical layer.

Inter-Service Communication

Synchronous Communication

The calling service sends a request and waits for a response.

REST (HTTP/JSON)

The most common approach. Simple, well-understood, works with any language.

```
Order Service                    Payment Service
+-------------+                  +-------------+
|             |  POST /payments  |             |
|  Create     | ---------------> |  Charge     |
|  Order      | <--------------- |  Customer   |
|             |  200 OK          |             |
+-------------+  {txn_id: 123}   +-------------+

Pros: Simple, widely supported, human-readable
Cons: Coupled, latency adds up, both must be online
```

gRPC (Protocol Buffers)

A high-performance RPC framework from Google. Uses binary serialization (Protocol Buffers) instead of JSON. Supports streaming.

```
Order Service                    Payment Service
+-------------+                  +-------------+
|             | ChargeCustomer() |             |
|  Create     | ---------------> |  Charge     |
|  Order      | <--------------- |  Customer   |
```

```
|             | PaymentResponse  |             |
+-------------+  (binary)        +-------------+

Pros: Fast (binary), type-safe (schema), streaming support
Cons: Not human-readable, requires code generation
```

Asynchronous Communication

The calling service sends a message and moves on. It does not wait for a response.

Message Queue

Services communicate through a message broker (RabbitMQ, Amazon SQS, Kafka).

```
Order Service         Message Queue          Payment Service
+-------------+      +-------------+         +-------------+
|             | ---> |  [Message]   | ------> |             |
|  Create     |      |  [Message]   |         |  Process    |
|  Order      |      |  [Message]   |         |  Payment    |
|             |      +-------------+          |             |
+-------------+                               +-------------+
     |
     v
Returns immediately
(does not wait for payment)

Pros: Decoupled, resilient (queue buffers messages), scalable
Cons: Eventual consistency, harder to debug, no immediate response
```

When to Use Each

```
Use Synchronous When:                Use Asynchronous When:

- You need an immediate response     - You can tolerate delay
- The operation is fast              - The operation is slow
- Failure should be visible          - Retries are acceptable
- Simple request/response            - Fan-out to many services

Examples:                            Examples:
- Get user profile                   - Send notification email
- Check inventory                    - Process payment (background)
- Authenticate user                  - Generate report
                                     - Update search index
```

Service Mesh and the Sidecar Pattern

As you add more microservices, cross-cutting concerns like load balancing, retries, circuit breaking, authentication, and observability become repetitive. Every service needs the same infrastructure code.

A service mesh handles these concerns outside of your application code.

The Sidecar Pattern

Each service gets a sidecar proxy — a small process that runs alongside it and handles all network communication.

```
Without Service Mesh                With Service Mesh (Sidecar)

+------------------+                +-------------------+--------+
| Order Service    |                | Order Service     | Sidecar|
|                  |                |                   | Proxy  |
| - Business logic |                | - Business logic  | - TLS  |
| - Retry logic    |                |                   | - Retry|
| - Circuit breaker|                |                   | - Auth |
| - TLS            |                |                   | - Trace|
| - Auth           |                |                   | - Load |
| - Tracing        |                |                   |   Bal. |
| - Load balancing |                |                   |        |
+------------------+                +-------------------+--------+
                                            |
All infrastructure code                     | Network calls go
mixed with business logic                   | through the sidecar
```

How a Service Mesh Works

```
+--------+-------+          +--------+-------+
| Service A      |          | Service B      |
| +------------+ |          | +------------+ |
| | App Code   | |          | | App Code   | |
| +-----+------+ |          | +-----+------+ |
|       |        |          |       ^        |
| +-----v------+ |          | +-----+------+ |
| | Sidecar    | | -------> | | Sidecar    | |
| | (Envoy)    | |  mTLS    | | (Envoy)    | |
| +------------+ |          | +------------+ |
+----------------+          +----------------+
       |                           |
       +----------+----------------+
                  |
          +-------v--------+
```

```
            |  Control Plane |
            |  (Istio, etc.) |
            |  - Config      |
            |  - Policies    |
            |  - Routing     |
            +----------------+

  Popular service meshes: Istio, Linkerd, Consul Connect
```

The service mesh has two parts:

1. Data plane: The sidecar proxies that handle actual traffic (usually Envoy).
2. Control plane: The management layer that configures the proxies (Istio, Linkerd).

When the Monolith Is Better

Microservices are not always the answer. Here are situations where a monolith is the better choice.

1. Early-Stage Startups

You are still figuring out what your product is. Service boundaries are unclear. A monolith lets you move fast and refactor easily.

2. Small Teams

If you have fewer than 10 engineers, the operational overhead of microservices will slow you down. A well-structured monolith is easier to manage.

3. Low Complexity

If your application is a CRUD app with straightforward business logic, microservices add unnecessary complexity.

4. Tight Coupling Is Natural

Some domains are inherently tightly coupled. Splitting them into services creates more network calls and coordination without real benefit.

```
When to Use What

+-------------------+----------------------------+
|  Monolith         |  Microservices             |
+-------------------+----------------------------+
| Small team (<10)  | Large team (50+)           |
| New product       | Mature product             |
| Unclear boundaries| Clear domain boundaries    |
| Simple domain     | Complex domain             |
| Speed matters most| Scale matters most         |
| Single deployment | Independent deployments    |
+-------------------+----------------------------+

Start with a monolith. Split when you feel the pain.
```

The Modular Monolith: A Middle Ground

A modular monolith is a single deployment unit with clear internal boundaries. Modules communicate through well-defined interfaces, not direct database access.

```
+-----------------------------------------------------------+
|                  MODULAR MONOLITH                          |
|                                                           |
|  +----------+  +----------+  +---------+  +----------+  |
|  |  User    |  |  Order   |  | Payment |  | Shipping |  |
|  |  Module  |  |  Module  |  | Module  |  | Module   |  |
|  |          |  |          |  |         |  |          |  |
|  | Own DB   |  | Own DB   |  | Own DB  |  | Own DB   |  |
|  | schema   |  | schema   |  | schema  |  | schema   |  |
|  +----+-----+  +----+-----+  +----+----+  +----+-----+  |
|       |             |             |             |        |
|  Communication through defined interfaces only           |
|  (No direct database access across modules)              |
+-----------------------------------------------------------+

Benefits: Clear boundaries + simple deployment
Migration: Easy to extract modules into services later
```

Common Mistakes

1. Starting with microservices on day one. You do not know your domain well enough. Start with a monolith and split when you understand the boundaries.
2. Creating a distributed monolith. If every service must call 5 other services to handle a request, you have not actually decoupled anything. You just made it harder to deploy.

3. Sharing databases between services. This creates tight coupling. If two services share a table, changing that table requires coordinating both teams.
4. Making services too small. A service that does one tiny thing and always needs to call other services is not a service — it is a function. Keep services at the level of business capabilities.
5. Ignoring operational requirements. Microservices need CI/CD, monitoring, logging, tracing, and alerting for every service. Budget for this infrastructure before you start splitting.
6. Synchronous chains. Service A calls B, which calls C, which calls D. If any service in the chain is slow or down, the entire request fails. Use async communication where possible.

Best Practices

1. Start with a well-structured monolith. Use clear module boundaries from the beginning. This makes future splitting easier.
2. One service, one database. Each service owns its data. No sharing tables. If another service needs your data, expose it through an API.
3. Design for failure. Every network call can fail. Use circuit breakers, retries with backoff, and timeouts.
4. Use asynchronous communication for non-critical paths. Sending a notification email after an order does not need to be synchronous.
5. Invest in observability early. Distributed tracing, centralized logging, and health checks are not optional. Build them from the start.
6. Automate everything. Each service needs its own CI/CD pipeline, automated tests, and deployment process. Manual processes do not scale with 50 services.
7. Define API contracts. Use OpenAPI specs or Protocol Buffer definitions. Version your APIs. Never make breaking changes without a migration path.

Quick Summary

Microservices break a large application into small, independent services that communicate over the network. They enable independent deployment, independent scaling, and technology diversity. But they introduce network complexity, data consistency challenges, and operational overhead. Service boundaries should align with business capabilities using bounded contexts

from Domain-Driven Design. Communication can be synchronous (REST, gRPC) or asynchronous (message queues). A service mesh handles cross-cutting concerns like retries, authentication, and tracing. For many teams, a well-structured monolith or modular monolith is the better starting point.

Key Points

- A monolith is a single deployable unit; microservices are many small, independent services.
- Microservices enable independent deployment, scaling, and technology choices per team.
- The main challenges are network failures, data consistency across services, and operational complexity.
- Bounded contexts from DDD help define natural service boundaries.
- Synchronous communication (REST, gRPC) is simpler but creates coupling; asynchronous (message queues) is more resilient.
- A service mesh with sidecar proxies handles cross-cutting concerns outside application code.
- A modular monolith offers clear boundaries without the operational overhead of microservices.
- Start with a monolith and migrate to microservices only when you feel specific pain points.

Practice Questions

1. Your monolith takes 45 minutes to deploy and a bug in the payment module brought down the entire application last week. Should you switch to microservices? What factors would influence your decision?
2. Two teams need to share customer data. Team A owns the User Service and Team B owns the Order Service. How should Team B access customer data without creating tight coupling?
3. The checkout flow in your microservices system involves 6 synchronous service calls in sequence. The 95th percentile latency is 3 seconds. How would you reduce this?

4. Your company has 5 engineers and is building an MVP. A senior engineer insists on using microservices. What counter-arguments would you present?
5. Explain the difference between a distributed monolith and a true microservices architecture. What are the symptoms of a distributed monolith?

Exercises

Exercise 1: Decompose a Monolith (On Paper)

Take a familiar application (e-commerce, social media, or ride-sharing) and identify:

- 5-8 potential microservices
- The data each service would own
- The API each service would expose
- The communication pattern between services (sync vs async)

Draw the architecture diagram showing services, databases, and communication flows.

Exercise 2: Compare Communication Patterns

Build two versions of a simple order system:

Version 1 (Synchronous):

- Order Service calls Payment Service via REST
- If payment fails, return error to user immediately

Version 2 (Asynchronous):

- Order Service publishes "OrderCreated" event to a message queue
- Payment Service subscribes and processes payment
- Payment Service publishes "PaymentCompleted" or "PaymentFailed" event

Compare: What happens when the Payment Service is down for 5 minutes? How does each version behave?

Exercise 3: Service Boundary Analysis

You are building a food delivery application. The product manager gives you these features:

- User registration and login
- Restaurant menu browsing

- Order placement
- Real-time delivery tracking
- Payment processing
- Driver assignment
- Restaurant dashboard
- Customer reviews

Group these features into microservices. For each service, define what data it owns and which other services it needs to communicate with. Justify your grouping.

What Is Next?

Now that you understand the trade-offs between monoliths and microservices, the next chapter covers the practical process of migrating from a monolith to microservices. You will learn the Strangler Fig pattern, database decomposition strategies, and how to avoid the pitfalls of a big-bang rewrite.

Chapter 15: Migrating from Monolith to Microservices

What You Will Learn

- The Strangler Fig pattern for gradual migration
- How to identify seams in a monolith — the natural places to split
- Database decomposition strategies and shared data challenges
- How to introduce an API gateway during migration
- Using feature flags to control the migration safely
- The dual-write problem and why it causes data inconsistency
- Event-driven decomposition as a migration strategy
- A real-world migration story walkthrough from start to finish

Why This Chapter Matters

The decision to move from a monolith to microservices is not the hard part. The hard part is doing the migration without breaking your running system. You cannot stop the business for six months while you rewrite everything.

Most failed migrations happen because teams try a "big bang" rewrite — throw away the old code and build everything new. History shows this approach fails far more often than it succeeds. The Netscape browser rewrite took three years and nearly killed the company.

This chapter teaches you the safe, incremental approach. You will learn how to migrate piece by piece, validate each step, and always have a rollback path.

The Strangler Fig Pattern

The Strangler Fig is a tropical plant that grows around an existing tree. Over time, it slowly replaces the host tree. Eventually, the original tree is completely replaced, but at every point during the process, the combined system is functional.

This is exactly how you should migrate a monolith.

```
Phase 1: Monolith handles everything

+------------------+
|     Monolith     |
|                  |
| [Users] [Orders] |
| [Pay]  [Search]  |
| [Ship] [Notify]  |
+------------------+

Phase 2: Extract one service, route traffic

       +--------+
       | Router |
       +---+----+
           |
   +-------+--------+
   |                |
+--+---+     +------+------+
| New  |     |  Monolith   |
| User |     |             |
| Svc  |     | [Orders]    |
|      |     | [Pay][Ship] |
+------+     | [Search]    |
             | [Notify]    |
             +-------------+

Phase 3: Extract more services

       +--------+
       | Router |
       +---+----+
           |
   +---+---+---+---+
   |   |       |   |
+--+-+ +--+-+ ++-+ +------+
|User| |Ord | |Pay| |Mono  |
|Svc | |Svc | |Svc| |      |
+----+ +----+ +---+ |[Ship]|
                     |[Srch]|
                     +------+

Phase 4: Monolith is gone
```

```
        +--------+
        | Router |
        +---+----+
            |
  +-+---+---+---+---+---+
  | |   |       |   |   |
 ++-+ +--+-+ ++-+ ++-+ ++-+
 |Us| |Ord | |Py| |Sh| |Sr|
 +--+ +----+ +--+ +--+ +--+
```

How It Works

1. Identify a piece of functionality in the monolith to extract.
2. Build the new service alongside the monolith.
3. Route traffic to the new service for that functionality.
4. Verify the new service works correctly.
5. Remove the old code from the monolith.
6. Repeat for the next piece.

The key insight: at every step, the system works. If the new service fails, you route traffic back to the monolith. There is always a rollback path.

Step 1: Identify Seams

A seam is a natural boundary in your monolith where the code can be split with minimal changes. Not all code is equally easy to extract.

Finding Good Seams

Look for code that:

- Has few dependencies on other parts of the monolith
- Communicates with the rest of the system through a small number of well-defined interfaces
- Owns its own data (or can be given its own data)
- Changes frequently (extracting it gives the most deployment speed)

```
  Dependency Analysis of a Monolith
```

```
+--------+     +--------+     +--------+
| User   |<----| Order  |---->| Payment|
| Module |     | Module |     | Module |
+---+----+     +---+----+     +--------+
    |              |
    |         +----+----+
    |         | Shipping|
    |         | Module  |
    |         +---------+
    |
+---+--------+
| Notification|    <--- Good candidate!
| Module      |         - Few incoming dependencies
|             |         - Only needs user data
+-----------+          - Changes often (new channels)

+-------------+
| Search      |    <--- Also good!
| Module      |         - Read-only
|             |         - Can use its own data store
+-------------+
```

What to Extract First

Start with the easiest, lowest-risk service. This is usually:

1. Leaf services — services that are called by others but do not call others (e.g., notification, search, reporting).
2. Read-heavy services — services that mostly read data and rarely write (e.g., product catalog, user profiles).
3. Independently deployable features — features that change on a different cadence than the rest of the system.

Do NOT start with the core business logic. Do not extract the order service or payment service first. Start at the edges.

Step 2: Database Decomposition

The database is the hardest part of any migration. The monolith typically has one big database with foreign keys between tables. Splitting the database requires careful planning.

The Problem

```
Monolith Database (Shared)

+-------------------------------------------+
|                                           |
|  users          orders         payments   |
|  +------+      +--------+    +--------+  |
|  | id   |<-----| user_id|    | order_id|->|
|  | name |      | total  |    | amount  |  |
|  | email|      | status |    | status  |  |
|  +------+      +---+----+    +--------+  |
|                     |                     |
|              order_items                  |
|              +----------+                 |
|              | order_id |                 |
|              | product  |                 |
|              | quantity |                 |
|              +----------+                 |
|                                           |
|  Foreign keys everywhere.                 |
|  Can't just split tables into separate    |
|  databases without breaking joins.        |
+-------------------------------------------+
```

Strategy 1: Shared Database (Temporary)

As a first step, let the new service access the monolith's database. This is not the final state, but it reduces risk during migration.

```
+----------+     +----------+
| New User |     | Monolith |
| Service  |     |          |
+----+-----+     +----+-----+
     |                |
     +-------+--------+
             |
      +------+------+
      |  Shared DB  |
      +-------------+

Temporary! Move to separate databases later.
```

Strategy 2: Database View or API

Create a read-only view or API that the new service uses to access data it does not own.

```
+----------+                    +----------+
| Order    |  GET /users/123    | User     |
```

```
| Service  | -----------------> | Service  |
+----+-----+                    +----+-----+
     |                               |
+----+-----+                    +----+-----+
| Order DB |                    | User DB  |
+----------+                    +----------+

Order Service no longer joins against the users table.
It calls the User Service API to get user data.
```

Strategy 3: Data Duplication

Copy the data the new service needs into its own database. Keep it synchronized through events or periodic sync.

```
+----------+     Event: UserUpdated     +----------+
| User     | --------------------------> | Order    |
| Service  |                             | Service  |
+----+-----+                             +----+-----+
     |                                        |
+----+-----+                             +----+-----+
| User DB  |                             | Order DB |
| (source  |                             | (has a   |
|  of truth|                             |  copy of |
|  for     |                             |  user    |
|  users)  |                             |  name,   |
+----------+                             |  email)  |
                                         +----------+
```

Handling Foreign Keys Across Services

When tables move to different databases, you lose foreign key constraints. Replace them with:

1. Application-level validation: The Order Service calls the User Service to verify a user exists before creating an order.
2. Eventual consistency: Accept that references might temporarily point to non-existent entities. Handle this gracefully.
3. Soft references: Store IDs as strings without database-enforced constraints. Validate at the application level.

Step 3: Introduce an API Gateway

An API gateway sits between clients and your services. During migration, it is essential for routing traffic between the monolith and new services.

```
Before API Gateway          After API Gateway

Client                      Client
  |                           |
  v                         +-v---------+
Monolith                    | API       |
                            | Gateway   |
                            +--+---+----+
                               |   |
                        +------+   +------+
                        |                 |
                    +---v---+      +-----v-----+
                    | New   |      | Monolith  |
                    | User  |      | (handles  |
                    | Svc   |      |  the rest)|
                    +-------+      +-----------+

The API Gateway routes:
/api/users/*  --> User Service
/api/*        --> Monolith (everything else)
```

Benefits During Migration

1. Transparent routing: Clients do not need to know which backend handles their request.
2. Gradual cutover: Route 10% of user traffic to the new service, then 50%, then 100%.
3. Rollback: If the new service has issues, route traffic back to the monolith instantly.
4. Cross-cutting concerns: Authentication, rate limiting, and logging in one place.

Step 4: Feature Flags for Safe Migration

Feature flags let you control which code path runs without deploying new code. They are essential for safe migrations.

```
if feature_flag("use_new_user_service"):
    response = call_new_user_service(request)
else:
    response = call_monolith_user_module(request)
```

Migration with Feature Flags

```
Phase 1: Flag off (0% new service)
All traffic goes to monolith.

Phase 2: Flag on for internal users (1%)
Employees test the new service.

Phase 3: Flag on for 10% of users
Monitor error rates, latency, correctness.

Phase 4: Flag on for 50% of users
Continue monitoring.

Phase 5: Flag on for 100% of users
New service handles all traffic.

Phase 6: Remove old code from monolith
Clean up the flag.

At any phase, if something goes wrong:
Turn off the flag --> instant rollback to monolith
```

Shadow Testing (Dark Launching)

Run both the old and new code paths simultaneously. Compare results. Only return the monolith's response to the user.

```
+----------+
|  Request |
+----+-----+
     |
+----v-----+
| Feature  |
| Flag     |
+--+---+---+
   |   |
   |   +------ Shadow mode ------+
   |                             |
+--v--------+            +-------v------+
| Monolith  |            | New Service  |
| (returns  |            | (result is   |
|  response)|            |  compared,   |
+--+--------+            |  not returned)|
   |                     +-------+------+
   v                             |
Client gets                Compare results.
monolith response.         Log differences.
```

```
                                Fix bugs before
                                real cutover.
```

The Dual-Write Problem

A common mistake during migration is writing to two databases simultaneously. This seems logical but causes serious consistency issues.

The Problem

```
Order Service needs to:
1. Write to the old monolith database
2. Write to the new order database

+-------------+
| Order       |
| Service     |
+--+------+---+
   |      |
   v      v
+--+--+ +-+----+
| Old | | New  |
| DB  | | DB   |
+-----+ +------+

What if step 1 succeeds but step 2 fails?
Now the databases are inconsistent.

What if step 2 succeeds but step 1 fails?
Same problem.

What if both succeed but with different timestamps?
A concurrent read might see different states.
```

Why Dual Writes Fail

1. There is no distributed transaction between two different databases (in practice).
2. If the process crashes between the two writes, one database has the data and the other does not.
3. Network issues can cause one write to succeed and the other to fail.

The Solution: Change Data Capture (CDC)

Write to one database (the source of truth). Use Change Data Capture to stream changes to the other database.

```
+-------------+
| Order       |
| Service     |
+------+------+
       |
       v  (single write)
+------+------+
|  Source DB  |
+------+------+
       |
       v  (CDC: Debezium, etc.)
+------+------+
| Change      |
| Data        |
| Capture     |
+------+------+
       |
       v
+------+------+
| Message     |
| Broker      |
| (Kafka)     |
+------+------+
       |
  +----+----+
  |         |
  v         v
+-+---+ +--+----+
| New | | Search|
| DB  | | Index |
+-----+ +------+

One source of truth. Changes flow to other systems.
No dual-write inconsistency.
```

Event-Driven Decomposition

Instead of making synchronous calls between the monolith and new services, use events. The monolith publishes events when things happen. New services subscribe to those events.

How It Works

```
Step 1: Add events to the monolith

+-----------------------------------------+
|              Monolith                    |
|                                         |
|  [Order Module]                         |
|       |                                 |
|       | When order is created:          |
|       | publish("OrderCreated", data)   |
|       |                                 |
+-------+---------------------------------+
        |
        v
+-------+-------+
|  Message       |
|  Broker        |
+-------+--------+
        |
        v
+-------+--------+
| New Notification|
| Service         |
| (subscribes to  |
|  OrderCreated)  |
+-----------------+

Step 2: Move the notification logic out of the monolith
Step 3: Repeat for other events and services
```

Benefits

1. Low coupling: The monolith does not need to know about the new services.
2. Gradual migration: You can add new services without changing the monolith (just add subscribers).
3. Reversible: If a new service fails, the monolith still works. Events queue up and are processed when the service recovers.

Real-World Migration Story: Online Bookstore

Let us walk through a complete migration for a fictional online bookstore called PageTurner.

The Starting Point

PageTurner is a monolith built in Java with a single PostgreSQL database. It handles:

- User accounts and authentication
- Book catalog and search
- Shopping cart
- Order processing
- Payment
- Email notifications
- Reviews and ratings

The team has 30 engineers. Deployments take 2 hours and happen once a week. A bug in the review feature last month brought down the entire site for 4 hours.

Phase 1: Assess and Plan (Week 1-2)

The team maps dependencies between modules:

```
Dependency Map

Auth ---------> User
                  ^
                  |
Catalog -------> |
   ^             |
   |             |
Search           |
                 |
Cart ----------> Catalog
   |
   v
Order ---------> Payment
   |              |
   v              v
Notification    Notification

Reviews ------> User, Catalog
```

They identify the best candidates for extraction:

1. Notifications (leaf node, no dependents)
2. Search (read-only, can have its own data store)
3. Reviews (low coupling, independent feature)

Phase 2: Set Up Infrastructure (Week 3-4)

- Deploy an API gateway (Kong)
- Set up a message broker (RabbitMQ)

- Set up CI/CD pipelines for new services
- Add monitoring and logging infrastructure
- Set up feature flags (LaunchDarkly)

Phase 3: Extract Notification Service (Week 5-8)

1. Add event publishing to the monolith's order module: publish `OrderCreated`, `OrderShipped` events.
2. Build a new Notification Service that subscribes to these events.
3. Test with shadow mode — both old and new notification code send emails. Compare.
4. Use feature flag to switch from old to new.
5. Remove notification code from the monolith.

```
Before:                          After:

+----------+                     +----------+     +-----------+
| Monolith |                     | Monolith | --> | Rabbit MQ |
|          |                     | (publishes     |           |
| sends    |                     |  events)  |    +-+---+-----+
| emails   |                     +----------+      |   |
| directly |                                        v   v
+----------+                     +-----------+ +---+-------+
                                 | Notif Svc | | Other     |
                                 | (sends    | | future    |
                                 |  emails)  | | services  |
                                 +-----------+ +-----------+
```

Phase 4: Extract Search Service (Week 9-12)

1. Set up Elasticsearch cluster.
2. Build a Search Service that indexes book data from the monolith database (using CDC).
3. Route search API calls through the API gateway to the new service.
4. Use feature flag: 10% -> 50% -> 100%.
5. Remove search code from the monolith.

Phase 5: Extract Reviews Service (Week 13-16)

1. Create a separate reviews database.
2. Migrate review data from the monolith database.
3. Build Reviews Service with its own API.
4. Update the monolith to call the Reviews Service API instead of querying the reviews table directly.

5. Feature flag rollout.
6. Remove reviews code and table from the monolith.

Phase 6: Continue Extracting (Month 5+)

With three successful extractions, the team has the infrastructure, processes, and confidence to continue. They tackle Cart, Catalog, and eventually Order and Payment — the harder services with more dependencies.

Results After 6 Months

- Deployment frequency: From weekly to multiple times per day (per service)
- Deployment time: From 2 hours to 15 minutes
- A review service outage no longer affects the rest of the site
- The search team switched from PostgreSQL full-text search to Elasticsearch without affecting other teams
- The notification team added push notifications without touching the monolith

---

Common Mistakes

1. Big bang rewrite. Throwing away the monolith and rewriting everything from scratch. This almost always fails. The Strangler Fig pattern is safer.
2. Extracting the hardest service first. Start with the easiest, lowest-risk service to build confidence and infrastructure.
3. Dual writes without a consistency strategy. Writing to two databases simultaneously leads to inconsistency. Use CDC or events instead.
4. No rollback path. Every migration step should be reversible. If the new service fails, you should be able to route traffic back to the monolith.
5. Migrating without feature flags. Feature flags give you fine-grained control over the migration. Without them, you are doing big-bang cutover at a smaller scale.
6. Ignoring the database. Extracting code into a new service but still reading from the monolith's database is not a real migration. Plan for database decomposition from the start.
7. Not investing in infrastructure first. CI/CD, monitoring, logging, and an API gateway need to exist before you start extracting services.

Best Practices

1. Follow the Strangler Fig pattern. Migrate incrementally. Always keep the system working.
2. Extract leaf services first. Start with services that have the fewest dependencies.
3. Use events for decoupling. Publish events from the monolith. New services subscribe to events instead of calling the monolith directly.
4. One source of truth for data. During migration, be clear about which system owns which data. Use CDC to propagate changes.
5. Shadow test before cutover. Run the old and new code paths in parallel. Compare results. Fix discrepancies before switching traffic.
6. Set up infrastructure before migrating. API gateway, message broker, CI/CD, monitoring, and feature flags should all be in place before you extract the first service.
7. Communicate the plan. The whole team should understand the migration strategy, the current phase, and the rollback plan.

Quick Summary

Migrating from a monolith to microservices should be gradual, not a big-bang rewrite. The Strangler Fig pattern lets you extract services one at a time while keeping the system functional. Start by identifying seams (natural boundaries) in the monolith, beginning with leaf services. Database decomposition is the hardest part — use strategies like API access, data duplication, and Change Data Capture to avoid dual-write problems. An API gateway enables transparent routing between the monolith and new services. Feature flags and shadow testing provide safe rollout and instant rollback.

Key Points

- The Strangler Fig pattern is the safest way to migrate: extract one service at a time, always keeping the system functional.
- Start with leaf services (notifications, search) that have the fewest dependencies.
- Database decomposition is the hardest part — plan for it early.
- Dual writes cause data inconsistency. Use Change Data Capture or events instead.

- An API gateway routes traffic between the monolith and new services transparently.
- Feature flags enable gradual rollout and instant rollback.
- Shadow testing (dark launching) lets you compare old and new implementations before switching.
- Event-driven decomposition decouples new services from the monolith.
- Always have a rollback path at every step of the migration.

Practice Questions

1. Your monolith has 15 modules. Which three characteristics would make a module a good candidate for first extraction?
2. During migration, your new Order Service needs customer data owned by the User module (still in the monolith). What are three ways to provide this data, and what are the trade-offs of each?
3. You are using dual writes to keep the old and new databases in sync. A developer reports that some orders appear in the new database but not the old one. What went wrong, and how would you fix it?
4. Your team wants to skip the API gateway and have clients call the new services directly. What problems will this cause during migration?
5. Explain how feature flags help with migration safety. What happens if you migrate without them?

Exercises

Exercise 1: Migration Plan

You have a monolith with these modules: Authentication, User Profile, Product Catalog, Search, Shopping Cart, Checkout, Payment, Shipping, Reviews, Recommendations, Notifications, Admin Dashboard.

Create a migration plan:

- Map the dependencies between modules
- Rank the modules by extraction difficulty (1 = easiest)
- Plan the order of extraction (which first, which last?)

- Identify which data needs to be shared and how

Exercise 2: Solve the Dual-Write Problem

You are migrating the Order module. During the transition, both the monolith and the new Order Service need to write to their respective databases. Design a solution using Change Data Capture that:

- Has one source of truth
- Keeps both databases synchronized
- Handles failures gracefully
- Can be rolled back if needed

Draw the architecture diagram and describe the data flow.

Exercise 3: Feature Flag Rollout Plan

Design a feature flag rollout plan for migrating the Search module:

- Define 5 rollout phases with percentage of traffic
- For each phase, list what you monitor and what would trigger a rollback
- Define the criteria for moving from one phase to the next
- Include a timeline estimate

What Is Next?

Now that you know how to break apart a monolith, the next chapter explores Event-Driven Architecture in depth. You will learn about event sourcing, CQRS, saga patterns, and how to build systems where services communicate through events rather than direct calls.

Chapter 16: Event-Driven Architecture

What You Will Learn

- The difference between events and commands, and when to use each
- What event sourcing is and how it stores state as a sequence of events
- How CQRS separates reads from writes and why events make it powerful
- What an event bus and event broker are and how they differ
- Choreography vs orchestration: two ways to coordinate services
- The saga pattern for managing distributed transactions
- How eventual consistency works in event-driven systems
- Why idempotent consumers are critical and how to build them
- What dead letter queues do in event-driven pipelines
- How to design an order processing pipeline using events

Why This Chapter Matters

Imagine a wedding. The bride and groom do not personally call every vendor to coordinate. They do not phone the florist, then the caterer, then the DJ one by one, waiting for each to confirm before calling the next. Instead, they send out invitations. Each vendor receives the invitation and independently prepares their part. The florist arranges flowers. The caterer prepares food. The DJ sets up music. Nobody waits for anyone else. Everyone reacts to the same event: "the wedding is happening."

This is event-driven architecture. Instead of services calling each other directly, they emit events: "order placed," "payment received," "item shipped." Other services listen for these events and react independently. This creates systems that are loosely coupled, highly scalable, and resilient to failure. In system design interviews, event-driven architecture shows

up constantly because it is how real-world companies like Amazon, Uber, and Netflix handle millions of operations per second.

Events vs Commands

The most fundamental concept in event-driven architecture is the difference between events and commands.

What Is an Event?

An event is a record of something that already happened. It is a fact. It cannot be rejected. It uses past tense.

- `OrderPlaced` -- an order was placed
- `PaymentReceived` -- a payment was received
- `UserRegistered` -- a user signed up
- `ItemShipped` -- an item was shipped

Events are immutable. You cannot undo the fact that an order was placed. You can place a cancellation, but the original event still happened.

What Is a Command?

A command is a request to do something. It has not happened yet. It can be accepted or rejected. It uses imperative tense.

- `PlaceOrder` -- please place this order
- `ProcessPayment` -- please process this payment
- `RegisterUser` -- please register this user
- `ShipItem` -- please ship this item

```
Commands vs Events:

COMMAND (Request)                    EVENT (Fact)
+------------------+                +------------------+
| PlaceOrder       |                | OrderPlaced      |
| "Please do this" |    ------>     | "This happened"  |
| Can be rejected  |                | Cannot be undone |
| Imperative tense |                | Past tense       |
+------------------+                +------------------+
```

```
Flow:
+---------+   Command    +---------+   Event      +---------+
| Client  |  PlaceOrder  | Order   |  OrderPlaced | Payment |
|         | -----------> | Service | -----------> | Service |
+---------+              +---------+              +---------+
                              |
                              |  Event: OrderPlaced
                              v
                         +---------+
                         | Notif.  |
                         | Service |
                         +---------+

The command goes TO a specific service.
The event goes FROM a service to anyone listening.
```

Key Differences

Aspect	Command	Event
Direction	Sent TO a specific service	Broadcast FROM a service
Tense	Imperative (do this)	Past tense (this happened)
Can be rejected?	Yes	No (it already happened)
Coupling	Sender knows the receiver	Sender does not know who listens
Cardinality	One sender, one receiver	One sender, many receivers

When to Use Each

Use commands when you need a specific service to do something and you care about the result. Use events when you want to notify the rest of the system that something happened and let each service decide how to react.

Event Sourcing

The Traditional Approach: Storing Current State

In a traditional system, you store the current state of an entity. When something changes, you update the record in place.

```
Traditional State Storage:

User places order:    UPDATE orders SET status = 'placed'    WHERE id = 123
Payment received:     UPDATE orders SET status = 'paid'      WHERE id = 123
Item shipped:         UPDATE orders SET status = 'shipped'   WHERE id = 123

Database at any time:
+----+--------+---------+
| id | amount | status  |
+----+--------+---------+
| 123| $49.99 | shipped |  <-- Only the current state
+----+--------+---------+

Question: When was the payment received? How long between
placing the order and shipping? What was the original address
before the customer changed it?

Answer: You do not know. The history is gone.
```

The Event Sourcing Approach: Storing Events

With event sourcing, you never update or delete records. Instead, you store every event that happened to an entity. The current state is derived by replaying all events.

Think of it like a bank account. Your bank does not just store your current balance. It stores every transaction: every deposit, every withdrawal, every transfer. Your balance is the result of replaying all those transactions from the beginning.

```
Event Sourcing Storage:

Event Store for Order #123:
+----+---------------------+-----------------------------------+
| #  | Event Type          | Data                              |
+----+---------------------+-----------------------------------+
| 1  | OrderCreated        | {id:123, items:[...], total:49.99}|
| 2  | PaymentReceived     | {id:123, method:"visa", at:"10:01"}|
| 3  | AddressChanged      | {id:123, new:"456 Oak St"}        |
| 4  | ItemPicked          | {id:123, warehouse:"WH-East"}     |
| 5  | ItemShipped         | {id:123, carrier:"FedEx"}         |
+----+---------------------+-----------------------------------+

Current state = replay events 1 through 5

Now you know:
- When payment was received (event #2, 10:01 AM)
- The original address before the change
```

```
  - Which warehouse picked the item
  - The complete history of everything that happened
```

How Replay Works

To get the current state of an entity, you start with an empty state and apply each event in order.

```
Rebuilding State from Events:

Start:   {}
Event 1: OrderCreated    --> {id:123, status:"created", total:49.99}
Event 2: PaymentReceived --> {id:123, status:"paid", total:49.99}
Event 3: AddressChanged  --> {id:123, status:"paid", address:"456 Oak St"}
Event 4: ItemPicked      --> {id:123, status:"picking", warehouse:"WH-East"}
Event 5: ItemShipped     --> {id:123, status:"shipped", carrier:"FedEx"}

Final state: {id:123, status:"shipped", total:49.99,
              address:"456 Oak St", warehouse:"WH-East",
              carrier:"FedEx"}
```

Benefits of Event Sourcing

1. Complete audit trail. You know exactly what happened, when, and in what order. This is critical for financial systems, healthcare, and compliance.
2. Time travel. You can rebuild the state of any entity at any point in time by replaying events up to that moment.
3. Debugging. When something goes wrong, you can replay the exact sequence of events that caused the bug.
4. New projections. When you add a new feature that needs historical data, you can replay all events to build new views. No backfilling needed.

Challenges of Event Sourcing

1. Complexity. It is a fundamentally different way of thinking about data. Most developers are trained to think in terms of current state.
2. Event schema evolution. What happens when you need to change the structure of an event? You still need to be able to replay old events.
3. Performance. Replaying thousands of events to rebuild state can be slow. You solve this with snapshots: periodically save the current state so you only need to replay events after the snapshot.

```
Snapshots for Performance:

Without snapshots: Replay all 10,000 events  (SLOW)

Event 1 -> Event 2 -> ... -> Event 10,000
|<------------- Replay all ------------->|

With snapshots: Replay only events after the snapshot (FAST)

Event 1 -> ... -> Event 9,500 -> [SNAPSHOT] -> Event 9,501 -> ... -> 10,000
                                 |<---- Replay only 500 events ---->|
```

CQRS: Command Query Responsibility Segregation

CQRS is a pattern that separates the read side (queries) from the write side (commands) of your application. It pairs naturally with event sourcing.

The Problem with a Single Model

In a traditional application, you use the same database tables for both reading and writing. But reads and writes have very different requirements.

- Writes need strong consistency, validation, and business rules.
- Reads need speed, denormalized data, and flexible queries.

Trying to optimize the same model for both creates compromises. Your write model becomes bloated with read-specific columns. Your read queries become slow because the data is normalized for writes.

How CQRS Works

CQRS splits your system into two sides.

```
CQRS Architecture:

                         +-------------------+
                         |                   |
         Commands        |   WRITE MODEL     |
User  ----------------> |   (Command Side)  |
                         |                   |
                         | - Validates       |
```

```
                          | - Enforces rules  |
                          | - Stores events   |
                          +--------+----------+
                                   |
                                   | Events published
                                   v
                          +-------------------+
                          |   EVENT BUS       |
                          +--------+----------+
                                   |
                                   v
                          +-------------------+
                          |                   |
          Queries         |   READ MODEL      |
User  <----------------   |   (Query Side)    |
                          |                   |
                          | - Denormalized    |
                          | - Fast queries    |
                          | - Multiple views  |
                          +-------------------+

Write Side:                     Read Side:
- Normalized tables             - Denormalized views
- Complex validation            - Simple lookups
- Event store                   - Optimized for queries
- Strong consistency            - Eventually consistent
```

CQRS with Events

When the write side processes a command, it emits events. The read side listens for those events and updates its denormalized views.

```
CQRS + Event Sourcing Flow:

1. User sends command: PlaceOrder(items, address)

2. Write side:
   - Validates order
   - Checks inventory
   - Stores event: OrderPlaced

3. Event bus delivers OrderPlaced to:
   - Read model updater (updates order dashboard)
   - Inventory service (reserves items)
   - Email service (sends confirmation)

4. User queries the read side:
   - GET /orders/123
   - Returns denormalized order with items, status, tracking
   - Served from a read-optimized store (Redis, Elasticsearch)
```

When to Use CQRS

CQRS adds complexity. Use it when:

- Read and write patterns are very different (many more reads than writes, or vice versa)
- You need different data models for querying (search, analytics, dashboards)
- You need to scale reads and writes independently
- You are already using event sourcing

Do not use CQRS for simple CRUD applications. It is overkill.

Event Bus and Event Broker

An event bus or event broker is the infrastructure that transports events from producers to consumers.

Event Bus (In-Process)

An event bus is a lightweight, often in-process mechanism for routing events. Think of it as a bulletin board in an office. Someone pins a notice, and everyone who checks the board sees it.

```
Event Bus (Simple):

+-----------+                   +-----------+
| Service A |---+               | Handler 1 |
+-----------+   |   +-------+  +-----------+
                +-->|       |-->|           |
+-----------+   |   | EVENT |  | Handler 2 |
| Service B |---+   |  BUS  |  +-----------+
+-----------+       |       |-->|           |
                    +-------+  | Handler 3 |
                               +-----------+

- In-process or single-application
- No persistence (events lost if app crashes)
- Simple, fast, no external dependencies
- Examples: MediatR (.NET), Guava EventBus (Java)
```

Event Broker (Distributed)

An event broker is a dedicated infrastructure component that stores and routes events between distributed services. It persists events, guarantees delivery, and handles retries.

```
Event Broker (Distributed):

+-----------+                              +-----------+
| Order     |---+                     +--->| Payment   |
| Service   |   |   +-------------+  |    | Service   |
+-----------+   +-->|             |--+     +-----------+
                |   |   EVENT     |  |
+-----------+   |   |   BROKER    |  |     +-----------+
| User      |---+   |             |--+---->| Inventory |
| Service   |       | (Kafka,     |  |     | Service   |
+-----------+       |  RabbitMQ,  |  |     +-----------+
                    |  Pulsar)    |  |
                    +-------------+  |     +-----------+
                                     +---->| Email     |
                                           | Service   |
                                           +-----------+

- Distributed, runs as separate infrastructure
- Persists events (can replay)
- Guarantees delivery (at-least-once)
- Handles retries, dead letter queues
- Scales independently
```

Event Bus vs Event Broker

Feature	Event Bus	Event Broker
Deployment	In-process	Separate infrastructure
Persistence	No (in-memory)	Yes (disk)
Delivery guarantee	None (best-effort)	At-least-once or exactly-once
Replay	No	Yes (Kafka)
Scaling	Scales with the app	Scales independently
Use case	Simple apps, monoliths	Microservices, distributed systems

Choreography vs Orchestration

When multiple services need to collaborate to complete a business process, there are two approaches: choreography and orchestration.

Choreography: Everyone Dances Independently

In choreography, there is no central coordinator. Each service listens for events and decides what to do next. It is like a dance where everyone knows the steps and reacts to the music independently.

```
Choreography (Order Processing):

1. Order Service emits: OrderPlaced
2. Payment Service hears OrderPlaced --> processes payment
3. Payment Service emits: PaymentReceived
4. Inventory Service hears PaymentReceived --> reserves items
5. Inventory Service emits: ItemsReserved
6. Shipping Service hears ItemsReserved --> ships order
7. Shipping Service emits: OrderShipped
8. Notification Service hears OrderShipped --> notifies customer

+-------+     +-------+     +---------+     +--------+     +--------+
| Order |     |Payment|     |Inventory|     |Shipping|     | Notif. |
+---+---+     +---+---+     +----+----+     +---+----+     +---+----+
    |             |              |              |              |
    | OrderPlaced |              |              |              |
    +------------>|              |              |              |
    |             |              |              |              |
    |             |PaymentRecvd  |              |              |
    |             +------------->|              |              |
    |             |              |              |              |
    |             |              |ItemsReserved |              |
    |             |              +------------->|              |
    |             |              |              |              |
    |             |              |              |OrderShipped  |
    |             |              |              +------------->|
    |             |              |              |              |

No central coordinator. Each service reacts to events.
```

Pros:

- Loose coupling. Each service only knows about events, not other services.
- Easy to add new services. Just subscribe to events.
- No single point of failure.

Cons:

- Hard to understand the full business flow. Logic is spread across services.
- Hard to debug. You need to trace events across multiple services.

- Hard to handle failures. What if the payment succeeds but inventory reservation fails?

Orchestration: One Conductor Leads

In orchestration, a central coordinator (the orchestrator) tells each service what to do and when. It is like an orchestra conductor directing each musician.

```
Orchestration (Order Processing):

The Order Orchestrator controls the flow:

+-------------------------------------------------------------+
|                   ORDER ORCHESTRATOR                         |
|                                                             |
|  Step 1: Call Payment Service --> ProcessPayment            |
|  Step 2: Call Inventory Service --> ReserveItems            |
|  Step 3: Call Shipping Service --> ShipOrder                |
|  Step 4: Call Notification Service --> NotifyCustomer       |
|                                                             |
|  If Step 2 fails: Call Payment Service --> RefundPayment    |
+-------------------------------------------------------------+
      |            |             |             |
      v            v             v             v
+---------+  +---------+  +---------+  +---------+
| Payment |  |Inventory|  | Shipping|  | Notif.  |
| Service |  | Service |  | Service |  | Service |
+---------+  +---------+  +---------+  +---------+

The orchestrator knows the entire flow and handles failures.
```

Pros:

- Easy to understand. The entire business flow is in one place.
- Easy to handle failures. The orchestrator knows what to undo.
- Easy to test. Test the orchestrator to test the flow.

Cons:

- Tight coupling to the orchestrator. It becomes a single point of failure.
- The orchestrator can become a "god service" that knows too much.
- Harder to scale. The orchestrator must handle all coordination.

Which to Choose?

Aspect	Choreography	Orchestration
Coupling	Low (services independent)	Higher (depends on orchestrator)

Visibility	Low (distributed logic)	High (centralized logic)
Failure handling	Complex (distributed compensation)	Simple (orchestrator manages)
Scalability	High (no bottleneck)	Moderate (orchestrator is bottleneck)
Best for	Simple flows, high scale	Complex flows, many failure cases

In practice, many systems use a hybrid approach: orchestration for complex workflows and choreography for simple event notifications.

The Saga Pattern

When a business process spans multiple services, you cannot use a traditional database transaction. The saga pattern is a way to manage distributed transactions using a sequence of local transactions coordinated by events.

The Problem

Consider an e-commerce order:

1. Charge the customer (Payment Service)
2. Reserve inventory (Inventory Service)
3. Schedule delivery (Shipping Service)

In a monolith, you would wrap all three in a single database transaction. If any step fails, everything rolls back. In microservices, there is no such thing as a distributed transaction (at least not a practical one).

How Sagas Work

A saga breaks the transaction into a series of steps. Each step has a compensating action -- something that undoes the step if a later step fails.

```
Saga: Order Processing

Step 1: Charge Payment
  Compensating action: Refund Payment
```

```
Step 2: Reserve Inventory
  Compensating action: Release Inventory

Step 3: Schedule Delivery
  Compensating action: Cancel Delivery

Happy Path:
Charge --> Reserve --> Schedule --> SUCCESS

Failure at Step 3:
Charge --> Reserve --> Schedule (FAILS!)
                |
                v
       Release Inventory (compensate step 2)
                |
                v
       Refund Payment (compensate step 1)
                |
                v
             FAILED (customer notified)
```

Choreography-Based Saga

Each service listens for events and triggers the next step or compensating action.

```
Choreography-Based Saga:

+----------+   OrderPlaced    +----------+   PaymentCharged   +-----------+
|  Order   | --------------> | Payment  | -----------------> | Inventory |
|  Service |                 | Service  |                    | Service   |
+----------+                 +----------+                    +-----------+
                                  |                               |
                             PaymentFailed                   ReservationFailed
                                  |                               |
                                  v                               v
                            Order Cancelled              Refund Payment
                                                         Release Inventory
                                                         Order Cancelled
```

Orchestration-Based Saga

A central saga orchestrator manages the steps and compensations.

```
Orchestration-Based Saga:

+-------------------------------------------------------------+
|                  SAGA ORCHESTRATOR                          |
```

```
|                                                            |
|  State Machine:                                            |
|                                                            |
|  PENDING --> CHARGING --> RESERVING --> SCHEDULING --> DONE|
|     |           |            |              |              |
|     v           v            v              v              |
|  CANCELLED  REFUNDING   RELEASING      CANCELLING          |
+------------------------------------------------------------+
     |           |            |              |
     v           v            v              v
+---------+ +---------+ +---------+    +---------+
| Payment | |Inventory| | Shipping|    | Notif.  |
+---------+ +---------+ +---------+    +---------+
```

Eventual Consistency

In event-driven systems, services do not update simultaneously. After an event is published, it takes time for all consumers to process it. During this window, different parts of the system have different views of the data. This is eventual consistency.

How It Feels to the User

```
Eventual Consistency Timeline:

Time 0: User places order
Time 1: Order Service stores order         (Order: "placed")
Time 2: Event "OrderPlaced" published
Time 3: Payment Service receives event     (Payment: "none" -> "processing")
Time 4: Inventory Service receives event   (Inventory: "available")
Time 5: Payment processed                  (Payment: "charged")
Time 6: Inventory reserved                 (Inventory: "reserved")

Between Time 2 and Time 6, different services show
different states. The system is temporarily inconsistent.

Eventually (by Time 6), everything is consistent.

+---------+  +---------+  +---------+
| Order:  |  | Payment:|  | Invent.:|
| placed  |  | charged |  | reserved|
+---------+  +---------+  +---------+
     All consistent (eventually)
```

Dealing with Eventual Consistency

1. Design UIs to show pending states. Show "Processing..." instead of assuming instant updates.
2. Use read-your-own-writes. After a user creates something, route their next read to the write model so they see their own change immediately.
3. Set expectations. Tell users "Your order is being processed" rather than pretending it is instant.

Idempotent Consumers

In an event-driven system with at-least-once delivery (which is the most common guarantee), your consumers will receive duplicate events. Network glitches, retries, and broker redeliveries all cause duplicates.

An idempotent consumer produces the same result whether it processes an event once or ten times.

Why This Matters

```
Without Idempotency:

Event: PaymentReceived {orderId: 123, amount: $50}

Delivery 1: Charge customer $50    (Total: $50)  CORRECT
Delivery 2: Charge customer $50    (Total: $100) WRONG!
Delivery 3: Charge customer $50    (Total: $150) VERY WRONG!

With Idempotency:

Event: PaymentReceived {orderId: 123, amount: $50, eventId: "evt-abc"}

Delivery 1: Check "evt-abc" processed? No.  Charge $50.  Record "evt-abc".
Delivery 2: Check "evt-abc" processed? Yes. Skip.
Delivery 3: Check "evt-abc" processed? Yes. Skip.

Total charged: $50 (CORRECT, no matter how many deliveries)
```

Strategies for Idempotency

1. Event ID deduplication. Store processed event IDs. Before processing, check if the ID already exists.

2. Natural idempotency. Some operations are naturally idempotent. Setting a status to "shipped" is the same whether you do it once or five times.
3. Idempotency keys. Use a unique key (like order ID + event type) to ensure each logical operation happens exactly once.

```
Idempotent Consumer Pattern:

+------------------+
|  Receive Event   |
+--------+---------+
         |
         v
+------------------+
| Check: eventId   |
| in processed_set?|
+--------+---------+
         |
    +----+----+
    |         |
   YES        NO
    |         |
    v         v
+-------+  +------------------+
| SKIP  |  | Process Event    |
| (ack) |  | Store eventId    |
+-------+  | in processed_set |
           | ACK to broker    |
           +------------------+
```

Dead Letter Queues in Event-Driven Systems

A dead letter queue (DLQ) is a special queue where events go when they cannot be processed after multiple retries. It is your safety net for events that would otherwise be lost or block the pipeline.

When Events End Up in the DLQ

```
Event Processing with DLQ:

+---------+     +--------+     +-----------+
| Event   |---->| Event  |---->| Consumer  |
| Source  |     | Broker |     |           |
+---------+     +--------+     +-----------+
                    |               |
```

```
                        |          Retry 1: FAIL
                        |          Retry 2: FAIL
                        |          Retry 3: FAIL
                        |               |
                        |               v
                        |         +----------+
                        |         |   DLQ    |
                        |         | (Dead    |
                        |         |  Letter  |
                        |         |  Queue)  |
                        |         +----------+
                        |               |
                        |               v
                        |         Alert! Engineers
                        |         investigate and
                        |         reprocess manually

  Common reasons for DLQ:
  - Malformed event data (missing required fields)
  - Consumer bug (null pointer, logic error)
  - Downstream service permanently unavailable
  - Schema mismatch (producer updated, consumer did not)
```

Best Practices for DLQs

1. Always configure a DLQ. Never let failed events disappear silently.
2. Set a reasonable retry count (3 to 5 retries with exponential backoff).
3. Monitor DLQ depth. A growing DLQ means something is wrong.
4. Build a reprocessing tool. After fixing the issue, you need a way to move events from the DLQ back to the main queue.
5. Include metadata. Store the failure reason, retry count, and original timestamp with each DLQ event.

Order Processing Pipeline: Putting It All Together

Let us design a complete order processing pipeline using event-driven architecture.

```
  Order Processing Pipeline:

  +--------+   PlaceOrder   +---------+  OrderPlaced  +----------+
  | Client | -------------> |  Order  | ------------> | Event    |
  |  (App) |   (command)    | Service |   (event)     | Broker   |
  +--------+                +---------+               +-----+----+
```

```
                                                          |
                  +----------------------------------------+
                  |                |                |
                  v                v                v
            +-----------+    +-----------+    +-----------+
            |  Payment  |    | Inventory |    |   Email   |
            |  Service  |    |  Service  |    |  Service  |
            +-----------+    +-----------+    +-----------+
                  |                |                |
         PaymentCharged    ItemsReserved    ConfirmationSent
                  |                |
                  v                v
            +----------+    +----------+
            | Event    |    | Event    |
            | Broker   |    | Broker   |
            +-----+----+    +-----+----+
                  |                |
                  v                v
            +-----------+    +-----------+
            | Shipping  |    | Analytics |
            |  Service  |    |  Service  |
            +-----------+    +-----------+
                  |
            OrderShipped
                  |
                  v
            +-----------+
            |   Push    |
            |  Notif.   |
            |  Service  |
            +-----------+


Detailed Event Flow:

Step 1: Client sends PlaceOrder command to Order Service
Step 2: Order Service validates and stores order
Step 3: Order Service emits OrderPlaced event
Step 4: Payment Service listens, charges customer
Step 5: Payment Service emits PaymentCharged
Step 6: Inventory Service listens, reserves items
Step 7: Inventory Service emits ItemsReserved
Step 8: Shipping Service listens, schedules shipment
Step 9: Shipping Service emits OrderShipped
Step 10: Notification Service sends push notification

Failure Handling (Saga):
- If payment fails: emit PaymentFailed, cancel order
- If inventory fails: emit ReservationFailed, refund payment
- If shipping fails: emit ShippingFailed, release inventory, refund
```

The Event Store for This Pipeline

```
Event Store (Order #789):

+----+--------------------+------+---------------------------------------+
| #  | Timestamp          | Type | Data                                  |
+----+--------------------+------+---------------------------------------+
| 1  | 2024-01-15 10:00:01| OrderPlaced    | {items: [...], total: $129} |
| 2  | 2024-01-15 10:00:03| PaymentCharged | {method: "visa_4242"}       |
| 3  | 2024-01-15 10:00:04| ItemsReserved  | {warehouse: "US-East"}      |
| 4  | 2024-01-15 10:00:10| OrderShipped   | {tracking: "FX123456"}      |
| 5  | 2024-01-15 10:00:11| CustomerNotified| {channel: "push+email"}    |
+----+--------------------+------+---------------------------------------+

Complete audit trail. You can:
- Replay to any point in time
- Debug any issue
- Build new reports from historical events
- Prove compliance to auditors
```

Trade-offs

Decision	Option A	Option B
Architecture	Request-response (simple, synchronous)	Event-driven (complex, async, scalable)
Coordination	Choreography (decoupled, hard to debug)	Orchestration (centralized, easier to debug)
State storage	Current state (simple, fast reads)	Event sourcing (audit trail, complex, replay)
Read/write model	Single model (simple, consistent)	CQRS (optimized, eventually consistent)
Consistency	Strong (immediate, slower)	Eventual (fast, temporary inconsistency)
Delivery	At-most-once (fast, may lose events)	At-least-once (safe, needs idempotency)

Common Mistakes

1. Using event-driven architecture for everything. Simple CRUD operations do not need events. If a user updates their profile name, a direct database update is fine. Reserve events for cross-service communication and important business state changes.
2. Not making consumers idempotent. This is the number one source of bugs in event-driven systems. Every consumer must handle duplicate events gracefully.
3. Ignoring event ordering. Events for the same entity should be processed in order. If `OrderShipped` arrives before `OrderPlaced`, your system will be confused. Use partition keys (like order ID) to ensure ordering within an entity.
4. Creating event chains that are too long. If Service A triggers B, which triggers C, which triggers D, which triggers E, a failure anywhere creates a complex debugging puzzle. Keep event chains short or use an orchestrator for complex workflows.
5. Not versioning events. When you change the structure of an event, old consumers will break. Always version your events (e.g., `OrderPlaced_v2`) and support backward compatibility.
6. Skipping the dead letter queue. Without a DLQ, failed events either block the pipeline or disappear silently. Both are unacceptable in production.

Best Practices

1. Start with commands, evolve to events. Begin with direct service-to-service calls. When you see a need for decoupling, introduce events for that specific flow.
2. Use unique event IDs. Every event should have a globally unique ID (UUID) for deduplication and tracing.
3. Include metadata in events. Every event should carry: event ID, timestamp, source service, correlation ID (to trace across services), and schema version.
4. Design events as contracts. Treat event schemas like API contracts. Use schema registries (like Confluent Schema Registry) to manage compatibility.
5. Keep events small. Include only the essential data in the event. If consumers need more detail, they can query the source service. This is called the "thin event" approach.
6. Use correlation IDs. When an event triggers a chain of downstream events, carry a correlation ID through the entire chain so you can trace the full flow.
7. Monitor consumer lag. Track how far behind each consumer is. A growing lag means the consumer cannot keep up with the event rate.
8. Test with chaos. Simulate failures, duplicate deliveries, and out-of-order events in your test environment to verify your system handles them correctly.

Quick Summary

Event-driven architecture builds systems around events: immutable records of things that happened. Events differ from commands: commands request action, events report facts. Event sourcing stores state as a sequence of events, giving you a complete audit trail and the ability to replay history. CQRS separates read and write models, optimizing each independently. Events flow through an event bus (in-process) or event broker (distributed like Kafka). Choreography lets services react independently to events; orchestration uses a central coordinator. The saga pattern manages distributed transactions through compensating actions. Eventual consistency means different services may temporarily have different views of the data. Idempotent consumers are critical because at-least-once delivery means duplicates will happen. Dead letter queues catch events that cannot be processed after retries.

Key Points

- Events are immutable facts about what happened; commands are requests that can be rejected
- Event sourcing stores every change as an event, enabling audit trails and time travel
- CQRS separates read and write models for independent optimization
- Choreography offers loose coupling; orchestration offers centralized control and easier debugging
- The saga pattern replaces distributed transactions with local transactions and compensating actions
- Eventual consistency is the trade-off for decoupling and scalability
- Idempotent consumers are not optional -- they are required for correctness
- Dead letter queues are your safety net for events that cannot be processed

---

Practice Questions

1. Your e-commerce platform uses event sourcing for orders. After one year, each order has an average of 50 events. Rebuilding order state is taking 200 milliseconds. Your SLA requires responses under 50 milliseconds. How would you solve this?

2. You are building a food delivery app. When a customer places an order, you need to: validate the order, charge payment, notify the restaurant, and assign a driver. Should you use choreography or orchestration? Justify your answer considering failure scenarios.
3. Your payment service processes `PaymentCharged` events. Due to a network glitch, the same event is delivered three times. Your current implementation charges the customer three times. How would you redesign the consumer to be idempotent?
4. You are designing a system where the order service and the inventory service need to stay in sync. A customer places an order, but by the time the inventory service processes the event, the item is out of stock. How would you handle this using the saga pattern? What compensating actions are needed?
5. Your dead letter queue has 50,000 events from the last 24 hours. All of them are `OrderPlaced` events that failed in the inventory service. What is your investigation and recovery plan?

Exercises

Exercise 1: Design an Event-Driven Notification System

Design the event flow for a social media application. When a user posts a photo, the following must happen: generate thumbnails, scan for inappropriate content, notify followers, update the news feed, and update the user's post count. Draw the architecture showing events, services, and the event broker. Identify which steps can happen in parallel and which must be sequential.

Exercise 2: Implement a Saga for Hotel Booking

Design a saga for a travel booking system. When a customer books a trip, you need to: reserve a flight, reserve a hotel, and reserve a rental car. Define the compensating action for each step. Draw the saga flow for both the happy path and a failure at each step. Choose between choreography-based and orchestration-based saga and justify your choice.

Exercise 3: Event Schema Evolution

You have an `OrderPlaced` event with the following schema: `{orderId, customerId, items[], total}`. The business now requires you to add a `currency` field and split `total` into `subtotal`, `tax`, and `total`. Design a migration strategy that does not break existing consumers. Consider versioning, backward compatibility, and how you would handle replaying old events.

What Is Next?

You now understand how services communicate through events, how to store state as events, and how to manage distributed transactions with sagas. But modern systems deal with more than structured data flowing between services. Users upload photos, videos, and documents. These large binary objects need a completely different storage approach. In the next chapter, you will learn about blob storage: how systems like S3 store files at scale, how to integrate with CDNs for fast delivery, and how platforms like Instagram handle millions of image uploads per day.

Chapter 17: Blob Storage

What You Will Learn

- What blob storage is and why it exists
- The difference between object storage, block storage, and file storage
- How S3-like architecture works under the hood
- What metadata is and how it makes objects searchable
- How pre-signed URLs provide secure, temporary access to files
- How CDN integration accelerates file delivery worldwide
- How an image processing pipeline handles uploads at scale
- How video transcoding works and why it is necessary
- What storage tiers are and how to save money with lifecycle policies
- How Instagram handles millions of photo uploads per day

Why This Chapter Matters

Imagine a library. Books are neatly organized on shelves with a catalog system. You look up a title in the catalog, find the shelf number, and walk to the right spot. Now imagine a warehouse full of shipping containers. Each container holds something different: furniture, electronics, clothing. You do not organize them by type. You give each container a unique label, record its location, and retrieve it by label when needed.

Blob storage works like that warehouse. "Blob" stands for Binary Large Object. It stores unstructured data: photos, videos, PDFs, backups, log files -- anything that is not a row in a database table. Every time you upload a profile picture, watch a video, or download a file from the cloud, you are interacting with blob storage. Companies like Netflix, Instagram, and Dropbox store petabytes of data in blob storage. Understanding how it works is essential for any system design involving media, files, or large data.

What Is Blob Storage?

Blob storage is a system designed to store large, unstructured binary data. Unlike a relational database (which stores structured rows and columns) or a file system (which organizes files in directories), blob storage treats each piece of data as an opaque "blob" identified by a unique key.

```
Relational Database:          File System:              Blob Storage:
(Structured Data)             (Hierarchical)            (Flat Namespace)

+----+-------+------+        /home/                    bucket: my-photos
| id | name  | age  |          /user/                    key: photo_001.jpg
+----+-------+------+            /photos/                 key: photo_002.jpg
| 1  | Alice | 30   |              img1.jpg              key: vacation/sunset.jpg
| 2  | Bob   | 25   |              img2.jpg              key: profile/avatar.png
+----+-------+------+            /docs/
                                     resume.pdf           bucket: my-docs
Best for: queries,            Best for: local            key: resume.pdf
joins, transactions           access, small files        key: report_2024.pdf

                                                          Best for: large files,
                                                          media, backups, any
                                                          unstructured data
```

Core Concepts

- Blob: A chunk of binary data. Could be an image, video, PDF, database backup, or any file.
- Bucket (Container): A top-level namespace that groups related blobs. Like a folder, but flat.
- Key (Object Key): A unique identifier for a blob within a bucket. Often looks like a file path: `users/123/profile.jpg`.
- Metadata: Key-value pairs attached to a blob describing it (content type, size, upload date, custom tags).

```
Blob Storage Anatomy:

+-------------------------------------------------+
|  BUCKET: "instagram-photos"                      |
|                                                  |
|  +----------------------------------------------+ |
|  | KEY: "users/alice/photo_001.jpg"             | |
```

```
   |  | DATA: [binary image data, 2.3 MB]            | |
   |  | METADATA:                                    | |
   |  |   content-type: image/jpeg                   | |
   |  |   size: 2,412,544 bytes                      | |
   |  |   uploaded: 2024-01-15T10:30:00Z             | |
   |  |   resolution: 4032x3024                      | |
   |  |   user-id: alice_123                         | |
   |  +----------------------------------------------+ |
   |                                                   |
   |  +----------------------------------------------+ |
   |  | KEY: "users/bob/video_042.mp4"               | |
   |  | DATA: [binary video data, 156 MB]            | |
   |  | METADATA:                                    | |
   |  |   content-type: video/mp4                    | |
   |  |   size: 163,577,856 bytes                    | |
   |  |   duration: 45s                              | |
   |  +----------------------------------------------+ |
   +---------------------------------------------------+
```

Object Storage vs Block Storage vs File Storage

These three storage types serve very different purposes. Understanding the differences is critical for system design.

Block Storage

Block storage splits data into fixed-size chunks called "blocks" and stores them on disk. It is the lowest level of storage. Your laptop's hard drive is block storage. In the cloud, it powers virtual machine disks.

Think of it as a stack of blank papers. You can write anything on any page, but there is no built-in organization. The operating system or database manages how blocks are used.

File Storage

File storage organizes data in a hierarchical directory structure with files and folders. It is what you see in your computer's file explorer. In the cloud, services like Amazon EFS or Azure Files provide shared file storage.

Think of it as a filing cabinet. Folders hold subfolders, which hold files. You navigate by path: `/reports/2024/january/sales.csv`.

Object Storage

Object storage stores data as objects in a flat namespace. Each object has data, metadata, and a unique key. There are no directories (although keys can use `/` to simulate them). It is designed for massive scale, durability, and HTTP access.

Think of it as a warehouse of labeled containers. You do not care where the container is physically stored. You just use the label to find it.

```
Comparison:

+------------------+-------------------+-------------------+
|   Block Storage  |   File Storage    |  Object Storage   |
+------------------+-------------------+-------------------+
| Fixed-size blocks| Files in folders  | Objects with keys |
| No metadata      | File metadata     | Rich metadata     |
| Low-level access | Path-based access | HTTP API access   |
| Fastest I/O      | Moderate I/O      | Higher latency    |
| Limited scale    | Moderate scale    | Unlimited scale   |
| VM disks, DBs    | Shared file sys   | Media, backups    |
+------------------+-------------------+-------------------+

Examples:
Block: AWS EBS, Azure Disk, GCP Persistent Disk
File:  AWS EFS, Azure Files, GCP Filestore
Object: AWS S3, Azure Blob, GCP Cloud Storage
```

Feature	Block Storage	File Storage	Object Storage
Access pattern	Random read/write	Sequential, path-based	Whole object read/write
Latency	Sub-millisecond	Milliseconds	Tens of milliseconds
Scale	Terabytes	Terabytes	Petabytes+
Protocol	SCSI, NVMe	NFS, SMB	HTTP REST API
Best for	Databases, VMs	Shared documents	Media files, backups
Modify part of file?	Yes (block-level)	Yes (seek + write)	No (replace entire object)

Key Insight

Object storage cannot modify part of a file. To change one pixel in a photo, you must re-upload the entire photo. This is a deliberate trade-off: giving up partial updates enables massive scalability and durability.

S3-Like Architecture

Amazon S3 (Simple Storage Service) is the most widely used blob storage service. Understanding its architecture helps you understand all object storage systems.

How S3 Works Internally

```
S3-Like Architecture (Simplified):

+--------+    PUT /bucket/key    +-------------------+
| Client | --------------------> |   API Gateway     |
+--------+                       | (Load Balanced)   |
                                 +--------+---------+
                                          |
                                 +--------v---------+
                                 |  Metadata Store   |
                                 |  (Key -> Location)|
                                 |  (DynamoDB-like)  |
                                 +--------+---------+
                                          |
                           +-------------+-------------+
                           |             |             |
                     +--------v--+  +-------v---+  +-----v-----+
                     | Storage   |  | Storage   |  | Storage   |
                     | Node A    |  | Node B    |  | Node C    |
                     | (Replica 1)|  | (Replica 2)|  | (Replica 3)|
                     +-----------+  +-----------+  +-----------+
                     | Disk 1    |  | Disk 1    |  | Disk 1    |
                     | Disk 2    |  | Disk 2    |  | Disk 2    |
                     | ...       |  | ...       |  | ...       |
                     +-----------+  +-----------+  +-----------+

Write Path:
1. Client sends PUT request with object data
2. API Gateway routes to metadata store
3. Metadata store assigns storage nodes
4. Data replicated to 3+ nodes across availability zones
5. Metadata (key, location, checksum) stored
6. Client gets 200 OK after all replicas confirm

Read Path:
1. Client sends GET request with bucket + key
2. API Gateway queries metadata store for location
3. Data fetched from nearest healthy storage node
4. Client receives object data
```

Durability and Availability

S3 provides 99.999999999% (eleven 9s) durability. This means if you store 10 million objects, you can expect to lose one object every 10,000 years.

How? By storing each object redundantly across multiple physical locations.

```
Durability through Redundancy:

Your photo "vacation.jpg" stored in S3:

+-------------------+     +-------------------+     +-------------------+
| Availability      |     | Availability      |     | Availability      |
| Zone A            |     | Zone B            |     | Zone C            |
| (Building 1)      |     | (Building 2)      |     | (Building 3)      |
|                   |     |                   |     |                   |
| [vacation.jpg]    |     | [vacation.jpg]    |     | [vacation.jpg]    |
| Copy 1            |     | Copy 2            |     | Copy 3            |
+-------------------+     +-------------------+     +-------------------+

Even if an entire building burns down,
your photo is safe in the other two.
```

Common S3 Operations

Operation	HTTP Method	Description
Upload	PUT	Store a new object
Download	GET	Retrieve an object
Delete	DELETE	Remove an object
List	GET (bucket)	List objects in a bucket
Head	HEAD	Get object metadata without downloading
Copy	PUT (with copy source)	Copy an object within S3

Metadata

Metadata is data about data. Every object in blob storage carries metadata alongside its binary content.

System Metadata

Automatically set by the storage system:

- `Content-Length`: Size of the object in bytes
- `Content-Type`: MIME type (image/jpeg, video/mp4, application/pdf)
- `Last-Modified`: When the object was last updated
- `ETag`: A hash of the object content (for cache validation)

User-Defined Metadata

Custom key-value pairs you attach to objects:

- `x-amz-meta-user-id: alice_123`
- `x-amz-meta-camera-model: iPhone 15 Pro`
- `x-amz-meta-gps-location: 37.7749,-122.4194`

Why Metadata Matters

Metadata enables searching, filtering, and processing without downloading the actual file.

```
Using Metadata:

"Find all photos uploaded by alice_123 that are larger
 than 5 MB and were taken with an iPhone"

Instead of downloading every photo to check:

Query metadata store:
  WHERE user-id = "alice_123"
  AND content-length > 5242880
  AND camera-model LIKE "iPhone%"

Returns matching keys without touching the actual files.
```

Pre-Signed URLs

A pre-signed URL is a temporary, self-expiring link that grants access to a private object without requiring authentication. It is one of the most important patterns in blob storage.

The Problem

Your objects are private. Only your server can access them. But you need to let users upload and download files directly without routing all traffic through your server.

How Pre-Signed URLs Work

```
Pre-Signed URL Flow (Download):

1. Client requests file access
+--------+     "I need photo_123.jpg"     +--------+
| Client | ----------------------------> | Server |
+--------+                                +--------+
                                              |
2. Server generates pre-signed URL            |
   (signed with secret key, expires in 15 min)|
                                              |
3. Server returns pre-signed URL              |
+--------+     "https://s3.../photo_123   +--------+
| Client | <-------- ?signature=abc&      | Server |
+--------+            expires=1705312000" +--------+
     |
4. Client downloads directly from S3
     |        GET (pre-signed URL)
     +-------------------------------> +-----+
                                       | S3  |
     <-------------------------------  +-----+
            (file data, 2.3 MB)

The server never touches the file data.
S3 verifies the signature and expiration.
```

```
Pre-Signed URL Flow (Upload):

1. Client asks server for upload permission
+--------+    "I want to upload a photo"  +--------+
| Client | ----------------------------> | Server |
+--------+                                +--------+
                                              |
2. Server generates pre-signed PUT URL        |
   (allows upload to specific key)            |
                                              |
3. Server returns URL                         |
+--------+    "PUT to this URL"           +--------+
| Client | <---------------------------- | Server |
+--------+                                +--------+
```

```
     |
4. Client uploads directly to S3
     |       PUT (file data)
     +-------------------------------> +-----+
                                        | S3  |
     <-------------------------------  +-----+
            (200 OK)

Benefits:
- Server never handles large file uploads
- Reduces server bandwidth and CPU
- S3 handles upload at scale
- URL expires, so access is temporary
```

Security Considerations

- Set short expiration times. 15 minutes for uploads, 1 hour for downloads is typical.
- Restrict to specific operations. An upload URL should not allow downloads.
- Include content type restrictions. Prevent users from uploading executables when you expect images.
- Use one-time URLs for sensitive content. Some systems invalidate the URL after first use.

CDN Integration

A Content Delivery Network (CDN) caches your blob storage content at edge locations around the world, delivering files to users from the nearest server.

Without a CDN

```
Without CDN:

User in Tokyo                          S3 in US-East
+--------+                             +--------+
|  User  | ---- 12,000 km ----------> |   S3   |
| Tokyo  | <--- 200ms latency ------- | Virginia|
+--------+                             +--------+

Every request travels across the Pacific Ocean.
Slow. Expensive (egress bandwidth).
```

With a CDN

```
With CDN:

User in Tokyo        CDN Edge in Tokyo           S3 in US-East
+--------+           +------------+              +--------+
|  User  | -- 5ms -> | CDN Edge   |              |   S3   |
| Tokyo  | <-------- | (Cached!)  |              | Virginia|
+--------+           +------------+              +--------+
                           |                          |
                      First request:                  |
                      Cache miss, fetch from S3 ----->|
                      Cache the response              |
                           |                          |
                      Subsequent requests:            |
                      Serve from cache (5ms)          |

Benefits:
- Latency: 200ms -> 5ms for cached content
- Bandwidth: S3 egress reduced by 90%+
- Scalability: CDN handles traffic spikes
- Cost: CDN egress is cheaper than S3 egress
```

CDN + Blob Storage Architecture

```
Complete CDN Integration:

+--------+                +----------+             +---------+
| Client |   GET image    |   CDN    |  Cache miss |   S3    |
|        | -------------> | (Edge)   | ----------> | (Origin)|
+--------+                +----------+             +---------+
     ^                         |
     |     Cached response     |
     +-------------------------+

Cache-Control: public, max-age=31536000
(Cache for 1 year -- images rarely change)

Cache Invalidation:
When a user updates their profile photo:
1. Upload new photo to S3 with new key (e.g., photo_v2.jpg)
2. Update application to reference new key
3. Old key eventually expires from CDN cache

OR: Use versioned keys
- avatar_abc123.jpg (hash of content in filename)
- When content changes, hash changes, new URL, no invalidation needed
```

Image Processing Pipeline

When a user uploads an image, you rarely store just the original. You need thumbnails, compressed versions, and different sizes for different devices.

```
Image Processing Pipeline:

+--------+                                    +---------+
| User   |  Upload original (12 MB)           |   S3    |
| Device | ---------------------------------> | (Raw)   |
+--------+                                    +----+----+
                                                   |
                                            Upload event
                                                   |
                                                   v
                                            +------+------+
                                            | Image       |
                                            | Processor   |
                                            | (Lambda/    |
                                            |  Worker)    |
                                            +------+------+
                                                   |
                              +----------------------+----------------------+
                              |                      |                      |
                              v                      v                      v
                     +---------+----+       +---------+----+       +---------+----+
                     | Thumbnail    |       | Medium       |       | Full Size    |
                     | 150x150      |       | 640x640      |       | 1080x1080    |
                     | 15 KB        |       | 80 KB        |       | 250 KB       |
                     | WebP format  |       | WebP format  |       | JPEG format  |
                     +---------+----+       +---------+----+       +---------+----+
                              |                      |                      |
                              v                      v                      v
                     +---------+----+       +---------+----+       +---------+----+
                     | S3: /thumb/  |       | S3: /medium/ |       | S3: /full/   |
                     | photo_001    |       | photo_001    |       | photo_001    |
                     +--------------+       +--------------+       +--------------+
                              |                      |                      |
                              +----------+-----------+
                                         |
                                         v
                                  +------+------+
                                  |     CDN     |
                                  | (Cached at  |
                                  |  edges)     |
                                  +-------------+

Processing steps:
1. Validate file type (is it really an image?)
2. Strip EXIF data (remove GPS coordinates for privacy)
3. Resize to multiple dimensions
4. Convert to efficient format (WebP for web, HEIC for iOS)
5. Compress (reduce file size without visible quality loss)
```

```
6. Store all versions in S3
7. Update metadata database with URLs
8. Invalidate CDN cache if replacing existing image
```

Video Transcoding

Video files are much larger than images and require transcoding: converting the video into multiple formats and quality levels so it plays smoothly on any device and network speed.

Why Transcoding Is Necessary

A raw video uploaded from a phone might be:

- 4K resolution (3840x2160)
- H.265 codec
- 150 Mbps bitrate
- 500 MB for a 30-second clip

But a user on a 3G connection with an old phone needs:

- 480p resolution
- H.264 codec (widely supported)
- 1 Mbps bitrate
- 5 MB for the same 30 seconds

Transcoding creates multiple versions so the player can switch quality based on network conditions (adaptive bitrate streaming).

```
Video Transcoding Pipeline:

+--------+   Upload     +---------+   Event    +-------------+
| User   | -----------> | S3 Raw  | --------> | Transcoding |
+--------+   (500 MB)   | Bucket  |           | Service     |
                         +---------+           +------+------+
                                                      |
                         +----------------------------+
                         |            |               |
                         v            v               v
                    +-----------+ +-----------+  +-----------+
                    | 1080p     | | 720p      |  | 480p      |
                    | H.264     | | H.264     |  | H.264     |
                    | 5 Mbps    | | 2.5 Mbps  |  | 1 Mbps    |
```

```
                    | 150 MB    | | 75 MB     |  | 30 MB     |
                    +-----------+ +-----------+  +-----------+
                          |            |               |
                          v            v               v
                    +-----------+ +-----------+  +-----------+
                    | S3 Output | | S3 Output |  | S3 Output |
                    | /1080p/   | | /720p/    |  | /480p/    |
                    +-----------+ +-----------+  +-----------+
                                   |
                                   v
                          +--------+--------+
                          | Manifest File   |
                          | (HLS/DASH)      |
                          | Lists all       |
                          | quality levels  |
                          +-----------------+
                                   |
                                   v
                             +-----+-----+
                             |    CDN    |
                             +-----------+

Adaptive Bitrate Streaming:
The video player checks network speed and switches quality:

Fast WiFi:   [1080p] ═══════════════════════
Slow 4G:     [1080p] ═══ [720p] ═══ [480p] ═══
3G:          [480p]  ═══════════════════════
```

Storage Tiers

Not all data is accessed equally. A photo uploaded today gets viewed frequently. A photo from five years ago might never be viewed again. Storage tiers let you pay less for data you rarely access.

```
Storage Tiers (S3 Example):

+-------------------------------------------------------------------+
|  Tier          | Access     | Cost/GB  | Retrieval  | Use Case    |
+-------------------------------------------------------------------+
|  Standard      | Frequent   | $0.023   | Instant    | Active      |
|                |            |          |            | content     |
+-------------------------------------------------------------------+
|  Infrequent    | Monthly    | $0.0125  | Instant    | Older       |
|  Access (IA)   |            |          | (+ fee)    | content     |
+-------------------------------------------------------------------+
|  Glacier       | Rare       | $0.004   | Minutes    | Archives    |
```

```
|  Instant       |            |          |            |            |
+-----------------------------------------------------------------+
|  Glacier       | Very rare  | $0.0036  | Hours      | Compliance |
|  Flexible      |            |          | (3-5 hrs)  |            |
+-----------------------------------------------------------------+
|  Glacier       | Almost     | $0.00099 | 12 hours   | Legal      |
|  Deep Archive  | never      |          |            | retention  |
+-----------------------------------------------------------------+

Cost savings: Standard vs Deep Archive = 23x cheaper!
```

Lifecycle Policies

You can automate moving data between tiers based on age.

```
Lifecycle Policy Example:

Day 0-30:     STANDARD         (frequently accessed)
    |
    | After 30 days
    v
Day 30-90:    INFREQUENT ACCESS (accessed sometimes)
    |
    | After 90 days
    v
Day 90-365:   GLACIER INSTANT   (rarely accessed)
    |
    | After 1 year
    v
Day 365+:     DEEP ARCHIVE      (compliance retention)
    |
    | After 7 years
    v
DELETE                           (retention period over)

You set this policy once. S3 automatically moves
objects between tiers. No code needed.
```

Instagram Upload Flow: Putting It All Together

Let us trace what happens when a user uploads a photo to Instagram, from tap to feed.

```
Instagram Photo Upload Flow:
```

```
Step 1: Upload Request
+----------+    POST /upload    +------------+
|  Mobile  | ----------------> |  API       |
|  App     |                   |  Gateway   |
+----------+                   +-----+------+
                                     |
Step 2: Get Pre-Signed URL           |
+----------+  Pre-signed URL   +-----+------+
|  Mobile  | <---------------- |  Upload    |
|  App     |                   |  Service   |
+----------+                   +-----+------+
     |                               |
Step 3: Direct Upload to S3          |
     |    PUT (12 MB photo)          |
     +-----------------------> +----+----+
                               | S3 Raw  |
                               | Bucket  |
                               +----+----+
                                    |
Step 4: Event Trigger               |
                                    v
                               +-----+------+
                               | Event:     |
                               | PhotoUploaded
                               +-----+------+
                                    |
       +----------------------------+-----------------------------+
       |                            |                             |
       v                            v                             v
+------+------+              +-------+------+              +------+------+
| Image       |              | Content      |              | Metadata    |
| Processor   |              | Moderation   |              | Extractor   |
| - Resize    |              | - NSFW check |              | - EXIF data |
| - Compress  |              | - Violence   |              | - GPS strip |
| - Thumbnail |              | - Copyright  |              | - Tags      |
+------+------+              +-------+------+              +------+------+
       |                            |                             |
       v                            v                             v
+------+------+              +-------+------+              +------+------+
| S3 Processed|              | Approved?    |              | Database    |
| Bucket      |              | Yes/No       |              | (PostgreSQL)|
| /thumb/     |              +--------------+              +--------------+
| /medium/    |
| /full/      |
+------+------+
       |
Step 5: CDN Distribution
       v
+------+------+
|     CDN     |
| Edge servers|
| worldwide   |
+------+------+
       |
Step 6: Feed Update
```

```
        v
+------+------+     +------+------+
| Feed        |     | Notification|
| Service     |     | Service     |
| (Fan-out to |     | (Push to    |
|  followers) |     |  followers) |
+-------------+     +-------------+

Timeline:
t=0s:    User taps "Share"
t=0.5s:  Pre-signed URL received
t=3s:    Photo uploaded to S3
t=5s:    Image processing complete
t=5.5s:  Content moderation approved
t=6s:    Photo appears in user's profile
t=6-30s: Photo appears in followers' feeds
```

Key Design Decisions in the Instagram Flow

1. Pre-signed URLs keep the upload service stateless. It never touches the file data.
2. Event-driven processing allows parallel execution of image processing, content moderation, and metadata extraction.
3. Multiple image sizes are generated once and cached forever, saving compute on every view.
4. CDN caching means popular photos are served from edge locations, not from S3.
5. Content moderation runs asynchronously. The photo is not visible until approved.

Trade-offs

Decision	Option A	Option B
Storage type	Object storage (scalable, HTTP, cheap)	File storage (low latency, hierarchical, POSIX)
Upload method	Through your server (control, small files)	Pre-signed URL (scalable, large files)
Image processing	Synchronous on upload (slower upload, instant access)	Async via event (fast upload, brief delay)
Image format	JPEG (universal, larger)	WebP (smaller, newer browser support)

CDN strategy	Pull-based (CDN fetches from origin on miss)	Push-based (pre-populate CDN edges)
Storage tier	Keep everything in Standard (fast, expensive)	Lifecycle policy (cheaper, retrieval delay for old data)
Video	Store original only (save storage)	Transcode to multiple qualities (better user experience)

Common Mistakes

1. Routing file uploads through your application server. A 100 MB video upload ties up a server thread for minutes. Use pre-signed URLs to let clients upload directly to blob storage.
2. Not generating multiple image sizes. Serving a 12 MB original photo to a mobile device displaying a 150-pixel thumbnail wastes bandwidth and slows the app. Generate thumbnails and medium-size versions on upload.
3. Using the same S3 key pattern for all files. Keys like `/photos/1.jpg`, `/photos/2.jpg` concentrate requests on the same S3 partition. Use random prefixes or hashed keys to distribute load: `/a3f2/photos/1.jpg`.
4. Forgetting to set Cache-Control headers. Without proper cache headers, CDN edges re-fetch from S3 on every request, negating the benefit of having a CDN.
5. Not using lifecycle policies. Storing years of backup data in Standard tier wastes money. Move old data to cheaper tiers automatically.
6. Ignoring content moderation. Allowing unmoderated uploads exposes your platform to legal risk. Always scan uploads before making them publicly visible.

Best Practices

1. Use pre-signed URLs for all uploads and private downloads. This keeps your servers stateless and offloads bandwidth to the storage provider.
2. Generate multiple sizes and formats on upload. Create thumbnails, medium, and full-size versions. Convert to WebP for web delivery. This is a one-time compute cost that saves bandwidth on every view.

3. Use content-addressable keys. Name objects by their content hash: `sha256(content).jpg`. This provides automatic deduplication and makes cache invalidation trivial (new content = new key = new URL).
4. Set aggressive Cache-Control headers for immutable content. Images and videos rarely change. Cache them for a year: `Cache-Control: public, max-age=31536000, immutable`.
5. Configure lifecycle policies from day one. Define rules for moving data to cheaper tiers as it ages. This saves significant cost at scale.
6. Separate raw uploads from processed files. Use different buckets for raw uploads and processed files. This makes it easy to reprocess if you change your image pipeline.
7. Monitor storage costs. Track storage usage by bucket, tier, and age. Set alerts for unexpected growth.
8. Enable versioning for critical data. S3 versioning keeps previous versions of objects, protecting against accidental overwrites and deletes.

Quick Summary

Blob storage stores unstructured binary data -- photos, videos, documents, backups -- as objects identified by unique keys within buckets. Object storage differs from block and file storage in that it scales to petabytes, uses HTTP APIs, and trades random access for durability and simplicity. S3-like systems replicate data across multiple availability zones for eleven-nines durability. Metadata attached to objects enables searching and filtering without downloading file contents. Pre-signed URLs allow clients to upload and download directly from blob storage, keeping your servers stateless. CDN integration caches content at edge locations worldwide, reducing latency from hundreds of milliseconds to single digits. Image processing pipelines generate multiple sizes and formats on upload. Video transcoding creates multiple quality levels for adaptive bitrate streaming. Storage tiers let you save money by automatically moving less-accessed data to cheaper storage classes.

Key Points

- Blob storage stores unstructured data as objects with keys, metadata, and binary content
- Object storage scales to petabytes but does not support partial updates -- you replace the entire object
- S3 achieves eleven-nines durability by replicating across availability zones

- Pre-signed URLs let clients upload and download directly from blob storage, bypassing your servers
- CDN integration caches content at edge locations, reducing latency dramatically
- Image processing pipelines create multiple sizes and formats on upload to save bandwidth on every view
- Video transcoding produces multiple quality levels for adaptive bitrate streaming
- Storage tiers save money by moving less-accessed data to cheaper classes automatically

---

Practice Questions

1. You are designing a system where users upload profile photos. Each photo is 5 MB. You have 100 million users and 10% update their photo each month. How much storage do you need? Should you keep old versions? What storage tiers would you use?
2. A user in Tokyo uploads a photo that a user in New York wants to view 5 seconds later. Trace the request path from upload to view, including S3, CDN, and any processing. What is the expected latency for the viewer?
3. Your image processing pipeline crashes after resizing an image but before generating the thumbnail. The user refreshes the page and sees a broken image. How would you design the system to handle partial processing failures?
4. You are building a video streaming platform. A creator uploads a 2 GB video. It needs to be available in 1080p, 720p, and 480p. How would you design the transcoding pipeline? How long should the creator wait before the video is available?
5. Your S3 bill jumped from $10,000 to $50,000 this month. What are the most likely causes? How would you investigate and reduce costs?

Exercises

Exercise 1: Design a Document Storage System

Design a blob storage architecture for a company document management system. Users upload Word documents, PDFs, and spreadsheets. Requirements: version history, full-text search of document contents, role-based access control, and audit logging. Draw the architecture showing upload flow, storage, indexing, and retrieval.

Exercise 2: Build a CDN Strategy

You run a photo-sharing platform with 50 million daily active users. 80% of photo views happen within the first 24 hours of upload. 15% happen in the first week. 5% happen after one week. Design a CDN and storage strategy that minimizes cost while keeping the experience fast. Include cache warming, TTL policies, and storage tier transitions.

Exercise 3: Image Processing at Scale

Your e-commerce platform has 10 million product images. A new business requirement asks you to add watermarks to all images and convert them from JPEG to WebP format. Design a batch processing pipeline that can reprocess all 10 million images within 24 hours. Calculate the compute resources needed assuming each image takes 2 seconds to process.

What Is Next?

You now understand how to store, process, and deliver large files at scale. But what happens when a user wants to find something specific among millions of items? Typing "blue running shoes size 10" and getting relevant results in under 100 milliseconds requires a completely different kind of system. In the next chapter, you will learn about search systems: how inverted indexes work, how Elasticsearch powers full-text search, and how to build search architectures that handle millions of queries per second.

Chapter 18: Search Systems

What You Will Learn

- Why full-text search is fundamentally different from SQL LIKE queries
- How an inverted index works (and why it is like a book's index)
- How Elasticsearch organizes data into indexes, shards, and replicas
- How an indexing pipeline keeps search data fresh
- How relevance scoring works with TF-IDF and BM25
- How fuzzy matching handles typos and misspellings
- How autocomplete and typeahead suggestions work
- How to design a search architecture for an e-commerce platform

Why This Chapter Matters

Imagine you are looking for a specific topic in a 500-page textbook. You have two options. Option one: start at page one and read every single page until you find what you need. Option two: open the index at the back of the book, look up the topic alphabetically, and jump directly to the right page.

Nobody chooses option one. It is absurdly slow. Yet that is exactly what a SQL `LIKE '%search term%'` query does -- it scans every single row in the table looking for a match. When your database has 100 million products, this takes minutes.

Search systems solve this by building the equivalent of a book's index for your data. They make it possible to search billions of documents and get relevant results in milliseconds. Every application with a search bar -- Google, Amazon, Netflix, Spotify -- depends on these concepts. Understanding search systems is essential for system design interviews and for building products that users actually enjoy.

Full-Text Search vs LIKE

The Problem with LIKE

SQL databases support the `LIKE` operator for text matching. It works for small datasets, but it falls apart at scale.

```
SQL LIKE Query:

SELECT * FROM products WHERE name LIKE '%running shoes%';

What the database does:

Row 1:  "Blue T-Shirt"           -- scan... no match
Row 2:  "Red Dress"              -- scan... no match
Row 3:  "Running Shoes Nike"     -- scan... MATCH!
Row 4:  "Green Hat"              -- scan... no match
Row 5:  "Black Jacket"           -- scan... no match
...
Row 10,000,000: "Yellow Socks"   -- scan... no match

Scanned ALL 10 million rows to find matches.
Time: 8 seconds. Unacceptable for a user-facing search.

Problems with LIKE:
1. Full table scan (no index can help with leading %)
2. No relevance ranking (which result is "best"?)
3. No fuzzy matching ("runnng shoes" returns nothing)
4. No stemming ("run" does not match "running")
5. No synonym support ("sneakers" does not match "shoes")
```

Full-Text Search

Full-text search pre-processes your data into a structure optimized for text queries. Instead of scanning every document at query time, it looks up pre-built indexes in milliseconds.

```
Full-Text Search:

Step 1: At index time, build an inverted index (done once, updated incrementally)

Step 2: At query time, look up the index (instant)

Query: "running shoes"

Inverted Index Lookup:
  "running" --> [doc 3, doc 47, doc 982, doc 5,401]
  "shoes"   --> [doc 3, doc 12, doc 47, doc 1,203]
```

```
Intersection: [doc 3, doc 47]  (both words appear)

Time: 5 milliseconds (even with 10 million documents)
```

Feature	SQL LIKE	Full-Text Search
Speed (10M docs)	Seconds to minutes	Milliseconds
Relevance ranking	No	Yes (scored results)
Fuzzy matching	No	Yes (handles typos)
Stemming	No	Yes (run = running)
Synonyms	No	Yes (shoes = sneakers)
Index usage	Cannot use index with leading %	Purpose-built inverted index

The Inverted Index

The inverted index is the core data structure that makes full-text search fast. It is called "inverted" because it inverts the relationship between documents and words.

Forward Index vs Inverted Index

A forward index maps documents to words (what words does this document contain?). An inverted index maps words to documents (which documents contain this word?).

```
Forward Index (Document -> Words):
Like reading a book page by page.

Doc 1: "The quick brown fox"     --> [the, quick, brown, fox]
Doc 2: "The lazy brown dog"      --> [the, lazy, brown, dog]
Doc 3: "Quick fox jumps high"    --> [quick, fox, jumps, high]

To find "fox": Must check EVERY document. Slow.


Inverted Index (Word -> Documents):
Like using a book's index at the back.

"brown"  --> [Doc 1, Doc 2]
```

```
"dog"    --> [Doc 2]
"fox"    --> [Doc 1, Doc 3]
"high"   --> [Doc 3]
"jumps"  --> [Doc 3]
"lazy"   --> [Doc 2]
"quick"  --> [Doc 1, Doc 3]
"the"    --> [Doc 1, Doc 2]

To find "fox": Look up one entry. Instant.
```

The Book Index Analogy

Think of a textbook on computer science. At the back, you find an index:

```
Book Index (This IS an inverted index!):

Algorithm ............. 12, 45, 78, 203
Binary Search .......... 45, 89
Cache ................. 34, 67, 201
Database ............... 12, 56, 99, 145
Hash Table ............ 78, 112
...

Want to learn about "Cache"?
Go directly to pages 34, 67, and 201.
No need to read all 500 pages.

This is exactly how a search engine works:
- "Pages" = documents in your database
- "Topics" = words/terms in the inverted index
- "Page numbers" = document IDs
```

Building an Inverted Index

The process of building an inverted index involves several text processing steps.

```
Building an Inverted Index:

Input Documents:
Doc 1: "Nike Running Shoes - Lightweight and comfortable"
Doc 2: "Adidas Running Sneakers for Marathon Runners"
Doc 3: "Nike Air Max - Comfortable Walking Shoes"

Step 1: Tokenization (split into words)
Doc 1: [Nike, Running, Shoes, Lightweight, and, comfortable]
Doc 2: [Adidas, Running, Sneakers, for, Marathon, Runners]
Doc 3: [Nike, Air, Max, Comfortable, Walking, Shoes]
```

```
Step 2: Lowercasing
Doc 1: [nike, running, shoes, lightweight, and, comfortable]
Doc 2: [adidas, running, sneakers, for, marathon, runners]
Doc 3: [nike, air, max, comfortable, walking, shoes]

Step 3: Stop word removal (remove "and", "for", "the", etc.)
Doc 1: [nike, running, shoes, lightweight, comfortable]
Doc 2: [adidas, running, sneakers, marathon, runners]
Doc 3: [nike, air, max, comfortable, walking, shoes]

Step 4: Stemming (reduce words to root form)
Doc 1: [nike, run, shoe, lightweight, comfort]
Doc 2: [adidas, run, sneaker, marathon, runner]
Doc 3: [nike, air, max, comfort, walk, shoe]

Step 5: Build Inverted Index
+-------------+------------------+
| Term        | Document IDs     |
+-------------+------------------+
| adidas      | [2]              |
| air         | [3]              |
| comfort     | [1, 3]          |
| lightweight | [1]              |
| marathon    | [2]              |
| max         | [3]              |
| nike        | [1, 3]          |
| run         | [1, 2]          |
| runner      | [2]              |
| shoe        | [1, 3]          |
| sneaker     | [2]              |
| walk        | [3]              |
+-------------+------------------+

Now searching for "comfortable running shoes":
- "comfort" -> [1, 3]
- "run"     -> [1, 2]
- "shoe"    -> [1, 3]

Intersection: Doc 1 matches all three terms. Best match!
Doc 3 matches "comfort" and "shoe" but not "run". Second best.
Doc 2 matches "run" only. Third.
```

Elasticsearch: Index, Shard, Replica

Elasticsearch is the most popular open-source search engine. It is built on Apache Lucene and designed for distributed, scalable search.

Core Concepts

```
Elasticsearch Architecture:

CLUSTER (group of nodes working together)
+-------------------------------------------------------------------+
|                                                                   |
|  NODE 1 (Server)        NODE 2 (Server)        NODE 3 (Server)    |
|  +-----------------+  +-----------------+   +-----------------+ |
|  |                 |  |                 |   |                 | |
|  |  Shard 0        |  |  Shard 1        |   |  Shard 2        | |
|  |  (Primary)      |  |  (Primary)      |   |  (Primary)      | |
|  |  +------------+  |  |  +------------+  |   |  +------------+  | |
|  |  | Inverted   |  |  |  | Inverted   |  |   |  | Inverted   |  | |
|  |  | Index      |  |  |  | Index      |  |   |  | Index      |  | |
|  |  +------------+  |  |  +------------+  |   |  +------------+  | |
|  |                 |  |                 |   |                 | |
|  |  Shard 1        |  |  Shard 2        |   |  Shard 0        | |
|  |  (Replica)      |  |  (Replica)      |   |  (Replica)      | |
|  |  +------------+  |  |  +------------+  |   |  +------------+  | |
|  |  | Inverted   |  |  |  | Inverted   |  |   |  | Inverted   |  | |
|  |  | Index      |  |  |  | Index      |  |   |  | Index      |  | |
|  |  +------------+  |  |  +------------+  |   |  +------------+  | |
|  |                 |  |                 |   |                 | |
|  +-----------------+  +-----------------+   +-----------------+ |
|                                                                   |
+-------------------------------------------------------------------+

INDEX: A collection of documents (like a database table)
  Example: "products" index with 10 million product documents

SHARD: A horizontal partition of an index
  The "products" index is split into 3 shards
  Each shard holds ~3.3 million documents
  Each shard is a self-contained Lucene index

REPLICA: A copy of a shard on a different node
  Each primary shard has one replica
  If Node 1 dies, Shard 0's replica on Node 3 takes over
```

Why Sharding?

```
Without Sharding:                 With Sharding:

One huge index:                   Split across nodes:

+-----------------+               +--------+ +--------+ +--------+
| 10M documents   |               | 3.3M   | | 3.3M   | | 3.3M   |
| One server      |               | Node 1 | | Node 2 | | Node 3 |
| All queries hit |               +--------+ +--------+ +--------+
```

```
| this one server  |                 |           |           |
+------------------+              Query hits all 3 in parallel
                                  Results merged and returned
Limited by single                 3x throughput, 3x capacity
server capacity
```

How a Search Query Works in Elasticsearch

```
Search Query Flow:

1. Client sends query to any node (coordinator)
+--------+   "running shoes"   +--------+
| Client | ------------------> | Node 1 |  (Coordinator)
+--------+                     +--------+
                                    |
2. Coordinator sends query to one copy of each shard
                                    |
                  +-----------------+-----------------+
                  |                                   |
                  v                                   v
            +--------+                          +--------+
            | Node 2 |                          | Node 3 |
            | Shard 1|                          | Shard 2|
            +--------+                          +--------+
                  |                                   |
3. Each shard searches its local inverted index
   and returns top matching document IDs + scores
                  |                                   |
                  v                                   v
            [doc 47: 8.5]                       [doc 982: 9.1]
            [doc 103: 7.2]                      [doc 501: 6.8]
                  |                                   |
                  +-----------------+-----------------+
                                    |
4. Coordinator merges results, sorts by score
                                    |
                                    v
            Final results: [doc 982: 9.1, doc 47: 8.5,
                            doc 103: 7.2, doc 501: 6.8]
                                    |
5. Coordinator fetches full documents for top results
                                    |
                                    v
+--------+   [results with data]   +--------+
| Client | <---------------------- | Node 1 |
+--------+                         +--------+
```

Indexing Pipeline

The indexing pipeline is how data gets from your primary database into the search engine. This is a critical but often overlooked part of search architecture.

The Challenge

Your primary data lives in a relational database (PostgreSQL, MySQL). Your search engine (Elasticsearch) needs a copy of that data in its inverted index format. How do you keep them in sync?

```
Indexing Pipeline:

+------------+                          +---------------+
| PostgreSQL |                          | Elasticsearch |
| (Source of |    Indexing Pipeline      | (Search       |
|  Truth)    | ----------------------> |  Engine)      |
+------------+                          +---------------+

Options:

1. Dual Write (Simple but Risky)
+------+    Write    +--------+
| App  | ----------> | Postgres|
|      | ----------> | Elastic |  <-- What if one write fails?
+------+             +--------+      Data gets out of sync.

2. Change Data Capture (CDC) - Recommended
+------+    Write    +----------+    CDC Stream    +----------+
| App  | ----------> | Postgres | --------------> | Indexer  |
+------+             +----------+    (Debezium)   +----+-----+
                                                        |
                                                        v
                                                   +----------+
                                                   | Elastic  |
                                                   +----------+

CDC reads the database's transaction log (WAL)
and streams changes to the indexer in real time.
No risk of dual-write inconsistency.

3. Periodic Batch Reindex
+----------+   Every 5 min   +----------+    Bulk Index    +----------+
| Postgres | -------------> | Batch Job | --------------> | Elastic  |
+----------+   (cron job)    +----------+                  +----------+

Simple but introduces up to 5 minutes of delay.
Acceptable for some use cases (product catalog).
```

Recommended Architecture

```
Production Indexing Pipeline:

+--------+  Write  +----------+  WAL Stream  +----------+
| App    | ------> | Postgres | -----------> | Debezium |
| Server |         | (Source  |  (CDC)       | (CDC     |
+--------+         |  of Truth|              |  Tool)   |
                   +----------+              +----+-----+
                                                  |
                                            +-----v-----+
                                            |   Kafka    |
                                            | (Buffer)   |
                                            +-----+------+
                                                  |
                                            +-----v------+
                                            |  Indexer   |
                                            |  Service   |
                                            | - Transform|
                                            | - Enrich   |
                                            | - Validate |
                                            +-----+------+
                                                  |
                                            +-----v--------+
                                            | Elasticsearch |
                                            +---------------+

The indexer service:
- Reads change events from Kafka
- Transforms data (flatten nested objects, combine fields)
- Enriches data (add category names, compute popularity)
- Validates (skip invalid records)
- Bulk-indexes into Elasticsearch
```

Relevance Scoring: TF-IDF and BM25

When a user searches for "running shoes," the search engine does not just find matching documents. It ranks them by relevance. The most relevant document appears first.

TF-IDF: Term Frequency - Inverse Document Frequency

TF-IDF is the foundational relevance scoring formula. It combines two ideas.

Term Frequency (TF): How often does the search term appear in this document? More occurrences = more relevant.

Inverse Document Frequency (IDF): How rare is this term across all documents? Rare terms are more important than common ones.

```
TF-IDF Example:

Search: "running shoes"

Document A: "Running shoes for marathon running. Best running shoes."
Document B: "Nike shoes for casual walking."
Document C: "The best running shoes guide for beginners."

Term Frequency (TF) for "running":
Doc A: 3 occurrences in 8 words  = 3/8 = 0.375
Doc B: 0 occurrences in 5 words  = 0/5 = 0.000
Doc C: 1 occurrence  in 7 words  = 1/7 = 0.143

Inverse Document Frequency (IDF) for "running":
3 total documents, 2 contain "running"
IDF = log(3/2) = 0.176  (not very rare)

IDF for "shoes":
3 total documents, 3 contain "shoes"
IDF = log(3/3) = 0.000  (appears everywhere, not discriminating)

IDF for "marathon":
3 total documents, 1 contains "marathon"
IDF = log(3/1) = 0.477  (rare, very discriminating!)

Key insight: Common words like "the" and "shoes" have low IDF.
Rare, specific words like "marathon" have high IDF.
```

BM25: The Modern Standard

BM25 (Best Matching 25) is an improved version of TF-IDF used by Elasticsearch and most modern search engines. It fixes two problems with basic TF-IDF.

Problem 1: TF saturation. In TF-IDF, a document mentioning "running" 100 times scores much higher than one mentioning it 5 times. But is it really 20 times more relevant? BM25 adds a saturation curve: after a certain point, additional occurrences matter less and less.

Problem 2: Document length bias. A 10,000-word document is more likely to contain a search term than a 50-word document, simply because it has more words. BM25 normalizes for document length.

```
TF-IDF vs BM25: Term Frequency Curves

Score
^
```

```
|          TF-IDF (linear)
|         /
|        /
|       /              BM25 (saturating)
|      /          ___________
|     /       ___/
|    /     ___/
|   / ___/
|  //
| /
+-------------------------> Term Frequency

BM25 says: "Yes, mentioning 'running' 5 times is better
than 1 time. But mentioning it 50 times is only slightly
better than 5 times."
```

Boosting

You can boost the importance of certain fields. A match in the product title is more relevant than a match in the description.

```
Field Boosting:

Search: "running shoes"

Product:
  title: "Nike Running Shoes"          (boost: 3x)
  description: "Great for running..."   (boost: 1x)
  category: "Footwear"                 (boost: 0.5x)

A match in the title contributes 3x more to the score
than a match in the description.
```

Fuzzy Matching

Users make typos. A search engine that returns zero results for "runnng shoes" is frustrating. Fuzzy matching finds results even when the query has spelling errors.

Edit Distance (Levenshtein Distance)

Fuzzy matching uses edit distance: the minimum number of single-character edits (insertions, deletions, or substitutions) needed to transform one word into another.

```
Edit Distance Examples:

"runnng" -> "running"    (1 insertion: add 'i')     Distance: 1
"shoez"  -> "shoes"      (1 substitution: z -> s)   Distance: 1
"rnning" -> "running"    (1 insertion: add 'u')     Distance: 1
"cat"    -> "car"        (1 substitution: t -> r)   Distance: 1
"cat"    -> "cart"       (1 insertion: add 't')     Distance: 1
"kitten" -> "sitting"    (3 edits)                  Distance: 3

Elasticsearch default: Allow edit distance of 2
"runnng shoes" matches "running shoes" (distance 1)
```

How Fuzzy Search Works

```
Fuzzy Search Flow:

Query: "runnng"  (typo)

Step 1: Generate variations within edit distance 2
  runng, running, runnig, runnin, runnmg, ...

Step 2: Look up each variation in the inverted index
  "running" --> [Doc 1, Doc 3, Doc 47]   FOUND!

Step 3: Return results with slightly lower score
  (penalize fuzzy matches vs exact matches)

User sees results for "running" even though they typed "runnng"
```

Autocomplete

Autocomplete (typeahead) suggests results as the user types, before they press Enter. It requires a different index structure optimized for prefix matching.

How Autocomplete Works

```
Autocomplete Flow:

User types: "r"  -> [running shoes, red dress, rain jacket]
User types: "ru" -> [running shoes, rubber boots, rug]
User types: "run" -> [running shoes, running watch, run tracker]
User types: "runn" -> [running shoes, running watch]
```

```
Each keystroke triggers a search.
Results must return in under 100ms or the UI feels laggy.
```

Implementation Approaches

1. Prefix Index (Edge N-grams)

Break each word into prefixes and index them separately.

```
Edge N-grams for "running":

r
ru
run
runn
runni
runnin
running

Each prefix is a separate entry in the inverted index.
When user types "run", instant lookup finds all words
starting with "run".
```

2. Completion Suggester (Elasticsearch)

Elasticsearch has a built-in completion suggester that uses an in-memory data structure (FST: Finite State Transducer) for extremely fast prefix lookups.

```
Completion Suggester Architecture:

+--------+   "run"    +-------------+   FST Lookup   +-----------+
| Client | ---------> | Elasticsearch| ------------> | In-Memory |
+--------+            | API          |               | FST       |
                      +-------------+               +-----------+
     ^                                                    |
     |                                                    v
     |              Suggestions:                    [running shoes,
     +---------------------------------------------- running watch,
                    (under 5ms)                       run tracker]

FST: Finite State Transducer
- Compresses all terms into a compact graph
- Prefix lookups in microseconds
- Lives entirely in memory
```

3. Popular Queries (Search-as-You-Type)

Instead of searching the product index, search a separate index of popular queries.

```
Popular Query Autocomplete:

+-------------------------------------------+
| Popular Queries Index                      |
+-------------------------------------------+
| Query              | Count  | Last 7 Days |
+-------------------------------------------+
| running shoes      | 45,000 | 12,000      |
| running watch      | 8,200  | 2,100       |
| running shorts     | 6,500  | 1,800       |
| run tracker app    | 3,100  | 900         |
+-------------------------------------------+

User types "run" -> return top queries by popularity
Much faster than searching millions of products
Shows what other users searched for (social proof)
```

E-Commerce Search Architecture

Let us design a complete search system for an e-commerce platform like Amazon or Shopify.

Requirements

- 50 million products
- 10,000 search queries per second at peak
- Results in under 200 milliseconds
- Relevance ranking, fuzzy matching, filtering, sorting
- Autocomplete with under 100 millisecond latency

Architecture

```
E-Commerce Search Architecture:

+----------+                              +------------------+
| Mobile/  |   "blue running shoes"       |   API Gateway    |
| Web App  | ---------------------------> | (Rate Limiting,  |
+----------+                              |  Auth)           |
                                          +--------+---------+
                                                   |
                                          +--------v---------+
```

```
                                        |  Search Service  |
                                        |                  |
                                        | - Parse query    |
                                        | - Apply filters  |
                                        | - Build ES query |
                                        +--------+---------+
                                                 |
                              +----------------------+
                              |                      |
                              v                      v
                     +--------+--------+    +--------+--------+
                     |  Elasticsearch   |    |  Autocomplete   |
                     |  Cluster         |    |  Index          |
                     |  (Main Search)   |    |  (Suggestions)  |
                     |                  |    |                  |
                     |  50M products    |    |  Popular queries |
                     |  5 shards        |    |  In-memory FST   |
                     |  1 replica each  |    +--------+--------+
                     +--------+--------+
                              |
                    +---------+---------+
                    |                   |
                    v                   v
              +-----------+      +-----------+
              | Relevance |      | Filters   |
              | Scoring   |      | & Facets  |
              | (BM25)    |      | (brand,   |
              |           |      |  price,   |
              |           |      |  size,    |
              |           |      |  color)   |
              +-----------+      +-----------+


Data Flow (Indexing Pipeline):

+-----------+   CDC    +----------+   Kafka   +----------+   Bulk    +----------+
| Product   | -------> | Debezium | -------> | Indexer  | -------> | Elastic  |
| Database  |          +----------+          | Service  |          | search   |
| (Postgres)|                                +----------+          +----------+
+-----------+                                      |
                                            +-----v------+
                                            | Enrichment |
                                            | - Synonyms |
                                            | - Spelling |
                                            | - Category |
                                            | - Popularity|
                                            +------------+
```

Query Processing Flow

```
What Happens When You Search "blue runnng shoes size 10":

Step 1: Query Parsing
+-------------------------------------------------------------+
| Input: "blue runnng shoes size 10"                          |
|                                                             |
| - Tokenize: [blue, runnng, shoes, size, 10]                 |
| - Spell check: runnng -> running                            |
| - Identify filters: size=10                                 |
| - Remaining search terms: [blue, running, shoes]            |
+-------------------------------------------------------------+

Step 2: Query Building
+-------------------------------------------------------------+
| Elasticsearch Query:                                        |
| {                                                           |
|   "query": {                                                |
|     "bool": {                                               |
|       "must": [                                             |
|         {"multi_match": {                                   |
|           "query": "blue running shoes",                    |
|           "fields": ["title^3", "description", "brand^2"],|
|           "fuzziness": "AUTO"                               |
|         }}                                                  |
|       ],                                                    |
|       "filter": [                                           |
|         {"term": {"sizes": "10"}},                          |
|         {"term": {"in_stock": true}}                        |
|       ]                                                     |
|     }                                                       |
|   },                                                        |
|   "aggs": {                                                 |
|     "brands": {"terms": {"field": "brand"}},                |
|     "price_ranges": {"range": {"field": "price", ...}}      |
|   }                                                         |
| }                                                           |
+-------------------------------------------------------------+

Step 3: Results
+-------------------------------------------------------------+
| Results (sorted by relevance score):                        |
|                                                             |
| 1. Nike Air Zoom Running Shoes - Blue (Score: 12.4)         |
|    Size 10 in stock. $129.99                                |
|                                                             |
| 2. Adidas Ultraboost Running Shoes - Blue (Score: 11.2)     |
|    Size 10 in stock. $179.99                                |
|                                                             |
| 3. New Balance Fresh Foam - Blue (Score: 9.8)               |
|    Size 10 in stock. $99.99                                 |
|                                                             |
| Facets:                                                     |
|   Brands: Nike (45), Adidas (38), New Balance (22)          |
|   Price: Under $100 (12), $100-$200 (28), Over $200 (5)  |
|   Colors: Blue (45), Black (32), White (18)                 |
```

```
+-----------------------------------------------------------+
```

Faceted Search

Facets are the filter counts you see on the left side of e-commerce search results. They tell the user how many products match each filter option.

```
Faceted Search:

Search: "running shoes"

+-------------------+     +-----------------------------+
|  FACETS (Filters) |     |  RESULTS                    |
|                   |     |                             |
|  Brand:           |     |  1. Nike Air Zoom           |
|  [ ] Nike (245)   |     |     $129.99 ****            |
|  [ ] Adidas (189) |     |                             |
|  [ ] New Bal (98) |     |  2. Adidas Ultraboost       |
|                   |     |     $179.99 ****             |
|  Price:           |     |                             |
|  [ ] Under $50    |     |  3. New Balance 1080        |
|  [ ] $50-$100     |     |     $149.99 ***              |
|  [ ] $100-$200    |     |                             |
|  [ ] Over $200    |     |  4. Brooks Ghost 15         |
|                   |     |     $139.99 ****             |
|  Size:            |     |                             |
|  [ ] 8  (120)     |     |                             |
|  [ ] 9  (145)     |     |                             |
|  [ ] 10 (138)     |     |                             |
|  [ ] 11 (98)      |     |                             |
+-------------------+     +-----------------------------+

Elasticsearch aggregations compute these counts
alongside the search results in a single query.
```

Trade-offs

Decision	Option A	Option B
Search engine	Elasticsearch (feature-rich, complex)	Database full-text (simpler, limited)

Indexing	Real-time CDC (fresh data, complex)	Batch reindex (delayed, simple)
Autocomplete	Edge n-grams (flexible, more storage)	Completion suggester (fast, less flexible)
Relevance	BM25 default (good enough for most)	Custom scoring with ML (better, complex)
Sharding	More shards (parallelism, overhead)	Fewer shards (simpler, limited throughput)
Consistency	Real-time index (always fresh)	Near-real-time (seconds delay, less load)

Common Mistakes

1. Using SQL LIKE for user-facing search. It does not scale, does not rank by relevance, and does not handle typos. Use a proper search engine for any search feature users interact with.
2. Not handling synonyms. Users search for "sneakers" but your products say "shoes." Without a synonym dictionary, they get zero results. Configure synonyms in your analyzer.
3. Over-sharding. Creating 20 shards for an index with 100,000 documents wastes resources. Each shard has overhead. A good rule: aim for 10 to 50 GB per shard.
4. Dual-writing to the database and search engine. If one write succeeds and the other fails, your data is out of sync. Use CDC (Change Data Capture) to stream changes from the database to the search engine.
5. Ignoring relevance tuning. The default BM25 scoring works but is rarely optimal. Invest time in field boosting, custom scoring, and A/B testing different relevance formulas.
6. Not monitoring search quality. Track click-through rates, zero-result rates, and search abandonment. If 20% of searches return no results, your search is broken.

Best Practices

1. Use Change Data Capture for indexing. CDC (tools like Debezium) reads the database transaction log and streams changes to your indexer. This is the most reliable way to

keep search in sync with your primary database.

2. Configure analyzers thoughtfully. Choose the right tokenizer, stemmer, and stop word list for your language. Test with real user queries.
3. Add synonyms and spelling correction. Build a synonym dictionary for your domain. Use fuzzy matching with edit distance 1-2 to handle typos.
4. Monitor zero-result rates. Track how often searches return no results. These are failed user experiences. Add the missing terms to your synonym dictionary or adjust your analyzer.
5. Use field boosting. Matches in the product title should score higher than matches in the description. Experiment with boost values and measure click-through rates.
6. Separate autocomplete from search. Use a dedicated completion suggester index for autocomplete. It is faster and can be optimized independently.
7. Plan shard allocation. Aim for 10 to 50 GB per shard. Use time-based indices for logs (one index per day). Monitor shard size and rebalance when needed.
8. Implement search analytics. Log every search query, the results shown, and which result the user clicked. This data is essential for improving relevance.

Quick Summary

Full-text search uses inverted indexes to find matching documents in milliseconds, unlike SQL LIKE which scans every row. An inverted index maps terms to document IDs, just like a book's index maps topics to page numbers. Building the index involves tokenization, lowercasing, stop word removal, and stemming. Elasticsearch distributes data across shards for parallelism and replicas for fault tolerance. An indexing pipeline using Change Data Capture keeps search data in sync with the primary database. BM25 scores relevance by combining term frequency with inverse document frequency, adding saturation and length normalization. Fuzzy matching handles typos using edit distance. Autocomplete uses edge n-grams or completion suggesters for sub-100-millisecond prefix lookups. E-commerce search combines full-text search with filters, facets, and relevance tuning to help users find products fast.

Key Points

- Inverted indexes map terms to documents, enabling millisecond lookups instead of full table scans
- Text processing (tokenization, stemming, stop words) is critical for accurate matching
- Elasticsearch shards data for parallelism and replicates for fault tolerance
- BM25 improves on TF-IDF by adding term frequency saturation and document length normalization
- Fuzzy matching with edit distance handles typos without requiring exact spelling
- Autocomplete needs a separate, optimized index for sub-100-millisecond responses
- Change Data Capture is the safest way to keep search indexes in sync with primary databases
- Relevance tuning, synonyms, and search analytics are essential for a good search experience

---

Practice Questions

1. Your e-commerce site has 50 million products. Users report that search results take 3 seconds. You are using a single Elasticsearch node with one shard. What changes would you make to bring latency under 200 milliseconds?
2. Users search for "iPhone charger" but your products are listed as "Lightning cable." How would you solve this? What are the trade-offs between synonyms, query expansion, and manual curation?
3. Your autocomplete suggestions take 500 milliseconds to return. The product index has 50 million documents. How would you redesign the autocomplete system to achieve sub-100-millisecond latency?
4. You are indexing product data from PostgreSQL into Elasticsearch using a cron job every 5 minutes. A product's price changes, but the old price shows in search results for up to 5 minutes. How would you reduce this delay without overloading either system?
5. Your search analytics show that 15% of queries return zero results. What strategies would you use to reduce this rate? How would you measure improvement?

Exercises

Exercise 1: Build an Inverted Index

Given these five product descriptions, manually build an inverted index. Apply tokenization, lowercasing, stop word removal, and stemming. Then trace how the search query "comfortable waterproof jacket" would be processed against your index. Which documents match? How would you rank them?

Products:

- Doc 1: "Waterproof Rain Jacket - Lightweight and Breathable"
- Doc 2: "Comfortable Cotton T-Shirt for Everyday Wear"
- Doc 3: "Men's Waterproof Hiking Jacket - Ultra Comfortable"
- Doc 4: "Leather Jacket - Classic Style"
- Doc 5: "Women's Comfortable Running Shoes - Waterproof"

Exercise 2: Design a Recipe Search Engine

Design the search architecture for a recipe website with 5 million recipes. Users should be able to search by ingredients ("chicken, garlic, lemon"), cooking time ("under 30 minutes"), dietary restrictions ("gluten-free"), and free text ("easy weeknight dinner"). Define the Elasticsearch index mapping, the indexing pipeline, and the query structure. Include autocomplete suggestions.

Exercise 3: Search Relevance Experiment

You run a bookstore with 1 million books. A user searches for "python programming." Currently, the top result is "Python: The Complete Reference" (published 2005, 2 reviews). But users consistently click on "Python Crash Course" (published 2023, 5000 reviews). Design a custom scoring formula that incorporates publication date, review count, and rating alongside BM25 text relevance. Explain how you would A/B test your new formula against the default.

What Is Next?

You now know how to build search systems that find relevant results in milliseconds across millions of documents. But finding information is only half the challenge. Users also need to be notified when things happen: when their order ships, when someone messages them, when a price drops. In the next chapter, you will learn how to design a notification system that delivers push notifications, SMS, emails, and in-app alerts at scale, handling millions of notifications per day with proper prioritization, throttling, and delivery tracking.

Chapter 19: Notification System

What You Will Learn

- The four types of notifications: push, SMS, email, and in-app
- How to design a notification service architecture from scratch
- How notification templates work and why they save engineering time
- How user preferences and opt-out management work
- How priority levels and throttling prevent notification fatigue
- How to track delivery status across multiple channels
- How retry logic handles transient failures
- How APNs, FCM, and SMTP deliver notifications to devices and inboxes
- How fan-out sends one notification to millions of users
- How Uber's ride notification flow works end to end

Why This Chapter Matters

Your phone buzzes. You glance down. "Your Uber is arriving in 2 minutes." That simple message seems effortless, but behind it is a system that chose the right channel (push notification, not email), personalized the message (your driver's name, car model, license plate), delivered it in under a second, tracked whether you received it, and decided not to send you an SMS because you already saw the push notification.

Notifications are how applications stay connected with their users. A well-designed notification system increases engagement and provides timely information. A poorly designed one annoys users into disabling notifications or uninstalling the app entirely. Every company with a mobile app or web presence needs a notification system. Understanding how to build one at scale is one of the most practical system design skills you can have.

Types of Notifications

There are four primary notification channels, each with different characteristics, costs, and user expectations.

Push Notifications

Messages sent directly to a user's mobile device or web browser. They appear on the lock screen or as banners even when the app is not open.

```
Push Notification Flow:

+--------+                    +---------+                    +--------+
| Your   |   Send message    | APNs or |   Deliver to      | User's |
| Server | ----------------> | FCM     | ----------------> | Device |
+--------+                   +---------+                   +--------+

APNs = Apple Push Notification service (iOS)
FCM  = Firebase Cloud Messaging (Android + Web)

Characteristics:
- Cost: Free (no per-message charge)
- Delivery: Near-instant (under 1 second)
- Rich content: Images, buttons, sounds
- Requires: User permission, device token
- Risk: Users can disable or uninstall
```

SMS (Text Messages)

Messages sent to a user's phone number via cellular networks. They work on any phone, even without internet.

```
SMS Flow:

+--------+                    +----------+                  +--------+
| Your   |   API call        | Twilio / |   Cellular       | User's |
| Server | ----------------> | AWS SNS  | ----------------> | Phone  |
+--------+                   +----------+                  +--------+

Characteristics:
- Cost: $0.01-$0.05 per message (expensive at scale)
- Delivery: 1-30 seconds
- Content: Text only, 160 characters
- Requires: Phone number
- Works without internet or smartphone
- Best for: Security codes (2FA), critical alerts
```

Email

Messages delivered to a user's email inbox. Rich HTML content with links, images, and formatting.

```
Email Flow:

+--------+                    +-----------+                 +--------+
| Your   |   SMTP/API        | SendGrid /|   Email          | User's |
| Server | ----------------> | SES /     | ----------------> | Inbox  |
+--------+                   | Mailgun   |                 +--------+
                             +-----------+

Characteristics:
- Cost: $0.0001-$0.001 per email (cheap)
- Delivery: Seconds to minutes
- Rich content: HTML, images, attachments
- Requires: Email address
- Risk: Spam filters may block delivery
- Best for: Receipts, newsletters, detailed updates
```

In-App Notifications

Messages displayed within your application when the user is actively using it. They appear as badges, banners, or notification feeds.

```
In-App Notification Flow:

+--------+                    +-----------+                 +--------+
| Your   |   WebSocket or    | Client    |   Show in        | User's |
| Server | ----------------> | App       | ----------------> | Screen |
+--------+   Server-Sent     +-----------+   notification   +--------+
             Events (SSE)                     center

Characteristics:
- Cost: Free (within your infrastructure)
- Delivery: Instant (if user is online)
- Rich content: Any UI element
- Requires: User to be in the app
- No permissions needed
- Best for: Activity feeds, social updates, badges
```

Channel Comparison

Feature	Push	SMS	Email	In-App

Cost per message	Free	$0.01-0.05	$0.0001	Free
Delivery speed	<1 second	1-30 seconds	Seconds-minutes	Instant
Reach	App installed	Any phone	Any email	App open
Rich content	Limited	No	Yes	Yes
User permission	Required	Varies by country	CAN-SPAM rules	None
Offline delivery	Yes (queued)	Yes	Yes	No

Notification Service Architecture

Let us design the architecture for a notification system that handles all four channels at scale.

```
Notification Service Architecture:

+----------+    +-----------+    +------------------+    +-----------+
| Order    |--->|           |    |                  |    |           |
| Service  |    |           |    |  NOTIFICATION    |    | Template  |
+----------+    |           |    |  SERVICE         |    | Engine    |
                |  API      |    |                  |    |           |
+----------+    |  GATEWAY  |--->| 1. Validate      |--->| Render    |
| Payment  |--->|           |    | 2. Check prefs   |    | message   |
| Service  |    |           |    | 3. Apply priority |    | from      |
+----------+    |           |    | 4. Route channel  |    | template  |
                |           |    | 5. Rate limit     |    +-----------+
+----------+    |           |    |                  |
| Auth     |--->|           |    +--------+---------+
| Service  |    +-----------+             |
+----------+                     +---------+---------+
                                 |                   |
                         +------v------+     +------v------+
                         | Priority    |     | User Prefs  |
                         | Queue       |     | Store       |
                         | (Kafka)     |     | (Database)  |
                         +------+------+     +-------------+
                                |
                   +-------------+-------------+
                   |             |             |
             +-----v----+ +-----v----+ +-----v----+
             | Push     | | SMS      | | Email    |
             | Worker   | | Worker   | | Worker   |
             +-----+----+ +-----+----+ +-----+----+
                   |             |             |
             +-----v----+ +-----v----+ +-----v----+
             | APNs/FCM | | Twilio/  | | SES/     |
```

```
              |          | | SNS      | | SendGrid |
              +----------+ +----------+ +----------+
                    |             |             |
              +-----v-----------v-------------v----+
              |        Delivery Tracker             |
              | (Track sent, delivered, opened,     |
              |  failed, bounced)                   |
              +-------------------------------------+
```

Request Flow

```
Notification Request:

{
  "user_id": "user_123",
  "type": "order_shipped",
  "data": {
    "order_id": "ORD-789",
    "tracking_number": "FX123456",
    "carrier": "FedEx",
    "eta": "2024-01-17"
  },
  "channels": ["push", "email"],
  "priority": "high"
}

Processing Steps:

1. VALIDATE
   - Is user_id valid?
   - Is notification type recognized?
   - Is required data present?

2. CHECK USER PREFERENCES
   - Has user opted out of this notification type?
   - Has user disabled this channel?
   - What is user's preferred language?
   - What is user's timezone?

3. APPLY PRIORITY AND THROTTLING
   - Is this high, medium, or low priority?
   - Has user received too many notifications today?
   - Is this a duplicate of a recent notification?

4. RENDER TEMPLATE
   - Load template for "order_shipped"
   - Fill in data: order ID, tracking number, carrier
   - Localize to user's language
   - Generate push text, email HTML, SMS text

5. ROUTE TO CHANNEL WORKERS
```

```
      - Enqueue push notification to push worker
      - Enqueue email to email worker
      - Each worker handles delivery independently

   6. TRACK DELIVERY
      - Record: sent at, delivered at, opened at
      - Handle failures: retry or fallback to another channel
```

Notification Templates

Templates separate the notification content from the notification logic. Instead of hardcoding messages, you define templates with placeholders that are filled in at send time.

```
   Template System:

   Template: "order_shipped"
   +-------------------------------------------------------+
   | Push Title: "Your order is on the way!"                |
   | Push Body:  "Order {{order_id}} shipped via {{carrier}}. |
   |              Expected delivery: {{eta}}"               |
   |                                                        |
   | Email Subject: "Your order {{order_id}} has shipped"   |
   | Email Body:   (HTML template with tracking link)       |
   |                                                        |
   | SMS: "Your order {{order_id}} shipped. Track: {{url}}"  |
   +-------------------------------------------------------+

   Data: {order_id: "ORD-789", carrier: "FedEx", eta: "Jan 17"}

   Rendered Push:
   Title: "Your order is on the way!"
   Body:  "Order ORD-789 shipped via FedEx. Expected delivery: Jan 17"

   Benefits:
   - Non-engineers can update notification text
   - Supports multiple languages (one template per language)
   - A/B test different messages without code changes
   - Consistent formatting across all notifications
```

Template Storage

```
   Template Storage:
```

```
+----+------------------+----------+------+-----------------+
| ID | Type             | Channel  | Lang | Content         |
+----+------------------+----------+------+-----------------+
| 1  | order_shipped    | push     | en   | "Your order..." |
| 2  | order_shipped    | push     | es   | "Tu pedido..."  |
| 3  | order_shipped    | email    | en   | "<html>..."     |
| 4  | order_shipped    | sms      | en   | "Order {{id}}.."|
| 5  | payment_received | push     | en   | "Payment of..." |
| 6  | payment_received | email    | en   | "<html>..."     |
+----+------------------+----------+------+-----------------+

Lookup: type + channel + language = template content
```

User Preferences

Users must be able to control what notifications they receive and how they receive them. This is not just good UX -- it is legally required in many jurisdictions.

```
User Preferences Model:

User: alice_123
+-------------------------+------+-----+-------+--------+
| Notification Type       | Push | SMS | Email | In-App |
+-------------------------+------+-----+-------+--------+
| Order updates           |  ON  | OFF |  ON   |   ON   |
| Promotional offers      | OFF  | OFF |  ON   |  OFF   |
| Security alerts         |  ON  |  ON |  ON   |   ON   |
| Social (likes, follows) |  ON  | OFF | OFF   |   ON   |
| Price drop alerts       |  ON  | OFF |  ON   |   ON   |
+-------------------------+------+-----+-------+--------+

Quiet Hours: 10:00 PM - 8:00 AM (user's timezone)
Language: English
Timezone: America/New_York

Processing Logic:
1. Receive notification request for alice_123
2. Look up preferences for this notification type
3. Filter out disabled channels
4. Check quiet hours (delay if in quiet period)
5. Send only to enabled channels
```

Global Opt-Out

Every notification must include an unsubscribe mechanism. For email, this is legally required (CAN-SPAM, GDPR). For push notifications, the OS provides a system-level toggle.

```
Unsubscribe Flow:

Email footer: "Unsubscribe from order updates"
     |
     v
+--------+  Click   +-------------+  Update    +----------+
| User   | -------> | Unsubscribe | ---------> | Prefs    |
| Email  |          | Service     |            | Database |
+--------+          +-------------+            +----------+
                          |
                    Generate one-time
                    token (no login needed)

Important:
- One-click unsubscribe (no login required)
- Confirm the unsubscribe on a landing page
- Take effect immediately
- Include "manage all preferences" link
```

Priority and Throttling

Not all notifications are equal. A security alert ("Someone logged into your account from a new device") is far more important than a promotional email ("20% off sneakers this weekend"). Priority and throttling systems ensure important notifications get through while preventing notification fatigue.

Priority Levels

```
Priority Levels:

+----------+-------------------+---------------------------+
| Priority | Examples          | Behavior                  |
+----------+-------------------+---------------------------+
| CRITICAL | Security alerts,  | Bypass all throttling,    |
|          | 2FA codes,        | send immediately, wake    |
|          | fraud detection   | device, override quiet hrs|
+----------+-------------------+---------------------------+
| HIGH     | Order shipped,    | Send immediately,         |
|          | payment received, | respect quiet hours,      |
|          | ride arriving     | bypass daily limits       |
+----------+-------------------+---------------------------+
```

```
| MEDIUM   | Social updates,   | Send within minutes,      |
|          | price drops,      | respect all limits,       |
|          | recommendations   | can be batched            |
+----------+-------------------+---------------------------+
| LOW      | Newsletters,      | Batch and send at optimal |
|          | promotions,       | time, first to be dropped |
|          | weekly digests    | if user near daily limit  |
+----------+-------------------+---------------------------+
```

Throttling

```
Throttling Rules:

Per-User Limits:
- Max 5 push notifications per hour
- Max 20 push notifications per day
- Max 1 SMS per hour (except CRITICAL)
- Max 3 emails per day (except transactional)

Throttling Decision Tree:
+-------------------+
| New notification  |
+--------+----------+
         |
         v
+--------+----------+
| Priority =        |
| CRITICAL?         |
+--------+----------+
    YES  |     | NO
         v     v
Send     +-----+----------+
now      | User at daily  |
         | limit?         |
         +-----+----------+
          YES  |     | NO
               v     v
         +-----+  +--+----------+
         | DROP |  | In quiet   |
         | (log)|  | hours?     |
         +------+  +--+---------+
                 YES  |     | NO
                      v     v
                +-----+  +--+------+
                |DELAY |  | SEND   |
                |until |  | NOW    |
                |8 AM  |  +--------+
                +------+
```

Batching

Low-priority notifications can be batched together instead of sending one at a time.

```
Batching Example:

Without batching (annoying):
10:01 AM - "Alice liked your photo"
10:03 AM - "Bob liked your photo"
10:07 AM - "Charlie liked your photo"
10:12 AM - "Diana liked your photo"
10:15 AM - "Eve liked your photo"

With batching (pleasant):
10:15 AM - "Alice, Bob, and 3 others liked your photo"

Implementation:
- Collect notifications of the same type for a time window (5-15 min)
- If only 1 notification, send individually
- If 2-3, list names: "Alice and Bob liked your photo"
- If 4+, summarize: "Alice, Bob, and 3 others liked your photo"
```

Delivery Tracking

After sending a notification, you need to know: Was it delivered? Was it opened? Did the user take action? This data drives optimization and debugging.

```
Notification Lifecycle:

+----------+    +--------+    +-----------+    +--------+    +--------+
| CREATED  |--->| QUEUED |--->| SENT      |--->|DELIVERED|-->| OPENED |
+----------+    +--------+    +-----------+    +--------+    +--------+
     |               |             |                              |
     v               v             v                              v
+---------+    +---------+   +-----------+                  +---------+
| FILTERED|    | DROPPED |   |  FAILED   |                  | CLICKED |
| (prefs) |    | (thrtl) |   | (bounce/  |                  | (CTA)   |
+---------+    +---------+   |  timeout) |                  +---------+
                             +-----------+
                                   |
                             +-----v-----+
                             |  RETRYING  |
                             +-----------+


Delivery Tracking Table:
```

```
+----------+----------+---------+--------+-----------+---------+
| notif_id | user_id  | channel | status | timestamp | details |
+----------+----------+---------+--------+-----------+---------+
| n_001    | user_123 | push    | opened | 10:05:03  |         |
| n_001    | user_123 | email   | sent   | 10:05:01  | SES ID  |
| n_002    | user_456 | push    | failed | 10:06:00  | expired |
|          |          |         |        |           | token   |
| n_003    | user_789 | sms     | delivd | 10:07:02  | Twilio  |
+----------+----------+---------+--------+-----------+---------+
```

Delivery Metrics

```
Key Metrics to Track:

+-----------------------------------+
| Metric            | Target        |
+-----------------------------------+
| Delivery rate     | > 95%         |
| Push open rate    | > 5%          |
| Email open rate   | > 20%         |
| Click-through rate| > 2%          |
| Unsubscribe rate  | < 0.5%       |
| Bounce rate       | < 2%          |
| Spam complaint    | < 0.1%       |
| Avg delivery time | < 5 seconds   |
+-----------------------------------+
```

Retry Logic

Notification delivery can fail for many reasons: the push service is temporarily down, the email bounces, or the SMS gateway is overloaded. A good retry strategy handles transient failures without overwhelming downstream services.

```
Retry Strategy:

Attempt 1: Send immediately
     |
     FAIL (timeout)
     |
Wait 1 second
     |
Attempt 2: Retry
     |
```

```
     FAIL (server error)
     |
Wait 4 seconds (exponential backoff)
     |
Attempt 3: Retry
     |
     FAIL (server error)
     |
Wait 16 seconds
     |
Attempt 4: Retry
     |
     FAIL
     |
Wait 64 seconds
     |
Attempt 5: Final retry
     |
     FAIL
     |
Move to Dead Letter Queue
Alert operations team
Consider fallback channel


Exponential Backoff with Jitter:

Delay = min(base * 2^attempt + random_jitter, max_delay)

Attempt 1: 1s  + jitter (0-500ms)
Attempt 2: 4s  + jitter (0-500ms)
Attempt 3: 16s + jitter (0-500ms)
Attempt 4: 64s + jitter (0-500ms)
Attempt 5: 120s (max)

Jitter prevents thundering herd:
Without jitter: 1000 failed notifications all retry at exactly 4s
With jitter:    1000 retries spread across 4.0s - 4.5s
```

Channel Fallback

If the primary channel fails, fall back to an alternative channel.

```
Channel Fallback:

Notification: "Your ride is arriving"
Primary channel: Push notification

+--------+     Push      +------+
| Server | ------------> | APNs |  FAIL (token expired)
+--------+               +------+
```

```
      |
      | Fallback to SMS
      v
+--------+     SMS       +--------+
| Server | ------------> | Twilio |  SUCCESS
+--------+               +--------+

Fallback rules:
- CRITICAL: Push -> SMS -> Email (try all)
- HIGH: Push -> Email
- MEDIUM: Push only (no fallback)
- LOW: Email only (no fallback)
```

APNs, FCM, and SMTP

These are the three core delivery protocols for push notifications and email.

Apple Push Notification Service (APNs)

```
APNs Architecture:

+--------+    HTTP/2     +------+    Persistent    +--------+
| Your   | -----------> | APNs | -- Connection --> | iPhone |
| Server |              |      |                   +--------+
+--------+              +------+
                            |
                       +----+----+
                       | Features|
                       +---------+
                       | - HTTP/2 multiplexing
                       | - JWT or certificate auth
                       | - Immediate feedback on errors
                       | - Device token per app
                       | - Priority: 10 (immediate)
                       |           5 (power-saving)
                       | - TTL: how long to keep trying
                       +---------+

Device Token:
When a user installs your app and allows notifications,
APNs gives the app a unique device token.
Your server stores this token and uses it to send pushes.

Token lifecycle:
- Generated on app install
- Can change (user reinstalls, new device)
```

```
  - Your server must handle token updates
  - Invalid tokens mean the app was uninstalled
```

Firebase Cloud Messaging (FCM)

```
  FCM Architecture:

  +--------+    HTTPS      +------+                  +---------+
  | Your   | -----------> | FCM  | ---- Push -----> | Android |
  | Server |              |      | ---- Push -----> | Web     |
  +--------+              +------+ ---- Push -----> | iOS*    |
                              |
                        +----+----+
                        | Features|
                        +---------+
                        | - Supports Android, iOS, Web
                        | - Topics (subscribe groups)
                        | - Conditions (topic combos)
                        | - Data messages (app handles)
                        | - Notification messages (FCM shows)
                        | - Message batching
                        +---------+

  * FCM can also deliver to iOS through APNs.

  Topics:
  Instead of sending to individual tokens,
  subscribe users to topics and send to the topic.

  Example:
  Topic: "breaking_news" -> 1 million subscribers
  One API call to FCM -> delivered to all subscribers
```

SMTP (Simple Mail Transfer Protocol)

```
  Email Delivery Path:

  +--------+   SMTP/API   +-----------+   SMTP    +----------+   SMTP    +--------+
  | Your   | -----------> | Email     | --------> | Receiving| --------> | User's |
  | Server |              | Service   |           | Mail     |           | Inbox  |
  +--------+              | (SES,     |           | Server   |           +--------+
                          | SendGrid) |           | (Gmail)  |
                          +-----------+           +----------+

  Email Deliverability Factors:
  +-----------------------------------------------------+
  | Factor           | Why It Matters                  |
```

```
+-----------------------------------------------------+
| SPF Record        | Proves your server is           |
|                   | authorized to send email        |
+-----------------------------------------------------+
| DKIM Signature    | Proves email was not modified   |
|                   | in transit                      |
+-----------------------------------------------------+
| DMARC Policy      | Tells receivers what to do      |
|                   | with unauthenticated email      |
+-----------------------------------------------------+
| Sender reputation| Based on bounce rate, spam      |
|                   | complaints, engagement          |
+-----------------------------------------------------+
| List hygiene      | Remove invalid addresses,       |
|                   | honor unsubscribes promptly     |
+-----------------------------------------------------+
```

Fan-Out: Sending to Millions

When you need to send a notification to millions of users (a product announcement, a breaking news alert), you need a fan-out strategy.

The Challenge

```
The Fan-Out Problem:

"Send a notification to all 10 million users"

Naive approach:
FOR each user in 10,000,000 users:
  send_notification(user)

Time: 10,000,000 x 50ms = 500,000 seconds = 5.8 DAYS

That is obviously not going to work.
```

Fan-Out Architecture

```
Fan-Out Architecture:

Step 1: Create notification
+----------+    "New feature launched!"    +-------------+
```

```
| Admin    | ---------------------------> | Notification|
| Console  |                               | Service     |
+----------+                               +------+------+
                                                  |
Step 2: Fan-out to user segments                  |
                                                  v
                                           +------+------+
                                           | Fan-Out     |
                                           | Service     |
                                           +------+------+
                                                  |
                  +------------------------------+-----------------------------+
                  |                              |                             |
                  v                              v                             v
          +-------+-------+             +--------+------+             +---------+-----+
          | Segment:      |             | Segment:      |             | Segment:      |
          | Users A-G     |             | Users H-P     |             | Users Q-Z     |
          | (3.3M users)  |             | (3.3M users)  |             | (3.3M users)  |
          +-------+-------+             +--------+------+             +---------+-----+
                  |                             |                            |
           Kafka partition 0            Kafka partition 1            Kafka partition 2
                  |                             |                            |
          +-------v-------+             +--------v------+             +---------v-----+
          | Worker Pool   |             | Worker Pool   |             | Worker Pool   |
          | (20 workers)  |             | (20 workers)  |             | (20 workers)  |
          | 50,000/sec    |             | 50,000/sec    |             | 50,000/sec    |
          +---------------+             +---------------+             +---------------+

Total throughput: 150,000 notifications/second
Time to send to 10M users: ~67 seconds

Key techniques:
1. Partition users into segments
2. Use Kafka to buffer and distribute work
3. Multiple worker pools process in parallel
4. Batch API calls to push services (FCM supports 500/batch)
5. Use topics for broadcast (FCM topics, APNs groups)
```

Uber Ride Notification Flow

Let us trace the complete notification flow for an Uber ride, from request to completion.

```
Uber Ride Notification Flow:

Event 1: Ride Requested
+---------+   Request    +----------+   Push    +----------+
| Rider   | -----------> | Ride     | -------> | Driver   |
| App     |              | Service  |          | App      |
```

```
+---------+              +----------+          +----------+
                               |
                        "Looking for a driver..."
                               |
                               v
                        Push to Rider: "Finding your ride..."

Event 2: Driver Accepted
+---------+   Accept     +----------+
| Driver  | -----------> | Ride     |
| App     |              | Service  |
+---------+              +----+-----+
                               |
                  +-----------+-----------+
                  |                       |
                  v                       v
            Push to Rider:          In-App Update:
            "John is on the way    Show driver photo,
             in a Toyota Camry    car details, ETA
             (ABC 1234)"          on map

Event 3: Driver Arriving
+---------+   Location   +----------+   Geofence    +----------+
| Driver  |   update     | Location | -- trigger --> | Notif.   |
| App     | -----------> | Service  |               | Service  |
+---------+              +----------+               +----+-----+
                                                         |
                  +-------------------------------------+
                  |                       |
                  v                       v
            Push to Rider:          SMS Fallback:
            "Your driver is         "Your Uber is
             arriving now!"         arriving. Look
                                    for Toyota Camry
                                    ABC 1234"

Event 4: Ride Started
+---------+   Start ride  +----------+
| Driver  | -----------> | Ride     |
| App     |              | Service  |
+---------+              +----+-----+
                               |
                               v
                        In-App to Rider:
                        Show route, ETA,
                        live tracking

Event 5: Ride Completed
+---------+   End ride    +----------+
| Driver  | -----------> | Ride     |
| App     |              | Service  |
+---------+              +----+-----+
                               |
                  +-----------+-----------+----------+
                  |                       |          |
```

```
                    v                        v          v
             Push to Rider:          Email:      In-App:
             "Trip complete!        Receipt     Rate your
              $24.50"              with map,    driver
                                   fare         (1-5 stars)
                                   breakdown

Event 6: Rating Received
+---------+   5 stars    +----------+
| Rider   | -----------> | Rating   |
| App     |              | Service  |
+---------+              +----+-----+
                              |
                              v
                        Push to Driver:
                        "You received a
                         5-star rating!"


Channel Selection Logic:

+-------------------+--------+------+-------+--------+
| Event             | Push   | SMS  | Email | In-App |
+-------------------+--------+------+-------+--------+
| Finding driver    | YES    | no   | no    | YES    |
| Driver accepted   | YES    | no   | no    | YES    |
| Driver arriving   | YES    | YES* | no    | YES    |
| Ride started      | no     | no   | no    | YES    |
| Ride completed    | YES    | no   | YES   | YES    |
| Rating received   | YES    | no   | no    | YES    |
+-------------------+--------+------+-------+--------+

* SMS only if push delivery fails (fallback)
```

Trade-offs

Decision	Option A	Option B
Delivery	Best-effort (fast, may lose)	Guaranteed (reliable, slower, complex)
Channel	Single channel (simple)	Multi-channel with fallback (reliable, costly)
Timing	Send immediately (real-time)	Batch and send at optimal time (fewer interruptions)

Template	Hardcoded messages (simple, inflexible)	Template engine (flexible, more moving parts)
Fan-out	Sequential (simple, slow)	Parallel with Kafka (fast, complex infrastructure)
Preference	Opt-out only (more reach)	Opt-in only (less reach, higher engagement)
Priority queue	Single queue (simple)	Multiple priority queues (complex, better UX)

Common Mistakes

1. Not respecting user preferences. Sending notifications to users who opted out is not just bad UX -- it violates regulations like GDPR and CAN-SPAM. Always check preferences before sending.
2. Sending too many notifications. Notification fatigue is the number one reason users disable push notifications or uninstall apps. Implement throttling and batching.
3. Not handling token expiration. Push notification device tokens expire when users uninstall the app or get a new device. If you keep sending to expired tokens, your sender reputation drops and delivery rates suffer.
4. No retry with backoff. Transient failures are normal. Retrying immediately (or not at all) either overwhelms the downstream service or loses the notification. Use exponential backoff with jitter.
5. Ignoring email deliverability. Without SPF, DKIM, and DMARC records, your emails land in spam. Monitor bounce rates and spam complaints.
6. No delivery tracking. If you do not track delivery status, you have no idea if notifications are actually reaching users. You cannot improve what you do not measure.

Best Practices

1. Implement user preference management from day one. Let users control notification types, channels, and quiet hours. This reduces unsubscribes and improves engagement.
2. Use templates for all notification content. Separate content from code. This enables localization, A/B testing, and non-engineer content updates.

3. Prioritize notifications. Not all notifications are equal. Security alerts should bypass throttling. Promotions should be batched. Use at least three priority levels.
4. Track delivery end-to-end. Record created, queued, sent, delivered, opened, and clicked timestamps. Monitor delivery rates and alert on drops.
5. Implement exponential backoff with jitter for retries. Start with a 1-second delay, double each time, add random jitter, and cap at a maximum delay. After a set number of retries, move to a dead letter queue.
6. Use channel fallback for critical notifications. If push fails, try SMS. If SMS fails, try email. Ensure the user receives critical information through at least one channel.
7. Clean up invalid tokens regularly. When APNs or FCM reports an invalid token, remove it immediately. Sending to invalid tokens damages your sender reputation.
8. Batch low-priority notifications. Combine multiple social notifications ("Alice, Bob, and 3 others liked your photo") instead of sending five separate notifications.

Quick Summary

A notification system delivers messages to users through four channels: push notifications (via APNs for iOS and FCM for Android), SMS (via providers like Twilio), email (via SMTP services like SES or SendGrid), and in-app notifications (via WebSockets or polling). The architecture consists of a notification service that validates requests, checks user preferences, applies priority and throttling rules, renders templates, and routes messages to channel-specific workers. Templates separate content from logic, enabling localization and A/B testing. User preferences give users control over what they receive and through which channels. Priority levels and throttling prevent notification fatigue. Delivery tracking records the lifecycle of each notification from creation through delivery and engagement. Retry logic with exponential backoff handles transient failures. Fan-out architectures using Kafka and parallel workers enable sending to millions of users in seconds.

Key Points

- Four notification channels (push, SMS, email, in-app) each have different costs, latencies, and capabilities
- User preferences and opt-out management are both a UX requirement and a legal obligation

- Priority levels ensure critical notifications like security alerts always get through
- Throttling and batching prevent notification fatigue and reduce unsubscribes
- Templates separate content from code, enabling localization and A/B testing
- Retry with exponential backoff and jitter handles transient delivery failures gracefully
- Channel fallback ensures critical notifications reach users even when one channel fails
- Delivery tracking (sent, delivered, opened, clicked) is essential for measuring effectiveness

---

Practice Questions

1. Your notification system sends 10 million push notifications per day. Your delivery rate has dropped from 95% to 70% over the past month. What are the most likely causes? How would you investigate and fix the problem?
2. You need to send a price drop alert to 5 million users within 30 minutes. Each push notification takes 50 milliseconds to send. How would you design the fan-out system? What infrastructure do you need?
3. A user receives a push notification for a security alert but does not see it (their phone was off). How would you ensure they receive the alert through another channel? What is the fallback timeline?
4. Your email open rate dropped from 25% to 8% after a new marketing campaign. Your spam complaint rate is at 0.3%. What happened? How would you recover your sender reputation?
5. Design the notification preference system for a social media app. What notification types would you define? What default settings would you use? How would you handle users who have not set preferences?

Exercises

Exercise 1: Design a Multi-Channel Notification Pipeline

Design the complete notification architecture for a banking app. Requirements: transaction alerts (immediate), monthly statements (batch email), security alerts (multi-channel with fallback), marketing offers (user-opted-in only). Include the priority system, throttling rules, template structure, and delivery tracking. Draw the architecture diagram.

Exercise 2: Build a Fan-Out System

Your news app has 20 million users. Breaking news must reach all users within 2 minutes. Design the fan-out architecture. Calculate the required throughput, number of workers, and Kafka partition count. Consider that FCM supports batches of 500 messages per API call. Handle the case where 5% of device tokens are invalid.

Exercise 3: Notification Analytics Dashboard

Design the data model and queries for a notification analytics dashboard. The dashboard must show: notifications sent per channel per hour, delivery rates, open rates, click-through rates, unsubscribe rates, and average delivery latency. Define the database schema, the data collection pipeline, and how you would compute real-time vs. daily aggregate metrics.

What Is Next?

You now know how to design notification systems that reach users through the right channel at the right time with the right message. But how do you know your notification system (and the rest of your infrastructure) is actually working? How do you detect problems before users complain? In the next chapter, you will learn about logging, monitoring, and observability: how to instrument your systems with metrics, logs, and traces so you can see what is happening inside your distributed system in real time.

Chapter 20: Logging, Monitoring, and Observability

What You Will Learn

- The difference between metrics, logs, and traces -- the three pillars of observability
- How Prometheus collects and stores metrics using a pull-based model
- How the ELK stack (Elasticsearch, Logstash, Kibana) handles log aggregation at scale
- How distributed tracing with tools like Jaeger tracks requests across services
- How alerting works and how to avoid alert fatigue
- How Grafana dashboards visualize system health at a glance
- What SLIs, SLOs, and SLAs are and how they define reliability targets
- How health checks detect unhealthy services before they cause outages
- The four golden signals: latency, traffic, errors, and saturation

Why This Chapter Matters

Imagine driving a car with no dashboard. No speedometer, no fuel gauge, no temperature warning, no check engine light. You would have no idea how fast you are going, whether you are about to run out of gas, or whether the engine is overheating. You would only find out something is wrong when the car breaks down on the highway.

Running a distributed system without monitoring is exactly like that. You have dozens of services, hundreds of servers, thousands of database connections, and millions of requests per minute. Without observability, you are blind. When something breaks (and it will), you have no idea what went wrong, where it went wrong, or how to fix it. Worse, you do not know about problems until your users start complaining.

Observability is not optional. It is the difference between a team that detects and fixes issues in minutes and a team that spends hours guessing in the dark. Every system design you build

needs monitoring built in from the start. In interviews, mentioning observability shows that you think about production systems holistically.

Metrics vs Logs vs Traces

These are the three pillars of observability. Each one answers a different question about your system.

Metrics

What happened at a high level?

Metrics are numerical measurements collected at regular intervals. They tell you the overall health of your system: how many requests per second, what is the error rate, how much CPU is being used.

Think of metrics as the gauges on your car dashboard. They give you a quick snapshot of how things are going.

```
Metrics Examples:

+-----------------------------------+----------+
| Metric                            | Value    |
+-----------------------------------+----------+
| http_requests_total               | 1,523,847|
| http_request_duration_seconds_avg | 0.045    |
| http_errors_total                 | 3,241    |
| cpu_usage_percent                 | 67.3     |
| memory_usage_bytes                | 4.2 GB   |
| active_database_connections       | 85       |
| queue_depth                       | 12,450   |
+-----------------------------------+----------+

Characteristics:
- Numeric values (counters, gauges, histograms)
- Collected at regular intervals (every 15-60 seconds)
- Compact (just numbers, very space-efficient)
- Aggregatable (can average, sum, percentile)
- Best for: dashboards, alerting, trend analysis
```

Logs

What exactly happened in detail?

Logs are timestamped text records of individual events. They contain the details you need when debugging a specific problem: what happened, when, where, and in what context.

Think of logs as a ship's logbook. Every event is recorded with a timestamp and details.

```
Log Examples:

2024-01-15 10:23:45.123 INFO  [order-service] [req-abc123]
  Order ORD-789 created for user user_456, total: $129.99

2024-01-15 10:23:45.234 INFO  [payment-service] [req-abc123]
  Processing payment for order ORD-789, method: visa_4242

2024-01-15 10:23:46.891 ERROR [payment-service] [req-abc123]
  Payment failed for order ORD-789: gateway timeout after 5000ms
  Stack trace:
    at PaymentGateway.charge(PaymentGateway.java:145)
    at PaymentService.processPayment(PaymentService.java:89)

2024-01-15 10:23:46.892 WARN  [payment-service] [req-abc123]
  Retrying payment for order ORD-789 (attempt 2 of 3)

Characteristics:
- Text-based (structured JSON or unstructured text)
- One entry per event
- Verbose (large volume, expensive to store)
- Not easily aggregatable
- Best for: debugging, audit trails, forensic analysis
```

Traces

What was the journey of a single request across services?

A trace follows a single request as it travels through multiple services. It shows the complete path, timing, and dependencies.

Think of it as a package tracking system. You can see every stop the package made, how long it stayed at each stop, and where it got delayed.

```
Trace Example: "Place Order" Request

Trace ID: trace-xyz-789
Total Duration: 1,247 ms

+-- API Gateway (12ms) ----------------------------------------+
|                                                               |
|  +-- Order Service (45ms) ---------------------------------+   |
|  |                                                         |   |
|  |  +-- User Service: Validate User (23ms) ----+           |   |
```

```
|  |  |                                          |           |   |
|  |  +------------------------------------------+           |   |
|  |                                                          |   |
|  |  +-- Inventory Service: Check Stock (89ms) -+           |   |
|  |  |                                          |           |   |
|  |  +------------------------------------------+           |   |
|  |                                                          |   |
|  |  +-- Payment Service: Charge Card (980ms) --+           |   |
|  |  |                                          |           |   |
|  |  |  +-- Payment Gateway (external) (945ms) -+           |   |
|  |  |  |   *** SLOW! ***                      |           |   |
|  |  |  +--------------------------------------+           |   |
|  |  |                                          |           |   |
|  |  +------------------------------------------+           |   |
|  |                                                          |   |
|  |  +-- Email Service: Send Confirmation (98ms)+           |   |
|  |  |                                          |           |   |
|  |  +------------------------------------------+           |   |
|  |                                                          |   |
|  +----------------------------------------------------------+   |
|                                                                 |
+-----------------------------------------------------------------+

Insight: The payment gateway took 945ms out of 1,247ms total.
That is 76% of the request time. The bottleneck is the
external payment gateway, not our code.

Characteristics:
- Request-scoped (follows one request end to end)
- Shows causality (which service called which)
- Shows timing (where time was spent)
- Essential for microservices debugging
- Best for: latency analysis, dependency mapping, bottleneck identification
```

How They Work Together

```
The Three Pillars Together:

METRICS tell you:  "Error rate jumped to 5% at 10:23 AM"
                    (Something is wrong!)

LOGS tell you:     "Payment gateway returning timeout errors
                    for orders with amount > $100"
                    (This is what is breaking!)

TRACES tell you:   "Request trace-xyz-789 shows the payment
                    gateway call took 5,000ms (timeout) while
                    the rest of the request took only 45ms"
                    (This is exactly where and why!)

Investigation Flow:
```

```
1. Alert fires (from metrics)       --> "What is wrong?"
2. Look at dashboard (metrics)      --> "Error rate is 5%"
3. Search logs for errors           --> "Payment timeouts"
4. Find a failing trace             --> "Gateway takes 5s"
5. Fix the issue                    --> "Increase timeout, add retry"
6. Verify fix in metrics            --> "Error rate back to 0.1%"
```

Prometheus: Metrics Collection

Prometheus is the most popular open-source metrics system. It uses a pull-based model: Prometheus scrapes metrics from your services at regular intervals.

How Prometheus Works

```
Prometheus Architecture:

+----------+     +----------+     +----------+
| Service A|     | Service B|     | Service C|
| /metrics |     | /metrics |     | /metrics |
+----+-----+     +----+-----+     +----+-----+
     ^               ^               ^
     |    Scrape     |    Scrape     |    Scrape
     |    (every 15s) |    (every 15s) |    (every 15s)
     |               |               |
+----+----------------+----------------+----+
|             PROMETHEUS                     |
|                                            |
|  +-----------+  +-----------+  +--------+  |
|  | Scraper   |  | Time      |  | Query  |  |
|  | (pulls    |  | Series    |  | Engine |  |
|  |  metrics) |  | Database |  | (PromQL)|  |
|  +-----------+  +-----------+  +--------+  |
|                                            |
+--------------------+-----------------------+
                     |
            +--------v--------+
            | Alertmanager    |
            | (sends alerts   |
            |  to Slack,      |
            |  PagerDuty)     |
            +-----------------+
                     |
            +--------v--------+
            | Grafana         |
            | (dashboards,    |
            |  visualization) |
```

```
              +-----------------+

Pull-Based Model:
- Each service exposes a /metrics endpoint
- Prometheus scrapes all endpoints every 15 seconds
- Metrics stored as time series (value over time)
- PromQL queries the data for dashboards and alerts
```

Metric Types

```
Prometheus Metric Types:

1. COUNTER (only goes up)
   Total number of HTTP requests: 1,523,847
   Total number of errors: 3,241
   Use: Request counts, error counts, bytes processed

2. GAUGE (goes up and down)
   Current CPU usage: 67.3%
   Active connections: 85
   Queue depth: 12,450
   Use: Temperature, memory, active sessions

3. HISTOGRAM (distribution of values)
   Request duration buckets:
     0-50ms:   45,000 requests  (60%)
     50-100ms: 22,500 requests  (30%)
     100-500ms: 6,000 requests  (8%)
     500ms+:    1,500 requests  (2%)
   Use: Latency percentiles, request sizes

4. SUMMARY (similar to histogram, calculates percentiles)
   P50 latency: 42ms
   P95 latency: 187ms
   P99 latency: 892ms
   Use: When you need precise percentiles client-side
```

PromQL: Querying Metrics

```
PromQL Examples:

# Total requests per second over the last 5 minutes
rate(http_requests_total[5m])

# Error rate (percentage of requests that are errors)
rate(http_requests_total{status=~"5.."}[5m])
/ rate(http_requests_total[5m]) * 100
```

```
# P99 latency
histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m]))

# CPU usage above 80% for any instance
cpu_usage_percent > 80

# Memory usage by service
memory_usage_bytes{service="order-service"} / 1024 / 1024 / 1024
```

The ELK Stack: Log Aggregation

ELK stands for Elasticsearch, Logstash, and Kibana. It is the most widely used open-source log aggregation stack.

The Problem with Logs at Scale

```
The Log Problem:

50 microservices x 10 instances each = 500 servers

Each server generates ~100 MB of logs per hour
Total: 50 GB of logs per hour = 1.2 TB per day

Without centralized logging:
- SSH into each of 500 servers to find a log entry? No.
- Correlate events across services? Impossible.
- Search for an error from last Tuesday? Good luck.

You need centralized log aggregation.
```

ELK Architecture

```
ELK Stack Architecture:

+----------+  +----------+  +----------+
| Service  |  | Service  |  | Service  |
| Instance |  | Instance |  | Instance |
+----+-----+  +----+-----+  +----+-----+
     |              |              |
     | Filebeat     | Filebeat     | Filebeat
     | (shipper)    | (shipper)    | (shipper)
```

```
        |              |              |
        +-------+------+------+------+
                |             |
                v             v
        +-------+-------------+-------+
        |         LOGSTASH            |
        |                             |
        |  1. PARSE (extract fields   |
        |     from unstructured text) |
        |                             |
        |  2. TRANSFORM (add metadata, |
        |     rename fields, filter)  |
        |                             |
        |  3. ENRICH (add geo-IP,     |
        |     add service name)       |
        +-------------+---------------+
                      |
                      v
        +-------------+---------------+
        |       ELASTICSEARCH         |
        |                             |
        |  Index: logs-2024-01-15     |
        |  Searchable within seconds  |
        |  Retention: 30 days         |
        +-------------+---------------+
                      |
                      v
        +-------------+---------------+
        |          KIBANA             |
        |                             |
        |  - Search logs in real time |
        |  - Build visualizations     |
        |  - Create dashboards        |
        |  - Set up alerts            |
        +-----------------------------+

    Modern alternative: Many teams now use
    Filebeat -> Elasticsearch directly (no Logstash)
    or Fluentd instead of Logstash (EFK stack).
```

Structured Logging

```
    Unstructured Log (Hard to Parse):

    2024-01-15 10:23:45 ERROR Payment failed for order ORD-789
    user user_456 amount $129.99 gateway timeout

    Which part is the order ID? Which is the user?
    Logstash must guess using regex patterns. Fragile.
```

```
Structured Log (JSON - Easy to Parse):

{
  "timestamp": "2024-01-15T10:23:45.123Z",
  "level": "ERROR",
  "service": "payment-service",
  "instance": "payment-3",
  "trace_id": "trace-xyz-789",
  "message": "Payment failed",
  "order_id": "ORD-789",
  "user_id": "user_456",
  "amount": 129.99,
  "currency": "USD",
  "error": "gateway_timeout",
  "duration_ms": 5000
}

Every field is explicitly named.
Elasticsearch can index and search every field.
No parsing guesswork needed.
```

Log Retention Strategy

```
Log Retention:

+--------------------+-------------+-------------------+
| Age                | Storage     | Searchable?       |
+--------------------+-------------+-------------------+
| Last 7 days        | Hot nodes   | Yes (fast)        |
|                    | (SSD)       |                   |
+--------------------+-------------+-------------------+
| 7-30 days          | Warm nodes  | Yes (slower)      |
|                    | (HDD)       |                   |
+--------------------+-------------+-------------------+
| 30-90 days         | Cold storage| After restore     |
|                    | (S3/Glacier)|                   |
+--------------------+-------------+-------------------+
| 90+ days           | Deleted     | No                |
+--------------------+-------------+-------------------+

Use Index Lifecycle Management (ILM) to automate:
- Roll over indices daily
- Move old indices to cheaper storage
- Delete indices after retention period
```

Distributed Tracing with Jaeger

In a microservices architecture, a single user request can travel through 10 or more services. Distributed tracing tracks the entire journey of a request, showing timing, dependencies, and bottlenecks.

How Distributed Tracing Works

```
Distributed Tracing Concepts:

TRACE: The entire journey of a request (has a unique Trace ID)
SPAN: One operation within the trace (one service call)

Trace ID: abc-123

+-- Span 1: API Gateway (12ms) -------------------------+
|   Span ID: span-1                                      |
|   Parent: none                                         |
|                                                         |
|   +-- Span 2: Order Service (245ms) ----------------+  |
|   |   Span ID: span-2                               |  |
|   |   Parent: span-1                                |  |
|   |                                                  |  |
|   |   +-- Span 3: DB Query (15ms) ------+           |  |
|   |   |   Span ID: span-3               |           |  |
|   |   |   Parent: span-2                |           |  |
|   |   +---------------------------------+           |  |
|   |                                                  |  |
|   |   +-- Span 4: Payment Service (200ms)+          |  |
|   |   |   Span ID: span-4               |           |  |
|   |   |   Parent: span-2                |           |  |
|   |   |                                  |           |  |
|   |   |   +-- Span 5: Stripe API (180ms)+           |  |
|   |   |   |   Parent: span-4            |           |  |
|   |   |   +-----------------------------+           |  |
|   |   |                                  |           |  |
|   |   +---------------------------------+           |  |
|   |                                                  |  |
|   +--------------------------------------------------+  |
|                                                         |
+---------------------------------------------------------+
```

How Trace Context Propagates

```
Context Propagation:

Every service passes the Trace ID and Span ID to the next service
via HTTP headers.

Request from Client:
```

```
GET /api/orders HTTP/1.1

API Gateway adds trace context:
GET /orders HTTP/1.1
traceparent: 00-abc123-span1-01

API Gateway calls Order Service:
GET /internal/orders HTTP/1.1
traceparent: 00-abc123-span2-01

Order Service calls Payment Service:
POST /internal/payments HTTP/1.1
traceparent: 00-abc123-span4-01

Every service:
1. Reads the traceparent header
2. Creates a new span (child of the parent span)
3. Records timing and metadata
4. Passes the trace context to downstream calls
5. Sends completed spans to the tracing backend (Jaeger)
```

Jaeger Architecture

```
Jaeger Architecture:

+----------+  +----------+  +----------+
| Service A|  | Service B|  | Service C|
| (traced) |  | (traced) |  | (traced) |
+----+-----+  +----+-----+  +----+-----+
     |              |              |
     | Spans        | Spans        | Spans
     |              |              |
+----v--------------v--------------v----+
|           JAEGER AGENT                 |
|     (sidecar or shared daemon)         |
|     Batches and forwards spans         |
+--------------------+-------------------+
                     |
                     v
+--------------------+-------------------+
|           JAEGER COLLECTOR             |
|     Validates and indexes spans        |
+--------------------+-------------------+
                     |
                     v
+--------------------+-------------------+
|           STORAGE (Backend)            |
|     Elasticsearch or Cassandra         |
+--------------------+-------------------+
                     |
                     v
```

```
+-------------------+-------------------+
|           JAEGER QUERY + UI           |
|     Search traces, visualize spans    |
+---------------------------------------+
```

Sampling

You cannot trace every request in a high-traffic system. At 100,000 requests per second, storing every trace would be prohibitively expensive. Sampling strategies select which requests to trace.

```
Sampling Strategies:

1. Probabilistic: Trace 1% of all requests
   Simple, even coverage, might miss rare errors

2. Rate-limiting: Trace 10 requests per second per service
   Predictable cost, might miss spikes

3. Adaptive: Increase sampling when errors occur
   Best coverage of problems, more complex

4. Head-based: Decide at the first service whether to trace
   Consistent (whole trace or nothing), can miss slow calls

5. Tail-based: Decide after the trace is complete
   Can trace only errors/slow requests, requires buffering
```

Alerting

Monitoring without alerting is just a screen nobody looks at. Alerts notify engineers when something needs attention.

Alert Design

```
Good Alert vs Bad Alert:

BAD ALERT:
"CPU usage is at 82% on server web-14"
- Is this actually a problem? Maybe 82% is normal.
- What should I do? No actionable information.
```

```
- Fires constantly? Engineers start ignoring it.

GOOD ALERT:
"Error rate exceeded 1% for the order-service
 (currently 3.5%) for the past 5 minutes.
 SLO burn rate: 10x normal.
 Dashboard: https://grafana.internal/order-service
 Runbook: https://wiki.internal/runbooks/order-errors"

- Clear problem statement
- Includes current value and threshold
- Links to dashboard and runbook
- Based on user-facing symptoms, not internal metrics
```

Alert Routing

```
Alert Routing:

+------------------+
| Prometheus       |
| (fires alerts)   |
+--------+---------+
         |
+--------v---------+
| Alertmanager     |
|                  |
| - Group related  |
|   alerts         |
| - Suppress       |
|   duplicates     |
| - Route by       |
|   severity       |
| - Silence during |
|   maintenance    |
+--------+---------+
         |
   +-----+-----+-----+
   |           |     |
   v           v     v
+------+  +------+ +--------+
| Slack |  |PagerD| | Email  |
| (info)|  |uty   | | (low)  |
+------+  |(crit) | +--------+
          +------+

Severity Routing:
- CRITICAL: PagerDuty (wakes someone up at 3 AM)
- WARNING:  Slack #alerts channel
- INFO:     Email digest (daily)
```

Avoiding Alert Fatigue

```
Alert Fatigue:

Monday:    47 alerts fired. 3 were real problems.
Tuesday:   52 alerts fired. 1 was a real problem.
Wednesday: 61 alerts fired. 2 were real problems.

By Thursday, the on-call engineer ignores all alerts.
On Friday, a real outage goes unnoticed for 45 minutes.

Rules to Prevent Alert Fatigue:
1. Every alert must be actionable (if not, delete it)
2. Alert on symptoms, not causes (error rate, not CPU)
3. Set thresholds based on data, not guesses
4. Use alert grouping (10 alerts for the same issue = 1 notification)
5. Review alerts monthly: delete noisy ones, adjust thresholds
6. Target: <5 actionable alerts per on-call shift
```

Grafana Dashboards

Grafana is the most popular open-source dashboard tool. It connects to Prometheus, Elasticsearch, and dozens of other data sources to visualize metrics and logs.

Dashboard Design Principles

```
Dashboard Layout:

+-------------------------------------------------------------------+
|                    ORDER SERVICE DASHBOARD                        |
+-------------------------------------------------------------------+
|                                                                   |
|  TOP ROW: Golden Signals (at a glance)                           |
|  +------------+  +------------+  +----------+  +-------------+  |
|  | Requests/s |  | Error Rate |  | P99      |  | Saturation  |  |
|  |   1,247    |  |   0.3%     |  | Latency  |  | CPU: 45%    |  |
|  |  (green)   |  |  (green)   |  |  187ms   |  | Mem: 62%    |  |
|  +------------+  +------------+  | (yellow) |  | (green)     |  |
|                                  +----------+  +-------------+  |
|                                                                   |
|  MIDDLE ROW: Request Details                                     |
|  +------------------------------+  +----------------------------+ |
|  | Requests per Second          |  | Response Time Distribution| |
|  |     ___                      |  |                           | |
|  |    /   \    ___              |  |  P50: [====] 42ms         | |
```

```
|  |   /     \__/   \            |  |  P95: [=========] 187ms    | |
|  |  /              \___        |  |  P99: [============] 892ms | |
|  | /                   \       |  |                            | |
|  +-------------------------------+  +---------------------------+ |
|                                                                     |
|  BOTTOM ROW: Dependencies                                          |
|  +-------------------------------+  +---------------------------+ |
|  | Database Connections          |  | Downstream Service Health | |
|  |  Active: 85 / Max: 200        |  |  Payment:  OK  (12ms)     | |
|  |  Wait: 3                      |  |  Inventory: OK  (8ms)     | |
|  |                               |  |  Email:    SLOW (450ms)   | |
|  +-------------------------------+  +---------------------------+ |
+---------------------------------------------------------------------+

Dashboard Design Rules:
1. Golden signals at the top (most important, visible first)
2. Time range selector (last 1h, 6h, 24h, 7d)
3. Color coding: Green (good), Yellow (warning), Red (critical)
4. One dashboard per service (not one giant dashboard)
5. Link to related dashboards (click database panel -> DB dashboard)
```

SLI, SLO, and SLA

These three terms define how you measure and commit to reliability.

SLI: Service Level Indicator

An SLI is a measurement of a specific aspect of your service's behavior. It is the raw metric.

```
SLI Examples:

- Request latency: P99 response time = 187ms
- Availability: Successful requests / Total requests = 99.95%
- Error rate: Failed requests / Total requests = 0.05%
- Throughput: 1,247 requests per second
```

SLO: Service Level Objective

An SLO is a target for your SLI. It is the goal your team aims to meet.

```
SLO Examples:
```

```
- "99.9% of requests will complete in under 500ms"
- "The service will be available 99.95% of the time"
- "The error rate will be below 0.1%"

SLO = SLI + Target

SLI: P99 latency is 187ms
SLO: P99 latency must be under 500ms
Status: MEETING SLO (187ms < 500ms)
```

SLA: Service Level Agreement

An SLA is a contract with customers that includes consequences (usually financial) if the SLO is not met.

```
SLA Examples:

"We guarantee 99.99% uptime. If uptime falls below:
 - 99.99%: No penalty
 - 99.9%:  10% credit on monthly bill
 - 99.0%:  25% credit on monthly bill
 - Below 99.0%: 50% credit on monthly bill"

Relationship:

SLI          SLO              SLA
(Metric)     (Target)         (Contract)
"What we     "What we         "What we
 measure"     aim for"         promise"

Availability  99.95% target    99.9% guaranteed
(measured)    (internal goal)  (with penalties)

Note: SLO is stricter than SLA.
Your internal target (99.95%) should be higher than
your external promise (99.9%). This gives you a buffer.
```

Error Budget

```
Error Budget:

SLO: 99.9% availability per month

Total minutes in a month: 43,200 (30 days)
Allowed downtime: 43,200 x 0.001 = 43.2 minutes

Error Budget = 43.2 minutes of downtime per month
```

```
+-----------------------------------------------------+
|  Error Budget for January                           |
|                                                      |
|  Budget: 43.2 minutes                               |
|  Used:   12.5 minutes (Jan 8: DB failover, 12.5m) |
|  Remaining: 30.7 minutes                            |
|                                                      |
|  [=========--------------------------------]        |
|   29% used                71% remaining             |
+-----------------------------------------------------+

If error budget is exhausted:
- Freeze new feature deployments
- Focus on reliability improvements
- Review what caused the budget to be consumed
```

Health Checks

Health checks are endpoints that report whether a service is functioning correctly. They are used by load balancers, orchestrators (Kubernetes), and monitoring systems to detect and replace unhealthy instances.

Types of Health Checks

```
Health Check Types:

1. LIVENESS CHECK: "Is the process running?"
   GET /health/live
   Response: 200 OK

   Checks: Process is alive, not deadlocked
   Action if failed: Restart the container

2. READINESS CHECK: "Can the service handle requests?"
   GET /health/ready
   Response: 200 OK or 503 Service Unavailable

   Checks: Database connected, cache warmed, dependencies available
   Action if failed: Remove from load balancer (but do not restart)

3. DEEP HEALTH CHECK: "Are all dependencies healthy?"
   GET /health/deep
   Response:
   {
```

```
      "status": "degraded",
      "checks": {
        "database": {"status": "healthy", "latency_ms": 5},
        "redis": {"status": "healthy", "latency_ms": 2},
        "payment_gateway": {"status": "unhealthy", "error": "timeout"},
        "disk_space": {"status": "healthy", "free_gb": 45}
      }
    }

    Checks: Every dependency individually
    Use: Debugging, not for load balancer decisions
    Warning: Can be slow, do not call frequently


Health Check Flow with Load Balancer:

+--------+
| Load   |    GET /health/ready (every 10s)
| Balancer| ----+----+----+
+---+----+     |    |    |
    |          v    v    v
    |     +----+ +----+ +----+
    |     | S1 | | S2 | | S3 |
    |     | OK | | OK | |FAIL|
    |     +----+ +----+ +----+
    |                      |
    |              Remove from rotation
    |              (no traffic until healthy)
    |
    | Route traffic only to S1 and S2
    v
+--------+  +--------+
| S1     |  | S2     |
+--------+  +--------+
```

The Four Golden Signals

Google's Site Reliability Engineering book defines four golden signals that every service should monitor. If you monitor nothing else, monitor these.

1. Latency

How long it takes to serve a request. Track both successful and failed requests separately (failed requests are often fast because they error out immediately).

```
Latency Monitoring:

Track percentiles, not averages!

Average latency: 85ms   <-- Looks fine!

But the distribution:
P50:  42ms    (half of requests are fast)
P90: 120ms    (10% of users wait over 120ms)
P95: 350ms    (5% of users wait over 350ms)
P99: 2,100ms  (1% of users wait over 2 seconds!)

The average hides the long tail.
1% of 1 million daily users = 10,000 users having a bad experience.

Always monitor P50, P95, and P99.
```

2. Traffic

How much demand is placed on your system. Measured as requests per second for web services, or transactions per second for databases.

```
Traffic Monitoring:

Requests per Second (RPS):
+-----------------------------------------------------+
|                                                      |
|  2000 |              ****                          |
|       |           ***    ***                       |
|  1500 |         **          **                     |
|       |       **              **                   |
|  1000 |     **                  **                 |
|       |   **                      **               |
|   500 | **                          ****           |
|       |*                                ****       |
|     0 +--+--+--+--+--+--+--+--+--+--+--+--+--+--+  |
|       00 02 04 06 08 10 12 14 16 18 20 22 24     |
|                    Hour of Day                      |
+-----------------------------------------------------+

Normal pattern: Low at night, peak during business hours.
Anomaly: Sudden spike at 3 AM = possible attack or bot traffic.
```

3. Errors

The rate of requests that fail. This includes explicit errors (HTTP 5xx), implicit errors (HTTP 200 but wrong content), and policy-based errors (responses slower than a threshold).

```
Error Rate Monitoring:

Error Rate = Failed Requests / Total Requests

+-----------------------------------------------------+
| Error Rate over Time                                |
|                                                      |
|  5% |                                                |
|     |                          *                     |
|  4% |                         * *                    |
|     |                        *   *                   |
|  3% |                       *     *                  |
|     |                      *       *                 |
|  2% |                     *         *                |
|     | SLO: 1% -------*-----------*---------          |
|  1% |              **             **                 |
|     |            **                 ***              |
|  0% |************                     *********      |
|     +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+  |
|     10:00    10:15    10:30    10:45    11:00       |
+-----------------------------------------------------+

At 10:25, error rate crossed the 1% SLO.
Alert fired. Engineer investigated.
Root cause: Database connection pool exhausted.
Fixed at 10:45. Error rate returned to normal.
```

4. Saturation

How "full" your service is. How close you are to capacity limits. Track the resources that will exhaust first.

```
Saturation Monitoring:

Resource Utilization:

CPU:           [==================--------] 67%    OK
Memory:        [=====================----] 82%    WARNING
Disk I/O:      [========----------------] 34%    OK
DB Connections:[========================] 95%    CRITICAL!
Network:       [============------------] 51%    OK

The most saturated resource is the bottleneck.
In this case: database connections at 95%.
If traffic increases 5%, connections will be exhausted.

Action: Increase connection pool size or add read replicas.
```

All Four Together

```
Four Golden Signals Dashboard:

+---------------------------+---------------------------+
|  LATENCY                  |  TRAFFIC                  |
|  P50: 42ms  P99: 187ms   |  1,247 req/s             |
|  [Status: OK]             |  [Status: OK]             |
+---------------------------+---------------------------+
|  ERRORS                   |  SATURATION               |
|  Rate: 0.3%               |  CPU: 45%  Mem: 62%       |
|  [Status: OK]             |  DB Conn: 85/200 (42%)    |
|                           |  [Status: OK]             |
+---------------------------+---------------------------+

If any of these turn red, you have a problem.
Start investigating immediately.
```

Putting It All Together: Production Observability Stack

```
Complete Observability Architecture:

+----------+  +----------+  +----------+  +----------+
| Service  |  | Service  |  | Service  |  | Service  |
| A        |  | B        |  | C        |  | D        |
+----+-----+  +----+-----+  +----+-----+  +----+-----+
     |              |              |              |
     | Metrics      | Logs         | Traces       | All three
     |              |              |              |
+----v----+    +----v----+    +----v----+    +----v----+
|Prometheus|   |Filebeat |    | Jaeger  |    |         |
| (scrape) |   | (ship)  |    | Agent   |    |         |
+----+-----+   +----+----+    +----+----+    |         |
     |              |              |          |         |
     v              v              v          |         |
+----------+  +----------+  +----------+     |         |
|Prometheus|  |Elastic   |  | Jaeger   |     |         |
|  TSDB    |  |search    |  | Collector|     |         |
+----+-----+  +----+-----+  +----+-----+     |         |
     |              |              |          |         |
     |              |              |          |         |
     +--------+-----+-------+-----+          |         |
              |              |                |         |
              v              v                |         |
         +----+----+   +----+----+            |         |
         | Grafana |   | Kibana  |            |         |
         | (Metrics|   | (Logs)  |            |         |
```

```
           |  & Dash)|   |         |            |         |
           +---------+   +---------+            |         |
                |                               |         |
                v                               |         |
           +----+----+                          |         |
           | Alert   |                          |         |
           | manager |                          |         |
           +----+----+                          |         |
                |                               |         |
        +------+------+                         |         |
        |      |      |                         |         |
        v      v      v                         |         |
     Slack  PagerDuty Email                     |         |
                                                |         |
                                               +---------+


  Investigation Workflow:

  1. Alert fires:     "Order service error rate > 1%"
  2. Open Grafana:    Check golden signals dashboard
  3. Narrow timeframe: Error started at 10:23 AM
  4. Search Kibana:   "level:ERROR AND service:order-service AND @timestamp > 10:23"
  5. Find error:      "Connection pool exhausted, max connections reached"
  6. Open Jaeger:     Find slow traces around 10:23
  7. See bottleneck:  DB queries taking 5s instead of 50ms
  8. Root cause:      A new query without an index causing table scans
  9. Fix:             Add database index, deploy
  10. Verify:         Error rate drops, latency normalizes on Grafana
```

Trade-offs

Decision	Option A	Option B
Metrics system	Prometheus (pull, open-source, flexible)	Datadog (push, managed, expensive)
Log aggregation	ELK stack (open-source, complex to operate)	Cloud-native (CloudWatch, managed, simpler)
Tracing	Jaeger (open-source, self-hosted)	AWS X-Ray (managed, vendor lock-in)
Log format	Structured JSON (searchable, larger)	Plain text (human-readable, hard to parse)

Sampling	Trace everything (complete data, expensive)	Sample 1% (cheap, may miss rare errors)
Alert threshold	Tight (catches everything, noisy)	Loose (quiet, may miss issues)
Dashboard	One per service (focused, many dashboards)	Global overview (context, less detail)

Common Mistakes

1. No monitoring until something breaks. By then it is too late. Build monitoring into your system from day one, not as an afterthought.
2. Alerting on causes instead of symptoms. Do not alert on "CPU at 80%." Alert on "error rate above 1%" or "P99 latency above 500ms." Users do not care about your CPU; they care about the experience.
3. Using averages instead of percentiles for latency. An average of 85ms hides the fact that 1% of users are waiting 2 seconds. Always use P50, P95, and P99.
4. Too many alerts (alert fatigue). If your on-call engineer gets 50 alerts per shift and only 2 are real problems, they will start ignoring all alerts. Every alert must be actionable. Delete the rest.
5. Not correlating metrics, logs, and traces. Metrics tell you something is wrong. Logs tell you what is wrong. Traces tell you where it is wrong. You need all three working together with shared correlation IDs.
6. No runbooks for alerts. An alert that says "Error rate is high" without a link to troubleshooting steps is useless at 3 AM. Every alert should link to a runbook.

Best Practices

1. Instrument the four golden signals for every service. Latency, traffic, errors, and saturation. If you monitor nothing else, monitor these.
2. Use structured logging (JSON). Make every log entry machine-parseable with explicit fields for service, level, trace ID, and relevant business data.
3. Include trace IDs in all logs. This lets you correlate logs across services for a single request. Search Kibana by trace ID to see the full picture.

4. Set SLOs based on user impact. Define what "good enough" means for your users, then measure against it. Use error budgets to balance reliability and feature velocity.
5. Alert on SLO burn rate, not raw thresholds. Instead of "error rate > 1%," alert on "we are consuming our monthly error budget 10x faster than normal." This reduces false positives.
6. Review alerts quarterly. Delete alerts nobody acts on. Adjust thresholds based on historical data. Add alerts for new failure modes you discovered.
7. Build dashboards top-down. Start with the overview (four golden signals), then drill down into subsystems (database, cache, queue). Link dashboards together for easy navigation.
8. Practice incident response. Run game days where you inject failures and practice using your observability tools to find and fix problems. The worst time to learn your dashboards is during a real outage.

Quick Summary

Observability has three pillars: metrics (numeric measurements over time), logs (detailed event records), and traces (request journeys across services). Prometheus collects metrics by scraping endpoints, stores them as time series, and supports alerting via PromQL and Alertmanager. The ELK stack aggregates logs from hundreds of servers into a searchable central store. Jaeger provides distributed tracing to follow requests across microservices, revealing bottlenecks and dependencies. Alerts should be actionable, symptom-based, and linked to runbooks. Grafana dashboards visualize system health at a glance. SLIs measure service behavior, SLOs set targets, and SLAs make contractual promises with consequences. Health checks let load balancers and orchestrators detect and replace unhealthy instances. The four golden signals -- latency, traffic, errors, and saturation -- are the minimum metrics every service must monitor.

Key Points

- Metrics, logs, and traces each answer different questions: what happened, what exactly happened, and what was the request's journey
- Prometheus uses a pull model to scrape metrics from service endpoints every 15 seconds

- The ELK stack centralizes logs from hundreds of servers into a searchable store
- Distributed tracing with Jaeger follows requests across services to find bottlenecks
- Alert on symptoms (error rate, latency) not causes (CPU, memory)
- SLOs define reliability targets; error budgets balance reliability with feature velocity
- The four golden signals (latency, traffic, errors, saturation) are the minimum monitoring for any service
- Always use percentiles (P50, P95, P99) instead of averages for latency

---

Practice Questions

1. Your e-commerce platform has 30 microservices. The checkout flow takes 3 seconds instead of the usual 500 milliseconds. You have Prometheus, ELK, and Jaeger deployed. Walk through exactly how you would diagnose the problem step by step.
2. Your SLO is 99.9% availability. It is January 20th and you have already consumed 80% of your monthly error budget. What actions would you take? When would you resume normal feature deployments?
3. Your on-call engineer received 147 alerts last week. Only 8 required action. How would you reduce alert noise while ensuring real problems are still caught?
4. You are deploying a new service. What metrics, logs, and traces would you instrument before the first deployment? What dashboards and alerts would you create?
5. Your service has a P99 latency of 2 seconds but a P50 of 50 milliseconds. The average is 120 milliseconds. Your SLO says P99 must be under 500 milliseconds. What could cause such a large gap between P50 and P99? How would you investigate?

Exercises

Exercise 1: Design an Observability Stack

You are building an e-commerce platform with 15 microservices. Design the complete observability stack: metrics collection, log aggregation, distributed tracing, alerting, and dashboards. Choose specific tools for each layer. Define the four golden signals for three critical services (API gateway, order service, payment service). Write the alert rules and specify the routing (what goes to Slack, what wakes someone up).

Exercise 2: Create an SLO Framework

Define SLOs for a food delivery application. Consider these user journeys: browsing restaurants, placing an order, tracking a delivery, and making a payment. For each journey, define the SLIs you would measure, the SLO targets, and the error budget. Calculate how much downtime is allowed per month for each SLO. Describe what happens when the error budget is exhausted.

Exercise 3: Incident Investigation

It is 2:00 AM. An alert fires: "Payment service error rate exceeded 5% for 10 minutes." Write a detailed incident investigation plan. What dashboards would you check first? What log queries would you run? How would you use traces to find the root cause? Write the post-incident review document including timeline, root cause, impact, and preventive measures.

What Is Next?

You now have the tools to see inside your distributed systems: metrics for the big picture, logs for the details, and traces for the journey. You can set targets with SLOs, alert on problems, and investigate incidents systematically. With the foundation chapters complete -- from scaling and databases to message queues, microservices, event-driven architecture, blob storage, search, notifications, and monitoring -- you are ready to put it all together. In the next chapters, you will design complete systems from scratch: URL shorteners, social media platforms, chat applications, and more. Each design will combine multiple concepts from these foundation chapters into a cohesive architecture.

Chapter 21: Designing a URL Shortener (TinyURL / Bit.ly)

You click a short link like `https://bit.ly/3xK9mQ2` and instantly land on a long, complicated URL. Behind that tiny redirect lives a fascinating system that handles billions of requests, stores hundreds of millions of mappings, and responds in single-digit milliseconds. In this chapter, we will design that system from scratch.

A URL shortener is one of the most popular system design interview questions, and for good reason. It is simple enough to explain in five minutes yet deep enough to explore caching, hashing, databases, analytics, and horizontal scaling. By the end of this chapter you will have a production-ready blueprint you can confidently walk through in any interview.

21.1 Understanding the Problem

Before we draw a single box on a whiteboard, let us make sure we agree on what we are building.

A URL shortener takes a long URL such as:

```
https://www.example.com/products/electronics/smartphones/iphone-15-pro-max?color=blue&s...
```

and converts it into something like:

```
https://short.ly/aB3kQ7z
```

When a user visits the short URL, the system looks up the original long URL and redirects the browser.

Why Do People Use URL Shorteners?

1. Sharing on character-limited platforms. A tweet or SMS has limited space.
2. Tracking clicks. Marketers want to know how many people clicked a link, from which country, and on what device.
3. Aesthetics. A clean, short link looks better in emails, presentations, and printed material.
4. Custom branding. Companies create branded short domains (`youtu.be`, `amzn.to`) to reinforce trust.

21.2 Requirements

Functional Requirements

#	Requirement	Description
F1	Shorten a URL	Given a long URL, generate a unique short URL.
F2	Redirect	When a user visits the short URL, redirect to the original long URL.
F3	Custom aliases	Allow users to pick a custom short code (e.g., `short.ly/my-brand`).
F4	Expiration	URLs can optionally expire after a set time.
F5	Analytics	Track click count, timestamp, referrer, country, and device for each short URL.

Non-Functional Requirements

#	Requirement	Target
NF1	High availability	99.99% uptime. Redirects must always work.
NF2	Low latency	Redirect in under 10 ms (p99).

NF3	Scalability	Handle 100 million new URLs per day and 10 billion redirects per day.
NF4	Durability	Once a short URL is created, it must never be lost.
NF5	No collisions	Two different long URLs must never map to the same short code.

Out of Scope

- User accounts and authentication (we will keep the design focused).
- Spam or malware detection (important in production but orthogonal to core design).

21.3 Back-of-the-Envelope Estimation

Good estimations show the interviewer you can think about scale before you start drawing boxes.

Traffic

Metric	Value
New URLs per day	100 million
New URLs per second	100M / 86,400 ≈ 1,160 URLs/s
Read-to-write ratio	~100:1 (reads dominate)
Redirects per second	1,160 x 100 = 116,000 redirects/s

This is a heavily read-heavy system. Caching will be critical.

Storage

Item	Size
Short code	7 characters = 7 bytes

Long URL (average)	200 bytes
Created timestamp	8 bytes
Expiration timestamp	8 bytes
User ID (optional)	8 bytes
Total per record	~250 bytes

Over five years:

```
100M URLs/day x 365 days x 5 years = 182.5 billion URLs
182.5 billion x 250 bytes ≈ 45.6 TB
```

That is manageable with modern distributed databases.

Key Space

If we use a 7-character short code with base62 encoding (a-z, A-Z, 0-9):

```
62^7 = 3.52 trillion unique codes
```

With 182.5 billion URLs over 5 years, we use about 5% of the key space. Plenty of room.

Bandwidth

Direction	Calculation	Result
Write (incoming)	1,160 URLs/s x 250 bytes	~290 KB/s
Read (outgoing)	116,000 req/s x 250 bytes	~29 MB/s

Very lightweight. This system is not bandwidth-bound; it is latency-bound.

Cache

If we follow the 80/20 rule (20% of URLs generate 80% of traffic), caching the top 20% of daily redirects:

```
116,000 req/s x 86,400 seconds = ~10 billion redirects/day
20% of unique URLs accessed daily ≈ 20M URLs
```

```
20M x 250 bytes ≈ 5 GB of cache
```

Five gigabytes fits easily in a single Redis instance, though we will use a cluster for redundancy.

21.4 High-Level Design

Here is the architecture at a glance:

```
                    ┌──────────────────┐
                    │   Load Balancer  │
                    └──────────────────┘
                              │
                              │
          ┌───────────────────┼──────────────────┐
          │                   │                  │
   ┌──────▼──────┐   ┌──────▼──────┐   ┌──────▼──────┐
   │ App Server  │   │ App Server  │   │ App Server  │
   │  (Write)    │   │  (Read)     │   │  (Read)     │
   └─────────────┘   └─────────────┘   └─────────────┘
          │                   │                  │
          │                   │                  │
          │            ┌──────▼──────────────────▼──────┐
          │            │       Cache Cluster            │
          │            │         (Redis)                │
          │            └────────────────────────────────┘
          │                          │
          │                          │ (cache miss)
   ┌──────▼──────────────────────────▼──────────────┐
   │              Database Cluster                  │
   │         (Key-Value / NoSQL Store)              │
   │   ┌─────────┐  ┌─────────┐  ┌─────────┐       │
   │   │Shard 1  │  │Shard 2  │  │Shard N  │       │
   │   └─────────┘  └─────────┘  └─────────┘       │
   └────────────────────────────────────────────────┘
          │
   ┌──────▼──────────────────────────┐
   │   Analytics Pipeline            │
   │  (Kafka -> Spark -> OLAP)       │
   └─────────────────────────────────┘
```

Request Flows

Flow 1: Shorten a URL (Write)

1. Client sends `POST /api/v1/shorten` with the long URL.
2. Load balancer routes to an app server.

3. App server generates a unique short code (we will discuss how shortly).
4. App server writes `(short_code -> long_url)` to the database.
5. App server returns the short URL to the client.

Flow 2: Redirect (Read)

1. User clicks `https://short.ly/aB3kQ7z`.
2. Load balancer routes to an app server.
3. App server checks the cache for `aB3kQ7z`.
4. Cache hit: Return the long URL immediately.
5. Cache miss: Query the database, populate the cache, return the long URL.
6. App server responds with an HTTP redirect (301 or 302).
7. App server asynchronously sends a click event to the analytics pipeline.

21.5 Deep Dive: Short Code Generation

This is the heart of the system. How do we turn a long URL into a unique 7-character code? There are several approaches, each with trade-offs.

Approach 1: Hash + Truncate

Take the long URL, hash it with MD5 or SHA-256, then take the first 7 characters of the base62-encoded hash.

```
long_url = "https://www.example.com/very/long/path"
hash     = MD5(long_url)          # 128-bit hash
base62   = base62_encode(hash)    # "aB3kQ7zXm9pL..."
code     = base62[:7]             # "aB3kQ7z"
```

Pros:

- Simple to implement.
- Same input always produces the same output (deterministic).
- No coordination needed between servers.

Cons:

- Collisions. Truncating a hash to 7 characters means collisions will happen. With the birthday paradox, collisions become likely after roughly `sqrt(62^7)` ≈ 1.9 million URLs. We generate 100 million per day, so collisions will be frequent.

Collision handling: When a collision is detected (the short code already exists in the database), we can append a counter or a random salt to the URL and re-hash:

```python
def generate_short_code(long_url):
    for attempt in range(MAX_RETRIES):
        candidate = base62(md5(long_url + str(attempt)))[:7]
        if not database.exists(candidate):
            return candidate
    raise Exception("Failed to generate unique code")
```

This works but adds latency on collisions and requires a database check for every generation.

Approach 2: Pre-Generated Key Service (Recommended)

Maintain a separate Key Generation Service (KGS) that pre-generates unique 7-character codes and stores them in a database. When an app server needs a new code, it grabs one from the pool.

```
┌──────────────────┐      ┌──────────────────────────┐
│ Key Generation   │      │      Key Database        │
│    Service       │──────>│  ┌──────────┐ ┌──────────┐ │
│                  │      │  │ Unused   │ │   Used   │ │
└──────────────────┘      │  │ Keys     │ │   Keys   │ │
                          │  └──────────┘ └──────────┘ │
                          └──────────────────────────┘
                                      │
                          ┌──────────▼──────────┐
                          │    App Servers      │
                          │ (grab keys in       │
                          │  batches of 1000)   │
                          └─────────────────────┘
```

How it works:

1. KGS generates millions of random 7-character base62 strings and stores them in an "unused" table.
2. When an app server starts (or runs low), it requests a batch of keys (say, 1,000) from KGS.
3. KGS moves those keys from "unused" to "used" in a single atomic transaction.
4. The app server assigns keys from its local batch. No coordination with other servers needed.
5. If the server crashes, we lose at most 1,000 keys. With 3.52 trillion possible keys, this is negligible.

Pros:

- Zero collisions (keys are unique by construction).
- Extremely fast (no hashing, no database check per URL).
- Scales horizontally (each server has its own batch).

Cons:

- Extra component (the KGS) to maintain.
- A small number of keys are wasted if a server crashes mid-batch.

Approach 3: Counter-Based with Base62

Use a global counter (or distributed counter like a Snowflake ID generator) and encode it in base62.

```
counter = 1000000      # auto-incrementing
code    = base62(1000000)  # "4c92"
```

Pros:

- Guaranteed unique.
- Sequential, so database writes are sequential (good for B-tree indexes).

Cons:

- Short codes are predictable. Competitors can enumerate your URLs.
- Requires a centralized counter or a distributed ID generator like Twitter Snowflake.

To hide the sequence, you can apply a bijective shuffle (a reversible permutation) before encoding.

Recommendation

Use the Pre-Generated Key Service for production systems. It gives zero collisions, no coordination overhead, and scales cleanly. Use the hash approach when you need deterministic mappings (same URL always gets the same code).

21.6 Deep Dive: 301 vs. 302 Redirect

When the app server returns the long URL, it sends an HTTP redirect. But which status code?

Status Code	Meaning	Browser Behavior
301 Moved Permanently	The short URL permanently maps to the long URL.	Browser caches the redirect. Future clicks skip our servers entirely.
302 Found (Temporary)	The mapping might change.	Browser does not cache. Every click hits our servers.

Which Should We Use?

It depends on our priorities:

- Use 301 if you want to reduce server load. Once the browser caches the redirect, it never asks our server again. But we lose analytics data for repeat visitors.
- Use 302 if you want accurate analytics. Every click hits our servers, so we can count every visit, track referrers, and measure unique vs. returning visitors.

Most URL shorteners use 302 because analytics are a key selling point. Bit.ly, for example, uses 302 so it can track every single click.

If you want the best of both worlds, use 302 for analytics but set a short `Cache-Control` header (e.g., 5 minutes) so the browser caches briefly to handle rapid double-clicks.

21.7 Deep Dive: Database Choice

Why Key-Value Store?

Our access pattern is simple:

- Write: `PUT(short_code, long_url_record)`
- Read: `GET(short_code) -> long_url_record`

There are no complex joins, no range queries, no transactions spanning multiple records. This is a textbook use case for a key-value store.

Options

Database	Strengths	Considerations

DynamoDB	Fully managed, auto-scaling, single-digit ms latency	Cost at extreme scale
Cassandra	High write throughput, linear scalability	Operational complexity
Redis (persistent)	Blazing fast, simple API	Memory cost for 45 TB
MongoDB	Flexible schema, good tooling	Less ideal for pure KV

Recommendation: DynamoDB or Cassandra. Both handle our scale (100M+ writes/day) with ease and shard automatically.

Database Schema

```
Table: url_mappings

┌──────────────┬──────────────────┬──────────────┐
│ Column       │ Type             │ Notes        │
├──────────────┼──────────────────┼──────────────┤
│ short_code   │ VARCHAR(7)       │ Primary Key  │
│ long_url     │ VARCHAR(2048)    │ NOT NULL     │
│ created_at   │ TIMESTAMP        │ NOT NULL     │
│ expires_at   │ TIMESTAMP        │ NULLABLE     │
│ user_id      │ VARCHAR(36)      │ NULLABLE     │
│ custom_alias │ BOOLEAN          │ DEFAULT false│
└──────────────┴──────────────────┴──────────────┘

Table: analytics_events  (in OLAP store like ClickHouse)

┌──────────────┬──────────────────┬──────────────┐
│ Column       │ Type             │ Notes        │
├──────────────┼──────────────────┼──────────────┤
│ event_id     │ UUID             │ Primary Key  │
│ short_code   │ VARCHAR(7)       │ Indexed      │
│ clicked_at   │ TIMESTAMP        │ NOT NULL     │
│ referrer     │ VARCHAR(2048)    │ NULLABLE     │
│ user_agent   │ VARCHAR(512)     │ NULLABLE     │
│ ip_address   │ VARCHAR(45)      │ For geo-loc  │
│ country      │ VARCHAR(2)       │ Derived      │
│ device_type  │ VARCHAR(20)      │ Derived      │
└──────────────┴──────────────────┴──────────────┘
```

21.8 API Design

Create Short URL

```
POST /api/v1/urls
Content-Type: application/json

Request:
{
    "long_url": "https://www.example.com/very/long/path",
    "custom_alias": "my-brand",       // optional
    "expires_at": "2025-12-31T23:59:59Z"  // optional
}

Response: 201 Created
{
    "short_url": "https://short.ly/my-brand",
    "short_code": "my-brand",
    "long_url": "https://www.example.com/very/long/path",
    "created_at": "2024-06-15T10:30:00Z",
    "expires_at": "2025-12-31T23:59:59Z"
}
```

Redirect

```
GET /{short_code}
Example: GET /aB3kQ7z

Response: 302 Found
Location: https://www.example.com/very/long/path
```

Get Analytics

```
GET /api/v1/urls/{short_code}/analytics

Response: 200 OK
{
    "short_code": "aB3kQ7z",
    "total_clicks": 42567,
    "unique_visitors": 31204,
    "clicks_by_country": {
        "US": 18234,
        "UK": 8912,
        "IN": 5601
    },
    "clicks_by_device": {
        "mobile": 25540,
        "desktop": 15123,
        "tablet": 1904
    },
    "clicks_over_time": [
```

```
        {"date": "2024-06-15", "clicks": 1234},
        {"date": "2024-06-16", "clicks": 2345}
    ]
}
```

Delete Short URL

```
DELETE /api/v1/urls/{short_code}

Response: 204 No Content
```

21.9 Deep Dive: Cache Layer

With 116,000 redirects per second and a read-to-write ratio of 100:1, caching is essential.

Cache Strategy

We use cache-aside (also called lazy loading):

```
┌──────────┐      ┌────────┐      ┌──────────┐
│ App      │──────>│ Redis  │      │ Database │
│ Server   │      │ Cache  │      │          │
│          │<──────│        │      │          │
│          │      └────────┘      │          │
│          │          │           │          │
│          │    (cache miss)      │          │
│          │──────────────────────>│          │
│          │<──────────────────────│          │
│          │                      │          │
│          │──(populate cache)    │          │
│          │──────>│ Redis  │     │          │
└──────────┘      └────────┘      └──────────┘
```

1. App server checks Redis for the short code.
2. Hit: Return the long URL. Done.
3. Miss: Query the database, write the result to Redis with a TTL, return the long URL.

Eviction Policy

Use LRU (Least Recently Used) eviction. Popular URLs stay in cache; rarely accessed URLs get evicted naturally.

Cache Sizing

From our estimation, we need about 5 GB for the top 20% of URLs. A Redis cluster with 3 nodes (each 4 GB, with replication) handles this comfortably.

Cache Invalidation

- On expiration: When a URL expires, we do not need to actively remove it from cache. Set the Redis TTL to match the URL's expiration time, and Redis will evict it automatically.
- On deletion: When a user deletes a short URL, explicitly remove it from cache.
- On update: If we allow updating the long URL for a short code, invalidate the cache entry on update.

21.10 Deep Dive: Analytics Pipeline

Analytics must not slow down the redirect path. We use an asynchronous pipeline.

```
┌──────────────┐      ┌──────────┐      ┌──────────────────┐      ┌──────────────┐
│ App Server   │──────>│  Kafka   │──────>│ Stream           │──────>│  OLAP DB     │
│ (click       │      │ (event   │      │ Processor        │      │(ClickHouse   │
│  event)      │      │  queue)  │      │ (Flink/Spark)    │      │ or Druid)    │
└──────────────┘      └──────────┘      └──────────────────┘      └──────────────┘
```

1. When a redirect happens, the app server fires a click event to Kafka (fire-and-forget, non-blocking).
2. A stream processor (Apache Flink or Spark Streaming) consumes events, enriches them (geo-IP lookup, device parsing), and writes aggregated results to an OLAP database.
3. The analytics API reads from the OLAP database.

This decoupling means:

- Redirect latency is unaffected by analytics processing.
- We can replay Kafka events if the OLAP database needs rebuilding.
- Analytics processing can scale independently.

21.11 Deep Dive: Custom Aliases

When a user requests a custom alias like `short.ly/my-brand`, we need to:

1. Validate the alias: Only allow alphanumeric characters, hyphens, and underscores. Reject reserved words (`api`, `admin`, `health`).
2. Check uniqueness: Query the database to ensure the alias is not taken.
3. Store it: Save it just like an auto-generated code. The `custom_alias` flag in the schema marks it as user-chosen.

Custom aliases bypass the Key Generation Service entirely. The user provides the key; we just validate and store it.

Potential Issues

- Squatting: Someone might reserve popular aliases. Consider requiring authentication for custom aliases.
- Length limits: Custom aliases should be between 3 and 30 characters. Too short risks collisions with auto-generated codes; too long defeats the purpose.

21.12 Deep Dive: URL Expiration

Some URLs should expire after a set period (e.g., a promotional link valid for 30 days).

How It Works

1. When creating a URL, the user optionally provides an `expires_at` timestamp.
2. During redirect, the app server checks if `expires_at` has passed.
3. If expired, return a 410 Gone response instead of redirecting.

Cleanup

Expired URLs still consume storage. We need a cleanup process:

- Lazy cleanup: Check expiration on read. If expired, delete and return 410. Simple but leaves dead data in the database.
- Active cleanup: A background job scans for expired URLs and deletes them periodically (e.g., every hour). Use a secondary index on `expires_at` for efficient scanning.

- Database TTL: Some databases (DynamoDB, Cassandra) support native TTL on records. The database automatically deletes expired records. This is the simplest approach.

21.13 Scaling Considerations

Database Sharding

With 45 TB over 5 years, we need to shard the database. The short code is the natural shard key because:

- All lookups are by short code (point queries).
- Short codes are random (with the KGS approach), ensuring even distribution.
- No cross-shard queries needed.

Use consistent hashing to distribute short codes across shards, allowing us to add shards without massive data migration.

App Server Scaling

App servers are stateless (they hold a batch of pre-generated keys in memory, but losing them is acceptable). We can auto-scale horizontally based on request rate.

Read Replicas

For the database, add read replicas to handle the read-heavy workload. Writes go to the primary; reads go to replicas.

Geographic Distribution

Deploy the service in multiple regions. Use a global load balancer (e.g., AWS Global Accelerator, Cloudflare) to route users to the nearest region. Each region has its own cache cluster. The database can use multi-region replication (DynamoDB Global Tables or Cassandra multi-datacenter).

```
                    ┌──────────────────┐
                    │  Global DNS/LB   │
                    └──────────────────┘
                              │
              ┌───────────────┴───────────────┐
              │               │               │
```

```
┌──────▼──────┐  ┌──────▼──────┐  ┌──────▼──────┐
│ US-East     │  │ EU-West     │  │ AP-South    │
│  ┌────────┐ │  │  ┌────────┐ │  │  ┌────────┐ │
│  │ App    │ │  │  │ App    │ │  │  │ App    │ │
│  │Servers │ │  │  │Servers │ │  │  │Servers │ │
│  └────────┘ │  │  └────────┘ │  │  └────────┘ │
│             │  │             │  │             │
│  ┌────────┐ │  │  ┌────────┐ │  │  ┌────────┐ │
│  │ Cache  │ │  │  │ Cache  │ │  │  │ Cache  │ │
│  └────────┘ │  │  └────────┘ │  │  └────────┘ │
│             │  │             │  │             │
│  ┌────────┐ │  │  ┌────────┐ │  │  ┌────────┐ │
│  │   DB   │ │  │  │   DB   │ │  │  │   DB   │ │
│  │Replica │ │  │  │Replica │ │  │  │Replica │ │
│  └────────┘ │  │  └────────┘ │  │  └────────┘ │
└─────────────┘  └─────────────┘  └─────────────┘
```

Rate Limiting

Protect the write endpoint with rate limiting (e.g., 100 URLs per minute per IP). The read (redirect) endpoint should not be rate-limited for normal traffic, but use DDoS protection at the load balancer level.

21.14 Complete Architecture

Putting it all together:

```
                    ┌──────────────────────┐
                    │   DNS / Global LB    │
                    └──────────────────────┘
                              │
                    ┌────────▼─────────────┐
                    │   Rate Limiter /     │
                    │   API Gateway        │
                    └──────────────────────┘
                              │
                  ┌───────────┼──────────────┐
                  │           │              │
            ┌─────▼─────┐ ┌───▼───────┐ ┌────▼──────┐
            │ App Server│ │App Server │ │App Server │
            │     1     │ │     2     │ │     N     │
            └───────────┘ └───────────┘ └───────────┘
                  │           │              │
        ┌─────────┼───────────┼──────────────┘
        │         │           │
   ┌────▼────┐ ┌──▼──────┐ ┌──▼──────────────┐
   │ Key Gen │ │ Redis   │ │ Analytics       │
   └─────────┘ └─────────┘ └─────────────────┘
```

```
      │ Service    │ │ Cache    │ │ (Kafka -> Flink    │
      │ (KGS)      │ │ Cluster │ │  -> ClickHouse)    │
      └────────────┘ └──────────┘ └────────────────────┘
            │              │
      ┌─────▼──────────────▼──────────────────────┐
      │          Database Cluster                 │
      │   (DynamoDB / Cassandra, sharded)         │
      │   ┌─────────┐ ┌─────────┐ ┌─────────┐     │
      │   │Shard 1  │ │Shard 2  │ │Shard N  │     │
      │   └─────────┘ └─────────┘ └─────────┘     │
      └───────────────────────────────────────────┘
```

21.15 Trade-offs

Decision	Option A	Option B	Our Choice
Code generation	Hash + truncate	Pre-generated KGS	KGS (zero collisions)
Redirect code	301 (permanent)	302 (temporary)	302 (better analytics)
Database	SQL (PostgreSQL)	NoSQL (DynamoDB)	NoSQL (simple KV, massive scale)
Cache eviction	LRU	LFU	LRU (simpler, works well for URLs)
Analytics	Synchronous	Asynchronous pipeline	Async (no impact on redirect latency)
Expiration cleanup	Lazy (on read)	Active (background job)	DB TTL (simplest, most reliable)
Key length	6 chars (56B keys)	7 chars (3.5T keys)	7 chars (room for growth)

21.16 Common Mistakes

1. Ignoring collisions with hash-based approaches. Candidates often say "just hash it" without addressing what happens when two URLs hash to the same short code. Always discuss collision handling.

2. Forgetting the read-to-write ratio. This is a 100:1 read-heavy system. If you do not mention caching, you have missed the most important optimization.
3. Using 301 without discussing analytics impact. A 301 redirect means the browser caches the mapping forever. You lose all future click data for that user. This is a critical trade-off.
4. Putting analytics on the hot path. Writing analytics data synchronously during a redirect adds latency. Always use an async pipeline.
5. Not discussing how to handle the same URL submitted twice. Should it get the same short code or a different one? Both are valid, but you need to decide and explain your reasoning.
6. Choosing SQL without justification. The access pattern is pure key-value. There is no compelling reason to use a relational database unless you need complex queries on metadata.

21.17 Best Practices

1. Use base62 encoding (a-z, A-Z, 0-9) instead of base64. Base64 includes `+` and `/` which are not URL-safe without encoding.
2. Validate URLs on write. Check that the long URL is well-formed, uses HTTP or HTTPS, and is not pointing to your own domain (to prevent redirect loops).
3. Add a health check endpoint. `/health` should return 200 OK with database and cache connectivity status. This allows your load balancer to route traffic away from unhealthy instances.
4. Use connection pooling for database and cache connections. Creating a new connection per request is prohibitively expensive at 116,000 requests per second.
5. Monitor cache hit ratio. If it drops below 80%, investigate. You may need to increase cache size or adjust TTL.
6. Log and alert on key exhaustion. Monitor the KGS pool. If unused keys drop below a threshold, generate more.

21.18 Quick Summary

A URL shortener maps short codes to long URLs and redirects users. The key design decisions are:

- Code generation: A pre-generated Key Generation Service eliminates collisions and coordination overhead.
- Storage: A NoSQL key-value store (DynamoDB/Cassandra) handles the simple access pattern at scale.
- Caching: Redis caches the top 20% of URLs, handling most of the 116K redirects/second.
- Redirect: Use 302 for analytics tracking; 301 if you prioritize reduced server load.
- Analytics: An async pipeline (Kafka, Flink, ClickHouse) tracks clicks without affecting redirect latency.
- Scaling: Stateless app servers, database sharding by short code, multi-region deployment.

21.19 Key Points

- URL shortening is a read-heavy system with a 100:1 read-to-write ratio.
- A 7-character base62 code gives 3.52 trillion unique keys, far more than needed.
- Pre-generated keys (KGS) are the cleanest solution for uniqueness and performance.
- Caching is the single biggest performance lever. A 5 GB Redis cache handles most traffic.
- Use 302 redirects if you need analytics; 301 if you need lower server load.
- Shard by short code for even distribution across database nodes.
- Decouple analytics from the redirect path using a message queue.

21.20 Practice Questions

1. How would you handle the case where two users submit the same long URL? Should they get the same short code or different ones? What are the trade-offs?
2. If the Key Generation Service goes down, what happens? How would you make it highly available?
3. How would you implement a "preview" feature where users can see where a short URL leads before being redirected?
4. What happens if a popular short URL suddenly gets millions of clicks per second (e.g., posted by a celebrity)? How does your cache handle it?
5. How would you implement rate limiting that is fair across multiple regions?

21.21 Exercises

1. Calculate the key space: If we use a 6-character base62 code, how many unique codes can we generate? Is that enough for 10 years at 200 million URLs per day?
2. Design the KGS: Write pseudocode for the Key Generation Service, including batch allocation and crash recovery.
3. Compare databases: Create a comparison table of DynamoDB, Cassandra, and Redis for this use case, evaluating cost, latency, throughput, and operational complexity.
4. Estimate cache hit ratio: Given that URL access follows a Zipf distribution with parameter s=1.0, what percentage of requests would be served from a cache holding the top 1% of URLs?
5. Design a monitoring dashboard: List the top 10 metrics you would display on an operations dashboard for this system.

21.22 What Is Next?

In the next chapter, we will design Pastebin, a system that shares several patterns with our URL shortener (unique key generation, key-value storage, expiration) but introduces new challenges: storing larger text content, syntax highlighting, and access control. You will see how the building blocks from this chapter can be reused and extended for a different product.

Chapter 22: Designing Pastebin

You need to share a code snippet with a colleague. You paste it into a website, click "create," and get a URL you can send to anyone. They open it and see your code, syntax-highlighted and neatly formatted. That website is Pastebin.

Pastebin is a deceptively simple product. On the surface, it stores text and gives you a link. Under the hood, it handles variable-length content that can range from a single line to megabytes of text, access control, expiration, syntax highlighting, and the same scale challenges as any modern web service. In this chapter, we will design it from the ground up.

If you completed the URL shortener chapter, you will recognize many patterns here: unique key generation, key-value lookups, and expiration. But Pastebin introduces new dimensions that make it a different beast. Let us dive in.

22.1 Understanding the Problem

Pastebin allows users to:

1. Create a paste by submitting text content. The system returns a unique URL.
2. Read a paste by visiting the URL. The text is displayed, optionally with syntax highlighting.
3. Control access: Pastes can be public, private (only the creator can see them), or unlisted (anyone with the link can see them, but the paste does not appear in search results).
4. Set expiration: Pastes can expire after a set period or live forever.

How Is This Different from a URL Shortener?

Aspect	URL Shortener	Pastebin
Stored data	A long URL (small, fixed size)	Text content (variable, potentially large)

Storage strategy	Key-value in database	Metadata in database, content in blob/object store
Content rendering	None (just redirect)	Syntax highlighting, line numbers, raw view
Access control	Typically public	Public, private, unlisted
Content size	~200 bytes	Up to 10 MB

22.2 Requirements

Functional Requirements

#	Requirement	Description
F1	Create paste	Submit text content and receive a unique URL.
F2	Read paste	Retrieve and display paste content by its URL.
F3	Syntax highlighting	Optionally specify a language for code highlighting.
F4	Access control	Support public, private, and unlisted pastes.
F5	Expiration	Pastes can expire after a configurable duration or never.
F6	Raw view	Provide a raw text endpoint (no HTML, no highlighting).
F7	Delete paste	Creator can delete their paste.

Non-Functional Requirements

#	Requirement	Target
NF1	High availability	99.9% uptime.

NF2	Low latency	Read a paste in under 100 ms (p99).
NF3	Scalability	Handle 10 million new pastes per day, 100 million reads per day.
NF4	Durability	Paste content must never be lost before expiration.
NF5	Size limits	Maximum paste size: 10 MB.

Out of Scope

- User registration and authentication (we will note where it plugs in).
- Search across pastes (complex and orthogonal to core design).
- Collaborative editing (that is Google Docs territory).

22.3 Back-of-the-Envelope Estimation

Traffic

Metric	Value
New pastes per day	10 million
New pastes per second	10M / 86,400 ≈ 116 pastes/s
Read-to-write ratio	~10:1
Reads per second	116 x 10 = 1,160 reads/s

This is a moderate scale system, much smaller than the URL shortener. But the content size makes storage the primary concern.

Storage

Item	Size
Average paste content	10 KB

Maximum paste content	10 MB
Metadata per paste	~500 bytes

Daily content storage:

```
10M pastes/day x 10 KB average = 100 GB/day
```

Over five years (assuming 50% of pastes expire within a year):

```
Year 1: 100 GB/day x 365 = 36.5 TB
Years 2-5: 36.5 TB x 0.5 (retained) x 4 = 73 TB
Total: ~110 TB of content
```

Plus metadata:

```
10M pastes/day x 365 x 5 = 18.25 billion records
18.25 billion x 500 bytes ≈ 9.1 TB of metadata
```

Content storage dominates. We need an object store (like S3) for content and a database for metadata.

Bandwidth

Direction	Calculation	Result
Write (incoming)	116 pastes/s x 10 KB	~1.16 MB/s
Read (outgoing)	1,160 reads/s x 10 KB	~11.6 MB/s

Very manageable bandwidth.

Key Space

Using a 8-character base62 code:

```
62^8 = 218 trillion unique keys
```

With 18.25 billion pastes over 5 years, we use 0.008% of the key space. No key exhaustion concerns.

22.4 High-Level Design

```
                    ┌──────────────────┐
                    │   Load Balancer  │
                    └──────────────────┘
                             │
                ┌────────────┼────────────┐
                │            │            │
            ┌───▼────┐   ┌───▼────┐   ┌───▼────┐
            │ App    │   │ App    │   │ App    │
            │ Server 1│  │ Server 2│  │ Server N│
            └────────┘   └────────┘   └────────┘
                │            │            │
        ┌───────┼────────────┼────────────┘
        │       │            │
    ┌───▼───┐ ┌─▼─────┐ ┌────▼──────────┐
    │ Key   │ │ Cache │ │ Object Store  │
    │ Gen   │ │(Redis)│ │ (S3 / Blob)   │
    │Service│ └───────┘ │ ┌─────┐ ┌─────┐│
    └───────┘     │     │ │Paste│ │Paste││
                  │     │ │ 1   │ │ 2   │
            ┌─────▼─────┴─┴─────┴─┴─────┴──┐
            │        Metadata DB           │
            │   (PostgreSQL / DynamoDB)    │
            └──────────────────────────────┘
                        │
            ┌───────────▼──────────────┐
            │  CDN (CloudFront /       │
            │  Cloudflare)             │
            └──────────────────────────┘
```

Two-Tier Storage Architecture

The key insight is separating metadata from content:

- Metadata database stores: paste ID, title, language, access level, timestamps, and a pointer to the content.
- Object store (S3, GCS, Azure Blob) stores the actual paste content. Object stores are optimized for large, variable-size blobs at low cost.

Why not store content in the database? A 10 MB paste in a database row is expensive and slow to retrieve. Object stores are 10-50x cheaper per GB than database storage and purpose-built for this workload.

Request Flows

Flow 1: Create a Paste

1. Client sends `POST /api/v1/pastes` with the text content, language, access level, and expiration.
2. App server gets a unique key from the Key Generation Service.
3. App server uploads the content to the object store: `s3://pastes/{key}`.
4. App server writes metadata to the database.
5. App server returns the paste URL to the client.

Flow 2: Read a Paste

1. User visits `https://paste.example.com/{key}`.
2. CDN checks if it has a cached version (for public pastes).
3. CDN hit: Return cached content.
4. CDN miss: Request reaches the app server.
5. App server checks Redis cache for metadata.
6. App server fetches content from the object store (or cache if recently accessed).
7. App server applies syntax highlighting (if a language is specified).
8. App server returns the rendered page.

22.5 Deep Dive: Unique Key Generation

We can reuse the Key Generation Service (KGS) approach from the URL shortener chapter. A dedicated service pre-generates unique 8-character base62 keys and distributes them in batches to app servers.

Why 8 characters instead of 7? Pastebin URLs are less commonly typed manually (unlike short URLs), so an extra character is a worthwhile trade for a larger key space.

The alternative approaches (hash-based, counter-based) also apply here with the same trade-offs discussed in Chapter 21.

22.6 Deep Dive: Content Storage

Why Object Storage?

Factor	Database (PostgreSQL)	Object Store (S3)
Cost per GB/month	$0.10 - $0.50	$0.023
Maximum object size	~1 GB (practical limit)	5 TB
Read latency	5-20 ms	50-100 ms (first byte)
Throughput	Limited by connections	Virtually unlimited
Durability	99.99% (with replication)	99.999999999% (11 nines)

Object storage wins on cost and durability. The higher latency is mitigated by our cache layer and CDN.

Content Organization

Store pastes in S3 with a flat key structure:

```
s3://pastebin-content/{paste_key}
```

For example: `s3://pastebin-content/aB3kQ7z9`

Do not use folder hierarchies (like `/2024/06/15/aB3kQ7z9`). S3 is not a filesystem; flat keys perform better and simplify lookups.

Compression

Text compresses extremely well. A 10 KB code snippet might compress to 2-3 KB with gzip or zstd. Compressing before storage:

- Reduces storage costs by 60-80%.
- Reduces bandwidth for reads.
- Adds minimal CPU overhead (compression is fast for text).

Store content compressed, with a `Content-Encoding` header noting the compression algorithm.

22.7 Deep Dive: Syntax Highlighting

Syntax highlighting can be done server-side or client-side.

Server-Side Highlighting

Use a library like Pygments (Python), highlight.js (Node.js), or Rouge (Ruby) to convert raw text into highlighted HTML.

```
Input:  def hello():\n    print("Hello")
Output: <span class="kw">def</span> <span class="fn">hello</span>()...
```

Pros: Works without JavaScript. SEO-friendly. Consistent rendering. Cons: CPU cost on the server. Need to regenerate if themes change.

Client-Side Highlighting

Send raw text to the browser and use a JavaScript library (Prism.js, highlight.js) to highlight on the client.

Pros: Zero server CPU cost. Easy to change themes. Lazy highlighting for large files. Cons: Requires JavaScript. Slower initial render for large files.

Recommendation

Use client-side highlighting for the web interface (offloads CPU to the client) and server-side highlighting for the API (when clients request pre-rendered HTML). Cache the highlighted output for popular pastes.

Supported Languages

Support at least the top 30 programming languages. Store the language as metadata so the highlighter knows which grammar to apply. If no language is specified, offer auto-detection (most highlighting libraries support this, though it is imperfect).

22.8 Deep Dive: Access Control

Three Access Levels

Level	Behavior
Public	Anyone can view. Appears in recent pastes and search.
Unlisted	Anyone with the link can view. Does not appear in listings or search.
Private	Only the creator can view. Requires authentication.

Implementation

```
if paste.access_level == "public":
    allow_access()
elif paste.access_level == "unlisted":
    allow_access()  # Security through obscurity (the URL is the secret)
elif paste.access_level == "private":
    if request.user_id == paste.creator_id:
        allow_access()
    else:
        return 403 Forbidden
```

Security Considerations

- Unlisted pastes rely on the URL being secret. With 8-character base62 keys (218 trillion combinations), guessing a URL is practically impossible. But anyone with the link can share it further.
- Private pastes require proper authentication. Use session tokens or API keys.
- Never cache private pastes in the CDN or shared cache. Only cache public and unlisted pastes.

22.9 Deep Dive: Size Limits and Rate Limiting

Size Limits

Limit	Value	Reason

Maximum paste size	10 MB	Prevents abuse, keeps storage manageable
Minimum paste size	1 byte	No reason to allow empty pastes
Maximum title length	256 characters	Keep metadata compact

Enforce size limits at multiple layers:

1. Client-side: Show a warning if content exceeds the limit (immediate feedback).
2. Load balancer / API gateway: Reject requests with `Content-Length` exceeding 10 MB (prevents large payloads from consuming server resources).
3. App server: Validate actual content size after parsing.

Rate Limiting

Operation	Limit	Window
Create paste (unauthenticated)	10 per minute	Per IP
Create paste (authenticated)	60 per minute	Per user
Read paste	300 per minute	Per IP

Use a token bucket or sliding window rate limiter at the API gateway level.

Why Rate Limit?

Without rate limiting, an attacker could:

- Exhaust storage by creating millions of large pastes.
- Consume all pre-generated keys.
- DDoS the write path.

22.10 Deep Dive: CDN for Popular Pastes

A CDN (Content Delivery Network) caches popular pastes at edge locations worldwide, reducing latency and offloading traffic from our servers.

What to Cache

Content Type	Cache in CDN?	TTL
Public pastes	Yes	1 hour
Unlisted pastes	Yes (with full URL as cache key)	1 hour
Private pastes	No	N/A
Raw paste content	Yes (for public/unlisted)	1 hour
Static assets (CSS, JS)	Yes	1 week

Cache Invalidation

When a paste is deleted or updated, we need to invalidate the CDN cache:

- Use the CDN's invalidation API (e.g., CloudFront invalidation, Cloudflare purge).
- Alternatively, include a version hash in the URL (`/paste/{key}?v=abc123`) and update it on modification.

CDN Architecture

```
┌─────────┐      ┌──────────────────────────┐      ┌──────────────┐
│ User    │ ───> │  CDN Edge (PoP)          │ ───> │ Origin       │
│(Brazil) │      │  Sao Paulo               │      │ Server       │
└─────────┘      │   ┌──────────────────┐   │      │ (US-East)    │
                 │   │ Cached: paste X  │   │      │              │
                 │   │ Cached: paste Y  │   │      │              │
                 │   └──────────────────┘   │      │              │
                 └──────────────────────────┘      └──────────────┘
```

For a viral paste (e.g., shared on a popular forum), the CDN handles 99% of traffic. Our origin servers barely feel it.

22.11 Database Schema

Metadata Table

```sql
CREATE TABLE pastes (
    paste_id        VARCHAR(8)   PRIMARY KEY,
    title           VARCHAR(256),
    language        VARCHAR(50),
    access_level    ENUM('public', 'unlisted', 'private') DEFAULT 'unlisted',
    content_url     VARCHAR(512) NOT NULL,  -- S3 URL
    content_size    INTEGER      NOT NULL,  -- bytes
    creator_id      VARCHAR(36),            -- nullable for anonymous
    created_at      TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
    expires_at      TIMESTAMP,              -- nullable = never expires
    view_count      INTEGER      DEFAULT 0,
    is_compressed   BOOLEAN      DEFAULT true
);

-- Index for expiration cleanup
CREATE INDEX idx_expires_at ON pastes(expires_at)
    WHERE expires_at IS NOT NULL;

-- Index for user's pastes
CREATE INDEX idx_creator ON pastes(creator_id, created_at DESC)
    WHERE creator_id IS NOT NULL;

-- Index for public paste listing
CREATE INDEX idx_public_recent ON pastes(created_at DESC)
    WHERE access_level = 'public';
```

Why PostgreSQL for Metadata?

Unlike the URL shortener (which is pure key-value), Pastebin metadata benefits from:

- Indexes on `expires_at`, `creator_id`, and `access_level`.
- SQL queries for listing a user's pastes, finding recent public pastes, and cleanup jobs.
- ACID transactions for create/delete operations.

At 10 million rows per day, PostgreSQL with proper sharding handles the load well. If scale exceeds a single cluster, switch to a horizontally scalable option like CockroachDB or use DynamoDB with secondary indexes.

22.12 API Design

Create a Paste

```
POST /api/v1/pastes
Content-Type: application/json

Request:
{
    "content": "def hello():\n    print('Hello, World!')",
    "title": "My Python Snippet",
    "language": "python",
    "access_level": "unlisted",
    "expires_in": "24h"  // "10m", "1h", "24h", "7d", "30d", "never"
}

Response: 201 Created
{
    "paste_id": "aB3kQ7z9",
    "url": "https://paste.example.com/aB3kQ7z9",
    "raw_url": "https://paste.example.com/raw/aB3kQ7z9",
    "title": "My Python Snippet",
    "language": "python",
    "access_level": "unlisted",
    "created_at": "2024-06-15T10:30:00Z",
    "expires_at": "2024-06-16T10:30:00Z"
}
```

Read a Paste (HTML)

```
GET /pastes/{paste_id}

Response: 200 OK
Content-Type: text/html

(Rendered HTML page with syntax-highlighted content)
```

Read a Paste (Raw)

```
GET /raw/{paste_id}

Response: 200 OK
Content-Type: text/plain

def hello():
    print('Hello, World!')
```

Read Paste Metadata (API)

```
GET /api/v1/pastes/{paste_id}

Response: 200 OK
{
    "paste_id": "aB3kQ7z9",
    "title": "My Python Snippet",
    "language": "python",
    "access_level": "unlisted",
    "content_size": 42,
    "view_count": 156,
    "created_at": "2024-06-15T10:30:00Z",
    "expires_at": "2024-06-16T10:30:00Z"
}
```

Delete a Paste

```
DELETE /api/v1/pastes/{paste_id}
Authorization: Bearer <token>

Response: 204 No Content
```

List User's Pastes

```
GET /api/v1/users/me/pastes?page=1&limit=20

Response: 200 OK
{
    "pastes": [
        {
            "paste_id": "aB3kQ7z9",
            "title": "My Python Snippet",
            "language": "python",
            "created_at": "2024-06-15T10:30:00Z",
            "view_count": 156
        }
    ],
    "total": 42,
    "page": 1,
    "limit": 20
}
```

22.13 Scaling Considerations

Object Store Scaling

S3 and similar object stores scale virtually infinitely. No sharding decisions needed. This is one of the biggest advantages of the two-tier architecture: the hardest scaling problem (content storage) is solved by a managed service.

Metadata Database Scaling

At 10 million inserts per day (~116/s), a single PostgreSQL instance handles writes easily. For reads (1,160/s), add read replicas.

If we grow beyond a single cluster's capacity:

- Shard by paste_id for even distribution.
- Use a consistent hashing ring to map paste IDs to shards.
- The "list user's pastes" query becomes a fan-out across shards (or maintain a separate index by user_id).

App Server Scaling

App servers are stateless (like the URL shortener). Scale horizontally behind a load balancer based on CPU and request rate.

Cache Scaling

Use a Redis cluster with 3-5 nodes. Cache paste metadata and recently accessed content. A 10 GB cache holds ~1 million recent pastes' metadata and ~100,000 content blobs.

Expiration at Scale

Running `DELETE FROM pastes WHERE expires_at < NOW()` on 18 billion rows is expensive. Better approaches:

1. Partition by time. Create monthly partitions. Drop entire partitions when all pastes in them have expired.
2. Lazy deletion. Do not delete expired pastes proactively. Check expiration on read. Run a background sweeper during off-peak hours.
3. Use object store lifecycle rules. S3 lifecycle policies can automatically delete objects after a set period. Set the object's expiration to match the paste's expiration.

```
┌──────────────────────────────────────────┐
│  Expiration Strategy                     │
```

```
│                                          │
│  1. Read request arrives for paste X     │
│  2. Check metadata: is expires_at past?  │
│  3. If yes: return 410 Gone              │
│  4. Background job (hourly):             │
│     - Scan expired pastes                │
│     - Delete from S3                     │
│     - Delete metadata row                │
│  5. S3 lifecycle rule (backup):          │
│     - Auto-delete objects older than     │
│       max_expiration + 1 day             │
└──────────────────────────────────────────┘
```

22.14 Complete Architecture

```
                          ┌──────────────────┐
                          │  CDN (CloudFront) │
                          └──────────────────┘
                                   │
                          ┌────────▼─────────┐
                          │  Load Balancer /  │
                          │  API Gateway      │
                          │ (Rate Limiting)   │
                          └──────────────────┘
                                   │
                     ┌─────────────┼─────────────┐
                     │             │             │
                ┌────▼────┐   ┌────▼────┐   ┌────▼────┐
                │ App Server│ │App Server│ │ App Server│
                │    1    │   │    2    │   │    N    │
                └────┬────┘   └────┬────┘   └────┬────┘
                     │ │ │         │             │
                ┌────┘ │ └─────────┼─────────────┘
                │      │           │
                │      │           │
           ┌────▼────┐ ┌────▼────┐ ┌────▼──────────────┐
           │  Key    │ │  Cache  │ │   S3 / Object Store │
           │  Gen    │ │ (Redis) │ │                    │
           │ Service │ │ Cluster │ │  ┌─────┐ ┌─────┐   │
           └─────────┘ └────┬────┘ │  │Paste│ │Paste│ ...│
                            │      │  │  1  │ │  2  │    │
                       ┌────▼──────┐└──└─────┘ └─────┘   │
                       │ Metadata  │└────────────────────┘
                       │ Database  │
                       │(PostgreSQL)│
                       │ ┌────┐ ┌────┐│
                       │ │ RW │ │ RO │
                       └────────────┘
```

```
                    │
              ┌─────▼──────────────┐
              │ Expiration Worker  │
              │ (Cron Job)         │
              └────────────────────┘
```

22.15 Trade-offs

Decision	Option A	Option B	Our Choice
Content storage	Database (BLOB column)	Object store (S3)	S3 (cheaper, more durable, infinite scale)
Metadata store	NoSQL (DynamoDB)	SQL (PostgreSQL)	PostgreSQL (need indexes, queries, transactions)
Syntax highlighting	Server-side	Client-side	Client-side for web, server-side for API
Compression	None	gzip/zstd	Compress (60-80% storage savings for text)
Expiration	Eager deletion	Lazy + background	Lazy + background (no read-path overhead)
CDN caching	All pastes	Public + unlisted only	Public + unlisted (never cache private)

22.16 Common Mistakes

1. Storing content in the database. This is the most common mistake. Variable-size text blobs (up to 10 MB) do not belong in a relational database. Use an object store for content and a database for metadata.
2. Forgetting access control for private pastes. Saying "the URL is the security" works for unlisted pastes but not for private ones. Private pastes need authentication.
3. No size limits. Without a maximum paste size, attackers can upload gigabytes of data. Always enforce limits at the API gateway and app server level.

4. Ignoring compression. Text compresses remarkably well. Not compressing content wastes 60-80% of storage costs.
5. Caching private pastes in CDN. A CDN is shared infrastructure. Private pastes in a CDN can be served to unauthorized users if cache keys are predictable.
6. No rate limiting. A simple script could create millions of pastes, exhausting storage and keys.

22.17 Best Practices

1. Use pre-signed URLs for S3 reads. Instead of proxying content through your app servers, generate a time-limited S3 URL and redirect the client. This offloads bandwidth from your servers to S3.
2. Store content with its hash for deduplication. If two users paste identical content, store it once and reference it twice. Use SHA-256 of the content as a secondary key.
3. Set reasonable defaults. Default expiration to 30 days (not "never"). This prevents abandoned pastes from accumulating forever.
4. Add abuse detection. Scan paste content for known malware patterns, phishing URLs, or illegal content. Use automated scanning on the write path.
5. Implement "burn after reading." Some pastes should self-destruct after being viewed once. Store a `burn_after_read` flag and delete the paste after the first successful read.
6. Version your API. Use `/api/v1/` prefixes so you can evolve the API without breaking existing clients.

22.18 Quick Summary

Pastebin stores text content and serves it via unique URLs. The key design decisions are:

- Two-tier storage: Metadata in PostgreSQL, content in S3. This separates structured queries from blob storage.
- Key generation: A pre-generated Key Generation Service provides unique 8-character codes.
- CDN caching: Public and unlisted pastes are cached at the edge for low-latency reads.
- Access control: Public, unlisted, and private levels. Private requires authentication; unlisted relies on URL secrecy.
- Syntax highlighting: Client-side for web UI, server-side for API.

- Expiration: Lazy check on read plus a background cleanup job.

22.19 Key Points

- Pastebin shares patterns with URL shorteners (key generation, expiration) but differs in content storage (variable-size blobs vs. small URLs).
- Object storage (S3) is the right choice for paste content: 10-50x cheaper than database storage, 11 nines of durability, and infinite scale.
- Compress text content before storing. Text compresses 60-80%, saving significant storage costs.
- Never cache private pastes in a CDN or shared cache.
- Rate limiting and size limits are essential to prevent abuse.
- Pre-signed S3 URLs offload read traffic from app servers to the object store.
- Syntax highlighting is best done client-side for web and server-side for API consumers.

22.20 Practice Questions

1. If two users paste identical content, should we store it once or twice? What are the privacy implications of deduplication?
2. How would you implement a "burn after reading" feature where the paste is deleted after the first view? What happens if two users click the link at the exact same time?
3. How would you add a search feature for public pastes? What kind of index would you use?
4. If a paste goes viral and gets 1 million views per second, walk through exactly what happens at each layer of the architecture.
5. How would you handle a paste that contains malware or phishing content? At what point in the pipeline would you detect it?

22.21 Exercises

1. Estimate S3 costs. Using current S3 pricing, calculate the monthly cost of storing 100 TB of paste content with standard storage, including PUT and GET request costs at our

traffic levels.

2. Design the deduplication system. Write pseudocode for a system that detects duplicate content and stores it only once, while maintaining separate metadata entries for each paste.
3. Compare object stores. Create a table comparing S3, Google Cloud Storage, and Azure Blob Storage on cost, latency, durability, and maximum object size.
4. Design the expiration worker. Write pseudocode for a background job that efficiently finds and deletes expired pastes across both the metadata database and S3.
5. Calculate CDN savings. If 10% of pastes account for 90% of reads, and a CDN has a 95% hit ratio for those popular pastes, how many requests per second does the origin server handle?

22.22 What Is Next?

In the next chapter, we will tackle a dramatically more complex system: Twitter/X. While Pastebin is a straightforward "store and retrieve" system, Twitter introduces fan-out (how do you deliver a tweet to millions of followers?), real-time timelines, trending topics, and the fascinating trade-off between fan-out on write and fan-out on read. Get ready for one of the most popular system design interview questions.

Chapter 23: Designing Twitter (X)

Every second, thousands of people post tweets, millions scroll through their timelines, and trending topics shift in real time. Twitter (now X) is one of the most fascinating systems to design because it combines a deceptively simple product -- short text posts -- with enormous engineering challenges around fan-out, real-time delivery, and search at scale.

In this chapter we will design Twitter from the ground up. You will learn how timeline generation works, why fan-out strategy matters so much, how the search index keeps up with hundreds of thousands of tweets per second, and how all the pieces fit together into a coherent architecture. Whether you are preparing for an interview or building a social platform, this chapter gives you a complete blueprint.

23.1 Understanding the Problem

At its core, Twitter lets users do four things:

1. Post a tweet -- a short message (up to 280 characters) optionally with images, videos, or links.
2. Follow other users -- subscribe to see their tweets.
3. View a timeline -- a personalized feed of tweets from people you follow.
4. Search tweets -- find tweets by keyword, hashtag, or user.

Around these four features orbit many secondary concerns: trending topics, notifications, direct messages, retweets, likes, and media storage. We will focus on the core four and touch on the others as we go.

Why Is Twitter Hard to Design?

The challenge is asymmetric fan-out. When a regular user with 200 followers tweets, delivering that tweet to 200 timelines is easy. When a celebrity with 50 million followers tweets, delivering to 50 million timelines in real time is a completely different problem. This asymmetry drives most of the interesting design decisions.

23.2 Requirements

Functional Requirements

#	Requirement	Description
F1	Post a tweet	Users can post text (280 chars), optionally with images or video.
F2	Home timeline	Users see a reverse-chronological feed of tweets from people they follow.
F3	Follow / unfollow	Users can follow or unfollow other users.
F4	Search	Users can search tweets by keyword, hashtag, or username.
F5	Like and retweet	Users can like or retweet a tweet.
F6	Trending topics	Surface the most-discussed topics in real time.
F7	Notifications	Notify users when someone follows, likes, retweets, or mentions them.

Non-Functional Requirements

#	Requirement	Target
NF1	High availability	99.99% uptime. Timeline must always load.
NF2	Low latency	Timeline load under 200 ms (p99).
NF3	Scalability	500 million daily active users, 500 million tweets per day.

NF4	Consistency	Eventual consistency is acceptable. A tweet may take a few seconds to appear in all followers' timelines.
NF5	Durability	No tweet is ever lost once accepted.

Out of Scope

- Direct messages (covered in Chapter 25).
- Ads and monetization.
- Content moderation and spam detection.

23.3 Back-of-the-Envelope Estimation

Traffic

Metric	Value
Daily active users (DAU)	500 million
Tweets per day	500 million
Tweets per second	500M / 86,400 ≈ 5,800 tweets/s
Timeline reads per day	Each user checks timeline ~10 times/day = 5 billion reads/day
Timeline reads per second	5B / 86,400 ≈ 58,000 reads/s
Read-to-write ratio	~10:1

Storage

Item	Size
Tweet text	280 bytes
User ID	8 bytes

Timestamp	8 bytes
Tweet ID	8 bytes
Metadata (likes, retweet count, flags)	50 bytes
Total per tweet (text only)	~350 bytes

Over five years (text only):

```
500M tweets/day x 365 x 5 = 912.5 billion tweets
912.5B x 350 bytes ≈ 319 TB
```

Media Storage

Media type	Percentage of tweets	Average size
Image	20%	500 KB
Video	5%	5 MB
None	75%	0

Daily media storage:

```
Images: 500M x 0.20 x 500 KB = 50 TB/day
Videos: 500M x 0.05 x 5 MB  = 125 TB/day
Total media: ~175 TB/day
```

Over five years: ~319 PB. Media dominates storage costs.

Timeline Fan-out

Average followers per user: ~200. When a regular user tweets:

```
Fan-out writes = 200 timeline entries
200 x 8 bytes (tweet ID) = 1,600 bytes per tweet
```

But a celebrity with 50 million followers:

```
Fan-out writes = 50,000,000 timeline entries
```

At 5,800 tweets per second, if even 1% come from celebrities, that is 58 celebrity tweets per second times 50 million fan-out each -- 2.9 billion writes per second. This is not feasible with pure fan-out on write. We need a hybrid approach.

23.4 High-Level Design

```
┌──────────┐          ┌──────────────┐
│  Client  │──────────►│ Load Balancer │
│(Mobile/Web)│         └──────────────┘
└──────────┘                  │
                ┌─────────────┼──────────────┐
                │             │              │
        ┌───────▼──────┐ ┌────▼────┐  ┌──────▼──────┐
        │ Tweet Service│ │Timeline│  │Search Service│
        │  (Write API) │ │Service │  │             │
        └──────────────┘ └────────┘  └─────────────┘
                │             │              │
        ┌───────▼──────┐      │       ┌──────▼──────┐
        │  Tweet Store │      │       │ Search Index │
        │  (Database)  │      │       │(Elasticsearch)│
        └──────────────┘      │       └─────────────┘
                │             │
        ┌───────▼──────┐ ┌────▼──────────┐
        │ Fan-out      │ │Timeline Cache│
        │ Service      │ │  (Redis)     │
        └──────────────┘ └───────────────┘
                │
        ┌───────▼──────┐     ┌──────────────┐
        │ Notification │     │ Media Service │
        │ Service      │     │ (S3 + CDN)   │
        └──────────────┘     └──────────────┘
```

Request Flow: Posting a Tweet

1. Client sends tweet to Tweet Service through the load balancer.
2. Tweet Service validates the request, assigns a unique tweet ID (using a distributed ID generator like Snowflake), and writes to the Tweet Store.
3. Tweet Service publishes an event to a message queue (Kafka).
4. Fan-out Service consumes the event. It looks up the author's followers and decides on the fan-out strategy:
    - Regular user (under 10,000 followers): fan-out on write. Push the tweet ID into each follower's timeline cache.

    - Celebrity (over 10,000 followers): skip fan-out. The tweet stays in the tweet store and is merged at read time.
5. Media Service processes any attached images or videos asynchronously.
6. Search Service indexes the tweet for full-text search.
7. Notification Service sends push notifications for mentions and followers with notifications enabled.

Request Flow: Reading the Timeline

1. Client requests home timeline from Timeline Service.
2. Timeline Service reads the pre-computed timeline from Redis (list of tweet IDs for that user).
3. For each celebrity the user follows, Timeline Service fetches their latest tweets from the Tweet Store.
4. The two lists are merged and sorted by time.
5. Tweet details (text, author info, like count) are fetched from cache or database.
6. The assembled timeline is returned to the client.

23.5 Deep Dive

23.5.1 Fan-out on Write vs. Fan-out on Read

This is the most critical design decision for Twitter. Let us compare:

Aspect	Fan-out on Write	Fan-out on Read
How it works	When a tweet is posted, push it into every follower's timeline cache.	When a user loads their timeline, pull tweets from all followed users.
Timeline read speed	Very fast -- just read the pre-built list.	Slower -- must query N users' tweet lists and merge.
Tweet post speed	Slower for popular users -- must update millions of caches.	Fast -- just write one record.
Storage	High -- duplicate tweet IDs across millions of timelines.	Low -- tweets stored once.

Best for	Regular users with few followers.	Celebrities with millions of followers.

The Hybrid Approach

Twitter uses a hybrid strategy:

```
┌──────────────────────────────────────────┐
│           Fan-out Decision Engine         │
│                                           │
│  if (author.followers < THRESHOLD):       │
│      fan_out_on_write(tweet, followers)   │
│  else:                                    │
│      mark_as_celebrity_tweet(tweet)        │
└──────────────────────────────────────────┘
```

- Threshold: Typically around 10,000 followers.
- Regular users: their tweets are pushed to followers' timeline caches immediately (fan-out on write).
- Celebrities: their tweets are NOT pushed. Instead, when a user reads their timeline, the system merges the pre-built cache with fresh celebrity tweets (fan-out on read).

This gives us the best of both worlds: fast reads for the 99% case and manageable write load for the 1% of celebrity tweets.

23.5.2 Timeline Service

The timeline is stored in Redis as a sorted set per user:

```
Key:    timeline:{user_id}
Value:  sorted set of (tweet_id, timestamp)
```

Each user's timeline cache holds the most recent 800 tweet IDs. When a new tweet is pushed via fan-out on write, the oldest entry is evicted.

Reading the timeline:

```
1. ZREVRANGE timeline:{user_id} 0 19       -- get top 20 tweet IDs
2. For each celebrity the user follows:
     GET latest tweets from tweets:{celebrity_id}
3. Merge the two lists by timestamp
4. MGET tweet:{id} for each tweet ID        -- fetch full tweet objects
5. Return assembled timeline to client
```

Why Redis? It keeps everything in memory, supports sorted sets natively, and handles hundreds of thousands of operations per second per node. A Redis cluster with sharding by user_id scales horizontally.

23.5.3 Tweet Storage

We need two storage layers:

Primary Database (MySQL / PostgreSQL with sharding):

- Source of truth for all tweets.
- Sharded by tweet_id for even distribution.
- Each shard holds a partition of all tweets.

Cache Layer (Redis / Memcached):

- Hot tweets (recent, viral) are cached.
- Cache-aside pattern: read from cache first, fall back to database.

23.5.4 Media Service

Tweets with images or videos go through a processing pipeline:

```
┌─────────┐     ┌──────────┐     ┌──────────────┐     ┌─────┐
│ Upload  │──►  │  Object  │──►  │  Processing  │──►  │ CDN │
│  API    │     │  Store   │     │  Pipeline    │     │     │
└─────────┘     │  (S3)    │     │              │     └─────┘
                └──────────┘     │ - Resize     │
                                 │ - Compress   │
                                 │ - Thumbnails │
                                 │ - Strip EXIF │
                                 └──────────────┘
```

1. Client uploads the media file to an upload API that returns a media ID.
2. The raw file is stored in object storage (S3).
3. An asynchronous pipeline generates multiple sizes (thumbnail, small, medium, large) and compresses them.
4. Processed files are pushed to a CDN for low-latency global delivery.
5. The tweet references the media ID; clients fetch media directly from the CDN.

23.5.5 Search Index

Twitter needs to index tweets for real-time search. The search architecture:

```
┌──────────────┐      ┌──────────────────┐      ┌──────────────────┐
│  Tweet Event │──────►│  Indexing        │──────►│Elasticsearch     │
│  (Kafka)     │      │  Workers         │      │  Cluster         │
└──────────────┘      └──────────────────┘      └──────────────────┘
                                                        │
                                                        ▼
                                                 ┌──────────────┐
                                                 │ Search API   │
                                                 └──────────────┘
```

- Inverted index: maps each word/hashtag to a list of tweet IDs.
- Near real-time: new tweets are searchable within seconds (Elasticsearch refresh interval).
- Ranking: results ranked by recency, engagement (likes, retweets), and relevance.
- Sharding: the index is sharded by time (recent tweets on faster hardware, older tweets on cheaper storage).

23.5.6 Trending Topics

Trending topics are computed by a streaming analytics pipeline:

```
┌──────────┐      ┌──────────────┐      ┌──────────────────┐
│  Tweets  │──────►│  Stream      │──────►│  Trending        │
│  (Kafka) │      │  Processor   │      │  Topics Cache    │
└──────────┘      │ (Flink/Spark)│      │  (Redis)         │
                  └──────────────┘      └──────────────────┘
```

Algorithm:

1. Extract hashtags and significant phrases from each tweet.
2. Count occurrences in a sliding window (e.g., last 5 minutes, last hour).
3. Apply a velocity formula: topics that spike suddenly rank higher than steadily popular ones.
4. Filter out evergreen terms (common words that always trend).
5. Store the top N trending topics in Redis, refreshed every 30-60 seconds.
6. Segment by geography for localized trends.

23.5.7 Notification Service

When certain events occur (mention, follow, like, retweet), the notification service alerts the user:

```
┌──────────────┐      ┌──────────────┐      ┌──────────────┐
│  Event       │──────▶│ Notification │──────▶│ Push Gateway  │
│  (Kafka)     │      │ Service      │      │ (APNs / FCM)  │
└──────────────┘      └──────────────┘      └──────────────┘
                             │
                             ▼
                      ┌──────────────┐
                      │ Notification  │
                      │ Store (DB)    │
                      └──────────────┘
```

- Priority levels: mentions and DMs are high priority (push immediately); likes on old tweets are low priority (batch).
- Rate limiting: if a tweet goes viral, do not send 10,000 individual like notifications. Aggregate them ("50 people liked your tweet").
- User preferences: respect mute, block, and notification settings.

23.6 Database Schema

Users Table

```sql
CREATE TABLE users (
    user_id       BIGINT PRIMARY KEY,    -- Snowflake ID
    username      VARCHAR(50) UNIQUE NOT NULL,
    display_name  VARCHAR(100),
    email         VARCHAR(255) UNIQUE,
    bio           VARCHAR(280),
    profile_image VARCHAR(500),
    followers_count  INT DEFAULT 0,
    following_count  INT DEFAULT 0,
    is_celebrity  BOOLEAN DEFAULT FALSE, -- fan-out threshold flag
    created_at    TIMESTAMP DEFAULT NOW()
);
```

Tweets Table

```sql
CREATE TABLE tweets (
    tweet_id      BIGINT PRIMARY KEY,    -- Snowflake ID (time-sortable)
    user_id       BIGINT NOT NULL,
    content       VARCHAR(280),
    media_ids     JSON,                  -- array of media references
```

```
    reply_to      BIGINT,                -- NULL if not a reply
    retweet_of    BIGINT,                -- NULL if not a retweet
    like_count    INT DEFAULT 0,
    retweet_count INT DEFAULT 0,
    reply_count   INT DEFAULT 0,
    created_at    TIMESTAMP DEFAULT NOW(),
    INDEX idx_user_time (user_id, created_at DESC)
);
-- Sharded by tweet_id
```

Follows Table

```sql
CREATE TABLE follows (
    follower_id   BIGINT NOT NULL,
    followee_id   BIGINT NOT NULL,
    created_at    TIMESTAMP DEFAULT NOW(),
    PRIMARY KEY (follower_id, followee_id),
    INDEX idx_followee (followee_id, follower_id)
);
-- Sharded by follower_id
```

Likes Table

```sql
CREATE TABLE likes (
    user_id       BIGINT NOT NULL,
    tweet_id      BIGINT NOT NULL,
    created_at    TIMESTAMP DEFAULT NOW(),
    PRIMARY KEY (user_id, tweet_id),
    INDEX idx_tweet (tweet_id)
);
```

Timeline Cache (Redis)

```
Key:    timeline:{user_id}
Type:   Sorted Set
Score:  tweet timestamp (epoch ms)
Member: tweet_id

Max size: 800 entries per user (oldest evicted)
```

23.7 API Design

Post a Tweet

```
POST /api/v1/tweets
Authorization: Bearer {token}

Request Body:
{
    "content": "Hello, world!",
    "media_ids": ["media_123", "media_456"],
    "reply_to": null
}

Response: 201 Created
{
    "tweet_id": "1234567890",
    "content": "Hello, world!",
    "created_at": "2025-01-15T10:30:00Z",
    "media": [
        {"id": "media_123", "url": "https://cdn.twitter.com/..."}
    ]
}
```

Get Home Timeline

```
GET /api/v1/timeline?cursor={last_tweet_id}&limit=20
Authorization: Bearer {token}

Response: 200 OK
{
    "tweets": [
        {
            "tweet_id": "1234567890",
            "user": {"user_id": "42", "username": "alice", ...},
            "content": "Hello, world!",
            "like_count": 15,
            "retweet_count": 3,
            "created_at": "2025-01-15T10:30:00Z"
        },
        ...
    ],
    "next_cursor": "1234567880"
}
```

Follow a User

```
POST /api/v1/users/{user_id}/follow
Authorization: Bearer {token}

Response: 200 OK
{
    "following": true,
    "user_id": "99"
}
```

Search Tweets

```
GET /api/v1/search?q=system+design&type=recent&cursor={}&limit=20
Authorization: Bearer {token}

Response: 200 OK
{
    "tweets": [...],
    "next_cursor": "..."
}
```

23.8 Scaling the System

Tweet Store Sharding

Shard tweets by `tweet_id` using consistent hashing. Since tweet IDs are generated with Snowflake (which embeds a timestamp), we get time-ordered IDs that spread evenly across shards.

Timeline Cache Sharding

Shard Redis by `user_id`. Each user's timeline lives on one Redis node, making reads fast (no cross-shard queries).

Fan-out Workers

The fan-out service is the most write-intensive component. Scale it by:

- Using a Kafka consumer group with many partitions.
- Each partition handles fan-out for a subset of users.

- During peak times (major events), auto-scale the consumer group.

Search Index Scaling

- Time-based sharding: recent tweets on SSDs, older tweets on HDDs.
- Replication: each shard has 2-3 replicas for read throughput.
- Query routing: search queries hit all shards in parallel; results are merged and ranked.

CDN for Media

- Media files are served from CDN edge locations worldwide.
- Upload goes to the origin (S3); CDN pulls on first request and caches.
- Multiple CDN providers for redundancy.

Database Read Replicas

- Each MySQL shard has 3-5 read replicas.
- Timeline Service reads from replicas; writes go to the primary.
- Replication lag (typically under 1 second) is acceptable for timeline reads.

23.9 Trade-offs

Decision	Option A	Option B	Our Choice	Why
Fan-out strategy	Pure write	Pure read	Hybrid	Handles both regular users and celebrities efficiently.
Tweet ID generation	UUID	Snowflake	Snowflake	Time-sortable, 64-bit (compact), no coordination needed.
Timeline storage	Database	Redis	Redis	In-memory speed is essential for timeline latency.

Search engine	Custom inverted index	Elasticsearch	Elasticsearch	Mature, battle-tested, supports near real-time indexing.
Consistency model	Strong	Eventual	Eventual	Users tolerate a few seconds of delay; strong consistency is too expensive at this scale.
Media storage	Own infrastructure	Cloud object storage	Cloud (S3)	Petabyte-scale, 11 nines durability, no operational burden.

23.10 Common Mistakes

1. Using only fan-out on write. This falls apart for celebrity accounts. Always discuss the hybrid approach.
2. Forgetting the celebrity problem entirely. If you do not mention that some users have millions of followers, the interviewer will ask.
3. Storing full tweets in the timeline cache. Store only tweet IDs and timestamps. Fetch full tweet data separately -- this keeps the cache small and avoids data consistency issues.
4. Ignoring media storage. Media is the largest storage cost by orders of magnitude. Mentioning CDN and object storage shows you think about real production systems.
5. Not discussing ID generation. Auto-increment IDs break with sharding. Snowflake IDs solve this elegantly.
6. Treating search as an afterthought. Real-time search is a core feature, not a nice-to-have.

23.11 Best Practices

1. Separate read and write paths. The timeline read path and tweet write path have very different performance profiles. Design them independently.
2. Use event-driven architecture. Kafka decouples the tweet write from fan-out, search indexing, notification, and trending computation. Each consumer processes events at its

own pace.

3. Cache aggressively but invalidate carefully. Timeline caches, tweet caches, and user profile caches all reduce database load. Use TTLs and event-driven invalidation.
4. Design for graceful degradation. If the fan-out service falls behind, timelines still work (they just miss very recent tweets). If search is down, the timeline still loads.
5. Monitor fan-out latency. Track the p99 time from tweet creation to appearance in followers' timelines. This is the key user experience metric.
6. Use cursor-based pagination. Offset-based pagination breaks at Twitter's scale. Use the last tweet ID as a cursor.

23.12 Quick Summary

Twitter's architecture revolves around three key insights: (1) use a hybrid fan-out strategy to handle both regular users and celebrities, (2) pre-compute timelines in Redis for fast reads while merging celebrity tweets at read time, and (3) use event-driven architecture (Kafka) to decouple tweet creation from downstream processing (fan-out, search, notifications, trending). Media storage dominates costs and is handled by object storage plus CDN. Search uses an inverted index (Elasticsearch) with time-based sharding for efficient real-time queries.

23.13 Key Points

- Hybrid fan-out is the heart of the design: fan-out on write for regular users, fan-out on read for celebrities.
- Redis sorted sets store pre-computed timelines (tweet IDs only, not full tweets).
- Snowflake IDs provide time-sortable, globally unique tweet identifiers without coordination.
- Kafka decouples tweet creation from fan-out, indexing, notifications, and trending.
- Celebrity threshold (~10K followers) determines the fan-out strategy per user.
- Media pipeline handles upload, processing (resize, compress, thumbnail), and CDN delivery.
- Elasticsearch provides near real-time full-text search with time-based sharding.
- Trending topics use streaming analytics with sliding windows and velocity detection.
- Eventual consistency is acceptable -- users tolerate a few seconds of delay.

- Storage is dominated by media (hundreds of TB per day for video and images).

23.14 Practice Questions

1. If a celebrity with 100 million followers tweets, and you use pure fan-out on write, how long would it take to update all timelines assuming each write takes 1 microsecond?
2. How would you handle the case where a user unfollows someone? Do you remove the unfollowed user's tweets from the timeline cache?
3. If Redis goes down for a user's timeline, how would you rebuild it?
4. How would you implement "quote tweets" in this design?
5. A breaking news event causes 10x normal tweet volume. Which components are most likely to become bottlenecks, and how would you handle it?

23.15 Exercises

1. Capacity planning: Calculate the number of Redis nodes needed to store timelines for 500 million users, where each timeline holds 800 tweet IDs (8 bytes each). Assume each Redis node has 64 GB of RAM.
2. Fan-out simulation: Write pseudocode for the fan-out decision engine that handles regular users, celebrities, and "protected" accounts (private tweets visible only to approved followers).
3. Trending algorithm: Design a data structure that can efficiently track the top 10 trending hashtags in a sliding 5-minute window, processing 5,000 tweets per second.
4. Schema extension: Extend the database schema to support Twitter threads (a chain of tweets by the same user).
5. Failure scenario: The fan-out service is down for 10 minutes. Design a recovery mechanism that processes the backlog without overwhelming downstream services.

23.16 What Is Next?

In this chapter we tackled the fan-out problem, timeline generation, and real-time search. In the next chapter, Chapter 24: Designing Instagram, we will explore a system with a similar

social graph but a very different content model -- images and photos. You will see how the news feed generation differs when every post is visual, how image processing pipelines work at scale, and how CDN strategies change when media is not optional but the entire product.

Chapter 24: Designing Instagram

A billion people use Instagram every month to share photos, browse their feed, and discover content. Behind every double-tap and every perfectly filtered sunset photo is a system that handles hundreds of millions of image uploads, generates personalized news feeds at massive scale, and delivers photos from CDN edge servers around the world in milliseconds.

In this chapter we will design Instagram from scratch. You will learn how the image upload and processing pipeline works, how news feed generation differs from Twitter's approach, how to shard a social graph, and how to handle the unique challenges that celebrities create in any follow-based system. This is one of the most frequently asked system design questions, and by the end you will have a thorough, production-grade blueprint.

24.1 Understanding the Problem

Instagram's core features are:

1. Upload a photo or video -- with captions, filters, and location tags.
2. News feed -- a personalized feed of posts from people you follow.
3. Follow / unfollow -- subscribe to other users' content.
4. Like and comment -- engage with posts.
5. Stories -- ephemeral content that disappears after 24 hours.
6. Explore -- discover new content based on interests.

We will focus on the first four features and address stories and explore at a high level.

How Is Instagram Different from Twitter?

While both are follow-based social networks, Instagram differs in important ways:

- Every post has media. Twitter posts are text-first; Instagram posts are image/video-first. This means media handling is not optional -- it IS the product.
- Higher media quality. Users expect high-resolution images loaded quickly.

- Less real-time pressure. A few seconds of delay in the news feed is more tolerable than on Twitter, where breaking news matters.
- Smaller post volume. Users post far fewer Instagram posts per day compared to tweets, but each post is much larger in bytes.

24.2 Requirements

Functional Requirements

#	Requirement	Description
F1	Upload photo/video	Users can upload images (up to 10 in a carousel) and short videos with captions.
F2	News feed	Users see posts from people they follow, ranked by relevance and recency.
F3	Follow / unfollow	Users can follow or unfollow other accounts.
F4	Like	Users can like a post.
F5	Comment	Users can comment on a post.
F6	User profile	View a user's posts, follower count, and bio.

Non-Functional Requirements

#	Requirement	Target
NF1	High availability	99.99% uptime. Feed and uploads must always work.
NF2	Low latency	News feed loads in under 300 ms. Images render in under 500 ms.
NF3	Scalability	1 billion monthly active users, 500 million DAU, 100 million photos/day.

NF4	Durability	No uploaded photo is ever lost.
NF5	Consistency	Eventual consistency. A new post may take a few seconds to appear in followers' feeds.

Out of Scope

- Stories and Reels.
- Direct messaging.
- Ads and monetization.
- Content recommendation (Explore page).

24.3 Back-of-the-Envelope Estimation

Traffic

Metric	Value
Daily active users	500 million
Photo uploads per day	100 million
Uploads per second	100M / 86,400 ≈ 1,160 uploads/s
Feed reads per day	500M users x 10 opens = 5 billion/day
Feed reads per second	5B / 86,400 ≈ 58,000 reads/s

Storage

Item	Size
Original photo	2 MB (average)
Processed versions (5 sizes)	500 KB + 200 KB + 100 KB + 50 KB + 20 KB ≈ 870 KB
Total per photo (all versions)	~3 MB

Daily storage:

```
100M photos/day x 3 MB = 300 TB/day
```

Over five years:

```
300 TB/day x 365 x 5 ≈ 547 PB
```

This is enormous. Object storage (S3) with lifecycle policies is essential.

Post Metadata

Item	Size
Post ID	8 bytes
User ID	8 bytes
Caption	500 bytes (average)
Location	50 bytes
Timestamps	16 bytes
Counters (likes, comments)	16 bytes
Media references	100 bytes
Total per post	~700 bytes

Over five years:

```
100M posts/day x 365 x 5 x 700 bytes ≈ 127 TB
```

Metadata is manageable. Media storage dominates by 4,000x.

24.4 High-Level Design

```
┌──────────┐          ┌──────────────┐
│  Client  │──────────►│  API Gateway  │
│(iOS/Android)│        │& Load Balancer│
└──────────┘          └──────────────┘
                              │
          ┌───────────────────┼────────────────────┐
          │                   │                    │
   ┌──────▼──────┐     ┌──────▼──────┐     ┌──────▼──────┐
   │   Post      │     │   Feed      │     │   User      │
   │   Service   │     │   Service   │     │   Service   │
   └─────────────┘     └─────────────┘     └─────────────┘
          │                   │                    │
   ┌──────▼──────┐     ┌──────▼──────┐     ┌──────▼──────┐
   │  Post DB    │     │ Feed Cache  │     │  User DB    │
   │  (MySQL)    │     │  (Redis)    │     │  (MySQL)    │
   └─────────────┘     └─────────────┘     └─────────────┘
          │
   ┌──────▼──────────────────────────────┐
   │        Image Processing Pipeline     │
   │                                      │
   │  Upload ──► S3 ──► Resize/Compress   │
   │                    ──► CDN Push      │
   └──────────────────────────────────────┘
          │
   ┌──────▼──────┐
   │  Message    │ ──► Fan-out Service
   │  Queue      │ ──► Notification Service
   │  (Kafka)    │ ──► Search Indexer
   └─────────────┘
```

Request Flow: Uploading a Photo

1. Client uploads the image to a pre-signed URL (direct to S3), bypassing application servers for large file transfers.
2. Client sends post metadata (caption, location, media references) to Post Service.
3. Post Service validates the request, stores metadata in Post DB, and publishes an event to Kafka.
4. Image Processing Pipeline (triggered by S3 event or Kafka):
    - Generates multiple resolutions (thumbnail: 150x150, small: 320px, medium: 640px, large: 1080px, original).
    - Applies compression (WebP format for supported clients, JPEG fallback).
    - Strips EXIF data for privacy.
    - Stores all versions in S3.
    - Invalidates or pre-warms CDN cache.
5. Fan-out Service pushes the post ID to followers' feed caches.
6. Notification Service notifies relevant users.

Request Flow: Loading the News Feed

1. Client requests feed from Feed Service.
2. Feed Service reads pre-computed feed from Redis (list of post IDs).
3. For celebrities the user follows, fetches their recent posts directly from Post DB.
4. Merges and ranks posts (by relevance score, not just chronological order).
5. Fetches full post metadata and user info from cache or database.
6. Returns the assembled feed. Image URLs point to the CDN.

Why Pre-signed URLs for Upload?

Uploading multi-megabyte images through application servers wastes bandwidth and CPU on machines that should be handling API logic. Pre-signed URLs let clients upload directly to S3:

```
1. Client ──▶ Post Service: "I want to upload a photo"
2. Post Service generates a pre-signed S3 URL (valid for 15 minutes)
3. Post Service ──▶ Client: returns the pre-signed URL
4. Client ──▶ S3: uploads the image directly
5. S3 ──▶ Lambda/Worker: triggers processing pipeline
```

24.5 Deep Dive

24.5.1 Image Processing Pipeline

This is the most Instagram-specific component. Every uploaded image goes through:

```
┌──────────┐    ┌──────────┐    ┌──────────┐    ┌──────────┐    ┌──────┐
│  Raw     │──▶│ Validate │──▶│  Resize  │──▶│ Compress │──▶│Store │
│ Upload   │    │ & Scan   │    │ (5 sizes)│    │(WebP/JPEG)│    │(S3)  │
└──────────┘    └──────────┘    └──────────┘    └──────────┘    └──┬───┘
                                                                    │
                                                                    ▼
                                                              ┌──────────┐
                                                              │ CDN Push │
                                                              └──────────┘
```

Validation: Check file type, size limits (max 30 MB), and scan for malware or prohibited content.

Resize: Generate five versions:

Version	Max dimension	Use case
Thumbnail	150 x 150	Profile grid, notifications
Small	320px wide	Feed on slow connections
Medium	640px wide	Standard feed view
Large	1080px wide	Full-screen view
Original	As uploaded	Zoom, download

Compression: Use WebP (30% smaller than JPEG at same quality) for modern clients, JPEG for older ones. Target quality: 80-85%.

EXIF stripping: Remove GPS coordinates, camera info, and other metadata for user privacy.

CDN push: Pre-warm the CDN by pushing processed images to edge locations in regions where the uploader's followers are concentrated.

24.5.2 News Feed Generation

Instagram's feed is ranked, not purely chronological. The ranking considers:

- Recency: newer posts score higher.
- Relationship: posts from accounts you interact with frequently score higher.
- Interest: posts similar to content you have liked before score higher.
- Engagement velocity: posts getting lots of likes quickly score higher.

The feed is generated in two phases:

Phase 1 -- Candidate generation (offline/near-real-time):

- Fan-out service pushes post IDs to followers' feed caches (same hybrid approach as Twitter).
- Each user's cache holds the last 500 candidate post IDs.

Phase 2 -- Ranking (at read time):

- When the user opens the app, Feed Service fetches the 500 candidates.
- A lightweight ranking model scores each candidate.
- The top N posts are returned, ordered by score.

This two-phase approach separates the expensive fan-out (done once per post) from the personalized ranking (done per feed request).

24.5.3 Database Schema and Sharding

Sharding Strategy

The primary shard key is user_id. This means:

- All of a user's posts are on the same shard.
- A user's profile query hits only one shard.
- The follow graph for a user is on one shard.

The trade-off: feed generation must query multiple shards (one per followed user). This is acceptable because:

- Fan-out pre-computes the feed in Redis (so the read path rarely hits the database).
- When we do need to query multiple shards, we do it in parallel.

Shard Count Estimation

With 500 million DAU and ~700 bytes per post metadata:

```
Posts over 5 years: 100M/day x 365 x 5 = 182.5 billion posts
Metadata: 182.5B x 700 bytes ≈ 127 TB
With indexes: ~250 TB

At 500 GB per shard: 250 TB / 500 GB = 500 shards
```

Add follows, likes, and comments and we need roughly 1,000 shards.

24.5.4 Celebrity Handling

Celebrities (accounts with millions of followers) create the same fan-out problem as in Twitter. Instagram uses a similar hybrid approach:

Regular accounts (under 50K followers):

- Fan-out on write: push post ID to all followers' feed caches.

Celebrity accounts (over 50K followers):

- Fan-out on read: do NOT push to followers' caches. Instead, when a user loads their feed, merge celebrity posts at read time.

Mixed timelines: Most users follow a mix of regular accounts and a few celebrities. The Feed Service:

1. Reads the pre-built feed from Redis (contains regular account posts).
2. Fetches recent posts from each celebrity the user follows.
3. Merges and ranks.

The celebrity threshold for Instagram can be higher than Twitter (50K vs 10K) because:

- Instagram has far fewer posts per user per day than tweets.
- Feed latency tolerance is higher (300 ms vs 200 ms).

24.5.5 CDN Strategy

With 100 million photos per day, CDN is critical:

Multi-tier caching:

```
┌─────────┐    ┌──────────────┐    ┌──────────────┐    ┌─────────┐
│ Client  │──▶│  CDN Edge    │──▶│ CDN Regional │──▶│   S3    │
│         │    │  (closest)   │    │  (mid-tier)  │    │(origin)│
└─────────┘    └──────────────┘    └──────────────┘    └─────────┘
```

- Edge servers: in 100+ cities worldwide. Serve ~95% of image requests.
- Regional servers: in 10-15 regions. Serve cache misses from edge.
- Origin (S3): only hit for very cold content.

Smart pre-warming: When a post is uploaded, predict where followers are located and push the image to those edge servers proactively, before anyone requests it.

Adaptive image serving: Serve different image sizes based on:

- Client screen resolution (1x, 2x, 3x).
- Network speed (smaller images on slow connections).
- Format support (WebP vs JPEG).

24.6 Database Schema

Users Table

```sql
CREATE TABLE users (
    user_id         BIGINT PRIMARY KEY,
    username        VARCHAR(30) UNIQUE NOT NULL,
    display_name    VARCHAR(100),
    email           VARCHAR(255) UNIQUE,
    phone           VARCHAR(20),
    bio             VARCHAR(300),
    profile_pic_url VARCHAR(500),
    is_private      BOOLEAN DEFAULT FALSE,
```

```
    is_celebrity    BOOLEAN DEFAULT FALSE,
    followers_count INT DEFAULT 0,
    following_count INT DEFAULT 0,
    posts_count     INT DEFAULT 0,
    created_at      TIMESTAMP DEFAULT NOW()
);
-- Sharded by user_id
```

Posts Table

```sql
CREATE TABLE posts (
    post_id         BIGINT PRIMARY KEY,     -- Snowflake ID
    user_id         BIGINT NOT NULL,
    caption         VARCHAR(2200),
    location_id     BIGINT,
    like_count      INT DEFAULT 0,
    comment_count   INT DEFAULT 0,
    created_at      TIMESTAMP DEFAULT NOW(),
    INDEX idx_user_time (user_id, created_at DESC)
);
-- Sharded by user_id
```

Post Media Table

```sql
CREATE TABLE post_media (
    media_id        BIGINT PRIMARY KEY,
    post_id         BIGINT NOT NULL,
    media_type      ENUM('IMAGE', 'VIDEO'),
    s3_key          VARCHAR(500) NOT NULL,
    width           INT,
    height          INT,
    display_order   TINYINT NOT NULL,      -- for carousel ordering
    INDEX idx_post (post_id)
);
-- Sharded by user_id (co-located with posts)
```

Follows Table

```sql
CREATE TABLE follows (
    follower_id     BIGINT NOT NULL,
    followee_id     BIGINT NOT NULL,
    created_at      TIMESTAMP DEFAULT NOW(),
```

```
    PRIMARY KEY (follower_id, followee_id),
    INDEX idx_followee (followee_id)
);
-- Sharded by follower_id
```

Likes Table

```sql
CREATE TABLE likes (
    user_id         BIGINT NOT NULL,
    post_id         BIGINT NOT NULL,
    created_at      TIMESTAMP DEFAULT NOW(),
    PRIMARY KEY (user_id, post_id),
    INDEX idx_post (post_id)
);
-- Sharded by post_id
```

Comments Table

```sql
CREATE TABLE comments (
    comment_id      BIGINT PRIMARY KEY,
    post_id         BIGINT NOT NULL,
    user_id         BIGINT NOT NULL,
    content         VARCHAR(2200),
    parent_id       BIGINT,                -- for nested replies
    created_at      TIMESTAMP DEFAULT NOW(),
    INDEX idx_post_time (post_id, created_at)
);
-- Sharded by post_id
```

Feed Cache (Redis)

```
Key:    feed:{user_id}
Type:   Sorted Set
Score:  post timestamp (epoch ms)
Member: post_id

Max size: 500 entries per user
TTL: 7 days (rebuild from database if expired)
```

24.7 API Design

Upload a Photo

Step 1: Get upload URL

```
POST /api/v1/media/upload-url
Authorization: Bearer {token}

Request Body:
{
    "content_type": "image/jpeg",
    "file_size": 2048000
}

Response: 200 OK
{
    "upload_url": "https://s3.amazonaws.com/instagram-uploads/...",
    "media_id": "media_789",
    "expires_at": "2025-01-15T11:00:00Z"
}
```

Step 2: Upload to S3 (client uploads directly)

Step 3: Create post

```
POST /api/v1/posts
Authorization: Bearer {token}

Request Body:
{
    "caption": "Beautiful sunset #photography",
    "media_ids": ["media_789"],
    "location": {"lat": 34.0522, "lng": -118.2437, "name": "Los Angeles"}
}

Response: 201 Created
{
    "post_id": "9876543210",
    "user": {"user_id": "42", "username": "alice"},
    "caption": "Beautiful sunset #photography",
    "media": [
        {
            "media_id": "media_789",
            "url": "https://cdn.instagram.com/p/media_789/large.webp",
            "width": 1080,
            "height": 1350
        }
    ],
    "created_at": "2025-01-15T10:30:00Z"
```

```
}
```

Get News Feed

```
GET /api/v1/feed?cursor={last_post_id}&limit=10
Authorization: Bearer {token}

Response: 200 OK
{
    "posts": [
        {
            "post_id": "9876543210",
            "user": {"user_id": "42", "username": "alice", "profile_pic": "..."},
            "caption": "Beautiful sunset #photography",
            "media": [...],
            "like_count": 342,
            "comment_count": 15,
            "liked_by_viewer": false,
            "created_at": "2025-01-15T10:30:00Z"
        }
    ],
    "next_cursor": "9876543200"
}
```

Like a Post

```
POST /api/v1/posts/{post_id}/like
Authorization: Bearer {token}

Response: 200 OK
{
    "liked": true,
    "like_count": 343
}
```

Comment on a Post

```
POST /api/v1/posts/{post_id}/comments
Authorization: Bearer {token}

Request Body:
{
    "content": "Stunning shot!",
    "parent_id": null
```

```
}

Response: 201 Created
{
    "comment_id": "comment_123",
    "content": "Stunning shot!",
    "user": {"user_id": "99", "username": "bob"},
    "created_at": "2025-01-15T10:35:00Z"
}
```

24.8 Scaling the System

Image Storage

- Object storage (S3) with intelligent tiering:
  - Hot tier: images less than 30 days old (frequently accessed).
  - Warm tier: 30-90 days old.
  - Cold tier: over 90 days old (rarely accessed, cheaper storage).
- Lifecycle policies move images between tiers automatically.

Database Sharding

- Shard by user_id using consistent hashing.
- Co-locate related tables (users, posts, post_media) on the same shard.
- Follows table is sharded by follower_id for efficient "who do I follow?" queries.
- Likes and comments are sharded by post_id for efficient "show me likes for this post" queries.

Feed Cache Scaling

- Redis cluster sharded by user_id.
- Each node handles ~1 million users' feeds.
- Total: 500 Redis nodes for 500 million DAU.
- Replication factor: 3 (one primary, two replicas).

Auto-Scaling

- Upload service: scale based on upload queue depth.

- Image processing: scale based on processing queue length. Use spot instances for cost savings.
- Feed service: scale based on request rate (predictable daily patterns -- peak at morning and evening).

Geographic Distribution

- Deploy application servers in multiple regions (US, Europe, Asia).
- Database primaries in one region; read replicas in all regions.
- CDN handles image delivery globally.
- Users are routed to the nearest region.

24.9 Trade-offs

Decision	Option A	Option B	Our Choice	Why
Upload path	Through app server	Pre-signed URL to S3	Pre-signed URL	Avoids wasting app server bandwidth on large files.
Image format	JPEG only	WebP + JPEG fallback	WebP + fallback	30% smaller files, faster loading, with backward compatibility.
Feed ranking	Chronological	ML-ranked	ML-ranked	Increases engagement; chronological option as user setting.
Feed pre-computation	Compute on read	Fan-out on write	Hybrid	Balances write cost (celebrities) with read speed (regular users).
Sharding key	post_id	user_id	user_id	Co-locates user's data; profile page hits one shard.

Like counts	Exact (DB query)	Approximate (counter cache)	Approximate	Exact counts at scale require expensive distributed counting.

24.10 Common Mistakes

1. Routing image uploads through application servers. This wastes bandwidth and creates bottlenecks. Use pre-signed URLs for direct S3 upload.
2. Storing only one image size. Serving a 5 MB original to a mobile device on 3G is a terrible user experience. Always generate multiple sizes.
3. Forgetting about image processing latency. The user should see their post immediately, even before all resized versions are ready. Show the original or a client-side preview while processing happens asynchronously.
4. Ignoring CDN costs. At 100 million images per day, CDN bandwidth is a significant cost. Discuss image compression, format optimization, and cache hit rates.
5. Using a single global database. At Instagram's scale, you need sharding. Discuss the shard key choice and its implications.
6. Not mentioning privacy. EXIF stripping removes GPS coordinates from uploaded photos. This is a real production concern.

24.11 Best Practices

1. Separate the upload path from the metadata path. Upload images directly to object storage; send metadata to application servers. This keeps the two concerns decoupled and independently scalable.
2. Generate multiple image sizes asynchronously. Do not block the upload response on image processing. Return success immediately and process in the background.
3. Use content-addressable storage for deduplication. Hash the image content and use the hash as the S3 key. If two users upload the same image, store it only once.
4. Implement progressive image loading. Load a tiny blurred placeholder first, then swap in the full image. This creates a perception of speed.

5. Cache user profiles aggressively. Every feed item shows the poster's username and profile picture. Cache these in Redis to avoid millions of database lookups per second.
6. Use cursor-based pagination for feeds. Never use OFFSET/LIMIT at scale -- it becomes slower as the offset increases.

24.12 Quick Summary

Instagram's architecture centers on three pillars: (1) a robust image processing pipeline that generates multiple sizes, compresses into modern formats, strips metadata, and distributes through a multi-tier CDN; (2) a hybrid fan-out strategy for news feed generation that pre-computes feeds for regular users while merging celebrity posts at read time; and (3) user_id-based sharding that co-locates each user's data for efficient profile and post queries. Pre-signed URLs enable direct-to-S3 uploads, keeping application servers lean. The feed uses ML-based ranking to surface the most relevant content.

24.13 Key Points

- Pre-signed URLs let clients upload images directly to S3, bypassing application servers.
- Image processing pipeline generates five sizes, compresses to WebP, and strips EXIF data asynchronously.
- Multi-tier CDN (edge, regional, origin) serves 95%+ of image requests from cache.
- Hybrid fan-out handles both regular and celebrity accounts for feed generation.
- ML-based feed ranking considers recency, relationship strength, interest, and engagement velocity.
- Sharding by user_id co-locates user profiles, posts, and media on the same shard.
- Media storage dominates costs at ~300 TB/day, requiring intelligent storage tiering.
- Approximate like counts (cached counters) are used instead of exact database counts at scale.
- Progressive image loading with blurred placeholders improves perceived performance.
- Content-addressable storage deduplicates identical images across users.

24.14 Practice Questions

1. If Instagram processes 100 million images per day and each image takes 5 seconds to process, how many processing workers do you need?
2. How would you handle the case where a user uploads a carousel of 10 images? Should all images be processed before the post appears in feeds?
3. A user with 10 million followers uploads a photo. Walk through the exact sequence of events from upload to the photo appearing in a follower's feed.
4. How would you implement a "like" button that feels instant to the user while being durable?
5. If the CDN cache hit rate drops from 95% to 80%, what happens to the origin servers, and how would you diagnose and fix it?

24.15 Exercises

1. Storage cost estimation: Calculate the monthly S3 storage cost for Instagram assuming 300 TB/day of new images, a 3-tier storage policy, and current S3 pricing.
2. CDN bandwidth: If the average user scrolls through 50 images per feed session (640px wide, WebP, ~80 KB each) and there are 5 billion feed sessions per day, calculate the daily CDN egress bandwidth.
3. Schema design: Extend the database schema to support Instagram Stories (ephemeral posts that expire after 24 hours). Consider how this affects storage and cleanup.
4. Cache warming: Design an algorithm that predicts which CDN edge servers should be pre-warmed when a celebrity uploads a photo, based on their follower distribution.
5. Deduplication: Design a system to detect and deduplicate near-identical images (not just exact matches) using perceptual hashing.

24.16 What Is Next?

In this chapter we designed a media-heavy social platform with image processing, CDN delivery, and ranked feeds. In the next chapter, Chapter 25: Designing a Chat System, we shift to a completely different paradigm: real-time, bidirectional communication. You will learn about WebSockets, message ordering guarantees, offline delivery, and how to build presence indicators -- challenges that require a fundamentally different architecture from the feed-based

systems we have designed so far.

Chapter 25: Designing a Chat System (WhatsApp / Messenger)

Billions of messages fly across chat systems every day. When you send "on my way" to a friend, you expect it to arrive instantly, in order, and exactly once. Behind that simple interaction lies a sophisticated system of persistent connections, message queues, ordering guarantees, and offline delivery mechanisms.

In this chapter we will design a chat system like WhatsApp or Facebook Messenger from scratch. You will learn why WebSockets are essential for real-time messaging, how to guarantee message ordering in a distributed system, how group chat fan-out works, how offline delivery ensures no message is ever lost, and how presence indicators ("online", "last seen") function at scale. This is one of the most rewarding system design exercises because it touches real-time communication, distributed systems, and encryption all in one problem.

25.1 Understanding the Problem

A chat system needs to support:

1. One-to-one (1:1) messaging -- send a text, image, or video to another user.
2. Group messaging -- send a message to a group of users (up to hundreds or thousands of members).
3. Online/offline status -- show whether a user is currently online and when they were last seen.
4. Read receipts -- indicate when a message has been delivered and read.
5. Push notifications -- alert users of new messages when the app is in the background.

What Makes Chat Systems Unique?

Unlike the feed-based systems in previous chapters, chat requires real-time, bidirectional communication. HTTP's request-response model does not work well here because the server

needs to push messages to clients as they arrive, without the client constantly polling. This drives the fundamental architecture choice: persistent connections (WebSockets or long polling).

Another unique challenge is ordering. In a feed, a post appearing a few seconds late is fine. In a chat, if message B arrives before message A, the conversation becomes confusing. Ordering guarantees are critical.

25.2 Requirements

Functional Requirements

#	Requirement	Description
F1	1:1 chat	Send text, image, and video messages between two users.
F2	Group chat	Groups of up to 500 members. Any member can send messages.
F3	Online status	Show whether a user is online, and if not, their "last seen" time.
F4	Read receipts	Show sent, delivered, and read indicators for each message.
F5	Push notifications	Notify users of new messages when the app is in the background.
F6	Message history	Users can scroll back through past messages.

Non-Functional Requirements

#	Requirement	Target
NF1	Low latency	Messages delivered in under 100 ms when both users are online.
NF2	Reliability	No message is ever lost. Exactly-once delivery semantics.

NF3	Ordering	Messages in a conversation appear in the order they were sent.
NF4	Scalability	1 billion connected users, 100 billion messages per day.
NF5	Availability	99.99% uptime. Chat must always work.

Out of Scope

- Voice and video calls.
- Stories and status updates.
- End-to-end encryption implementation details (we will discuss the concept).
- Payment features.

25.3 Back-of-the-Envelope Estimation

Traffic

Metric	Value
Connected users (concurrent)	500 million
Messages per day	100 billion
Messages per second	100B / 86,400 ≈ 1.16 million msg/s
Average message size	200 bytes (text)
Messages with media	~10%

Storage

Text messages over five years:

```
100B msg/day x 365 x 5 = 182.5 trillion messages
182.5T x 200 bytes ≈ 36.5 PB (text only)
```

Media messages (10% of total):

```
10B media msg/day x average 200 KB = 2 PB/day
Over 5 years: ~3,650 PB = 3.65 EB
```

Connection Management

Each connected user maintains one WebSocket connection. With 500 million concurrent connections:

```
Each connection uses ~10 KB of memory
500M x 10 KB = 5 TB of RAM just for connections
At 100K connections per server: 5,000 chat servers
```

Bandwidth

```
1.16M msg/s x 200 bytes = 232 MB/s (text only)
Plus media: roughly 10x = ~2.3 GB/s total
```

25.4 High-Level Design

```
┌──────────┐
│ Client A │◄═══ WebSocket ═══►│ Chat      │
└──────────┘                   │ Server    │
                               │ Cluster   │
┌──────────┐                   │           │
│ Client B │◄═══ WebSocket ═══►│ (5,000    │
└──────────┘                   │ servers)  │
                               └──────┬────┘
                                      │
                  ┌───────────────────┼──────────────────┐
                  │                   │                  │
            ┌─────▼─────┐       ┌─────▼──────┐     ┌─────▼─────┐
            │ Message   │       │ Session/   │     │ Presence  │
            │ Queue     │       │ Connection │     │ Service   │
            │ (Kafka)   │       │ Registry   │     │           │
            └─────┬─────┘       │ (Redis)    │     └─────┬─────┘
                  │             └────────────┘           │
            ┌─────▼─────┐                          ┌─────▼─────┐
```

```
              │  Message    │                              │  Presence  │
              │  Store      │                              │  Cache     │
              │ (Cassandra) │                              │  (Redis)   │
              └─────────────┘                              └────────────┘
                     │
              ┌──────▼──────┐            ┌─────────────────┐
              │  Push       │            │  Media Service  │
              │ Notification│            │  (S3 + CDN)     │
              │  Service    │            └─────────────────┘
              └─────────────┘
```

WebSocket Connection Flow

1. Client opens the app and initiates a WebSocket handshake with a chat server (via load balancer).
2. The load balancer assigns the client to a specific chat server using sticky routing (by user ID).
3. The chat server registers the connection in the Session Registry (Redis): `user_id -> chat_server_id`.
4. The WebSocket connection stays open for the duration of the session.
5. If the connection drops, the client reconnects (possibly to a different server), and the registry is updated.

Message Flow: 1:1 Chat

```
Alice sends "Hello" to Bob:

1. Alice ──WebSocket──▶ Chat Server A
2. Chat Server A:
   a. Validates the message
   b. Generates a message ID (Snowflake)
   c. Stores the message in Message Store (Cassandra)
   d. Looks up Bob's connection in Session Registry
3. IF Bob is online:
   Chat Server A ──▶ Chat Server B (where Bob is connected)
   Chat Server B ──WebSocket──▶ Bob
4. IF Bob is offline:
   Chat Server A ──▶ Push Notification Service ──▶ Bob's phone
5. Chat Server A ──WebSocket──▶ Alice: "message delivered" ✓
```

Message Flow: Group Chat

```
Alice sends "Meeting at 3pm" to a group of 200 members:

1. Alice ──WebSocket──▶ Chat Server A
2. Chat Server A:
   a. Validates the message
   b. Stores in Message Store
   c. Looks up all 200 group members' connections
3. For each online member:
   Route message to their chat server
4. For each offline member:
   Send push notification
```

25.5 Deep Dive

25.5.1 WebSocket vs. Long Polling

Aspect	WebSocket	Long Polling
Connection	Persistent, full-duplex	Repeated HTTP requests
Latency	Very low (sub-100 ms)	Higher (HTTP overhead per request)
Server resources	One connection per user	New connection per poll cycle
Firewall/proxy support	Sometimes blocked	Works everywhere (standard HTTP)
Scalability	More efficient at scale	Wastes bandwidth with empty responses
Complexity	More complex (connection management, reconnection)	Simpler to implement

Our choice: WebSocket with long polling as a fallback for clients behind restrictive firewalls.

Why WebSocket wins: At 500 million concurrent users, long polling would create enormous overhead from constantly opening and closing HTTP connections. WebSocket keeps a single TCP connection open, enabling instant message delivery in both directions.

Fallback strategy: If a WebSocket connection fails after three attempts, fall back to long polling. The client polls every 3-5 seconds, and the server holds the request open for up to 30 seconds if no new messages are available (reducing unnecessary round trips).

25.5.2 Chat Server Architecture

Each chat server handles ~100,000 concurrent WebSocket connections. The server:

- Maintains an in-memory map of `connection_id -> user_id`.
- Listens for incoming messages from connected clients.
- Publishes messages to the message queue (Kafka) for persistence and fan-out.
- Receives messages from other chat servers (for cross-server delivery) via an internal messaging layer.

Cross-server message routing:

```
┌──────────────┐                    ┌──────────────┐
│ Chat Server A │                    │ Chat Server B │
│              │                    │              │
│ Alice ═══╗   │   Internal Bus     │  ╔═══ Bob    │
│          ╚═══│ ◄═══════════════► │  ╚═══        │
│              │    (Redis Pub/Sub  │              │
│              │     or gRPC)       │              │
└──────────────┘                    └──────────────┘
```

Options for cross-server communication:

1. Redis Pub/Sub: Each chat server subscribes to a channel named after itself. To send a message to a user on Server B, Server A publishes to Server B's channel.
2. Direct gRPC: Server A calls Server B directly. Faster but requires service discovery.
3. Message queue (Kafka): Each server consumes from a partition. More durable but slightly higher latency.

We use Redis Pub/Sub for its simplicity and low latency, with Kafka as the durable backing store.

25.5.3 Message Storage

Chat messages have a unique access pattern:

- Most reads are for recent messages (the latest 50 messages in a conversation).
- Historical messages are read infrequently (scrolling back).
- Writes are heavy (1.16 million messages per second).
- Messages are rarely updated or deleted.

This makes Cassandra an excellent choice:

```sql
CREATE TABLE messages (
    conversation_id  UUID,
    message_id       TIMEUUID,     -- time-sortable unique ID
    sender_id        BIGINT,
    content          TEXT,
    media_url        TEXT,
    message_type     TEXT,          -- 'text', 'image', 'video'
    status           TEXT,          -- 'sent', 'delivered', 'read'
    created_at       TIMESTAMP,
    PRIMARY KEY (conversation_id, message_id)
) WITH CLUSTERING ORDER BY (message_id DESC);
```

Why Cassandra?

- Optimized for write-heavy workloads (LSM tree storage engine).
- Partition by `conversation_id`: all messages in a conversation are co-located.
- Clustering by `message_id` (time-sorted): reading the latest N messages is a simple range scan.
- Linear horizontal scalability.
- Tunable consistency (QUORUM for writes, ONE for reads is a good balance).

25.5.4 Message Ordering

Ordering is crucial in chat. We need to guarantee that messages in a conversation appear in the order they were sent.

Challenge: In a distributed system, clocks are not perfectly synchronized. If Alice sends message A at 10:00:00.001 and Bob sends message B at 10:00:00.002 (on a different server), can we guarantee A appears before B?

Solution: Per-conversation sequence numbers

```
Each conversation maintains a monotonically increasing sequence number.

When a message arrives:
1. Acquire a lock on the conversation (Redis distributed lock or DB row lock)
2. Increment the sequence number
3. Assign the sequence number to the message
4. Release the lock
5. Store the message with the sequence number

Clients display messages ordered by sequence number, not by timestamp.
```

Optimization for 1:1 chats: Since only two people can send messages, we can use a simpler approach:

- Each sender maintains their own local sequence counter.

- The message includes `(sender_id, sender_seq_num)`.
- The server assigns a global sequence number using an atomic counter in Redis.

For group chats: The per-conversation lock approach works because the throughput per conversation is bounded (humans can only type so fast).

25.5.5 Offline Message Delivery

When a user is offline, messages must be stored and delivered when they come back online:

```
1. Alice sends message to Bob (offline)
2. Chat Server stores message in Cassandra with status = 'sent'
3. Chat Server sends push notification to Bob
4. Bob opens the app hours later
5. Bob's client connects via WebSocket
6. Client sends "sync request" with last_received_message_id
7. Chat Server queries Cassandra:
   SELECT * FROM messages
   WHERE conversation_id = ?
   AND message_id > last_received_message_id
   ORDER BY message_id ASC
8. Server streams pending messages to Bob over WebSocket
9. For each message, server updates status to 'delivered'
10. Server notifies Alice: "message delivered ✓✓"
```

Handling long offline periods: If a user has been offline for days and has thousands of pending messages across many conversations:

- Fetch conversations with unread messages first (metadata query).
- Load messages per conversation as the user opens each one.
- Do not dump all pending messages at once (would overwhelm the client and the connection).

25.5.6 Group Chat Fan-out

Group messaging introduces a fan-out challenge similar to Twitter, but with key differences:

- Group sizes are smaller (max 500 members vs millions of followers).
- Every message must be delivered (no "eventual consistency" for chat).
- Ordering within the group must be consistent for all members.

Fan-out approach for groups:

```
Message arrives for group G (200 members):

1. Store message in messages table (partition: group_conversation_id)
```

```
2. Look up all 200 members in group_members table
3. For each member:
   a. If online: route to their chat server
   b. If offline: increment unread counter, send push notification
4. Batch the routing: group members by chat server to minimize cross-server calls

Optimization:
- For small groups (< 50): fan-out synchronously
- For large groups (50-500): fan-out asynchronously via Kafka
```

Why not fan-out on write (like Twitter)? In Twitter, fan-out on write means copying tweet IDs to followers' timeline caches. In chat, we do not copy messages -- we just route them. The message lives in one place (the conversation partition in Cassandra), and all group members read from that same partition. The "fan-out" is really just notification routing, not data duplication.

25.5.7 End-to-End Encryption Overview

Modern chat systems use end-to-end encryption (E2EE) so that even the server cannot read messages:

```
┌──────────┐                      ┌──────────┐
│  Alice   │                      │   Bob    │
│          │                      │          │
│ Plaintext │    ┌──────────┐     │Plaintext │
│ "Hello"  │──►│  Server  │──►│ "Hello"  │
│          │    │          │     │          │
│ Encrypted │    │ Encrypted│     │Encrypted │
│ with Bob's│    │ cannot   │     │decrypted │
│ public key│    │ read it  │     │with Bob's│
│          │    └──────────┘     │priv. key │
└──────────┘                      └──────────┘
```

Key concepts:

- Each user has a public/private key pair generated on their device.
- Public keys are exchanged via the server (key distribution).
- Messages are encrypted with the recipient's public key before leaving the sender's device.
- The server stores and forwards encrypted blobs -- it never sees plaintext.
- Group chats use a shared group key, distributed via pairwise encryption.

Impact on system design:

- The server cannot index or search message content.
- The server cannot filter spam or malware from messages.

- Message storage is opaque (encrypted blobs).
- Key management adds complexity (key rotation, multi-device sync, key verification).

We will not implement E2EE in this design, but mentioning it in an interview shows awareness of privacy requirements.

25.5.8 Presence Service

The presence service tracks which users are online and when they were last active:

```
┌──────────────┐     ┌────────────────┐     ┌────────────────┐
│ Chat Server  │────►│  Presence      │────►│   Redis        │
│ (heartbeat)  │     │  Service       │     │   Cache        │
└──────────────┘     └────────────────┘     └────────────────┘
```

How it works:

1. When a user connects via WebSocket, the presence service sets their status to "online" in Redis:

```
SET presence:{user_id} "online" EX 60
```

2. The chat server sends a heartbeat every 30 seconds to refresh the TTL.
3. If no heartbeat arrives for 60 seconds, Redis expires the key, and the user is considered offline.
4. The "last seen" timestamp is updated when the key expires.

Challenges at scale:

- 500 million users checking friends' online status would create enormous read traffic.
- Solution: Subscribe to presence updates only for users in the current chat. Do not poll for all contacts.
- Use Redis Pub/Sub: when a user's status changes, publish to a channel. Only chat servers with interested clients subscribe.

Reducing noise:

- Do not broadcast every micro-disconnect (e.g., switching from WiFi to cellular).
- Use a grace period: if a user disconnects and reconnects within 30 seconds, do not change their status.

25.6 Database Schema

Conversations Table

```sql
-- Cassandra
CREATE TABLE conversations (
    conversation_id    UUID PRIMARY KEY,
    conversation_type  TEXT,           -- '1:1' or 'group'
    name               TEXT,           -- group name (null for 1:1)
    created_by         BIGINT,
    created_at         TIMESTAMP,
    last_message_at    TIMESTAMP,
    last_message_preview TEXT
);
```

Conversation Members Table

```sql
-- Cassandra
CREATE TABLE conversation_members (
    conversation_id    UUID,
    user_id            BIGINT,
    role               TEXT,           -- 'admin', 'member'
    joined_at          TIMESTAMP,
    unread_count       INT,
    last_read_message  TIMEUUID,
    PRIMARY KEY (conversation_id, user_id)
);

-- Reverse index: find all conversations for a user
CREATE TABLE user_conversations (
    user_id            BIGINT,
    last_message_at    TIMESTAMP,
    conversation_id    UUID,
    PRIMARY KEY (user_id, last_message_at)
) WITH CLUSTERING ORDER BY (last_message_at DESC);
```

Messages Table

```sql
-- Cassandra
CREATE TABLE messages (
    conversation_id    UUID,
    message_id         TIMEUUID,
    sender_id          BIGINT,
```

```
    content            TEXT,
    media_url          TEXT,
    media_type         TEXT,
    reply_to           TIMEUUID,
    sequence_num       BIGINT,
    status             TEXT,           -- 'sent', 'delivered', 'read'
    created_at         TIMESTAMP,
    PRIMARY KEY (conversation_id, message_id)
) WITH CLUSTERING ORDER BY (message_id DESC);
```

Session Registry (Redis)

```
Key:    session:{user_id}
Value:  chat_server_id
TTL:    None (managed by connect/disconnect events)
```

Presence Cache (Redis)

```
Key:    presence:{user_id}
Value:  {"status": "online", "last_seen": "2025-01-15T10:30:00Z"}
TTL:    60 seconds (refreshed by heartbeat)
```

Unread Counters (Redis)

```
Key:    unread:{user_id}:{conversation_id}
Value:  integer count
```

25.7 API Design

Send a Message

```
WebSocket Frame (client -> server):
{
    "action": "send_message",
    "conversation_id": "conv_123",
```

```
    "content": "Hello, Bob!",
    "media_id": null,
    "reply_to": null,
    "client_message_id": "client_uuid_456"  -- for idempotency
}

WebSocket Frame (server -> sender, acknowledgment):
{
    "action": "message_ack",
    "client_message_id": "client_uuid_456",
    "message_id": "msg_789",
    "status": "sent",
    "timestamp": "2025-01-15T10:30:00.123Z"
}

WebSocket Frame (server -> recipient):
{
    "action": "new_message",
    "conversation_id": "conv_123",
    "message_id": "msg_789",
    "sender_id": "42",
    "content": "Hello, Bob!",
    "timestamp": "2025-01-15T10:30:00.123Z"
}
```

Get Message History (REST API for initial load)

```
GET /api/v1/conversations/{conversation_id}/messages?before={message_id}&limit=50
Authorization: Bearer {token}

Response: 200 OK
{
    "messages": [
        {
            "message_id": "msg_789",
            "sender_id": "42",
            "content": "Hello, Bob!",
            "status": "read",
            "created_at": "2025-01-15T10:30:00Z"
        },
        ...
    ],
    "has_more": true
}
```

Mark Messages as Read

```
WebSocket Frame (client -> server):
{
    "action": "mark_read",
    "conversation_id": "conv_123",
    "last_read_message_id": "msg_789"
}
```

Get Online Status

```
GET /api/v1/users/{user_id}/presence
Authorization: Bearer {token}

Response: 200 OK
{
    "user_id": "99",
    "status": "offline",
    "last_seen": "2025-01-15T09:45:00Z"
}
```

Create a Group

```
POST /api/v1/conversations
Authorization: Bearer {token}

Request Body:
{
    "type": "group",
    "name": "Project Team",
    "member_ids": ["42", "99", "155", "201"]
}

Response: 201 Created
{
    "conversation_id": "conv_456",
    "type": "group",
    "name": "Project Team",
    "members": [...]
}
```

25.8 Scaling the System

Chat Server Scaling

- Horizontal scaling: add more chat servers as concurrent users grow.
- Sticky sessions: route each user to the same server via consistent hashing on user_id.
- Graceful drain: before shutting down a server for maintenance, migrate connections to other servers.
- Connection limits: cap each server at 100K connections to prevent memory exhaustion.

Message Store (Cassandra) Scaling

- Partition key: `conversation_id`. All messages in a conversation are on the same partition.
- Partition size monitoring: very active group chats could create hot partitions. If a conversation exceeds a partition size threshold, archive older messages to cold storage.
- Replication factor: 3 (across different racks/availability zones).
- Compaction strategy: leveled compaction for read-heavy access patterns.

Session Registry Scaling

- Redis cluster with consistent hashing by user_id.
- With 500 million users: each entry is ~50 bytes, total ~25 GB. Fits in a small Redis cluster.

Message Queue (Kafka) Scaling

- Partition by conversation_id to maintain ordering within a conversation.
- Number of partitions: start with 1,000, scale as needed.
- Consumer groups: one for persistence, one for push notifications, one for analytics.

Push Notification Scaling

- Push notifications go through APNs (Apple) and FCM (Google).
- Both have rate limits. Use a queue to smooth out bursts.
- Batch notifications: if a user receives 50 messages while offline, send one summary notification, not 50.

25.9 Trade-offs

Decision	Option A	Option B	Our Choice	Why
Real-time protocol	Long polling	WebSocket	WebSocket	Lower latency, less overhead at scale. Long polling as fallback.
Message store	MySQL (sharded)	Cassandra	Cassandra	Write-optimized, excellent for time-series data, linear scaling.
Cross-server routing	Redis Pub/Sub	gRPC direct	Redis Pub/Sub	Simpler, decoupled. gRPC is faster but tightly couples servers.
Message ordering	Timestamps	Sequence numbers	Sequence numbers	Clock skew makes timestamps unreliable across distributed servers.
Group fan-out	Copy message to each member	Shared partition, route notifications	Shared partition	Avoids data duplication; routing is lightweight.
Presence protocol	Poll-based	Heartbeat + Pub/Sub	Heartbeat + Pub/Sub	Efficient, low overhead, handles scale well.
Encryption	Server-side	End-to-end	E2EE (concept)	Users expect privacy. Server cannot read messages.

25.10 Common Mistakes

1. Using HTTP polling for real-time chat. Polling every second creates 500 million requests per second from polling alone. WebSocket is not optional at this scale.
2. Ignoring message ordering. Saying "just use timestamps" fails because distributed clocks have skew. Use per-conversation sequence numbers.
3. Storing messages in a relational database. At 1.16 million writes per second, a relational database with B-tree indexes would collapse. Cassandra's LSM tree architecture handles write-heavy workloads far better.

4. Forgetting offline delivery. If you only describe the "both users online" case, you miss half the problem. Always discuss what happens when the recipient is offline.
5. Not discussing the session registry. How does Server A know that Bob is connected to Server B? The session registry is the answer.
6. Treating group chat as a simple extension of 1:1 chat. Group chat has unique challenges: fan-out to hundreds of members, ordering across multiple senders, and admin controls.

25.11 Best Practices

1. Use idempotency keys for messages. The client generates a `client_message_id` with each message. If the client retries (due to a network glitch), the server deduplicates using this key.
2. Separate the connection layer from business logic. Chat servers handle WebSocket connections; business logic (validation, storage, routing) runs in separate services. This makes each layer independently scalable.
3. Implement graceful reconnection. When a WebSocket drops, the client should reconnect and sync missed messages using the `last_received_message_id`. Make this seamless to the user.
4. Batch push notifications. Do not send a push notification for every single message in a group. If a user is offline and receives 20 messages in a group, send one notification: "20 new messages in Project Team."
5. Use heartbeats wisely. A heartbeat interval of 30 seconds balances between accuracy (knowing when a user goes offline) and overhead (network and server resources).
6. Archive old messages to cold storage. Messages older than 1 year can be moved to cheaper storage (S3 or cheaper Cassandra tiers) and loaded on demand.

25.12 Quick Summary

A chat system's architecture revolves around three pillars: (1) persistent WebSocket connections for real-time bidirectional messaging, managed by a fleet of chat servers with a Redis-based session registry for cross-server routing; (2) Cassandra for message storage, partitioned by conversation_id for co-located, time-sorted message retrieval; and (3) per-conversation sequence numbers for ordering guarantees, combined with offline delivery through push notifications and sync-on-reconnect. Group chat uses shared message storage

(not duplication) with notification fan-out. Presence is tracked via heartbeats with Redis TTLs.

25.13 Key Points

- WebSocket provides persistent, full-duplex connections for real-time message delivery.
- Session registry (Redis) maps each user to their chat server for cross-server message routing.
- Cassandra stores messages partitioned by conversation_id, optimized for write-heavy time-series data.
- Per-conversation sequence numbers guarantee message ordering despite clock skew.
- Offline delivery uses push notifications plus sync-on-reconnect to ensure no message is lost.
- Group chat stores messages once (shared partition) and fans out notifications, not data.
- Heartbeat-based presence with Redis TTLs tracks online/offline status efficiently.
- Read receipts use a watermark approach: "I have read everything up to message X."
- Idempotency keys (client_message_id) prevent duplicate message delivery on retry.
- End-to-end encryption ensures even the server cannot read message content.

25.14 Practice Questions

1. If a chat server crashes and 100,000 users lose their connections, what happens? How do they reconnect, and how are pending messages delivered?
2. How would you handle a group with 10,000 members (like a broadcast channel)? Does the architecture change?
3. Alice sends a message on her phone, then immediately opens the app on her laptop. How do you ensure the message appears on both devices?
4. How would you implement "typing indicators" ("Alice is typing...") without overwhelming the system?
5. If Cassandra is temporarily unavailable, should the chat server still accept messages? If so, where do they go?

25.15 Exercises

1. Connection capacity: Calculate how many chat servers you need for 500 million concurrent connections. Assume each server has 32 GB of RAM and each connection uses 10 KB.
2. Ordering simulation: Write pseudocode for the sequence number assignment mechanism for a group chat with concurrent senders. Show how you handle the case where two messages arrive at the server simultaneously.
3. Offline sync protocol: Design the protocol for syncing messages when a user comes back online after 3 days. Consider that they might be in 50 group chats, each with hundreds of unread messages.
4. Presence at scale: If each user has an average of 200 contacts and checks their presence every 10 seconds, calculate the read load on the presence service. Design a solution that keeps this load manageable.
5. Multi-device sync: Extend the design to support users logged in on 3 devices simultaneously. How does message delivery change? How does read receipt sync work?

25.16 What Is Next?

In this chapter we built a real-time messaging system with WebSocket connections, message ordering, offline delivery, and presence tracking. In the next chapter, Chapter 26: Designing YouTube, we shift from text-based communication to the world of video. You will learn about video upload and transcoding pipelines, adaptive bitrate streaming (HLS/DASH), CDN delivery for large video files, and recommendation engines -- a completely different set of challenges that make video platforms among the most complex systems to design.

Chapter 26: Designing YouTube

Every minute, over 500 hours of video are uploaded to YouTube. Billions of people watch over a billion hours of video every day, streamed seamlessly from data centers to devices on every continent. YouTube is one of the largest, most complex systems ever built -- combining massive storage, real-time video processing, adaptive streaming, global CDN delivery, and machine learning-powered recommendations into a single product.

In this chapter we will design YouTube from the ground up. You will learn how the video upload and transcoding pipeline works, why adaptive bitrate streaming (HLS/DASH) is essential for a good viewing experience, how CDN delivery works for large video files, how the search and recommendation engines function, and how all the pieces fit together. This is a system design classic that tests your ability to think about storage, compute, and bandwidth at truly enormous scale.

26.1 Understanding the Problem

YouTube's core features are:

1. Upload a video -- creators upload video files that are processed and made available to viewers.
2. Stream a video -- viewers watch videos with adaptive quality based on their connection speed.
3. Search videos -- find videos by title, description, tags, and content.
4. Recommendations -- personalized suggestions for what to watch next.
5. Engagement -- like, comment, subscribe, and share.

What Makes YouTube Uniquely Challenging?

- Enormous file sizes. A single 10-minute 4K video is ~3 GB. Multiply by 500 hours uploaded per minute.

- Transcoding compute. Every uploaded video must be transcoded into dozens of formats and resolutions. This is CPU-intensive and time-consuming.
- Global streaming. A video uploaded in Tokyo must stream smoothly to a viewer in rural Brazil on a 3G connection.
- Storage never shrinks. Unlike chat messages that can be archived, videos are expected to be available forever. Storage only grows.

26.2 Requirements

Functional Requirements

#	Requirement	Description
F1	Upload video	Creators upload video files (up to 256 GB, 12 hours).
F2	Stream video	Viewers watch videos with adaptive quality.
F3	Search	Find videos by keywords, filters (date, duration, quality).
F4	Recommendations	Show personalized "up next" and home page suggestions.
F5	Like / comment	Viewers engage with videos.
F6	Subscribe	Viewers subscribe to channels.
F7	Thumbnails	Auto-generated and custom thumbnails for each video.

Non-Functional Requirements

#	Requirement	Target
NF1	High availability	99.99% uptime for streaming.
NF2	Low startup latency	Video starts playing within 2 seconds.

NF3	Adaptive quality	Adjust bitrate in real time based on network conditions.
NF4	Scalability	2 billion monthly active users, 1 billion hours watched daily.
NF5	Durability	No uploaded video is ever lost.
NF6	Global reach	Low latency streaming from any location on Earth.

Out of Scope

- Live streaming (different architecture).
- Monetization and ad serving.
- Content moderation (important but orthogonal).
- YouTube Shorts.

26.3 Back-of-the-Envelope Estimation

Upload Traffic

Metric	Value
Video uploads per minute	500 hours of content
Average video length	10 minutes
Videos uploaded per minute	500 x 60 / 10 = 3,000 videos
Videos uploaded per second	~50 videos/s
Average raw file size (1080p)	1.5 GB
Upload bandwidth	50 x 1.5 GB = 75 GB/s ingress

Storage

Raw uploads per day:

```
50 videos/s x 86,400 s x 1.5 GB = 6,480 TB/day ≈ 6.5 PB/day (raw)
```

After transcoding, each video is stored in multiple resolutions and formats:

Resolution	Bitrate	Size (10 min video)
144p	0.1 Mbps	7.5 MB
240p	0.3 Mbps	22.5 MB
360p	0.7 Mbps	52.5 MB
480p	1.5 Mbps	112.5 MB
720p	3 Mbps	225 MB
1080p	6 Mbps	450 MB
1440p	10 Mbps	750 MB
4K	20 Mbps	1,500 MB
Total (all resolutions)		~3.1 GB

With ~2x overhead for different codecs (H.264, VP9, AV1):

```
Per video: ~6 GB total stored
Daily: 50 videos/s x 86,400 s x 6 GB = 25.9 PB/day
Yearly: ~9.5 EB/year
```

Streaming Traffic

```
1 billion hours watched daily
Average bitrate: 5 Mbps (mix of qualities)
1B hours x 3600 s/hour x 5 Mbps = 18 exabits/day
= 2.25 exabytes/day = 26 TB/s sustained
```

This is the scale that requires a massive CDN.

Thumbnail Storage

Each video generates 3 auto-thumbnails plus 1 custom upload, each in 3 sizes:

```
4 thumbnails x 3 sizes x 50 KB = 600 KB per video
50 videos/s x 86,400 s x 600 KB = 2.6 TB/day
```

Thumbnails are tiny compared to video but served much more frequently (every search result, recommendation, and browse page shows thumbnails).

26.4 High-Level Design

```
┌──────────┐          ┌──────────────┐
│ Creator  │──────────►│ API Gateway  │
│ (Upload) │          └──────────────┘
└──────────┘                  │
                              │
                   ┌──────────┼──────────┐
                   │          │          │
              ┌────▼────┐     │     ┌────▼────┐
              │ Upload  │     │     │ Video   │
              │ Service │     │     │ Metadata│
              └─────────┘     │     │ Service │
                   │          │     └─────────┘
                   │          │          │
              ┌────▼────┐     │     ┌────▼────┐
              │Original │     │     │Metadata │
              │Storage  │     │     │Database │
              │(S3/GCS) │     │     │(MySQL)  │
              └─────────┘     │     └─────────┘
                   │          │
                   │          │
              ┌────▼──────────────────────────┐
              │        Transcoding Pipeline   │
              │                               │
              │ Raw ──► Decode ──► Encode (N formats) │
              │                ──► Thumbnails │
              │                ──► Quality checks │
              └───────────────────────────────┘
                              │
              ┌───────────────▼───────────────┐
              │        Transcoded Storage (S3/GCS) │
              └───────────────────────────────┘
                              │
                        ┌─────▼─────┐
                        │   CDN     │
                        │ (Global)  │
                        └───────────┘
                              │
                        ┌─────▼─────┐
                        │ Viewer    │
                        │(Streaming)│
                        └───────────┘
```

```
┌──────────────────┐      ┌──────────────────────┐
│ Search Index     │      │ Recommendation       │
│(Elasticsearch)   │      │ Engine (ML)          │
└──────────────────┘      └──────────────────────┘
```

Request Flow: Uploading a Video

1. Creator initiates upload through the client app.
2. Upload Service handles the upload:
    - For large files, uses resumable upload (chunked upload with checkpointing).
    - Raw video is stored in original storage (S3/GCS).
3. Upload Service creates a metadata record in the Metadata Database with status "processing".
4. Upload Service publishes a "video uploaded" event to the message queue (Kafka).
5. Transcoding Pipeline picks up the event and processes the video:
    - Decodes the original video.
    - Encodes into 8 resolutions x 2-3 codecs = 16-24 output files.
    - Generates thumbnails.
    - Runs quality checks (bitrate verification, corruption detection).
    - Stores all output files in transcoded storage.
6. Transcoding Pipeline publishes "transcoding complete" event.
7. Metadata Service updates the video status to "published".
8. Search Indexer indexes the video metadata for search.
9. The video is now available for streaming.

Request Flow: Watching a Video

1. Viewer clicks on a video. Client requests the video manifest from the Metadata Service.
2. The manifest contains URLs for all available qualities (a `.m3u8` for HLS or `.mpd` for DASH).
3. Client's video player selects the appropriate quality based on screen size and bandwidth.
4. Player requests video segments (small chunks, typically 2-10 seconds each) from the CDN.
5. CDN serves segments from edge cache. On cache miss, pulls from origin (transcoded storage).
6. Player continuously monitors bandwidth and switches quality up or down (adaptive bitrate).

7. View event is logged for analytics and recommendation engine.

26.5 Deep Dive

26.5.1 Upload and Transcoding Pipeline

This is the most compute-intensive component. Let us break it down:

Resumable Upload

Large video files (potentially gigabytes) cannot be uploaded in a single HTTP request. We use resumable uploads:

```
1. Client: POST /upload/init
   Server: returns upload_id, chunk_size (5 MB)

2. Client uploads chunks:
   PUT /upload/{upload_id}/chunk/{chunk_number}
   (each chunk is 5 MB)

3. If upload is interrupted:
   Client: GET /upload/{upload_id}/status
   Server: returns last successful chunk number
   Client resumes from the next chunk

4. After all chunks uploaded:
   Client: POST /upload/{upload_id}/complete
   Server: concatenates chunks, starts processing
```

Transcoding Architecture

```
┌──────────────┐     ┌──────────────┐     ┌──────────────┐
│   Task       │     │  Transcoding │     │  Transcoding │
│   Queue      │────►│  Worker 1    │     │  Worker N    │
│   (Kafka)    │     │  (GPU/CPU)   │     │  (GPU/CPU)   │
└──────────────┘     └──────────────┘     └──────────────┘
                            │                    │
                     ┌──────▼────────────────────▼──────┐
                     │      Transcoded Storage (S3)     │
                     └──────────────────────────────────┘
```

Parallelization strategy:

1. Split the video into segments (e.g., 10-second chunks).
2. Transcode each segment independently across multiple workers.

3. Each worker encodes its segment into all required resolutions.
4. Merge the transcoded segments back together.

This turns a 30-minute transcoding job into a 2-3 minute job with enough workers.

Codec choices:

Codec	Compression	CPU cost	Compatibility
H.264	Good	Low	Universal
VP9	30% better than H.264	High	Chrome, Android
AV1	30% better than VP9	Very high	Growing support

YouTube transcodes to multiple codecs and serves the most efficient one the client supports.

26.5.2 Adaptive Bitrate Streaming (HLS / DASH)

Adaptive bitrate streaming is the key technology that makes video playback smooth across varying network conditions.

How it works:

```
┌──────────────────────────────────────────────────────────┐
│                    Video File                            │
│                                                          │
│  Split into segments:                                    │
│  [Seg 1][Seg 2][Seg 3][Seg 4][Seg 5]...                 │
│  (each 2-10 seconds)                                     │
│                                                          │
│  Each segment encoded at multiple bitrates:              │
│                                                          │
│  4K:    [Seg1_4K ][Seg2_4K ][Seg3_4K ]...  20 Mbps      │
│  1080p: [Seg1_1080][Seg2_1080][Seg3_1080]...  6 Mbps    │
│  720p:  [Seg1_720 ][Seg2_720 ][Seg3_720 ]...  3 Mbps    │
│  480p:  [Seg1_480 ][Seg2_480 ][Seg3_480 ]...  1.5Mbps   │
│  360p:  [Seg1_360 ][Seg2_360 ][Seg3_360 ]...  0.7Mbps   │
│                                                          │
│  Manifest file (.m3u8 / .mpd) lists all variants         │
└──────────────────────────────────────────────────────────┘
```

Playback flow:

1. Player requests the manifest file.
2. Manifest lists all available qualities and segment URLs.
3. Player starts with a conservative quality (e.g., 720p).
4. Player downloads segment 1 at 720p.

5. Player measures actual download speed:
    - If bandwidth > 6 Mbps: switch to 1080p for the next segment.
    - If bandwidth < 1 Mbps: switch to 360p for the next segment.
6. Quality switches happen at segment boundaries (seamless to the viewer).

HLS vs DASH:

Aspect	HLS (Apple)	DASH
Manifest format	.m3u8 (text)	.mpd (XML)
Segment format	.ts or .fmp4	.m4s (fmp4)
Browser support	Safari native, others via JS	Chrome, Firefox, Edge native
DRM support	FairPlay	Widevine, PlayReady
Market share	Dominant on Apple devices	Dominant elsewhere

YouTube serves both HLS and DASH depending on the client.

26.5.3 CDN Delivery

YouTube's CDN strategy is multi-layered:

```
┌─────────┐    ┌──────────────┐    ┌──────────────┐    ┌──────────┐
│ Viewer  │──→│  CDN Edge    │──→│  CDN Regional│──→│  Origin  │
│         │    │  (ISP-level)│    │  (City-level)│    │  (GCS)   │
└─────────┘    └──────────────┘    └──────────────┘    └──────────┘
```

Tier 1 -- Edge cache (ISP peering):

- YouTube deploys cache servers inside ISP networks (called Google Global Cache, or GGC).
- These serve the majority of popular video segments directly from the ISP's network.
- Zero transit cost for the ISP and minimal latency for the viewer.

Tier 2 -- Regional edge (city PoPs):

- Google-owned data centers in major cities.
- Serve cache misses from Tier 1.

Tier 3 -- Origin storage:

- Video master copies in Google Cloud Storage.
- Only accessed for very cold content (old, unpopular videos).

Cache strategy:

- Popular videos (top 20% that generate 80% of views): proactively pushed to edge caches.
- Long-tail videos (millions of videos with few views each): cached on demand, evicted with LRU.
- Segment-level caching: only cache the segments actually requested, not entire videos.

26.5.4 Video Metadata and Search

Metadata Database (MySQL):

- Stores video title, description, tags, category, upload date, view count, like/dislike count.
- Sharded by video_id.

Search Index (Elasticsearch):

- Indexes video metadata for full-text search.
- Fields indexed: title (boosted weight), description, tags, channel name, auto-generated captions.
- Ranking factors: relevance score, view count, recency, engagement rate, channel authority.

Auto-generated captions:

- YouTube uses speech-to-text ML models to generate captions for uploaded videos.
- Captions are indexed for search, making video content searchable even if the title is vague.
- This is a significant competitive advantage (most of the searchable content is in the audio, not the title).

26.5.5 Recommendation Engine

Recommendations drive most views on YouTube (over 70% of watch time comes from recommendations).

Architecture:

```
┌──────────────┐    ┌──────────────┐    ┌──────────────┐
│ User Watch   │──►│ Feature      │──►│ ML Model     │
│ History      │    │ Engineering  │    │ Serving      │
└──────────────┘    └──────────────┘    └──────────────┘
                                              │
                                              ▼
┌──────────────┐    ┌──────────────┐    ┌──────────────┐
│ Video        │──►│ Candidate    │──►│ Ranking      │
```

```
│  Features     │      │  Generation     │      │  (Top N)      │
└───────────────┘      └─────────────────┘      └───────────────┘
```

Two-stage pipeline:

1. Candidate generation: from millions of videos, narrow down to a few hundred candidates.
    - Collaborative filtering: users who watched video A also watched video B.
    - Content-based: videos with similar titles, tags, and categories.
    - Social signals: videos popular among the user's demographic.
2. Ranking: score each candidate and select the top N.
    - Features: video age, view velocity, user engagement history, watch time prediction.
    - Model: deep neural network trained on billions of watch events.
    - Output: predicted watch time (not click probability -- this reduces clickbait).

Serving: Pre-compute recommendations periodically (every few hours) and cache them. Augment with real-time signals (what the user just watched) at request time.

26.5.6 Thumbnail Generation

Thumbnails are critical for click-through rate. YouTube generates them automatically:

```
┌──────────────┐      ┌──────────────┐      ┌──────────────┐
│  Transcoded  │──────►│  Thumbnail   │──────►│  Thumbnail   │
│  Video       │      │  Extractor   │      │  Storage     │
└──────────────┘      │              │      │  (S3 + CDN)  │
                      │ - Sample N   │      └──────────────┘
                      │   frames     │
                      │ - Score by   │
                      │   quality    │
                      │ - Pick top 3 │
                      └──────────────┘
```

Process:

1. Sample frames at regular intervals (every 2 seconds).
2. Score each frame using an ML model (clarity, composition, faces, text readability).
3. Select the top 3 as auto-generated thumbnails.
4. Creator can also upload a custom thumbnail.
5. Generate each thumbnail in multiple sizes (120x90, 320x180, 480x360, 1280x720).
6. Serve from CDN with aggressive caching (thumbnails are immutable once generated).

26.6 Database Schema

Videos Table

```sql
CREATE TABLE videos (
    video_id        BIGINT PRIMARY KEY,     -- Snowflake ID
    channel_id      BIGINT NOT NULL,
    title           VARCHAR(100) NOT NULL,
    description     TEXT,
    status          ENUM('uploading', 'processing', 'published', 'failed', 'removed'),
    duration_seconds INT,
    original_s3_key VARCHAR(500),
    view_count      BIGINT DEFAULT 0,
    like_count      INT DEFAULT 0,
    dislike_count   INT DEFAULT 0,
    comment_count   INT DEFAULT 0,
    category        VARCHAR(50),
    tags            JSON,
    language        VARCHAR(10),
    published_at    TIMESTAMP,
    created_at      TIMESTAMP DEFAULT NOW(),
    INDEX idx_channel_time (channel_id, published_at DESC),
    INDEX idx_status (status)
);
-- Sharded by video_id
```

Video Formats Table

```sql
CREATE TABLE video_formats (
    video_id        BIGINT NOT NULL,
    format_id       INT NOT NULL,           -- e.g., 1=144p_h264, 2=240p_h264, ...
    resolution      VARCHAR(10),            -- '1080p', '720p', etc.
    codec           VARCHAR(20),            -- 'h264', 'vp9', 'av1'
    bitrate_kbps    INT,
    s3_key          VARCHAR(500),
    manifest_url    VARCHAR(500),
    file_size_bytes BIGINT,
    PRIMARY KEY (video_id, format_id)
);
-- Sharded by video_id (co-located with videos)
```

Channels Table

```sql
CREATE TABLE channels (
    channel_id      BIGINT PRIMARY KEY,
    user_id         BIGINT NOT NULL,
    name            VARCHAR(100) NOT NULL,
    description     TEXT,
    subscriber_count BIGINT DEFAULT 0,
    video_count     INT DEFAULT 0,
    created_at      TIMESTAMP DEFAULT NOW()
);
```

Comments Table

```sql
CREATE TABLE comments (
    comment_id      BIGINT PRIMARY KEY,
    video_id        BIGINT NOT NULL,
    user_id         BIGINT NOT NULL,
    content         VARCHAR(10000),
    parent_id       BIGINT,                -- for threaded replies
    like_count      INT DEFAULT 0,
    created_at      TIMESTAMP DEFAULT NOW(),
    INDEX idx_video_time (video_id, created_at DESC)
);
-- Sharded by video_id
```

Watch History Table

```sql
CREATE TABLE watch_history (
    user_id         BIGINT,
    video_id        BIGINT,
    watched_seconds INT,
    total_seconds   INT,
    watched_at      TIMESTAMP,
    PRIMARY KEY (user_id, watched_at)
) -- In Cassandra or BigTable for write throughput
WITH CLUSTERING ORDER BY (watched_at DESC);
```

Thumbnails Table

```sql
CREATE TABLE thumbnails (
    video_id        BIGINT NOT NULL,
    thumbnail_type  ENUM('auto_1', 'auto_2', 'auto_3', 'custom'),
    size            VARCHAR(20),            -- '120x90', '320x180', etc.
```

```
    s3_key          VARCHAR(500),
    is_default      BOOLEAN DEFAULT FALSE,
    PRIMARY KEY (video_id, thumbnail_type, size)
);
```

26.7 API Design

Upload a Video

Step 1: Initialize upload

```
POST /api/v1/videos/upload/init
Authorization: Bearer {token}

Request Body:
{
    "title": "System Design Tutorial",
    "description": "Learn system design...",
    "file_size": 1500000000,
    "content_type": "video/mp4",
    "category": "Education",
    "tags": ["system design", "interview"]
}

Response: 200 OK
{
    "upload_id": "upload_abc123",
    "video_id": "vid_789",
    "chunk_size": 5242880,
    "upload_url": "https://upload.youtube.com/v1/upload/upload_abc123"
}
```

Step 2: Upload chunks

```
PUT /api/v1/videos/upload/{upload_id}/chunk/{chunk_number}
Content-Type: application/octet-stream
Content-Length: 5242880

[binary chunk data]

Response: 200 OK
{
    "chunk_number": 42,
    "status": "received"
```

```
}
```

Step 3: Complete upload

```
POST /api/v1/videos/upload/{upload_id}/complete

Response: 202 Accepted
{
    "video_id": "vid_789",
    "status": "processing",
    "estimated_processing_time": "5 minutes"
}
```

Stream a Video

```
GET /api/v1/videos/{video_id}/manifest
Authorization: Bearer {token}

Response: 200 OK
{
    "video_id": "vid_789",
    "title": "System Design Tutorial",
    "manifest_urls": {
        "hls": "https://cdn.youtube.com/vid_789/master.m3u8",
        "dash": "https://cdn.youtube.com/vid_789/manifest.mpd"
    },
    "available_qualities": ["4k", "1080p", "720p", "480p", "360p", "240p", "144p"],
    "thumbnails": {
        "default": "https://cdn.youtube.com/vid_789/thumb_default.jpg"
    }
}
```

Search Videos

```
GET /api/v1/search?q=system+design&sort=relevance&duration=medium&limit=20&cursor=xxx
Authorization: Bearer {token}

Response: 200 OK
{
    "results": [
        {
            "video_id": "vid_789",
            "title": "System Design Tutorial",
            "channel": {"name": "TechChannel", "subscriber_count": 500000},
            "view_count": 1200000,
```

```
            "duration": 3600,
            "thumbnail": "https://cdn.youtube.com/vid_789/thumb_320x180.jpg",
            "published_at": "2025-01-10T08:00:00Z"
        }
    ],
    "next_cursor": "..."
}
```

Get Recommendations

```
GET /api/v1/recommendations?video_id={video_id}&limit=20
Authorization: Bearer {token}

Response: 200 OK
{
    "recommendations": [
        {
            "video_id": "vid_456",
            "title": "Advanced System Design",
            "reason": "Similar to what you watched",
            ...
        }
    ]
}
```

26.8 Scaling the System

Upload and Transcoding

- Auto-scaling worker fleet: scale transcoding workers based on queue depth.
- GPU acceleration: use GPU instances for video encoding (10-50x faster than CPU).
- Spot/preemptible instances: transcoding is fault-tolerant (can restart from the last checkpoint), making it ideal for cheap spot instances.
- Priority queues: creator partners and paying users get faster transcoding.

Storage Tiering

```
┌──────────────────────────────────────────────────────────┐
│                    Storage Tiers                         │
```

```
│                                                          │
│  Hot (SSD):    Videos < 7 days old (most views)          │
│  Warm (HDD):  Videos 7-90 days old                       │
│  Cold (Archive): Videos > 90 days old, < 10 views/month │
│  Glacier:     Videos > 1 year, near zero views           │
└──────────────────────────────────────────────────────────┘
```

Lifecycle policies automatically migrate videos between tiers. When a cold video is requested, it is promoted back to warm/hot storage.

CDN Scaling

- Deploy cache servers at ISP peering points (Google Global Cache).
- Use consistent hashing to distribute video segments across CDN nodes.
- Serve popular videos from L1 cache (hit rate > 90%).
- Pre-warm CDN for videos that are predicted to go viral (based on early view velocity).

View Count Scaling

View counts are updated billions of times per day. Exact real-time counting is expensive:

```
Approximate counting strategy:
1. Client logs view events to Kafka
2. Stream processor (Flink) counts views in 1-minute windows
3. Aggregated counts written to Redis (near real-time display)
4. Batch job reconciles exact counts every hour and writes to MySQL
```

Database Scaling

- Videos metadata: MySQL sharded by video_id. 500 shards with read replicas.
- Watch history: Cassandra (write-heavy, time-series). Partition by user_id.
- View counts: Redis for real-time approximate, MySQL for exact historical.
- Search index: Elasticsearch cluster with time-based sharding.

26.9 Trade-offs

Decision	Option A	Option B	Our Choice	Why

Upload method	Single request	Resumable chunked	Resumable	Large files need checkpoint/resume capability.
Transcoding	On-demand	Pre-transcode all formats	Pre-transcode	Pay compute cost once; serve instantly forever.
Codec strategy	Single codec (H.264)	Multi-codec	Multi-codec	30-50% bandwidth savings with VP9/AV1 for supported clients.
Streaming protocol	Progressive download	Adaptive bitrate (HLS/DASH)	Adaptive	Adjusts to network conditions; better experience on variable connections.
CDN strategy	Third-party CDN	Own CDN (ISP peering)	Own CDN	At YouTube's scale, owning is cheaper and gives more control.
View counting	Exact real-time	Approximate near real-time	Approximate	Exact counting at billions/day is too expensive; approximate is good enough for display.
Recommendation	Collaborative filtering only	Deep learning	Deep learning	Better personalization, predicts watch time (not just clicks).

26.10 Common Mistakes

1. Ignoring transcoding complexity. Saying "we just store the video" misses the most compute-intensive part of the system. Always discuss the transcoding pipeline, codec choices, and parallelization.
2. Forgetting adaptive bitrate. Serving a single quality file means either wasting bandwidth on low-end connections or providing poor quality on fast ones. ABR is essential.
3. Underestimating storage. YouTube-scale storage is measured in exabytes. Discuss storage tiering and lifecycle management.

4. Not mentioning resumable uploads. A 2 GB video upload over a flaky connection needs resume capability. This is not optional.
5. Treating CDN as a black box. At YouTube's scale, CDN is not just "put it on Cloudflare." Discuss ISP peering, cache hierarchy, and pre-warming strategies.
6. Ignoring the recommendation engine. Recommendations drive 70%+ of views. Skipping this means missing a core component.

26.11 Best Practices

1. Transcode in parallel using segmented encoding. Split the video into chunks, transcode each independently, then merge. This reduces processing time from hours to minutes.
2. Use multiple codecs. Serve AV1 to clients that support it (50% bandwidth savings over H.264), VP9 as fallback, and H.264 for universal compatibility.
3. Implement storage tiering aggressively. Most videos get 90% of their views in the first week. Move cold content to cheaper storage tiers automatically.
4. Pre-warm CDN for trending content. When a video's view velocity spikes, proactively push it to more edge locations before it overwhelms the origin.
5. Log watch events for everything. Watch history, completion rate, pause/seek events, and quality switches are all valuable signals for the recommendation engine and content analytics.
6. Separate the upload pipeline from the serving pipeline. Uploading and transcoding are CPU/bandwidth-intensive; serving is latency-sensitive. Keep them on separate infrastructure.

26.12 Quick Summary

YouTube's architecture is built around four pillars: (1) a resumable upload pipeline that feeds into a massively parallel transcoding system, producing video segments in multiple resolutions and codecs; (2) adaptive bitrate streaming (HLS/DASH) that serves video in small segments, dynamically adjusting quality based on the viewer's network conditions; (3) a multi-tier CDN with ISP-level peering that serves 90%+ of video traffic from edge caches; and (4) a deep learning-based recommendation engine that drives 70%+ of watch time by predicting user engagement. Storage is measured in exabytes and managed through aggressive tiering. View counts use approximate near-real-time counting for display with periodic exact reconciliation.

26.13 Key Points

- Resumable chunked uploads handle large video files (up to 256 GB) with checkpoint/resume capability.
- Parallel segmented transcoding splits videos into chunks, encodes independently, and merges -- reducing processing time by 10x+.
- Multiple codecs (H.264, VP9, AV1) serve the most efficient format each client supports.
- Adaptive bitrate streaming (HLS/DASH) adjusts video quality per segment based on real-time network conditions.
- ISP-level CDN peering (Google Global Cache) serves most traffic from within the viewer's ISP network.
- Storage tiering (hot/warm/cold/archive) manages exabyte-scale storage costs.
- Approximate view counting (stream processing + periodic batch reconciliation) handles billions of daily view events.
- Two-stage recommendation (candidate generation + deep learning ranking) predicts watch time, not just clicks.
- Auto-generated thumbnails use ML to select the best frames; multiple sizes are served from CDN.
- Video segments (2-10 seconds each) are the unit of caching, streaming, and quality switching.

26.14 Practice Questions

1. A creator uploads a 4K, 2-hour video (50 GB raw). Estimate the total storage needed after transcoding into all resolutions and codecs.
2. How would you handle the case where a video goes viral 5 minutes after upload, before transcoding into all formats is complete?
3. If the CDN cache hit rate drops from 95% to 85%, what is the impact on origin servers (in TB/s of additional traffic)?
4. How would you design the "watch later" and "playlist" features? What database and caching strategy would you use?
5. A video has 1 billion views. How would you efficiently count and display this without overwhelming the database?

26.15 Exercises

1. Transcoding capacity: If each transcoding worker can process 1 minute of video per minute (real-time speed), and YouTube receives 500 hours of video per minute that needs to be transcoded into 20 formats, how many workers are needed? What if GPU workers are 20x faster?
2. CDN cost estimation: Calculate the monthly CDN egress cost for serving 1 billion hours of video at an average bitrate of 5 Mbps, assuming $0.02 per GB of egress.
3. Manifest file design: Write a sample HLS manifest file (.m3u8) for a video with 4 quality levels (360p, 720p, 1080p, 4K), showing the segment URLs and bitrate declarations.
4. Storage lifecycle: Design the rules for a storage lifecycle policy that minimizes cost while ensuring videos are always accessible. Consider view frequency, video age, and access latency requirements.
5. Recommendation evaluation: Design an A/B testing framework for the recommendation engine. What metrics would you track? How would you ensure the test is statistically significant?

26.16 What Is Next?

In this chapter we designed a video platform handling exabytes of storage, millions of concurrent streams, and sophisticated content recommendations. In the next chapter, Chapter 27: Designing Uber, we move to a fundamentally different domain: real-time geospatial systems. You will learn about geohashing and quadtrees for spatial indexing, real-time driver matching, ETA estimation, surge pricing algorithms, and how to build a dispatch system that matches riders with drivers in seconds -- challenges that require thinking about geography, time, and economics simultaneously.

Chapter 27: Designing Uber

When you open a ride-hailing app, tap a button, and a car appears at your doorstep within minutes, a remarkable amount of engineering is working behind the scenes. In the time between your request and your driver's arrival, the system has matched you with the best available driver, calculated an estimated arrival time, set dynamic pricing based on supply and demand, and begun tracking both your location and the driver's location in real time.

In this chapter we will design Uber from scratch. You will learn how geospatial indexing works (geohash and quadtree), how the dispatch system matches riders with nearby drivers in milliseconds, how ETA estimation combines routing with real-time traffic data, how surge pricing balances supply and demand, and how real-time location tracking is delivered over WebSocket connections. This is one of the most challenging system design problems because it combines geography, real-time systems, economics, and payments into a single product.

27.1 Understanding the Problem

Uber's core flow is:

1. Rider requests a ride -- specifying pickup and dropoff locations.
2. System matches a driver -- finds the best available driver nearby.
3. Driver accepts and navigates to pickup -- both rider and driver see each other's location in real time.
4. Trip begins -- the system tracks the route and calculates the fare.
5. Trip ends -- payment is processed automatically.

What Makes Uber Uniquely Challenging?

- Geospatial queries at scale. "Find all available drivers within 3 km of this location" must execute in milliseconds, millions of times per minute.

- Real-time location updates. Millions of drivers send GPS updates every 3-5 seconds. That is hundreds of thousands of writes per second.
- Two-sided marketplace. Unlike social networks where users are interchangeable, Uber must balance supply (drivers) and demand (riders) in real time, per location.
- Time-critical matching. A rider expects a match within seconds, not minutes. Delayed matching means lost riders.

27.2 Requirements

Functional Requirements

#	Requirement	Description
F1	Request a ride	Rider specifies pickup and dropoff. System returns fare estimate and ETA.
F2	Match driver	Find and assign the best nearby available driver.
F3	Real-time tracking	Both rider and driver see each other's live location during the trip.
F4	Trip management	Start trip, navigate, end trip, calculate fare.
F5	Payment	Automatically charge the rider and pay the driver after the trip.
F6	Surge pricing	Dynamically adjust prices based on real-time supply and demand.
F7	Driver availability	Drivers can go online/offline. System tracks their status and location.

Non-Functional Requirements

#	Requirement	Target

NF1	Low latency matching	Match a driver within 5 seconds of request.
NF2	Real-time updates	Location updates delivered within 1 second.
NF3	High availability	99.99% uptime. Ride requests must always work.
NF4	Scalability	20 million rides per day, 5 million concurrent drivers.
NF5	Accuracy	ETA accurate within 2 minutes. Fare accurate to the cent.
NF6	Consistency	A driver can only be matched to one rider at a time. No double-booking.

Out of Scope

- Uber Eats (food delivery).
- Driver onboarding and background checks.
- Autonomous vehicles.
- Multi-stop rides.

27.3 Back-of-the-Envelope Estimation

Traffic

Metric	Value
Daily rides	20 million
Rides per second	20M / 86,400 ≈ 230 rides/s
Concurrent drivers online	5 million
Driver location updates	Every 4 seconds
Location writes per second	5M / 4 = 1.25 million writes/s

Rider location queries (during trips)	~2 million concurrent trips x 1 query/4s = 500,000 reads/s

Storage

Item	Size
Trip record	~1 KB (pickup, dropoff, route, fare, timestamps)
Location history per trip	~2 KB (GPS points every 4 seconds for ~20 min)
Total per trip	~3 KB

Daily storage:

```
20M trips/day x 3 KB = 60 GB/day
Yearly: ~22 TB/year
```

Driver location data (ephemeral, in-memory):

```
5M drivers x 50 bytes (lat, lng, timestamp, status) = 250 MB
```

This fits entirely in memory. The geospatial index is an in-memory system.

Geospatial Query Load

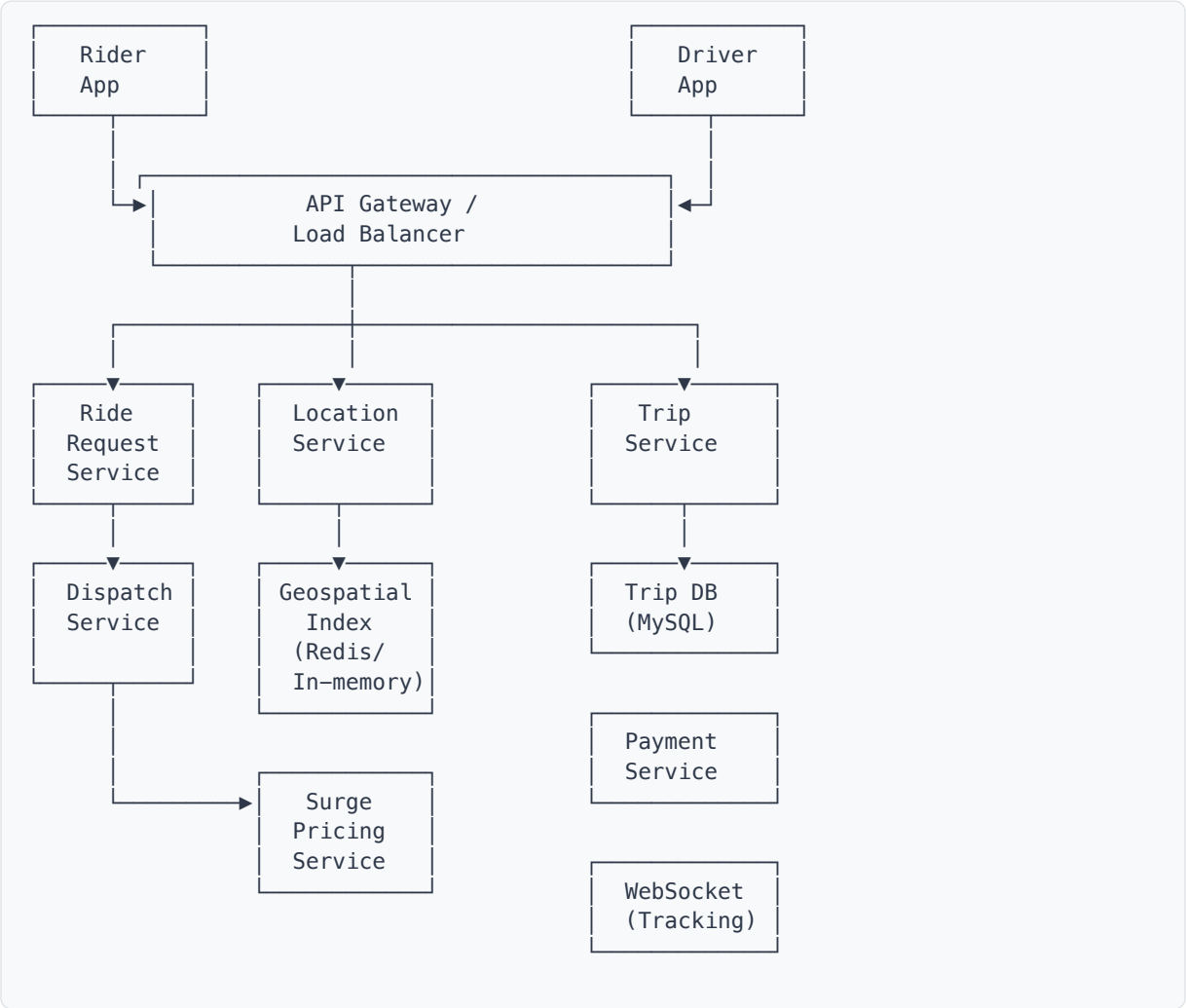
When a rider requests a ride, the system queries nearby drivers:

```
230 ride requests/s x average 50 nearby drivers checked = 11,500 proximity lookups/s
```

Plus periodic re-indexing as drivers move:

```
1.25 million location updates/s requiring index updates
```

27.4 High-Level Design



Request Flow: Requesting a Ride

```
1. Rider opens app, enters pickup and dropoff
2. Rider App ──► Ride Request Service:
   - Calculate fare estimate (distance + time + surge multiplier)
   - Display to rider
3. Rider confirms ──► Ride Request Service:
   - Create ride request record
   - Send to Dispatch Service
4. Dispatch Service:
   a. Query Geospatial Index: "available drivers within 3 km of pickup"
   b. Score candidates (proximity, rating, vehicle type, direction of travel)
   c. Send ride offer to the top driver
5. Driver App ◄── offer via WebSocket
6. Driver accepts ──► Dispatch Service:
   a. Mark driver as "on_trip" in Geospatial Index
   b. Create Trip record
   c. Notify rider via WebSocket: "Driver matched!"
```

```
7. Real-time tracking begins:
   - Driver location ──▶ Location Service ──▶ WebSocket ──▶ Rider App
   - Rider location ──▶ Location Service ──▶ WebSocket ──▶ Driver App
```

Request Flow: Driver Rejecting or Timeout

```
If driver does not accept within 15 seconds:
1. Dispatch Service marks the offer as expired
2. Dispatch Service selects the next-best driver
3. Sends new offer
4. Repeat up to 3 times
5. If no driver accepts: notify rider "No drivers available"
```

27.5 Deep Dive

27.5.1 Geospatial Indexing

The fundamental operation is: "Find all available drivers near location (lat, lng) within radius R." There are two main approaches:

Geohash

A geohash encodes a (latitude, longitude) pair into a string where nearby locations share a common prefix:

```
Location: (37.7749, -122.4194)  -- San Francisco
Geohash:  9q8yyk8

Precision levels:
  "9"        -- covers ~5,000 km x 5,000 km
  "9q"       -- covers ~1,250 km x 625 km
  "9q8"      -- covers ~156 km x 156 km
  "9q8y"     -- covers ~39 km x 19.5 km
  "9q8yy"    -- covers ~5 km x 5 km
  "9q8yyk"   -- covers ~1.2 km x 0.6 km
  "9q8yyk8"  -- covers ~150 m x 150 m
```

How to find nearby drivers using geohash:

```
1. Compute the geohash of the rider's location at precision 6 (≈1 km cell)
2. Find the 8 neighboring geohash cells (to handle edge effects)
3. Query Redis: SMEMBERS drivers:available:{geohash} for each of the 9 cells
4. Filter by exact distance (some drivers in neighboring cells may be too far)
5. Sort by distance and score
```

Redis data structure:

```
Key:    drivers:available:{geohash_prefix}
Type:   Set
Members: driver_id values

Example:
  drivers:available:9q8yyk → {driver_101, driver_205, driver_389}
```

When a driver's location updates:

1. Compute old geohash and new geohash.
2. If different: SREM from old cell, SADD to new cell.
3. If same: no index change needed.

Quadtree

A quadtree recursively divides the map into four quadrants. Each leaf node contains a manageable number of drivers:

```
┌──────────────────────────────┐
│              │               │
│   NW (12     │   NE (8       │
│   drivers)   │   drivers)    │
│              │               │
├──────────────┼───────────────┤
│              │               │
│   SW         │   SE          │
│  ┌────┬────┐ │               │
│  │ 5  │ 3  │ │   (45         │
│  ├────┼────┤ │   drivers)    │
│  │ 7  │ 15 │ │     ──►       │
│  └────┴────┘ │   subdivide!  │
│              │               │
└──────────────────────────────┘
```

Rules:

- Each cell has a maximum capacity (e.g., 100 drivers).
- When a cell exceeds capacity, subdivide it into 4 children.

- Dense areas (downtown) have many small cells; sparse areas (rural) have few large cells.

Pros vs. geohash:

Aspect	Geohash	Quadtree
Implementation	Simple (string operations)	More complex (tree structure)
Density adaptation	Fixed grid size	Adapts to driver density
Query	String prefix matching	Tree traversal
Update cost	O(1) hash compute	O(log N) tree traversal
Best for	Uniform-ish distribution	Highly variable density

Our choice: Geohash with Redis for simplicity and speed. At Uber's scale, the slight inefficiency in sparse areas is outweighed by the simplicity and Redis's performance.

27.5.2 Driver Matching (Dispatch)

Finding nearby drivers is just the first step. The dispatch service must pick the best driver:

Scoring factors:

```
score = w1 * proximity_score
      + w2 * eta_score
      + w3 * driver_rating
      + w4 * heading_score
      + w5 * acceptance_rate
```

Factor	Description
Proximity	Straight-line distance to pickup.
ETA	Actual driving time (accounts for roads, traffic).
Rating	Higher-rated drivers preferred.
Heading	Driver already heading toward pickup scores higher (less time to turn around).
Acceptance rate	Drivers who frequently decline offers are deprioritized.

Dispatch algorithm:

```
1. Query geospatial index for available drivers within 3 km
2. If fewer than 3 candidates, expand to 5 km
3. For each candidate:
   a. Calculate ETA via routing service
   b. Compute score
4. Sort by score (descending)
5. Offer to top-scored driver
6. If declined/timeout (15s), offer to next driver
7. If no match after 3 attempts, expand radius and retry
8. If still no match, inform rider
```

Preventing double-booking: When a ride offer is sent to a driver, that driver is temporarily "locked" (status = "offered"). No other ride request can be offered to them. If they decline or timeout, the lock is released.

27.5.3 ETA Estimation

ETA (Estimated Time of Arrival) is one of Uber's most critical calculations. An inaccurate ETA leads to frustrated riders and poor driver utilization.

Components:

```
┌──────────────┐      ┌──────────────┐      ┌──────────────┐
│  Road Graph  │      │  Real-time   │      │  Historical  │
│  (Map Data)  │      │  Traffic     │      │  Travel Time │
└──────────────┘      └──────────────┘      └──────────────┘
       │                    │                     │
       └────────────────────┼─────────────────────┘
                            │
                            ▼
                     ┌──────────────┐
                     │  Routing     │
                     │  Engine      │
                     │  (Dijkstra/  │
                     │   A*)        │
                     └──────────────┘
                            │
                            ▼
                     ┌──────────────┐
                     │    ETA       │
                     └──────────────┘
```

Road graph: A weighted directed graph where:

- Nodes are intersections.
- Edges are road segments.
- Edge weights are travel times (not distances).

Real-time traffic: Edge weights are updated every few minutes based on:

- GPS data from Uber drivers currently on the road.

- Third-party traffic data.
- Time of day and day of week patterns.

Algorithm: Modified A* search with traffic-aware edge weights. For long routes, use hierarchical routing (compute on highway graph first, then local roads).

ETA caching: For popular routes (e.g., airport to downtown), cache ETAs and refresh every 5 minutes. Avoids recomputing the same routes millions of times.

27.5.4 Surge Pricing

Surge pricing dynamically adjusts fares when demand exceeds supply in a geographic area:

```
┌──────────────────────────────────────────────┐
│              Surge Pricing Engine            │
│                                              │
│  For each geographic zone (hexagonal grid):  │
│                                              │
│  demand = ride_requests_last_5min            │
│  supply = available_drivers_in_zone          │
│                                              │
│  ratio = demand / supply                     │
│                                              │
│  if ratio > 1.5:                             │
│      surge_multiplier = f(ratio)             │
│      (e.g., 2.0x, 2.5x, 3.0x)              │
│  else:                                       │
│      surge_multiplier = 1.0                  │
│                                              │
│  Fare = base_fare * surge_multiplier         │
│       + per_mile * distance                  │
│       + per_minute * duration                │
│       + booking_fee                          │
└──────────────────────────────────────────────┘
```

Geographic zones: The city is divided into hexagonal zones (hexagons tile without gaps, unlike squares). Each zone is about 1 km across.

Update frequency: Surge multipliers recalculated every 1-2 minutes.

Smoothing: To prevent jarring price jumps, apply exponential moving average:

```
new_surge = 0.7 * calculated_surge + 0.3 * previous_surge
```

Communication: The rider sees the surge multiplier before confirming. They can wait for the surge to decrease or accept the higher fare.

27.5.5 Trip Service

The trip service manages the lifecycle of a ride:

```
States: REQUESTED ──► MATCHED ──► DRIVER_EN_ROUTE ──► ARRIVED
                                                        │
        ──► TRIP_STARTED ──► TRIP_COMPLETED ──► PAYMENT_PROCESSED
```

Fare calculation (at trip end):

```
fare = base_fare
     + (distance_miles * per_mile_rate)
     + (duration_minutes * per_minute_rate)
     + surge_multiplier_at_request_time * variable_fare
     + booking_fee
     + tolls
     - promotions / discounts

-- The surge multiplier is locked in at request time, not trip end
```

Route tracking: During the trip, the driver's GPS coordinates are recorded every 4 seconds. This forms the actual route, used for:

- Fare calculation (actual distance traveled).
- Detecting route deviations.
- Historical trip display.

27.5.6 Payment Service

Payment is processed automatically at the end of each trip:

```
┌──────────────┐     ┌──────────────┐     ┌──────────────┐
│  Trip        │──►  │  Payment     │──►  │  Payment     │
│  Service     │     │  Service     │     │  Gateway     │
│  (fare)      │     │              │     │ (Stripe/     │
└──────────────┘     │  - Auth hold │     │  Braintree)  │
                     │  - Capture   │     └──────────────┘
                     │  - Split     │
                     │    (driver/  │
                     │     Uber)    │
                     └──────────────┘
```

Authorization hold: When a rider requests a ride, a hold is placed on their payment method for the estimated fare. This ensures the rider can pay.

Capture: After the trip, the actual fare is captured (may differ from the estimate due to route changes).

Split: The fare is split between the driver (typically 75-80%) and Uber (20-25%).

Reliability: Payment processing uses the saga pattern:

1. Charge rider's payment method.
2. If successful, credit driver's account.
3. If charge fails, retry with exponential backoff.
4. If still fails, mark trip as "payment pending" and notify rider.

27.5.7 Real-Time Location Tracking (WebSocket)

During a trip, both rider and driver see each other's live location:

```
┌──────────┐      ┌──────────────┐      ┌──────────┐
│  Driver  │ ═══► │  Location    │ ═══► │  Rider   │
│   App    │ WS   │  Service     │ WS   │   App    │
│          │      │              │      │          │
│ GPS every│      │ - Validate   │      │ Updates  │
│ 4 seconds│      │ - Store      │      │ on map   │
│          │      │ - Forward    │      │          │
└──────────┘      └──────────────┘      └──────────┘
                        │
                        ▼
                  ┌──────────────┐
                  │ Geospatial   │
                  │ Index Update │
                  └──────────────┘
```

Protocol: WebSocket for persistent bidirectional connection. Falls back to SSE (Server-Sent Events) if WebSocket is blocked.

Location update flow:

1. Driver app sends GPS coordinates to Location Service via WebSocket every 4 seconds.
2. Location Service validates the update (reject impossible speeds, out-of-bounds coordinates).
3. Updates the driver's position in the Geospatial Index.
4. If the driver is on a trip, forwards the location to the matched rider via WebSocket.
5. Stores the location in the trip's route history.

Bandwidth optimization:

- Send only (lat, lng, timestamp, heading) -- about 30 bytes per update.

- Compress updates using delta encoding: send only the change from the previous position.
- Batch updates on the client side: if the driver has not moved significantly, skip the update.

27.5.8 Complete Architecture

Putting it all together:

```
┌──────────────────────────────────────────────────────────────────────────┐
│                                API Gateway                               │
│                             (Load Balancer)                              │
└──────────────────────────────────────────────────────────────────────────┘
        │           │           │           │           │
        │           │           │           │           │
  ┌─────▼─────┐ ┌───▼───────┐ ┌─▼─────────┐│   ┌───────▼───┐
  │  Ride     │ │Location   │ │  Trip     │└   │ Payment   │
  │  Request  │ │Service    │ │ Service   │    │ Service   │
  │  Service  │ │           │ │           │    │           │
  └───────────┘ └───────────┘ └───────────┘    └───────────┘
      │           │           │      │           │
  ┌───▼───────┐ ┌─────▼─────┐ ┌───▼───┐    │   ┌─────▼─────┐
  │ Dispatch  │ │Geospatial│ │Trip   │    │   │Payment    │
  │ Service   │ │  Index    │ │ DB    │    │   │Gateway    │
  │           │ │(Redis)    │ │(SQL)│    │   │(Stripe)│
  └───────────┘ └───────────┘ └───────┘    │   └───────────┘
      │                                    │
  ┌───▼───────┐                       ┌───▼───────┐
  │  Surge    │                       │ WebSocket│
  │ Pricing   │                       │ Gateway   │
  │ Service   │                       │(Tracking)│
  └───────────┘                       └───────────┘
      │
  ┌───▼───────┐                       ┌───────────┐
  │ Routing   │                       │  Kafka    │
  │ Engine    │                       │(Events)   │
  │(ETA/Maps)│                       └───────────┘
  └───────────┘
```

27.6 Database Schema

Riders Table

```sql
CREATE TABLE riders (
    rider_id        BIGINT PRIMARY KEY,
    name            VARCHAR(100),
    email           VARCHAR(255) UNIQUE,
    phone           VARCHAR(20) UNIQUE,
    rating          DECIMAL(3,2) DEFAULT 5.00,
    payment_method_id VARCHAR(100),
    created_at      TIMESTAMP DEFAULT NOW()
);
```

Drivers Table

```sql
CREATE TABLE drivers (
    driver_id       BIGINT PRIMARY KEY,
    name            VARCHAR(100),
    email           VARCHAR(255) UNIQUE,
    phone           VARCHAR(20) UNIQUE,
    license_plate   VARCHAR(20),
    vehicle_type    ENUM('economy', 'comfort', 'xl', 'premium'),
    vehicle_make    VARCHAR(50),
    vehicle_model   VARCHAR(50),
    vehicle_color   VARCHAR(30),
    rating          DECIMAL(3,2) DEFAULT 5.00,
    status          ENUM('offline', 'available', 'offered', 'on_trip'),
    created_at      TIMESTAMP DEFAULT NOW(),
    INDEX idx_status (status)
);
```

Trips Table

```sql
CREATE TABLE trips (
    trip_id             BIGINT PRIMARY KEY,
    rider_id            BIGINT NOT NULL,
    driver_id           BIGINT,
    status              ENUM('requested', 'matched', 'driver_en_route',
                             'arrived', 'in_progress', 'completed',
                             'cancelled'),
    pickup_lat          DECIMAL(10, 7),
    pickup_lng          DECIMAL(10, 7),
    pickup_address      VARCHAR(500),
    dropoff_lat         DECIMAL(10, 7),
    dropoff_lng         DECIMAL(10, 7),
    dropoff_address     VARCHAR(500),
    vehicle_type        ENUM('economy', 'comfort', 'xl', 'premium'),
    surge_multiplier    DECIMAL(3,1) DEFAULT 1.0,
    estimated_fare      DECIMAL(10,2),
```

```
    actual_fare         DECIMAL(10,2),
    distance_miles      DECIMAL(8,2),
    duration_minutes    DECIMAL(8,2),
    requested_at        TIMESTAMP,
    matched_at          TIMESTAMP,
    pickup_at           TIMESTAMP,
    dropoff_at          TIMESTAMP,
    created_at          TIMESTAMP DEFAULT NOW(),
    INDEX idx_rider (rider_id, created_at DESC),
    INDEX idx_driver (driver_id, created_at DESC),
    INDEX idx_status (status)
);
-- Sharded by trip_id
```

Trip Route Points Table

```sql
-- High-write, time-series data: use Cassandra
CREATE TABLE trip_route_points (
    trip_id         BIGINT,
    recorded_at     TIMESTAMP,
    lat             DECIMAL(10, 7),
    lng             DECIMAL(10, 7),
    speed_mph       DECIMAL(5, 1),
    heading         INT,
    PRIMARY KEY (trip_id, recorded_at)
) WITH CLUSTERING ORDER BY (recorded_at ASC);
```

Geospatial Index (Redis)

```
-- Available drivers indexed by geohash
Key:    drivers:geo:{geohash_6}
Type:   Set
Members: driver_id

-- Driver details (for quick lookup)
Key:    driver:location:{driver_id}
Type:   Hash
Fields: lat, lng, geohash, heading, speed, status, vehicle_type, rating
TTL:    60 seconds (refreshed with each location update)

-- Surge pricing zones
Key:    surge:{zone_id}
Type:   String
Value:  multiplier (e.g., "2.5")
TTL:    120 seconds (refreshed by surge pricing service)
```

Payments Table

```sql
CREATE TABLE payments (
    payment_id      BIGINT PRIMARY KEY,
    trip_id         BIGINT NOT NULL,
    rider_id        BIGINT NOT NULL,
    driver_id       BIGINT NOT NULL,
    amount          DECIMAL(10,2),
    driver_payout   DECIMAL(10,2),
    platform_fee    DECIMAL(10,2),
    payment_method  VARCHAR(50),
    status          ENUM('authorized', 'captured', 'failed', 'refunded'),
    processed_at    TIMESTAMP,
    created_at      TIMESTAMP DEFAULT NOW(),
    INDEX idx_trip (trip_id)
);
```

27.7 API Design

Request a Ride

```
POST /api/v1/rides/estimate
Authorization: Bearer {token}

Request Body:
{
    "pickup": {"lat": 37.7749, "lng": -122.4194},
    "dropoff": {"lat": 37.3382, "lng": -121.8863},
    "vehicle_type": "economy"
}

Response: 200 OK
{
    "estimates": [
        {
            "vehicle_type": "economy",
            "fare_range": {"min": 42.00, "max": 56.00},
            "surge_multiplier": 1.5,
            "eta_minutes": 4,
            "distance_miles": 45.2,
            "duration_minutes": 55
        },
        {
            "vehicle_type": "comfort",
            "fare_range": {"min": 58.00, "max": 72.00},
            ...
```

```
        }
    ]
}
```

Confirm Ride Request

```
POST /api/v1/rides
Authorization: Bearer {token}

Request Body:
{
    "pickup": {"lat": 37.7749, "lng": -122.4194, "address": "123 Market St"},
    "dropoff": {"lat": 37.3382, "lng": -121.8863, "address": "456 First St"},
    "vehicle_type": "economy",
    "payment_method_id": "pm_abc123"
}

Response: 201 Created
{
    "trip_id": "trip_789",
    "status": "requested",
    "surge_multiplier": 1.5,
    "estimated_fare": 49.00,
    "message": "Finding your driver..."
}
```

Driver Accept/Decline (WebSocket)

```
-- Server to Driver (offer):
{
    "action": "ride_offer",
    "trip_id": "trip_789",
    "pickup": {"lat": 37.7749, "lng": -122.4194, "address": "123 Market St"},
    "dropoff_address": "456 First St",
    "estimated_fare": 49.00,
    "estimated_distance": 45.2,
    "expires_in_seconds": 15
}

-- Driver to Server (accept):
{
    "action": "accept_ride",
    "trip_id": "trip_789"
}
```

Update Driver Location (WebSocket)

```
-- Driver to Server (every 4 seconds):
{
    "action": "location_update",
    "lat": 37.7751,
    "lng": -122.4190,
    "heading": 45,
    "speed": 25.5,
    "timestamp": 1705312200000
}
```

End Trip

```
POST /api/v1/trips/{trip_id}/complete
Authorization: Bearer {driver_token}

Response: 200 OK
{
    "trip_id": "trip_789",
    "status": "completed",
    "actual_fare": 52.30,
    "distance_miles": 46.1,
    "duration_minutes": 58,
    "payment_status": "captured"
}
```

27.8 Scaling the System

Geospatial Index Scaling

- Redis cluster sharded by geographic region (each city or metro area gets its own Redis cluster).
- Within a city, further shard by geohash prefix (e.g., all geohash cells starting with "9q8" on one shard).
- Each shard handles ~500,000 location updates per second.

Location Service Scaling

- Stateless service behind a load balancer.
- Auto-scale based on WebSocket connection count.
- Each server handles ~200,000 concurrent WebSocket connections.

- With 5 million concurrent drivers: ~25 location service servers.

Dispatch Service Scaling

- Partition by city/region. Each region has its own dispatch cluster.
- Within a region, dispatch requests are load-balanced across workers.
- Use distributed locking (Redis SETNX) to prevent double-booking drivers.

Trip Database Scaling

- MySQL sharded by trip_id.
- Read replicas for queries (trip history, analytics).
- Trip route points stored in Cassandra (write-heavy time-series data).

Regional Deployment

Uber operates in hundreds of cities. Each city is semi-independent:

```
┌──────────────────────────────────────────────────┐
│                Global Services                   │
│  (Accounts, Payments, Analytics)                 │
└──────────────────────────────────────────────────┘
                   │
                   │
     ┌─────────────┼──────────────────┐
     │             │                  │
┌────▼──────┐  ┌──▼────────────┐  ┌──▼────────────┐
│ US-West   │  │ US-East       │  │ Europe        │
│ Region    │  │ Region        │  │ Region        │
│           │  │               │  │               │
│ - SF      │  │ - NYC         │  │ - London      │
│ - LA      │  │ - Chicago     │  │ - Paris       │
│ - Seattle│  │ - Miami       │  │ - Berlin      │
└───────────┘  └───────────────┘  └───────────────┘
```

Each region has its own:

- Geospatial index (drivers are local).
- Dispatch service (matching is local).
- Location service (low latency for real-time updates).

Global services (payments, accounts) are centralized.

27.9 Trade-offs

Decision	Option A	Option B	Our Choice	Why
Geospatial index	Quadtree	Geohash + Redis	Geohash	Simpler to implement, Redis provides speed and scalability.
Driver matching	Nearest driver	Scored matching	Scored	Nearest is not always best (ETA, heading, rating matter).
Pricing	Fixed	Dynamic (surge)	Dynamic	Balances supply and demand in real time.
Location updates	HTTP polling	WebSocket	WebSocket	1.25M updates/s is only feasible with persistent connections.
Trip data	Single DB	MySQL + Cassandra	Hybrid	Trip metadata in MySQL (relational queries), GPS points in Cassandra (write-heavy time-series).
Dispatch scope	Global	Per-city	Per-city	Matching is inherently local; global dispatch adds unnecessary latency.
Fare calculation	Pre-trip estimate only	Post-trip actual	Both	Estimate for transparency; actual for accuracy. Surge locked at request time.

27.10 Common Mistakes

1. Using a relational database for the geospatial index. SQL spatial queries are too slow for 1.25 million location updates per second. Use an in-memory solution like Redis.
2. Matching the nearest driver only. The nearest driver by straight-line distance may be across a river with no bridge. Use ETA (actual driving time) as the primary matching criterion.
3. Ignoring driver double-booking. Without proper locking, two concurrent ride requests could match the same driver. Use distributed locks or atomic operations.
4. Forgetting surge pricing. If you design a fixed-pricing system, the interviewer will ask what happens at peak times when demand exceeds supply. Discuss surge pricing proactively.
5. Not separating the dispatch into geographic regions. A rider in San Francisco should never be matched with a driver in New York. Regional partitioning is not optional.
6. Treating payment as trivial. Payment involves authorization holds, capture, splits, refunds, and fraud detection. At least mention the saga pattern and idempotency.

27.11 Best Practices

1. Lock the surge multiplier at request time. If the rider sees a 1.5x surge and confirms, they pay 1.5x even if the surge increases to 2.0x during the trip. This builds trust.
2. Use exponential backoff for driver matching. Start with a small radius (2 km). If no drivers, expand to 5 km, then 10 km. Do not search the entire city from the start.
3. Store raw GPS data for every trip. This is essential for fare disputes, safety investigations, and route optimization training data.
4. Implement graceful degradation. If the surge pricing service is down, fall back to 1.0x (no surge). If the routing engine is slow, use straight-line distance estimates temporarily.
5. Pre-compute popular ETAs. Airport-to-downtown, major landmarks, business districts -- these routes are requested thousands of times per day. Cache the results.
6. Use hexagonal grids for surge zones. Hexagons have uniform neighbor distances (unlike squares with diagonal neighbors), making surge calculations more consistent.

27.12 Quick Summary

Uber's architecture centers on four pillars: (1) a geohash-based geospatial index in Redis that supports 1.25 million driver location updates per second and millisecond proximity queries; (2)

a scored dispatch system that matches riders with the best available driver based on ETA, rating, heading, and acceptance rate, with distributed locking to prevent double-booking; (3) real-time location tracking over WebSocket connections, enabling both rider and driver to see each other's positions during a trip; and (4) a dynamic surge pricing engine that recalculates supply/demand ratios per geographic zone every 1-2 minutes. The system is partitioned by city/region for locality, with global services for payments and accounts.

27.13 Key Points

- Geohash + Redis provides fast geospatial indexing for finding nearby drivers in milliseconds.
- Scored dispatch considers ETA, rating, heading, and acceptance rate -- not just distance.
- Distributed locking prevents double-booking drivers across concurrent ride requests.
- WebSocket connections deliver real-time location updates every 4 seconds during trips.
- Surge pricing uses hexagonal zones with supply/demand ratios recalculated every 1-2 minutes.
- ETA estimation combines road graph routing (A*) with real-time traffic data from driver GPS signals.
- Payment processing uses authorization holds at request time and capture at trip end, with saga pattern for reliability.
- Regional partitioning keeps dispatch, geospatial index, and location services local to each city.
- Trip route tracking stores GPS points in Cassandra for fare calculation, disputes, and analytics.
- Surge multiplier is locked at request time to build rider trust and prevent surprise charges.

27.14 Practice Questions

1. How would you handle the case where a driver's GPS signal is lost during a trip? How do you calculate the fare if you are missing route data?
2. At 3 AM on New Year's Eve, demand spikes 20x and almost no drivers are available. How does the system behave? What prevents the surge multiplier from going infinitely

high?

3. A rider requests a ride at the exact border of two geohash cells. How do you ensure you find the nearest driver who might be in either cell?
4. How would you implement ride-sharing (UberPool) where multiple riders share a vehicle? How does matching and routing change?
5. If a driver's app crashes mid-trip, what happens? How does the system detect this and recover?

27.15 Exercises

1. Geohash implementation: Write pseudocode to encode a (latitude, longitude) pair into a geohash string of precision 6. Then write the function to find the 8 neighboring geohash cells.
2. Dispatch simulation: Given a list of 10 available drivers (each with lat, lng, rating, heading), a rider's pickup location, and a simple scoring formula, write pseudocode for the dispatch algorithm that returns the best driver.
3. Surge pricing model: Design a surge pricing model for a city divided into 100 hexagonal zones. Define the data structures, the update frequency, and the formula for calculating the surge multiplier. Handle edge cases: what happens when supply is zero?
4. Capacity planning: Calculate the number of Redis nodes needed for the geospatial index in a city with 500,000 concurrent drivers, assuming each driver occupies 100 bytes and each Redis node has 32 GB of RAM. Include overhead for geohash sets.
5. Failure scenario: The Redis cluster holding the geospatial index goes down for 5 minutes. Design a recovery strategy that minimizes rider impact. Can you still match rides? How do you rebuild the index?

27.16 What Is Next?

In this chapter we designed a real-time ride-hailing system with geospatial indexing, driver dispatch, surge pricing, and live location tracking. This was one of the most complex designs in this book, combining real-time systems, geography, economics, and payments into a single coherent architecture. In the next chapter, Chapter 28: Designing a Notification Service, we will design the system responsible for delivering push notifications, emails, and SMS messages at scale -- a service that many of the systems we have designed in previous chapters depend on.

Chapter 28: Design a Notification Service

What You Will Learn

By the end of this chapter, you will understand how to design a scalable notification service that delivers messages across multiple channels. You will learn how to build a priority queue system, implement template engines, handle user preferences, manage delivery retries with exponential backoff, enforce rate limiting, support A/B testing, and track delivery analytics. You will walk away with a complete architecture that handles millions of notifications per day.

Why This Chapter Matters

Notifications are everywhere. Every app you use sends push notifications, emails, or SMS messages. Behind the scenes, these systems must handle enormous scale, respect user preferences, guarantee delivery, and avoid spamming users. A notification service is one of the most common system design interview questions because it touches on queues, distributed systems, template rendering, third-party integrations, and analytics. Understanding this design will sharpen your skills across many system design topics.

---

1. Requirements Gathering

Functional Requirements

Before drawing any diagrams, let us clarify what our notification service must do.

Core features:

- Send notifications via three channels: push notification, SMS, and email
- Support notification templates with variable substitution
- Respect user preferences (opt-in, opt-out, quiet hours, channel preferences)
- Schedule notifications for future delivery
- Track delivery status (sent, delivered, failed, opened)
- Support A/B testing of notification content
- Provide analytics dashboards

Additional features:

- Priority levels (urgent, high, normal, low)
- Rate limiting per user to prevent spamming
- Idempotent delivery (no duplicate notifications)
- Batch notifications (digest mode)

Non-Functional Requirements

- Scale: 10 million notifications per day (approximately 115 per second average, with peaks of 1,000+ per second)
- Latency: Urgent notifications delivered within 1 second, normal within 5 minutes
- Reliability: At-least-once delivery with deduplication
- Availability: 99.9% uptime

Back-of-Envelope Estimation

```
Daily notifications: 10 million
Average per second:  10M / 86,400 = ~115 notifications/sec
Peak (10x average):  ~1,150 notifications/sec

Storage per notification metadata: ~1 KB
Daily storage: 10M x 1 KB = 10 GB
Monthly storage: 300 GB
Yearly storage: 3.6 TB

Template storage: negligible (thousands of templates, ~1 KB each)
```

---

2. High-Level Architecture

Let us start with the big picture and then drill into each component.

```
+-----------------+     +-----------------+     +-----------------+
|   API Clients   |     |  Scheduled Jobs |     |  Event Triggers |
|  (Web, Mobile,  |     |  (Cron-based    |     |  (User signup,  |
|   Internal APIs) |     |   campaigns)    |     |   order placed) |
+--------+--------+     +--------+--------+     +--------+--------+
         |                       |                       |
         v                       v                       v
+--------+-----------------------+-----------------------+--------+
|                     API Gateway / Load Balancer                  |
+--------+-------------------------------------------------------+
         |
         v
+--------+-------------------------------------------------------+
|                    Notification Service API                      |
|  +-------------+  +-------------+  +-------------+  +----------+ |
|  | Validation  |  | Idempotency |  | Rate Limit  |  | Auth &   | |
|  | Layer       |  | Check       |  | Check       |  | Authz    | |
|  +-------------+  +-------------+  +-------------+  +----------+ |
+--------+-------------------------------------------------------+
         |
         v
+--------+-------------------------------------------------------+
|                     Priority Queue System                        |
|  +----------+  +----------+  +----------+  +----------+          |
|  | URGENT   |  | HIGH     |  | NORMAL   |  | LOW      |          |
|  | Queue    |  | Queue    |  | Queue    |  | Queue    |          |
|  +----------+  +----------+  +----------+  +----------+          |
+--------+-------------------------------------------------------+
         |
         v
+--------+-------------------------------------------------------+
|                    Notification Workers                          |
|  +-----------------+  +-----------------+  +-----------------+ |
|  | User Preference |  | Template Engine |  | A/B Test Engine | |
|  | Resolver        |  | (Render Content) |  | (Select Variant) | |
|  +-----------------+  +-----------------+  +-----------------+ |
+--------+-------------------------------------------------------+
         |
         +-------------------+-------------------+
         |                   |                   |
         v                   v                   v
+--------+------+  +---------+-----+  +---------+-----+
| Push Delivery |  | SMS Delivery  |  | Email Delivery|
| Service       |  | Service       |  | Service       |
| (APNs, FCM)   |  | (Twilio,      |  | (SendGrid,    |
|               |  |  Vonage)      |  |  SES)         |
+--------+------+  +---------+-----+  +---------+-----+
         |                   |                   |
         v                   v                   v
+--------+-------------------+-------------------+------+
|                  Delivery Tracker                      |
|  (Status updates, webhooks, delivery receipts)         |
+--------+-----------------------------------------------+
         |
```

```
          v
+--------+-------------------------------------------------+
|                   Analytics Service                      |
|  (Sent, delivered, opened, clicked, failed rates)        |
+----------------------------------------------------------+
```

Data Flow

1. A client sends a notification request to the API
2. The API validates the request, checks idempotency, and enforces rate limits
3. The notification is placed in the appropriate priority queue
4. Workers consume from queues, resolve user preferences, render templates, and select A/B variants
5. Channel-specific delivery services send the actual notification
6. Delivery tracker records status updates
7. Analytics service aggregates metrics

---

3. Detailed Component Design

3.1 Notification Service API

The API is the entry point. It accepts notification requests and performs initial processing.

API Endpoints:

```
POST /api/v1/notifications
  Send a single notification

POST /api/v1/notifications/batch
  Send notifications to multiple users

GET  /api/v1/notifications/{id}/status
  Check delivery status

PUT  /api/v1/users/{id}/preferences
  Update user notification preferences

POST /api/v1/templates
  Create or update a notification template

GET  /api/v1/analytics/notifications
```

```
  Retrieve notification analytics
```

Request Schema:

```json
{
  "notification_id": "uuid-123",         // Client-generated for idempotency
  "user_id": "user-456",
  "template_id": "welcome-email-v2",
  "channels": ["push", "email"],         // Desired channels
  "priority": "high",                    // urgent | high | normal | low
  "data": {                              // Template variables
    "user_name": "Alice",
    "action_url": "https://app.com/verify"
  },
  "schedule_at": "2025-01-15T09:00:00Z", // Optional: future delivery
  "ab_test_id": "test-789"               // Optional: A/B test
}
```

3.2 Idempotency Layer

Duplicate notifications are a terrible user experience. If a client retries a request due to a network timeout, we must not send the notification twice.

How it works:

```
Client sends notification with notification_id = "uuid-123"

Step 1: Check idempotency store
  Key: "idempotency:uuid-123"
  If EXISTS -> return cached response (already processed)
  If NOT EXISTS -> continue processing

Step 2: Process the notification

Step 3: Store result in idempotency store
  SET "idempotency:uuid-123" = response
  EXPIRE after 24 hours

+-------------------+     +-----------------+
| Incoming Request  |---->| Redis Idempotency|
| id = "uuid-123"   |     | Store            |
+-------------------+     +--------+--------+
                                   |
                          +--------+--------+
                          | Key exists?      |
                          +--------+--------+
                          |YES              |NO
                          v                 v
```

```
                    Return cached     Process request
                    response          and store result
```

We use Redis for the idempotency store because lookups are fast (sub-millisecond) and we can set automatic expiration on keys.

3.3 Priority Queue System

Not all notifications are equal. A two-factor authentication code is urgent. A weekly digest is low priority. Our queue system must handle this.

Queue Architecture:

```
                     Notification arrives
                            |
                     +------+------+
                     | Priority    |
                     | Router      |
                     +------+------+
                            |
           +-------+--------+--------+-------+
           |       |                 |       |
           v       v                 v       v
      +--------+ +------+     +--------+ +-----+
      |URGENT  | |HIGH  |     |NORMAL  | |LOW  |
      |Queue   | |Queue |     |Queue   | |Queue|
      |Weight:8| |Wt: 4 |     |Wt: 2   | |Wt:1 |
      +--------+ +------+     +--------+ +-----+
           |       |                 |       |
           +-------+--------+-------+-------+
                            |
                     Workers consume using
                     weighted round-robin
```

Implementation with Kafka:

We use Apache Kafka with four topics, one per priority level. Consumer groups use weighted consumption to ensure higher-priority messages are processed first.

```
Topics:
  notifications.urgent   (8 partitions, consumed first)
  notifications.high     (8 partitions)
  notifications.normal   (16 partitions)
  notifications.low      (4 partitions)

Consumer group: notification-workers
  - 20 worker instances
  - Workers poll urgent queue 8x, high 4x, normal 2x, low 1x
```

Why not a single queue with priority field?

A single queue with millions of messages makes priority sorting expensive. Separate queues let us control consumption rates independently. If the urgent queue backs up, we can add workers specifically for that queue.

3.4 Scheduling Service

For scheduled notifications (send this email at 9 AM tomorrow), we need a reliable scheduler.

```
+--------------------+
| Scheduled          |
| Notification       |
| Request            |
+---------+----------+
          |
          v
+---------+----------+     +------------------+
| Scheduler Service  |---->| Scheduled Store  |
| (Write to store)   |     | (Database table) |
+--------------------+     +---------+---------+
                                     |
                           +---------+---------+
                           | Scheduler Worker |
                           | (Polls every     |
                           |  minute)         |
                           +---------+---------+
                                     |
                           Check: schedule_at <= NOW
                           AND status = 'pending'
                                     |
                                     v
                           Enqueue to Priority Queue
                           Mark status = 'queued'
```

Database Schema for Scheduled Notifications:

```sql
CREATE TABLE scheduled_notifications (
    id              UUID PRIMARY KEY,
    notification_id UUID NOT NULL,
    user_id         VARCHAR(64) NOT NULL,
    template_id     VARCHAR(128) NOT NULL,
    channels        JSONB NOT NULL,
    priority        VARCHAR(16) NOT NULL,
    data            JSONB NOT NULL,
    schedule_at     TIMESTAMP NOT NULL,
    status          VARCHAR(16) DEFAULT 'pending',
    created_at      TIMESTAMP DEFAULT NOW(),
```

```
    INDEX idx_schedule_at_status (schedule_at, status)
);
```

The scheduler worker runs every minute, queries for notifications whose `schedule_at` has passed, enqueues them, and marks them as processed. The index on `(schedule_at, status)` ensures this query is fast even with millions of rows.

3.5 User Preference Service

Users must be able to control what notifications they receive, through which channels, and when.

Preference Model:

```json
{
  "user_id": "user-456",
  "global_enabled": true,
  "quiet_hours": {
    "enabled": true,
    "start": "22:00",
    "end": "07:00",
    "timezone": "America/New_York"
  },
  "channels": {
    "push": { "enabled": true },
    "email": { "enabled": true, "frequency": "immediate" },
    "sms": { "enabled": false }
  },
  "categories": {
    "marketing": { "enabled": false },
    "transactional": { "enabled": true },
    "social": { "enabled": true, "channels": ["push"] }
  },
  "digest": {
    "enabled": true,
    "frequency": "daily",
    "time": "09:00"
  }
}
```

Preference Resolution Flow:

```
Notification for user-456, category="social", channels=["push","email"]
         |
         v
+--------+-----------+
| Load User Prefs    |
```

```
| (Cache in Redis,   |
|  fallback to DB)   |
+--------+-----------+
         |
         v
+--------+-----------+
| Global enabled?    |---NO---> Drop notification
+--------+-----------+
         |YES
         v
+--------+-----------+
| Category enabled?  |---NO---> Drop notification
+--------+-----------+
         |YES
         v
+--------+-----------+
| Channel filtering  |
| User wants: push   |
| Requested: push,   |
|   email             |
| Result: push only  |
+--------+-----------+
         |
         v
+--------+-----------+
| Quiet hours check  |
| Current time in    |
| user timezone:     |
| 23:30 EST          |
| Quiet: 22:00-07:00 |---IN QUIET HOURS---> Queue for 07:00
+--------+-----------+
         |NOT IN QUIET HOURS
         v
    Proceed to template rendering
```

3.6 Template Engine

Templates let us separate notification content from delivery logic. Marketing teams can update message copy without deploying code.

Template Storage:

```json
{
  "template_id": "order-confirmation-v3",
  "category": "transactional",
  "channels": {
    "push": {
      "title": "Order Confirmed!",
      "body": "Hi {{user_name}}, your order #{{order_id}} is confirmed."
```

```
    },
    "email": {
      "subject": "Your order #{{order_id}} is confirmed",
      "html_body": "<h1>Thanks, {{user_name}}!</h1><p>Order total: {{total}}</p>",
      "text_body": "Thanks, {{user_name}}! Order total: {{total}}"
    },
    "sms": {
      "body": "Order #{{order_id}} confirmed. Total: {{total}}"
    }
  },
  "version": 3,
  "created_at": "2025-01-10T12:00:00Z"
}
```

Template Rendering Process:

```
Input:
  template_id = "order-confirmation-v3"
  channel = "email"
  data = { "user_name": "Alice", "order_id": "12345", "total": "$99.00" }

Step 1: Load template from cache (Redis) or database
Step 2: Select channel-specific content
Step 3: Substitute variables using Mustache/Handlebars syntax
Step 4: Validate output (no unresolved {{variables}})

Output:
  subject: "Your order #12345 is confirmed"
  html_body: "<h1>Thanks, Alice!</h1><p>Order total: $99.00</p>"
  text_body: "Thanks, Alice! Order total: $99.00"
```

Templates are versioned. When you update a template, the old version remains available. Scheduled notifications that were created with version 2 will still render with version 2, even if version 3 now exists.

3.7 A/B Testing Engine

A/B testing lets you compare different notification variants to see which performs better.

```
+--------------------+
| Notification with  |
| ab_test_id="t-789" |
+---------+----------+
          |
          v
+---------+----------+
| A/B Test Config    |
| Test "t-789":      |
```

```
|   Variant A: 50%   |
|     template: v1   |
|   Variant B: 50%   |
|     template: v2   |
+--------+-----------+
         |
         v
+--------+-----------+
| Consistent Hash    |
| hash(user_id) %100 |
| Result: 42         |
| 42 < 50 -> A       |
+--------+-----------+
         |
         v
+--------+-----------+
| Record Assignment  |
| user-456 -> A      |
| (for analytics)    |
+--------+-----------+
         |
         v
    Render Variant A template
```

Key design decisions for A/B testing:

1. Consistent assignment: The same user always sees the same variant. We use a hash of the user ID, not random selection, so a user who receives multiple notifications from the same test always gets the same variant.
2. Statistical tracking: We track delivery rate, open rate, and click rate per variant. The analytics service computes statistical significance.
3. Gradual rollout: Start with 10/90 split, then move to 50/50 once the new variant looks safe.

3.8 Channel Delivery Services

Each channel has its own delivery service that handles the specifics of that channel.

```
+-------------------------------------------------------------+
|                 Push Notification Service                    |
|                                                             |
|  +------------------+     +---------------------------+    |
|  | Device Token     |     | Platform Router           |    |
|  | Registry         |     |   iOS -> APNs             |    |
|  | (user -> tokens) |     |   Android -> FCM          |    |
|  +------------------+     |   Web -> Web Push API     |    |
|                           +---------------------------+    |
|                                                             |
|  Handles: Token refresh, invalid token cleanup,             |
```

```
|           payload size limits (4KB APNs, 4KB FCM)         |
+-----------------------------------------------------------+

+-----------------------------------------------------------+
|                    SMS Delivery Service                     |
|                                                             |
|  +------------------+     +---------------------------+   |
|  | Phone Number     |     | Provider Router           |   |
|  | Validator        |     |   US/CA -> Twilio         |   |
|  | (E.164 format)   |     |   EU -> Vonage            |   |
|  +------------------+     |   Asia -> Local provider  |   |
|                           +---------------------------+   |
|                                                             |
|  Handles: Character limits (160 GSM, 70 Unicode),         |
|           country code routing, opt-out (STOP keyword)    |
+-----------------------------------------------------------+

+-----------------------------------------------------------+
|                   Email Delivery Service                    |
|                                                             |
|  +------------------+     +---------------------------+   |
|  | Email Validator  |     | Provider Router           |   |
|  | (MX record check)|     |   Transactional -> SES    |   |
|  +------------------+     |   Marketing -> SendGrid   |   |
|                           +---------------------------+   |
|                                                             |
|  Handles: DKIM/SPF/DMARC, bounce processing,              |
|           unsubscribe headers, attachment limits            |
+-----------------------------------------------------------+
```

Why separate services per channel?

Each channel has unique constraints, failure modes, and rate limits. Push notifications can fail because a device token expired. SMS can fail because a phone number is invalid. Email can fail because the receiving server is temporarily down. Separate services let us handle each channel's quirks independently and scale them separately.

3.9 Retry with Exponential Backoff

Notifications can fail. The third-party provider might be down, the user's device might be unreachable, or we might hit a rate limit. We need smart retries.

```
Attempt 1: Send immediately
  Failed? Wait 1 second

Attempt 2: Retry after 1s
  Failed? Wait 2 seconds

Attempt 3: Retry after 2s
  Failed? Wait 4 seconds
```

```
Attempt 4: Retry after 4s
  Failed? Wait 8 seconds

Attempt 5: Retry after 8s
  Failed? Move to Dead Letter Queue

Formula: delay = base_delay * 2^(attempt - 1) + random_jitter

Jitter prevents all retries from hitting the provider at the same time
(thundering herd problem).
```

Retry Flow:

```
+-------------------+
| Send Notification |
+--------+----------+
         |
    +----+----+
    | Success? |
    +----+----+
    YES  |    NO
     |   |     |
     v   |     v
  Record |  +--+------------------+
  success|  | Retryable error?    |
         |  | (timeout, 5xx,      |
         |  |  rate limited)      |
         |  +--+------------------+
         |  YES |            NO |
         |      v               v
         |  +---+----------+  Record permanent
         |  | Max retries  |  failure
         |  | reached?     |
         |  +---+----------+
         |  NO  |       YES |
         |      v           v
         |  Add back to   Move to Dead
         |  queue with     Letter Queue
         |  delay          (DLQ)
         |
         v
      Done
```

Dead Letter Queue (DLQ): Notifications that fail all retry attempts go to a DLQ. Operations teams monitor the DLQ and can manually retry or investigate failures. Common DLQ reasons include invalid device tokens, deactivated phone numbers, and permanent email bounces.

3.10 Rate Limiting Per User

We must protect users from being spammed, even if our own systems or clients try to send too many notifications.

```
Rate Limiting Rules:
  - Max 5 push notifications per user per hour
  - Max 3 SMS per user per day
  - Max 10 emails per user per day
  - Max 1 notification of same type per user per 5 minutes

+-------------------+
| Notification for  |
| user-456          |
+--------+----------+
         |
         v
+--------+----------+     +------------------+
| Rate Limit Check  |---->| Redis Counters   |
| (per user,        |     |                  |
|  per channel,     |     | push:user-456:   |
|  per type)        |     |   hour -> 3      |
+--------+----------+     | sms:user-456:    |
         |                 |   day -> 1       |
    +----+----+            +------------------+
    | Within  |
    | limits? |
    +----+----+
    YES  |    NO
     |   |     |
     v   |     v
  Continue|  +--+------------------+
  sending |  | Priority override?  |
          |  | (urgent bypasses    |
          |  |  some limits)       |
          |  +--+------------------+
          |  YES |            NO |
          |      v               v
          |  Continue         Drop or defer
          |  sending          notification
          |
          v
       Proceed to delivery
```

Urgent notifications (like security alerts or OTP codes) can bypass rate limits. But even urgent notifications have a hard cap to prevent abuse.

3.11 Delivery Tracking and Analytics

Tracking every notification through its lifecycle gives us the data to improve delivery rates and user engagement.

Notification Lifecycle States:

```
CREATED --> QUEUED --> PROCESSING --> SENT --> DELIVERED --> OPENED --> CLICKED
                          |            |
                          v            v
                       FILTERED     FAILED --> RETRYING --> SENT
                       (user prefs)    |
                                       v
                                   DEAD_LETTERED
```

Analytics Data Model:

```sql
CREATE TABLE notification_events (
    event_id        UUID PRIMARY KEY,
    notification_id UUID NOT NULL,
    user_id         VARCHAR(64) NOT NULL,
    channel         VARCHAR(16) NOT NULL,
    event_type      VARCHAR(32) NOT NULL,
    ab_test_id      VARCHAR(64),
    ab_variant      VARCHAR(16),
    metadata        JSONB,
    created_at      TIMESTAMP DEFAULT NOW()
);

-- Partitioned by created_at for efficient time-range queries
-- Indexed on (notification_id), (user_id, created_at)
```

Analytics Dashboard Metrics:

```
+-----------------------------------------------------------------+
|              Notification Analytics Dashboard                   |
+-----------------------------------------------------------------+
| Time Range: Last 7 Days                                         |
+-----------------------------------------------------------------+
|                                                                 |
| Total Sent: 70,000,000    Delivery Rate: 97.2%                  |
|                                                                 |
| By Channel:                                                     |
|   Push:  40M sent | 38.5M delivered | 12M opened | 96.3% DR  |
|   Email: 25M sent | 24.5M delivered | 8M opened  | 98.0% DR  |
|   SMS:    5M sent |  4.8M delivered | N/A         | 96.0% DR  |
|                                                                 |
| Top Failure Reasons:                                            |
|   Invalid device token:  45%                                    |
|   Email bounce:          25%                                    |
|   Provider timeout:      15%                                    |
|   Rate limited:          10%                                    |
|   Other:                  5%                                    |
|                                                                 |
| A/B Test Results (test-789):                                    |
|   Variant A: 52% open rate (p < 0.05, significant)              |
|   Variant B: 48% open rate                                      |
```

```
+-------------------------------------------------------------+
```

---

4. Complete Architecture Diagram

Here is the full system architecture with all components connected.

```
+-------------------------------------------------------------------+
|                        CLIENTS                                     |
|  +----------+  +-------------+  +-------------+  +------------+  |
|  | Mobile   |  | Web App     |  | Internal    |  | Cron Jobs  |  |
|  | App      |  |             |  | Services    |  | (Campaigns)|  |
|  +----+-----+  +------+------+  +------+------+  +-----+------+  |
+-------|---------------|---------------|---------------|------------+
        |               |               |               |
        v               v               v               v
+-------+---------------+---------------+---------------+-----------+
|                    API Gateway / Load Balancer                     |
|                   (Rate limiting, auth, routing)                   |
+-----------------------------------+-------------------------------+
                                    |
                                    v
+-----------------------------------+-------------------------------+
|                    NOTIFICATION SERVICE API                        |
|  +--------+  +-----------+  +----------+  +----------+  +------+  |
|  | Auth   |  | Validate  |  | Idempot- |  | Rate     |  | Pref |  |
|  | Check  |->| Request   |->| ency     |->| Limit    |->| Check|  |
|  +--------+  +-----------+  | Check    |  | Check    |  +--+---+  |
|                             +----------+  +----------+     |      |
+-----------------------------------+-------------------------+------+
                                    |                         |
                        +---------+--------+           +-----+-----+
                        |                  |           | Scheduler |
                        v                  v           | Service   |
              +----------+--+    +----------+--+        +-----+-----+
              | Immediate   |    | Scheduled    |             |
              | Queue       |    | Store (DB)   |<------------+
              +----------+--+    +-------------+    (polls every min)
                        |
                        v
+-----------------------+-----------------------------------------+
|                   PRIORITY QUEUE (Kafka)                          |
|  +---------+  +--------+  +---------+  +--------+               |
|  | URGENT  |  | HIGH   |  | NORMAL  |  | LOW    |               |
|  | topic   |  | topic  |  | topic   |  | topic  |               |
|  +---------+  +--------+  +---------+  +--------+               |
+-----------------------+-----------------------------------------+
                        |
                        v
```

```
+-----------------------+-----------------------------------------+
|                 NOTIFICATION WORKERS (20 instances)               |
|                                                                   |
|  1. Consume from priority queues (weighted round-robin)           |
|  2. Resolve user preferences (Redis cache -> DB fallback)         |
|  3. Select A/B test variant (consistent hashing)                  |
|  4. Render template (variable substitution)                       |
|  5. Route to channel delivery service                             |
+------+-------------------+-------------------+--------------------+
       |                   |                   |
       v                   v                   v
+------+------+    +-------+------+    +-------+------+
| Push        |    | SMS          |    | Email        |
| Delivery    |    | Delivery     |    | Delivery     |
| Service     |    | Service      |    | Service      |
+------+------+    +-------+------+    +-------+------+
       |                   |                   |
       v                   v                   v
+------+------+    +-------+------+    +-------+------+
| APNs / FCM  |    | Twilio /     |    | SES /        |
| / Web Push  |    | Vonage       |    | SendGrid     |
+------+------+    +-------+------+    +-------+------+
       |                   |                   |
       +-------------------+-------------------+
                           |
                           v
+------+-------------------------------------------------------+
|                DELIVERY TRACKER                                |
|  +-------------+  +--------------+  +------------------+  |
|  | Status DB   |  | Webhook      |  | Retry Manager    |  |
|  | (events)    |  | Receiver     |  | (exp. backoff)   |  |
|  +-------------+  +--------------+  +------------------+  |
+------+-------------------------------------------------------+
       |
       v
+------+-------------------------------------------------------+
|                ANALYTICS SERVICE                               |
|  +------------------+  +------------------+               |
|  | Real-time Metrics|  | Batch Analytics  |               |
|  | (Redis counters) |  | (Data warehouse) |               |
|  +------------------+  +------------------+               |
+-----------------------------------------------------------+

+-----------------------+  +--------------------------------+
| SUPPORTING SERVICES   |  | DATA STORES                    |
| +-------------------+ |  | +-------------------+          |
| | User Preference   | |  | | PostgreSQL        |          |
| | Service           | |  | | (preferences,     |          |
| +-------------------+ |  | |  templates,       |          |
| | Template Service  | |  | |  schedules)       |          |
| +-------------------+ |  | +-------------------+          |
| | A/B Test Service  | |  | | Redis             |          |
| +-------------------+ |  | | (cache, rate      |          |
| | Device Token      | |  | |  limits,          |          |
| | Registry          | |  | |  idempotency)     |          |
```

```
| +-------------------+ |  | +-------------------+          |
+-----------------------+  | | Kafka             |          |
                           | | (priority queues)  |          |
                           | +-------------------+          |
                           | | ClickHouse /       |          |
                           | | Analytics DB       |          |
                           | +-------------------+          |
                           +--------------------------------+
```

---

5. Database Schema

Core Tables

```sql
-- Notification requests
CREATE TABLE notifications (
    id              UUID PRIMARY KEY,
    user_id         VARCHAR(64) NOT NULL,
    template_id     VARCHAR(128) NOT NULL,
    channels        JSONB NOT NULL,
    priority        VARCHAR(16) NOT NULL DEFAULT 'normal',
    data            JSONB NOT NULL,
    ab_test_id      VARCHAR(64),
    status          VARCHAR(32) NOT NULL DEFAULT 'created',
    created_at      TIMESTAMP DEFAULT NOW(),
    updated_at      TIMESTAMP DEFAULT NOW()
);

-- Notification delivery attempts per channel
CREATE TABLE notification_deliveries (
    id              UUID PRIMARY KEY,
    notification_id UUID REFERENCES notifications(id),
    channel         VARCHAR(16) NOT NULL,
    status          VARCHAR(32) NOT NULL,
    provider        VARCHAR(64),
    provider_msg_id VARCHAR(256),
    attempt_count   INT DEFAULT 1,
    error_message   TEXT,
    sent_at         TIMESTAMP,
    delivered_at    TIMESTAMP,
    opened_at       TIMESTAMP,
    clicked_at      TIMESTAMP,
    created_at      TIMESTAMP DEFAULT NOW()
);

-- User preferences
```

```
CREATE TABLE user_preferences (
    user_id         VARCHAR(64) PRIMARY KEY,
    preferences     JSONB NOT NULL,
    updated_at      TIMESTAMP DEFAULT NOW()
);

-- Templates
CREATE TABLE notification_templates (
    template_id     VARCHAR(128) NOT NULL,
    version         INT NOT NULL,
    category        VARCHAR(64) NOT NULL,
    content         JSONB NOT NULL,
    created_at      TIMESTAMP DEFAULT NOW(),
    PRIMARY KEY (template_id, version)
);

-- Device tokens for push notifications
CREATE TABLE device_tokens (
    id              UUID PRIMARY KEY,
    user_id         VARCHAR(64) NOT NULL,
    platform        VARCHAR(16) NOT NULL, -- ios, android, web
    token           TEXT NOT NULL,
    is_active       BOOLEAN DEFAULT TRUE,
    last_used_at    TIMESTAMP,
    created_at      TIMESTAMP DEFAULT NOW(),
    UNIQUE (user_id, token)
);
```

---

6. Scaling Considerations

Horizontal Scaling

```
Component          | How to Scale
-------------------+---------------------------------------------
API Servers        | Add more instances behind load balancer
Priority Queues    | Add Kafka partitions, add consumer instances
Workers            | Add more worker instances (stateless)
Push Service       | Scale independently per platform
SMS Service        | Scale based on geographic load
Email Service      | Scale based on send volume
Redis              | Redis Cluster for sharding
PostgreSQL         | Read replicas for preferences/templates
Analytics DB       | Column-store DB (ClickHouse) for analytics
```

Handling Traffic Spikes

Major events (Black Friday, New Year) can cause 10-100x normal traffic.

Strategies:

1. Pre-warm queues: For known campaigns, pre-generate notifications and queue them
2. Auto-scaling workers: Kubernetes HPA scales workers based on queue depth
3. Backpressure: If queues are too deep, reject low-priority notifications
4. Circuit breakers: If a provider is down, stop sending and buffer messages

---

Common Mistakes

1. No idempotency. Without deduplication, retries send duplicate notifications. Users get the same message twice or more. Always use client-generated IDs.
2. Single queue for all priorities. A massive marketing campaign floods the queue, and urgent security alerts get stuck behind millions of marketing emails. Use separate queues per priority.
3. Ignoring user preferences. Sending notifications to users who opted out violates trust and potentially regulations like GDPR and CAN-SPAM. Always check preferences before sending.
4. No retry backoff. Retrying immediately and repeatedly when a provider is down creates a thundering herd and may get your account rate-limited or banned. Use exponential backoff with jitter.
5. Tight coupling to providers. Hardcoding Twilio or SendGrid everywhere makes it impossible to switch providers. Abstract each channel behind an interface.
6. Synchronous delivery. Trying to send the notification in the API request path makes the API slow and unreliable. Always queue notifications and process them asynchronously.

Best Practices

1. Use separate delivery services per channel. Each channel has different failure modes, rate limits, and constraints. Independent services let you scale and debug each channel separately.
2. Implement circuit breakers for external providers. When a provider goes down, stop hammering it. Buffer notifications and retry when the circuit closes.

3. Cache user preferences aggressively. Preferences change rarely but are checked on every notification. Cache them in Redis with a TTL and invalidate on update.
4. Monitor delivery rates by channel and provider. A sudden drop in delivery rate for push notifications might mean your APNs certificate expired. Set up alerts on delivery rate thresholds.
5. Design templates to degrade gracefully. If a variable is missing, show a sensible default rather than crashing. Use fallback values in templates.
6. Log every state transition. Every notification should have a complete audit trail: created, queued, processed, sent, delivered, opened. This is essential for debugging and compliance.

Quick Summary

A notification service accepts requests from clients, validates them, checks for duplicates, enforces rate limits, queues them by priority, resolves user preferences, renders templates, selects A/B variants, and dispatches through channel-specific delivery services. Failed deliveries are retried with exponential backoff. Every state transition is tracked for analytics and debugging.

Key Points

- Use separate priority queues (not a single sorted queue) for different urgency levels
- Idempotency keys prevent duplicate notification delivery on retries
- User preference resolution must happen before rendering and sending
- Each delivery channel (push, SMS, email) should be an independent service
- Exponential backoff with jitter prevents thundering herd on retries
- Rate limiting per user prevents notification spam
- A/B testing uses consistent hashing so users always see the same variant
- Dead letter queues catch permanently failed notifications for manual review
- Templates separate content from delivery logic, enabling non-engineer updates

---

Practice Questions

1. How would you handle a scenario where a user has multiple devices and should receive a push notification on all of them? What happens if one device token is invalid?
2. A marketing team wants to send 50 million promotional emails over the weekend. How would you design the system to handle this batch without impacting transactional notifications?
3. Your SMS provider reports a 30% failure rate for messages to a specific country. How would you detect this, alert on it, and route around the problem?
4. A notification contains time-sensitive information (a flash sale ending in 2 hours). How would you ensure the notification is not delivered after the sale ends?
5. How would you implement notification grouping (showing "Alice and 5 others liked your post" instead of 6 separate notifications)?

Exercises

Exercise 1: Design the Rate Limiter

Design a Redis-based rate limiter that enforces the following rules for notifications:

- Maximum 5 push notifications per user per hour
- Maximum 10 emails per user per day
- Maximum 1 notification of the same template per user per 10 minutes

Write pseudocode for the rate limit check function. Consider what happens when Redis is temporarily unavailable.

Exercise 2: Build a Retry Simulator

Write a program (in any language) that simulates the retry logic for notification delivery. Your simulator should:

- Accept a failure probability (e.g., 30%)
- Implement exponential backoff with jitter
- Track how many attempts each notification needs
- Calculate the average delivery time across 1000 notifications
- Report how many notifications end up in the dead letter queue

Exercise 3: Design the Preference Resolution Cache

Design a caching strategy for user preferences that balances freshness with performance. Address:

- How to invalidate the cache when a user updates preferences
- How to handle cache misses without overloading the database
- How to prevent a cache stampede when Redis restarts
- What default preferences to use for a new user who has not set any

---

What Is Next?

Now that you understand how to design a notification service that handles multiple channels, priorities, and user preferences, the next chapter will tackle another critical infrastructure component: a distributed rate limiter. You will learn the algorithms (token bucket, sliding window) and implementation details (Redis, Lua scripts) that power rate limiting at scale. Rate limiting is not just a feature of notification services -- it protects every API and service in your architecture.

Chapter 29: Design a Distributed Rate Limiter

What You Will Learn

By the end of this chapter, you will understand how to design a distributed rate limiter that protects your services from being overwhelmed. You will learn two core algorithms in depth -- token bucket and sliding window counter -- and implement them with Redis. You will see how to handle race conditions with Lua scripts, build a configurable rule engine, design multi-tier rate limiting (per user, per API, and global), and create a complete architecture that works across multiple servers.

Why This Chapter Matters

Rate limiting is the immune system of your infrastructure. Without it, a single misbehaving client can take down your entire service. Every major API (GitHub, Stripe, Twitter, AWS) enforces rate limits. In system design interviews, rate limiting comes up both as a standalone question and as a component of larger designs. Understanding the algorithms, trade-offs, and distributed implementation details will make you stronger in any interview.

---

1. Requirements Gathering

Functional Requirements

- Limit requests by user ID, IP address, or API key
- Support configurable rules (e.g., 100 requests per minute per user)

- Return appropriate HTTP responses when limits are exceeded
- Support multiple rate limiting tiers (user-level, API-level, global)
- Allow different limits for different API endpoints
- Provide rate limit headers in responses (remaining quota, reset time)

Non-Functional Requirements

- Low latency: Rate limit check must complete in under 1 millisecond
- High availability: If the rate limiter is down, requests should still be processed (fail-open vs fail-closed)
- Distributed: Must work consistently across multiple application servers
- Accuracy: Minor over-counting is acceptable; significant under-counting is not
- Scale: Handle 1 million+ requests per second across all services

Back-of-Envelope Estimation

```
Total requests per second: 1,000,000
Rate limit check per request: 1 (minimum)
Redis operations per check: 2–3

Redis throughput needed: 2–3 million ops/sec
Redis single instance: ~100,000 ops/sec
Redis instances needed: 20–30 (with Redis Cluster)

Memory per rate limit counter: ~100 bytes
Active users at any time: 10 million
Memory for all counters: 10M x 100B = 1 GB
```

---

2. Rate Limiting Algorithms

Before building the distributed system, let us understand the algorithms.

2.1 Token Bucket (Deep Dive)

The token bucket is the most widely used rate limiting algorithm. It is simple, memory-efficient, and allows controlled bursts.

How It Works:

Imagine a bucket that holds tokens. The bucket has a maximum capacity. Tokens are added to the bucket at a fixed rate. When a request arrives, it takes one token from the bucket. If the bucket is empty, the request is rejected.

```
Token Bucket Parameters:
  - capacity:    Maximum tokens the bucket can hold (burst size)
  - refill_rate: Tokens added per second

Example: capacity = 10, refill_rate = 2 tokens/sec

Time 0s:  Bucket = [T T T T T T T T T T]  (10 tokens, full)
          5 requests arrive, 5 tokens consumed
          Bucket = [T T T T T _ _ _ _ _]  (5 tokens left)

Time 1s:  2 tokens added (refill_rate = 2)
          Bucket = [T T T T T T T _ _ _]  (7 tokens)
          3 requests arrive
          Bucket = [T T T T _ _ _ _ _ _]  (4 tokens)

Time 2s:  2 tokens added
          Bucket = [T T T T T T _ _ _ _]  (6 tokens)
          8 requests arrive, only 6 succeed
          Bucket = [_ _ _ _ _ _ _ _ _ _]  (0 tokens)
          2 requests REJECTED

Time 3s:  2 tokens added
          Bucket = [T T _ _ _ _ _ _ _ _]  (2 tokens)
```

Token Bucket Implementation (Pseudocode):

```
class TokenBucket:
    capacity = 10          # max tokens
    refill_rate = 2        # tokens per second
    tokens = 10            # current tokens
    last_refill_time = NOW()

    function allow_request():
        # Step 1: Refill tokens based on elapsed time
        now = NOW()
        elapsed = now - last_refill_time
        tokens_to_add = elapsed * refill_rate
        tokens = MIN(capacity, tokens + tokens_to_add)
        last_refill_time = now

        # Step 2: Check if we have a token
        if tokens >= 1:
            tokens = tokens - 1
            return ALLOW
        else:
            return REJECT
```

Why Token Bucket?

Property	Token Bucket Behavior
Steady rate	Sustained rate equals refill_rate
Bursts	Allows bursts up to capacity
Memory	O(1) per bucket (just 3 values)
Precision	Smooth, not tied to fixed time windows

Redis Implementation of Token Bucket:

```
-- Lua script for atomic token bucket check
-- KEYS[1] = rate limit key (e.g., "ratelimit:user:123")
-- ARGV[1] = capacity
-- ARGV[2] = refill_rate (tokens per second)
-- ARGV[3] = current timestamp (in microseconds)

local key = KEYS[1]
local capacity = tonumber(ARGV[1])
local refill_rate = tonumber(ARGV[2])
local now = tonumber(ARGV[3])

-- Get current state
local state = redis.call('HMGET', key, 'tokens', 'last_refill')
local tokens = tonumber(state[1])
local last_refill = tonumber(state[2])

-- Initialize if first request
if tokens == nil then
    tokens = capacity
    last_refill = now
end

-- Refill tokens
local elapsed = (now - last_refill) / 1000000  -- convert to seconds
local new_tokens = elapsed * refill_rate
tokens = math.min(capacity, tokens + new_tokens)

-- Try to consume a token
local allowed = 0
if tokens >= 1 then
    tokens = tokens - 1
    allowed = 1
end

-- Save state
redis.call('HMSET', key, 'tokens', tokens, 'last_refill', now)
redis.call('EXPIRE', key, math.ceil(capacity / refill_rate) + 1)
```

```
-- Return: allowed (0 or 1), remaining tokens, time until next token
local retry_after = 0
if allowed == 0 then
    retry_after = math.ceil((1 - tokens) / refill_rate)
end

return {allowed, math.floor(tokens), retry_after}
```

2.2 Sliding Window Counter (Deep Dive)

The sliding window counter provides more precise rate limiting than fixed windows without the memory cost of tracking every individual request.

The Problem with Fixed Windows:

```
Fixed Window: 10 requests per minute

Window 1 (0:00 - 0:59):  Requests at 0:50-0:59 = 10 (all near end)
Window 2 (1:00 - 1:59):  Requests at 1:00-1:09 = 10 (all near start)

In the 20-second span from 0:50 to 1:09, user made 20 requests!
The fixed window allowed DOUBLE the intended limit.

     Window 1                Window 2
|............XXXXXXXXXX|XXXXXXXXXX............|
              10 reqs    10 reqs
              ^------- 20 in 20 sec -------^
```

Sliding Window Counter Solution:

The sliding window counter uses a weighted combination of the current and previous window counts.

```
Sliding Window Formula:
  weighted_count = (prev_window_count * overlap_percentage)
                 + current_window_count

Example: Limit = 10 requests per minute

Previous window (0:00 - 0:59): 8 requests
Current window  (1:00 - 1:59): 5 requests
Current time: 1:15 (25% into current window)

Overlap of previous window = 1 - 0.25 = 0.75 (75%)

weighted_count = (8 * 0.75) + 5 = 6 + 5 = 11

11 > 10, so REJECT the request!
```

```
     Previous Window          Current Window
|XXXXXXXX..............|XXXXX..................|
  8 requests              5 requests
         ^-- 75% overlap --^-- 25% elapsed
```

Sliding Window Counter Implementation (Pseudocode):

```
class SlidingWindowCounter:
    limit = 10             # max requests per window
    window_size = 60       # window size in seconds

    function allow_request(user_id):
        now = NOW()
        current_window = floor(now / window_size)
        previous_window = current_window - 1

        # Get counts for current and previous windows
        current_count = redis.GET(key(user_id, current_window)) or 0
        previous_count = redis.GET(key(user_id, previous_window)) or 0

        # Calculate position within current window
        elapsed_in_window = now % window_size
        overlap = 1 - (elapsed_in_window / window_size)

        # Weighted count
        weighted = (previous_count * overlap) + current_count

        if weighted < limit:
            redis.INCR(key(user_id, current_window))
            redis.EXPIRE(key(user_id, current_window), window_size * 2)
            return ALLOW
        else:
            return REJECT
```

Redis Implementation of Sliding Window Counter:

```
-- Lua script for atomic sliding window counter
-- KEYS[1] = current window key
-- KEYS[2] = previous window key
-- ARGV[1] = limit
-- ARGV[2] = window_size (seconds)
-- ARGV[3] = current timestamp

local current_key = KEYS[1]
local previous_key = KEYS[2]
local limit = tonumber(ARGV[1])
local window_size = tonumber(ARGV[2])
local now = tonumber(ARGV[3])

-- Get counts
```

```
local current_count = tonumber(redis.call('GET', current_key) or "0")
local previous_count = tonumber(redis.call('GET', previous_key) or "0")

-- Calculate overlap
local elapsed = now % window_size
local overlap = 1 - (elapsed / window_size)

-- Weighted count
local weighted = math.floor(previous_count * overlap) + current_count

if weighted < limit then
    redis.call('INCR', current_key)
    redis.call('EXPIRE', current_key, window_size * 2)
    return {1, limit - weighted - 1}  -- allowed, remaining
else
    local retry_after = window_size - elapsed
    return {0, 0, retry_after}  -- rejected, 0 remaining, retry after
end
```

2.3 Algorithm Comparison

```
+--------------------+--------------+------------------+-------------+
| Property           | Token Bucket | Sliding Window   | Fixed Window|
+--------------------+--------------+------------------+-------------+
| Memory per user    | O(1)         | O(1)             | O(1)        |
| Burst handling     | Allows burst | Smooth, no burst | Edge burst  |
| Accuracy           | Good         | Very good        | Poor at edge|
| Implementation     | Simple       | Moderate         | Simplest    |
| Best for           | API rate     | Precise limits   | Simple      |
|                    | limiting     | anti-abuse       | use cases   |
+--------------------+--------------+------------------+-------------+
```

When to use which:

- Token bucket: When you want to allow controlled bursts (most API rate limiting)
- Sliding window counter: When you need precise counts without burst tolerance
- Fixed window: Only when simplicity matters more than accuracy

---

3. Distributed Rate Limiting with Redis

3.1 Why Redis?

Rate limiting requires shared state across all application servers. Redis is the standard choice because:

1. Sub-millisecond latency: Rate limit checks must be fast
2. Atomic operations: INCR, Lua scripts prevent race conditions
3. TTL support: Counters automatically expire
4. High throughput: 100K+ operations per second per instance
5. Replication: Redis Sentinel or Cluster for high availability

3.2 Architecture with Redis

```
+-------------------+     +-------------------+     +-------------------+
| App Server 1      |     | App Server 2      |     | App Server 3      |
| +---------------+ |     | +---------------+ |     | +---------------+ |
| | Rate Limiter  | |     | | Rate Limiter  | |     | | Rate Limiter  | |
| | Middleware    | |     | | Middleware    | |     | | Middleware    | |
| +-------+-------+ |     | +-------+-------+ |     | +-------+-------+ |
+---------+---------+     +---------+---------+     +---------+---------+
          |                         |                         |
          +-------------------------+-------------------------+
                                    |
                                    v
                      +---------------+---------------+
                      |         Redis Cluster          |
                      |  +-------+  +-------+  +---+  |
                      |  |Shard 1|  |Shard 2|  | 3 |  |
                      |  |A-F    |  |G-M    |  |N-Z|  |
                      |  +-------+  +-------+  +---+  |
                      +-------------------------------+
```

All application servers talk to the same Redis cluster. This ensures that no matter which server handles a request, the rate limit count is consistent.

3.3 Handling Race Conditions

Without atomicity, two servers can read the same count, both decide the request is allowed, and both increment -- exceeding the limit.

The Race Condition:

```
Time    Server A                    Server B
----    --------                    --------
T1      GET counter = 9
T2                                  GET counter = 9
T3      9 < 10, ALLOW
T4                                  9 < 10, ALLOW
```

```
T5      INCR counter -> 10
T6                                  INCR counter -> 11  (OVER LIMIT!)
```

Solution: Lua Scripts

Redis executes Lua scripts atomically. The entire read-check-increment happens as a single operation.

```
-- Atomic rate limit check with Lua
-- This entire script runs atomically in Redis

local key = KEYS[1]
local limit = tonumber(ARGV[1])
local window = tonumber(ARGV[2])

local current = tonumber(redis.call('GET', key) or "0")

if current < limit then
    -- Atomically increment and set expiry
    local new_count = redis.call('INCR', key)
    if new_count == 1 then
        redis.call('EXPIRE', key, window)
    end
    return {1, limit - new_count}  -- allowed, remaining
else
    local ttl = redis.call('TTL', key)
    return {0, 0, ttl}  -- rejected, 0 remaining, retry after
end
```

With this Lua script, the race condition is eliminated. Redis processes the script atomically -- no other command can run between the GET and the INCR.

Alternative: Redis MULTI/EXEC

For simpler cases, you can use Redis transactions, but Lua scripts are preferred because they can include conditional logic.

3.4 Handling Redis Failures

What happens when Redis is unavailable?

```
+-------------------+
| Request arrives   |
+--------+----------+
         |
         v
+--------+----------+
| Rate limit check  |
```

```
| (Redis call)      |
+--------+----------+
         |
    +----+----+
    | Redis   |
    | healthy?|
    +----+----+
    YES  |    NO
     |   |     |
     v   |     v
  Normal |  +--+------------------+
  check  |  | Fail-open policy    |
         |  | ALLOW the request   |
         |  | Log the failure     |
         |  | Alert ops team      |
         |  +---------------------+
         |
         v
      Continue
```

Fail-open vs fail-closed:

- Fail-open (recommended): Allow requests when Redis is down. You temporarily lose rate limiting but your service stays available.
- Fail-closed: Reject all requests when Redis is down. Your service becomes unavailable, which is usually worse than temporarily exceeding rate limits.

Local fallback:

For extra resilience, keep an in-memory rate limiter as a fallback. It will not be perfectly accurate (each server counts independently), but it provides basic protection.

```
function check_rate_limit(user_id):
    try:
        return redis_rate_limit_check(user_id)
    except RedisConnectionError:
        log.warn("Redis unavailable, using local fallback")
        return local_rate_limit_check(user_id)  # in-memory counter
```

---

4. Rule Engine

Different API endpoints need different limits. Free users get fewer requests than paid users. The rule engine makes this configurable.

4.1 Rule Configuration

```yaml
# rate_limit_rules.yaml

rules:
  # Global rules (apply to all requests)
  - name: "global-rate-limit"
    scope: "global"
    limit: 10000
    window: 1          # per second
    priority: 1        # evaluated first

  # Per-API endpoint rules
  - name: "search-api"
    scope: "endpoint"
    match:
      path: "/api/v1/search"
      method: "GET"
    limit: 30
    window: 60         # 30 per minute
    priority: 2

  - name: "write-api"
    scope: "endpoint"
    match:
      path: "/api/v1/posts"
      method: "POST"
    limit: 10
    window: 60         # 10 per minute
    priority: 2

  # Per-user tier rules
  - name: "free-user"
    scope: "user"
    match:
      tier: "free"
    limit: 100
    window: 3600       # 100 per hour
    priority: 3

  - name: "pro-user"
    scope: "user"
    match:
      tier: "pro"
    limit: 1000
    window: 3600       # 1000 per hour
    priority: 3

  # Per-IP rules (for unauthenticated requests)
  - name: "ip-rate-limit"
    scope: "ip"
    limit: 50
    window: 60         # 50 per minute per IP
    priority: 4
```

4.2 Rule Evaluation

```
Incoming request:
  IP: 192.168.1.100
  User: user-123 (tier: pro)
  Endpoint: POST /api/v1/posts

Rule evaluation (ordered by priority):

1. Global rate limit check
   Key: "rl:global"
   Limit: 10,000/sec
   Current: 5,432 -> PASS

2. Endpoint rate limit check
   Key: "rl:endpoint:POST:/api/v1/posts"
   Limit: 10/min
   Current: 7 -> PASS

3. User tier rate limit check
   Key: "rl:user:user-123"
   Limit: 1,000/hour (pro tier)
   Current: 245 -> PASS

4. IP rate limit check
   Key: "rl:ip:192.168.1.100"
   Limit: 50/min
   Current: 12 -> PASS

All checks pass -> ALLOW request
```

If any rule fails, the request is rejected immediately. There is no need to check remaining rules.

4.3 Rule Storage and Hot Reload

```
+-------------------+     +------------------+
| Rules Config      |     | Rules Config     |
| (YAML file or DB) |     | Service          |
+--------+----------+     +--------+---------+
         |                         |
         v                         v
+--------+----------+     +--------+---------+
| On startup:       |     | On rule change:  |
| Load all rules    |     | Publish event    |
| into memory       |     | to Pub/Sub       |
+--------+----------+     +--------+---------+
         |                         |
```

```
         v                          v
+--------+----------+     +--------+---------+
| In-memory rule    |<----| All servers      |
| cache on each     |     | receive update   |
| app server        |     | and reload rules |
+-------------------+     +------------------+
```

Rules are loaded into memory on each server at startup and refreshed via pub/sub when changes occur. This avoids a database call on every request.

---

5. Multi-Tier Rate Limiting

Production systems need multiple layers of rate limiting. Here is how they work together.

```
+-------------------------------------------------------------------+
|                    MULTI-TIER RATE LIMITING                        |
+-------------------------------------------------------------------+
|                                                                    |
|  Tier 1: GLOBAL                                                   |
|  +----------------------------------------------------------------+ |
|  | Protects the entire system from total overload                | |
|  | Limit: 50,000 requests/second across all users                | |
|  | Key: "rl:global"                                              | |
|  +----------------------------------------------------------------+ |
|                              |                                     |
|  Tier 2: PER-SERVICE                                               |
|  +----------------------------------------------------------------+ |
|  | Protects individual microservices                             | |
|  | Limit: 5,000 req/sec for user-service                         | |
|  | Key: "rl:service:user-service"                                | |
|  +----------------------------------------------------------------+ |
|                              |                                     |
|  Tier 3: PER-ENDPOINT                                              |
|  +----------------------------------------------------------------+ |
|  | Protects expensive operations                                 | |
|  | Limit: 10 req/min for POST /api/search                        | |
|  | Key: "rl:endpoint:POST:/api/search"                           | |
|  +----------------------------------------------------------------+ |
|                              |                                     |
|  Tier 4: PER-USER                                                  |
|  +----------------------------------------------------------------+ |
|  | Fair usage per user based on tier                             | |
|  | Limit: 100 req/hour for free users                            | |
|  | Key: "rl:user:user-123"                                       | |
|  +----------------------------------------------------------------+ |
|                              |                                     |
|  Tier 5: PER-IP                                                    |
```

```
|  +-------------------------------------------------------------+ |
|  | Protects against unauthenticated abuse                        | |
|  | Limit: 50 req/min per IP                                      | |
|  | Key: "rl:ip:192.168.1.100"                                   | |
|  +-------------------------------------------------------------+ |
|                                                                    |
+--------------------------------------------------------------------+

Evaluation order: Global -> Service -> Endpoint -> User -> IP
First failure = REJECT (short-circuit)
```

Why multiple tiers?

Consider these scenarios:

- A single user makes 1000 requests per second (caught by user tier)
- A botnet of 10,000 IPs each makes 10 requests per second (caught by global tier)
- A bug in a client repeatedly hits a search endpoint (caught by endpoint tier)

No single tier catches all abuse patterns. Multiple tiers provide defense in depth.

---

6. HTTP Response Design

When a request is rate-limited, the response must be informative.

```
Allowed Response:
  HTTP/1.1 200 OK
  X-RateLimit-Limit: 100
  X-RateLimit-Remaining: 57
  X-RateLimit-Reset: 1705312800

Rejected Response:
  HTTP/1.1 429 Too Many Requests
  X-RateLimit-Limit: 100
  X-RateLimit-Remaining: 0
  X-RateLimit-Reset: 1705312800
  Retry-After: 45

  {
    "error": "rate_limit_exceeded",
    "message": "You have exceeded your rate limit of 100 requests per hour.",
    "retry_after": 45
  }
```

Header meanings:

- `X-RateLimit-Limit`: The maximum number of requests allowed in the window
- `X-RateLimit-Remaining`: How many requests the client can still make
- `X-RateLimit-Reset`: Unix timestamp when the window resets
- `Retry-After`: Seconds until the client should retry (only on 429 responses)

---

7. Complete Architecture Diagram

```
+-------------------------------------------------------------------+
|                      CLIENT REQUESTS                              |
|  +----------+  +----------+  +----------+  +----------+          |
|  | Mobile   |  | Web App  |  | API      |  | Partner  |          |
|  | Client   |  | Client   |  | Client   |  | API      |          |
|  +----+-----+  +----+-----+  +----+-----+  +----+-----+          |
+-------|-------------|-------------|-------------|------------------+
        |             |             |             |
        v             v             v             v
+-------+-------------+-------------+-------------+-----------------+
|                    API GATEWAY / LOAD BALANCER                    |
|              (First-tier: IP-based rate limiting)                 |
+-------------------------------+-----------------------------------+
                                |
                                v
+-------------------------------+-----------------------------------+
|                    RATE LIMITER MIDDLEWARE                        |
|                                                                   |
|  +------------------+     +------------------+                   |
|  | Request Context  |     | Rule Engine      |                   |
|  | Extractor        |---->| (in-memory rules)|                   |
|  | (user, IP, API   |     | Match request to |                   |
|  |  key, endpoint)  |     | applicable rules |                   |
|  +------------------+     +--------+---------+                   |
|                                    |                              |
|  For each matched rule:            v                              |
|  +------------------+     +--------+---------+                   |
|  | Build Redis Key  |---->| Execute Lua      |                   |
|  | (scope:id:window)|     | Script on Redis  |                   |
|  +------------------+     | (atomic check)   |                   |
|                           +--------+---------+                   |
|                                    |                              |
|                           +--------+---------+                   |
|                           | ALLOW or REJECT  |                   |
|                           | Set response     |                   |
|                           | headers          |                   |
|                           +------------------+                   |
+-------------------------------+-----------------------------------+
                                |
                +---------------+---------------+
                |                               |
```

```
          ALLOWED                            REJECTED
               |                                 |
               v                                 v
+------+-------+--------+          +-------------+--+
| Application Server    |          | 429 Response   |
| (process request      |          | with headers:  |
|  normally)            |          | Retry-After,   |
+------+----------------+          | RateLimit-*    |
       |                           +---------+------+
       v                                     |
  Normal response                            v
  with RateLimit                        Client waits
  headers                              and retries

+-------------------------------------------------------------------+
|                      REDIS CLUSTER                                 |
|                                                                     |
|  +----------+  +----------+  +----------+  +----------+         |
|  | Shard 1  |  | Shard 2  |  | Shard 3  |  | Shard 4  |         |
|  |          |  |          |  |          |  |          |         |
|  | Keys:    |  | Keys:    |  | Keys:    |  | Keys:    |         |
|  | rl:user: |  | rl:user: |  | rl:ip:   |  | rl:global|         |
|  |  A-F     |  |  G-M     |  |  all     |  | rl:svc:  |         |
|  +----+-----+  +----+-----+  +----+-----+  +----+-----+         |
|       |              |              |              |              |
|  +----+-----+  +----+-----+  +----+-----+  +----+-----+         |
|  | Replica  |  | Replica  |  | Replica  |  | Replica  |         |
|  +----------+  +----------+  +----------+  +----------+         |
+-------------------------------------------------------------------+

+-------------------------------------------------------------------+
|                   SUPPORTING COMPONENTS                            |
|                                                                     |
|  +-------------------+  +-------------------+  +---------------+ |
|  | Rules Config DB   |  | Metrics Collector |  | Dashboard     | |
|  | (PostgreSQL)      |  | (Prometheus)      |  | (Grafana)     | |
|  |                   |  |                   |  |               | |
|  | Store and update  |  | Track:            |  | Visualize:    | |
|  | rate limit rules  |  | - Requests/sec    |  | - Rate limit  | |
|  | per endpoint,     |  | - Rejections/sec  |  |   hit rates   | |
|  | user tier, etc.   |  | - Latency of      |  | - Top limited | |
|  |                   |  |   rate checks     |  |   users/IPs   | |
|  +-------------------+  +-------------------+  +---------------+ |
+-------------------------------------------------------------------+
```

8. Advanced Topics

8.1 Distributed Rate Limiting Across Data Centers

When your service runs in multiple data centers, you have two choices:

```
Option A: Global counters (consistent but slower)

+-------------------+            +-------------------+
| Data Center US    |            | Data Center EU    |
| +-------+         |            |         +-------+ |
| | App   +----+    |            |    +----+ App   | |
| +-------+    |    |            |    |    +-------+ |
|              v    |            |    v               |
|        +-----+--+ |            | +--+-----+         |
|        | Redis  +--+----------+--+ Redis   |         |
|        | (sync) | |  Cross-DC  | | (sync) |         |
|        +--------+ |  replication| +--------+         |
+-------------------+            +-------------------+

Pros: Globally accurate counts
Cons: Cross-DC latency on every request (50-100ms)


Option B: Local counters with sync (fast but approximate)

+-------------------+            +-------------------+
| Data Center US    |            | Data Center EU    |
| +-------+         |            |         +-------+ |
| | App   +----+    |            |    +----+ App   | |
| +-------+    |    |            |    |    +-------+ |
|              v    |            |    v               |
|        +-----+--+ |            | +--+-----+         |
|        | Local  | |            | | Local  |         |
|        | Redis  | |  Periodic  | | Redis  |         |
|        +---+----+ |   sync     | +----+---+         |
|            |      +--+-----+--+      |              |
|            +-------->|Sync |<--------+              |
|                   |  |Svc  |  |                     |
|                   +--+-----+--+                     |
+-------------------+            +-------------------+

Pros: Low latency (local Redis)
Cons: Slightly inaccurate (users can exceed limit by up to 2x briefly)
```

Most systems choose Option B and accept the slight inaccuracy. For the total limit, split it across data centers proportional to their traffic.

8.2 Rate Limiting WebSocket Connections

WebSocket connections are long-lived. Rate limiting applies differently:

```
Connection-level:
  - Limit new connections per user: 5 per minute
```

```
  - Limit total active connections per user: 3

Message-level:
  - Limit messages per connection per second: 10
  - Limit message size: 64 KB
```

8.3 Client-Side Rate Limiting

Smart clients implement their own rate limiting to avoid wasting requests:

```
Client-side approach:

1. Read X-RateLimit-Remaining from responses
2. If remaining < threshold, slow down proactively
3. If 429 received, back off using Retry-After header
4. Implement exponential backoff for retries:
   wait = min(base * 2^attempt + jitter, max_wait)
```

Common Mistakes

1. Using fixed windows without understanding boundary issues. Clients can send 2x the intended rate by timing requests at window boundaries. Use sliding windows or token buckets instead.
2. Non-atomic read-then-write in Redis. Separate GET and INCR commands create race conditions. Always use Lua scripts for atomic operations.
3. Fail-closed by default. If Redis is down and you reject all traffic, your rate limiter has become a denial-of-service attack on your own service. Default to fail-open.
4. Same limits for all endpoints. A search endpoint and a health check have vastly different costs. Expensive endpoints need tighter limits.
5. No rate limit headers in responses. Clients cannot implement smart backoff without knowing their remaining quota. Always include `X-RateLimit-*` headers.
6. Forgetting about distributed clock skew. Servers in different data centers may have slightly different clocks. Use Redis server time or tolerate small inaccuracies.

Best Practices

1. Use Lua scripts for atomicity. Every rate limit check should be a single atomic operation. Lua scripts in Redis guarantee this.
2. Include rate limit information in every response. Even successful responses should include `X-RateLimit-Remaining` so clients can self-regulate.
3. Implement multiple tiers. Global, per-service, per-endpoint, per-user, and per-IP tiers catch different abuse patterns.
4. Use token bucket for most API rate limiting. It is simple, memory efficient, and handles bursts gracefully. Sliding window is better when you need strict precision.
5. Monitor and alert on rate limit hit rates. A sudden increase in 429 responses might mean a legitimate user is having problems, not just abuse. Track and investigate.
6. Allow configuration changes without deployments. Store rules in a database or config service with hot reloading. This lets you respond to abuse quickly.

Quick Summary

A distributed rate limiter checks every incoming request against configurable rules using shared counters in Redis. The two main algorithms are token bucket (allows bursts, simple) and sliding window counter (precise, no burst). Lua scripts ensure atomicity. Multi-tier limiting (global, service, endpoint, user, IP) provides defense in depth. The system fails open when Redis is unavailable and includes informative headers so clients can self-regulate.

Key Points

- Token bucket allows controlled bursts and uses O(1) memory per user
- Sliding window counter avoids the boundary problem of fixed windows
- Lua scripts in Redis eliminate race conditions by making read-check-increment atomic
- Multi-tier rate limiting (global, service, endpoint, user, IP) catches different abuse patterns
- Fail-open is usually better than fail-closed when Redis is unavailable
- Rate limit headers (Remaining, Reset, Retry-After) enable smart client behavior
- Rules should be configurable without code deployments
- Local in-memory counters serve as a fallback when Redis is down

---

Practice Questions

1. Your Redis cluster is experiencing high latency (50ms instead of the usual 1ms). How does this affect your rate limiter, and what can you do about it?
2. A sophisticated attacker rotates through thousands of IP addresses to bypass per-IP rate limits. How would you detect and mitigate this?
3. You need to implement a rate limiter for a real-time bidding system where latency above 5ms is unacceptable. How would you modify the architecture?
4. Your API has both read and write endpoints. Reads are cheap, writes are expensive. How would you design the rate limiting rules?
5. A partner integration sends legitimate bursts of 1000 requests in 1 second, then nothing for 59 seconds. How would you configure the rate limiter to allow this pattern while still protecting against abuse?

Exercises

Exercise 1: Implement Token Bucket in Redis

Write a complete Redis Lua script that implements the token bucket algorithm. Test it by simulating 100 requests arriving at different intervals and verifying that the correct number are allowed and rejected. Your script should handle the case where the key does not exist yet (first request).

Exercise 2: Compare Algorithms

Build a simulation (in any language) that compares fixed window, sliding window counter, and token bucket. For each algorithm:

- Simulate 1000 requests with a limit of 10 per minute
- Vary the arrival pattern (uniform, bursty, boundary-crossing)
- Report how many requests each algorithm allows
- Calculate the maximum actual rate achieved in any 60-second window

Exercise 3: Design a Rate Limit Dashboard

Design a monitoring dashboard for your rate limiter. Specify:

- What metrics to collect (counters, histograms, gauges)
- How to identify the top 10 rate-limited users in real time
- How to detect when a legitimate user is being unfairly limited

- What alerts to configure and at what thresholds

What Is Next?

You now understand how to protect your services with a distributed rate limiter. The next chapter tackles a very different kind of system: a web crawler. You will learn how to systematically crawl billions of web pages while being polite, avoiding traps, and handling the immense scale of the internet. The web crawler design will test your ability to think about distributed systems, queues, and deduplication at truly massive scale.

Chapter 30: Designing a Web Crawler

Every time you type a query into Google, Bing, or DuckDuckGo, the results you see were discovered by a web crawler -- a tireless piece of software that visits billions of web pages, downloads their content, and follows every link it finds. Without crawlers, search engines would have nothing to search. But crawling the entire web is one of the hardest distributed systems problems in existence: you must be fast enough to keep up with a constantly changing internet, polite enough not to overwhelm the websites you visit, and smart enough to avoid traps, duplicates, and irrelevant content.

In this chapter we will design a web crawler that can handle billions of pages. Along the way we will tackle URL frontiers, politeness policies, duplicate detection with bloom filters, robots.txt compliance, and distributed crawling architecture. This is a favorite system design interview question because it blends networking, data structures, distributed systems, and practical engineering trade-offs into a single problem.

What You Will Learn

- How a web crawler discovers and downloads pages at massive scale.
- The architecture of the URL frontier and why a simple queue is not enough.
- How bloom filters provide space-efficient duplicate detection.
- Why politeness and robots.txt compliance are non-negotiable.
- BFS vs DFS traversal strategies and their trade-offs.
- How to distribute crawling across hundreds of machines.
- Techniques to detect and avoid crawler traps.

Why This Chapter Matters

Web crawlers are not just for search engines. Companies use crawlers for price monitoring, content aggregation, SEO auditing, machine learning dataset collection, and legal compliance checking. Understanding crawler design teaches you about rate limiting, distributed coordination, deduplication, and working with unreliable external systems -- skills that transfer to dozens of other system design problems.

In interviews, this question tests whether you can think about systems that interact with the outside world at scale, where you do not control the servers you talk to.

30.1 Understanding the Problem

A web crawler starts with a set of URLs, downloads each page, extracts new URLs from the page content, and adds those URLs to the list of pages to visit. Repeat this process billions of times, and you have crawled the web.

Sounds simple, right? The devil is in the details.

The Scale of the Web

Metric	Approximate Value
Indexed pages (Google)	50+ billion
New pages created daily	hundreds of millions
Average page size	~2 MB (with images), ~100 KB (HTML only)
Unique domains	200+ million active

A crawler must download, parse, and index this content while being a good citizen of the internet.

30.2 Requirements

Functional Requirements

#	Requirement	Description
F1	Crawl web pages	Given seed URLs, discover and download web pages by following hyperlinks.
F2	Extract content	Parse HTML to extract text, metadata, and outgoing links.
F3	Respect robots.txt	Honor each domain's crawling rules.
F4	Detect duplicates	Avoid downloading the same page twice.
F5	Store content	Save downloaded pages for indexing or processing.
F6	Freshness	Re-crawl pages periodically to keep content up to date.

Non-Functional Requirements

#	Requirement	Target
NF1	Scalability	Crawl 1 billion pages per day.
NF2	Politeness	Never overwhelm any single domain.
NF3	Robustness	Handle malformed HTML, timeouts, and server errors gracefully.
NF4	Extensibility	Easy to add new content types (images, PDFs, videos).
NF5	Trap avoidance	Detect and escape infinite URL loops and spider traps.

30.3 Back-of-the-Envelope Estimation

Traffic

Metric	Calculation	Value
Pages per day	Given	1 billion
Pages per second	1B / 86,400	~11,600 pages/s
Average HTML size	Estimated	100 KB
Bandwidth (download)	11,600 x 100 KB	~1.16 GB/s

Storage

Metric	Calculation	Value
Raw HTML per day	1B x 100 KB	100 TB/day
Compressed (5:1)	100 TB / 5	~20 TB/day
Metadata per page	~500 bytes	500 GB/day

DNS Lookups

Every URL requires a DNS resolution. At 11,600 pages/s, we need a local DNS cache to avoid overwhelming public DNS servers.

30.4 High-Level Architecture

Here is the big picture of a web crawler:

```
+----------+     +--------------+     +--------------+     +----------------+
|  Seed    |---->|  URL         |---->|  DNS         |---->|  HTML          |
|  URLs    |     |  Frontier    |     |  Resolver    |     |  Downloader    |
+----------+     +--------------+     +--------------+     +----------------+
                       ^                                          |
                       |                                          v
                 +-----------+     +---------------+     +----------------+
                 |  URL      |<----|  Content      |<----|  Content       |
                 |  Filter   |     |  Extractor    |     |  Parser        |
                 +-----------+     +---------------+     +----------------+
                       |                                          |
```

```
                    v                                   v
              +-----------+                       +----------------+
              |  Duplicate|                       |  Content       |
              |  Detector |                       |  Storage       |
              |  (Bloom)  |                       |  (S3/HDFS)     |
              +-----------+                       +----------------+
```

Component Walkthrough

1. Seed URLs -- The starting points. These are a curated list of high-quality, diverse URLs that cover many domains and topics. Examples include top websites from Alexa rankings, government portals, and university pages.

2. URL Frontier -- The heart of the crawler. This is not a simple FIFO queue. It is a priority queue combined with a politeness scheduler (more on this below).

3. DNS Resolver -- Translates domain names to IP addresses. We run our own caching DNS resolver to avoid hitting public DNS too hard.

4. HTML Downloader -- Fetches the page content via HTTP/HTTPS. Handles redirects, timeouts, retries, and different content types.

5. Content Parser -- Validates and cleans the HTML. Rejects malformed or binary content.

6. Content Extractor -- Pulls out the useful data: page text, title, metadata, and most importantly, outgoing links.

7. URL Filter -- Removes unwanted URLs: different file types (.zip, .exe), blacklisted domains, URLs that are too long, etc.

8. Duplicate Detector -- Uses a bloom filter (and optionally a URL database) to check if we have already seen or crawled this URL.

9. Content Storage -- Stores the downloaded HTML and extracted data in a distributed file system (HDFS, S3) for later indexing.

30.5 Deep Dive: The URL Frontier

The URL frontier is the most interesting data structure in a web crawler. It serves two purposes that conflict with each other:

1. Prioritization -- Important pages should be crawled first.

2. Politeness -- We must not hit the same domain too frequently.

Architecture of the URL Frontier

```
                  +-------------------+
                  |  Incoming URLs    |
                  +-------------------+
                          |
                          v
                  +-------------------+
                  |  Prioritizer      |
                  |  (assigns         |
                  |   priority score) |
                  +-------------------+
                          |
            +-------------+-------------+
            |             |             |
            v             v             v
      +---------+   +---------+   +---------+
      | Queue 1 |   | Queue 2 |   | Queue 3 |
      | (High)  |   | (Medium)|   | (Low)   |
      +---------+   +---------+   +---------+
            |             |             |
            +-------------+-------------+
                          |
                          v
                  +-------------------+
                  |  Queue Selector   |
                  |  (weighted random)|
                  +-------------------+
                          |
                          v
                  +-------------------+
                  |  Politeness       |
                  |  Router           |
                  +-------------------+
                          |
            +-------------+-------------+
            |             |             |
            v             v             v
      +---------+   +---------+   +---------+
      | Domain  |   | Domain  |   | Domain  |
      | Queue A |   | Queue B |   | Queue C |
      +---------+   +---------+   +---------+
            |             |             |
            v             v             v
      (one worker per domain queue)
```

Front Queues (Priority)

The front queues handle prioritization. Each URL gets a priority score based on:

- PageRank or similar metric -- How many other pages link to this URL?
- Update frequency -- Pages that change often (news sites) get higher priority.
- Depth from seed -- Pages closer to seed URLs tend to be more important.
- Domain authority -- Well-known domains get priority.

The queue selector picks from higher-priority queues more often using a weighted random or round-robin with bias.

Back Queues (Politeness)

The back queues ensure politeness. Each back queue is dedicated to a single domain (or host). A URL for `example.com` always goes to the same back queue. Each back queue has its own worker thread, and that worker enforces a delay between requests to the same domain.

```
Domain Queue for example.com:
  - Last request: 10:00:05.000
  - Minimum delay: 2 seconds
  - Next allowed request: 10:00:07.000
  - Queue: [/page1, /page2, /page3, ...]
```

This guarantees we never send more than one request every N seconds to any single domain.

30.6 Deep Dive: DNS Resolution

DNS resolution is a hidden bottleneck. Each page requires a DNS lookup, and at 11,600 pages/s, we cannot afford the 10-100ms latency of a public DNS query for every request.

Solution: Local DNS Cache

```
+----------------+     +------------------+     +------------------+
|  Crawler       |---->|  Local DNS Cache |---->|  Public DNS      |
|  Worker        |     |  (in-memory,     |     |  (Google 8.8.8.8)|
|                |     |   TTL-aware)     |     |                  |
+----------------+     +------------------+     +------------------+
```

- Cache DNS results in memory with TTL (time-to-live) awareness.
- Pre-fetch DNS for URLs in the frontier before they are needed.
- Run our own recursive DNS resolver for control and performance.

- Typical DNS cache hit rate: 90%+ because we crawl many pages per domain.

30.7 Deep Dive: Duplicate Detection with Bloom Filters

With billions of URLs, we need a space-efficient way to check "have I seen this URL before?" A hash set would work but requires too much memory.

What Is a Bloom Filter?

A bloom filter is a probabilistic data structure that can tell you:

- Definitely not in the set -- 100% accurate.
- Probably in the set -- Small chance of false positive.

It never has false negatives. If the bloom filter says "not seen," the URL is definitely new.

How It Works

```
URL: "https://example.com/page1"
         |
         v
  +------+------+------+
  | Hash |  Hash|  Hash|   (k=3 hash functions)
  |  h1  |  h2  |  h3  |
  +------+------+------+
     |       |       |
     v       v       v
  Bit Array (m bits):
  [0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0]
           ^       ^                     ^
         h1=2    h2=4                  h3=11

  To check: if ALL k positions are 1, "probably seen."
  If ANY position is 0, "definitely not seen."
```

Sizing the Bloom Filter

For 1 billion URLs with a 1% false positive rate:

Parameter	Value

n (expected items)	1 billion
p (false positive rate)	1%
m (bits needed)	~9.6 billion bits = ~1.2 GB
k (hash functions)	7

Only 1.2 GB of memory to track a billion URLs. A hash set would need 50+ GB.

Content-Based Deduplication

URLs can be different but point to the same content (e.g., with different query parameters or session IDs). For content deduplication, compute a fingerprint (e.g., SimHash or MinHash) of the page content and check it against a separate bloom filter or database.

30.8 Deep Dive: Robots.txt and Politeness

Robots.txt

The `robots.txt` file at the root of every website tells crawlers what they are allowed to crawl:

```
# Example robots.txt for example.com
User-agent: *
Disallow: /admin/
Disallow: /private/
Crawl-delay: 2

User-agent: MyCustomBot
Allow: /public/
Disallow: /
```

Rules our crawler must follow:

1. Before crawling any page on a domain, fetch and parse its `robots.txt`.
2. Cache the `robots.txt` for each domain (refresh every 24 hours).
3. Respect `Disallow` directives -- never crawl forbidden paths.
4. Respect `Crawl-delay` -- wait at least this many seconds between requests.
5. Identify ourselves with a proper `User-Agent` header.

Politeness Beyond Robots.txt

Even if `robots.txt` allows aggressive crawling, a well-behaved crawler should:

Rule	Implementation
Rate limit per domain	Max 1 request per 1-2 seconds per domain
Rate limit per IP	Some domains share IPs; limit per IP too
Respect HTTP 429	Back off when server says "too many requests"
Respect HTTP 503	Server is overloaded; retry later with exponential backoff
Off-peak crawling	For small sites, crawl during their off-peak hours
Connection reuse	Use HTTP keep-alive to reduce connection overhead

---

30.9 BFS vs DFS Traversal

BFS (Breadth-First Search)

BFS explores all links at the current depth before going deeper. This is the standard approach for web crawling.

```
Seed URL (depth 0)
    |
    +-- Link A (depth 1)
    |       |
    |       +-- Link D (depth 2)
    |       +-- Link E (depth 2)
    |
    +-- Link B (depth 1)
    |       |
    |       +-- Link F (depth 2)
    |
    +-- Link C (depth 1)

BFS order: Seed -> A -> B -> C -> D -> E -> F
```

Advantages:

- Discovers important pages (close to seeds) first.

- Natural fit for level-by-level crawling.
- Less likely to get trapped in deep link chains.

Disadvantages:

- Requires large memory for the frontier queue.
- May hit many domains simultaneously at shallow depths.

DFS (Depth-First Search)

DFS follows one link chain as deep as possible before backtracking.

```
DFS order: Seed -> A -> D -> E -> B -> F -> C
```

Advantages:

- Uses less memory (only needs the current path stack).
- Good for focused crawling of a single domain.

Disadvantages:

- Can get trapped in deep link chains or spider traps.
- May miss important pages on other branches.
- Not suitable for general-purpose web crawling.

Verdict: Use BFS for general web crawling. Use DFS only for focused, single-domain crawls with a depth limit.

30.10 Crawler Traps

A crawler trap is a set of URLs that cause the crawler to loop infinitely or generate an unlimited number of pages.

Common Traps

Trap Type	Example	Solution
Infinite query strings	`/page?id=1`, `/page?id=2`, ... forever	Limit unique URLs per domain

Calendar traps	`/calendar/2024/01`, `/calendar/2024/02`, ...	Detect date patterns, set depth limits
Session ID URLs	`/page?sid=abc123` (new session each visit)	Strip known session parameters
Soft 404s	Server returns 200 OK with "page not found" content	Content similarity detection
Redirect loops	A -> B -> C -> A	Track redirect chains, limit to 5 hops
Dynamic content	JavaScript generates infinite links	Set per-domain URL budget

Anti-Trap Strategies

1. URL budget per domain -- Allow at most N URLs (e.g., 10,000) per domain.
2. Depth limit -- Stop following links beyond depth D (e.g., 15) from seed URLs.
3. URL pattern detection -- If URLs from a domain follow a repetitive pattern with only a counter changing, flag it.
4. Manual blacklist -- Maintain a list of known trap domains.
5. Content fingerprinting -- If many pages from the same domain have nearly identical content, slow down or stop crawling that domain.

30.11 Distributed Architecture

A single machine cannot crawl a billion pages per day. We need a distributed system.

```
                        +-------------------+
                        |  Coordinator      |
                        |  (assigns domains |
                        |   to workers)     |
                        +-------------------+
                                |
          +-------------------+-------------------+
          |                   |                   |
          v                   v                   v
    +---------------+   +---------------+   +---------------+
    |  Crawler      |   |  Crawler      |   |  Crawler      |
    |  Worker 1     |   |  Worker 2     |   |  Worker N     |
    |               |   |               |   |               |
    | - URL Frontier|   | - URL Frontier|   | - URL Frontier|
    | - DNS Cache   |   | - DNS Cache   |   | - DNS Cache   |
```

```
    | - Downloader  |   | - Downloader  |   | - Downloader  |
    | - Parser      |   | - Parser      |   | - Parser      |
    +---------------+   +---------------+   +---------------+
            |                   |                   |
            +-------------------+-------------------+
                                |
                                v
                  +------------------------+
                  |  Shared Storage        |
                  |  (HDFS / S3)           |
                  +------------------------+
                  |  Shared Bloom Filter   |
                  |  (Redis / distributed) |
                  +------------------------+
                  |  URL Database          |
                  |  (for persistence)     |
                  +------------------------+
```

Partitioning Strategy

The most common approach is domain-based partitioning: each worker is responsible for a set of domains. This has several advantages:

1. Natural politeness -- Each worker manages rate limiting for its own domains.
2. DNS cache efficiency -- Domains stay on the same worker, so DNS results are cached locally.
3. robots.txt caching -- Each worker caches robots.txt for its domains.
4. No coordination overhead for politeness -- Workers do not need to coordinate rate limits.

Use consistent hashing to assign domains to workers. When a worker discovers a URL for a domain owned by another worker, it sends the URL to that worker via a message queue.

Fault Tolerance

Failure	Recovery
Worker crash	Reassign its domains to other workers. Persist frontier to disk periodically.
DNS resolver down	Fall back to public DNS servers.
Storage unavailable	Buffer downloaded pages locally, flush when storage recovers.
Network partition	Workers continue crawling their local frontier independently.

30.12 Handling Freshness

The web is constantly changing. A crawler must re-visit pages to keep its index fresh.

Re-Crawl Strategies

Strategy	How It Works	Pros	Cons
Uniform	Re-crawl all pages every N days	Simple	Wastes resources on static pages
Frequency-based	Crawl pages that change often more frequently	Efficient	Requires change history
Priority-based	Re-crawl important pages more often	Good use of budget	Need a quality signal
HTTP headers	Use `Last-Modified` and `ETag` headers	Minimal bandwidth for unchanged pages	Not all servers support them

Conditional Requests

```
GET /page HTTP/1.1
Host: example.com
If-Modified-Since: Mon, 01 Jan 2024 00:00:00 GMT
If-None-Match: "abc123"

Response (if unchanged):
HTTP/1.1 304 Not Modified
(no body -- saves bandwidth)
```

Using conditional requests, we can check if a page has changed without downloading the full content.

30.13 Content Processing Pipeline

Once a page is downloaded, it goes through a processing pipeline:

```
+----------------+     +----------------+     +----------------+
|  Raw HTML      |---->|  Encoding      |---->|  HTML          |
|  Download      |     |  Detection     |     |  Cleaning      |
+----------------+     +----------------+     +----------------+
                                                       |
                                                       v
+----------------+     +----------------+     +----------------+
|  Store in      |<----|  Content       |<----|  Link          |
|  Index         |     |  Fingerprint   |     |  Extraction    |
+----------------+     +----------------+     +----------------+
```

1. Encoding Detection -- Determine character encoding (UTF-8, ISO-8859-1, etc.).
2. HTML Cleaning -- Remove scripts, styles, ads, and boilerplate. Extract the main content.
3. Link Extraction -- Find all `<a href>` tags. Resolve relative URLs to absolute URLs.
4. Content Fingerprinting -- Compute SimHash or MinHash to detect near-duplicate pages.
5. Indexing -- Store the cleaned content and metadata for the search engine or downstream processing.

30.14 Putting It All Together

Here is the complete flow for crawling a single page:

```
1. URL Frontier picks next URL (respecting priority and politeness)
        |
2. Check robots.txt cache (fetch if missing or expired)
        |
3. Is this path allowed? --NO--> Discard URL
        |YES
4. Resolve DNS (check local cache first)
        |
5. Download page via HTTP (with timeout, retries)
        |
6. Is it HTML? --NO--> Store raw or discard based on config
        |YES
7. Parse HTML, extract text and links
        |
8. Compute content fingerprint
        |
9. Is content a duplicate? --YES--> Discard
        |NO
10. Store content in distributed storage
        |
11. For each extracted link:
    a. Normalize URL (lowercase host, remove fragment, etc.)
    b. Check URL bloom filter
```

```
    c. Already seen? --YES--> Skip
    d. Pass URL filter (extension, blacklist, length)
    e. Add to URL frontier with priority score
```

Common Mistakes

1. Ignoring robots.txt. This is not optional. Ignoring it can get your crawler banned and your company sued.
2. No politeness controls. Hammering a small website with 100 requests per second will take it down. You are responsible for the damage.
3. Using a simple FIFO queue as the frontier. Without priority and politeness, you will crawl unimportant pages and overwhelm popular domains.
4. Not handling DNS efficiently. DNS resolution becomes the bottleneck if you do not cache aggressively.
5. Ignoring crawler traps. Without depth limits and URL budgets, your crawler will get stuck in infinite loops and waste all its resources on a single domain.
6. Storing all raw HTML forever. Without compression and retention policies, storage costs explode.
7. Not normalizing URLs. `http://Example.COM/Page` and `http://example.com/page` are the same URL. Without normalization, you crawl duplicates.

Best Practices

1. Identify your crawler. Set a descriptive `User-Agent` header and provide a contact URL or email.
2. Implement exponential backoff. When a server returns errors, back off exponentially rather than retrying immediately.
3. Normalize URLs consistently. Lowercase the scheme and host, remove default ports, remove fragments, sort query parameters.
4. Use conditional requests. `If-Modified-Since` and `If-None-Match` headers save bandwidth on re-crawls.
5. Monitor everything. Track pages per second, error rates per domain, bloom filter false positive rate, DNS cache hit rate, and frontier size.

6. Design for extensibility. Use a plugin architecture so you can easily add support for new content types (PDF, images, JavaScript-rendered pages).
7. Persist the frontier. If a worker crashes, you do not want to lose millions of URLs. Write the frontier to disk periodically.

Quick Summary

A web crawler starts from seed URLs, uses a URL frontier (priority + politeness queues) to decide what to crawl next, downloads pages, extracts links, and adds new URLs to the frontier. Bloom filters provide space-efficient duplicate detection. Robots.txt and rate limiting ensure politeness. The system is distributed by partitioning domains across workers. BFS is preferred over DFS for general crawling. Trap avoidance requires depth limits, URL budgets, and pattern detection. Freshness is maintained through re-crawling strategies and conditional HTTP requests.

Key Points

- The URL frontier is a two-tier system: front queues for priority, back queues for per-domain politeness.
- Bloom filters use ~1.2 GB to track 1 billion URLs with a 1% false positive rate.
- Robots.txt must be fetched, cached, and respected for every domain.
- Domain-based partitioning provides natural politeness isolation in distributed crawling.
- BFS is the default traversal strategy; DFS is only for focused single-domain crawls.
- Crawler traps are real and dangerous. Always set depth limits and URL budgets per domain.
- Conditional HTTP requests save bandwidth when re-crawling pages for freshness.
- URL normalization prevents duplicate downloads of the same page under different URL formats.

Practice Questions

1. A website has no `robots.txt` file. Does this mean you can crawl everything on it without limits? What should your crawler do?
2. Your bloom filter has a 1% false positive rate. This means 1% of new URLs are incorrectly identified as "already seen" and skipped. How would you reduce the impact of false positives on crawl coverage?
3. You discover that 80% of your crawler's time is spent on DNS lookups. What three changes would you make to fix this?
4. A domain generates URLs like `/products?page=1`, `/products?page=2`, all the way to `/products?page=999999`. Your crawler is stuck crawling this one domain. How do you detect and handle this situation?
5. Your crawler needs to handle JavaScript-rendered pages (single-page applications where content loads dynamically). How would you modify the architecture to support this?

Exercises

Exercise 1: Design a Politeness Module

Design the politeness component of a web crawler. Specify the data structures, the algorithm for deciding when a domain can be crawled next, and how you handle `Crawl-delay` directives from robots.txt. Include pseudocode for the `getNextURL()` function.

Exercise 2: Bloom Filter Sizing

You need to crawl 10 billion unique URLs over the lifetime of your crawler. Calculate the optimal bloom filter size (in GB) for false positive rates of 0.1%, 1%, and 5%. How many hash functions does each configuration need? What happens to memory usage if you need to support 50 billion URLs?

Exercise 3: Freshness Strategy

Design a re-crawl scheduler for a news aggregation crawler. News sites update every few minutes, blogs update weekly, and corporate pages update monthly. Describe your priority scheme, how you detect update frequency, and how you allocate your crawl budget (10 million pages per day) across these categories.

What Is Next?

You now understand how to design a system that interacts with the entire internet at scale. In the next chapter, we will step back from specific designs and focus on estimation math -- the back-of-the-envelope calculations that let you quickly reason about scale, storage, bandwidth, and capacity in any system design interview.

Chapter 31: Estimation Math for System Design

Every system design interview begins with the same unspoken question: "Does this person understand scale?" Before you draw a single box on the whiteboard, you need to estimate how much traffic, storage, and bandwidth your system will handle. These back-of-the-envelope calculations are not about getting exact numbers -- they are about proving you can reason about whether you need one server or a thousand, one gigabyte or one petabyte, one request per second or one million.

In this chapter, we will build your estimation toolkit from the ground up. We will cover the essential numbers every engineer should have memorized, walk through the math step by step, and practice with real-world worked examples for Instagram, Twitter, and YouTube.

What You Will Learn

- Powers of 2 and how they map to real-world storage and traffic numbers.
- Latency numbers every engineer should know by heart.
- How to estimate QPS (queries per second) from DAU (daily active users).
- How to estimate storage requirements for any system.
- How to estimate bandwidth needs.
- Step-by-step worked examples for Instagram, Twitter, and YouTube.

Why This Chapter Matters

Estimation math is the foundation of every system design discussion. When an interviewer asks you to "design Instagram," they want to see that you think about scale before you start drawing architecture diagrams. A candidate who can quickly say "we need about 50,000 QPS

for reads and 500 QPS for writes, so reads dominate and we need caching" immediately establishes credibility.

More importantly, estimation math prevents you from over-engineering or under-engineering your design. If your system handles 100 requests per second, you do not need Kafka and a distributed database. If it handles 1 million requests per second, a single PostgreSQL instance will not cut it.

31.1 Powers of 2 Reference Table

Every estimation starts with understanding data sizes. Memorize this table.

```
+-------+---------------------+-----------------------+
| Power |  Exact Value        |  Approximate Size     |
+-------+---------------------+-----------------------+
| 2^10  |  1,024              |  1 Thousand     (1 KB)|
| 2^20  |  1,048,576          |  1 Million      (1 MB)|
| 2^30  |  1,073,741,824      |  1 Billion      (1 GB)|
| 2^40  |  1,099,511,627,776  |  1 Trillion     (1 TB)|
| 2^50  |  ~1.13 x 10^15      |  1 Quadrillion  (1 PB)|
+-------+---------------------+-----------------------+
```

Common Data Units

Unit	Power of 2	Bytes	Everyday Analogy
1 Byte	2^0	1	A single character
1 KB (Kilobyte)	2^10	~1,000	A short email
1 MB (Megabyte)	2^20	~1,000,000	A high-res photo
1 GB (Gigabyte)	2^30	~1,000,000,000	A movie (SD quality)
1 TB (Terabyte)	2^40	~10^12	500 hours of video
1 PB (Petabyte)	2^50	~10^15	All US academic libraries

Quick Conversion Tricks

- 1 million seconds = ~11.6 days

- 1 billion seconds = ~31.7 years
- Seconds in a day = 86,400 (~10^5, use 100,000 for estimation)
- Seconds in a month = ~2.6 million (~2.5 x 10^6)
- Seconds in a year = ~31.5 million (~3 x 10^7)

31.2 Latency Numbers Every Engineer Should Know

These numbers help you understand where time goes in your system. They were originally compiled by Jeff Dean at Google and remain directionally correct even as hardware evolves.

```
+---------------------------------------------+----------------+
| Operation                                   | Latency        |
+---------------------------------------------+----------------+
| L1 cache reference                          |       1 ns     |
| L2 cache reference                          |       4 ns     |
| Branch mispredict                           |       5 ns     |
| Mutex lock/unlock                           |      25 ns     |
| Main memory (RAM) reference                 |     100 ns     |
| Compress 1 KB with Snappy                   |   3,000 ns     |
|                                             |     = 3 us     |
| Send 1 KB over 1 Gbps network              |  10,000 ns     |
|                                             |    = 10 us     |
| SSD random read                             | 100,000 ns     |
|                                             |   = 100 us     |
| Read 1 MB sequentially from memory          | 250,000 ns     |
|                                             |   = 250 us     |
| Round trip within same datacenter           | 500,000 ns     |
|                                             |   = 500 us     |
| Read 1 MB sequentially from SSD             |   1,000,000 ns |
|                                             |     = 1 ms     |
| HDD seek                                   |  10,000,000 ns |
|                                             |    = 10 ms     |
| Read 1 MB sequentially from HDD            |  20,000,000 ns |
|                                             |    = 20 ms     |
| Send 1 MB over 1 Gbps network              |  10,000,000 ns |
|                                             |    = 10 ms     |
| Round trip US East <-> US West              |  40,000,000 ns |
|                                             |    = 40 ms     |
| Round trip US <-> Europe                    | 100,000,000 ns |
|                                             |   = 100 ms    |
+---------------------------------------------+----------------+
```

Key Takeaways from These Numbers

```
Latency Scale (logarithmic):

1 ns    |==  L1 cache
10 ns   |====  L2 cache, mutex
100 ns  |======  RAM
1 us    |========
10 us   |==========  Network send 1KB
100 us  |============  SSD random read
1 ms    |==============  SSD sequential 1MB
10 ms   |================  HDD seek, network 1MB
100 ms  |==================  Cross-continent round trip
```

1. Memory is 1000x faster than SSD. RAM access is 100 ns; SSD random read is 100 us.
2. SSD is 100x faster than HDD. SSD random read is 100 us; HDD seek is 10 ms.
3. Network within a datacenter is fast. A round trip is about 500 us (0.5 ms).
4. Cross-continent adds real latency. US East to West is ~40 ms; US to Europe is ~100 ms.
5. Compression is cheap. Compressing 1 KB takes only 3 us. Almost always worth it for network transfers.
6. Sequential reads are much faster than random reads. Design your data access patterns accordingly.

31.3 Estimating QPS from DAU

QPS (Queries Per Second) is the most fundamental capacity metric. Here is the formula:

```
QPS = DAU x (average queries per user per day) / seconds per day

Where:
  DAU = Daily Active Users
  Seconds per day = 86,400 (round to 100,000 for easy math)
```

Step-by-Step Process

1. Start with DAU. This is usually given or you estimate it.
2. Estimate actions per user per day. How many times does a user read/write?
3. Divide by 86,400 (or use 100,000 for round numbers).
4. Calculate peak QPS. Multiply average QPS by 2x to 5x for peak hours.
5. Separate reads and writes. Most systems are read-heavy (10:1 to 100:1 ratio).

Example: Basic QPS Calculation

```
Given:
  DAU = 10 million
  Average reads per user per day = 20
  Average writes per user per day = 2

Read QPS:
  = 10,000,000 x 20 / 86,400
  = 200,000,000 / 86,400
  = ~2,300 reads/second

Write QPS:
  = 10,000,000 x 2 / 86,400
  = 20,000,000 / 86,400
  = ~230 writes/second

Peak QPS (assume 3x average):
  Peak reads  = 2,300 x 3 = ~7,000 reads/second
  Peak writes = 230 x 3   = ~700 writes/second
```

31.4 Estimating Storage

Storage estimation follows a simple pattern:

```
Daily storage = (number of new items per day) x (size per item)
Yearly storage = daily storage x 365
N-year storage = yearly storage x N
```

Estimating Item Size

Break down each item into its components:

```
Example: A social media post

  Field              | Size
  ------------------|--------
  post_id            | 8 bytes (int64)
  user_id            | 8 bytes
  text               | 280 bytes (max)
  timestamp          | 8 bytes
  likes_count        | 4 bytes
  comments_count     | 4 bytes
```

```
  media_url         | 256 bytes
  ------------------|--------
  Total per post    | ~568 bytes
  Round up to       | ~600 bytes (for overhead)
```

Always round up. Real storage includes indexes, replication, and overhead.

31.5 Estimating Bandwidth

Bandwidth is about data flowing in and out of your system per second.

```
Incoming bandwidth = Write QPS x average write size
Outgoing bandwidth = Read QPS x average response size
```

Example

```
Given:
  Write QPS = 230 writes/second
  Average write size = 10 KB (includes image thumbnail)
  Read QPS = 2,300 reads/second
  Average response size = 50 KB (includes images)

Incoming bandwidth:
  = 230 x 10 KB
  = 2,300 KB/s
  = ~2.3 MB/s

Outgoing bandwidth:
  = 2,300 x 50 KB
  = 115,000 KB/s
  = ~115 MB/s
  = ~920 Mbps
```

At ~1 Gbps outgoing, you need multiple servers behind a load balancer to serve this traffic.

31.6 Worked Example: Estimate Storage for Instagram

Let us estimate how much storage Instagram needs for photos.

Assumptions

Parameter	Value	Reasoning
DAU	500 million	Instagram's approximate DAU
% of users who post daily	10%	Most users consume, few create
Photos posted per active poster	1.5	Average, including stories
Average photo size (compressed)	1.5 MB	After server-side compression
Number of size variants stored	4	Original + 3 thumbnails
Average variant size	500 KB	Thumbnails are smaller

Calculation

```
Step 1: Photos per day
  = DAU x posting_rate x photos_per_poster
  = 500,000,000 x 0.10 x 1.5
  = 75,000,000 photos/day (75 million)

Step 2: Storage per photo (all variants)
  = original + (3 thumbnails x average variant size)
  = 1.5 MB + (3 x 500 KB)
  = 1.5 MB + 1.5 MB
  = 3 MB per photo

Step 3: Daily photo storage
  = 75,000,000 x 3 MB
  = 225,000,000 MB
  = 225 TB/day

Step 4: Yearly photo storage
  = 225 TB x 365
  = ~82 PB/year

Step 5: With replication (3x for durability)
  = 82 PB x 3
  = ~246 PB/year
```

Summary

```
+-------------------------+--------------------+
| Metric                  | Estimate           |
+-------------------------+--------------------+
| Photos per day          | 75 million         |
| Daily photo storage     | 225 TB             |
| Yearly photo storage    | ~82 PB             |
| With 3x replication     | ~246 PB/year       |
+-------------------------+--------------------+
```

This explains why Instagram runs on massive distributed object storage systems, not traditional databases.

31.7 Worked Example: Estimate QPS for Twitter

Let us estimate the QPS for Twitter's home timeline feature.

Assumptions

Parameter	Value	Reasoning
DAU	250 million	Twitter's approximate DAU
Timeline refreshes per user per day	10	Open app, scroll, refresh
Tweets viewed per refresh	20	Average scroll session
New tweets posted per user per day	0.5	Not everyone tweets daily
Read:write ratio	~400:1	Heavily read-dominant

Read QPS (Timeline Loads)

```
Step 1: Timeline API calls per day
  = DAU x refreshes_per_day
  = 250,000,000 x 10
  = 2,500,000,000 (2.5 billion)

Step 2: Average QPS
  = 2,500,000,000 / 86,400
```

```
  = ~29,000 requests/second

Step 3: Peak QPS (assume 3x for major events)
  = 29,000 x 3
  = ~87,000 requests/second
```

Write QPS (New Tweets)

```
Step 1: New tweets per day
  = DAU x tweets_per_user
  = 250,000,000 x 0.5
  = 125,000,000 (125 million)

Step 2: Average QPS
  = 125,000,000 / 86,400
  = ~1,450 tweets/second

Step 3: Peak QPS
  = 1,450 x 3
  = ~4,350 tweets/second
```

Fan-out Consideration

Each tweet must be delivered to followers. If the average user has 200 followers:

```
Fan-out writes per second:
  = 1,450 tweets/s x 200 followers
  = 290,000 fan-out writes/second
```

This is why Twitter uses a pre-computed timeline (fan-out on write) for most users but switches to fan-out on read for celebrities with millions of followers.

Summary

```
+-------------------------+--------------------+
| Metric                  | Estimate           |
+-------------------------+--------------------+
| Timeline read QPS       | ~29,000            |
| Peak timeline QPS       | ~87,000            |
| Write (tweet) QPS       | ~1,450             |
| Fan-out writes/s        | ~290,000           |
| Read:Write ratio        | ~400:1             |
+-------------------------+--------------------+
```

31.8 Worked Example: Estimate Bandwidth for YouTube

Let us estimate YouTube's bandwidth requirements.

Assumptions

Parameter	Value	Reasoning
DAU	800 million	YouTube's approximate DAU
Videos watched per user per day	5	Average across all users
Average video length	5 minutes	Mix of short and long content
Average video bitrate (streaming)	5 Mbps	720p-1080p adaptive
Video uploads per day	500,000	Creators upload ~500K videos/day
Average upload size	500 MB	Before transcoding
Number of transcoded variants	5	240p, 360p, 480p, 720p, 1080p

Streaming (Outgoing) Bandwidth

```
Step 1: Total viewing time per day
  = DAU x videos_per_user x avg_length
  = 800,000,000 x 5 x 5 minutes
  = 20,000,000,000 minutes/day
  = 20 billion minutes/day

Step 2: Total viewing time in seconds
  = 20,000,000,000 x 60
  = 1,200,000,000,000 seconds/day (1.2 trillion)

Step 3: Average concurrent viewers (spread over 24h)
  = 1,200,000,000,000 / 86,400
  = ~13,900,000 concurrent streams

Step 4: Outgoing bandwidth
  = 13,900,000 x 5 Mbps
  = 69,500,000 Mbps
```

```
  = ~69.5 Tbps (terabits per second)

Step 5: With CDN (most traffic served from edge)
  Origin bandwidth = 69.5 Tbps x 10% (CDN cache hit ~90%)
  = ~7 Tbps from origin
```

Upload (Incoming) Bandwidth

```
Step 1: Upload volume per day
  = 500,000 videos x 500 MB
  = 250,000,000 MB
  = 250 TB/day

Step 2: Upload bandwidth (average)
  = 250 TB / 86,400 seconds
  = ~2.9 GB/s
  = ~23 Gbps

Step 3: Transcoding output (5 variants, each ~60% of original size)
  = 250 TB x 5 x 0.6
  = 750 TB/day of transcoded output
```

Storage

```
Step 1: Daily storage (all variants)
  = original + transcoded
  = 250 TB + 750 TB
  = 1,000 TB = 1 PB/day

Step 2: Yearly storage
  = 1 PB x 365
  = 365 PB/year

Step 3: With replication (3x)
  = 365 PB x 3
  = ~1,095 PB/year (~1.1 EB/year)
```

Summary

```
+-------------------------+--------------------+
| Metric                  | Estimate           |
+-------------------------+--------------------+
| Concurrent streams      | ~14 million        |
| Streaming bandwidth     | ~69.5 Tbps         |
```

```
| Origin bandwidth (CDN)  | ~7 Tbps           |
| Upload bandwidth        | ~23 Gbps          |
| Daily storage (all)     | ~1 PB             |
| Yearly storage (3x rep) | ~1.1 EB           |
+-------------------------+-------------------+
```

These numbers explain why YouTube operates its own CDN (Google Global Cache), builds custom hardware, and is one of the largest consumers of internet bandwidth worldwide.

31.9 Estimation Cheat Sheet

Use this quick reference during interviews:

```
+-----------------------------------+----------------------------+
| What You Need                     | Formula                    |
+-----------------------------------+----------------------------+
| QPS                               | DAU x actions/user / 86400 |
| Peak QPS                          | Average QPS x 2~5         |
| Daily storage                     | New items/day x item size  |
| Yearly storage                    | Daily x 365               |
| Incoming bandwidth                | Write QPS x write size     |
| Outgoing bandwidth                | Read QPS x response size   |
| Number of servers (CPU-bound)     | Peak QPS / QPS per server  |
| Number of servers (memory-bound)  | Working set / RAM per node |
+-----------------------------------+----------------------------+
```

Common Reference Numbers

```
+-----------------------------------+----------------------------+
| Item                              | Typical Size               |
+-----------------------------------+----------------------------+
| A tweet / short text post         | 300 bytes - 1 KB           |
| A user profile record             | 1 - 5 KB                  |
| A compressed photo (JPEG)         | 200 KB - 2 MB              |
| A 1-minute video (720p)           | 30 - 60 MB                |
| A database row (relational)       | 100 bytes - 1 KB          |
| An API response (JSON)            | 1 - 50 KB                 |
+-----------------------------------+----------------------------+

+-----------------------------------+----------------------------+
| System Capacity                   | Typical Range              |
+-----------------------------------+----------------------------+
| Single web server QPS             | 1,000 - 10,000             |
```

```
| Single DB server QPS (simple)    | 5,000 - 20,000             |
| Single Redis instance QPS        | 50,000 - 100,000           |
| Single machine RAM               | 64 - 512 GB               |
| Single machine SSD               | 1 - 8 TB                  |
| 1 Gbps network link              | ~125 MB/s throughput       |
| 10 Gbps network link             | ~1.25 GB/s throughput      |
+----------------------------------+----------------------------+
```

31.10 Tips for Estimation in Interviews

Do

1. State your assumptions clearly. "I will assume DAU is 100 million" is much better than just using a number.
2. Round aggressively. Use 100,000 instead of 86,400. Use 300 million instead of 328 million. Precision does not matter.
3. Show your work. Write each step on the whiteboard. The interviewer wants to see your thought process.
4. Sanity check results. If you calculate that a text messaging app needs 1 PB of storage per day, something is wrong.
5. Use powers of 10. Everything is easier when you think in powers of 10.

Do Not

1. Do not spend more than 5 minutes on estimation. It is a tool, not the design itself.
2. Do not argue about exact numbers. "Is it 3 MB or 5 MB?" does not matter. The order of magnitude matters.
3. Do not forget peak traffic. Average QPS is useful, but your system must handle peaks.
4. Do not ignore replication. Storage estimates should include replication factor (usually 3x).
5. Do not forget the read/write ratio. This determines your caching and scaling strategy.

Common Mistakes

1. Confusing bits and bytes. Network speeds are in bits per second (Mbps, Gbps). Storage is in bytes (MB, GB). There are 8 bits in a byte.
2. Forgetting to account for metadata. A 1 MB photo needs more than 1 MB of storage when you add database indexes, file system overhead, and metadata.
3. Using daily averages for capacity planning. Peak traffic can be 3-10x the average. Size your system for peaks.
4. Ignoring compression. Text compresses 5-10x. Images are already compressed. Not accounting for compression leads to wildly wrong bandwidth estimates.
5. Treating all users as equal. Power users (celebrities, brands) generate disproportionate traffic. A single viral tweet can create millions of reads.

Best Practices

1. Memorize the latency numbers table. You will use it in every interview.
2. Practice converting between units quickly. You should be able to go from "500 million requests per day" to "~6,000 QPS" in your head.
3. Always estimate both reads and writes. The ratio between them drives fundamental design decisions.
4. Work top-down. Start with users, then actions per user, then QPS, then storage, then bandwidth.
5. Use a consistent framework. Same steps every time: Traffic -> Storage -> Bandwidth -> Number of machines.
6. Keep a mental model of "big company" numbers. Google: billions of searches. Instagram: hundreds of millions of DAU. YouTube: billions of hours watched per day. These anchor your estimates.

Quick Summary

Estimation math is about reasoning at the right order of magnitude, not getting exact answers. Memorize the powers of 2 table, the latency numbers, and the conversion shortcuts. Follow the top-down framework: start with DAU, calculate QPS (average and peak, reads and writes), estimate storage (daily, yearly, with replication), and compute bandwidth. Always state your assumptions, round aggressively, and sanity check your results. Five minutes of good estimation sets the foundation for the entire design discussion.

Key Points

- Seconds in a day: ~86,400 (round to 100,000 for easy math).
- RAM is 1000x faster than SSD; SSD is 100x faster than HDD.
- QPS = DAU x actions per user / 86,400. Peak QPS = 2x to 5x average.
- Storage grows linearly: new items per day x item size x 365 x replication factor.
- Network bandwidth is measured in bits; storage in bytes. 1 byte = 8 bits.
- Always estimate reads and writes separately. The ratio drives design decisions.
- Compression saves 5-10x for text, but images and video are already compressed.
- Your estimates should be within the right order of magnitude (10x), not exact.

Practice Questions

1. A messaging app has 50 million DAU. Each user sends 40 messages per day and receives 40 messages per day. Average message size is 200 bytes. Calculate the write QPS, read QPS, daily storage, and yearly storage.
2. A video conferencing platform has 10 million DAU. Each user averages 2 meetings per day, each lasting 30 minutes. Video is streamed at 2 Mbps per participant. Average meeting size is 4 participants. What is the peak concurrent bandwidth?
3. An e-commerce site receives 50 million product page views per day. Each page view returns 100 KB of data. The site has 10 million products, each with a 5 KB database record. How much RAM do you need to cache all products? What is the outgoing bandwidth?
4. Why is cross-continent latency (~100 ms) a critical factor for global system design, but L1 cache latency (1 ns) is rarely discussed in system design interviews?
5. You are designing a system that stores 10 billion records, each 1 KB in size. You want all data accessible with less than 1 ms latency. Can you keep all data in RAM? How many machines do you need? (Assume 256 GB RAM per machine.)

Exercises

Exercise 1: Estimate Slack's Infrastructure

Slack has 30 million DAU. Each user sends 50 messages per day across 10 channels. Messages average 500 bytes. Users also upload 0.5 files per day averaging 2 MB each. Calculate: write QPS, read QPS (assume 10:1 read/write), daily message storage, daily file storage, and outgoing bandwidth. How many database servers do you need if each handles 10,000 write QPS?

Exercise 2: Cost Estimation

Using your Instagram storage estimate from Section 31.6 (225 TB/day, ~82 PB/year), calculate the approximate annual storage cost using cloud pricing of $0.02 per GB per month for standard storage. How much would you save by moving data older than 90 days to cold storage at $0.004 per GB per month?

Exercise 3: Build Your Own Cheat Sheet

Create a one-page estimation cheat sheet for three systems you might encounter in an interview: a ride-sharing app, a photo-sharing app, and a real-time multiplayer game. For each, list the key assumptions (DAU, actions per user, data sizes), and pre-calculate the QPS, storage, and bandwidth. Practice presenting each estimate in under 3 minutes.

What Is Next?

Now that you can reason about scale, the next chapter gives you a complete framework for structuring your system design interview from start to finish. You will learn the four-step process that top candidates follow, the communication patterns that impress interviewers, and how to manage your time in a 45-minute session.

Chapter 32: The System Design Interview Framework

You have spent dozens of hours learning about caching, databases, load balancers, message queues, and distributed systems. You can explain consistent hashing and the CAP theorem in your sleep. But when you sit down in a 45-minute system design interview and the interviewer says "Design WhatsApp," your mind goes blank. Where do you start? What do you say first? How do you know if you are going too deep or staying too shallow?

This chapter gives you a repeatable, four-step framework that works for every system design question. It is the same framework used by candidates who get offers at top tech companies, and it works because it mirrors how senior engineers actually approach problems in the real world: clarify the problem, sketch the big picture, dive into the hard parts, and reflect on trade-offs.

What You Will Learn

- The four-step framework for system design interviews: Clarify, High-Level Design, Deep Dive, Wrap Up.
- Exactly what to say and do in each step, with time allocations.
- How to communicate effectively with your interviewer throughout.
- Red flags that hurt your chances and green flags that help.
- A complete sample 45-minute walkthrough.
- What interviewers are actually evaluating.

Why This Chapter Matters

Technical knowledge without a delivery framework is like having all the ingredients but no recipe. The framework is what transforms your knowledge into a coherent, impressive interview performance. Without it, even brilliant engineers ramble, skip critical steps, or spend 30 minutes on a detail that does not matter.

The framework also reduces anxiety. When you know exactly what to do in each phase, you do not waste mental energy wondering "what should I talk about next?" You can focus entirely on the actual design.

32.1 The Four Steps at a Glance

```
+-------------------------------------------------------------+
|                45-Minute Interview Timeline                 |
+-------------------------------------------------------------+
|                                                             |
|  Step 1: CLARIFY (5-7 min)                                  |
|  [=======]                                                  |
|  Ask questions. Define scope. Estimate scale.               |
|                                                             |
|  Step 2: HIGH-LEVEL DESIGN (15-20 min)                      |
|  [=====================]                                    |
|  Draw the architecture. Define APIs. Data model.            |
|                                                             |
|  Step 3: DEEP DIVE (15-20 min)                              |
|  [=====================]                                    |
|  Go deep on 2-3 critical components.                        |
|                                                             |
|  Step 4: WRAP UP (3-5 min)                                  |
|  [=====]                                                    |
|  Summarize. Discuss trade-offs. Identify improvements.      |
|                                                             |
+-------------------------------------------------------------+
```

Each step has a clear purpose, a set of activities, and a time budget. Let us explore each one in detail.

32.2 Step 1: Clarify the Requirements (5-7 minutes)

This is the most underrated step. Many candidates skip it entirely and jump straight to drawing boxes. That is a mistake. Clarifying requirements shows the interviewer that you think before

you build -- a hallmark of senior engineers.

What to Do

1. Restate the problem in your own words to confirm understanding.
2. Ask clarifying questions about functional requirements.
3. Ask about non-functional requirements (scale, latency, availability).
4. Identify what is in scope and out of scope.
5. Do back-of-the-envelope estimation (from Chapter 31).

Questions to Ask

Here is a checklist of questions organized by category. You will not ask all of them -- pick the ones most relevant to the problem.

Functional scope:

- What are the core features we need to support?
- Who are the users? (consumers, businesses, internal?)
- What platforms? (mobile, web, API?)
- Do we need real-time functionality?

Scale and traffic:

- How many DAU should we design for?
- What is the expected read/write ratio?
- How much data are we storing?
- Are there any spiky traffic patterns?

Non-functional requirements:

- What is the latency target? (p50, p99)
- What availability do we need? (99.9%? 99.99%?)
- Is consistency or availability more important?
- Do we need to support multiple regions?

Constraints:

- Are there any specific technology constraints?
- Do we need backward compatibility with existing systems?
- What is the budget? (for cost-sensitive designs)

Example: Clarifying "Design a Chat System"

```
Candidate: "To make sure I understand, we are designing a real-time
            messaging system similar to WhatsApp or Slack. Let me
            ask a few questions to narrow the scope."

Candidate: "Is this 1:1 chat, group chat, or both?"
Interviewer: "Both. Start with 1:1 and we can extend to groups."

Candidate: "What about media -- just text, or also images/files?"
Interviewer: "Text for now. Mention how you would add media."

Candidate: "How many DAU are we targeting?"
Interviewer: "50 million."

Candidate: "What about message delivery guarantees? At-most-once,
            at-least-once, or exactly-once?"
Interviewer: "At-least-once with deduplication on the client."

Candidate: "Do we need message history / persistence?"
Interviewer: "Yes, users should see their message history."

Candidate: "Any latency requirements?"
Interviewer: "Messages should be delivered in under 500ms."
```

After this exchange, the candidate and interviewer have a shared understanding of what to build.

Back-of-the-Envelope Estimation

After clarifying requirements, spend 2-3 minutes on quick estimates:

```
Candidate: "Let me do some quick math.

  50M DAU x 40 messages sent per day = 2 billion messages/day
  QPS = 2B / 86,400 = ~23,000 messages/second
  Peak QPS = ~70,000 messages/second

  Storage per message: ~200 bytes (text + metadata)
  Daily storage: 2B x 200 bytes = 400 GB/day
  Yearly: ~146 TB

  This is a write-heavy system with real-time delivery needs.
  We will need WebSockets and a pub/sub mechanism."
```

Common Mistake: Going Too Deep Too Early

Do NOT start designing the database schema or debating Redis vs Memcached in this step. You are setting the stage, not building the house.

32.3 Step 2: High-Level Design (15-20 minutes)

This is the main act. You will draw the architecture, define APIs, and outline the data model.

What to Do

1. Draw the architecture diagram on the whiteboard (or shared doc).
2. Define the API for core operations.
3. Outline the data model (key tables/collections and their relationships).
4. Walk through the main user flows end-to-end.
5. Check in with the interviewer to see if they want to go deeper on anything.

Drawing the Architecture

Start with the user on the left and the data store on the right. Add components as needed:

```
+---------+       +-----------+       +------------+       +----------+
| Client  |------>|  Load     |------>|  App       |------>| Database |
| (Mobile |       |  Balancer |       |  Server(s) |       |          |
|  / Web) |       +-----------+       +------------+       +----------+
+---------+                                   |
                                              v
                                         +----------+
                                         |  Cache   |
                                         +----------+
```

Then add detail as you talk through the design:

```
+---------+     +--------+     +------------+     +-----------+
| Client  |<--->|  API   |<--->|  Chat      |<--->| Message   |
| (WS)    |     | Gateway|     |  Service   |     | Store     |
+---------+     +--------+     +------------+     | (Cassandra|
                                     |            +-----------+
                                     v
                               +------------+     +-----------+
                               | Presence   |<--->| Redis     |
                               | Service    |     | (online   |
                               +------------+     |  status)  |
                                                  +-----------+
                               +------------+     +-----------+
                               | Push       |<--->| APNs/FCM  |
                               | Service    |     | (offline  |
```

```
                                +-------------+     |  push)    |
                                                    +-----------+
```

Defining the API

Keep APIs simple. Use REST-style or describe the operations:

```
POST /api/messages
  Body: { sender_id, receiver_id, content, timestamp }
  Response: { message_id, status }

GET /api/messages?user_id=X&contact_id=Y&before=timestamp&limit=50
  Response: { messages: [...], has_more: true/false }

WebSocket: /ws/chat
  Connect: { user_id, auth_token }
  Send: { type: "message", receiver_id, content }
  Receive: { type: "message", sender_id, content, timestamp }
```

Outlining the Data Model

```
Messages Table:
+-------------+-----------+-----------------------------------+
| Column      | Type      | Notes                             |
+-------------+-----------+-----------------------------------+
| message_id  | UUID      | Primary key (time-based UUID)     |
| sender_id   | BIGINT    | Who sent it                       |
| receiver_id | BIGINT    | Who receives it                   |
| content     | TEXT      | Message text                      |
| created_at  | TIMESTAMP | When it was sent                  |
| status      | ENUM      | sent / delivered / read           |
+-------------+-----------+-----------------------------------+

Partition key: (sender_id, receiver_id) for conversation-based queries
```

Walking Through User Flows

Pick the 2-3 most important user flows and trace them through your architecture:

```
Flow: User A sends a message to User B

1. User A's client sends message via WebSocket to API Gateway
2. API Gateway routes to Chat Service
```

```
3. Chat Service stores message in Message Store
4. Chat Service checks if User B is online (Presence Service)
5. If online: deliver via WebSocket connection to User B
6. If offline: send push notification via Push Service
7. User B's client acknowledges receipt
8. Chat Service updates message status to "delivered"
```

Check In With the Interviewer

After completing the high-level design (about 15-20 minutes in), pause and ask:

```
"This is the high-level architecture. Before I go deeper,
is there a particular component you would like me to dive into?
I was thinking of exploring [X] and [Y] in more detail."
```

This shows collaboration and lets the interviewer steer toward what they care about.

32.4 Step 3: Deep Dive (15-20 minutes)

This is where you demonstrate depth. Pick 2-3 components and go deep on design decisions, trade-offs, and edge cases.

What to Do

1. Pick 2-3 areas to dive deep (or follow the interviewer's guidance).
2. Discuss design choices and trade-offs for each area.
3. Handle edge cases and failure scenarios.
4. Optimize for the non-functional requirements identified in Step 1.

What to Dive Into

Choose areas that are:

- Critical to the system -- If this component fails, the whole system breaks.
- Technically interesting -- Shows depth of knowledge.
- Relevant to the requirements -- Addresses the specific scale/latency/consistency needs.

Common deep-dive topics:

Topic	What to Discuss
Database choice	SQL vs NoSQL, partition strategy, replication
Caching strategy	What to cache, eviction policy, consistency
Scaling bottlenecks	What breaks first, how to scale horizontally
Data consistency	Strong vs eventual, conflict resolution
Failure handling	What happens when X goes down?
Real-time delivery	WebSockets, long polling, SSE
Rate limiting	How to protect the system from abuse
Monitoring	Key metrics, alerting, debugging

Example Deep Dive: Message Delivery Guarantee

```
Candidate: "Let me dive into how we guarantee message delivery.

The challenge is that the recipient might be offline, or the
network might drop the WebSocket connection mid-delivery.

Here is our delivery flow:

1. Sender sends message. Server stores it in the database
   BEFORE attempting delivery. This guarantees durability.

2. Server assigns a monotonically increasing sequence number
   per conversation. This lets the client detect gaps.

3. If recipient is online, push via WebSocket. The recipient
   client sends an ACK back. Server updates status to
   'delivered.'

4. If no ACK within 5 seconds, retry 3 times with
   exponential backoff.

5. If recipient is offline, queue the message. When the
   recipient reconnects, they pull all undelivered messages
   using the last sequence number they received.

6. The client deduplicates using message_id to handle
   at-least-once delivery.

Edge case: What if the server crashes after storing the
message but before delivering it? On restart, the server
scans for messages in 'sent' status older than 30 seconds
and retries delivery.
```

```
Edge case: What if both clients are on different servers?
We use a pub/sub system (Redis Pub/Sub or Kafka) so that
the server holding User B's WebSocket receives the message
from the server that accepted User A's request."
```

This kind of deep dive shows the interviewer that you think about failure modes, edge cases, and real-world reliability.

32.5 Step 4: Wrap Up (3-5 minutes)

The wrap-up is your chance to show big-picture thinking and self-awareness.

What to Do

1. Summarize the design in 2-3 sentences.
2. Discuss trade-offs you made and why.
3. Identify improvements you would make with more time.
4. Mention operational concerns (monitoring, deployment, testing).

Example Wrap Up

```
Candidate: "To summarize, we designed a real-time chat system
that handles 50M DAU with WebSocket-based delivery, Cassandra
for message storage, Redis for presence and pub/sub, and a
push notification service for offline users.

Key trade-offs:
- We chose eventual consistency for message delivery status
  to favor availability and speed over strict ordering.
- We used Cassandra over PostgreSQL because our access pattern
  is append-heavy and partition-friendly.

If I had more time, I would explore:
- End-to-end encryption and key management
- Group chat fan-out optimization
- Media message support with CDN integration
- Message search using Elasticsearch
- Rate limiting and spam detection

For monitoring, I would track: message delivery latency (p50,
p99), WebSocket connection count, undelivered message queue
```

```
depth, and database write latency."
```

32.6 Communication Tips

How you communicate is as important as what you design. Here are the behaviors that set top candidates apart.

Think Out Loud

The interviewer cannot read your mind. Narrate your thought process:

```
GOOD: "I am choosing Cassandra here because our write volume
      is very high and our access pattern is partition-friendly.
      We always query messages by conversation, which maps
      perfectly to Cassandra's partition key."

BAD:  *draws Cassandra on the whiteboard without explanation*
```

Collaborate, Do Not Lecture

Treat the interview as a working session with a colleague, not a presentation:

```
GOOD: "I am thinking of using a message queue here to decouple
      the write path from the delivery path. Does that align
      with what you are thinking, or would you like me to
      explore a different approach?"

BAD:  *talks for 20 minutes straight without pause*
```

Manage Your Time

Keep a mental clock. If you are 25 minutes in and still on the high-level design, you need to move to the deep dive.

```
+--------+--------+-----------+--------+
| 0-7min | 7-25min| 25-42min  | 42-45  |
| Clarify| Design | Deep Dive | Wrap   |
+--------+--------+-----------+--------+
```

Use the Whiteboard Effectively

- Draw big. Small diagrams are hard to read and modify.
- Use consistent notation. Boxes for services, cylinders for databases, arrows for data flow.
- Label everything. Do not make the interviewer guess what a box represents.
- Leave space. You will need to add components as you go deeper.

32.7 Red Flags and Green Flags

Red Flags (Things That Hurt You)

Red Flag	Why It Hurts
Jumping to solution without asking questions	Shows lack of engineering rigor
Only one approach considered	Unable to reason about trade-offs
Over-engineering simple problems	Using Kafka for 100 QPS
Under-engineering complex problems	Single database for 1M QPS
Ignoring non-functional requirements	Not discussing scalability or availability
Cannot handle pushback	Gets defensive when interviewer challenges a decision
Silent thinking for long periods	Interviewer cannot evaluate your thought process
Ignoring the interviewer's hints	Missing guidance toward what they want to discuss
Memorized solution delivery	Reciting a textbook answer without understanding trade-offs
No estimation or numbers	Cannot reason about scale

Green Flags (Things That Help You)

Green Flag	Why It Helps

Asks thoughtful clarifying questions	Shows product sense and engineering maturity
Estimates scale before designing	Demonstrates quantitative thinking
Explains trade-offs for each decision	Shows depth and experience
Handles edge cases proactively	Thinks about what can go wrong
Adapts when interviewer steers	Collaborative and flexible
Discusses monitoring and operations	Thinks beyond just building the system
Draws clean, organized diagrams	Communicates complex ideas clearly
Self-corrects and iterates	Shows intellectual honesty
Considers cost implications	Practical engineering judgment
Mentions security and privacy	Awareness of real-world concerns

---

32.8 Sample 45-Minute Walkthrough: "Design a News Feed"

Here is how a strong candidate would spend their 45 minutes on "Design a News Feed" (like Facebook or Twitter).

Minutes 0-6: Clarify

```
Candidate: "I want to make sure I understand the scope. We are
designing the home feed -- the page where users see posts from
people they follow, sorted by relevance or time. Is that right?"

Interviewer: "Yes."

Candidate: "A few questions:
- What types of content? Text, images, videos?
- Do we need a ranking algorithm, or is reverse chronological OK?
- How many DAU?
- Is this global or single-region?"

Interviewer: "Text and images. Ranked feed. 200M DAU. Global."

Candidate: "OK, let me estimate scale quickly.
- 200M DAU, each views feed ~10 times/day = 2B feed loads/day
```

```
- Feed load QPS = 2B / 86,400 = ~23,000 QPS
- Peak = ~70,000 QPS
- Each feed shows 20 posts = 1.4 billion post reads/day
  (but cached, so actual DB reads are much lower)
- Writes: ~50M posts/day = ~580 posts/second
- This is extremely read-heavy. Caching is critical."
```

Minutes 6-22: High-Level Design

```
Candidate draws:

+--------+    +----------+    +----------+    +---------+
| Client |--->| API      |--->| Feed     |--->| Post    |
|        |    | Gateway  |    | Service  |    | Storage |
+--------+    +----------+    +----------+    +---------+
                                   |
                                   v
                              +----------+    +---------+
                              | Feed     |--->| Feed    |
                              | Cache    |    | Cache   |
                              | Builder  |    | (Redis) |
                              +----------+    +---------+
                                   |
                                   v
                              +----------+    +---------+
                              | Social   |--->| Graph   |
                              | Graph    |    | DB      |
                              | Service  |    |         |
                              +----------+    +---------+

"When a user requests their feed, the Feed Service checks
the Feed Cache first. If it is a cache hit, we return the
pre-computed feed. If it is a miss, the Feed Builder
constructs the feed by:

1. Querying the Social Graph for the user's following list
2. Fetching recent posts from each followed user
3. Ranking the posts
4. Caching the result
5. Returning the top 20 posts

For writes: when a user creates a post, we store it and
then fan out to followers' cached feeds.

API:
  GET /feed?user_id=X&cursor=abc&limit=20
  POST /posts { user_id, content, media_ids }

Data model:
  Posts: { post_id, author_id, content, media_urls,
           created_at, like_count }
```

```
  Follow: { follower_id, followee_id, created_at }
  Feed Cache: per-user sorted list of post_ids in Redis"
```

Candidate walks through the create-post and view-feed flows.

Minutes 22-40: Deep Dive

```
Candidate: "I would like to dive into two areas:
1. Fan-out strategy (push vs pull)
2. Feed ranking

DEEP DIVE 1: Fan-out

The key decision is push (fan-out on write) vs pull
(fan-out on read).

Push model:
  When User A posts, write the post_id to the feed cache
  of every follower. If A has 1,000 followers, that is
  1,000 cache writes.

  Pros: Feed reads are instant (pre-computed).
  Cons: Celebrity problem. If a user has 10M followers,
        one post triggers 10M writes.

Pull model:
  When User B requests their feed, query all followees'
  recent posts, merge, rank, return.

  Pros: No celebrity problem. No write amplification.
  Cons: Slow feed reads. Must query N users' posts.

Hybrid approach (what Facebook and Twitter actually do):
  - For normal users (< 10K followers): push model
  - For celebrities (> 10K followers): pull model
  - When building a feed, merge the cached feed (push)
    with live queries for celebrity posts (pull)

This gives us fast reads for most content while avoiding
the celebrity write amplification problem.

DEEP DIVE 2: Feed ranking

Instead of reverse chronological, we rank by relevance.
Features used for ranking:

  - Recency (newer posts score higher)
  - Engagement (likes, comments, shares)
  - Relationship strength (how often user interacts
    with the author)
  - Content type preference (does user engage more
```

```
    with images or text?)

The ranking model runs at feed-build time. We can use a
lightweight ML model (logistic regression or small neural
net) that scores each candidate post and returns the
top 20.

For latency: pre-compute features offline, store in a
feature store, and do only the scoring online. Target:
<100ms for ranking 500 candidate posts."
```

Minutes 40-45: Wrap Up

```
Candidate: "To summarize:
- Feed system for 200M DAU, 23K QPS average, 70K peak
- Hybrid fan-out: push for regular users, pull for celebrities
- Redis-based feed cache for sub-10ms feed reads
- ML-based ranking for relevance

Trade-offs:
- Hybrid complexity vs pure push/pull simplicity
- Eventual consistency in feed (acceptable for social media)
- Cache memory cost vs read latency

With more time, I would explore:
- Content moderation pipeline before feed insertion
- A/B testing framework for ranking models
- Multi-region feed consistency
- Ads insertion into the feed
- Rate limiting on post creation to prevent spam

For monitoring: feed build latency, cache hit rate,
fan-out queue depth, ranking model latency, and
feed freshness (time between post creation and feed
appearance)."
```

32.9 What Interviewers Are Actually Looking For

Different companies weight these differently, but here are the core evaluation criteria:

Technical Competence

- Can you design a system that actually works at the stated scale?

- Do you understand the fundamental building blocks (databases, caches, queues, load balancers)?
- Can you make appropriate technology choices and justify them?

Problem-Solving Approach

- Do you break down the problem systematically?
- Can you identify the hard parts of the problem?
- Do you consider multiple approaches before committing?

Trade-off Analysis

- Can you articulate the pros and cons of your design decisions?
- Do you understand that every choice has a cost?
- Can you adapt your design when constraints change?

Communication

- Can you explain complex ideas clearly?
- Do you organize your thoughts logically?
- Do you collaborate with the interviewer?

Scope Management

- Can you identify what is in scope and out of scope?
- Do you prioritize the most important features?
- Can you manage your time effectively during the interview?

Seniority Signals

Level	What Interviewers Look For
Junior	Understands basic components. Can draw a reasonable architecture.
Mid	Makes good technology choices. Handles common edge cases.
Senior	Deep trade-off analysis. Discusses operational concerns. Drives the conversation.

Staff+	Considers organizational impact. Discusses multi-year evolution. Addresses cost and business trade-offs.

Common Mistakes

1. Skipping Step 1. Jumping to the solution without understanding the problem. You might design the wrong system entirely.
2. Spending too long on estimation. Estimation should take 2-3 minutes, not 15. It supports the design, it is not the design.
3. Drawing before thinking. Taking 30 seconds to mentally outline before you start drawing leads to a much cleaner diagram.
4. Going too deep too early. Discussing database indexing strategies before you have a high-level architecture wastes precious time.
5. Ignoring the interviewer's hints. If the interviewer says "interesting, how would you handle the case where..." -- that is where they want you to go. Follow their lead.
6. One-size-fits-all answers. Using the exact same architecture for a 100-user internal tool and a 100-million-user consumer app shows lack of judgment.
7. No wrap-up. Ending abruptly without summarizing makes it feel incomplete.

Best Practices

1. Practice with a timer. Run mock interviews with a 45-minute clock. Time management is a skill that requires practice.
2. Use a consistent notation. Develop your own visual language: boxes for services, cylinders for databases, arrows for data flow, clouds for external services.
3. Prepare "deep dive" topics. For each system you study, prepare 2-3 topics you can dive deep on. Database choice, caching strategy, and failure handling are almost always relevant.
4. Study real systems. Read engineering blogs from Netflix, Uber, Twitter, and Facebook. Understanding how real systems work gives you authentic material.
5. Practice explaining trade-offs. For every design decision, be ready to say: "I chose X over Y because Z. The downside of X is W, but in our case that is acceptable because..."

6. Record yourself. Watch your practice sessions. Are you clear? Do you ramble? Do you use the whiteboard effectively?

Quick Summary

The four-step framework structures your system design interview into Clarify (5-7 min), High-Level Design (15-20 min), Deep Dive (15-20 min), and Wrap Up (3-5 min). In Step 1, ask clarifying questions and estimate scale. In Step 2, draw the architecture, define APIs, and outline the data model. In Step 3, go deep on 2-3 critical components with trade-off analysis. In Step 4, summarize, discuss trade-offs, and mention improvements. Throughout, think out loud, collaborate with the interviewer, and manage your time.

Key Points

- Never jump to the solution. Always start by clarifying requirements and estimating scale.
- The high-level design should include an architecture diagram, API definitions, and a data model.
- Deep dives should focus on 2-3 areas that are critical, technically interesting, and relevant to the requirements.
- Always discuss trade-offs. Every design decision has pros and cons.
- Communicate like a colleague, not a lecturer. Check in with the interviewer regularly.
- Green flags: thoughtful questions, quantitative thinking, trade-off analysis, edge case handling.
- Red flags: no clarification, silent thinking, ignoring hints, memorized answers.
- The wrap-up is your chance to show big-picture thinking and self-awareness.

Practice Questions

1. You are 20 minutes into a 45-minute interview and you are still defining APIs for your high-level design. What should you do?
2. The interviewer asks you to "Design Uber." Write down the 5 most important clarifying questions you would ask before starting the design.

3. You chose PostgreSQL for your database, and the interviewer pushes back: "At 500,000 writes per second, will PostgreSQL handle this?" How do you respond? (Hint: this is not about defending your choice -- it is about demonstrating trade-off thinking.)
4. What is the difference between how a junior engineer and a senior engineer would wrap up a system design interview?
5. An interviewer gives you the hint: "Think about what happens when one of your services goes down." What are they actually asking you to discuss?

Exercises

Exercise 1: Timed Mock Interview

Set a 45-minute timer and design "an online file storage system like Google Drive" using the four-step framework. Record the time you spend on each step. After the timer, evaluate: did you cover all four steps? Did you go too deep on any one area? Did you discuss trade-offs?

Exercise 2: Trade-off Practice

For each of these design decisions, write 2 pros and 2 cons:

- Using a NoSQL database (like DynamoDB) vs a relational database (like PostgreSQL) for a social media post store.
- Fan-out on write vs fan-out on read for a news feed.
- WebSockets vs long polling for real-time notifications.
- Monolithic architecture vs microservices for a startup's MVP.
- Strong consistency vs eventual consistency for a messaging app.

Exercise 3: Interviewer Perspective

You are the interviewer evaluating a candidate designing a URL shortener. Write a scoring rubric with 5 criteria, each scored 1-5. What would a score of 1 look like vs a score of 5 for each criterion? Use this rubric to evaluate your own practice interviews.

What Is Next?

You now have the framework to deliver a structured, impressive system design interview. In the next chapter, we will catalog the most common design patterns that appear across system design problems -- reusable solutions like Circuit Breaker, CQRS, Saga, and Sidecar that you

can pull out of your toolkit whenever they fit.

Chapter 33: Common System Design Patterns

Patterns are reusable solutions to recurring problems. When a senior engineer hears "we need to protect our service from cascading failures," they immediately think "Circuit Breaker." When someone says "our reads and writes have completely different performance requirements," the response is "CQRS." These patterns are the vocabulary of system design -- learning them gives you ready-made building blocks that you can combine and adapt to solve almost any design problem.

This chapter is a catalog of 17 essential patterns. For each pattern, you will find: when to use it, how it works with an ASCII diagram, and a real-world example. Think of this chapter as a reference you come back to whenever you encounter a design challenge.

What You Will Learn

- 17 design patterns that appear repeatedly in system design interviews and production systems.
- When to apply each pattern and when to avoid it.
- How each pattern works, illustrated with ASCII diagrams.
- Real-world examples from companies like Netflix, Amazon, Uber, and Stripe.

Why This Chapter Matters

In interviews, dropping the right pattern name at the right time signals experience. But more importantly, understanding these patterns helps you design better systems. Instead of reinventing solutions from scratch, you can reach for a proven pattern, adapt it to your context, and explain the trade-offs confidently.

These patterns are not academic exercises. Every one of them is used in production at scale by companies processing millions of requests per second.

33.1 Ambassador

When to Use

You want to offload cross-cutting concerns (logging, monitoring, retries, TLS termination) from your application to a helper process.

How It Works

```
+-------------------+       +-------------------+
|   Application     |       |   Ambassador      |
|   Container       |------>|   Container       |------>  External
|                   |       |                   |         Service
|  (business logic  |       |  (retries, TLS,   |
|   only)           |       |   logging, auth)  |
+-------------------+       +-------------------+
         Pod / Host
```

The ambassador sits alongside your application as a proxy. All outbound traffic goes through it. The application itself stays simple and focused on business logic.

Real-World Example

Envoy Proxy is used as an ambassador in Kubernetes service meshes (Istio). It handles mTLS, retries, circuit breaking, and observability for every service without changing application code.

33.2 Backends for Frontends (BFF)

When to Use

Different clients (mobile, web, smart TV) need different API shapes, response sizes, or aggregation logic from the same backend services.

How It Works

```
+-----------+     +----------+
| Mobile    |---->| Mobile   |---+
| App       |     | BFF      |   |
+-----------+     +----------+   |     +-----------------+
                                 +---->| Shared Backend  |
+-----------+     +----------+   |     | Services        |
| Web       |---->| Web      |---+     | (Users, Orders, |
| App       |     | BFF      |   |     |  Inventory...)  |
+-----------+     +----------+   |     +-----------------+
                                 |
+-----------+     +----------+   |
| Smart TV  |---->| TV       |---+
| App       |     | BFF      |
+-----------+     +----------+
```

Each BFF is tailored for its client. The mobile BFF returns smaller payloads with fewer images. The web BFF returns richer data. The TV BFF returns large media-focused responses.

Real-World Example

Netflix uses BFF extensively. Their mobile, TV, and web clients each have dedicated backend-for-frontend services that optimize API responses for each platform's screen size, bandwidth, and interaction patterns.

33.3 Bulkhead

When to Use

You want to isolate failures so that a problem in one part of the system does not take down everything else. Named after the watertight compartments in a ship's hull.

How It Works

```
                +-----------------------------------+
                |         Application Server        |
                |                                   |
                |  +----------+   +----------+     |
```

```
                 |  | Thread   |   | Thread   |     |
  Requests ---->|  | Pool A   |   | Pool B   |     |
  for Service A |  | (50      |   | (50      |     |
                 |  |  threads)|   |  threads)|     |
  Requests ---->|  +----+-----+   +----+-----+     |
  for Service B |       |              |           |
                +-------|--------------|-----------+
                        v              v
                    +---------+    +---------+
                    |Service A|    |Service B|
                    |(slow!)  |    |(healthy)|
                    +---------+    +---------+

Without bulkhead: Service A's slowness consumes ALL threads.
                  Service B requests also fail.

With bulkhead: Service A's slowness only exhausts Pool A.
               Pool B continues serving Service B normally.
```

Real-World Example

Amazon isolates customer-facing services from internal batch-processing workloads using separate thread pools, connection pools, and even separate clusters. A spike in internal analytics never affects the shopping cart.

33.4 Cache-Aside (Lazy Loading)

When to Use

You want to reduce database load by caching frequently read data, but you need control over what gets cached and when.

How It Works

```
  Read Path:
  +---------+     +----------+     +----------+
  | App     |---->| Cache    |     | Database |
  | Server  |     | (Redis)  |     |          |
  +---------+     +----------+     +----------+
       |               |                |
       |  1. Check     |                |
       |  cache ------>|                |
```

```
        |               |                |
        |  2a. HIT:     |                |
        |  return data <-                |
        |               |                |
        |  2b. MISS:    |                |
        |  query DB ----------------------->
        |               |                |
        |  3. Get result <-----------------
        |               |                |
        |  4. Write to  |                |
        |  cache ------>|                |
        |               |                |
        |  5. Return    |                |
        |  to client    |                |

   Write Path:
   1. Write to database
   2. Invalidate cache (delete the key)
```

The application is responsible for managing the cache. On a miss, it loads from the database and populates the cache. On a write, it updates the database and invalidates the cache entry.

Real-World Example

Almost every web application uses cache-aside with Redis or Memcached. Facebook uses Memcached as a cache-aside layer in front of MySQL for user profile data, handling billions of lookups per day.

33.5 Circuit Breaker

When to Use

You want to prevent cascading failures when a downstream service is unhealthy. Instead of sending requests that will fail (and waiting for timeouts), stop calling the failing service and fail fast.

How It Works

```
                  +-----------------------------------------+
                  |           Circuit Breaker States          |
                  |                                         |
```

```
                 |  +--------+    failures    +---------+   |
                 |  | CLOSED |  exceed ------>|  OPEN   |   |
                 |  | (normal|  threshold     | (reject |   |
                 |  |  flow) |                |  all    |   |
                 |  +---+----+                |  calls) |   |
                 |      ^                     +----+----+   |
                 |      |                          |        |
                 |      |  success    +----------+ | timer  |
                 |      +-------------|HALF-OPEN | | expires|
                 |                    |(test one |<+        |
                 |                    | request) |          |
                 |                    +----------+          |
                +-------------------------------------------+

CLOSED:    All requests pass through normally.
           Track failure count.

OPEN:      All requests immediately fail (no call to
           downstream). Return cached/default response.

HALF-OPEN: After a timeout, allow ONE test request
           through. If it succeeds, go to CLOSED.
           If it fails, go back to OPEN.
```

Real-World Example

Netflix's Hystrix (now replaced by Resilience4j) implemented circuit breakers for every downstream service call. When a recommendation service was slow, the circuit opened and Netflix showed cached or default recommendations instead of making users wait.

33.6 CQRS (Command Query Responsibility Segregation)

When to Use

Your read and write workloads have fundamentally different requirements. Reads need fast, denormalized data. Writes need normalized, consistent data. Scaling them together forces painful compromises.

How It Works

```
                    Writes (Commands)
                         |
                         v
+---------+     +---------------+     +-----------+
| Client  |---->| Command       |---->| Write     |
|         |     | Service       |     | Database  |
+---------+     +---------------+     +-----------+
                                            |
                                       (events)
                                            |
                                            v
+---------+     +---------------+     +-----------+
| Client  |<----| Query         |<----| Read      |
|         |     | Service       |     | Database  |
+---------+     +---------------+     +-----------+
                   Reads (Queries)

Write DB: normalized, optimized for writes (PostgreSQL)
Read DB:  denormalized, optimized for reads (Elasticsearch, Redis)
Sync:     events propagate changes from write to read store
```

The write side and read side can be scaled, optimized, and even use different technologies independently.

Real-World Example

An e-commerce product catalog uses PostgreSQL for writes (add/update products) and Elasticsearch for reads (search, filter, faceted browsing). Changes are synced via events with a short delay (eventual consistency).

33.7 Event Sourcing

When to Use

You need a complete, immutable history of every change to your data. Instead of storing current state, you store every event that led to the current state.

How It Works

```
Traditional (state-based):
+----+--------+--------+
| id | name   | balance|
```

```
+----+--------+--------+
| 1  | Alice  | $150   |  <-- only current state
+----+--------+--------+

Event Sourcing:
+-----+--------+----------+---------+
| seq | entity | event    | data    |
+-----+--------+----------+---------+
| 1   | acct-1 | Created  | $0      |
| 2   | acct-1 | Deposit  | +$200   |
| 3   | acct-1 | Withdraw | -$50    |  <-- full history
+-----+--------+----------+---------+
Current balance = replay events: $0 + $200 - $50 = $150

+------------------+     +------------------+
| Event Store      |     | Materialized     |
| (append-only     |---->| View             |
|  log of events)  |     | (current state   |
+------------------+     |  for queries)    |
                         +------------------+
```

You never update or delete events. To get current state, you replay events from the beginning (or from a snapshot).

Real-World Example

Banking and financial systems use event sourcing to maintain a complete audit trail. Every transaction is an event. The account balance is a materialized view computed by replaying all events. This makes auditing, debugging, and regulatory compliance straightforward.

33.8 Fan-Out

When to Use

A single event needs to be delivered to many recipients. A user posts a tweet, and all their followers need to see it.

How It Works

```
Fan-Out on Write (Push Model):
+--------+     +-----------+     +---+---+---+---+
| User A |---->| Fan-Out   |---->| B | C | D | E |
```

```
| posts  |     | Service   |     +---+---+---+---+
+--------+     +-----------+     Follower feed caches
Time: O(N) at write time, O(1) at read time

Fan-Out on Read (Pull Model):
+--------+     +-----------+     +---+---+---+---+
| User B |---->| Feed      |---->| A | X | Y | Z |
| reads  |     | Builder   |     +---+---+---+---+
| feed   |     +-----------+     Query all followees
+--------+
Time: O(1) at write time, O(N) at read time

Hybrid (Real-World):
- Regular users (<10K followers): fan-out on write
- Celebrities (>10K followers): fan-out on read
```

Real-World Example

Twitter uses a hybrid fan-out approach. When a regular user tweets, the tweet is pushed into each follower's timeline cache (fan-out on write). When a celebrity with millions of followers tweets, the tweet is fetched at read time (fan-out on read) to avoid millions of cache writes.

33.9 Gateway Aggregation

When to Use

A client needs data from multiple backend services for a single page view. Instead of the client making 5 separate API calls, a gateway aggregates the responses.

How It Works

```
              Without Gateway Aggregation:
+--------+     5 round trips     +---+---+---+---+---+
| Client |<--------------------->| A | B | C | D | E |
+--------+                       +---+---+---+---+---+
Latency = sum of all 5 calls (on mobile, very slow)

              With Gateway Aggregation:
+--------+     1 round trip     +-----------+
| Client |<-------------------->| API       |
+--------+                      | Gateway   |
                                +-----+-----+
```

```
                                        |
                              +----+----+----+----+----+
                              | A  | B  | C  | D  | E  |
                              +----+----+----+----+----+
                              (parallel internal calls)

  Latency = max of 5 calls (parallel) + gateway overhead
```

The gateway makes parallel calls to backend services, merges the results, and returns a single response. This dramatically reduces latency on high-latency networks (mobile, cross-continent).

Real-World Example

An e-commerce product detail page needs data from: product service, pricing service, reviews service, inventory service, and recommendation service. A GraphQL gateway or API gateway aggregates these into a single response, reducing the client from 5 round trips to 1.

33.10 Leader Election

When to Use

You have a distributed system where exactly one node needs to perform a specific role (like scheduling tasks, writing to a shared resource, or coordinating workers).

How It Works

```
  +----------+    +----------+    +----------+
  | Node A   |    | Node B   |    | Node C   |
  | (LEADER) |    | (follower|    | (follower|
  +----+-----+    +----+-----+    +----+-----+
       |               |               |
       +-------+-------+-------+-------+
               |               |
               v               v
       +-----------------------------+
       |  Coordination Service       |
       |  (ZooKeeper / etcd / Consul) |
       |                             |
       |  Lock: "/leader" -> Node A  |
       +-----------------------------+
```

```
If Node A fails:
1. Its heartbeat / lease expires
2. Node B and C race to acquire the lock
3. Winner (say Node B) becomes new leader
4. Node B takes over the leader's responsibilities
```

Leader election typically uses a distributed lock or lease mechanism. The leader must renew its lease periodically. If it fails to renew (crash, network partition), another node takes over.

Real-World Example

Apache Kafka uses ZooKeeper for leader election among brokers. Each partition has a leader broker that handles all reads and writes. If the leader fails, ZooKeeper triggers a new election and one of the in-sync replicas becomes the new leader.

33.11 Outbox Pattern

When to Use

You need to update a database AND publish an event/message atomically. Without the outbox pattern, you risk publishing an event but failing to update the database (or vice versa).

How It Works

```
The Problem (dual write):
+--------+     +---------+
| App    |---->| Database|  1. Write succeeds
| Server |     +---------+
|        |--X->| Message |  2. Publish fails!
+--------+     | Queue   |     (data inconsistency)
               +---------+

The Solution (outbox):
+--------+     +---------+     +----------+     +---------+
| App    |---->| Database|     | Outbox   |---->| Message |
| Server |     |         |     | Relay    |     | Queue   |
+--------+     | Orders  |     | (CDC or  |     |         |
               | table   |     |  poller) |     +---------+
               |---------|     +----------+
               | Outbox  |         ^
               | table   |---------+
```

```
                  +---------+
                  (same transaction)

  1. App writes to Orders table AND Outbox table in a
     SINGLE database transaction (atomic).
  2. A separate relay process reads the Outbox table and
     publishes events to the message queue.
  3. After successful publish, mark outbox row as processed.
```

Because the business data and the outbox entry are written in the same transaction, they are always consistent. The relay process handles delivery to the message queue.

Real-World Example

Stripe uses the outbox pattern to ensure that payment state changes are reliably published as events. When a payment succeeds, the payment record and the event are written in one transaction. A background process then delivers the event to webhooks and internal consumers.

33.12 Retry Pattern

When to Use

A transient failure (network glitch, temporary server overload) causes a request to fail. Instead of immediately returning an error, retry the request after a delay.

How It Works

```
  +--------+         +----------+
  | Client |-------->| Service  |
  +--------+         +----------+
       |                    |
       | Request 1          | 503 (overloaded)
       |<-----------------|
       |                    |
       | (wait 1 second)    |
       |                    |
       | Request 2          | 503 (still overloaded)
       |<-----------------|
       |                    |
       | (wait 2 seconds)   |
```

```
        |                    |
        | Request 3          | 200 OK
        |<------------------|

    Exponential Backoff with Jitter:
    Delay = min(base * 2^attempt + random_jitter, max_delay)

    Attempt 1: wait ~1s  (+/- random jitter)
    Attempt 2: wait ~2s  (+/- random jitter)
    Attempt 3: wait ~4s  (+/- random jitter)
    Attempt 4: wait ~8s  (+/- random jitter)
    Max delay: 30s (cap to prevent absurd waits)
```

Critical: Only retry on transient errors (5xx, timeouts, connection resets). Never retry on client errors (4xx) -- those will fail every time. And always add jitter to prevent the thundering herd problem (all clients retrying at the exact same time).

Real-World Example

AWS SDKs use exponential backoff with jitter for all API calls by default. When a DynamoDB write is throttled, the SDK automatically retries with increasing delays and random jitter, preventing all clients from retrying simultaneously.

33.13 Saga Pattern

When to Use

You have a business transaction that spans multiple services, and you cannot use a traditional distributed transaction (2PC). Instead, you break it into a sequence of local transactions, each with a compensating action to undo it if a later step fails.

How It Works

```
    Choreography-based Saga:

    Order          Payment         Inventory        Shipping
    Service        Service         Service          Service
       |               |               |               |
       | 1. Create     |               |               |
       |    Order      |               |               |
       |-------------->|               |               |
```

```
      |                | 2. Charge     |                |
      |                |    Payment    |                |
      |                |-------------->|                |
      |                |               | 3. Reserve     |
      |                |               |    Stock       |
      |                |               |-------------->|
      |                |               |                | 4. Ship
      |                |               |                |
      |                |               |  FAILS!        |
      |                |               |<----- X        |
      |                |               |                |
      |                | 3c. Refund    | 3c. Release    |
      |                |    Payment    |    Stock        |
      |                |<--------------|                |
      | 1c. Cancel     |               |                |
      |    Order       |               |                |
      |<--------------|               |                |

   Each step has a compensating transaction (suffix "c"):
   Step 1: Create Order    -> 1c: Cancel Order
   Step 2: Charge Payment  -> 2c: Refund Payment
   Step 3: Reserve Stock   -> 3c: Release Stock
```

If any step fails, the saga executes the compensating transactions for all previously completed steps, in reverse order.

Real-World Example

Uber uses sagas for ride booking. The saga coordinates: matching a rider with a driver, authorizing payment, updating driver availability, and starting the trip. If any step fails (payment declined, driver cancels), the saga reverses the completed steps.

33.14 Scatter-Gather

When to Use

You need to query multiple sources (shards, services, or indexes) and combine the results into a single response.

How It Works

```
+--------+     +-----------+
| Client |---->| Scatter-  |
+--------+     | Gather    |
               | Controller|
               +-----+-----+
                     |
        +------------+------------+
        |            |            |
        v            v            v
  +---------+  +---------+  +---------+
  | Shard 1 |  | Shard 2 |  | Shard 3 |
  | or      |  | or      |  | or      |
  | Service |  | Service |  | Service |
  +---------+  +---------+  +---------+
        |            |            |
        +------------+------------+
                     |
                     v
               +-----------+
               | Merge     |
               | Results   |
               +-----------+
                     |
                     v
               (combined response)

SCATTER: Send the query to all shards/services in parallel
GATHER:  Collect all responses, merge, sort, return
```

The total latency is the maximum latency across all shards, not the sum. But you must handle stragglers: set a timeout and return partial results if one shard is slow.

Real-World Example

Elasticsearch uses scatter-gather for every search query. The coordinating node sends the query to all relevant shards in parallel, each shard returns its top results, and the coordinator merges and re-ranks them into the final result set.

33.15 Sidecar

When to Use

You want to add functionality to an existing application (logging, monitoring, networking, security) without modifying the application's code. The sidecar is a separate process that runs alongside the application.

How It Works

```
+-----------------------------------------+
|                 Pod / Host              |
|                                         |
|  +-----------------+  +--------------+ |
|  |  Application    |  |  Sidecar     | |
|  |  Container      |  |  Container   | |
|  |                 |  |              | |
|  |  (business      |  |  (logging,   | |
|  |   logic)        |  |   metrics,   | |
|  |                 |  |   proxy,     | |
|  |                 |  |   config)    | |
|  +-----------------+  +--------------+ |
|        |                    |          |
|        +--------+-----------+          |
|                 |                       |
+-----------------------------------------+
                  |
            Shared network namespace
            (localhost communication)
```

The sidecar shares the same lifecycle and network namespace as the main application. They communicate over localhost, which has near-zero latency.

Real-World Example

Kubernetes service meshes (Istio, Linkerd) inject an Envoy sidecar proxy into every pod. The sidecar handles mTLS encryption, traffic management, retries, and observability -- all without changing a single line of application code.

33.16 Strangler Fig

When to Use

You need to migrate from a legacy (monolith) system to a new system incrementally, without a risky big-bang cutover. Named after the strangler fig tree that grows around an existing tree and eventually replaces it.

How It Works

```
Phase 1: All traffic goes to monolith
+--------+     +-------------------+
| Client |---->|    Monolith       |
+--------+     +-------------------+

Phase 2: New features go to new service
+--------+     +---------+     +-------------------+
| Client |---->| Facade/ |---->|    Monolith       |
+--------+     | Router  |     | (existing features|
               |         |     +-------------------+
               |         |
               |         |---->+-------------------+
               +---------+     | New Service       |
                               | (new + migrated   |
                               |  features)        |
                               +-------------------+

Phase 3: Most traffic goes to new services
+--------+     +---------+     +-------------------+
| Client |---->| Facade/ |---->| Monolith          |
+--------+     | Router  |     | (shrinking)       |
               |         |     +-------------------+
               |         |
               |         |---->+---+---+---+-------+
               +---------+     | A | B | C | ...   |
                               +---+---+---+-------+
                               New Microservices

Phase 4: Monolith is decommissioned
+--------+     +---------+     +---+---+---+-------+
| Client |---->| Facade/ |---->| A | B | C | ...   |
+--------+     | Router  |     +---+---+---+-------+
               +---------+     New Microservices
```

The facade/router layer is the key. It intercepts all requests and decides whether to route them to the monolith or the new services. As you migrate features one by one, the monolith shrinks until it can be turned off.

Real-World Example

Amazon famously migrated from a monolithic bookstore to microservices using the strangler fig pattern over several years. They incrementally extracted services (product catalog, ordering, recommendations) while keeping the monolith running. The facade layer routed traffic to the appropriate service based on the feature.

33.17 Throttling

When to Use

You need to protect your service from being overwhelmed by too many requests. Throttling limits the rate at which clients can make requests.

How It Works

```
Token Bucket Algorithm:
+-----------------------------------------+
|  Bucket (capacity = 10 tokens)          |
|                                         |
|  Tokens: [*] [*] [*] [*] [*] [ ] [ ]    |
|           5 tokens remaining            |
|                                         |
|  Refill rate: 2 tokens/second           |
|                                         |
|  Request arrives:                       |
|    Token available? -> Consume 1, allow |
|    No tokens?       -> Reject (429)     |
+-----------------------------------------+

Per-Client Throttling:
+---------+     +------------+     +---------+
| Client  |---->| Throttle   |---->| Service |
| A (100  |     | Layer      |     |         |
|  req/s) |     | (checks    |     |         |
+---------+     |  rate per  |     |         |
                |  client)   |     |         |
+---------+     |            |     |         |
| Client  |---->| Client A:  |---->|         |
| B (200  |     |  100/100   |     |         |
|  req/s) |     | Client B:  |     |         |
+---------+     |  200/150   |     +---------+
                |  (throttled|
                |   50 req/s)|
                +------------+
```

Common algorithms: Token Bucket (allows bursts), Leaky Bucket (smooths traffic), Fixed Window Counter (simple but imprecise), Sliding Window (precise but more complex).

Real-World Example

Every major API (GitHub, Stripe, Twitter) uses throttling. GitHub's API allows 5,000 requests per hour per authenticated user. When you exceed the limit, you receive a `429 Too Many Requests` response with a `Retry-After` header telling you when to try again.

Pattern Quick Reference

```
+-------------------+----------------------------------------+
| Pattern           | Use When...                            |
+-------------------+----------------------------------------+
| Ambassador        | Offload cross-cutting concerns         |
| BFF               | Different clients need different APIs   |
| Bulkhead          | Isolate failures between components    |
| Cache-Aside       | Reduce DB load with controlled caching |
| Circuit Breaker   | Prevent cascading downstream failures  |
| CQRS              | Read/write workloads differ greatly    |
| Event Sourcing    | Need full audit trail of all changes   |
| Fan-Out           | One event -> many recipients           |
| Gateway Aggregate | Client needs data from many services   |
| Leader Election   | Exactly one node must coordinate       |
| Outbox            | Atomic DB update + event publish       |
| Retry             | Transient failures are expected        |
| Saga              | Multi-service transaction without 2PC  |
| Scatter-Gather    | Query multiple sources, merge results  |
| Sidecar           | Add features without code changes      |
| Strangler Fig     | Incremental monolith-to-micro migration|
| Throttling        | Protect service from overload          |
+-------------------+----------------------------------------+
```

Common Mistakes

1. Over-applying patterns. Not every system needs CQRS or Event Sourcing. Use the simplest solution that meets your requirements.
2. Using Saga when a simple transaction will do. If all your data is in one database, use a regular transaction. Sagas are for cross-service coordination.
3. Circuit breaker without fallback. Opening the circuit is only half the solution. You need a fallback (cached data, default response, graceful degradation).
4. Retry without backoff. Retrying immediately at full speed makes the overloaded service worse. Always use exponential backoff with jitter.
5. Throttling without feedback. Return a `429` status code AND a `Retry-After` header so clients know when to come back.
6. Strangler fig without monitoring. You must monitor both the old and new paths to detect regressions before cutting over.

Best Practices

1. Start simple. Use cache-aside and retry before reaching for CQRS and sagas.
2. Combine patterns. Circuit breaker + retry + bulkhead is a powerful combination for resilient service-to-service communication.
3. Know the trade-offs. Every pattern adds complexity. Event sourcing gives you an audit trail but makes queries harder. CQRS gives you read/write optimization but introduces eventual consistency.
4. Name patterns in interviews. Saying "I would use the outbox pattern here" signals experience and gives the interviewer a shorthand for your approach.
5. Match patterns to requirements. Do not use event sourcing because it sounds cool. Use it because you need an audit trail or time-travel debugging.

Quick Summary

This chapter covered 17 design patterns that appear repeatedly in system design: Ambassador, BFF, Bulkhead, Cache-Aside, Circuit Breaker, CQRS, Event Sourcing, Fan-Out, Gateway Aggregation, Leader Election, Outbox, Retry, Saga, Scatter-Gather, Sidecar, Strangler Fig, and Throttling. Each pattern solves a specific recurring problem. The key is knowing when to apply each one and understanding the trade-offs involved. In interviews, naming the right pattern at the right time demonstrates experience and provides a shared vocabulary with the interviewer.

Key Points

- Patterns are reusable solutions to recurring design problems, not silver bullets.
- Circuit Breaker + Retry + Bulkhead is the standard combination for resilient inter-service communication.
- CQRS and Event Sourcing add significant complexity; only use them when the requirements demand it.
- The Outbox pattern solves the dual-write problem (database + message queue atomicity).
- Sagas replace distributed transactions for multi-service workflows, using compensating actions for rollback.

- Strangler Fig is the safest way to migrate from a monolith to microservices incrementally.
- Fan-out on write is fast for reads but expensive for writes (celebrity problem). Hybrid approaches are common.
- Always combine throttling with informative error responses (429 + Retry-After).

Practice Questions

1. A microservice calls three downstream services. Service A responds in 10ms, Service B in 50ms, and Service C is intermittently failing with 30-second timeouts. Which patterns would you apply to protect your service?
2. You are designing an e-commerce checkout flow that involves: creating an order, charging payment, reserving inventory, and sending a confirmation email. Which pattern would you use, and what are the compensating transactions for each step?
3. Your team is migrating a 10-year-old monolith to microservices. The monolith handles 500 requests per second and cannot have downtime. Which pattern do you use, and what is your first step?
4. A social media platform has 100 million users. When a user posts, the post needs to appear in all followers' feeds. A celebrity has 50 million followers. How do you handle this using the Fan-Out pattern?
5. Your service writes to a PostgreSQL database and publishes events to Kafka. Occasionally, the Kafka publish fails after the database write succeeds, causing data inconsistency. Which pattern solves this, and how does it work?

Exercises

Exercise 1: Pattern Matching

For each scenario below, identify which pattern(s) apply and explain why:

- A payment service needs to call a fraud detection API that is occasionally slow.
- A mobile app loads a home screen that requires data from 7 different backend services.
- A hotel booking system needs to reserve a room, charge the credit card, and send a confirmation, and must roll back if any step fails.
- A legacy system processes 10,000 orders per day and the team wants to gradually move to a new architecture.

Exercise 2: Design a Resilient Service Call

Design the complete error-handling strategy for a service that calls a downstream payment API. Include: retry policy (how many retries, backoff strategy, which errors to retry), circuit breaker configuration (failure threshold, timeout, half-open behavior), bulkhead configuration (thread pool size), and fallback behavior (what to return when the circuit is open).

Exercise 3: Implement a Saga

Design a saga for a food delivery order that involves: (1) accept order, (2) charge customer, (3) notify restaurant, (4) assign delivery driver. Define the compensating transaction for each step. Draw the sequence diagram for both the success path and a failure at step 3. Would you use choreography (event-based) or orchestration (coordinator-based)? Why?

What Is Next?

In the final chapter, we provide a comprehensive glossary of system design terms -- over 100 definitions organized alphabetically. Whether you need a quick refresher on "consistent hashing" or want to look up "quorum," the glossary has you covered.

Chapter 34: Glossary of System Design Terms

This glossary contains over 100 terms that appear throughout this book and in system design interviews. Use it as a quick reference when you encounter an unfamiliar term or need a concise refresher. Terms are organized alphabetically for easy lookup.

What You Will Learn

- Clear, concise definitions for the most important system design terms.
- How each concept fits into the broader system design landscape.
- Cross-references to related terms and the chapters where each topic is covered in depth.

Why This Chapter Matters

System design has its own vocabulary. In interviews and on the job, you need to speak this language fluently. Saying "we should use consistent hashing for our cache cluster" is only useful if you and your audience share the same understanding of what consistent hashing means. This glossary ensures you have a precise, interview-ready definition for every major concept.

A

ACL (Access Control List) -- A list of permissions attached to a resource that specifies which users or systems are allowed to access it and what operations they can perform.

Active-Active -- A deployment model where multiple instances or data centers are all actively serving traffic simultaneously. Provides high availability and geographic distribution. Contrast with active-passive.

Active-Passive -- A deployment model where one instance handles all traffic (active) while a standby instance (passive) takes over only if the active one fails. Simpler than active-active but wastes standby resources.

API (Application Programming Interface) -- A defined contract that specifies how software components communicate with each other. Common styles include REST, GraphQL, gRPC, and WebSocket.

API Gateway -- A server that acts as the single entry point for all client requests. Handles cross-cutting concerns like authentication, rate limiting, request routing, and response aggregation.

Asynchronous Processing -- A pattern where work is accepted immediately and processed later, typically using a message queue. The caller does not wait for the result. Improves responsiveness and decouples producers from consumers.

Availability -- The percentage of time a system is operational and accessible. Expressed as "nines": 99.9% (three nines) allows ~8.7 hours of downtime per year; 99.99% (four nines) allows ~52 minutes.

Availability Zone (AZ) -- An isolated location within a cloud region, with independent power, cooling, and networking. Deploying across multiple AZs protects against localized failures.

B

Back Pressure -- A mechanism where a system signals upstream components to slow down when it is overwhelmed. Prevents cascading failures by controlling the flow of work through the system.

Batch Processing -- Processing data in large groups (batches) at scheduled intervals, rather than in real-time. Suitable for analytics, ETL, and workloads where latency is not critical. Tools include Hadoop MapReduce and Apache Spark.

Bloom Filter -- A space-efficient probabilistic data structure that tests whether an element is a member of a set. Can produce false positives but never false negatives. Used for duplicate detection in web crawlers and cache lookups.

Blue-Green Deployment -- A deployment strategy where two identical environments (blue and green) exist. New code is deployed to the inactive environment, tested, then traffic is

switched. Enables instant rollback by switching back.

Broker -- A middleware component that receives, stores, and forwards messages between producers and consumers. Examples: Kafka broker, RabbitMQ broker, Redis Pub/Sub.

C

Cache -- A fast, temporary storage layer (usually in-memory) that stores frequently accessed data to reduce latency and database load. Common implementations: Redis, Memcached.

Cache Eviction -- The policy for removing items from a cache when it is full. Common policies: LRU (Least Recently Used), LFU (Least Frequently Used), TTL (Time To Live), FIFO (First In First Out).

Cache Invalidation -- The process of removing or updating stale data in a cache when the underlying source data changes. One of the hardest problems in computer science.

Cache-Aside -- A caching pattern where the application manages the cache explicitly: check cache first, on miss load from database and populate cache, on write update database and invalidate cache.

Canary Deployment -- A deployment strategy where new code is rolled out to a small percentage of users first. If no issues are detected, the rollout gradually expands to all users.

CAP Theorem -- States that a distributed system can provide at most two of three guarantees simultaneously: Consistency, Availability, and Partition tolerance. In practice, partition tolerance is required, so the trade-off is between consistency and availability.

CDN (Content Delivery Network) -- A geographically distributed network of proxy servers that cache and serve content from locations close to users, reducing latency for static assets like images, videos, CSS, and JavaScript.

Choreography -- A coordination pattern where services communicate through events without a central coordinator. Each service reacts to events and publishes its own events. Contrast with orchestration.

Circuit Breaker -- A pattern that prevents cascading failures by stopping requests to a failing downstream service. Has three states: Closed (normal), Open (all requests blocked), Half-Open (test one request).

Consistent Hashing -- A hashing technique where adding or removing a node in a distributed system only requires remapping a small fraction of keys, rather than rehashing everything. Used in distributed caches and databases.

Containerization -- Packaging an application and its dependencies into a lightweight, portable container that runs consistently across environments. Docker is the most common container runtime; Kubernetes orchestrates containers at scale.

CQRS (Command Query Responsibility Segregation) -- A pattern that separates the read model (queries) from the write model (commands), allowing each to be optimized, scaled, and maintained independently.

D

Data Lake -- A centralized repository that stores raw, unstructured, and structured data at any scale. Unlike a data warehouse, data is stored in its native format and transformed only when needed.

Data Partitioning -- See Sharding.

Data Replication -- Copying data across multiple nodes or data centers to improve availability, durability, and read performance. Common strategies: single-leader, multi-leader, and leaderless replication.

Database Index -- A data structure (typically B-tree or hash) that speeds up data retrieval operations at the cost of additional storage and slower writes. Without indexes, queries must scan entire tables.

Dead Letter Queue (DLQ) -- A queue that stores messages that could not be processed successfully after a configured number of retries. Allows human or automated investigation of failed messages without blocking the main queue.

Denormalization -- Intentionally introducing redundancy in a database schema to improve read performance by reducing the need for joins. Common in read-heavy systems and NoSQL databases.

DNS (Domain Name System) -- The distributed system that translates human-readable domain names (example.com) into IP addresses (93.184.216.34). Acts as the internet's phone book.

Distributed Lock -- A lock mechanism that works across multiple processes or machines. Used when only one node should perform a specific action at a time. Implemented using tools like Redis (Redlock), ZooKeeper, or etcd.

Distributed Transaction -- A transaction that spans multiple databases or services. Traditionally uses Two-Phase Commit (2PC). In modern microservices, often replaced by the Saga pattern due to 2PC's blocking nature.

E

Elasticity -- The ability of a system to automatically scale resources up during high demand and scale down during low demand, typically to optimize cost.

Endpoint -- A specific URL or network address where a service can be reached. For example, `https://api.example.com/users` is an endpoint for the users API.

Eventual Consistency -- A consistency model where reads may return stale data temporarily, but all replicas will converge to the same value given enough time without new updates. Used in systems that prioritize availability over strict consistency.

Event-Driven Architecture -- An architecture where services communicate by producing and consuming events (facts about something that happened). Promotes loose coupling between services. Key components: event producers, event bus/broker, event consumers.

Event Sourcing -- A pattern where state changes are stored as an immutable sequence of events rather than as mutable current state. Current state is derived by replaying events. Provides a complete audit trail.

F

Failover -- The process of automatically switching to a standby system when the primary system fails. Can be active-passive (standby takes over) or active-active (traffic redistributes among remaining nodes).

Fan-Out -- The process of distributing a single event or message to multiple recipients. Fan-out on write pushes data to recipients at write time; fan-out on read pulls data from sources at read time.

Fault Tolerance -- A system's ability to continue operating correctly even when some components fail. Achieved through redundancy, replication, and graceful degradation.

Firewall -- A network security system that monitors and controls incoming and outgoing traffic based on predetermined rules. Protects internal services from unauthorized external access.

G

Geo-Replication -- Replicating data across multiple geographic regions to reduce latency for global users and provide disaster recovery. Introduces challenges around consistency and conflict resolution.

gRPC -- A high-performance, open-source remote procedure call (RPC) framework developed by Google. Uses Protocol Buffers for serialization and HTTP/2 for transport. More efficient than REST for service-to-service communication.

GraphQL -- A query language for APIs where the client specifies exactly what data it needs. Reduces over-fetching and under-fetching compared to REST. Created by Facebook.

H

Hash Function -- A function that maps data of arbitrary size to a fixed-size value. Used in hash tables, consistent hashing, checksums, and data integrity verification.

Heartbeat -- A periodic signal sent between distributed components to indicate they are alive and functioning. If heartbeats stop, the component is considered failed, triggering failover or recovery.

Horizontal Scaling (Scale Out) -- Adding more machines to a system to handle increased load. Contrast with vertical scaling (adding more power to a single machine). Horizontal scaling provides near-unlimited capacity but adds complexity.

Hot Spot -- An uneven distribution of load where a small number of nodes, keys, or partitions receive a disproportionate amount of traffic. Common in poorly designed sharding schemes.

HTTP (Hypertext Transfer Protocol) -- The foundation protocol of the web. Defines how clients and servers communicate. HTTP/1.1 uses text-based format; HTTP/2 uses binary framing and multiplexing; HTTP/3 uses QUIC over UDP.

I

Idempotency -- A property where performing the same operation multiple times produces the same result as performing it once. Critical for retry safety: if a request is retried due to a timeout, an idempotent operation will not create duplicate effects.

Indexing -- The process of creating data structures (indexes) that allow fast lookup of records by specific fields. Without indexes, databases must scan all rows (full table scan).

J

Jitter -- A random delay added to retry intervals or scheduled operations to prevent multiple clients from acting simultaneously (thundering herd). Exponential backoff with jitter is the standard retry strategy.

K

Kafka -- A distributed event streaming platform used as a message broker, event store, and stream processor. Provides high throughput, fault tolerance, and durability. Data is organized into topics and partitions.

L

Latency -- The time elapsed between sending a request and receiving a response. Measured in milliseconds. P50 (median), P95, and P99 percentiles are commonly tracked.

Leader Election -- The process of choosing one node in a distributed system to serve as the coordinator or primary. Typically implemented using distributed consensus algorithms or external coordination services (ZooKeeper, etcd).

Load Balancer -- A component that distributes incoming requests across multiple servers to prevent any single server from being overwhelmed. Common algorithms: round-robin, least connections, consistent hashing, weighted distribution.

Load Shedding -- Intentionally dropping requests when a system is overloaded to protect the system from total failure. Better to reject some requests quickly than to fail on all requests slowly.

Long Polling -- A technique where the client sends a request and the server holds the connection open until new data is available or a timeout occurs. A compromise between regular polling and WebSockets.

M

MapReduce -- A programming model for processing large datasets in parallel across a distributed cluster. The Map step transforms data; the Reduce step aggregates results.

Originally developed by Google.

Message Queue -- A component that stores messages between producers and consumers, enabling asynchronous, decoupled communication. Examples: RabbitMQ, Amazon SQS, Apache Kafka.

Microservices -- An architectural style where an application is composed of small, independently deployable services, each responsible for a specific business capability. Contrast with monolithic architecture.

Middleware -- Software that sits between an application and the network or operating system, providing services like message routing, authentication, and data transformation.

Monolithic Architecture -- An application built as a single, tightly coupled unit where all components share the same codebase and deployment. Simpler to develop initially but harder to scale and maintain as the system grows.

Multi-Tenancy -- A software architecture where a single instance of the application serves multiple customers (tenants), with data isolation between them. Common in SaaS applications.

N

NoSQL -- A category of databases that do not use the traditional relational table-based model. Types include: key-value stores (Redis, DynamoDB), document stores (MongoDB), column-family stores (Cassandra), and graph databases (Neo4j).

Normalization -- The process of organizing database tables to minimize data redundancy and dependency. Normal forms (1NF, 2NF, 3NF) define progressively stricter rules.

O

Object Storage -- A storage architecture that manages data as objects (with metadata and a unique identifier) rather than as files in a hierarchy or blocks on a disk. Amazon S3 is the most common example. Ideal for unstructured data like images, videos, and backups.

Observability -- The ability to understand the internal state of a system by examining its outputs: logs (what happened), metrics (quantitative measurements), and traces (request flow across services).

Orchestration -- A coordination pattern where a central coordinator directs the workflow across multiple services. The coordinator knows the full process and tells each service what to do. Contrast with choreography.

Outbox Pattern -- A pattern that ensures atomicity between a database write and an event publish by writing both the data and the event to the same database transaction. A separate process reads the outbox table and publishes events.

P

Partition Tolerance -- A system's ability to continue operating despite network partitions (communication failures between nodes). One of the three guarantees in the CAP theorem; generally considered non-negotiable in distributed systems.

Peer-to-Peer (P2P) -- A distributed architecture where each node acts as both client and server. Nodes communicate directly with each other without a central server. Used in file sharing, blockchain, and some CDN architectures.

Polyglot Persistence -- Using different database technologies for different services or data stores within the same application, choosing the best tool for each job.

Primary-Replica -- A replication model where one node (primary/leader) handles all writes and propagates changes to replica nodes (followers/secondaries) that handle reads. Also called master-slave replication.

Protocol Buffers (Protobuf) -- A binary serialization format developed by Google. More compact and faster to serialize/deserialize than JSON or XML. Used with gRPC.

Pub/Sub (Publish-Subscribe) -- A messaging pattern where publishers send messages to topics without knowing who will receive them, and subscribers receive messages from topics they are interested in. Decouples producers from consumers.

Q

QPS (Queries Per Second) -- The number of requests a system handles per second. A fundamental metric for capacity planning. Average QPS matters for provisioning; peak QPS matters for scaling.

Quorum -- The minimum number of nodes that must agree on an operation for it to be considered successful. In a system with N replicas, a quorum requires agreement from (N/2 +

1) nodes. Ensures consistency in replicated systems.

Queue -- A data structure or service that holds items for processing in a defined order (typically FIFO). In distributed systems, message queues decouple producers from consumers and handle load spikes.

R

Rate Limiting -- Controlling the number of requests a client can make to a service within a given time window. Protects services from abuse and overload. Algorithms: token bucket, leaky bucket, sliding window.

Read Replica -- A copy of a database that serves read-only queries, offloading read traffic from the primary database. Introduces replication lag (eventual consistency).

Redis -- An open-source, in-memory data structure store used as a cache, message broker, and database. Supports strings, lists, sets, sorted sets, hashes, and streams. Known for sub-millisecond latency.

Redundancy -- Duplicating critical components or data so that if one fails, another can take over. The foundation of fault tolerance and high availability.

REST (Representational State Transfer) -- An architectural style for APIs that uses standard HTTP methods (GET, POST, PUT, DELETE) and treats everything as a resource identified by a URL. The most common API style for web services.

Reverse Proxy -- A server that sits in front of backend servers, forwarding client requests to them. Provides benefits like load balancing, SSL termination, caching, and DDoS protection. Examples: Nginx, HAProxy.

RPC (Remote Procedure Call) -- A protocol that allows a program to call a function on a remote server as if it were a local function call. gRPC is the most popular modern implementation.

S

Saga Pattern -- A pattern for managing distributed transactions across multiple services. Instead of a single atomic transaction, a saga is a sequence of local transactions with compensating transactions to undo completed steps if a later step fails.

Scalability -- A system's ability to handle increased load by adding resources. Horizontal scalability adds more machines; vertical scalability adds more power to existing machines.

Schema -- The structure that defines how data is organized in a database: tables, columns, data types, relationships, and constraints. Schema-on-write (relational) vs schema-on-read (NoSQL) is a key design decision.

Server-Sent Events (SSE) -- A technology where the server pushes updates to the client over a single long-lived HTTP connection. Simpler than WebSockets but only supports one-way communication (server to client).

Service Discovery -- The mechanism by which services find and connect to each other in a dynamic environment. Can be client-side (client queries a registry) or server-side (load balancer queries registry). Examples: Consul, etcd, Kubernetes DNS.

Service Level Agreement (SLA) -- A formal contract between a service provider and its users that defines expected availability, latency, and other quality metrics, along with penalties for violations.

Service Level Objective (SLO) -- An internal target for a service's performance or availability. More specific than an SLA. For example: "p99 latency under 200ms" or "99.95% availability."

Service Mesh -- An infrastructure layer that handles service-to-service communication in a microservices architecture. Typically implemented as sidecar proxies (Envoy) managed by a control plane (Istio, Linkerd).

Sharding -- Splitting a database across multiple machines (shards), where each shard holds a subset of the data. Enables horizontal scaling of databases. Key decisions: sharding key, resharding strategy, cross-shard queries.

Sidecar Pattern -- Deploying a helper container alongside the main application container in the same pod/host. The sidecar handles cross-cutting concerns (logging, monitoring, networking) without modifying the application.

SLA/SLO/SLI -- Service Level Agreement (contract with users), Service Level Objective (internal target), and Service Level Indicator (actual measured metric). SLI measures reality; SLO sets the target; SLA defines the contract.

Snowflake ID -- A unique ID generation scheme (created by Twitter) that produces 64-bit IDs composed of: timestamp (41 bits), machine ID (10 bits), and sequence number (12 bits). IDs are time-sortable and globally unique without coordination.

SQL (Structured Query Language) -- The standard language for interacting with relational databases. Provides commands for querying (SELECT), modifying (INSERT, UPDATE, DELETE), and defining data structures (CREATE TABLE).

SSE -- See Server-Sent Events.

Sticky Session -- A load balancing technique where all requests from a specific client are routed to the same server. Useful when server-side state (sessions) exists but complicates scaling and failover.

Stream Processing -- Processing data records continuously in real-time as they arrive, rather than in batches. Tools include Apache Kafka Streams, Apache Flink, and Apache Storm.

Strong Consistency -- A guarantee that after a write completes, all subsequent reads will return the updated value. Provides the simplest programming model but requires coordination that reduces availability and increases latency.

T

Throughput -- The amount of work a system can handle per unit of time. Measured in requests per second (RPS), transactions per second (TPS), or data volume per second (MB/s).

Throttling -- Intentionally limiting the rate of operations to protect a system from overload. Applied per-client, per-API, or globally. Returns HTTP 429 (Too Many Requests) when the limit is exceeded.

TLS (Transport Layer Security) -- A cryptographic protocol that provides secure communication over a network. Ensures confidentiality (encryption), integrity (tamper detection), and authentication (certificates). The successor to SSL.

TTL (Time To Live) -- A duration after which cached data, DNS records, or messages automatically expire. Prevents serving stale data indefinitely. Common TTL values: cache (5-60 minutes), DNS (1-24 hours).

Two-Phase Commit (2PC) -- A distributed transaction protocol that ensures all participants either commit or abort a transaction. Phase 1: coordinator asks all participants to prepare. Phase 2: coordinator tells all to commit (or abort if any failed). Blocking and slow, often replaced by the Saga pattern.

U

UUID (Universally Unique Identifier) -- A 128-bit identifier that is unique across space and time without requiring a central authority. Version 4 UUIDs are randomly generated; version 7 UUIDs are time-ordered.

V

Vertical Scaling (Scale Up) -- Adding more CPU, RAM, or storage to an existing machine. Simpler than horizontal scaling but has a hardware ceiling and creates a single point of failure.

Virtual IP (VIP) -- An IP address that is not tied to a specific physical machine. Used in load balancing and failover: the VIP floats between servers, so clients always connect to the same address regardless of which server is active.

W

WAL (Write-Ahead Log) -- A technique where changes are written to an append-only log before being applied to the database. Ensures durability: if the system crashes, the log can be replayed to recover uncommitted changes.

WebSocket -- A communication protocol that provides full-duplex (two-way) communication over a single, long-lived TCP connection. Ideal for real-time applications like chat, live feeds, and multiplayer games.

Write-Behind (Write-Back) Cache -- A caching strategy where writes go to the cache first and are asynchronously flushed to the database. Provides fast writes but risks data loss if the cache fails before flushing.

Write-Through Cache -- A caching strategy where writes go to both the cache and the database simultaneously. Ensures consistency but adds write latency since every write waits for both operations.

Z

ZooKeeper -- A centralized service for distributed coordination, including: configuration management, naming, leader election, distributed locking, and group membership. Widely used in the Hadoop and Kafka ecosystems. Being replaced by etcd and Raft-based alternatives in newer systems.

Quick Summary

This glossary covers over 100 terms spanning every major area of system design: networking, databases, caching, messaging, deployment, security, and distributed systems. Use it as a reference whenever you encounter an unfamiliar term in this book, in an interview, or on the job.

Key Points

- System design has a rich vocabulary. Knowing the precise meaning of terms like "eventual consistency," "quorum," and "idempotency" is essential for interviews and professional communication.
- Many terms are related: understanding one helps you understand others. For example, CAP theorem connects to consistency, availability, and partition tolerance.
- This glossary is a starting point. Each term could fill a chapter (and many of them do, earlier in this book). Use the glossary for quick reference and dive into the full chapters for depth.

Practice Questions

1. Explain the difference between horizontal scaling and vertical scaling. When would you choose one over the other?
2. What is the relationship between SLI, SLO, and SLA? Give an example of each for a web application.
3. Compare and contrast WebSockets, Server-Sent Events, and long polling. When would you use each?
4. What does idempotency mean, and why is it important for APIs that may be retried?
5. Explain the difference between eventual consistency and strong consistency. Give a real-world example of a system that uses each.

Exercises

Exercise 1: Term Association

Match each term to the problem it solves:

- Bloom filter -> ?
- Circuit breaker -> ?
- Consistent hashing -> ?
- Dead letter queue -> ?
- Write-ahead log -> ?

Problems: (a) Distributing keys evenly when nodes change, (b) Space-efficient set membership testing, (c) Preventing cascading failures, (d) Recovering from crashes without data loss, (e) Handling messages that repeatedly fail processing.

Exercise 2: Build a Mini Glossary

Pick a system you use daily (email, social media, streaming video). List 20 terms from this glossary that are relevant to how that system works. For each term, write one sentence explaining how it applies specifically to that system.

Congratulations

You have made it through the entire book. From the fundamentals of scaling and load balancing, through databases, caching, message queues, and microservices, to designing real systems like URL shorteners, chat applications, and video platforms -- you now have a comprehensive toolkit for system design.

But this is not the end. It is the beginning of a much longer journey. Here is what to learn next:

Keep Building Your Knowledge

1. Read engineering blogs. Companies like Netflix, Uber, Stripe, Airbnb, Meta, and Google regularly publish detailed posts about their architecture. These are the best source of real-world system design knowledge.
2. Study open-source systems. Read the documentation and architecture overviews of systems like Kafka, Cassandra, Elasticsearch, Kubernetes, and Redis. Understanding how these tools work internally makes you a stronger designer.
3. Practice regularly. Pick a new system design problem every week and work through it using the four-step framework from Chapter 32. Time yourself. Practice with a friend who can play the interviewer role.
4. Build things. The best way to understand distributed systems is to build one. Implement a simple key-value store with replication, or build a chat application with WebSockets, or

create a URL shortener that handles thousands of requests per second.

5. Go deeper on fundamentals. Read "Designing Data-Intensive Applications" by Martin Kleppmann for a rigorous treatment of distributed systems fundamentals. Read the original papers on Raft consensus, Google's Spanner, Amazon's Dynamo, and Facebook's TAO.
6. Stay current. The field evolves rapidly. Serverless architectures, edge computing, AI-driven infrastructure, and new database technologies continue to reshape how we build systems.

The Mindset That Matters

The most important thing this book teaches is not any specific pattern or technology. It is a way of thinking: start with requirements, estimate scale, design for the common case, plan for failure, and always understand the trade-offs. Every design decision has a cost. The art of system design is making the right trade-offs for your specific context.

You are ready. Go build great systems.